INFOSYS SP / DSE CODING ASSESSMENT · 6 DECEMBER 2026

Data Structures and Algorithms in Java

A 29-chapter textbook for the Infosys SP/DSE coding assessment

Volume I · Theory

One chapter for each day of the 29-day plan, from the Java toolkit on 7 November to the day before the exam. Each chapter teaches the ideas, proves why they work, measures what they cost, and gives the Java you will type.

Companion volume: **Volume II · Practice Workbook** (same chapter numbers)

Prepared for Purvam Prajapati · Edition of 7 November 2026

Data Structures and Algorithms in Java · Volume I: Theory. Edition of 7 November 2026.

This book expands the 29-day plan (*infosys-dsa-29-day-plan.html*) and the daily pattern handbook (*infosys-dsa-daily-handbook.html*) of the Skill-Upgradation repository into a textbook. The chapter numbers are the day numbers of the plan.

Every Java listing was extracted from the book's sources and compiled with `javac --release 8`; listings that are complete programs were run on their sample inputs. Problem statements from Infosys and other sites are paraphrased. Facts about the assessment come from the sources listed in Appendix D.

Typeset from Markdown with WeasyPrint. Text in Source Serif 4, headings in Source Sans 3, code in JetBrains Mono (all SIL Open Font License).

Contents

Part I Foundations 6

1 **Java for Problem Solving: Collections, Input and Output** 7

1.1 The assessment and how these two books work 8 1.2 Measuring cost: big-O notation 9 1.3 Primitive types and overflow 11 1.4 The collections toolkit 13 1.5 Strings and characters 20 1.6 Reading input fast 21 1.7 Linked lists 25 1.8 Recursion and the call stack 28

2 **Prefix Sums, Difference Arrays and Hashing** 34

2.1 The range-sum problem 34 2.2 Counting subarrays with a given sum 35 2.3 One key, many questions 37 2.4 Hashing in Java 38 2.5 Difference arrays 39 2.6 Two dimensions 40

3 **Two Pointers and Sliding Windows** 43

3.1 Two pointers on a sorted array 43 3.2 3Sum: sorting plus two pointers 45 3.3 Variable-size sliding windows 47 3.4 Longest repeating character replacement 50 3.5 Counting windows: exactly K 51 3.6 The shortest valid window 53 3.7 Fixed-size windows and the monotonic deque 54 3.8 Choosing between windows, two pointers and prefix sums 56

4 **Sorting and Greedy Choice: Intervals** 59

4.1 Sorting in Java 59 4.2 What makes a greedy algorithm correct 61 4.3 Merging intervals 62 4.4 Interval scheduling: keep the most 63 4.5 Partitioning into the fewest groups 65 4.6 Custom orders: the Largest Number 68 4.7 Greedy invariants beyond sorting 69

6 **Binary Search and Searching on the Answer** 73

6.1 Binary search on a sorted array 73 6.2 First and last position, and counting 77 6.3 Searching on the answer 78 6.4 Minimise the maximum, maximise the minimum 82 6.5 Overflow, bounds and real numbers 84 6.6 Counting instead of searching: the k-th smallest 86 6.7 Cost, and functions that rise then fall 88

7 **Monotonic Stacks** 92

7.1 Next greater element 92 7.2 Worked example: daily temperatures 96 7.3 Largest rectangle in a histogram 97 7.4 The contribution technique 99 7.5 Greedy stacks for the smallest result 102 7.6 Relatives: the monotonic deque and the Cartesian tree 104

8 **Bits, XOR, Tries and Linear Bases** 108

8.1 Binary representation and Java’s bit operators 108 8.2 The algebra of XOR 110 8.3 Bit independence: sums of XORs and choosing X 112 8.4 The binary trie 114 8.5 The XOR basis 118 8.6 Choosing the right XOR tool 120

9 **Brute Force, Stress Testing and the Mock Exam** 123

9.1 Why brute force first 123 9.2 Complete search patterns 124 9.3 Stress testing 127 9.4 Estimating complexity from the limits 130 9.5 The mock protocol 130 9.6 The post-mortem and an error taxonomy 132 9.7 A map of Part I 133 9.8 After the mock 134

Part II Number Theory, Dynamic Programming and First Graphs 137

10 **Number Theory for Programmers** 138

10.1 Divisibility and the greatest common divisor 138 10.2 Modular arithmetic 142 10.3 Fast exponentiation 143 10.4 Inverses and binomial coefficients 145 10.5 Primes, sieves and factorisation 148 10.6 Perfect squares and the square-free kernel 152 10.7 Gcds of subarrays and loops over multiples 155 10.8 Small primes as bit masks 158

11 **Dynamic Programming I: States, Transitions and Knapsacks** 162

11.1 What dynamic programming is 162 11.2 The recipe 164 11.3 One-dimensional DP: House Robber 166 11.4 The 0/1 knapsack 168 11.5 Unbounded knapsack and coin change 170 11.6 Subset sum and partitions 173 11.7 Solutions, counts, memory and bit-parallel subset sums 175

13 **Dynamic Programming III: Increasing Subsequences and Extra Dimensions** 179

13.1 Subsequences and the quadratic DP 180 13.2 Patience sorting in O(n log n) 181 13.3 Recovering the subsequence and the length at each position 183 13.4 Peaks: combining increasing runs from both sides 185 13.5 DP over values with a Fenwick tree of maxima 186 13.6 Two keys: sort by one, LIS on the other 189 13.7 An extra state for a side condition 190

14 **Dynamic Programming IV: Divisors, Intervals and Games** 196

14.1 Chains under divisibility 196 14.2 Looping over multiples 198 14.3 Counting divisor arrays 200 14.4 Interval DP 201 14.5 Cutting a stick, and the last operation 203 14.6 Two-player games 204 14.7 Choosing the order and the limits 207

15 **Graphs I: Breadth-First Search, Depth-First Search and Topological Order** 210

15.1 Graphs and how to store them 211 15.2 Breadth-first search 213 15.3 Depth-first search without recursion 216 15.4 Directed acyclic graphs and topological order 218 15.5 0-1 BFS 221 15.6 Cost, and a checklist for traversals 223

16 **Analysing a Problem Before You Code** 227

16.1 From story to model 227 16.2 Amortised analysis 229 16.3 Operation and memory budgets in Java 231 16.4 Choosing types and checking modular arithmetic 233 16.5 Which pattern? 234 16.6 Designing your own tests 235 16.7 Mock 2 and its review 236 16.8 After the mock 237

Part III Graphs, Trees and Range Queries 240

17 **Graphs II: Shortest Paths, Union-Find and Spanning Trees** 241

17.1 Weighted shortest paths and Dijkstra’s algorithm 242 17.2 Changing the cost of a path 245 17.3 State-expanded graphs 248 17.4 Union-find 252 17.5 Minimum spanning trees 255 17.6 Offline processing in sorted order 259

18 **Trees I: Recursion, Parent Arrays and Ancestors** 263

18.1 Trees and how they are stored 263 18.2 Recursion on binary trees 265 18.3 General trees without recursion 270 18.4 Lowest common ancestors by binary lifting 276 18.5 Path queries from root prefixes 279 18.6 Subtree sizes as answers 281

19 **Trees II: Tree DP, Rerooting and Small-to-Large** 284

19.1 Tree DP with states 285 19.2 Per-subtree aggregates 288 19.3 Rerooting 289 19.4 Small-to-large merging 291 19.5 Parity and edge operations 295 19.6 Choosing a technique 297

C **Answers to review questions** 300

Index 310

Preface

Stub preface.

PART I

Foundations

Days 1–9 · Java toolkit, arrays, greedy, binary search, stacks, bits

Part I builds the tools that solve the first two problems of a typical Infosys paper. Each chapter takes one family of techniques (prefix sums, windows, sorting and greedy choice, heaps, binary search, monotonic stacks and bit tricks) and shows when it applies, why it is correct and what it costs. The part ends with the first timed mock and the brute-force and stress-testing habits that turn a wrong answer into a found bug.

1	Java for Problem Solving: Collections, Input and Output
2	Prefix Sums, Difference Arrays and Hashing
3	Two Pointers and Sliding Windows
4	Sorting and Greedy Choice: Intervals
5	Greedy with heaps
6	Binary Search and Searching on the Answer
7	Monotonic Stacks
8	Bits, XOR, Tries and Linear Bases
9	Brute Force, Stress Testing and the Mock Exam

CHAPTER 1 · DAY 1 OF 29 · SATURDAY 7 NOVEMBER 2026 · LEARNING DAY

1

Java for Problem Solving: Collections, Input and Output

Accounts, the Java collections you will use every day, and how FastScanner replaces Scanner.

Part I · Foundations
Handbook cards · Java collections toolkit, Fast input and output (FastScanner), Linked list moves
Before this chapter · Basic Java syntax (classes, methods, loops, arrays)
Practice · Workbook Chapter 1 · 8 problems · core ≈ 3.9 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- describe the Infosys SP/DSE coding paper and the daily routine these two books follow;
- turn input limits into a target running time using big-O notation and the rule of about 10^8 simple steps per second;
- avoid the Java traps that silently cost marks, namely `int` overflow, negative remainders, `==` on `Integer` and `String`, and `+=` on strings in a loop;
- choose a collection by the question it must answer, and state the cost of each call you use;
- read large inputs with `FastScanner` and print through one `PrintWriter`;
- reverse, split, merge and check linked lists with pointer invariants, and recognise when recursion depth will crash a Java program.

Every later chapter of this book assumes that you can turn an idea into correct Java quickly. That skill has three parts. You must know what a solution will cost before you write it, so that you do not spend forty minutes on code that is too slow. You must know the standard library well enough that "count the values", "find the nearest booking" or "always take the largest" is one call you type without thinking. And you must know the handful of places where Java behaves differently from school arithmetic: integers that wrap around, remainders that come out negative, objects that look equal but are not `==`.

This chapter covers those three parts. It starts with the assessment itself and the routine that the two volumes follow, then sets up the language for talking about running time. The middle of the chapter is a tour of the primitive types, the collections and the string classes, with four of the day's problems as worked examples. The last three sections cover reading input in an exam that gives you only standard input, linked lists, and the call stack.

Much of this material is reference. Read it once today, and come back to Table 1.5 and Table 1.9 whenever a later chapter uses a structure you are unsure of.

1.1 The assessment and how these two books work

What the paper looks like

The Infosys Specialist Programmer (SP) and Digital Specialist Engineer (DSE) roles share one coding assessment. The reports collected in the plan agree on the following points.

- **Three problems in three hours**, in increasing difficulty. A 2026 off-campus candidate described an in-person, proctored lab round with about four DSA problems, and the official sample paper has a fourth "Complex" problem as well.
- **Marks**: the first problem is worth 20, the second 30 and the third 50.
- **Partial credit**: marks come from the test cases you pass, so a correct but slow solution still scores on the small tests. Sectional and overall cut-offs are reported. Negative marking applies only to plagiarism.
- **Languages**: C, C++, Java, Python and JavaScript.
- **Problem style**: long, story-style statements. Input arrives **one value per line**. You usually implement a function such as `get_ans(...)` that returns the answer. Typical limits are $n \le 10^5$, with answers taken modulo $10^9 + 7$.
- **Conditions**: strict invigilation; phones, calculators and smartwatches were not allowed at past in-person rounds.

The official sample paper is harder than its labels suggest. Its "Easy" problem combines a number-theory trick (the square-free part of a number) with a tree technique (small-to-large merging) and a traversal without recursion; the plan estimates it at about Codeforces 1800. That gap is why the plan reaches LeetCode Hards and Codeforces 1600–2200 problems by the third week.

> **INFOSYS LENS**
>
> Partial marks are real marks. A brute force that is correct for small inputs passes the small test files, and on the sample paper's Hard problem trying every subsequence already works up to $N \approx 20$. Never leave a problem blank. Chapter 9 teaches you to write the brute force first and to use it to test your fast solution.

Function stub or full program

Two kinds of question appear, and they need different code.

1. **A function stub.** The platform shows a method such as `get_ans(int n, int m, int[] a)` and reads the input for you. Keep the stub's name, parameters and return type exactly as given and fill in the body. Changing the signature can make every test fail.
2. **A full program.** You write `main` yourself and read every number from standard input. Then you need the `FastScanner` of Section 1.6 and a `PrintWriter` for output.

On LeetCode you write only the method, inside a class named `Solution`, and LeetCode passes you the input as arguments. On Codeforces you always write the full program. Practising both forms is part of the plan.

How the two volumes work together

Each chapter number is a day number. Theory Chapter N (this book) teaches the ideas for Day N; Workbook Chapter N holds the day's plan, a spaced-review list of earlier patterns, exercises to complete by hand, a worksheet for each problem and an error log. Hints are in Workbook Appendix A and exercise answers in Workbook Appendix B. Answers to this book's review questions are in Appendix C.

Table 1.1 The daily routine (about four hours, plus an optional bonus hour)

Block	What you do	Time
1. Handbook review	Write each pattern that is due from memory, then compare and rate yourself.	15 min
2. Learn	Read the theory chapter. Type each new listing once and run it.	30–45 min
3. Core problems	Two sittings of about 75 minutes with a 10-minute break. Limits per problem: Easy 20, Medium 40, Hard 60 minutes. When time is up, open the hint, then the editorial; close it and write the solution yourself.	about 2.5 h
4. Review	Log every mistake in the error log. Re-solve one problem from three days ago without your old code.	20 min
Bonus hour	Only if the core is done and you still feel sharp. These are the Specialist Programmer-level problems.	optional

Stop starting new problems when you notice the signs of fatigue: two problems in a row ran past their time limit, you re-read the same line of a statement, or you start pasting code you do not understand. Do the review block and end the day.

> **EXAM TIP**
>
> Practise where the exam will put you. LeetCode problems are solved in LeetCode's editor (click *Run*, then *Submit*). Codeforces problems have a *Custom test* page for trying the sample input. Infosys problems with no online judge go into OneCompiler (onecompiler.com/java), which has a STDIN box for the sample input.

1.2 Measuring cost: big-O notation

Before you write a solution you should know, within a factor of ten, how long it will run. The limits in the statement tell you how fast the solution must be; big-O notation tells you how fast your idea is. Matching the two is the first thing to do with every problem.

Counting steps

Count the **basic steps** an algorithm performs: an arithmetic operation, a comparison, an array access, an assignment. Each takes a small, fixed amount of time, so the total time is roughly proportional to the number of steps. We count the steps for the worst input of each size n and write the count as a function $f(n)$.

Consider the brute force for "is there a pair $i < j$ with $a[i] + a[j] = t$?":

```
static boolean hasPairBrute(int[] a, int t) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] + a[j] == t) return true;
    return false;
}
```

For a fixed i the inner loop runs $n - 1 - i$ times. Over all i that is $(n-1) + (n-2) + \cdots + 0 = n(n-1)/2$ comparisons when no pair exists. For $n = 10^4$ this is 49,995,000 comparisons, which runs in well under a second. For $n = 10^5$ it is 4,999,950,000, which at the speeds measured below takes between 5 and 50 seconds. The exact count is less important than its shape: it grows like n^2.

The definition

DEFINITION 1.1 Big-O, big-Omega and big-Theta

Let f and g be functions from positive integers to non-negative numbers. In **big-O notation**, $f(n) = O(g(n))$ means there are constants $c > 0$ and n_0 such that $f(n) \le c \cdot g(n)$ for every $n \ge n_0$. Similarly $f(n) = \Omega(g(n))$ means $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$ and some $c > 0$, and $f(n) = \Theta(g(n))$ means both hold.

O is an upper bound, Ω a lower bound and Θ an exact order of growth. In practice people write O when they mean Θ, and this book follows the custom when the bound is tight.

EXAMPLE 1.1 A bound with explicit constants

Take $f(n) = 3n^2 + 5n + 2$. To show $f(n) = O(n^2)$, choose $c = 4$. The inequality $3n^2 + 5n + 2 \le 4n^2$ is equivalent to $n^2 - 5n - 2 \ge 0$, which first holds at $n = 6$ (at $n = 5$ the left side is -2). So $c = 4$ and $n_0 = 6$ work. The same f is also $O(n^3)$, because O is only an upper bound, but it is not $O(n)$: for any c, $3n^2 > c \cdot n$ as soon as $n > c/3$.

Three rules follow directly from the definition and cover almost every analysis in this book.

1. **Constants and lower-order terms disappear.** $3n^2 + 5n + 2 = \Theta(n^2)$, and $\log_2 n$ and $\log_{10} n$ differ only by a constant factor, so the base of a logarithm is never written.
2. **Sequential parts add, and the larger wins.** If one phase costs $O(n \log n)$ and the next $O(n)$, the total is $O(n \log n)$.
3. **Nested parts multiply.** A loop of n iterations whose body costs $O(\log n)$ costs $O(n \log n)$.

LEMMA 1.1 The sum rule

If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$, then $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$.

Proof. There are constants with $f_1(n) \le c_1 g_1(n)$ for $n \ge n_1$ and $f_2(n) \le c_2 g_2(n)$ for $n \ge n_2$. For $n \ge \max(n_1, n_2)$, $f_1(n) + f_2(n) \le c_1 g_1(n) + c_2 g_2(n) \le (c_1 + c_2) \cdot \max(g_1(n), g_2(n))$. So $c = c_1 + c_2$ and $n_0 = \max(n_1, n_2)$ satisfy the definition. ∎

Logarithms appear whenever a quantity is halved each step, which is the heart of binary search (Chapter 6), heaps (Chapter 5) and balanced trees.

CLAIM 1.2 Halving loops

For $n \ge 1$ the loop `for (int x = n; x > 0; x /= 2)` runs exactly $\lfloor \log_2 n \rfloor + 1$ times.

Proof. After k iterations, $x = \lfloor n/2^k \rfloor$, because integer division by 2 repeated k times equals one division by 2^k, rounded down. The loop runs iteration $k + 1$ exactly when $\lfloor n/2^k \rfloor \ge 1$, that is when $2^k \le n$, that is when $k \le \log_2 n$. So iterations $k + 1 = 1, 2, \ldots, \lfloor \log_2 n \rfloor + 1$ run and no more. For $n = 10^6$ that is 20 iterations. ∎

How fast is Java?

A useful planning figure is **10^8 to $3 \cdot 10^8$ basic steps per second** for typical contest Java. The range is wide because "a step" hides very different costs. These measurements were taken for this book with OpenJDK 25 (middle of three runs):

- a loop of 10^9 iterations doing one XOR, one shift and one addition: 0.98 s;
- 10^6 increments of random cells of an `int[]` of size 10^6: 32 ms;

- 10^6 `HashMap.merge` calls on random keys: 170 ms;
- 10^6 `TreeMap.merge` calls on random keys: 630 ms;
- `Arrays.sort` on 10^6 random `int` values: 95 ms.

So a trivial loop runs at 10^9 steps per second, while a `TreeMap` operation costs about twenty times as much as an array access. When an estimate lands near the limit, count collection calls separately from array steps.

From limits to a target

Reverse the reasoning: the largest n in the statement tells you which complexities can fit in about a second. Table 1.2 is the short version. The full table, with memory and more cases, is in Theory Appendix A.

Table 1.2 Rough limits for a 1–2 second time limit

Largest n	Fits	Typical technique
≤ 10	$O(n!)$	try every order
≤ 20	$O(2^n \cdot n)$	every subset, bitmask DP
≤ 500	$O(n^3)$	three nested loops, interval DP
≤ 5 000	$O(n^2)$	all pairs, two-dimensional DP
≤ 10^6	$O(n \log n)$	sorting, heaps, binary search, balanced trees
≤ 10^7–10^8	$O(n)$	one pass, prefix sums, two pointers
larger	$O(\log n)$ or $O(1)$	a formula, or binary search on the answer

Memory is estimated the same way: cells times bytes per cell. An `int` takes 4 bytes and a `long` 8, so `new long[100000][100000]` asks for $10^{10} \times 8 = 8 \cdot 10^{10}$ bytes, about 80 GB. An array of 10^6 `long` values is 8 MB and is always fine.

> **EXAM TIP**
>
> Write the target complexity on paper before you write code. The workbook's problem worksheets have a box for it. If your idea does not meet the target, it is a partial-marks fallback, not the solution.

1.3 Primitive types and overflow

The integer types

Table 1.3 Java's numeric types

Type	Bits	Range	Use it for
`int`	32	−2,147,483,648 to 2,147,483,647 (about $\pm 2.1 \cdot 10^9$)	indices, counts, values ≤ 10^9
`long`	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (about $\pm 9.2 \cdot 10^{18}$)	sums, products, anything modulo $10^9 + 7$
`char`	16	0 to 65,535 (unsigned)	characters; it is a number
`double`	64	about $\pm 1.8 \cdot 10^{308}$, 53 bits of precision	real numbers; integers are exact only up to $2^{53} \approx 9 \cdot 10^{15}$

The default rule for this book is **`long` for every sum and product, `int` for indices and counts**. A `long` holds the product of two values below $3 \cdot 10^9$, which covers every product of two numbers below the modulus $10^9 + 7$.

Two's complement and wrap-around

DEFINITION 1.2 Two's complement

A Java `int` is stored as 32 bits $b_{31} \dots b_0$ in **two's complement**: the pattern represents the value $-b_{31} \cdot 2^{31} + b_{30} \cdot 2^{30} + \cdots + b_0 \cdot 2^0$. The top bit counts as -2^{31} instead of $+2^{31}$, so patterns with the top bit set are the negative numbers. A `long` works the same way with 64 bits.

With this representation the processor adds and multiplies bit patterns exactly as it would for unsigned numbers and keeps the low 32 bits. The consequence is the rule you must remember:

KEY IDEA

Every `int` addition, subtraction and multiplication returns the true result **modulo 2^{32}**, moved into the range $[-2^{31}, 2^{31})$. Java never throws an exception on integer overflow; you get a wrong number silently.

These lines were run to produce the values in the comments:

```
int big = 2_000_000_000;
int s = big + big;                      // -294967296  (4e9 - 2^32)
int a = 100_000, b = 100_000;
int p = a * b;                          // 1410065408  (1e10 - 2 * 2^32)
long ok = (long) a * b;                 // 10000000000 (a becomes long before *)
long late = (long) (a * b);             // 1410065408  (overflow happened first)
int wrap = Integer.MAX_VALUE + 1;       // -2147483648
int abs = Math.abs(Integer.MIN_VALUE);  // -2147483648 (there is no +2^31 in int)
```

The fourth line is the trap that catches people who know about overflow. In `(long) (a * b)` the multiplication happens in `int`, overflows, and only the wrong result is converted. In `(long) a * b` the cast applies to `a` alone, so `b` is promoted and the multiplication happens in `long`.

PITFALL

Write `(long) a * b`, never `(long) (a * b)`. And in modular code, reduce after **every** multiplication: with $x, y < 10^9 + 7$, `x * y % MOD` fits in a `long` (the product is at most about $1.0 \cdot 10^{18}$, far below $9.2 \cdot 10^{18}$), but `x * y * z % MOD` does not.

Division and remainders

Java's integer division rounds **towards zero**, and `%` is defined so that `a == (a / b) * b + a % b`. So the remainder takes the sign of the left operand: `-7 / 2` is −3, `-7 % 3` is −1 and `-3 % 5` is −3. Mathematics, and every hash-map key of Chapter 2, wants the remainder in $[0, m)$. Use `Math.floorMod(-7, 3)`, which is 2, and `Math.floorDiv(-7, 2)`, which is −4 (rounding down).

A comparator written as subtraction fails for the same reason as any other `int` arithmetic. For $p = 2 \cdot 10^9$ and $q = -2 \cdot 10^9$, `p - q` is −294,967,296, which claims $p < q$. `Integer.compare(p, q)` returns 1, the right answer. Chapter 4 returns to comparators.

Boxing and the Integer cache

Collections hold objects, not primitives, so `List<Integer>` and `Map<Integer, Integer>` store `Integer` objects. Converting an `int` to an `Integer` is called **autoboxing**, and the reverse is unboxing; the compiler inserts both for you. Boxing calls `Integer.valueOf`, which by the Java Language Specification returns a **shared, cached object** for every value from −128 to 127 and may create a new object for any other value.

The operator `==` on two objects asks "are these the same object?", not "do they hold the same number?". Together, these two facts give the following results, all checked by running them:

```
Integer i1 = 127, i2 = 127;
boolean x1 = i1 == i2;                  // true: both are the cached object for 127
Integer i3 = 128, i4 = 128;
boolean x2 = i3 == i4;                  // false: two different objects
boolean x3 = i3.equals(i4);             // true: same value
boolean x4 = i3.intValue() == i4;       // true: one side is int, so both are unboxed
Integer i5 = -129, i6 = -129;
boolean x5 = i5 == i6;                  // false; -128 == -128 would be true
```

`Long.valueOf` caches the same range. The upper end of the `Integer` cache can be raised with a JVM option, so code that happens to pass with `==` on your machine may fail elsewhere. The bug shows up in practice as `map.get(a) == map.get(b)` for two counts: it is correct while the counts are small and fails once both reach 128.

> **PITFALL**
>
> Compare boxed values with `.equals()`, or unbox one side first (`int ca = map.get(a);`). And never unbox a value that may be `null`: `int c = map.get(k);` throws a `NullPointerException` when `k` is missing. Use `map.getOrDefault(k, 0)`.

1.4 The collections toolkit

Pick the structure by the question

Each collection answers a few questions in one fast step. Decide which question your algorithm asks most often, then pick the structure that answers it.

Table 1.4 Which structure answers which question

The question	Structure	The call
Have I seen *x*?	`HashSet<Integer>`	`add`, `contains`
How many times did *x* appear? Where was *x*?	`HashMap<Integer, Integer>`, or `int[26]` for letters	`merge`, `getOrDefault`
What was added most recently?	`ArrayDeque` as a **stack**	`push`, `pop`, `peek`
What was added earliest?	`ArrayDeque` as a **queue**	`offer`, `poll`, `peek`
What is the smallest (or largest) item so far?	`PriorityQueue`	`add`, `poll`, `peek`
What is the nearest key below or above *x*?	`TreeMap` / `TreeSet`	`floorKey`, `ceilingKey`, `lower`, `higher`
What is element *i*?	array or `ArrayList`	`a[i]`, `get(i)`

The costs, with the measurements from Section 1.2, are collected in Table 1.5. Keep it next to you for the first week.

Table 1.5 Cost of the calls you will use most (measured: 10^6 operations, OpenJDK 25)

Structure	Calls	Cost per call	10^6 operations
`int[]`	`a[i]`, `a[i]++`	O(1)	32 ms (random cells)
`ArrayList`	`get`, `set`, `add` at the end	O(1), `add` amortised	45 ms (add, then get)
	`add(i, x)`, `remove(i)`	O(n)	
`HashMap`, `HashSet`	`get`, `put`, `merge`, `contains`	O(1) expected	170 ms (`merge`)
`ArrayDeque`	`push`, `pop`, `offer`, `poll`, `peek`	O(1) amortised	50 ms (push, then pop)
`PriorityQueue`	`add`, `poll`	O(log n)	545 ms (add, then poll)
	`peek`	O(1)	
	`remove(x)`, `contains(x)`	O(n)	
`TreeMap`, `TreeSet`	`put`, `get`, `remove`, `floorKey`, `ceilingKey`, `first`, `last`	O(log n)	630 ms (`merge`)
`Arrays.sort`	primitive array	O(n log n)	95 ms (10^6 `int` values)

Arrays and ArrayList

An `ArrayList` is an array that grows: when it is full, it allocates an array about 1.5 times larger and copies the elements across. One copy costs O(n), but it happens so rarely that n additions cost O(n) in total, which is what "O(1) amortised" means. Inserting or removing in the middle shifts every later element and costs O(n).

> **PITFALL**
>
> `List<Integer>` has two `remove` methods. `list.remove(3)` removes the element at **index** 3; `list.remove(Integer.valueOf(3))` removes the first element equal to 3. The compiler picks the index version whenever you pass an `int`.

Arrays of primitives sort in ascending order with `Arrays.sort(a)`. A custom order needs objects, so it works on `Integer[]` or on arrays of arrays such as `int[][]`, never on `int[]`. These lines are from the handbook's toolkit card:

```
int[][] pairs = {{2, 5}, {1, 9}, {2, 1}};
Arrays.sort(pairs, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0]) : Integer.compare(x[1], y[1]));
// pairs is now {1,9}, {2,1}, {2,5}
Integer[] boxed = {5, 2, 8};
Arrays.sort(boxed, Collections.reverseOrder());    // 8, 5, 2
List<String> words = new ArrayList<>(Arrays.asList("bbb", "a", "cc"));
words.sort((x, y) -> Integer.compare(x.length(), y.length()));   // a, cc, bbb
```

Chapter 4 explains which algorithm each sort uses and why that matters.

HashMap and HashSet

A hash map stores key–value pairs and finds a key in O(1) expected time; Chapter 2 explains how. Four calls do most of the work:

```
int[] a = {3, 1, 3, 2, 3};
Map<Integer, Integer> freq = new HashMap<>();
for (int x : a) freq.merge(x, 1, Integer::sum);   // same as put(x, getOrDefault(x, 0) + 1)
int threes = freq.getOrDefault(3, 0);              // 3
int nines = freq.getOrDefault(9, 0);               // 0 (missing key, no exception)
Map<String, List<String>> groups = new HashMap<>();
groups.computeIfAbsent("eilnst", k -> new ArrayList<>()).add("listen");   // build groups
```

Iteration order over a `HashMap` is unspecified. If the output must be sorted, sort the keys or use a `TreeMap`.

ArrayDeque as a stack and as a queue

A **stack** returns the most recently added item first (last in, first out); a **queue** returns the earliest (first in, first out). `ArrayDeque` is both, with O(1) operations at either end. Use it instead of `Stack`, which is an old synchronised class built on `Vector` and pays for locking on every call. `ArrayDeque` does not accept `null`.

```
ArrayDeque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
int top = stack.pop();                             // 2 (last in, first out)
ArrayDeque<Integer> queue = new ArrayDeque<>();
queue.offer(1);
queue.offer(2);
int front = queue.poll();                          // 1 (first in, first out)
```

Do not mix the two families on one deque by accident: `push` and `pop` work at the front, `offer` adds at the back and `poll` removes from the front.

PriorityQueue

A **priority queue** returns the smallest item first, whatever the order of insertion. Java's `PriorityQueue` is a binary heap (Chapter 5 builds one from scratch). The default is a min-heap; pass a comparator for anything else.

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();                          // poll() = smallest
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // poll() = largest
PriorityQueue<long[]> byFirst = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));
```

Only the top is ordered. Printing a heap holding 5, 1, 4, 2, 3 gives `[1, 2, 4, 5, 3]`, the internal array, not a sorted list. To get the items in order, `poll` them one by one.

TreeMap and TreeSet

DEFINITION 1.3 Ordered map

An **ordered map** keeps its keys sorted and answers, for any value x, "which key is nearest to x from below or above?". Java's `TreeMap` (and `TreeSet` for keys alone) is a balanced binary search tree, so every such query and every update costs $O(\log n)$.

The four neighbour queries differ only in whether x itself counts. For the set {10, 20, 30}, running the calls gives:

Table 1.6 Neighbour queries on the set {10, 20, 30}

Call	Meaning	Result for 20	Result for 5
`floor(x)`	largest key $\le x$	20	`null`
`lower(x)`	largest key $< x$	10	`null`

Call	Meaning	Result for 20	Result for 5
`ceiling(x)`	smallest key ≥ *x*	20	10
`higher(x)`	smallest key > *x*	30	10

On a `TreeMap` the same queries are `floorKey`, `lowerKey`, `ceilingKey` and `higherKey`, and `floorEntry` and friends return the key with its value. `first()`, `last()`, `pollFirst()`, `headSet(x)` (keys < *x*) and `tailSet(x)` (keys ≥ *x*) complete the toolkit. Every result can be `null`, so test before you unbox.

Worked example: Two Sum in one pass

LeetCode 1 (Two Sum) gives an array and a target and asks for the indices of two different elements that add up to the target; exactly one answer exists. The brute force of Section 1.2 checks every pair in $O(n^2)$. The one-pass method asks a hash map, at each index j, whether the partner value $t - a[j]$ appeared earlier.

Listing 1.1 Two Sum in one pass (LeetCode 1)

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();      // value -> index where it was seen
        for (int j = 0; j < nums.length; j++) {
            Integer i = seen.get(target - nums[j]);        // a partner seen earlier?
            if (i != null) return new int[]{i, j};
            seen.put(nums[j], j);                          // store after the look-up
        }
        return null;                                       // no pair (LeetCode promises one)
    }
}
```

THEOREM 1.3 **Correctness of the one-pass Two Sum**

If some pair $i < j$ has $a[i] + a[j] = t$, Listing 1.1 returns such a pair; otherwise it returns `null`*. Every pair it returns is valid.*

Proof. Invariant: when the loop starts index j, `seen` contains exactly the values $a[0], \ldots, a[j-1]$, each mapped to an index where it occurs. It holds for $j = 0$ (empty map) and the `put` at the end of each iteration maintains it. *Validity*: a returned pair (i, j) has $a[i] = t - a[j]$ by the look-up and $i < j$ by the invariant. *Completeness*: let j^* be the smallest index that is the right end of some valid pair (i, j^*). The loop does not return before j^*, because a return at $j < j^*$ would be a valid pair ending at j. At j^* the value $a[i] = t - a[j^*]$ is in `seen` by the invariant, so the method returns. ∎

Storing after the look-up matters. If `put` came first, an element could pair with itself: for $a = [3, 5]$ and $t = 6$, the element 3 would find itself at index 0 and return [0, 0].

EXAMPLE 1.2 **Tracing Two Sum**

Take $a = [4, 11, 7, 2, 8, 5]$ and $t = 13$. The table shows the map at the start of each step.

```
j  a[j]  need = t - a[j]   map before this step       found
0    4         9           {}                          -
1   11         2           {4:0}                       -
2    7         6           {4:0, 11:1}                 -
3    2        11           {4:0, 7:2, 11:1}            11 at index 1  -> return [1, 3]
```

The loop does at most n map look-ups and n insertions, so it runs in $O(n)$ expected time and $O(n)$ memory, against $O(n^2)$ time and $O(1)$ memory for the brute force.

Worked example: Valid Parentheses with a stack

LeetCode 20 (Valid Parentheses) asks whether a string of the characters `()[]{}` is balanced: every opener is closed by the matching closer, in the right order. The last opener that is still open must be closed first, which is exactly the question a stack answers. The listing pushes the closer it expects; a closer in the input must then equal the top of the stack.

Listing 1.2 Valid Parentheses with an ArrayDeque (LeetCode 20)

```
static boolean isValid(String s) {
    ArrayDeque<Character> expect = new ArrayDeque<>();   // closers we still owe, top = most recent
    for (char c : s.toCharArray()) {
        if (c == '(') expect.push(')');
        else if (c == '[') expect.push(']');
        else if (c == '{') expect.push('}');
        else if (expect.isEmpty() || expect.pop() != c) return false;  // wrong or unexpected closer
    }
    return expect.isEmpty();                             // every opener must be closed
}
```

Call a string *balanced* if it can be reduced to the empty string by repeatedly deleting an adjacent matching pair `()`, `[]` or `{}`. This is equivalent to the usual definition by nesting, and it is also the brute force the listing was tested against on 70,000 random strings.

LEMMA 1.4 **Deleting a matched pair does not change the result**

If $s = u\,x\,y\,v$, where x is an opener and y its matching closer, then `isValid(s)` = `isValid(uv)`.

Proof. On s the method processes u exactly as on uv. If it returns `false` inside u, both runs return `false`. Otherwise, at x it pushes y; at the next character, y, it pops y and the comparison succeeds. The stack is now exactly as it was after u, so the rest of the run on s processes v from the same state as the run on uv and ends the same way. ∎

THEOREM 1.5 **Correctness of the stack check**

`isValid(s)` returns `true` exactly when s is balanced.

Proof. *Balanced ⇒ true.* A balanced string reaches the empty string by some sequence of pair deletions. By Lemma 1.4 no deletion changes the method's result, and `isValid("")` is `true`. *True ⇒ balanced*, by induction on the length of s. The empty string is balanced. Let s be non-empty and accepted. Its first character is an opener, because a closer on an empty stack returns `false`, and s contains a closer, because otherwise the stack would be non-empty at the end. Let y be the first closer and x the character just before it, which is an opener. At y the method pops the closer pushed for x, and since it does not return `false`, y matches x. Deleting xy gives a shorter string that is accepted by Lemma 1.4, hence balanced by induction; so s is balanced too. ∎

Each character causes one push or one pop, so the method runs in $O(n)$ time and $O(n)$ memory. The stack also has a direct meaning at every moment: it holds, from bottom to top, the closers still owed for the openers that have not yet been matched.

EXAMPLE 1.3 **Three traces**

The stack is written from bottom to top.

```
"{[()]()}"            "([)]"                   "(()"
{  push }   }         (  push )   )            (  push )   )
[  push ]   }]        [  push ]   )]           (  push )   ))
(  push )   }])       )  pop ] != ) -> false   )  pop )    )
)  pop )    }]                                 end: stack not empty -> false
]  pop ]    }
(  push )   })
)  pop )    }
}  pop }    (empty)  -> true
```

Worked example: the k largest values with a size-k heap

Many problems end with “report the k best”. Sorting everything costs O(n log n). A min-heap that never holds more than k items costs O(n log k), and it also works when the items arrive one at a time.

Listing 1.3 The k largest values with a size-k min-heap

```java
static int[] kLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();   // min-heap: smallest of the kept k on top
    for (int x : a) {
        heap.add(x);
        if (heap.size() > k) heap.poll();                  // drop the smallest of k + 1
    }
    int[] res = new int[heap.size()];
    for (int i = res.length - 1; i >= 0; i--) res[i] = heap.poll();   // largest first
    return res;
}
```

CLAIM 1.6 **The heap keeps the k largest**

After the loop has processed a[0..j], the heap holds a multiset of min(k, j + 1) values that are the largest among a[0..j] (ties broken arbitrarily).

Proof. By induction on j. Suppose the heap holds the k largest of the first j values and x arrives. The k largest of the first $j+1$ values are among the heap’s k values and x, because any value outside the heap was already beaten by k others. Of these $k+1$ candidates the smallest is the one to drop, and `poll` removes exactly the smallest. While fewer than k values have arrived, nothing is dropped. ∎

EXAMPLE 1.4 **Tracing the size-k heap**

Take $a = [5, 1, 9, 3, 7, 9, 2, 8]$ and $k = 3$. The heap contents are listed in sorted order for readability.

```
x   dropped   heap after the step
5      -      [5]
1      -      [1, 5]
9      -      [1, 5, 9]
3      1      [3, 5, 9]
7      3      [5, 7, 9]
9      5      [7, 9, 9]
2      2      [7, 9, 9]
8      7      [8, 9, 9]
```

The result is [9, 9, 8].

Each of the n additions and at most n polls works on a heap of at most $k+1$ items, so the time is O(n log k) and the memory O(k).

LeetCode 347 (Top K Frequent Elements) applies this to counts: first count each value with a `HashMap`, then run the size-k heap over the map's entries, ordered by count instead of by value. There is also an O(n) alternative. Every count lies between 1 and n, so you can put each value into a bucket indexed by its count, `List<Integer>[] bucket = new List[n + 1]`, and read the buckets from n down to 1 until k values are collected. Writing both versions is a good exercise.

Worked example: My Calendar with TreeMap neighbours

LeetCode 729 (My Calendar I) receives bookings [*start*, *end*) one at a time and must accept a booking only if it overlaps no accepted booking. Two half-open intervals $[s, e)$ and $[s', e')$ overlap exactly when $s' < e$ and $s < e'$. Checking every accepted booking costs O(n) per request. An ordered map from start to end needs only two neighbours.

Listing 1.4 My Calendar I with floorKey and ceilingKey (LeetCode 729)

```
class MyCalendar {
    private final TreeMap<Integer, Integer> booked = new TreeMap<>();   // start -> end, [start, end)

    public boolean book(int start, int end) {
        Integer before = booked.floorKey(start);        // latest booking starting at or before start
        if (before != null && booked.get(before) > start) return false;
        Integer after = booked.ceilingKey(start);       // earliest booking starting at or after start
        if (after != null && after < end) return false;
        booked.put(start, end);
        return true;
    }
}
```

THEOREM 1.7 **Two neighbours suffice**

Let the accepted bookings be pairwise disjoint and let [s, e) be a request. If [s, e) overlaps any accepted booking, it overlaps the booking with the largest start ≤ s or the booking with the smallest start ≥ s.

Proof. Let $[s', e')$ be an accepted booking that overlaps $[s, e)$, so $s' < e$ and $s < e'$. *Case* $s' \le s$. Let f be the largest start $\le s$, so $s' \le f \le s$. If $s' < f$, the booking starting at f begins after $[s', e')$ and is disjoint from it, so $e' \le f \le s$, contradicting $s < e'$. Hence $s' = f$: the floor booking is the overlapping one. *Case* $s' > s$. Let c be the smallest start $\ge s$, so $s \le c \le s' < e$. Call that booking $[c, d)$. It satisfies $c < e$ and $s \le c < d$, so it overlaps the request. ∎

The listing tests exactly these two bookings. The floor booking overlaps when its end is greater than s; the ceiling booking overlaps when its start is less than e. Each request costs two O(log n) queries and one insertion, so n requests cost O(n log n) in total, against O(n^2) for checking every booking.

EXAMPLE 1.5 **Tracing seven requests**

Requests arrive in this order. "floor" and "ceiling" are the neighbours found for the request's start, shown as start (end).

```
request   floorKey   ceilingKey   result
[10,20)   null       null         accept
[25,30)   10 (20)    null         accept
[15,26)   10 (20)    25 (30)      reject: [10,20) ends after 15
[20,25)   10 (20)    25 (30)      accept: touching ends do not overlap
[5,10)    null       10 (20)      accept
[0,6)     null       5 (10)       reject: [5,10) starts before 6
[30,31)   25 (30)    null         accept
```

The final map is {5: 10, 10: 20, 20: 25, 25: 30, 30: 31}.

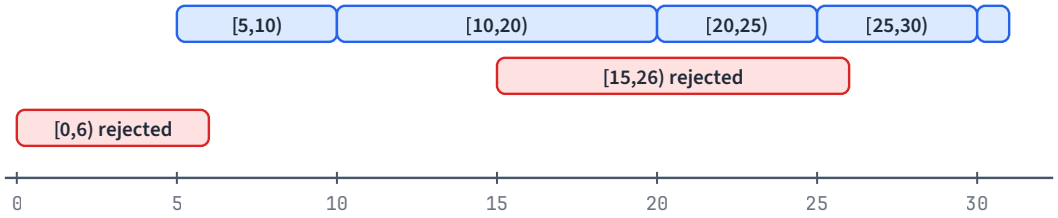


Figure 1.1 The accepted bookings of Example 1.5 (blue) and the two rejected requests (red). Each rejected request overlaps one of its two TreeMap neighbours.

1.5 Strings and characters

Immutable strings and the cost of +=

DEFINITION 1.4 **Immutable object**

An object is **immutable** if its contents can never change after it is created. Java's `String` is immutable: every method that seems to change a string, such as `concat`, `replace` or `+`, returns a new string and leaves the old one untouched.

Immutability makes strings safe to share and to use as hash-map keys. It also makes one common loop quadratic.

CLAIM 1.8 **Repeated += is quadratic**

Building a string of length n with `s += c` for one character at a time copies $1 + 2 + \cdots + n = n(n+1)/2$ characters.

Proof. The i-th `+=` cannot extend `s` in place, because `s` is immutable. It allocates a new string of length i and copies the $i - 1$ old characters and the new one. Summing i from 1 to n gives $n(n+1)/2$. ∎

`StringBuilder` keeps a mutable buffer with spare capacity and roughly doubles it when it fills, so n appends copy fewer than $2n$ characters in total: O(1) amortised per append. The measured difference on this book's machine:

Characters appended one at a time	`s += c`	`StringBuilder.append`
25,000	0.33 s	under 1 ms
50,000	0.92 s	under 1 ms
100,000	3.1 s	under 1 ms
200,000	6.5 s	6 ms

```
StringBuilder sb = new StringBuilder();
sb.append("ab").append(3).reverse();
String s = sb.toString();                    // "3ba"
boolean same = s.equals("3ba");              // true (== compares references, not text)
char[] chars = s.toCharArray();
Arrays.sort(chars);
String sorted = new String(chars);           // "3ab"
```

The same reasoning applies to output: build one large `StringBuilder` or use one `PrintWriter` (Section 1.6), never thousands of separate `System.out.println` calls.

equals versus ==

Because strings are objects, `==` compares references. String **literals** with the same text are shared (the compiler and JVM keep one copy), so `"abc" == "abc"` is `true`, which hides the bug in small tests. A string built at run time is a new object: with `s4 = "ab"; s4 = s4 + "c";` the test `"abc" == s4` is `false` while `"abc".equals(s4)` is `true`. Always use `equals`, and `compareTo` for alphabetical order (it returns a negative number, zero or a positive number).

`s.charAt(i)` costs O(1). `s.substring(l, r)` (characters $l..r-1$) **copies** them, so it costs $O(r-l)$; calling it inside a loop over all positions makes the loop quadratic.

Characters are numbers

A `char` is an unsigned 16-bit number, the character's code. Arithmetic on `char` happens in `int`, which gives three idioms you will use daily:

```
char c = 'd';
int idx = c - 'a';                // 3: position in the alphabet, an index into int[26]
char back = (char) ('a' + 3);     // 'd': the cast is needed, 'a' + 3 is an int
int digit = '7' - '0';            // 7: the digit's value
int code = 'a' + 1;               // 98, not 'b': the result of + is an int
```

`Character.isDigit`, `Character.isLetter`, `Character.toLowerCase` and `Character.isUpperCase` cover the rest.

Counting letters with int[26]

When the keys are lowercase letters, an `int[26]` indexed by `c - 'a'` replaces a `HashMap<Character, Integer>`. It needs no boxing, no hashing and no `null` checks.

Listing 1.5 Counting letters with an int[26]

```
static int[] letterCounts(String s) {          // s contains only 'a'..'z'
    int[] cnt = new int[26];
    for (char c : s.toCharArray()) cnt[c - 'a']++;
    return cnt;                                // for "banana": cnt[0] = 3, cnt[1] = 1, cnt[13] = 2
}
```

Two strings are anagrams of each other when one is a rearrangement of the other, that is when every letter occurs equally often in both. So two strings are anagrams exactly when their count arrays are equal. LeetCode 242 (Valid Anagram) needs only this fact. The neat version uses one array: add 1 for each letter of the first string, subtract 1 for each letter of the second, and check that every entry is 0. It runs in $O(n + 26)$ time with 26 integers of memory.

Counting is also the way to build a key for a group of anagrams. Sorting the letters gives the same key to every anagram ("listen" and "silent" both become "eilnst"), and `computeIfAbsent(key, k -> new ArrayList<>()).add(word)` collects each group.

1.6 Reading input fast

On LeetCode the input arrives as method arguments. In a full-program question you read it from standard input, and for 10^5 or 10^6 numbers the choice of reader decides whether the program finishes in time.

Why Scanner is slow

`java.util.Scanner` is convenient but does a lot of work per token. It matches every token against a regular expression, converts numbers in a locale-aware way, and reads the input through a small buffer of 1024

characters. `BufferedReader` instead reads standard input in large chunks and returns a whole line at a time, and `StringTokenizer` splits a line on spaces with no regular expressions at all. The plan's author timed three readers on input written one number per line, as the Infosys input is (OpenJDK 25, middle of three runs, reading time only):

Table 1.7 Time to read numbers written one per line

Reader	10^5 numbers	10^6 numbers	When to use it
`Scanner`	182 ms	818 ms	small inputs only; risky above about 10^5 numbers with a 1–2 s limit
`FastScanner` (`BufferedReader` + `StringTokenizer`)	46 ms	144 ms	the default: memorise it
`FastReader` (byte buffer)	10 ms	33 ms	only when reading millions of numbers

Output matters as much. Printing 10^6 lines with `System.out.println` took 2.0 s; the same output through one `PrintWriter` took 0.15 s.

FastScanner: the version to memorise

Listing 1.6 FastScanner and PrintWriter: a complete program that sums n numbers

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws IOException {
        FastScanner fs = new FastScanner(System.in);
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        int n = fs.nextInt();                    // one value per line or all on one line: both work
        long[] a = new long[n];
        for (int i = 0; i < n; i++) a[i] = fs.nextLong();

        long sum = 0;
        for (long x : a) sum += x;
        out.println(sum);
        out.flush();                             // without this you print nothing
    }

    /** BufferedReader reads big chunks; StringTokenizer splits a line on spaces. */
    static final class FastScanner {
        private final BufferedReader br;
        private StringTokenizer st;

        FastScanner(InputStream in) {
            br = new BufferedReader(new InputStreamReader(in));
        }

        String next() throws IOException {
            while (st == null || !st.hasMoreTokens()) {   // current line used up: read another
                String line = br.readLine();
                if (line == null) return null;           // end of input
                st = new StringTokenizer(line);
            }
            return st.nextToken();
        }

        int nextInt() throws IOException { return Integer.parseInt(next()); }

        long nextLong() throws IOException { return Long.parseLong(next()); }

        double nextDouble() throws IOException { return Double.parseDouble(next()); }

```

```
        String nextLine() throws IOException {          // the next whole line, spaces included
            st = null;                                  // (drops anything left on the current line)
            return br.readLine();
        }
    }
}
```

Run on the input `3 10 20 30`, and again on the same four numbers written one per line, the program prints 60 both times. Line by line:

- `new BufferedReader(new InputStreamReader(System.in))` reads standard input in large blocks instead of a few characters at a time.
- `next()` returns one word. If the current line has no words left, it reads the next line and wraps it in a new `StringTokenizer`. That loop is why it does not matter whether the numbers come one per line or all on one line, and why blank lines are skipped.
- `nextInt()`, `nextLong()` and `nextDouble()` convert the next word.
- `nextLine()` returns the next whole line, spaces included. Use it only for text such as a sentence; it discards anything left on the current line.
- `PrintWriter` collects output in memory. `out.flush()` at the end sends it all at once. Forget it and nothing is printed, which scores zero.
- `main` declares `throws IOException` because `readLine` can throw it.

KEY IDEA

`next()` keeps reading lines until it finds a word. That single loop makes `FastScanner` independent of how the values are spread over lines, which is exactly what one-value-per-line input needs.

Table 1.8 Moving from Scanner to FastScanner

With `Scanner` you wrote	With `FastScanner` you write
`Scanner sc = new Scanner(System.in);`	`FastScanner fs = new FastScanner(System.in);`
`sc.nextInt()`, `sc.nextLong()`	`fs.nextInt()`, `fs.nextLong()`
`sc.next()` (one word)	`fs.next()`
`sc.nextLine()`	`fs.nextLine()`; there is no trap where `nextLine()` right after `nextInt()` returns an empty string
`System.out.println(x)` in a loop	`out.println(x)`, then one `out.flush()` at the end
`main(String[] args)`	`main(String[] args) throws IOException`

When the number of values is not given, read until `next()` returns `null`: `for (String w = fs.next(); w != null; w = fs.next()) { ... }`.

One value per line

The official sample paper's Hard problem gives N, then M, then N values, each on its own line. With `FastScanner` the reading code is just the order of the statement: `int n = fs.nextInt(); int m = fs.nextInt();` and a loop of `fs.nextInt()`. If you are sure that every line holds exactly one value, `BufferedReader.readLine` alone is enough:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());       // line 1: N
int m = Integer.parseInt(br.readLine().trim());       // line 2: M
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = Integer.parseInt(br.readLine().trim());   // next N lines
```

This version fails on a blank line or on two values sharing a line, which `FastScanner` handles. Prefer `FastScanner` unless you are already sure of the layout.

INFOSYS LENS

Trust the samples, not the format text. In the sample paper's Medium problem the written input format puts p after the array, but all three samples only work if p comes right after n: the written order gives 1, 0, 0 against the expected 2, 3, 1. Run your reading code on every sample before you write the logic.

FastReader: the byte version

`FastReader` reads raw bytes 64 KB at a time and builds each number digit by digit (`r = r * 10 + digit`), without creating any `String`. It is about four times faster than `FastScanner` in Table 1.7 and harder to type from memory, so treat it as optional.

Listing 1.7 FastReader: optional byte-buffer reader

```
import java.io.*;
import java.util.*;

public class Main {
    static final long MOD = 1_000_000_007L;

    public static void main(String[] args) throws IOException {
        FastReader in = new FastReader(System.in);
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        int n = in.nextInt();              // one value per line or all on one line
        long[] a = new long[n];
        for (int i = 0; i < n; i++) a[i] = in.nextLong();

        long sum = 0;
        for (long x : a) sum += x;
        out.println(sum);

        out.flush();                        // forgetting this = empty output = 0 marks
    }

    /** Token reader: skips spaces AND newlines, so "one value per line" input is handled. */
    static final class FastReader {
        private final DataInputStream in;
        private final byte[] buf = new byte[1 << 16];
        private int len = 0, ptr = 0;

        FastReader(InputStream is) { in = new DataInputStream(is); }

        private int read() throws IOException {
            if (ptr == len) {
                len = in.read(buf, 0, buf.length);
                ptr = 0;
                if (len <= 0) return -1;
            }
            return buf[ptr++];
        }

        long nextLong() throws IOException {
            int c = read();
            while (c != '-' && (c < '0' || c > '9')) {
```

```
                if (c == -1) throw new EOFException();
                c = read();
            }
            boolean neg = c == '-';
            if (neg) c = read();
            long r = 0;
            while (c >= '0' && c <= '9') { r = r * 10 + (c - '0'); c = read(); }
            return neg ? -r : r;
        }

        int nextInt() throws IOException { return (int) nextLong(); }

        String next() throws IOException {
            int c = read();
            while (c != -1 && c <= ' ') c = read();
            if (c == -1) throw new EOFException();
            StringBuilder sb = new StringBuilder();
            while (c > ' ') { sb.append((char) c); c = read(); }
            return sb.toString();
        }
    }
}
```

`read()` refills the buffer whenever it runs out. `nextLong()` skips every byte that is not a digit or a minus sign, then accumulates digits. Run on `3 10 20 30` it prints 60; on 3, 10, 20, −30 one per line it prints 0.

PITFALL

Three input bugs cost full problems: forgetting `out.flush()`, calling `nextLine()` in the middle of a line (it drops the rest of that line), and leaving `throws IOException` off `main` (a compile error).

1.7 Linked lists

Nodes and references

DEFINITION 1.5 Singly linked list

A **linked list** is a chain of nodes in which each node holds a value and a reference `next` to the following node; the last node's `next` is `null`. The list is known only through a reference to its first node, the *head*.

A linked list gives up O(1) access by index (reaching node i takes i steps) in exchange for O(1) insertion and deletion at any node you already hold. Contest code rarely builds its own lists, but LeetCode list problems and interviews test the pointer manipulation directly, and the same moves reappear on trees.

The one skill these problems test is changing references without losing part of the list. Draw the boxes, and before each assignment ask: after this line, can I still reach every node I need? Listing 1.8 is the handbook's tested template, with LeetCode's node class and the four moves this section proves correct. Every method runs in O(n) time and O(1) extra memory.

Listing 1.8 Linked list moves: reverse, middle, cycle check, merge

```
static final class ListNode {                    // LeetCode's list node
    int val;
    ListNode next;

    ListNode(int val) { this.val = val; }
}

static ListNode reverse(ListNode head) {
```

```
    ListNode prev = null;
    while (head != null) {
        ListNode nxt = head.next;                       // save, point back, move on
        head.next = prev;
        prev = head;
        head = nxt;
    }
    return prev;
}

static ListNode middle(ListNode head) {                 // second middle for even length
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

static boolean hasCycle(ListNode head) {                // Floyd: fast catches slow inside a cycle
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

static ListNode mergeSorted(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;     // a dummy head removes special cases
    while (a != null && b != null) {
        if (a.val <= b.val) {
            tail.next = a;
            a = a.next;
        } else {
            tail.next = b;
            b = b.next;
        }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;
    return dummy.next;
}
```

Reversing a list

`reverse` (lines 8–17) walks the list once and turns each `next` reference around. Three references are alive at every moment: `prev`, the node before; `head`, the node being processed; and `nxt`, saved so that the rest of the list is not lost.

LEMMA 1.9 Reversal invariant

Let the original list be $v_1 \to v_2 \to \cdots \to v_n$. At the start of each iteration, after k iterations, `prev` is the head of the list $v_k \to v_{k-1} \to \cdots \to v_1 \to$ `null`, and `head` is the head of the untouched list $v_{k+1} \to \cdots \to v_n \to$ `null`.

Proof. Before the first iteration ($k = 0$), `prev` is the empty list and `head` is the whole list. In an iteration, `nxt` saves v_{k+2} before the only reference to it, `head.next`, is overwritten. Then v_{k+1}`.next` is set to `prev`, which makes $v_{k+1} \to v_k \to \cdots \to v_1$ the new reversed part, and `prev` and `head` move to v_{k+1} and v_{k+2}. The loop ends when `head` is `null`, that is when $k = n$, and `prev` is then the head of the fully reversed list. ∎

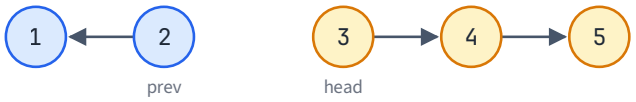


Figure 1.2 Reversing 1 → 2 → 3 → 4 → 5 after two iterations. The reversed part 2 → 1 starts at prev; the untouched part 3 → 4 → 5 starts at head.

The order of the four lines in the loop matters. Overwriting `head.next` before saving it loses the rest of the list, which is the classic bug.

Fast and slow pointers: the middle

`middle` (lines 19–26) moves `slow` one node and `fast` two nodes per iteration. When `fast` runs off the end, `slow` has covered half the distance.

> **CLAIM 1.10** **The slow pointer stops at index ⌊n/2⌋**
>
> *For a list of $n \geq 1$ nodes indexed $0..n-1$, `middle` returns node $\lfloor n/2 \rfloor$: the middle node for odd n, the second of the two middle nodes for even n.*

Proof. After t iterations, `slow` is at index t and `fast` at index $2t$ (with index n meaning `null`). The loop continues while indices $2t$ and $2t+1$ both exist, that is while $2t + 1 \leq n - 1$. It therefore stops at the first t with $2t \geq n - 1$, which is $t = \lceil (n-1)/2 \rceil = \lfloor n/2 \rfloor$. For $n = 5$ it returns index 2; for $n = 4$, index 2. ∎

The condition `fast != null && fast.next != null` must be written in that order: Java evaluates `&&` from left to right and skips the second test when the first is false, so `fast.next` is never read on a `null`.

Floyd's cycle detection

A list has a *cycle* if some node's `next` points back to an earlier node, so following `next` never reaches `null`. Remembering every visited node in a `HashSet` detects this with $O(n)$ memory. Floyd's method, `hasCycle` (lines 28–36), uses the same two pointers and $O(1)$ memory: if there is a cycle, the fast pointer enters it and eventually laps the slow one.

> **THEOREM 1.11** **Floyd's method is correct and fast**
>
> *If the list has no cycle, `hasCycle` returns `false`. If the list has a tail of μ nodes followed by a cycle of λ nodes, the pointers meet after exactly t steps, where t is the smallest multiple of λ that is at least $\max(\mu, 1)$. In particular $t \leq \mu + \lambda$.*

Proof. Number the nodes by their distance from the head: node i is reached after i steps. After t steps `slow` is at node t and `fast` at node $2t$. *No cycle*: `fast` is always strictly ahead of `slow` ($2t > t$ for $t \geq 1$) on a path that ends, so the two never coincide, and the loop stops when `fast` reaches the end. *Cycle*: the nodes from index μ onwards repeat with period λ, so for positions $x, y \geq \mu$ the nodes are the same exactly when $x \equiv y \pmod{\lambda}$. The pointers coincide after $t \geq 1$ steps exactly when $t \geq \mu$ (so that `slow` is on the cycle) and $2t \equiv t \pmod{\lambda}$, that is when λ divides t. The smallest such t is the smallest multiple of λ that is $\geq \max(\mu, 1)$. It is at most $\mu + \lambda$, because among the λ consecutive integers $\max(\mu, 1), \ldots, \max(\mu, 1) + \lambda - 1$ one is a multiple of λ, and $\max(\mu, 1) + \lambda - 1 \leq \mu + \lambda$. ∎

The formula was checked by running the method on every list with $0 \leq \mu \leq 12$ and $1 \leq \lambda \leq 12$: the meeting step always equalled it.

EXAMPLE 1.6 Tracing Floyd's method

Take nodes 0..7 with a tail 0, 1, 2 (μ = 3) and a cycle 3 → 4 → 5 → 6 → 7 → 3 (λ = 5). The smallest multiple of 5 that is ≥ 3 is 5.

```
t     0  1  2  3  4  5
slow  0  1  2  3  4  5
fast  0  2  4  6  3  5    <- meet at node 5 after 5 steps
```

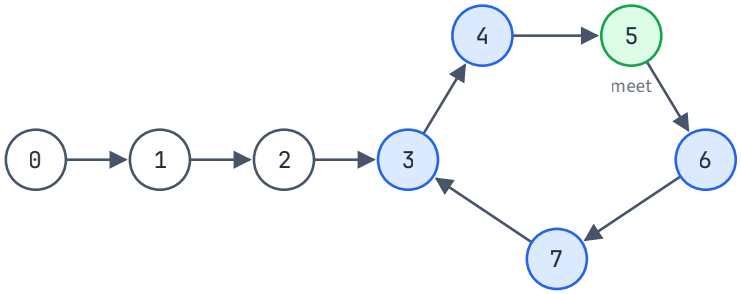


Figure 1.3 The list of Example 1.6: a tail of three nodes and a cycle of five. Slow and fast meet at node 5.

Merging with a dummy head

`mergeSorted` (lines 38–52) builds a new chain by repeatedly attaching the smaller front node of the two lists. The first attachment would be a special case, because there is no tail yet, so the method starts from a *dummy* node whose `next` becomes the answer.

The invariant is that `dummy.next` … `tail` is the sorted merge of the nodes already taken, and every node still in `a` or `b` is at least `tail.val`. Taking the smaller front node keeps both halves true. When one list runs out, the other is already sorted and not smaller than `tail`, so it is attached whole. Each node is attached once: $O(n + m)$ time and $O(1)$ extra memory. Using `<=` takes from `a` on ties, which keeps the merge stable.

All four methods were tested on 20,000 random lists of up to 10 nodes against array-based versions (reverse an array, take index $\lfloor n/2 \rfloor$, add a random back edge, sort the concatenation).

PITFALL

Before every `fast.next.next` check `fast != null && fast.next != null`. And reach for a dummy head whenever the first node of the result might change; it removes the special case for an empty result.

1.8 Recursion and the call stack

Frames

DEFINITION 1.6 Call stack

Each method call that has started and not yet returned owns a *frame* holding its parameters, local variables and the place to return to. The frames form the **call stack**: a call pushes a frame and a return pops it. The *depth* of a recursion is the largest number of its frames alive at once.

A Java thread's stack has a fixed size, set when the thread starts. When a recursion goes deeper than the stack allows, the JVM throws `StackOverflowError` and the program dies. The limit depends on the frame size and the JVM. On this book's machine, with default settings, a recursion with one `int` parameter succeeded at depth 10^4 and failed at $2 \cdot 10^4$; a parameterless method reached about 20,000 frames. Plan for **trouble somewhere between 10^4 and 10^5**.

Trees in the Infosys paper arrive as **parent arrays**: `parent[v]` is the parent of node v, and the root has parent −1. The depth of a node, its number of edges to the root, has a one-line recursive definition and an equally short loop.

Listing 1.9 The depth of one node: recursive and iterative

```
static int depthRec(int[] parent, int v) {
    return parent[v] == -1 ? 0 : 1 + depthRec(parent, parent[v]);   // one frame per ancestor
}

static int depthIter(int[] parent, int v) {
    int d = 0;
    for (int u = v; parent[u] != -1; u = parent[u]) d++;           // one frame in total
    return d;
}
```

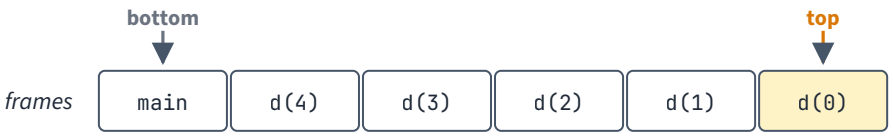


Figure 1.4 The call stack while depthRec runs on node 4 of a chain 0 ← 1 ← 2 ← 3 ← 4. Each call waits for the one to its right; the rightmost, on the root, returns first.

Both methods take time proportional to the depth. The recursive one also needs one frame per ancestor. On a chain of 10^5 nodes, run for this book, `depthIter` returned 99999 and `depthRec` threw `StackOverflowError`. Random trees are shallow, so a recursive solution can pass every small test and crash on the one test that is a long chain. That is how the official sample paper's trees, with up to 10^5 nodes, catch recursive solutions.

Rewriting recursion as a loop

There are two standard rewrites.

1. **The recursive call is the last thing that happens** (or can be made so with an accumulator, as `d` in `depthIter`). Then the method is a loop in disguise: replace the call by updating the variables and going round again.
2. **Work remains after the call returns.** Then keep the pending calls on an explicit `ArrayDeque` used as a stack. The deque lives on the heap, which is limited only by the memory limit, not by the thread stack.

Listing 1.10 computes the depth of every node, remembering the depths already known. In the recursive version the assignment to `depth[v]` must wait until the depth of the parent is known, so it is work that remains after the call. The iterative version walks up from v, pushing the nodes whose calls would be pending, until it reaches a node of known depth or the root; then it pops them, deepest call first, and does the remaining work in the order the calls would return.

Listing 1.10 The depth of every node: recursion with memory, and an explicit stack

```
static void allDepthsRec(int[] parent, int[] depth, int v) {        // depth[] starts filled with -1
    if (depth[v] != -1) return;
    if (parent[v] == -1) { depth[v] = 0; return; }
```

```
    allDepthsRec(parent, depth, parent[v]);
    depth[v] = depth[parent[v]] + 1;                          // runs after the call returns
}

static int[] allDepthsIter(int[] parent) {
    int n = parent.length;
    int[] depth = new int[n];
    Arrays.fill(depth, -1);
    ArrayDeque<Integer> pending = new ArrayDeque<>();
    for (int v = 0; v < n; v++) {
        int u = v;
        while (depth[u] == -1 && parent[u] != -1) {           // the calls that would be made
            pending.push(u);
            u = parent[u];
        }
        if (depth[u] == -1) depth[u] = 0;                     // u is the root
        while (!pending.isEmpty()) {                          // the returns, deepest call first
            int w = pending.pop();
            depth[w] = depth[parent[w]] + 1;
        }
    }
    return depth;
}
```

Every node is pushed at most once over the whole run, because its depth is known from the moment it is popped, so `allDepthsIter` costs O(n) time and O(n) memory whatever the shape of the tree. For `parent` = [−1, 0, 1, 1, 3, 0] it returns [0, 1, 2, 2, 3, 1]. Both versions were compared on 5,000 random trees of up to 30 nodes, a quarter of them chains. Chapter 15 and Chapter 18 use the same idea for depth-first search and tree traversal without recursion.

The big-stack thread

When rewriting would take too long in an exam, run the whole solution on a new thread with a large stack. The fourth argument of the `Thread` constructor is the requested stack size in bytes.

Listing 1.11 Deep recursion on a thread with a 128 MB stack (fallback)

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        // Run the real work on a thread with a 128 MB stack so deep recursion doesn't crash.
        Thread t = new Thread(null, () -> {
            try { solve(); } catch (IOException e) { throw new UncheckedIOException(e); }
        }, "solver", 1 << 27);
        t.start();
    }

    static void solve() throws IOException {
        int n = 1_000_000;                                    // a path of 1e6 nodes
        System.out.println(depth(n));
    }

    static int depth(int k) { return k == 0 ? 0 : 1 + depth(k - 1); }
}
```

This program recurses to depth 10^6 and prints 1000000. The JVM treats the stack size as a request, so prefer the iterative rewrite when you have the time, and keep this as the fallback.

Java 8 compatibility and the traps table

You do not know which Java version the exam's compiler uses, so write Java 8: no `var`, no records, no `List.of` or `Map.of`, no switch expressions, no text blocks. Every listing in this book compiles with `javac --release 8`. Table 1.9 collects the traps from the whole chapter.

Table 1.9 Java traps that cost marks

Trap	What happens	Fix
`int` products and sums	silent overflow, wrong answers ($10^9 \times 10^9$)	`long` by default; cast before multiplying: `(long) a * b`
`%` on negative numbers	`-3 % 5 == -3`	`Math.floorMod(x, m)`
comparator `(x, y) -> x[0] - y[0]`	overflows for large values	`Integer.compare(x[0], y[0])`
`==` on `Integer` / `Long` objects	`false` for values outside −128..127	`.equals()`, or unbox to `int`
recursive DFS on 10^5 nodes	`StackOverflowError`	an explicit stack, or the big-stack thread
`Stack` / `Vector`	synchronised, so slower	`ArrayDeque`
`new long[100000][100000]`	out of memory	estimate first: cells × 8 bytes
newer syntax (`var`, records, `List.of`)	compile error on an old JDK	write Java 8 style
`s += c` in a loop	quadratic time	`StringBuilder`
`System.out.println` 10^6 times	about 2 s of output alone	one `PrintWriter`, one `flush()`

Summary

- The Infosys paper has three (sometimes four) problems worth 20, 30 and 50 marks, scored by test cases passed. Input comes one value per line; some questions give a function stub whose signature you must keep.
- $f(n) = O(g(n))$ means $f \le c \cdot g$ from some n_0 on. Plan with about 10^8–$3 \cdot 10^8$ basic steps per second, and turn the largest n into a target complexity before coding.
- `int` arithmetic wraps modulo 2^{32} without warning. Use `long` for sums and products, cast before multiplying, and use `Math.floorMod` for remainders.
- `==` compares references: it fails on `Integer` values outside −128..127 and on strings built at run time. Use `equals`.
- Choose the collection by its question: `HashMap` for counts, `ArrayDeque` for last-in or first-in, `PriorityQueue` for the smallest so far, `TreeMap` for the nearest key. A size-k heap finds the k largest in $O(n \log k)$.
- Strings are immutable, so `+=` in a loop is quadratic; use `StringBuilder`. Count letters in an `int[26]`.
- `FastScanner` reads tokens regardless of line layout and is about five times faster than `Scanner`; print through one `PrintWriter` and flush it.
- Linked-list code is correct when it never loses a reference: save `next` before overwriting it, check `fast` and `fast.next` before stepping twice, and use a dummy head.
- Recursion deeper than about 10^4–10^5 frames crashes; rewrite it with a loop or an explicit stack, or run on a big-stack thread.

Key terms

big-O notation p. 10
two's complement p. 12
ordered map p. 15
immutable p. 20
linked list p. 25
call stack p. 28
autoboxing p. 13
stack p. 13
queue p. 13
priority queue p. 15

Review questions

1. Show that $n^2 + 100n = \mathrm{O}(n^2)$ by giving explicit constants c and n_0. Is $n^2 + 100n = \mathrm{O}(n)$? Prove your answer.
2. A solution sorts $n = 2 \cdot 10^5$ numbers and then, for each number, does a binary search. Estimate the number of basic steps and say whether it fits in one second.
3. What does `int x = 50_000 * 50_000;` store, and why? Write the line correctly for a `long` result.
4. `Integer a = 100, b = 100, c = 1000, d = 1000;` What are `a == b`, `c == d` and `c.equals(d)`, and why does the first differ from the second?
5. In Listing 1.1, what goes wrong if `seen.put(nums[j], j)` is moved before the look-up? Give a failing input.
6. Why does Theorem 1.7 need the accepted bookings to be pairwise disjoint? Give three bookings, two of which overlap, for which checking only the floor and ceiling neighbours misses an overlap with the request $[5, 6)$.
7. You must report the 10 largest of 10^6 numbers arriving one at a time. Compare sorting with a size-10 heap in time and memory.
8. Why is `FastScanner` indifferent to whether the input has one value per line or all values on one line, while the `readLine` fragment of Section 1.6 is not?
9. Using Theorem 1.11, after how many steps do slow and fast meet on a list with $\mu = 7$ and $\lambda = 3$? With $\mu = 0$ and $\lambda = 4$?
10. A recursive function on a tree with 10^5 nodes works on random trees but crashes on some tests. What is the likely shape of the failing trees, and what are two ways to fix the program?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 1 "Introduction" (input and output, working with numbers) and Chapter 2 "Time complexity" (calculation rules and estimating efficiency from the input size). Chapter 4 "Data structures" covers dynamic arrays, sets, maps, deques and priority queues in C++ terms. Free at cses.fi/book.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms* (4th edition), the chapter *Characterizing Running Times*, for the formal treatment of O, Ω and Θ.
- Java 8 API documentation, docs.oracle.com/javase/8/docs/api/: the class pages for `java.util.ArrayDeque`, `java.util.PriorityQueue`, `java.util.TreeMap` and `java.util.HashMap` list every method with its exact behaviour on missing keys and `null`.
- Workbook Chapter 1 has gap-filling, tracing and bug-finding exercises on this chapter's listings.

Problems

These are the problems for Day 1. Work them in Workbook Chapter 1, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
1.P1	LC 1 · Two Sum	EASY	CORE
1.P2	LC 242 · Valid Anagram	EASY	CORE
1.P3	LC 20 · Valid Parentheses	EASY	CORE
1.P4	LC 347 · Top K Frequent Elements	MEDIUM	CORE
1.P5	LC 729 · My Calendar I	MEDIUM	CORE
1.P6	LC 206 · Reverse Linked List	EASY	BONUS
1.P7	LC 141 · Linked List Cycle	EASY	BONUS
1.P8	LC 49 · Group Anagrams	MEDIUM	BONUS

CHAPTER 2 · DAY 2 OF 29 · SUNDAY 8 NOVEMBER 2026 · LEARNING DAY

2

Prefix Sums, Difference Arrays and Hashing

Count subarrays by remembering earlier prefix values in a HashMap.

Part I · Foundations
Handbook cards · Prefix sums + HashMap, Difference array (many range adds)
Before this chapter · Chapter 1 (HashMap, long arithmetic)
Practice · Workbook Chapter 2 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- build a prefix-sum array and answer any range-sum query in O(1) time;
- count subarrays whose sum, remainder or XOR equals a target in O(n) time with a hash map of earlier prefixes;
- explain why the map starts with the empty prefix, and handle negative remainders in Java;
- apply many range additions with a difference array and rebuild the array in one pass;
- extend both ideas to two-dimensional grids with inclusion–exclusion.

Many array problems ask about *contiguous* pieces of the array: a range of days, a run of characters, a block of cells. Asking about every piece directly is expensive, because an array of length n has $n(n+1)/2$ non-empty contiguous pieces. This chapter shows how one pass of preprocessing turns each question about a piece into a question about two *points*, and how a hash map then answers "how many earlier points match this one?" in constant time. The same trick, run backwards, applies thousands of range updates in time proportional to the array length.

The chapter is short on new code and long on one idea. That idea comes back in sliding windows (Chapter 3), Fenwick trees (Chapter 20) and XOR problems (Chapter 8), so it is worth understanding completely.

2.1 The range-sum problem

Suppose an array a of n integers is fixed and you must answer q questions of the form "what is $a[l] + a[l+1] + \cdots + a[r]$?". Adding the elements for each question costs $O(n)$ per query and $O(nq)$ in total, which for $n = q = 10^5$ is 10^{10} additions: far beyond the roughly 10^8 simple operations Java performs in a second.

The fix is to precompute running totals once.

DEFINITION 2.1 **Prefix sum**

For an array $a[0..n-1]$, the **prefix-sum array** P has $n+1$ entries: $P[0] = 0$ and $P[i] = a[0] + a[1] + \cdots + a[i-1]$ for $1 \le i \le n$. Equivalently, $P[i+1] = P[i] + a[i]$.

The extra leading zero is deliberate. $P[i]$ is the sum of the first i elements, a **half-open** range $[0, i)$. With that convention every range sum is a single subtraction and there is no special case for ranges that start at index 0.

THEOREM 2.1 Range sums from prefix sums

For $0 \le l \le r < n$, $a[l] + \cdots + a[r] = P[r+1] - P[l]$.

Proof. $P[r+1] = a[0] + \cdots + a[r]$ and $P[l] = a[0] + \cdots + a[l-1]$. Subtracting cancels the common terms $a[0]$, ..., $a[l-1]$ and leaves exactly $a[l] + \cdots + a[r]$. When $l = 0$ the second sum is empty and $P[0] = 0$, so the formula still holds. ∎

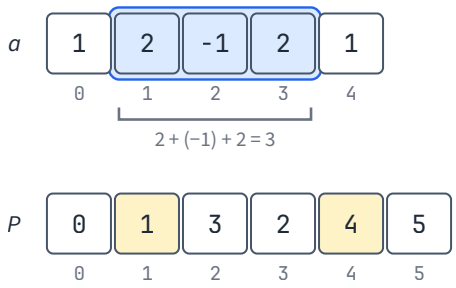


Figure 2.1 The array a and its prefix sums P. The sum of a[1..3] (shaded) is P[4] − P[1] = 4 − 1 = 3.

Listing 2.1 Building prefix sums and answering a range query

```
// P[i] = a[0] + ... + a[i-1], so sum(a[l..r]) = P[r + 1] - P[l]
static long[] prefixSums(int[] a) {
    long[] p = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    return p;
}

static long rangeSum(long[] p, int l, int r) {   // inclusive l..r
    return p[r + 1] - p[l];
}
```

Building P costs $O(n)$ time and $O(n)$ memory; each query afterwards costs $O(1)$. For q queries the total is $O(n + q)$ instead of $O(nq)$.

JAVA NOTE

The prefix array is `long[]` even though a is `int[]`. With $n = 10^5$ values of size up to 10^9, a running total reaches 10^{14}, which overflows `int` (maximum about $2.1 \cdot 10^9$) silently: Java does not throw an exception on integer overflow. Use `long` for every running sum unless you have checked the bound.

The prefix array is a small example of a general bargain in algorithm design: spend time once on preprocessing so that each later question is cheap. The bargain only pays when the array does not change. If values change between queries, every prefix after the changed position is wrong, and you need the Fenwick tree of Chapter 20.

2.2 Counting subarrays with a given sum

Now a harder question. Given a and a target k, how many subarrays have sum exactly k? This is LeetCode 560, the first problem of Day 2, and the template for a whole family.

By Theorem 2.1, the subarray $a[l..r]$ has sum k exactly when $P[r+1] - P[l] = k$. Writing $j = r + 1$ and $i = l$, the question becomes:

> How many pairs of indices $i < j$ in the prefix array satisfy $P[j] - P[i] = k$, that is, $P[i] = P[j] - k$?

A question about subarrays has become a question about **pairs of points**. Checking every pair is $O(n^2)$, which is the brute force you would write for partial marks. The faster idea is to scan j from left to right and ask, at each j, how many *earlier* prefix values equal $P[j] - k$. A hash map from "prefix value" to "how many times seen so far" answers that in O(1) on average.

KEY IDEA

Scan left to right. Before you record the current prefix, look up how many earlier prefixes equal `prefix - k`. Each one is the left end of a subarray that ends here with sum k.

Listing 2.2 Number of subarrays with sum exactly k (negative values allowed)

```java
static long countSubarraysWithSum(int[] a, long k) {
    Map<Long, Integer> seen = new HashMap<>();   // prefix value -> how many times seen so far
    seen.put(0L, 1);                             // the empty prefix P[0] = 0
    long prefix = 0, count = 0;
    for (int x : a) {
        prefix += x;
        count += seen.getOrDefault(prefix - k, 0);   // earlier prefix p with prefix - p == k
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

EXAMPLE 2.1 **Tracing the count for k = 3**

Take $a = [1, 2, -1, 2, 1]$ and $k = 3$. The map starts as {0: 1}. Each row shows the state after reading $a[i]$.

```
i  a[i]  prefix  look up prefix-k  found  total   map after this step
0    1      1          -2            0      0     {0:1, 1:1}
1    2      3           0            1      1     {0:1, 1:1, 3:1}        a[0..1] = 1+2
2   -1      2          -1            0      1     {0:1, 1:1, 2:1, 3:1}
3    2      4           1            1      2     {..., 4:1}              a[1..3] = 2-1+2
4    1      5           2            1      3     {..., 5:1}              a[3..4] = 2+1
```

The answer is 3. Notice row 1: the subarray $a[0..1]$ starts at index 0, and it is found only because the map was seeded with the empty prefix.

THEOREM 2.2 **Correctness of the prefix-count scan**

When the loop has processed $a[0..j-1]$, `count` equals the number of subarrays with sum k that end at an index $\le j-1$, and `seen` maps each value v to the number of indices $i \le j$ with $P[i] = v$.

Proof. By induction on j. Before the loop ($j = 0$) no subarray has been processed, `count` = 0, and `seen` = {0: 1} records $P[0] = 0$, so the invariant holds. Suppose it holds after j elements and the loop reads $a[j]$. Then `prefix` becomes $P[j+1]$. The subarrays with sum k that end exactly at index j are $a[i..j]$ with $P[i] = P[j+1] - k$ and $0 \le i \le j$; by the invariant `seen` holds exactly the count of those i, so adding it to `count` keeps the first half of the invariant. Recording $P[j+1]$ in `seen` restores the second half. After the last element, `count` is the number of all subarrays with sum k. ∎

Order matters inside the loop. Looking up *before* recording the current prefix guarantees $i < j + 1$, so the empty subarray is never counted when $k = 0$. Swap the two lines and LeetCode 560 with $k = 0$ counts every position once too often.

PITFALL

Forgetting `seen.put(0L, 1)` is the most common bug in this pattern. It silently misses every subarray that starts at index 0, and the samples often still pass.

Why a sliding window does not work here

If every $a[i]$ were positive, the sum of a window would grow when the right end moves and shrink when the left end moves, and the two-pointer window of Chapter 3 would find all windows with sum k in $O(n)$. With negative numbers that monotonicity is gone: extending a window can lower its sum. The prefix-count method makes no assumption about signs, which is why it is the default whenever the problem allows negatives or zeros.

Cost

The loop does n map operations. A Java `HashMap` performs `get`, `put` and `merge` in $O(1)$ *expected* time, so the scan is $O(n)$ expected time and $O(n)$ memory for up to $n + 1$ distinct prefix values. Section Section 2.4 looks at what "expected" hides.

2.3 One key, many questions

The scan in Listing 2.2 has three moving parts: the **key** stored for each prefix, the **look-up** made at each step, and **what is stored** (a count or an index). Changing them solves a family of problems.

Table 2.1 Variants of the prefix-and-map scan

Question	Key stored for each prefix	Look up	Store
subarrays with sum k	P	$P - k$	count
subarrays with sum divisible by k	P mod k (non-negative)	the same remainder	count
subarrays with odd sum	parity of P	the opposite parity	count
longest subarray with sum k	P	$P - k$	first index
longest subarray with equal 0s and 1s	P after mapping $0 \to -1$	the same P	first index
subarrays with XOR k	$X = a[0] \oplus \cdots \oplus a[i-1]$	$X \oplus k$	count

Remainders

A subarray sum $P[j] - P[i]$ is divisible by k exactly when $P[j] \equiv P[i] \pmod{k}$, so the key is the remainder of the prefix and the look-up is the same remainder. The number of good subarrays is then $\Sigma\, c(c-1)/2$ over the counts c of each remainder, or equivalently the running sum of "how many earlier prefixes had my remainder".

JAVA NOTE

Java's `%` keeps the sign of the left operand: `-3 % 5` is `-3`, not `2`. Two prefixes with remainders -3 and 2 are congruent mod 5 but would be stored under different keys. Use `Math.floorMod(prefix, k)`, which always returns a value in $[0, k)$.

Listing 2.3 Subarrays whose sum is divisible by k (LeetCode 974)

```
static int subarraysDivByK(int[] a, int k) {
    int[] cnt = new int[k];          // remainders are 0..k-1, so an array beats a map
    cnt[0] = 1;                      // the empty prefix
```

```
    int prefix = 0, total = 0;
    for (int x : a) {
        prefix = Math.floorMod(prefix + x, k);
        total += cnt[prefix];
        cnt[prefix]++;
    }
    return total;
}
```

When the keys come from a small range, as remainders $0..k-1$ do, an `int[]` replaces the map. It is faster, uses no boxing, and has no worst case.

Longest instead of how many

To find the *longest* subarray with sum k, keep for each prefix value the *first* index where it appeared and never overwrite it. At index j, if $P[j]-k$ was first seen at index i, the subarray $a[i..j-1]$ has length $j-i$, the longest ending at j. LeetCode 525 (Contiguous Array) is this question after rewriting every 0 as −1: a subarray with as many 0s as 1s is a subarray with sum 0.

Listing 2.4 Longest subarray with sum k: store first occurrences

```
static int longestWithSum(int[] a, long k) {
    Map<Long, Integer> first = new HashMap<>();
    first.put(0L, 0);                       // P[0] = 0 appears at index 0
    long prefix = 0;
    int best = 0;
    for (int j = 1; j <= a.length; j++) {
        prefix += a[j - 1];                 // prefix == P[j]
        Integer i = first.get(prefix - k);
        if (i != null) best = Math.max(best, j - i);
        first.putIfAbsent(prefix, j);       // keep the earliest index only
    }
    return best;
}
```

Parity, and why counting can be even simpler

For "how many subarrays have an odd sum" (LeetCode 1524) the key is the parity of P, which has only two values. A subarray is odd exactly when its two end prefixes have different parities, so the answer is (number of even prefixes) × (number of odd prefixes), counting $P[0] = 0$ as even. A one-line formula replaces the map entirely. When a variant's key space is tiny, look for such a closed form.

2.4 Hashing in Java

The $O(n)$ bound in Section 2.2 rests on hash-map operations taking constant time. It is worth knowing what that means.

A **hash table** stores entries in an array of *buckets*. To find key x, Java computes `x.hashCode()`, mixes its bits, and uses the low bits as a bucket index. Entries whose hashes land in the same bucket form a short list. When the table is more than 75 % full (the default **load factor**), it doubles its bucket array and redistributes the entries, which costs $O(n)$ once but $O(1)$ **amortised** per insertion.

If the keys spread evenly, each bucket holds $O(1)$ entries and every operation is $O(1)$ on average. If an adversary picks keys that all collide, a bucket becomes long. Since Java 8, a bucket with more than eight entries is converted into a balanced tree, so the worst case is $O(\log n)$ per operation, not $O(n)$. Contest problems occasionally include such "anti-hash" tests for `HashMap<Long, …>`; the Infosys tests are not reported to.

There are also constant-factor costs that the O(1) hides:

- **Boxing.** `HashMap<Long, Integer>` stores objects, not primitives. Every `put` may allocate a `Long` and an `Integer`, and every look-up unboxes. For 10^6 operations this is noticeable but fine; for 10^7 it can exceed a 1–2 second limit.
- **Small key ranges.** When keys lie in a known range of size up to about 10^7, an array indexed by the key (shifted by an offset if keys can be negative) is several times faster.
- **Pre-sizing.** `new HashMap<>(2 * n)` avoids the repeated doublings when you know roughly how many keys are coming.

JAVA NOTE

Three map calls do most of the work in this chapter. `getOrDefault(k, 0)` returns 0 for a missing key instead of `null` (unboxing `null` throws a `NullPointerException`). `merge(k, 1, Integer::sum)` adds 1 to the count, inserting 1 if the key is new. `putIfAbsent(k, v)` writes only when the key is missing, which is exactly "remember the first index".

2.5 Difference arrays

The second half of the chapter runs the prefix idea in reverse. Suppose you receive m updates of the form "add v to every element of $a[l..r]$" and only need the final array. Applying each update element by element costs $O(n)$ per update and $O(nm)$ in total.

DEFINITION 2.2 Difference array

The **difference array** of $a[0..n-1]$ is $d[0..n]$ with $d[0] = a[0]$, $d[i] = a[i] - a[i-1]$ for $1 \le i < n$, and $d[n] = 0$ as a spare slot.

Taking prefix sums of d gives back a: $d[0] + d[1] + \cdots + d[i]$ telescopes to $a[i]$. So d and a carry the same information, and some changes are much cheaper to make in d.

THEOREM 2.3 Range add as two point updates

Adding v to every element of $a[l..r]$ changes the difference array in exactly two places: $d[l]$ increases by v and $d[r+1]$ decreases by v.

Proof. Inside the range, $a[i]$ and $a[i-1]$ both increase by v, so their difference is unchanged. At $i = l$, only $a[l]$ increases, so $d[l]$ grows by v. At $i = r+1$, only $a[r]$ increased, so $d[r+1] = a[r+1] - a[r]$ falls by v. Every other difference involves two unchanged elements. ∎

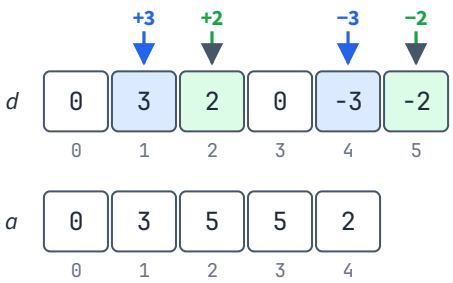


Figure 2.2 Two range additions on an array of five zeros: +3 on [1..3] and +2 on [2..4]. Each update writes two numbers into d; one running sum rebuilds a.

Listing 2.5 Applying many range additions with a difference array

```java
// Apply many "add v to a[l..r]" updates, then read the array once. O(n + updates).
static long[] applyRangeAdds(int n, int[][] updates) {   // each update = {l, r, v}, inclusive
    long[] d = new long[n + 1];
    for (int[] u : updates) {
        d[u[0]] += u[2];          // start adding at l
        d[u[1] + 1] -= u[2];      // stop after r
    }
    long[] a = new long[n];
    long running = 0;
    for (int i = 0; i < n; i++) {
        running += d[i];
        a[i] = running;
    }
    return a;
}
```

The array d has $n + 1$ slots so that $d[r + 1]$ exists when $r = n - 1$; that last slot is never read back. The total cost is $O(n + m)$.

EXAMPLE 2.2 Corporate flight bookings

LeetCode 1109 gives n flights numbered $1..n$ and bookings (*first*, *last*, *seats*): reserve *seats* on every flight from *first* to *last*. It asks for the total seats on each flight. This is Listing 2.5 after shifting the 1-based flight numbers to 0-based indices: `d[first - 1] += seats; d[last] -= seats;` followed by one running sum.

Counting coverage

A special case appears constantly: given intervals $[s, e]$, how many intervals cover each point? Each interval is a range addition of 1. After the running sum, $a[x]$ is the number of intervals covering x, and the maximum of a is the largest overlap. That maximum is the answer to "minimum number of meeting rooms" and "minimum railway platforms", an Infosys repeat problem you will meet again with a sweep line in Chapter 4.

EXAM TIP

A difference array needs every update before any read. If updates and reads are interleaved ("add 5 to [2..7]; what is a[4] now?"), switch to a Fenwick tree with range update (Chapter 20) or a lazy segment tree (Chapter 21).

2.6 Two dimensions

Both ideas extend to a grid g with R rows and C columns. Define $S[i][j]$ as the sum of the rectangle of rows $0..i - 1$ and columns $0..j - 1$, with a row and a column of zeros at index 0.

THEOREM 2.4 Rectangle sums by inclusion–exclusion

$S[i + 1][j + 1] = g[i][j] + S[i][j + 1] + S[i + 1][j] - S[i][j]$, and the sum of rows r1..r2 and columns c1..c2 is $S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1] + S[r1][c1]$.

The subtraction removes the strip above the rectangle and the strip to its left; their overlap was removed twice, so it is added back once. This is the **inclusion–exclusion principle** in its smallest form.

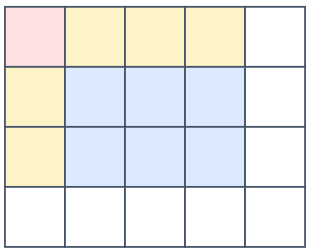


Figure 2.3 Inclusion–exclusion on a grid. The query rectangle (blue) is the big prefix minus the strip above (amber) minus the strip to the left (amber), plus the corner that was subtracted twice (red).

Listing 2.6 Two-dimensional prefix sums (LeetCode 304)

```
static long[][] prefix2D(int[][] g) {
    int R = g.length, C = g[0].length;
    long[][] s = new long[R + 1][C + 1];
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            s[i + 1][j + 1] = g[i][j] + s[i][j + 1] + s[i + 1][j] - s[i][j];
    return s;
}

static long rectSum(long[][] s, int r1, int c1, int r2, int c2) {   // inclusive corners
    return s[r2 + 1][c2 + 1] - s[r1][c2 + 1] - s[r2 + 1][c1] + s[r1][c1];
}
```

The two-dimensional difference array is the same reasoning backwards: to add v to a rectangle, add v at $(r1, c1)$, subtract it at $(r1, c2 + 1)$ and $(r2 + 1, c1)$, and add it back at $(r2 + 1, c2 + 1)$. A 2-D prefix sum of the result rebuilds the grid.

Summary

- A prefix-sum array with a leading zero turns any range sum into one subtraction, $P[r + 1] - P[l]$, after $O(n)$ preprocessing.
- "Subarray with property X" usually becomes "pair of prefixes with property Y". Scanning left to right with a hash map of earlier prefixes counts such pairs in $O(n)$ expected time.
- Seed the map with the empty prefix, look up before you record, and use `long` for sums and `Math.floorMod` for remainders.
- Store counts to answer "how many", first indices to answer "how long".
- Prefer an `int[]` to a `HashMap` when the keys lie in a small known range.
- A difference array applies a range addition with two point updates; one running sum rebuilds the array. It needs all updates before any read.
- In two dimensions, inclusion–exclusion gives rectangle sums and rectangle additions in $O(1)$ each.

Key terms

prefix-sum array p. 34
difference array p. 39
hash table p. 38
load factor p. 38
inclusion–exclusion principle p. 40

Review questions

1. Why does the prefix array have $n + 1$ entries instead of n? What would the range-sum formula look like without the leading zero?
2. In Listing 2.2, what goes wrong if the `getOrDefault` line and the `merge` line are swapped? Give an input where the answer changes.
3. An array contains only positive numbers. Name two methods that count subarrays with sum k in $O(n)$ and say which one uses less memory.
4. Why is `Math.floorMod(prefix, k)` needed in Listing 2.3? Give values of `prefix` and k for which `prefix % k` and `Math.floorMod(prefix, k)` differ.
5. How many subarrays of [1, 1, 1] have an odd sum? Compute it with the parity formula of Section 2.3 and check by listing them.
6. You need the longest subarray with sum 0. Why must the map keep the *first* index of each prefix and not the latest?
7. Five updates add 1 to the ranges [0..2], [1..3], [2..4], [0..4] and [3..3] of an array of length 5. Write the difference array after all updates and the final array.
8. Give the 2-D formula for the sum of a single cell (i, j) using S, and check that it reduces to $g[i][j]$.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 9 "Range queries", the section on sum queries and prefix sum arrays, including the two-dimensional case. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Search the subsegment with the maximum/minimum sum" (cp-algorithms.com/others/maximum_average_segment.html), which uses prefix sums to find the best subarray.
- Workbook Chapter 2 has traces of Listing 2.2 and Listing 2.5 to complete by hand before you start the problems.

Problems

These are the problems for Day 2. Work them in Workbook Chapter 2, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
2.P1	LC 560 · Subarray Sum Equals K	MEDIUM	CORE
2.P2	LC 974 · Subarray Sums Divisible by K	MEDIUM	CORE
2.P3	LC 1524 · Number of Sub-arrays With Odd Sum	MEDIUM	CORE
2.P4	LC 525 · Contiguous Array	MEDIUM	CORE
2.P5	LC 1109 · Corporate Flight Bookings	MEDIUM	BONUS
2.P6	LC 304 · Range Sum Query 2D - Immutable	MEDIUM	BONUS
2.P7	CF 466C · Number of Ways	CF 1700	BONUS

CHAPTER 3 · DAY 3 OF 29 · MONDAY 9 NOVEMBER 2026 · LEARNING DAY

3

Two Pointers and Sliding Windows

Grow the window on the right; shrink it on the left while it is invalid.

Part I · Foundations
Handbook cards · Two pointers on a sorted array, Sliding window (variable size), Window maximum (monotonic deque)
Before this chapter · Chapter 1 (HashMap, ArrayDeque, big-O) and Chapter 2 (prefix sums)
Practice · Workbook Chapter 3 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- find and count pairs with a target sum in a sorted array with two pointers, and prove that no valid pair is skipped;
- solve 3Sum in $O(n^2)$ with sorting, two pointers and duplicate skipping;
- state the monotonicity condition under which a variable-size sliding window is correct, and use the grow-and-shrink template in O(n);
- count windows with exactly K of something as atMost(K) − atMost(K − 1), and find the shortest valid window with need/have counters;
- compute the maximum of every fixed-size window with a monotonic deque in O(n), and explain the amortised bound;
- decide between two pointers, a sliding window and prefix sums, in particular when negative numbers are present.

A brute force over pairs or over subarrays looks at $\Theta(n^2)$ candidates. For $n = 10^5$ that is $5 \cdot 10^9$, far too many. This chapter is about two ideas that look at only $O(n)$ of them and still never miss the answer.

The first idea works on a **sorted** array. Two indices start at the two ends and move towards each other; at every step one comparison proves that one of the two elements cannot be part of any answer, so it is discarded for good. The second idea works on **contiguous** pieces of an array or string. A window $[l, r]$ slides from left to right; the right end always advances, and the left end advances only when the window breaks a rule. Both pointers move at most n times, so the whole scan is linear.

Neither idea is correct for every problem, and the important part of the chapter is the condition that makes each one correct. For two pointers it is sortedness. For windows it is *monotonicity*: shrinking a valid window must keep it valid. Each section states the condition, proves the algorithm correct under it, and shows the counterexample when it fails. The chapter ends with the monotonic deque, which answers "maximum of every window of size k" in $O(n)$, and with a guide to choosing between this chapter's tools and the prefix sums of Chapter 2.

3.1 Two pointers on a sorted array

Is there a pair with a given sum?

Given a sorted array $a[0..n-1]$ and a target t, decide whether some $i < j$ has $a[i] + a[j] = t$. The **two pointers** puts l at the first index and r at the last. If $a[l] + a[r]$ is too small, the only way to increase the sum is to move l right; if it is too large, move r left.

Listing 3.1 Two pointers on a sorted array: find a pair, count pairs

```java
// Sorted array, pointers at both ends: does any pair add up to target?
static boolean hasPairWithSum(int[] sorted, long target) {
    int l = 0, r = sorted.length - 1;
    while (l < r) {
        long s = (long) sorted[l] + sorted[r];
        if (s == target) return true;
        if (s < target) l++;      // need a bigger sum
        else r--;                 // need a smaller sum
    }
    return false;
}

// Count pairs i < j with sorted[i] + sorted[j] <= target
static long countPairsAtMost(int[] sorted, long target) {
    long count = 0;
    int l = 0, r = sorted.length - 1;
    while (l < r) {
        if ((long) sorted[l] + sorted[r] <= target) {
            count += r - l;       // sorted[l] pairs with every index in (l, r]
            l++;
        } else {
            r--;
        }
    }
    return count;
}
```

The argument for correctness is an *elimination* argument: each move throws away one index, and we must show that the index thrown away is not part of any answer.

THEOREM 3.1 **Two pointers discard no valid pair**

In `hasPairWithSum`, the following holds at the start of every iteration: every pair $i < j$ with $a[i] + a[j] = t$ satisfies $l \le i$ and $j \le r$. Consequently the method returns `true` exactly when such a pair exists.

Proof. Initially $l = 0$ and $r = n - 1$, so every pair satisfies the bounds. Suppose the invariant holds and $s = a[l] + a[r] < t$. Any valid pair that uses index l has the form (l, j) with $j \le r$ by the invariant, and since the array is sorted, $a[l] + a[j] \le a[l] + a[r] < t$. So no valid pair uses l, and every valid pair has $i \ge l + 1$: the invariant survives `l++`. The case $s > t$ is symmetric: any valid pair (i, r) has $a[i] + a[r] \ge a[l] + a[r] > t$, so `r--` discards nothing. If the method returns `true`, it has found a valid pair. If the loop ends with $l \ge r$, the invariant says every valid pair has $l \le i < j \le r \le l$, which is impossible, so none exists. ∎

Each iteration moves one pointer one step towards the other, so there are at most $n - 1$ iterations: $O(n)$ time and $O(1)$ memory, after an $O(n \log n)$ sort if the input is not sorted already.

EXAMPLE 3.1 **Finding a pair**

Take $a = [1, 3, 4, 6, 8, 11]$ and $t = 14$.

```
l  r  a[l]  a[r]  sum   action
0  5    1    11    12   too small: l++   (1 + anything <= 1 + 11 = 12)
1  5    3    11    14   found
```

The first step discards index 0: the largest partner available to 1 is 11, and even that falls short.

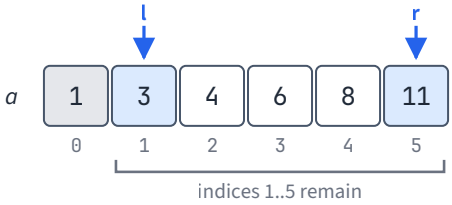


Figure 3.1 The second step of Example 3.1. Index 0 (grey) has been discarded; a[l] + a[r] = 3 + 11 = 14.

Counting pairs with sum at most X

`countPairsAtMost` uses the same picture to count instead of search. If $a[l] + a[r] \le X$, then $a[l] + a[j] \le X$ for **every** j in $(l, r]$, because those $a[j]$ are no larger than $a[r]$. So $a[l]$ forms $r - l$ good pairs at once, and index l has nothing more to contribute. If the sum is greater than X, then $a[i] + a[r] > X$ for every i in $[l, r)$, so index r forms no good pair with anything left.

CLAIM 3.2 **Counting pairs**

At the start of every iteration of `countPairsAtMost`, `count` *equals the number of good pairs* (i, j) *with* $i < l$, *and no good pair has* $l \le i < j$ *with* $j > r$. *At the end,* `count` *is the number of good pairs.*

Proof. Both statements hold initially. In the first branch, the good pairs with $i = l$ are exactly (l, j) for $l < j \le r$ (pairs with $j > r$ are excluded by the second statement), which is $r - l$ pairs; adding them and moving l keeps the first statement. In the second branch, no good pair uses r together with an index in $[l, r)$, so the second statement survives `r--`. When $l \ge r$, every pair with $i \ge l$ has $j > i \ge l \ge r$, so it is not good, and all good pairs have been counted. ∎

For $a = [1, 3, 4, 6, 8, 11]$ and $X = 9$ the scan adds 4 (index 0 with indices 1..4) and then 2 (index 1 with indices 2..3), for a total of 6, matching a brute force over all 15 pairs.

PITFALL

Sorting changes indices. LeetCode 1 (Two Sum) asks for original indices, so either use the hash map of Chapter 1 or sort an array of indices by value. And cast before adding: `(long) a[l] + a[r]`, since two values near $2 \cdot 10^9$ overflow `int`.

3.2 3Sum: sorting plus two pointers

LeetCode 15 (3Sum) asks for all *distinct* triples of values from the array that add up to 0. Checking every triple is $\Theta(n^3)$. Fixing the smallest element of the triple turns the rest into a pair problem: after sorting, for each i, find all pairs in $a[i + 1..n - 1]$ with sum $-a[i]$. Each pair search is the scan of Section 3.1, so the total is $O(n^2)$.

The difficulty is **duplicates**. The array $[-1, -1, 0, 1, 1]$ contains the triple of values $(-1, 0, 1)$ four times over as index triples, but it must be reported once. Sorting puts equal values next to each other, which makes duplicates cheap to skip.

Distinct pairs with a given sum

Listing 3.2 is the inner loop: it reports every pair of *values* (x, y) with $x \le y$ and $x + y = t$ inside $a[from..n - 1]$, each exactly once.

Listing 3.2 All distinct value pairs with a given sum in a sorted range

```
static List<int[]> distinctPairs(int[] a, int from, long target) {   // a sorted; search a[from..]
    List<int[]> out = new ArrayList<>();
    int l = from, r = a.length - 1;
    while (l < r) {
        long s = (long) a[l] + a[r];
        if (s < target) l++;
        else if (s > target) r--;
        else {
            out.add(new int[]{a[l], a[r]});
            l++;
            r--;
            while (l < r && a[l] == a[l - 1]) l++;   // skip copies of the value just used
            while (l < r && a[r] == a[r + 1]) r--;
        }
    }
    return out;
}
```

CLAIM 3.3 **Each value pair is reported once**

Listing 3.2 reports every value pair (x, y) with $x \le y$, $x + y = t$, that occurs at two different positions of $a[from..n-1]$, and reports each such pair exactly once.

Proof. The two non-hit branches discard an index exactly as in Theorem 3.1, so no pair that is still unreported is lost there. At a hit with $x = a[l]$ and $y = a[r]$, every other pair whose smaller value is x must have larger value $t - x = y$, and every pair whose larger value is y has smaller value x: both are the pair just reported. So moving l past every copy of x and r past every copy of y loses no new pair. Because l has moved past all copies of x, every later report has a strictly larger first value, so no value pair is reported twice. ∎

Assembling 3Sum

The full method is now three steps, which you will write yourself for LeetCode 15:

1. Sort the array.
2. For each i, skip it if $a[i] = a[i-1]$: every triple starting with that value was already found from the first copy, because that copy saw a superset of the remaining elements.
3. Otherwise collect the distinct pairs with sum $-a[i]$ in $a[i+1..n-1]$ and report each together with $a[i]$.

A triple of values $x \le y \le z$ is reported only from the first index i holding x (step 2), and only once there (Claim 3.3), so every distinct triple appears exactly once. For [−2, 0, 1, 1, 2, −1, −4, 2], sorted to [−4, −2, −1, 0, 1, 1, 2, 2], the method finds the four triples (−4, 2, 2), (−2, 0, 2), (−2, 1, 1) and (−1, 0, 1). An assembled version of these steps matched a brute force over all triples on 20,000 random arrays of up to 9 values in [−5, 5].

The outer loop runs n times and each inner scan is $O(n)$, so the time is $O(n^2)$: about $n^2/2 = 4.5 \cdot 10^6$ pointer moves for $n = 3\,000$. Apart from the output and the sort, the memory is $O(1)$.

Why not hashing?

A hash set can also find pairs: for each i, scan $j > i$ and ask whether $-a[i] - a[j]$ was seen between them. That is $O(n^2)$ as well, but it is slower and messier. Every step is a hash operation instead of an array comparison (in Chapter 1's measurements, about five times slower per operation), and removing duplicate triples needs a set of sorted triples, with extra memory and boxing. Sorting makes duplicates adjacent, so a single comparison skips them.

EXAM TIP

"Pairs or triples with a sum" plus "the order of the input does not matter" is the signal to sort first. The sort costs $O(n \log n)$, which never dominates an $O(n^2)$ search.

3.3 Variable-size sliding windows

The second family asks about **contiguous** pieces: the longest substring without a repeated character, the longest run of 1s if you may flip k zeros, the shortest stretch whose sum reaches a target. A brute force tries every window $[l, r]$: $\Theta(n^2)$ windows, and often more work per window.

The template

DEFINITION 3.1 Sliding window

A **sliding window** is a contiguous piece $a[l..r]$ of an array or string, described by its two ends, together with a *state* that summarises its contents (a sum, a count of zeros, a table of counts). The window *slides* when r or l advances by one; the state is updated for the one element that enters or leaves, instead of being recomputed.

Listing 3.3 Sliding window template: longest subarray with at most k distinct values

```
// Longest subarray with at most k distinct values.
// Template: grow r, shrink l while the window is invalid, then record the answer.
static int longestAtMostKDistinct(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        cnt.merge(a[r], 1, Integer::sum);                    // 1. add a[r]
        while (cnt.size() > k) {                             // 2. shrink while invalid
            if (cnt.merge(a[l], -1, Integer::sum) == 0) cnt.remove(a[l]);
            l++;
        }
        best = Math.max(best, r - l + 1);                    // 3. [l, r] is valid
    }
    return best;
}
```

The three numbered steps are the whole pattern. The *state* of the window (here the map of counts) must always describe exactly the elements of $a[l..r]$: step 1 adds $a[r]$, and every `l++` in step 2 first removes $a[l]$.

PITFALL

Bringing a count down to 0 does not remove the key from a `HashMap`. Remove it yourself, as the template does, or `cnt.size()` counts values that are no longer in the window.

When is it correct?

The template never tries most windows. It is correct only because of a property of the rule.

DEFINITION 3.2 Monotone window condition

A rule that decides whether a window $a[l..r]$ is *valid* is **monotone** if every contiguous sub-window of a valid window is valid, and the empty window is valid.

"At most k distinct values", "no repeated character", "at most k zeros" and "sum $\le X$ when all values are positive" are monotone: removing elements from either end cannot create a second distinct value, a repeat, a zero or a larger positive sum. "Sum exactly k" is not, and neither is "sum $\le X$" when negative values are allowed.

For each right end r, let $L(r)$ be the smallest $l \le r + 1$ for which $[l, r]$ is valid. (The empty window $[r + 1, r]$ is valid, so $L(r)$ exists.)

LEMMA 3.4 The left boundary only moves right

If the window condition is monotone, then [l, r] is valid exactly when $L(r) \le l \le r + 1$, and $L(r) \le L(r + 1)$ for every r.

Proof. If $L(r) \le l$, then $[l, r]$ is a sub-window of the valid window $[L(r), r]$, so it is valid; if $l < L(r)$, it is invalid by the minimality of $L(r)$. For the second part, $[L(r + 1), r + 1]$ is valid, so its sub-window $[L(r + 1), r]$ is valid, and by minimality $L(r) \le L(r + 1)$. ∎

THEOREM 3.5 Correctness of the template

If the window condition is monotone, then after step 2 of iteration r the template has $l = L(r)$. Hence `r - l + 1` is the length of the longest valid window ending at r, and `best` is the length of the longest valid window overall.

Proof. By induction on r, with $L(-1) = 0$. Before iteration r, $l = L(r - 1) \le L(r)$ by Lemma 3.4. Step 2 increments l while $[l, r]$ is invalid, which by the lemma means while $l < L(r)$, and stops as soon as $l = L(r)$, because that window is valid. The longest valid window ending at r starts at $L(r)$, and every valid window ends at some r, so the maximum over r is the overall answer. ∎

THEOREM 3.6 The template runs in O(n) steps

Over the whole run, step 2 executes at most n times in total, so the template performs O(n) additions and removals.

Proof. Each execution of the loop body in step 2 increases l by one, l never decreases, and $l \le r + 1 \le n$ throughout. So the body runs at most n times over the whole run, not n times per iteration of r. Each index enters the window once (step 1) and leaves at most once (step 2). ∎

This is an **amortised** bound: a single iteration of r may shrink the window by many steps, but the total over all iterations is at most n. With a `HashMap` state each step costs O(1) expected time; with an `int[]` of counts it costs O(1) worst case.

Example: the longest substring without repeating characters

LeetCode 3 (Longest Substring Without Repeating Characters) is the standard first window problem. The rule "no character occurs twice" is monotone. The state is a count per character; LeetCode 3's strings are ASCII, so an `int[128]` replaces the map. When $a[r]$ makes a count 2, only that character can be repeated, so step 2 shrinks until its count is 1 again.

Listing 3.4 Longest substring without repeating characters (LeetCode 3)

```
static int lengthOfLongestSubstring(String s) {
    int[] cnt = new int[128];                    // LeetCode 3 uses ASCII characters only
    int best = 0;
    for (int l = 0, r = 0; r < s.length(); r++) {
```

```
        char c = s.charAt(r);
        cnt[c]++;
        while (cnt[c] > 1) cnt[s.charAt(l++)]--;  // shrink until c is unique again
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

EXAMPLE 3.2 **Tracing on “abcbdab”**

```
r  s[r]  l before -> after   window   best
0   a         0 -> 0          a         1
1   b         0 -> 0          ab        2
2   c         0 -> 0          abc       3
3   b         0 -> 2          cb        3    (drop a, b)
4   d         2 -> 2          cbd       3
5   a         2 -> 2          cbda      4
6   b         2 -> 4          dab       4    (drop c, b)
```

The answer is 4 (“cbda”).

A common variant stores the last index where each character was seen and jumps l straight to one past it: `l = Math.max(l, last[c] + 1)`. It is the same algorithm with step 2 done in one jump; the `Math.max` stops l from moving left when the previous copy is already outside the window.

Example: at most k zeros

LeetCode 1004 (Max Consecutive Ones III) gives a 0/1 array and asks for the longest run of 1s if you may flip at most k zeros. Flipping zeros inside a window turns it into all 1s, so the question is “the longest window with at most k zeros”, a monotone rule. The state is one integer.

Listing 3.5 Longest window with at most k zeros (LeetCode 1004)

```
static int longestOnes(int[] a, int k) {
    int zeros = 0, best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        if (a[r] == 0) zeros++;
        while (zeros > k) if (a[l++] == 0) zeros--;   // shrink until at most k zeros
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

EXAMPLE 3.3 **Tracing with k = 2**

Take a = [1, 0, 1, 1, 0, 0, 1, 1, 0, 1].

```
r  a[r]  zeros  l before -> after  length  best
0   1      0        0 -> 0           1       1
1   0      1        0 -> 0           2       2
2   1      1        0 -> 0           3       3
3   1      1        0 -> 0           4       4
4   0      2        0 -> 0           5       5
5   0      2        0 -> 2           4       5    (third zero: drop indices 0, 1)
6   1      2        2 -> 2           5       5
7   1      2        2 -> 2           6       6
8   0      2        2 -> 5           4       6    (drop indices 2, 3, 4)
9   1      2        5 -> 5           5       6
```

The answer is 6, the window a[2..7].

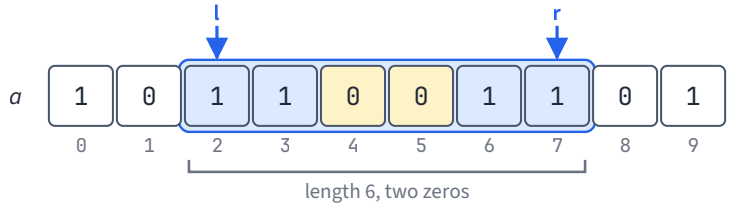


Figure 3.2 The best window of Example 3.3, a[2..7], contains exactly k = 2 zeros (amber). Flipping them gives six consecutive 1s.

INFOSYS LENS

The Infosys "Andy's vacation" problem (cancel at most K obligations, find the longest run of free days) is LeetCode 1004 on a calendar: a day with obligations plays the role of a zero. The trap reported with it is that several obligations can fall on one day, and all of them must be cancelled to free that day, so a day costs its number of obligations, not 1. You will solve it in Chapter 24.

3.4 Longest repeating character replacement

LeetCode 424 (Longest Repeating Character Replacement) gives a string of uppercase letters and an integer k: you may replace at most k characters, and you want the longest substring of one repeated letter. This section derives the window condition and then proves a shortcut that most published solutions use without explanation.

The window condition

To make a window uniform, keep its most frequent letter and replace everything else. So a window of length w whose most frequent letter occurs f times needs exactly $w - f$ replacements:

$$[l, r] \text{ is valid} \iff (r - l + 1) - \text{maxCount}(l, r) \le k,$$

where maxCount(l, r) is the largest count of a single letter in the window. The rule is monotone: removing an end character lowers w by 1 and f by at most 1, so $w - f$ cannot grow. By Theorem 3.5 the template is correct if step 2 recomputes the maximum of the 26 counts each time. That costs 26 steps per check, and there are at most $2n$ checks (one that fails per r, one per `l++`), which is fast enough for any limit in this problem.

The shortcut: never decrease maxCount

The well-known O(n) version keeps a variable M that is the largest count **ever** seen, never decreases it, and replaces the `while` by an `if`: when $(r - l + 1) - M > k$, it removes one character on the left. So the window never shrinks: each step it either grows by one or slides one place to the right. The window may even be invalid for a while. The answer is the largest length reached.

EXAMPLE 3.4 The shortcut on "ABBACBBA" with k = 1

The table shows the state after each step. "true max" is the real maxCount of the window, for comparison.

```
r  ch   M   slide?  window  length  true max  best
0  A    1   no      A         1       1        1
1  B    1   no      AB        2       1        2
2  B    2   no      ABB       3       2        3
```

```
3  A    2   yes     BBA       3         2         3
4  C    2   yes     BAC       3         1         3   <- invalid window, length kept
5  B    2   yes     ACB       3         1         3
6  B    2   yes     CBB       3         2         3
7  A    2   yes     BBA       3         2         3
```

The answer is 3, which a brute force over all windows confirms. At $r = 4$ and 5 the window needs two replacements, more than k, yet nothing goes wrong: the algorithm keeps the length 3 that it has already proved achievable and only looks for something longer.

Why is this correct? The intuition is that the length can only grow when M grows, and a stale M only describes a window that really existed. Write w_r and M_r for the window length and M after step r.

LEMMA 3.7 Three facts about the shortcut

(a) $w_r - M_r \le k$ after every step. (b) M_r is at least the count of every letter in the window $[l, r]$ just before the possible slide at step r. (c) At a step t where M increases, there is no slide, and the window after step t contains M_t copies of $s[t]$.

Proof. (a) If there is no slide, the test $(r - l + 1) - M_r \le k$ is exactly the statement. If there is a slide, $w_r = w_{r-1}$ and $M_r \ge M_{r-1}$, so $w_r - M_r \le w_{r-1} - M_{r-1} \le k$ by induction. (b) Take a letter x in the window and let $t \le r$ be its last position there. When $s[t]$ was added, the window then in use started no later than the current one (the left end never moves left), so it contained at least as many copies of x as the current window, and M was raised to at least that count. M never decreases. (c) A count rises by one per step, so $M_t = M_{t-1} + 1$. Before the possible slide the length is $w_{t-1} + 1$, and $(w_{t-1} + 1) - M_t = w_{t-1} - M_{t-1} \le k$ by (a), so there is no slide. The count of $s[t]$ in that window is the new maximum M_t. ∎

THEOREM 3.8 The shortcut finds the optimum

Let A be the length of the longest substring that can be made uniform with at most k replacements. The window length after the last step equals A.

Proof. *No more than A.* Fix any step r, and let $t \le r$ be the last step at which M increased, so $M_r = M_t$. By (c) the window W after step t has M_r copies of some letter c, and its length $w_t \le w_r$ because lengths never decrease. Any substring of length w_r that contains W (one exists, since $w_r \le r + 1 \le n$) has at least M_r copies of c, so it needs at most $w_r - M_r \le k$ replacements by (a). Hence $A \ge w_r$. *At least A.* Let $[i, j]$ be an optimal window. Suppose that at some step $r \le j$ the window started at i and slid, so that $(r - i + 1) - M_r > k$. The window $[i, r]$ is a sub-window of $[i, j]$, so it is valid: $(r - i + 1) - \text{maxCount}(i, r) \le k$. By (b), $M_r \ge \text{maxCount}(i, r)$, which contradicts the slide. So the left end never passes i before step j, and $w_j \ge j - i + 1 = A$. Lengths never decrease, so the final length is at least A. ∎

Both versions, the exact one with 26 counts and the shortcut (with `if` or with `while`), matched a brute force on 20,000 random strings over {A, B, C}. Writing the code is left for the problem worksheet. In an exam, the exact version is the safer one to write first: its proof is Theorem 3.5.

3.5 Counting windows: exactly K

Some problems ask **how many** windows satisfy a rule. With a monotone rule, Lemma 3.4 says the valid windows ending at r are exactly $[l, r]$ for $L(r) \le l \le r$, so there are $r - L(r) + 1$ of them. Replace step 3 of the template by `count += r - l + 1` and the scan counts every valid window in $O(n)$.

"Exactly K" rules are not monotone: a window with exactly two odd numbers has sub-windows with one. The fix is to count two monotone rules and subtract.

THEOREM 3.9 Exactly = at most minus at most

For any non-negative quantity q attached to windows (the number of odd values, of zeros, of distinct values), the number of windows with $q = K$ equals atMost(K) − atMost($K - 1$), where atMost(m) is the number of windows with $q \le m$, and atMost(-1) = 0.

Proof. The windows with $q \le K$ split into two disjoint groups: those with $q \le K - 1$ and those with $q = K$. Counting both sides gives atMost(K) = atMost($K - 1$) + exactly(K). ∎

When q only grows as a window is extended (counts of odd values, zeros or distinct values all do), "$q \le m$" is monotone and each atMost is one window scan. Here is the method on a new example: count the subarrays that contain exactly k odd numbers.

Listing 3.6 Subarrays with exactly k odd numbers

```
static long atMostOdd(int[] a, int k) {
    if (k < 0) return 0;                          // no window has fewer than 0 odd values
    long count = 0;
    int odd = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        if ((a[r] & 1) == 1) odd++;
        while (odd > k) if ((a[l++] & 1) == 1) odd--;
        count += r - l + 1;                       // windows [l..r], [l+1..r], ..., [r..r]
    }
    return count;
}

static long exactlyKOdd(int[] a, int k) {
    return atMostOdd(a, k) - atMostOdd(a, k - 1);
}
```

The test `(a[r] & 1) == 1` is used instead of `a[r] % 2 == 1` because, as Chapter 1 showed, `-3 % 2` is −1 in Java; the bit test is correct for negative values.

EXAMPLE 3.5 Exactly two odd numbers in [2, 1, 1, 4, 1, 6]

The two scans, with the number of windows added at each r:

```
atMostOdd(a, 2)                          atMostOdd(a, 1)
r  a[r]  odd  l  add  total              r  a[r]  odd  l  add  total
0   2     0   0   1     1                0   2     0   0   1     1
1   1     1   0   2     3                1   1     1   0   2     3
2   1     2   0   3     6                2   1     1   2   1     4
3   4     2   0   4    10                3   4     1   2   2     6
4   1     2   2   3    13                4   1     1   3   2     8
5   6     2   2   4    17                5   6     1   3   3    11
```

So exactly(2) = 17 − 11 = 6. Running the method for every k gives exactly(0) = 3, exactly(1) = 8, exactly(2) = 6 and exactly(3) = 4. These add up to 21 = 6 · 7/2, the number of non-empty subarrays, as they must.

LeetCode 992 (Subarrays with K Different Integers) has the same shape with q = the number of distinct values, which also only grows as a window is extended.

3.6 The shortest valid window

"Shortest window that reaches a target" problems have the opposite monotonicity: making a valid window **longer** keeps it valid. A window whose positive values sum to at least T still does when extended; a window that contains every required value still does when extended. For such rules the template records the answer **inside** the loop: while the window is valid, record its length, then shrink.

Listing 3.7 Shortest subarray with sum at least target (positive values)

```java
// Shortest subarray with sum >= target (all values positive)
static int shortestSumAtLeast(int[] a, long target) {
    int best = Integer.MAX_VALUE;
    long sum = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum >= target) {                          // valid: record, then shrink
            best = Math.min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

LEMMA 3.10 **Correctness of the shrink-while-valid loop**

Suppose every window that contains a valid window is valid. For each r let $G(r)$ be the largest l such that $[l, r]$ is valid (or -1 if none). Then G is non-decreasing, and after iteration r of Listing 3.7, $l = G(r) + 1$ and the shortest valid window ending at r, of length $r - G(r) + 1$, has been recorded.

Proof. $[G(r), r]$ is valid, so the larger window $[G(r), r + 1]$ is valid and $G(r + 1) \geq G(r)$. Also $[l, r]$ is valid exactly when $l \leq G(r)$: windows with smaller l contain $[G(r), r]$. Assume that before iteration r, $l = G(r - 1) + 1 \leq G(r) + 1$. The loop records and advances while $[l, r]$ is valid, that is while $l \leq G(r)$, so its last recording is the window $[G(r), r]$ and it stops at $l = G(r) + 1$. (For $r = 0$ start from $l = 0 = G(-1) + 1$ with $G(-1) = -1$.) ∎

The same $O(n)$ amortised argument as Theorem 3.6 applies: l advances at most n times in total.

Need and have

When validity means "the window contains everything on a list", keep two numbers instead of comparing whole count tables. These **need/have counters** are *need*, how many requirements there are, and *have*, how many the window currently meets. A requirement becomes met when its count rises to the required level and stops being met when the count falls below it, so each add or remove updates *have* in $O(1)$. The window is valid exactly when *have* = *need*.

Here is the pattern on a new problem: the shortest subarray that contains every distinct value of the array at least once. The requirements are "value v occurs at least once", one per distinct value.

Listing 3.8 Shortest window containing every distinct value (need/have counters)

```java
static int shortestWithAllValues(int[] a) {
    Map<Integer, Integer> cnt = new HashMap<>();
    Set<Integer> values = new HashSet<>();
    for (int x : a) values.add(x);
    int need = values.size();                            // values the window must contain
    int have = 0, best = Integer.MAX_VALUE;
    for (int l = 0, r = 0; r < a.length; r++) {
        if (cnt.merge(a[r], 1, Integer::sum) == 1) have++;      // a value enters the window
        while (have == need) {                           // valid: record, then shrink
            best = Math.min(best, r - l + 1);
            if (cnt.merge(a[l], -1, Integer::sum) == 0) have--; // a value leaves completely
```

```
            l++;
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

EXAMPLE 3.6 Tracing on [2, 1, 2, 2, 3, 1, 3, 2]

There are three distinct values, so *need* = 3. The table shows, for each r, the windows recorded inside the loop and the state when the loop stops.

```
r  a[r]  windows recorded (length)          have after  l after  best
0   2    -                                      1          0       -
1   1    -                                      2          0       -
2   2    -                                      2          0       -
3   2    -                                      2          0       -
4   3    [2,1,2,2,3] (5), [1,2,2,3] (4)         2          2       4
5   1    [2,2,3,1] (4), [2,3,1] (3)             2          4       3
6   3    -                                      2          4       3
7   2    [3,1,3,2] (4), [1,3,2] (3)             2          6       3
```

The answer is 3, for example [2, 3, 1] at indices 3..5.

For a requirement list with multiplicities, such as "at least two copies of x and one of y", store the required count per item and increase *have* when an item's count reaches its requirement exactly. That is the key observation behind LeetCode 76 (Minimum Window Substring).

3.7 Fixed-size windows and the monotonic deque

Now the window has a fixed size k, and the question is the maximum of every window: for $a[0..n-1]$, output $\max(a[i..i+k-1])$ for $i = 0..n-k$. Sums of fixed windows are easy (add the new element, subtract the old one), but a maximum cannot be "subtracted": when the current maximum leaves the window, which element takes over? Rescanning the window costs $O(k)$ per step and $O(nk)$ in total; a `TreeMap` of counts gives $O(n \log k)$. A deque does it in $O(n)$.

The idea: dominated elements never matter

Suppose $j < i$ and $a[j] \le a[i]$. Every window that contains j and is still to come also contains i, because i is later and windows move right. So j can never again be the unique maximum: i is at least as large and leaves the window later. Such a j can be deleted the moment i arrives. What remains after the deletions is a list of indices whose values strictly decrease from oldest to newest.

DEFINITION 3.3 Monotonic deque

A **monotonic deque** for window maxima is a double-ended queue of indices $j_1 < j_2 < \cdots < j_m$ from the current window whose values strictly decrease: $a[j_1] > a[j_2] > \cdots > a[j_m]$. New indices enter at the back; indices leave at the back when dominated and at the front when they fall out of the window.

Listing 3.9 Maximum of every window of size k with a monotonic deque

```
// Maximum of every window of size k, O(n). The deque holds indices; their values decrease.
static int[] windowMax(int[] a, int k) {
    int n = a.length;
    int[] res = new int[n - k + 1];
    ArrayDeque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();   // they can never win now
```

```
        dq.addLast(i);
        if (dq.peekFirst() <= i - k) dq.pollFirst();                    // front left the window
        if (i >= k - 1) res[i - k + 1] = a[dq.peekFirst()];
    }
    return res;
}
```

THEOREM 3.11 The front of the deque is the window maximum

After iteration i of Listing 3.9: (1) the deque holds indices of the window $(i-k, i]$ in increasing order with strictly decreasing values, and its last index is i; (2) every index of the window that is not in the deque has a later index of the window with a value at least as large. Consequently a[front] is the maximum of the window.

Proof. By induction on i. Suppose (1) and (2) hold after iteration $i-1$ for the window $(i-1-k, i-1]$. The `while` loop removes from the back every index j with $a[j] \le a[i]$; each such j is dominated by the later index i, which is in the new window, so (2) holds for it. The back elements that stay have values greater than $a[i]$, so appending i keeps the values strictly decreasing. All deque indices were $> i-1-k$, so the only one that can now lie outside the new window is $i-k$, and if present it is the smallest index, at the front; the `if` removes it. Indices outside the window do not matter for (2): an index j of the new window that was dominated by a later j' is still dominated, since $j' > j$ is also in the new window. This proves (1) and (2). For the consequence, start from any window index and follow "a later index with a value at least as large" while the current index is not in the deque. Indices increase, so this stops at a deque index with a value at least the starting value. So the window maximum is attained in the deque, and the deque's largest value is at its front. ∎

THEOREM 3.12 The deque method is linear

Listing 3.9 performs at most $2n$ deque insertions and removals in total, so it runs in $O(n)$ time and uses $O(k)$ memory.

Proof. Each index is added to the deque exactly once. Each index is removed at most once, either from the back or from the front, because a removed index is never added again. So there are at most n additions and n removals, and each `while` test that fails ends the loop for that iteration, adding at most n more comparisons. The deque never holds more than the k indices of one window. ∎

EXAMPLE 3.7 Tracing with k = 3

Take $a = [4, 2, 12, 3, 8, 7, 1, 5]$. The deque is shown as index:value pairs from front to back.

```
i  a[i]  removed at back  removed at front  deque after          max
0    4        -                 -           0:4
1    2        -                 -           0:4 1:2
2   12     2, 4                -           2:12                  12
3    3        -                 -           2:12 3:3              12
4    8        3                 -           2:12 4:8              12
5    7        -                12           4:8 5:7                8
6    1        -                 -           4:8 5:7 6:1            8
7    5        1                 8           5:7 7:5                7
```

The output is [12, 12, 12, 8, 8, 7]. The run made 14 deque operations, within the bound $2n = 16$.

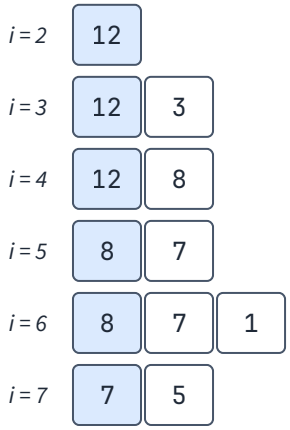


Figure 3.3 The deque values after each step of Example 3.7, front on the left. Values always decrease from front to back; the front (blue) is the maximum of the current window.

PITFALL

Store indices, not values: the front check `dq.peekFirst() <= i - k` needs to know where the front element sits. For window minima, flip the comparison and remove from the back while `a[dq.peekLast()] >= a[i]`.

The same deque appears in dynamic programming when $dp[i]$ needs the best $dp[j]$ over the last k positions j: the deque holds candidate indices j, and each $dp[i]$ is computed in O(1) amortised instead of O(k).

3.8 Choosing between windows, two pointers and prefix sums

The three techniques of Chapters 2 and 3 overlap, and the wrong choice produces code that passes the samples and fails hidden tests. The deciding questions are about structure, not about the story in the statement.

Table 3.1 Which technique for which question

The question	Condition you need	Technique	Cost
pair or triple with a target sum, order irrelevant	none (sort first)	two pointers from both ends	O($n \log n$) + O(n) per pair search
longest window with a rule	rule is monotone (sub-windows of valid windows are valid)	grow-and-shrink window	O(n)
shortest window with a rule	extending a valid window keeps it valid	shrink-while-valid window	O(n)
number of windows with $q = K$	q never decreases when a window is extended	atMost(K) − atMost($K-1$)	O(n)
max or min of every window of size k	fixed size	monotonic deque	O(n)
windows with sum exactly k, any signs	none	prefix sums + hash map (Chapter 2)	O(n) expected

Negative numbers break windows

Sum-based window rules are monotone only when every value is positive (or non-negative). With negatives, removing an element can **raise** the sum, and the template silently gives wrong answers. Two small inputs, both checked by running the code:

- *Longest subarray with sum ≤ 0* on [5, −10, 5]. The whole array has sum 0, so the answer is 3. The window template removes the 5 at index 0 as soon as it sees it (sum 5 > 0) and ends with 2.
- *Shortest subarray with sum ≥ 1* on [−3, 3]. The answer is 1, the subarray [3]. Listing 3.7 never sees a valid window, because the sum of [−3, 3] is 0 and it never drops the −3, so it returns 0 ("none").

When negatives are allowed, go back to the prefix sums of Chapter 2. Sums of subarrays become differences $P[r+1] - P[l]$, and questions about them become questions about pairs of prefix values, which a hash map (exact sums), sorting or a monotonic deque over the prefix array (shortest sum ≥ T) can answer.

> **KEY IDEA**
>
> Before you write a window, test the rule: "if $[l, r]$ is valid, is $[l+1, r]$ valid?". If the answer depends on the signs of the values, the window is wrong for some input.

Summary

- On a sorted array, two pointers from both ends find or count pairs in $O(n)$: each comparison proves that one end index belongs to no answer (Theorem 3.1).
- 3Sum is "fix the smallest, then two pointers", $O(n^2)$. Sorting makes duplicate values adjacent, so skipping equal neighbours reports each triple once.
- A variable window is correct when its rule is monotone: sub-windows of valid windows are valid. Then the left end $L(r)$ never moves left, and grow-and-shrink finds the longest valid window ending at every r.
- Both pointers move at most n times, so a window scan costs $O(n)$ amortised, whatever happens in one iteration.
- For character replacement the rule is length − maxCount ≤ k. Recomputing maxCount from 26 counts is correct and simple; keeping a never-decreasing maxCount and sliding the window is also correct, by Theorem 3.8.
- Count valid windows by adding $r - l + 1$ per step; count "exactly K" as atMost(K) − atMost($K-1$).
- For "shortest valid window" rules that survive extension, record inside the loop and shrink while valid; need/have counters make the validity test $O(1)$.
- The monotonic deque keeps only undominated indices, so its front is the window maximum and each index enters and leaves once: $O(n)$.
- With negative values, sum-based windows fail; use prefix sums.

Key terms

sliding window p. 47
monotone window condition p. 47
monotonic deque p. 54
two-pointer technique p. 43
amortised analysis p. 48
need/have counters p. 53

Review questions

1. In Theorem 3.1, why is it safe to discard index l when $a[l] + a[r] < t$, even though $a[l]$ has not been tried with any index other than r?
2. `countPairsAtMost` adds $r - l$ in one step. Which property of the array justifies counting all those pairs without looking at them?
3. In 3Sum, what goes wrong if the outer loop does not skip i when $a[i] = a[i-1]$? Give an input and the duplicated output.
4. Which of these window rules are monotone? (a) at most 3 distinct values; (b) at least 3 distinct values; (c) sum ≤ 10 with values in [1, 100]; (d) sum ≤ 10 with values in [−5, 5]; (e) every character occurs an even number of times.
5. Explain in one paragraph why the window template is $O(n)$ even though the inner `while` loop can run n times in a single iteration.
6. In the replacement shortcut of Section 3.4, the window can be invalid. Why does that not make the final answer too large?
7. Use Theorem 3.9 to compute the number of subarrays of [1, 1, 1] with exactly one odd value, showing atMost(1) and atMost(0).
8. In Listing 3.9 the back is popped while `a[back] <= a[i]`. What changes if it pops only while `a[back] < a[i]`? Is the output still correct, and is the running time still $O(n)$?
9. Give an input on which the window template for "longest subarray with sum ≤ X" fails because of a negative value, and name the method you would use instead.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 8 "Amortized analysis": the two pointers method (subarray sum, 2SUM) and the sliding window minimum with a deque. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Minimum Stack / Minimum Queue" (cp-algorithms.com/data_structures/stack_queue_modification.html), which builds a queue with O(1) minimum, an alternative to the monotonic deque.
- Workbook Chapter 3 has traces of Listing 3.3 and Listing 3.9 to complete by hand, and a window rule to test for monotonicity.

Problems

These are the problems for Day 3. Work them in Workbook Chapter 3, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
3.P1	LC 3 · Longest Substring Without Repeating Characters	MEDIUM	CORE
3.P2	LC 1004 · Max Consecutive Ones III	MEDIUM	CORE
3.P3	LC 424 · Longest Repeating Character Replacement	MEDIUM	CORE
3.P4	LC 15 · 3Sum	MEDIUM	CORE
3.P5	LC 992 · Subarrays with K Different Integers	HARD	BONUS
3.P6	LC 239 · Sliding Window Maximum	HARD	BONUS
3.P7	LC 76 · Minimum Window Substring	HARD	BONUS

CHAPTER 4 · DAY 4 OF 29 · TUESDAY 10 NOVEMBER 2026 · LEARNING DAY

4

Sorting and Greedy Choice: Intervals

Sort by the right key, then make one decision per element. Check the greedy by swapping two choices.

Part I · Foundations
Handbook cards · Intervals: sort, then sweep
Before this chapter · Chapter 1 (comparators, PriorityQueue) and Chapter 3 (two pointers)
Practice · Workbook Chapter 4 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- explain which algorithm Java's sorts use for primitives and for objects, what stability means, and how to write a safe comparator;
- prove a greedy algorithm correct with an exchange argument, and refute a wrong greedy with a small counterexample found by brute force;
- merge overlapping intervals and choose the most non-overlapping intervals by earliest end, with proofs;
- compute the fewest rooms or platforms as the maximum overlap, with a sweep over sorted events or a heap of end times, handling ties at equal times;
- prove that the Largest Number comparator is a valid order and that sorting by it is optimal;
- use reachability invariants and two-pass greedy algorithms for constraints that come from both sides.

A greedy algorithm builds its answer one decision at a time and never takes a decision back. It is the fastest kind of algorithm to write and the easiest to get wrong: many natural greedy rules are false, and they often pass the samples. This chapter is about telling the two apart.

Almost every greedy algorithm in an exam starts with a sort, because the sort decides the order in which decisions are made. So the chapter starts with sorting in Java: what `Arrays.sort` does, how to write comparators that cannot overflow or contradict themselves, and what can go wrong on adversarial inputs. The second section gives the tool for proving a greedy correct, the **exchange argument**, and the habit that catches false greedies before they cost marks: checking against a brute force on small inputs.

The rest of the chapter applies both to the problems that appear most often in practice: intervals. Merging them, keeping as many disjoint ones as possible, and packing all of them into the fewest rooms are three different questions with three different sort keys. The chapter ends with a custom order (the Largest Number problem, an Infosys repeat) and with two greedy invariants that are not about intervals at all.

4.1 Sorting in Java

Two algorithms behind one method name

`Arrays.sort` chooses its algorithm by the element type.

- **Primitive arrays** (`int[]`, `long[]`, `double[]`, `char[]`) are sorted with **dual-pivot quicksort**, a quicksort that splits the range around two pivots into three parts. Its average time is $O(n \log n)$ and it is very

fast in practice: sorting 10^6 random `int` values took between 11 and 91 ms over three runs on this book's machine.

- **Object arrays** (`Integer[]`, `String[]`, `int[][]`) and `List.sort` / `Collections.sort` use **TimSort**, a merge sort that first detects runs that are already sorted. It is $O(n \log n)$ in the worst case and $O(n)$ on input that is already sorted. Boxing and comparator calls make it slower: 10^6 `Integer` values took 410–570 ms, and 10^6 `int[]` pairs sorted by a lambda comparator 470–700 ms.

DEFINITION 4.1 Stable sort

A sort is **stable** if elements that compare equal keep their original relative order.

TimSort is stable, and Java guarantees it for every object sort. Quicksort is not a stable algorithm, but for primitives it cannot matter: two equal `int` values are indistinguishable. Stability matters when you sort records by one field and want ties to stay in input order. Sorting (Asha, 70), (Ben, 85), (Chen, 70), (Dev, 85), (Esi, 60) by score gives Esi, Asha, Chen, Ben, Dev: Asha stays before Chen and Ben before Dev.

The worst case of quicksort, and a precaution

Quicksort is fast because its pivots usually split the range into balanced parts. For every fixed pivot rule there are inputs on which the splits are badly unbalanced, and then quicksort takes $\Theta(n^2)$ time. The dual-pivot quicksort of Java 8 has no defence against such inputs, and contest problems have used crafted "anti-quicksort" tests to make `Arrays.sort(int[])` time out. Since JDK 14 the implementation switches to heap sort when the partitioning becomes too deep, which caps the worst case at $O(n \log n)$. You do not know which JDK an exam uses, so when the input could be adversarial, shuffle first: a random order makes the bad case astronomically unlikely, whatever the pivot rule.

Listing 4.1 Shuffle before sorting primitives (a precaution against crafted inputs)

```
static void shuffleThenSort(int[] a) {
    Random rnd = new Random();
    for (int i = a.length - 1; i > 0; i--) {                    // Fisher-Yates shuffle
        int j = rnd.nextInt(i + 1);
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
    Arrays.sort(a);
}
```

The shuffle costs $O(n)$ and is cheap insurance: shuffling and sorting 10^6 values took 35–74 ms. The alternative, sorting an `Integer[]` so that TimSort's guarantee applies, costs about five times as much.

Why O(n log n) is the best a comparison sort can do

Both algorithms compare elements and move them according to the results. No algorithm of that kind can beat $n \log n$ by more than a constant factor.

THEOREM 4.1 Lower bound for comparison sorting

Any algorithm that sorts n distinct values using only comparisons needs at least $\log_2(n!)$ comparisons on some input, and $\log_2(n!) \geq (n/2)\log_2(n/2)$.

Proof. Run the algorithm on the $n!$ orderings of n distinct values. Each comparison has two outcomes, so after c comparisons the algorithm can be in at most 2^c different states. Two orderings that lead to the same sequence of outcomes are rearranged in the same way, so at most one of them ends sorted. Hence $2^c \geq n!$,

that is $c \geq \log_2(n!)$. For the estimate, the product $n! = 1 \cdot 2 \cdots n$ contains at least $n/2$ factors that are each at least $n/2$, so $\log_2(n!)$ is at least $n/2$ times $\log_2(n/2)$. ∎

Sorting is therefore $\Theta(n \log n)$, and a greedy algorithm that sorts once and then makes one $O(1)$ or $O(\log n)$ decision per element costs $\Theta(n \log n)$ in total.

Comparators

A comparator `(x, y) -> ...` must return a negative number when x goes first, a positive number when y goes first and 0 for a tie. It must also be **consistent**: if it says x before y and y before z, it must say x before z. Two rules prevent almost every comparator bug.

1. **Never subtract.** `(x, y) -> x[0] - y[0]` overflows when the values differ by more than $2^{31} - 1$ (Chapter 1 showed $2 \cdot 10^9 - (-2 \cdot 10^9) = -294{,}967{,}296$) and then reports the wrong order. Use `Integer.compare` and `Long.compare`.
2. **Break ties explicitly** when the order of ties matters, with a second key.

```
int[][] iv = {{5, 9}, {1, 4}, {5, 6}};
Arrays.sort(iv, (x, y) -> Integer.compare(x[1], y[1]));             // by end: {1,4}, {5,6}, {5,9}
Arrays.sort(iv, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                                       : Integer.compare(y[1], x[1])); // start up, then end down
Integer[] idx = {0, 1, 2};
int[] key = {30, 10, 20};
Arrays.sort(idx, (p, q) -> Integer.compare(key[p], key[q]));         // indices in key order: 1, 2, 0
```

The last two lines show the standard way to sort while remembering original positions: sort an array of indices by the values they point to.

> **PITFALL**
>
> An inconsistent comparator can make TimSort throw `IllegalArgumentException: Comparison method violates its general contract!`, usually only on large inputs. The cause is a comparator that is not transitive or that returns different answers for the same pair, for example one that compares `double` values with subtraction and casts to `int`.

4.2 What makes a greedy algorithm correct

> **DEFINITION 4.2 Greedy algorithm**
>
> A **greedy algorithm** builds a solution by a sequence of choices, each made by a fixed local rule (“take the interval that ends first”, “take the largest coin”) and never revised.

A greedy is correct when two properties hold.

- **Greedy-choice property**: some optimal solution contains the greedy’s first choice.
- **Optimal substructure**: after the first choice, what remains is a smaller instance of the same problem, and an optimal solution of it, combined with the first choice, is optimal for the whole.

Given both, induction on the size of the instance proves the greedy optimal. The second property is usually obvious from the problem. The first is where the work is, and the standard tool for it is the exchange argument.

KEY IDEA

Exchange argument. Take any optimal solution O. If it does not agree with the greedy's choice, change O by exchanging one of its elements for the greedy's choice, and show that the result is still feasible and no worse. Then an optimal solution that agrees with the greedy exists.

The same idea works for orderings: if an optimal order has two neighbours that the greedy would put the other way round, show that swapping them never makes the answer worse. Repeated swaps then turn any optimal order into the greedy's order without losing anything (Section 4.6 uses exactly this).

Testing a greedy before trusting it

A greedy rule that sounds right is often wrong. For "choose as many pairwise disjoint intervals as possible", three plausible rules are: take the interval that starts first, take the shortest, take the one that ends first. Only the third is correct, and a brute force over all subsets finds counterexamples to the others in seconds.

Listing 4.2 Brute force for interval scheduling: try every subset (n <= 20)

```
static int bruteMaxNonOverlap(int[][] iv) {          // touching ends allowed, as in [1,2] and [2,3]
    int n = iv.length, best = 0;
    for (int m = 0; m < (1 << n); m++) {              // m = the subset, one bit per interval
        boolean ok = true;
        for (int i = 0; i < n && ok; i++)
            for (int j = i + 1; j < n && ok; j++)
                if ((m >> i & 1) == 1 && (m >> j & 1) == 1
                        && iv[i][0] < iv[j][1] && iv[j][0] < iv[i][1]) ok = false;   // overlap
        if (ok) best = Math.max(best, Integer.bitCount(m));
    }
    return best;
}
```

Comparing a rule with this brute force exposes it at once. Two of the smallest counterexamples, both checked by running the greedy and the brute force:

- **Earliest start** on [1, 10], [2, 3], [4, 5]: it takes [1, 10] and then nothing else fits, so 1. The optimum is 2.
- **Shortest first** on [1, 5], [4, 7], [6, 10]: it takes [4, 7] (length 3), which overlaps both others, so 1. The optimum is 2.

EXAM TIP

Before you submit a greedy, write the brute force (it takes five minutes) and compare the two on a few thousand random inputs of size 5–10. If they disagree, the brute force hands you a small counterexample. Chapter 9 turns this into a reusable stress tester. A brute force that is correct also scores the small tests on its own.

4.3 Merging intervals

LeetCode 56 (Merge Intervals) asks to merge all overlapping intervals and return the disjoint blocks that remain. Here intervals are closed and touching ones merge: [1, 3] and [3, 5] become [1, 5]. The key is to sort by **start**.

Listing 4.3 Merging overlapping intervals (LeetCode 56)

```java
// Merge overlapping intervals {start, end}
static int[][] merge(int[][] iv) {
    Arrays.sort(iv, (x, y) -> Integer.compare(x[0], y[0]));    // sort by start
    List<int[]> out = new ArrayList<>();
    for (int[] cur : iv) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last != null && cur[0] <= last[1]) last[1] = Math.max(last[1], cur[1]);
        else out.add(new int[]{cur[0], cur[1]});
    }
    return out.toArray(new int[0][]);
}
```

THEOREM 4.2 **Correctness of the merge**

After processing the first i intervals in order of start, `out` is the list of maximal merged blocks of those intervals, sorted by start, and the last block has the largest start.

Proof. By induction on i; for $i = 0$ the list is empty. Let the next interval be $c = [s, e]$. Every interval processed so far has start $\le s$. Every block except the last ends before the last block starts (they are disjoint and sorted), and the last block starts at a value $\le s$, so every earlier block ends before s: none of them can touch c. So c either overlaps the last block, when $s \le$ last end, and the two merge into one block whose end is the larger of the two ends; or it overlaps nothing processed so far and starts a new block, which has the largest start. Either way the list again consists of the maximal blocks in order. ∎

The sort costs $O(n \log n)$ and the loop $O(n)$, so the total is $O(n \log n)$ with $O(n)$ memory for the output.

EXAMPLE 4.1 **Tracing the merge**

Take [5, 7], [1, 3], [2, 4], [6, 8], [10, 12], [12, 13]. After sorting by start:

```
cur        action       out after
[1, 3]     new block    [1,3]
[2, 4]     extend       [1,4]
[5, 7]     new block    [1,4] [5,7]
[6, 8]     extend       [1,4] [5,8]
[10, 12]   new block    [1,4] [5,8] [10,12]
[12, 13]   extend       [1,4] [5,8] [10,13]      (touching ends merge)
```

If the statement says that touching intervals do **not** merge, the test becomes `cur[0] < last[1]`. That one character is the most common difference between interval problems, so read the statement for it every time.

4.4 Interval scheduling: keep the most

Given n intervals, choose as many as possible so that no two overlap. With half-open intervals $[s, e)$, or with closed ones where touching ends are allowed, [1, 2] and [2, 3] can both be chosen. The correct greedy is "always take the interval that **ends** first among those that still fit".

Listing 4.4 The most non-overlapping intervals: sort by end

```java
// Most intervals you can keep with no overlap (touching ends allowed): sort by END
static int maxNonOverlapping(int[][] iv) {
    Arrays.sort(iv, (x, y) -> Integer.compare(x[1], y[1]));
    int count = 0;
    long end = Long.MIN_VALUE;
```

```
    for (int[] cur : iv) {
        if (cur[0] >= end) {
            count++;
            end = cur[1];
        }
    }
    return count;
}
```

THEOREM 4.3 Earliest end is optimal

Listing 4.4 returns the largest number of pairwise non-overlapping intervals.

Proof. Let the greedy choose $g_1, g_2, \ldots, g_k$ in this order, and let $O = o_1, o_2, \ldots, o_m$ be any optimal solution sorted by end, so its intervals are disjoint and o_1 ends before o_2 starts, and so on. *Exchange step.* g_1 is an interval with the smallest end overall, so $\text{end}(g_1) \le \text{end}(o_1)$. Replace o_1 by g_1 in O. The result is still feasible: o_2 starts at or after $\text{end}(o_1) \ge \text{end}(g_1)$, so g_1 does not overlap o_2 or anything later. It still has m intervals, so it is optimal and agrees with the greedy's first choice. *Induction.* Remove g_1 and every interval that starts before $\text{end}(g_1)$. The remaining intervals are exactly those compatible with g_1, the greedy's later choices $g_2, \ldots, g_k$ are its run on them, and $o_2, \ldots, o_m$ is a feasible choice among them. By induction on the number of intervals, the greedy chooses at least $m - 1$ of them, so $k \ge m$. Since O is optimal, $k = m$. ∎

The sort is $O(n \log n)$, the scan $O(n)$. The variable `end` is a `long` starting at `Long.MIN_VALUE`, so the first interval is always accepted even if its start is `Integer.MIN_VALUE`.

EXAMPLE 4.2 Tracing the earliest-end greedy

Eleven intervals, already sorted by end:

```
interval   decision   last end   count
[1, 4]     take          4         1
[3, 5]     skip          4         1    (starts before 4)
[0, 6]     skip          4         1
[5, 7]     take          7         2
[3, 9]     skip          7         2
[5, 9]     skip          7         2
[6, 10]    skip          7         2
[8, 11]    take         11         3
[8, 12]    skip         11         3
[2, 14]    skip         11         3
[12, 16]   take         16         4
```

The greedy keeps 4 intervals, which the subset brute force of Listing 4.2 confirms is the maximum.

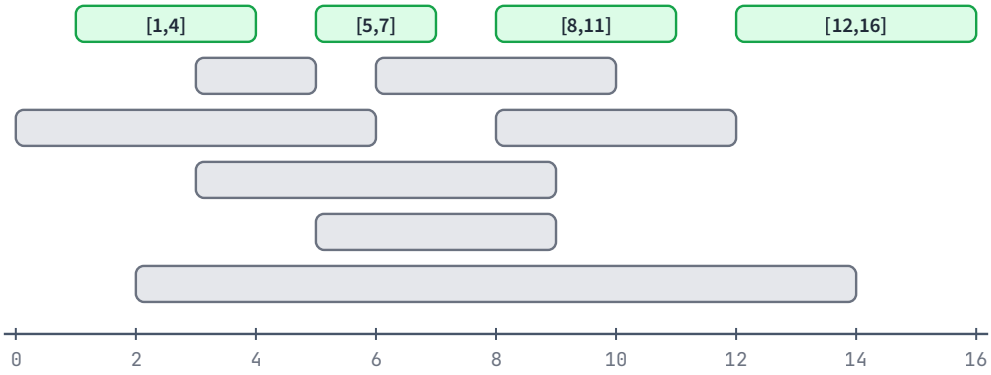


Figure 4.1 The intervals of Example 4.2. The greedy's choices (green) each end as early as possible, leaving the most room for the rest.

Removing the fewest, and the closed-endpoint variant

LeetCode 435 (Non-overlapping Intervals) asks for the fewest intervals to **remove** so that the rest do not overlap. Removing a set R leaves a non-overlapping set exactly when the kept set $n - |R|$ is non-overlapping, so the fewest removals is n minus the most you can keep. For the eleven intervals above that is $11 - 4 = 7$.

When touching endpoints count as overlapping (closed intervals that must not share even a point), the same greedy works with the test changed from `cur[0] >= end` to `cur[0] > end`. This is the key observation for LeetCode 452 (Minimum Number of Arrows to Burst Balloons): one arrow at the end of the earliest-ending balloon bursts every balloon that contains that point, and touching balloons share an arrow.

4.5 Partitioning into the fewest groups

Now every interval must be kept, and the question is how many groups (meeting rooms, railway platforms) are needed so that the intervals in each group do not overlap. LeetCode 2406 (Divide Intervals Into Minimum Number of Groups) is this question for closed intervals.

DEFINITION 4.3 Depth of a set of intervals

The **depth** of a set of intervals is the largest number of them that contain one common point.

THEOREM 4.4 Fewest groups equals depth

The fewest groups into which a set of intervals can be split, with no two overlapping intervals in the same group, equals its depth.

Proof. *Lower bound.* If d intervals contain a common point, every two of them overlap, so they need d different groups. *Upper bound.* Process the intervals in order of start, and put each into any group whose last interval does not overlap it, opening a new group only when every group's last interval overlaps it. Suppose interval $c = [s, e]$ opens group number g. Then each of the $g - 1$ existing groups has a last interval that overlaps c; since those intervals started at or before s, overlapping c means they end at or after s, so they all contain the point s. Together with c, that is g intervals containing s, so $g \le$ depth. Every group was opened this way, so the greedy uses at most depth groups. ∎

The proof gives two algorithms: compute the depth directly (only the number is needed), or run the greedy with a heap when the actual assignment is needed.

Sweeping sorted starts and ends

The depth is reached at some start point, because coverage only increases at starts. The handbook's template sorts the starts and the ends separately and walks through the starts, retiring every interval that ended before the current start.

Listing 4.5 Maximum overlap by sweeping sorted starts and ends

```java
// Most intervals covering one point = minimum platforms / meeting rooms.
// Closed intervals: a train that arrives at the minute another leaves needs its own platform.
static int maxOverlap(int[][] iv) {
    int n = iv.length;
    int[] starts = new int[n], ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = iv[i][0];
        ends[i] = iv[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);
    int best = 0, busy = 0;
    for (int i = 0, j = 0; i < n; i++) {
        while (j < n && ends[j] < starts[i]) {   // use <= if touching intervals can share
            j++;
            busy--;
        }
        busy++;
        best = Math.max(best, busy);
    }
    return best;
}
```

At the moment interval i (in start order) is counted, `busy` is $(i + 1)$ minus the number of ends strictly before s_i, which is the number of intervals with start $\le s_i$, among the first $i + 1$, that are still open at s_i. At the last of several equal starts this is exactly the coverage of the point s_i, so the maximum over i is the depth. Two sorts and one pass: $O(n \log n)$.

Events and tie-breaking

An equivalent and more flexible **sweep line** turns every interval into two **events**, $(s, +1)$ and $(e, -1)$, sorts them by time and adds them up. The only subtlety is two events at the same time. For closed intervals a start at time t overlaps an interval ending at t, so **starts must come before ends** at equal times; for half-open intervals (or "a platform freed at minute t can be used at minute t") ends come first.

Listing 4.6 Maximum overlap with events and an explicit tie-break

```java
static int maxOverlapEvents(int[][] iv) {                          // closed intervals [s, e]
    int n = iv.length;
    int[][] ev = new int[2 * n][];
    for (int i = 0; i < n; i++) {
        ev[2 * i] = new int[]{iv[i][0], +1};                         // start
        ev[2 * i + 1] = new int[]{iv[i][1], -1};                     // end
    }
    Arrays.sort(ev, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                                           : Integer.compare(y[1], x[1]));   // ties: starts first
    int busy = 0, best = 0;
    for (int[] e : ev) {
        busy += e[1];
        best = Math.max(best, busy);
    }
    return best;
}
```

A heap of end times

To produce the groups themselves, run the greedy of Theorem 4.4 with a min-heap holding the last end time of every group. For the next interval in start order, the group whose last interval ended first is the best candidate to reuse: if even that one overlaps, they all do.

Listing 4.7 Fewest groups with a min-heap of end times (closed intervals)

```
static int minGroups(int[][] iv) {
    int[][] s = iv.clone();
    Arrays.sort(s, (x, y) -> Integer.compare(x[0], y[0]));          // by start
    PriorityQueue<Integer> ends = new PriorityQueue<>();              // last end of each group
    for (int[] cur : s) {
        if (!ends.isEmpty() && ends.peek() < cur[0]) ends.poll();  // reuse the earliest-freed group
        ends.add(cur[1]);
    }
    return ends.size();
}
```

Each interval costs at most one `poll` and one `add` on a heap of at most depth elements, so the total is O(n log n). All three methods (sorted starts and ends, events, heap) were compared with a brute force over every assignment of up to 7 intervals to groups, and with direct counting at every integer point, on 20,000 random sets.

EXAMPLE 4.3 **Railway platforms**

Five trains with arrival and departure times (closed intervals: a train arriving at the minute another departs needs its own platform): [600, 640], [610, 700], [645, 720], [700, 730], [740, 800].

```
events (starts first at equal times)      heap of last end times, by arrival
t=600 arrive  busy 1                      [600,640]  new platform     {640}
t=610 arrive  busy 2                      [610,700]  new platform     {640, 700}
t=640 depart  busy 1                      [645,720]  reuse (640)      {700, 720}
t=645 arrive  busy 2                      [700,730]  new platform     {700, 720, 730}
t=700 arrive  busy 3   <- maximum         [740,800]  reuse (700)      {720, 730, 800}
t=700 depart  busy 2
t=720 depart  busy 1
t=730 depart  busy 0
t=740 arrive  busy 1
t=800 depart  busy 0
```

Three platforms are needed. If the statement allowed a platform freed at minute 700 to take the train arriving at minute 700, ends would sort first and the answer would be 2.

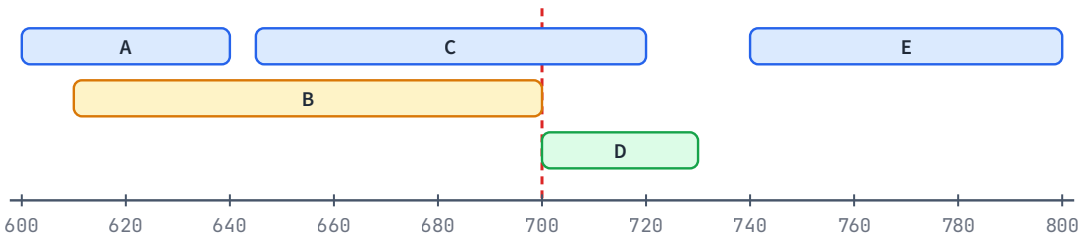


Figure 4.2 The trains of Example 4.3. At minute 700 (red line) three trains occupy platforms, so three platforms are needed for closed intervals.

INFOSYS LENS

Minimum railway platforms is one of the most frequently reported Infosys problems (eight reports in the plan's question bank). It is the maximum overlap, the same as LeetCode 2406. Read the statement for what happens when a train arrives at the exact minute another departs: that decides `<` or `<=` in Listing 4.5, and the tie-break in Listing 4.6.

4.6 Custom orders: the Largest Number

LeetCode 179 (Largest Number) gives non-negative integers and asks for the largest number that can be formed by concatenating all of them in some order, returned as a string. Sorting by numeric value fails: for 9 and 90 the larger value 90 should go second, because 990 > 909. Sorting the strings alphabetically in reverse fails too: "121" > "12" alphabetically, but 12121 > 12112, so "12" should go first.

The right comparison looks at the two possible concatenations: put x before y when $x + y > y + x$ as strings. Both concatenations have the same length, so comparing them as strings compares them as numbers.

```
String[] s = {"12", "121", "9", "90", "0"};
Arrays.sort(s, (x, y) -> (y + x).compareTo(x + y));   // x first when x + y is larger
// s is now 9, 90, 12, 121, 0, which concatenates to "990121210"
```

Is this a legitimate order? A comparator defined by "which concatenation is larger" is not obviously transitive, and TimSort may throw or return garbage if it is not. The following theorem settles it by giving each string a number that the comparator really compares.

THEOREM 4.5 The concatenation order is a valid order

For a non-empty digit string x with numeric value X, let $t(x)$ be 10 to the power of the length of x (so $t(\text{"121"}) = 1000$), and let $v(x) = X / (t(x) - 1)$. Then $x + y > y + x$ exactly when $v(x) > v(y)$, and $x + y = y + x$ exactly when $v(x) = v(y)$. Hence the comparator orders strings by decreasing v, which is transitive.

Proof. Appending y to x shifts x left by the length of y, so the numeric value of the concatenation $x + y$ is $X \cdot t(y) + Y$. So $x + y > y + x$ as numbers exactly when $X \cdot t(y) + Y > Y \cdot t(x) + X$, that is when $X(t(y) - 1) > Y(t(x) - 1)$. Both brackets are positive, so dividing by them gives $X / (t(x) - 1) > Y / (t(y) - 1)$. The same steps with $=$ in place of $>$ give the second statement. Comparing real numbers is transitive, so the comparator is. ∎

The number $v(x)$ is the value of $0.xxx\ldots$, the digits of x repeated for ever: $v(9) = 1$, $v(90) = 0.909\ldots$, $v(12) = 0.1212\ldots$, $v(121) = 0.121121\ldots$, $v(0) = 0$. The order 9, 90, 12, 121, 0 is exactly decreasing v.

THEOREM 4.6 Sorting by the concatenation order is optimal

Concatenating the strings in order of decreasing v gives the largest possible number.

Proof. Take any arrangement with two neighbours x, y in that order and $v(x) < v(y)$. Swapping them changes the string $P + x + y + S$ into $P + y + x + S$: same length, same prefix and suffix, and the middle part grows because $y + x > x + y$ by Theorem 4.5. So an optimal arrangement never has such a pair: its v values are non-increasing. Any two arrangements with non-increasing v differ only in the order inside groups of equal v, and swapping two neighbours with equal v does not change the string, because then $x + y = y + x$. So every arrangement with non-increasing v, including the sorted one, gives the same, optimal, string. ∎

One case remains. If every number is 0, the concatenation is "000…0", but the answer must be "0". Check the first string after sorting: if it is "0", every string is "0". The sort makes $O(n \log n)$ comparisons, each

building two strings of length up to twice the longest number. The sort with this comparator matched a brute force over all orders on 5,000 random inputs of up to 6 numbers; writing the rest of LeetCode 179 is left for the problem worksheet.

INFOSYS LENS

"Arrange numbers to form the largest number" is an Infosys repeat problem (four reports in the plan's question bank). It is LeetCode 179, with the same comparator and the same all-zeros case.

4.7 Greedy invariants beyond sorting

Two more greedy patterns are worth knowing because their correctness rests on an invariant that is easy to state and to check.

Reachability: all values 0..x

Suppose you have coins with positive values and ask which totals can be paid with a subset of them. Process the coins in increasing order and keep x, the largest value such that **every** total 0, 1, ..., x can be paid with the coins used so far.

LEMMA 4.7 Extending the reachable range

If every total 0..x can be paid with some set of coins and a new coin c satisfies $c \le x + 1$, then every total 0..x + c can be paid. If instead every remaining coin is larger than $x + 1$ and the coins used so far sum to x, the total $x + 1$ can never be paid.

Proof. For a total $t \le x$, use the old way. For $x < t \le x + c$, the total $t - c$ lies in 0..x because $t - c \ge x + 1 - c \ge 0$, so pay $t - c$ the old way and add c. For the second part: any subset paying $x + 1$ cannot use a remaining coin (each is larger than $x + 1$), and the used coins sum to only x. ∎

Starting from $x = 0$ and adding the coins in increasing order while $c \le x + 1$ keeps x equal to the sum of the coins used, so the lemma applies at every step. The scan stops at the first coin that is larger than $x + 1$: by the second half of the lemma, $x + 1$ is then the smallest total that cannot be paid, even though the remaining coins can pay many larger totals. Sorting is what makes this work: a coin processed out of order could be larger than $x + 1$ only because a smaller coin had not been added yet.

EXAMPLE 4.4 Which totals can coins pay?

Coins 1, 2, 5 and 13, in increasing order:

```
coin   test           x
 1     1 <= 0 + 1     0 -> 1
 2     2 <= 1 + 1     1 -> 3
 5     5 >  3 + 1     stop: every total 0..3 can be paid, 4 cannot
```

With one more coin of value 4 the scan continues: 1, 2, 4, 5, 13 give x = 1, 3, 7, 12, 25, so every total 0..25 can be paid. Adding the single coin $x + 1 = 4$ at the point where the scan stopped extended the range from 3 to 25.

The scan costs O(n log n) for the sort and O(n) after it. This invariant is the key observation of LeetCode 1798 (Maximum Number of Consecutive Values You Can Make). Workbook Chapter 4 uses it for a design exercise: the fewest coins to add so that every total 1..n can be paid.

Constraints from both sides: two passes

Some problems put a constraint between each element and **both** of its neighbours. A single left-to-right pass can only see the left neighbour. The trick is two passes, one per direction, and a proof that the second pass does not break what the first one achieved.

Here is a new example. Given caps $h[0..n-1] \geq 0$, find the array b with $0 \leq b[i] \leq h[i]$ and $|b[i] - b[i+1]| \leq 1$ that is as large as possible in every position. (Think of terrain that must be levelled so that neighbouring columns differ by at most one, without ever adding material.)

Listing 4.8 Two passes: the largest array under caps with neighbours differing by at most 1

```
static int[] capHeights(int[] h) {                    // largest b <= h with |b[i] - b[i+1]| <= 1
    int n = h.length;
    int[] b = h.clone();
    for (int i = 1; i < n; i++) b[i] = Math.min(b[i], b[i - 1] + 1);       // pass 1: left neighbours
    for (int i = n - 2; i >= 0; i--) b[i] = Math.min(b[i], b[i + 1] + 1);  // pass 2: right neighbours
    return b;
}
```

THEOREM 4.8 **The two passes give the pointwise largest feasible array**

The array b returned by Listing 4.8 is feasible, and every feasible array c satisfies c[i] ≤ b[i] for every i.

Proof. Write L for the array after pass 1 and b for the result. *Feasible.* Pass 2 makes $b[i] \leq b[i+1] + 1$ directly. For the other direction, $b[i] = \min(L[i], b[i+1] + 1)$. If the minimum is $b[i+1] + 1$, then $b[i+1] \leq b[i] + 1$ trivially. Otherwise $b[i] = L[i]$, and $b[i+1] \leq L[i+1] \leq L[i] + 1 = b[i] + 1$, using that pass 2 only lowers values and pass 1 made $L[i+1] \leq L[i] + 1$. Values never rise above h and never fall below 0, because each is a minimum of non-negative quantities. *Largest.* Let c be feasible. By induction from the left, $c[i] \leq L[i]$: $c[0] \leq h[0] = L[0]$, and $c[i] \leq \min(h[i], c[i-1] + 1) \leq \min(h[i], L[i-1] + 1) = L[i]$. Then by induction from the right, $c[i] \leq \min(L[i], c[i+1] + 1) \leq \min(L[i], b[i+1] + 1) = b[i]$. ∎

For $h = [3, 0, 4, 6, 5, 1, 3]$, pass 1 gives $[3, 0, 1, 2, 3, 1, 2]$ and pass 2 gives $[1, 0, 1, 2, 2, 1, 2]$. Feasibility and maximality were checked against an enumeration of every feasible array on 20,000 random inputs of up to six positions. Both passes are $O(n)$.

LeetCode 135 (Candy) is the mirror image: the constraints are lower bounds (“a child with a higher rating than a neighbour gets more candies”), and the smallest feasible array comes from two passes that each take a maximum instead of a minimum.

Summary

- `Arrays.sort` uses dual-pivot quicksort for primitives (fast, average $O(n \log n)$; quadratic on crafted inputs in Java 8, so shuffle first when that matters) and stable TimSort for objects ($O(n \log n)$ worst case, slower because of boxing).
- No comparison sort beats $\log_2(n!) \approx n \log_2 n$ comparisons. Write comparators with `Integer.compare`, never subtraction, and make them consistent.
- A greedy is correct when an exchange argument shows that some optimal solution agrees with its first choice. Test every greedy against a brute force on small inputs; false rules fail within seconds.
- Merge intervals by sorting by start; only the last block can overlap the next interval.
- Keep the most non-overlapping intervals by sorting by **end**; fewest removals = n − most kept. Whether touching ends overlap changes `>=` into `>`.

- The fewest groups (rooms, platforms) equals the maximum overlap: a lower bound from one crowded point and a greedy that achieves it. Sweep sorted events with the right tie-break, or keep a min-heap of end times.
- Largest Number sorts by $x + y$ versus $y + x$; it is a valid order because it compares $v(x) = X/(t(x) - 1)$ with $t(x) = 10$ to the length of x, and sorting by it is optimal by adjacent exchanges. Handle the all-zeros case.
- "Every total $0..x$ can be paid" extends to $0..x + c$ with any coin $c \le x + 1$; scanning sorted coins finds the smallest total that cannot be paid. Constraints from both neighbours are handled by two passes and a proof that the second keeps the first.

Key terms

stable sort p. 60
greedy algorithm p. 61
Exchange argument p. 62
depth of intervals p. 65
Greedy-choice property p. 61
sweep line p. 66

Review questions

1. Why does stability not matter when sorting an `int[]`, but matter when sorting `int[][]` rows by their first column?
2. A comparator for `long` values is written `(x, y) -> (int) (x - y)`. Give two different reasons why it can sort incorrectly.
3. For interval scheduling, give a counterexample to the rule "take the interval that overlaps the fewest others", or explain why you could not find one quickly. What does this tell you about testing greedies?
4. In the exchange step of Theorem 4.3, why is the modified solution still feasible after replacing o_1 by g_1?
5. LeetCode 435 counts [1, 2] and [2, 3] as non-overlapping. What single change to Listing 4.4 would treat them as overlapping, and what would the fewest removals then be for [1, 2], [2, 3], [3, 4]?
6. Explain why the depth of a set of intervals is a lower bound on the number of rooms, and why the greedy never exceeds it.
7. In Listing 4.6, what goes wrong if equal-time events are sorted with ends first for closed intervals? Give the trains of Example 4.3 as an example.
8. Using $v(x)$ from Theorem 4.5, order the strings "4", "40", "45" and "7", and write the largest number they form.
9. Coins are [1, 1, 3, 9]. Using Lemma 4.7, what is the largest x such that every total $0..x$ can be paid? Which total is the first that cannot?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 3 "Sorting" (sorting theory, the $\Omega(n \log n)$ lower bound, sorting in C++) and Chapter 6 "Greedy algorithms" (coin problem, scheduling, tasks and deadlines, minimising sums). Free at cses.fi/book.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms* (4th edition), the chapter *Greedy Algorithms*, whose activity-selection problem is the interval scheduling of Section 4.4 with a full treatment of greedy-choice and optimal substructure.
- Java 8 API documentation, docs.oracle.com/javase/8/docs/api/: `java.util.Arrays` (the notes on `sort` for primitives and objects) and `java.util.Comparator`.
- Workbook Chapter 4 has an exchange-argument exercise, a sweep with tie-breaking to trace, and a false greedy to refute.

Problems

These are the problems for Day 4. Work them in Workbook Chapter 4, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
4.P1	LC 56 · Merge Intervals	MEDIUM	CORE
4.P2	LC 435 · Non-overlapping Intervals	MEDIUM	CORE
4.P3	LC 2406 · Divide Intervals Into Minimum Number of Groups	MEDIUM	CORE
4.P4	LC 179 · Largest Number	MEDIUM	CORE
4.P5	LC 1798 · Maximum Number of Consecutive Values You Can Make	MEDIUM	BONUS
4.P6	LC 452 · Minimum Number of Arrows to Burst Balloons	MEDIUM	BONUS
4.P7	LC 135 · Candy	HARD	BONUS

CHAPTER 6 · DAY 6 OF 29 · THURSDAY 12 NOVEMBER 2026 · LEARNING DAY

6

Binary Search and Searching on the Answer

When "can the answer be x?" is monotone, binary search x and check it greedily.

Part I · Foundations
Handbook cards · Binary search: lower and upper bound, Binary search on the answer
Before this chapter · Chapter 1 (long arithmetic, Arrays), Chapter 4 (greedy arguments)
Practice · Workbook Chapter 6 · 7 problems · core ≈ 4.1 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- state the loop invariant of a half-open binary search and use it to prove that lower bound and upper bound are correct and always stop;
- find the first and last position of a value, and count occurrences, with two lower-bound searches;
- turn an optimisation question into a monotone yes/no predicate, choose safe bounds, and search for the smallest or largest value that passes;
- prove that a greedy check is correct for “minimise the maximum” and “maximise the minimum” problems;
- avoid overflow and off-by-one errors with long bounds, ceiling division, correct rounding of the midpoint and a fixed number of iterations for real numbers;
- find the k-th smallest element of a set too large to list by counting the elements ≤ x.

Binary search is the first algorithm most programmers learn and one of the last they get right under pressure. The idea fits in a sentence: if a question about position i is answered “no” up to some point and “yes” after it, you can find that point by asking about the middle and discarding half of the remaining positions. An array of 10^6 elements needs at most 20 questions.

The idea reaches much further than sorted arrays. Many optimisation problems ask for “the smallest speed”, “the least capacity” or “the largest gap” such that something is possible. The set of candidate answers is a range of integers, and the question “is x good enough?” flips once from no to yes as x grows. Binary search over that range turns a hard optimisation problem into a sequence of easy yes/no checks, each of which is usually a greedy pass over the input. This chapter calls it **searching on the answer**.

The chapter starts with the sorted-array case and proves it correct with a loop invariant, because every later use rests on the same four lines of reasoning. It then builds the general framework, applies it to the “minimise the maximum” and “maximise the minimum” families, deals with overflow and real numbers, and ends with a counting trick that finds the k-th smallest element of a set too large to store.

6.1 Binary search on a sorted array

Suppose $a[0..n-1]$ is sorted in non-decreasing order and you want the first index whose value is at least x. A left-to-right scan answers this in $O(n)$. Binary search answers it in $O(\log n)$ by using the order: if $a[mid] \geq x$, then every later element is also $\geq x$, so the answer is mid or to its left; if $a[mid] < x$, every earlier element is also $< x$, so the answer is to the right of mid.

DEFINITION 6.1 Lower bound and upper bound

For a sorted array $a[0..n-1]$ and a value x, the **lower bound** of x is the smallest index i with $a[i] \geq x$, or n if there is none. The **upper bound** of x is the smallest index i with $a[i] > x$, or n if there is none.

Both are positions where x could be inserted while keeping the array sorted: the lower bound is the leftmost such position and the upper bound the rightmost. The answer n ("insert at the end") is a legal result, so the search range has $n + 1$ possible answers, 0 to n.

The two searches are instances of one problem. Let $P(i)$ be a yes/no property of indices that is false for every i below some threshold t and true for every $i \geq t$; for the lower bound, $P(i)$ is "$a[i] \geq x$", which has this shape because a is sorted. The task is to find t. The template keeps the answer inside a **half-open** range $[lo, hi)$: every index left of lo is known to be false and every index from hi on is known to be true.

Listing 6.1 Lower and upper bound on a sorted int array (handbook template)

```
// First index with a[i] >= x (a.length if none). a must be sorted.
static int lowerBound(int[] a, int x) {
    int lo = 0, hi = a.length;              // answer is in [lo, hi]
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] >= x) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

// First index with a[i] > x. Count of x = upperBound(a, x) - lowerBound(a, x).
static int upperBound(int[] a, int x) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] > x) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

The comment "answer is in [lo, hi]" refers to the closed range of possible *results*; the indices still to be examined form the half-open range $[lo, hi)$. When the loop stops, the two ends meet and the result is lo.

Why it is correct

A **loop invariant** is a statement about the variables that is true before the loop starts and stays true after every iteration. If it is chosen well, the invariant together with the stopping condition gives the result.

THEOREM 6.1 Correctness of the half-open binary search

Let P be a property of the indices $0..n-1$ that is false for $i < t$ and true for $t \leq i < n$, where $0 \leq t \leq n$. The loop of Listing 6.1, with the test `a[mid] >= x` replaced by $P(mid)$, stops and returns t. In particular `lowerBound` returns the lower bound and `upperBound` returns the upper bound.

Proof. The invariant is: (I1) $0 \leq lo \leq hi \leq n$; (I2) $P(i)$ is false for every $i < lo$; (I3) $P(i)$ is true for every i with $hi \leq i < n$. Before the loop, $lo = 0$ and $hi = n$, so (I2) and (I3) speak about no indices at all and hold trivially.

Suppose the invariant holds and $lo < hi$. Then $mid = \lfloor (lo + hi)/2 \rfloor$ satisfies $lo \leq mid < hi$, so mid is a real index. If $P(mid)$ is true, then $mid \geq t$, so every $i \geq mid$ is true and setting $hi = mid$ keeps (I3). If $P(mid)$ is false, then $mid < t$, so every $i \leq mid$ is false and setting $lo = mid + 1$ keeps (I2). In both cases (I1) still holds, and $hi - lo$

has strictly decreased because $lo \le mid < hi$. A non-negative integer cannot decrease for ever, so the loop stops.

When it stops, $lo = hi$. By (I2) all indices below lo are false and by (I3) all indices from lo on are true, so lo is the first true index, which is t. For `lowerBound`, $P(i)$ is "$a[i] \ge x$"; for `upperBound` it is "$a[i] > x$". Both are false-then-true because a is sorted. ∎

LEMMA 6.2 Number of iterations

On an array of length $n \ge 1$, the loop of Listing 6.1 runs at most $\lfloor \log_2 n \rfloor + 1$ times.

Proof. Let $L = hi - lo$. Since $mid = lo + \lfloor L/2 \rfloor$, the branch $hi = mid$ leaves a range of length $\lfloor L/2 \rfloor$, and the branch $lo = mid + 1$ leaves $L - \lfloor L/2 \rfloor - 1 = \lceil L/2 \rceil - 1 \le \lfloor L/2 \rfloor$. So each iteration replaces L by at most $\lfloor L/2 \rfloor$. Starting from $L = n$, after k iterations $L \le \lfloor n/2^k \rfloor$, which is 0 as soon as $2^k > n$, that is, after $\lfloor \log_2 n \rfloor + 1$ iterations. ∎

For $n = 10^6$ that is 20 iterations, for $n = 10^9$ it is 30. A simulation that always follows the longer half reaches exactly these counts, so the bound is tight.

EXAMPLE 6.1 Tracing the lower bound

Take $a = [1, 3, 3, 3, 5, 8, 8, 9]$. The first trace searches for $x = 3$, the second for $x = 4$, which is not in the array.

```
lowerBound(a, 3)                    lowerBound(a, 4)
lo  hi  mid  a[mid]  decision        lo  hi  mid  a[mid]  decision
 0   8   4     5     hi = 4          0   8   4     5     hi = 4
 0   4   2     3     hi = 2          0   4   2     3     lo = 3
 0   2   1     3     hi = 1          3   4   3     3     lo = 4
 0   1   0     1     lo = 1         result 4
result 1
```

For $x = 3$ the search keeps moving left even after it has seen a 3 at index 2, because the question is "≥ 3?", not "= 3?". It stops at index 1, the first 3. For $x = 4$ the result is 4, the position where 4 would be inserted; $a[4] = 5 \ne 4$ tells you that 4 is absent. The upper bound of 3 is also 4: its trace is the right-hand one, because "> 3" and "≥ 4" are the same question for integers.

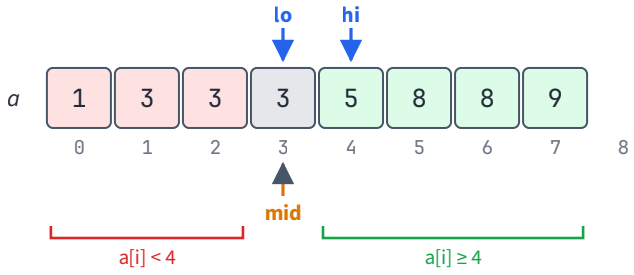


Figure 6.1 The invariant of lowerBound(a, 4) after two iterations. Indices left of lo are known to fail (a[i] < 4, red), indices from hi on are known to pass (green), and only [lo, hi) is still unknown.

The classic off-by-one failures

Every binary search bug breaks one of the three facts the proof used: the invariant holds at the start, each branch keeps it, and each branch shrinks the range. Table 6.1 shows four common edits of Listing 6.1 and

the smallest sorted input on which each fails. The inputs were found by trying every sorted array of length up to 4 with values 0..2.

Table 6.1 Four plausible edits of the lower-bound template and their smallest failing inputs

Edit	Smallest failing input	What happens
`hi = a.length - 1`, loop `while (lo <= hi)`, still `hi = mid`	$a = [0], x = -1$	$lo = hi = mid = 0$ passes, $hi = mid$ changes nothing: infinite loop
`hi = a.length - 1`, loop `while (lo < hi)`	$a = [0], x = 1$	returns 0, but no element is ≥ 1; the answer $n = 1$ was outside the range
`hi = mid - 1` when $a[mid] \geq x$	$a = [0, 1], x = 1$	$mid = 1$ passes and is thrown away; returns 0
"last index with $a[i] \leq x$" using `lo = mid` and *mid* rounded down	$a = [0, 0], x = 0$	$lo = 0$, $hi = 1$, $mid = 0$ passes, $lo = mid$: infinite loop

The last row deserves a closer look because it is the bug that makes programs hang. Here is the method as people often write it.

Listing 6.2 WRONG: the last index with a[i] <= x, with the midpoint rounded down

```
static int lastAtMostWrong(int[] a, int x) {
    int lo = -1, hi = a.length - 1;         // answer is in [lo, hi]; -1 means "none"
    while (lo < hi) {
        int mid = (lo + hi) / 2;            // rounds down: mid can equal lo
        if (a[mid] <= x) lo = mid;          // so this branch may not move
        else hi = mid - 1;
    }
    return lo;
}
```

On $a = [1, 2]$ with $x = 5$ the variables go $lo = -1$, $hi = 1$, $mid = 0$, then $lo = 0$, $hi = 1$, $mid = 0$, and from then on nothing changes:

```
lo  hi  mid  a[mid]
-1   1   0     1      a[0] <= 5, lo = 0
 0   1   0     1      a[0] <= 5, lo = 0   (no progress)
 0   1   0     1      ...
```

When the branch that moves *lo* is `lo = mid`, the midpoint must be rounded **up**, so that $lo < mid \leq hi$ whenever $lo < hi$. Then both branches shrink the range.

Listing 6.3 The last index with a[i] <= x (-1 if none): round the midpoint up

```
static int lastAtMost(int[] a, int x) {
    int lo = -1, hi = a.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo + 1) / 2;   // rounds up: lo < mid <= hi
        if (a[mid] <= x) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
```

The result always equals `upperBound(a, x) - 1`, and in practice that one-liner is the safer choice. The rounding rule matters for searches on the answer in Section 6.4, where "largest x that works" is the natural form.

KEY IDEA

Pair the branches with the rounding. If the branches are `hi = mid` and `lo = mid + 1`, round *mid* down. If they are `lo = mid` and `hi = mid - 1`, round *mid* up. Any other pairing can stop moving.

Java's `Arrays.binarySearch`

The standard library has `Arrays.binarySearch(a, key)` for every primitive array type and `Collections.binarySearch(list, key)` for lists. The array must be sorted; on an unsorted array the result is meaningless. The return value encodes two different answers:

- If the key is present, the result is *some* index holding it. The documentation says explicitly that when several elements equal the key, there is no guarantee which one is found.
- If the key is absent, the result is $-(p) - 1$, where p is the insertion point, the index of the first element greater than the key. That insertion point is exactly the lower bound, so `lowerBound = -r - 1` when $r < 0$. The "-1" makes the result negative even when $p = 0$.

On $a = [1, 3, 3, 3, 5, 8, 8, 9]$ the calls return:

```
Arrays.binarySearch(a, 3)  ->  3      (the 3s are at indices 1..3; the first is 1)
Arrays.binarySearch(a, 8)  ->  5      (happens to be the first 8)
Arrays.binarySearch(a, 4)  -> -5      (insertion point 4)
Arrays.binarySearch(a, 10) -> -9      (insertion point 8 = n)
Arrays.binarySearch(a, 0)  -> -1      (insertion point 0)
```

PITFALL

`Arrays.binarySearch` returns *any* matching index, so it cannot answer "first position", "last position" or "how many". On [2, 2, 2, 2] it returns 1 for the key 2. Write `lowerBound` and `upperBound` yourself; they take two minutes and are always right.

JAVA NOTE

The template computes the midpoint as `(lo + hi) >>> 1`. The operator `>>>` shifts right and fills with zeros, so it treats the 32 bits of `lo + hi` as an unsigned number. For two non-negative `int`s the true sum is below 2^{32}, so the result is correct even when `lo + hi` has overflowed into the negative range. Section 6.5 compares this with `lo + (hi - lo) / 2`.

6.2 First and last position, and counting

LeetCode 34 (Find First and Last Position of Element in Sorted Array) asks for the first and last index of a target in a sorted array, or $[-1, -1]$ if it is absent, in $O(\log n)$ time. Both ends are lower bounds.

THEOREM 6.3 Upper bound as a lower bound

For a sorted `int` array and an integer $x < 2^{31} - 1$, upperBound(a, x) = lowerBound(a, x + 1). Consequently, if x occurs in a, its occurrences are exactly the indices lowerBound(a, x) .. lowerBound(a, x + 1) − 1, and the number of occurrences is lowerBound(a, x + 1) − lowerBound(a, x).

Proof. For integers, $a[i] > x$ holds exactly when $a[i] \geq x + 1$, so the two searches test the same property at every index and return the same index by Theorem 6.1. Indices below lowerBound(a, x) hold values $< x$;

indices from upperBound(a, x) on hold values $> x$; every index in between holds a value that is $\geq x$ and $\leq x$, that is, equal to x. ∎

So the first position is f = lowerBound(a, x); if $f = n$ or $a[f] \neq x$, the value is absent. Otherwise the last position is lowerBound(a, $x + 1$) − 1. On the array of Example 6.1, the value 3 occupies indices 1..3 and the value 8 occupies 5..6, while for 4 the first candidate is index 4 with $a[4] = 5 \neq 4$, so 4 is absent.

PITFALL

`x + 1` overflows when x = `Integer.MAX_VALUE` and becomes -2^{31}, whose lower bound is 0. LeetCode 34 keeps targets far from that limit, but in general prefer `upperBound(a, x) - 1` for the last position, or do the arithmetic in `long`.

The same two searches count the elements in any value range: the number of i with $L \leq a[i] \leq R$ is upperBound(a, R) − lowerBound(a, L). Sorting once and answering each range query with two binary searches costs $O(n \log n + q \log n)$; LeetCode 2300 (Successful Pairs of Spells and Potions), on Day 28, is this idea. When the set changes between queries, a `TreeMap` or `TreeSet` (Chapter 1) gives `floorKey`, `ceilingKey`, `lowerKey` and `higherKey` in $O(\log n)$ each, at a larger constant cost.

6.3 Searching on the answer

Now the main idea of the chapter. Many problems ask for the smallest (or largest) value of a parameter for which some task can be done: the slowest eating speed that finishes in time, the smallest ship that delivers everything within a deadline, the largest gap that still fits every item. Computing the optimum directly is often hard. Checking whether a *given* value works is often easy.

DEFINITION 6.2 Monotone predicate

A yes/no function ok(x) on the integers lo..hi is a **monotone predicate** if ok(x) true implies ok($x + 1$) true. Equivalently, there is a threshold t such that ok is false below t and true from t on. (For "maximise" problems the predicate runs the other way: true up to a threshold, false after it.)

Binary search on the answer finds the threshold of a monotone predicate with $O(\log(hi - lo))$ calls of ok. It is Theorem 6.1 with the array index replaced by the candidate answer. Four steps turn a problem into this form.

1. **Define the predicate.** Write the question "can the task be done with parameter x?" so that the answer to the problem is the smallest x with a yes.
2. **Prove monotonicity.** Show that a yes for x implies a yes for $x + 1$. Usually a solution that works for x also works for $x + 1$ unchanged ("a bigger ship can carry the same loads").
3. **Choose the bounds.** Pick $lo \leq$ the answer (often a value below which the task is impossible for an obvious reason) and hi with ok(hi) true (a value that certainly works).
4. **Write the check.** Implement ok(x), usually as one greedy pass in $O(n)$, and prove that the greedy pass answers the question exactly.

The search itself never changes, so it is worth having in a form that takes the predicate as a parameter. `LongPredicate` is the functional interface in `java.util.function` for a test on a `long`.

Listing 6.4 Smallest x that passes, largest x that passes

```
// Smallest x in [lo, hi] with ok(x) true.
// Requires: ok is false...false true...true on [lo, hi], and ok(hi) is true.
static long firstTrue(long lo, long hi, LongPredicate ok) {
```

```
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;          // rounds down: lo <= mid < hi
        if (ok.test(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

// Largest x in [lo, hi] with ok(x) true.
// Requires: ok is true...true false...false on [lo, hi], and ok(lo) is true.
static long lastTrue(long lo, long hi, LongPredicate ok) {
    while (lo < hi) {
        long mid = lo + (hi - lo + 1) / 2;      // rounds up: lo < mid <= hi
        if (ok.test(mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
```

THEOREM 6.4 Correctness of the answer search

Under its stated requirements, `firstTrue` *returns the smallest x in [lo, hi] with ok(x) true, and* `lastTrue` *returns the largest. Each makes at most* $\lfloor \log_2(hi - lo) \rfloor + 1$ *calls of ok when* $hi > lo$.

Proof. For `firstTrue` the invariant is: every y below the current lo (and at least the original lo) has ok(y) false, and ok(hi) is true. It holds at the start by the requirements. With $lo < hi$, the midpoint satisfies $lo \le mid < hi$. If ok(mid) is true, $hi = mid$ keeps "ok(hi) true"; if false, monotonicity makes every $y \le mid$ false, so $lo = mid + 1$ keeps the first part. The gap $hi - lo$ shrinks by at least half each time (the argument of Lemma 6.2), so the loop stops with $lo = hi$, a true value whose predecessors are all false. For `lastTrue` the invariant is mirrored: ok(lo) is true and every y above hi is false. Because mid is rounded up, $lo < mid \le hi$, so both `lo = mid` and `hi = mid - 1` shrink the gap. ∎

With these in hand, a solution is a predicate and two bounds, for example `firstTrue(1, maxPile, k -> hours(piles, k) <= h)`. In an exam you may prefer to write the loop inline with the check; the reasoning is the same.

EXAMPLE 6.2 Eating speed, step by step

This is the shape of LeetCode 875 (Koko Eating Bananas). There are piles of bananas and h hours. At speed k bananas per hour, a pile of p bananas takes $\lceil p/k \rceil$ hours (a partly eaten hour is not shared between piles). Find the smallest k that finishes within h hours. Take the piles [4, 9, 2, 13, 7] and $h = 9$.

Predicate. ok(k) is "hours(k) $\le h$", where hours(k) $= \Sigma \lceil p/k \rceil$.

Monotonicity. For each pile, $\lceil p/k \rceil$ does not increase when k grows, so hours(k) is non-increasing and ok is false-then-true.

Bounds. $lo = 1$, since a speed must be positive. hi = the largest pile, 13: at that speed every pile takes one hour, so hours = 5, the number of piles, and the problem guarantees there are at most h piles.

Check. One pass over the piles. The table shows hours(k) for every candidate.

k	1	2	3	4	5	6	7	8	9	10	11	12	13
hours(k)	35	19	14	11	9	9	7	7	6	6	6	6	5
ok?	n	n	n	n	y	y	y	y	y	y	y	y	y

The search visits only four of the thirteen candidates:

```
lo  hi  mid  hours(mid)  decision
 1  13    7       7      ok          -> hi = 7
 1   7    4      11      too slow    -> lo = 5
 5   7    6       9      ok          -> hi = 6
 5   6    5       9      ok          -> hi = 5
answer 5
```

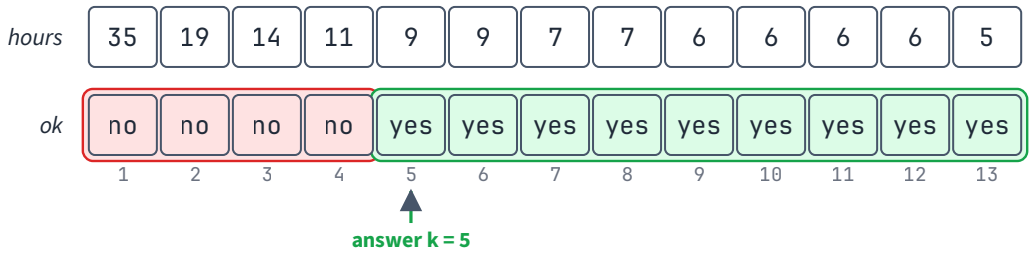


Figure 6.2 The predicate "hours(k) ≤ 9" for the piles [4, 9, 2, 13, 7], indexed by the speed k. It is false up to k = 4 and true from k = 5 on; the search finds the boundary.

PITFALL

Two overflow traps hide in this check. The total of hours can reach 10^4 piles × 10^9 hours = 10^{13} at speed 1, so it must be a `long`. And `(p + k - 1) / k`, the usual ceiling, needs `p + k - 1` to fit; for p, $k \le 10^9$ it is at most 1,999,999,999 < $2^{31} - 1$, but only just. Computing it in `long` removes the doubt.

Ship capacity and the greedy check

LeetCode 1011 (Capacity To Ship Packages Within D Days) gives package weights in a fixed order and a number of days. Each day the ship takes a contiguous run of the remaining packages whose total weight is at most its capacity. Find the least capacity that ships everything within the given number of days. The handbook template is the complete solution, and it is worth knowing line by line because the same check reappears in Section 6.4.

Listing 6.5 Least ship capacity: the predicate is "greedy needs at most `days` days" (handbook template)

```java
// "Minimise the maximum": smallest capacity that ships all packages, in order, within `days` days
static long minCapacity(int[] weights, int days) {
    long lo = 0, hi = 0;
    for (int w : weights) {
        lo = Math.max(lo, w);                  // must carry the heaviest package
        hi += w;                               // one day for everything always works
    }
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;
        if (daysNeeded(weights, mid) <= days) hi = mid;   // mid works: try smaller
        else lo = mid + 1;                               // mid fails: need bigger
    }
    return lo;
}

static int daysNeeded(int[] weights, long cap) {         // greedy check, O(n)
    int d = 1;
    long load = 0;
    for (int w : weights) {
        if (load + w > cap) {
            d++;
            load = 0;
        }
        load += w;
```

```
    }
    return d;
}
```

The search is `firstTrue` written inline. The interesting part is the check: why is "load each day as full as possible" the fewest days?

LEMMA 6.5 The greedy day count is optimal

Let cap ≥ the largest weight. Then `daysNeeded(weights, cap)` equals the minimum number of days over all valid schedules with that capacity.

Proof. Describe a schedule by the index of the last package shipped on each day. Let $g_1 < g_2 < \cdots$ be these indices for the greedy schedule and $o_1 < o_2 < \cdots$ for any valid schedule. We show by induction that $g_j \geq o_j$ for every j for which both exist ("greedy stays ahead").

For $j = 1$: the other schedule's first day is the packages $0..o_1$ with total ≤ cap. Greedy keeps adding packages while the total stays ≤ cap, so it ships at least these, and $g_1 \geq o_1$.

For the step, assume $g_j \geq o_j$. If $g_j \geq o_{j+1}$, then $g_{j+1} > g_j \geq o_{j+1}$. Otherwise greedy's day $j + 1$ starts at $g_j + 1 \leq o_{j+1}$, and the packages $g_j + 1..o_{j+1}$ are part of the other schedule's day $j + 1$, which starts at $o_j + 1 \leq g_j + 1$. Their total is at most that day's total, which is ≤ cap, so greedy can take all of them and $g_{j+1} \geq o_{j+1}$.

If the other schedule uses D days, its last index is $o_D = n - 1$, so $g_D \geq n - 1$: greedy has finished within D days. Because cap ≥ every weight, each greedy day contains at least one package and respects the capacity, so greedy's schedule is itself valid. Hence its day count is the minimum. ∎

Monotonicity is immediate: a schedule that works for capacity c works unchanged for $c + 1$. The bounds are safe: below the heaviest package nothing works, and the total weight ships everything in one day. So the answer is the smallest capacity in [max, total] whose greedy day count is at most `days`.

EXAMPLE 6.3 Ship capacity on weights 1..10 with 5 days

The search runs over [10, 55]:

```
lo  hi  mid  days needed  decision
10  55   32       2       works -> hi = 32
10  32   21       3       works -> hi = 21
10  21   15       5       works -> hi = 15
10  15   12       6       fails -> lo = 13
13  15   14       6       fails -> lo = 15
answer: 15
```

With capacity 15 the greedy days are [1, 2, 3, 4, 5], [6, 7], [8], [9], [10]: five days, the heaviest load exactly 15.

PITFALL

The lower bound `lo = max weight` is not an optimisation; it is needed for correctness. `daysNeeded` assumes every package fits. With `lo = 0`, the weights [1, 2] and 2 days give the answer 1: `daysNeeded` puts the 2 on its own day, counts 2 days, and never notices that the load exceeds the capacity. Lemma 6.5 only holds for cap ≥ the largest weight.

6.4 Minimise the maximum, maximise the minimum

Two phrases in a statement almost always signal a search on the answer: “minimise the largest” and “maximise the smallest”. The reason is that the inner quantity turns into a threshold. “The largest part is as small as possible” becomes “every part is ≤ S; how small can S be?”, and “every part ≤ S” is exactly the kind of condition a greedy pass can check.

Splitting an array into k parts

LeetCode 410 (Split Array Largest Sum) asks you to cut an array of non-negative integers into k non-empty contiguous parts so that the largest part sum is as small as possible. It looks like the ship problem with one difference: the ship had *at most* `days` days, while here the number of parts is *exactly* k. The next lemma removes the difference.

LEMMA 6.6 At most k parts is as good as exactly k

Let the elements be non-negative and $n \geq k$. If a can be cut into $j \leq k$ contiguous parts each with sum ≤ S, it can be cut into exactly k such parts.

Proof. While there are fewer than k parts, there are fewer than n parts, so some part has at least two elements. Cut it into two non-empty pieces. Each piece’s sum is at most the sum of the part, because the removed elements are non-negative, so both are ≤ S. Each cut adds one part; after $k - j$ cuts there are exactly k. ∎

THEOREM 6.7 Split array largest sum

For non-negative a and $1 \leq k \leq n$, the minimum possible largest part sum equals the smallest $S \geq \max(a)$ for which the greedy count of Listing 6.5, `daysNeeded(a, S)`, is at most k.

Proof. Call S feasible if some cut into exactly k parts has every part sum ≤ S; the answer is the smallest feasible S. By Lemma 6.6, S is feasible exactly when some cut into at most k parts has every sum ≤ S. For $S \geq \max(a)$, Lemma 6.5 (with packages = elements and days = parts) says the fewest parts with sums ≤ S is `daysNeeded(a, S)`, so S is feasible exactly when that count is ≤ k. No S below $\max(a)$ is feasible, because the part containing the largest element has a sum at least that element. Feasibility is monotone in S, so the smallest feasible S is what `firstTrue` over $[\max(a), \Sigma a]$ returns. ∎

EXAMPLE 6.4 Three parts of [4, 1, 6, 2, 5, 3]

The bounds are max = 6 and total = 21. The greedy part counts are:

```
S       6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
parts   5  4  3  3  3  2  2  2  2  2  2  2  2  2  2  1
```

and the search needs four checks:

```
lo  hi  mid  parts  decision
 6  21   13      2    ok    -> hi = 13
 6  13    9      3    ok    -> hi = 9
 6   9    7      4    fails -> lo = 8
 8   9    8      3    ok    -> hi = 8
answer 8
```

With $S = 8$ the greedy parts are [4, 1], [6, 2], [5, 3] with sums 5, 8 and 8. No cut does better: with $S = 7$ the greedy needs four parts, and by Lemma 6.5 nothing needs fewer.

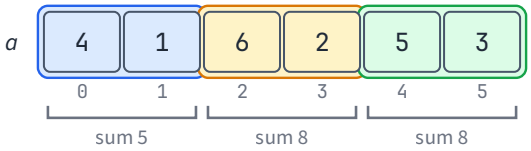


Figure 6.3 The greedy cut of [4, 1, 6, 2, 5, 3] for S = 8: each part takes elements until the next one would push its sum above 8.

Maximise the minimum

Now the mirror image, on a small problem of its own. Cut an array of non-negative integers into exactly k contiguous pieces; you will receive the piece with the smallest sum. How large can that smallest sum be made?

Here the predicate is ok(s) = "the array can be cut into k pieces each with sum ≥ s". It is true-then-false: if every piece reaches s, every piece reaches $s - 1$. So the answer is the *largest* s that passes, found with `lastTrue` and a midpoint rounded up.

The check is again greedy: walk left to right and close a piece as soon as its sum reaches s. Closing early never hurts, because a piece that has reached s gains nothing from more elements, while the next piece gains a head start.

Listing 6.6 Maximise the smallest piece sum (non-negative values, 1 <= k <= n)

```
// Largest s such that a can be cut into k contiguous pieces, each with sum >= s
static long maxMinPiece(int[] a, int k) {
    long lo = 0, hi = 0;                        // s = 0 always works
    for (int x : a) hi += x;                    // no piece can beat the total
    while (lo < hi) {
        long mid = lo + (hi - lo + 1) / 2;      // round UP: the branch below is lo = mid
        if (piecesAtLeast(a, mid) >= k) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

// Greedy: the most pieces with sum >= s. A leftover tail (sum < s) joins the last piece.
static int piecesAtLeast(int[] a, long s) {
    int pieces = 0;
    long cur = 0;
    for (int x : a) {
        cur += x;
        if (cur >= s) {
            pieces++;
            cur = 0;
        }
    }
    return pieces;
}
```

LEMMA 6.8 **The greedy piece count**

For non-negative a and $s \geq 0$, `piecesAtLeast(a, s)` is the largest number of disjoint contiguous pieces with sum ≥ s that a cut of the array can contain. Moreover a can be cut into exactly k pieces with every sum ≥ s if and only if `piecesAtLeast(a, s)` ≥ k.

Proof. The first claim is "greedy stays ahead" again: by induction, greedy's j-th piece ends no later than the j-th qualifying piece of any other cut, because greedy closes a piece at the first moment its sum reaches s, and starting no later can only help (all values are non-negative). For the second claim: if greedy finds $p \geq k$

pieces, merge pieces $k..p$ and any leftover tail into one; sums only grow when pieces merge, so all k pieces still reach s. Conversely, a cut into k pieces that all reach s shows that at least k qualifying pieces exist, so greedy finds at least k. ∎

On $a = [5, 2, 4, 1, 3, 6]$ with $k = 3$ the search runs:

```
lo  hi  mid  pieces  decision
 0  21   11     1    fails -> hi = 10
 0  10    5     3    ok    -> lo = 5
 5  10    8     2    fails -> hi = 7
 5   7    6     3    ok    -> lo = 6
 6   7    7     2    fails -> hi = 6
answer 6
```

The pieces for $s = 6$ are [5, 2], [4, 1, 3] and [6], with sums 7, 8 and 6. Look at the fourth row. With $lo = 6$ and $hi = 7$, a midpoint rounded down would be 6, which passes, so `lo = mid` would set lo to 6 again and the loop would never end. Rounding up gives 7, which fails and closes the range.

The same shape elsewhere

The "largest gap" problems follow this pattern with positions instead of sums. To place m items at sorted positions so that the closest two are as far apart as possible, the predicate is "a greedy left-to-right placement, taking every position at least d after the last item placed, places at least m items"; it is true-then-false in d, so you search the largest d with the midpoint rounded up. LeetCode 1552 (Magnetic Force Between Two Balls), a bonus problem today, has exactly this shape. In LeetCode 2226 (Maximum Candies Allocated to K Children) the check is a sum of quotients, $\Sigma \lfloor \text{pile} / x \rfloor \geq k$; watch the candidate $x = 0$, which would divide by zero.

Table 6.2 The two optimisation shapes

Statement says	Predicate	Shape	Search	Midpoint
minimise the largest …	every part ≤ x is achievable	false … true	smallest true	round down
maximise the smallest …	every part ≥ x is achievable	true … false	largest true	round up

6.5 Overflow, bounds and real numbers

A binary search on the answer often runs over values far larger than array indices: capacities up to 10^{14}, speeds up to 10^9, positions down to -10^9. Four details decide whether the code survives such ranges.

The midpoint

The textbook midpoint `(lo + hi) / 2` overflows when `lo + hi` exceeds $2^{31} - 1$.

```
int lo = 1_500_000_000, hi = 2_000_000_000;
int bad = (lo + hi) / 2;          // -397483648: lo + hi wrapped around
int good = lo + (hi - lo) / 2;    // 1750000000
int alsoGood = (lo + hi) >>> 1;   // 1750000000: the sum's bits read as unsigned
```

`(lo + hi) >>> 1` is safe only when both ends are non-negative. Searches over negative values break it, and they also expose a second problem: Java's `/` rounds toward zero, not down. With $lo = -3$ and $hi = -2$, `(lo + hi) / 2` is −2, which equals hi; in `firstTrue` the branch `hi = mid` then makes no progress. `lo + (hi - lo) / 2` gives −3, because `hi - lo` is non-negative and its half rounds down. Use `lo + (hi - lo) / 2` (or `+ 1` inside for rounding up) everywhere, with `long` variables whenever the range might pass 2^{31}.

Long bounds and sums inside the check

The bounds must be `long` when the answer can exceed `int`: the total of 10^5 weights of 10^9 is 10^{14}. Inside the check, every running sum must be `long` as well, and so must every product such as `mid * count`. When *hi* is generous (say $2 \cdot 10^{18}$ to be safe), a product in the check can overflow even `long`; compare by division instead (`count > limit / mid`), or keep *hi* close to a proven bound.

A generous *hi* costs very little time: doubling the range adds one iteration. A *hi* that does not satisfy ok(*hi*) costs correctness, since `firstTrue` then returns *hi* even though nothing works. If the problem allows "impossible", check ok(*hi*) once before searching and report the impossibility separately.

Ceiling division

For $a \ge 0$ and $b > 0$, $\lceil a/b \rceil = \lfloor (a + b - 1)/b \rfloor$. Java's `/` truncates toward zero, which is the floor only for non-negative results: `-7 / 2` is −3, while $\lfloor -7/2 \rfloor = -4$. `Math.floorDiv` always rounds down, and ceilings of possibly negative values follow from $\lceil a/b \rceil = -\lfloor -a/b \rfloor$.

Listing 6.7 Ceiling division

```
static long ceilDiv(long a, long b) {          // a >= 0, b > 0
    return (a + b - 1) / b;
}

static long ceilDivAnySign(long a, long b) {   // any a, b > 0 (Java 8 has no Math.ceilDiv)
    return -Math.floorDiv(-a, b);
}
```

Searching over real numbers

Some answers are real numbers: a radius, a speed, an average. The predicate is the same, but the range $[lo, hi]$ never shrinks to a single value, so the loop needs a different stopping rule. The safe rule is a **fixed number of iterations**. Each iteration halves the gap, so 100 iterations shrink it by a factor of $2^{100} \approx 10^{30}$, far below any tolerance a problem asks for.

The tempting rule `while (hi - lo > 1e-9)` can loop for ever. A `double` has about 16 significant decimal digits, so near 10^9 consecutive doubles are `Math.ulp(1e9)` $= 1.19 \cdot 10^{-7}$ apart. Once *lo* and *hi* are neighbouring doubles, their midpoint rounds to one of them, the gap stays at $1.19 \cdot 10^{-7}$, and the condition never becomes false.

A good example links this chapter with Chapter 2: find the largest average of a subarray of length at least L. Checking a candidate average m directly is awkward, but subtracting m from every element turns it into a prefix-sum question.

CLAIM 6.9 **The average check**

Some subarray of length $\ge L$ has average $\ge m$ if and only if, with P the prefix sums of $a[i] - m$, some $j \ge L$ has $P[j] - \min\{P[i] : 0 \le i \le j - L\} \ge 0$.

Proof. The subarray $a[i..j-1]$ has length $j - i$ and average $\ge m$ exactly when its sum is $\ge m(j - i)$, that is, when $\Sigma(a[t] - m)$ over the subarray is ≥ 0, which is $P[j] - P[i] \ge 0$. Its length is $\ge L$ exactly when $i \le j - L$. For a fixed j, some valid i works if and only if the smallest $P[i]$ over the valid i works. ∎

The predicate is true-then-false in m (an average that reaches m reaches anything smaller), so the search keeps the largest m that passes.

Listing 6.8 Largest average of a subarray with length >= L, by a real-valued search

```
static double maxAverage(int[] a, int L) {
    double lo = Integer.MAX_VALUE, hi = Integer.MIN_VALUE;
    for (int x : a) {
        lo = Math.min(lo, x);                  // every average is >= min(a) ...
        hi = Math.max(hi, x);                  // ... and <= max(a)
    }
    for (int iter = 0; iter < 100; iter++) {   // fixed count, never "while (hi - lo > eps)"
        double mid = (lo + hi) / 2;
        if (canReach(a, L, mid)) lo = mid;
        else hi = mid;
    }
    return lo;
}

static boolean canReach(int[] a, int L, double m) {     // some length >= L has average >= m?
    int n = a.length;
    double[] p = new double[n + 1];                     // prefix sums of a[i] - m
    for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i] - m;
    double minPrefix = Double.MAX_VALUE;
    for (int j = L; j <= n; j++) {
        minPrefix = Math.min(minPrefix, p[j - L]);      // best left end i <= j - L
        if (p[j] - minPrefix >= 0) return true;
    }
    return false;
}
```

EXAMPLE 6.5 **The first iterations of the real search**

For $a = [3, -1, 4, -1, 5, -9, 2, 6]$ and $L = 3$ the range starts at $[-9, 6]$:

```
iter   lo        hi        mid       canReach(mid)
  0   -9.0000    6.0000   -1.5000    yes -> lo = mid
  1   -1.5000    6.0000    2.2500    yes -> lo = mid
  2    2.2500    6.0000    4.1250    no  -> hi = mid
  3    2.2500    4.1250    3.1875    no  -> hi = mid
  4    2.2500    3.1875    2.7188    no  -> hi = mid
  5    2.2500    2.7188    2.4844    yes -> lo = mid
  6    2.4844    2.7188    2.6016    yes -> lo = mid
  7    2.6016    2.7188    2.6602    yes -> lo = mid
after 100 iterations: 2.666666667
```

The answer is 8/3, the average of $a[2..4] = [4, -1, 5]$; a brute force over all subarrays of length ≥ 3 agrees.

The cost is $100 \times O(n)$. Each check is linear because the minimum prefix is carried along instead of recomputed.

6.6 Counting instead of searching: the k-th smallest

Some problems ask for the k-th smallest element of a collection that is far too large to list: all products $i \cdot j$ in a table, all pairwise differences, all sums of two arrays. The trick is to binary search on the *value* and ask a counting question at each step.

Let count(x) be the number of elements $\leq x$. It is non-decreasing in x, so "count(x) $\geq k$" is a monotone predicate.

THEOREM 6.10 The k-th smallest by counting

Let $s_1 \le s_2 \le \cdots \le s_N$ be the elements of a finite collection of integers, sorted, and $1 \le k \le N$. Then s_k is the smallest integer x with count$(x) \ge k$. In particular that smallest x is always an element of the collection.

Proof. count$(s_k) \ge k$, because $s_1, \ldots, s_k$ are all $\le s_k$. Now take any $x < s_k$. Every element $\le x$ is smaller than s_k, so it is among $s_1, \ldots, s_{k-1}$ (the elements from s_k on are $\ge s_k > x$). Hence count$(x) \le k - 1 < k$. So s_k passes and nothing smaller does. ∎

Take the multiplication table with n rows and m columns, where the entry in row i and column j is $i \cdot j$ (1-based). Row i holds $i, 2i, \ldots, m{\cdot}i$, so the number of entries $\le x$ in that row is $\min(m, \lfloor x/i \rfloor)$, and count$(x)$ is the sum over the rows.

Listing 6.9 The k-th smallest entry of an n x m multiplication table

```java
// Entries are i * j for 1 <= i <= n, 1 <= j <= m. Requires 1 <= k <= n * m.
static long kthInTable(int n, int m, long k) {
    long lo = 1, hi = (long) n * m;             // the smallest and largest entries
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;
        if (countAtMost(n, m, mid) >= k) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

static long countAtMost(int n, int m, long x) {   // entries <= x: row i holds i, 2i, ..., m*i
    long c = 0;
    for (int i = 1; i <= n; i++) c += Math.min(m, x / i);
    return c;
}
```

EXAMPLE 6.6 The 7th smallest entry of a 3 × 4 table

The table has rows [1, 2, 3, 4], [2, 4, 6, 8] and [3, 6, 9, 12]. Its counts are:

```
x          1   2   3   4   5   6   7   8   9  10  11  12
count(x)   1   3   5   7   7   9   9  10  11  11  11  12
```

For $k = 7$ the search goes:

```
lo  hi  mid  count(mid)  decision
 1  12    6       9      enough  -> hi = 6
 1   6    3       5      too few -> lo = 4
 4   6    5       7      enough  -> hi = 5
 4   5    4       7      enough  -> hi = 4
answer 4
```

Sorting all twelve entries gives [1, 2, 2, 3, 3, 4, 4, 6, 6, 8, 9, 12], whose 7th element is 4. Note that count(5) is also 7, yet 5 is not in the table; the search returns 4 because it looks for the *smallest* x with enough elements, exactly as Theorem 6.10 requires.

	j=1	j=2	j=3	j=4
i=1	1	2	3	4
i=2	2	4	6	8
i=3	3	6	9	12

Figure 6.4 The 3 × 4 multiplication table. The seven entries ≤ 4 (blue) show count(4) = 7, so 4 is the 7th smallest.

The cost is $O(n \log(nm))$: about 30 iterations of an $O(n)$ count. For $n = m = 3 \cdot 10^4$ the table has $9 \cdot 10^8$ entries, which would need 3.6 GB as an `int[]`, while the search does about $30 \times 3 \cdot 10^4 \approx 10^6$ divisions.

The same plan works whenever count(x) is cheap:

- **A matrix with sorted rows and sorted columns.** Count the entries ≤ x by a staircase walk: start at the bottom-left corner; if the entry is ≤ x, the whole column above it counts, so add the row number plus one and move right; otherwise move up. The walk takes $O(n + m)$ steps.
- **Pairwise distances.** For LeetCode 719 (Find K-th Smallest Pair Distance), a bonus problem, the key observation is that after sorting, the pairs with difference ≤ d can be counted with two pointers (Chapter 3) in $O(n)$, so the distance itself can be binary searched.

6.7 Cost, and functions that rise then fall

A search on the answer over V candidate values makes at most $\lceil \log_2 V \rceil$ calls of the check. If one check costs C, the total is $O(C \cdot \log V)$. With an $O(n)$ greedy check this is the familiar **$O(n \log V)$**.

Table 6.3 Iterations of the search for typical answer ranges

Range of the answer	Candidates V	Iterations $\lceil \log_2 V \rceil$	With an $O(n)$ check, $n = 10^5$
speeds, capacities up to 10^9	10^9	30	$3 \cdot 10^6$ steps
up to $2 \cdot 10^9$	$2 \cdot 10^9$	31	$3.1 \cdot 10^6$ steps
sums of 10^5 values ≤ 10^9	10^{14}	47	$4.7 \cdot 10^6$ steps
anything that fits in `long`	10^{18}	60	$6 \cdot 10^6$ steps

The logarithm makes the width of the range almost free. What matters is the cost of the check, so spend your effort there: an $O(n \log n)$ check inside a 60-step search is still fine for $n = 10^5$, but an $O(n^2)$ check is not.

INFOSYS LENS

The plan's description of the paper gives typical limits of $n \le 10^5$ with answers taken mod $10^9 + 7$. At those limits an $O(n \log V)$ search with a linear check finishes in a few million steps, well inside any time limit, which is why "binary search plus a greedy check" is a safe first idea whenever a statement asks for the least or greatest value that makes something possible.

EXAM TIP

Signals that suggest a search on the answer: "minimum possible maximum", "maximum possible minimum", "smallest x such that ...", "at least k ... within d ...", a deadline or budget that the answer

must meet, and an answer range far larger than n. If you can write "if x works, then $x + 1$ works" in one line, you have the predicate.

When binary search fails: unimodal functions

Binary search needs a predicate that flips once. Some problems instead ask for the maximum of a function that rises and then falls.

DEFINITION 6.3 Unimodal function

A function f on the integers $lo..hi$ is **unimodal** if, for some peak p, it is strictly increasing on $lo..p$ and strictly decreasing on $p..hi$.

The values of f are not monotone, but its *slope* is: the predicate "$f(x) \geq f(x+1)$" is false before the peak and true from the peak on. So the peak is the first x where f stops increasing, and `firstTrue` finds it.

Listing 6.10 The peak of a unimodal function, by binary search on the slope

```
// f strictly increases on [lo, p] and strictly decreases on [p, hi]. Returns p.
static long argMaxUnimodal(long lo, long hi, LongUnaryOperator f) {
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;                                  // mid + 1 <= hi
        if (f.applyAsLong(mid) < f.applyAsLong(mid + 1)) lo = mid + 1; // still climbing
        else hi = mid;                                                  // at or past the peak
    }
    return lo;
}
```

The classical alternative is **ternary search**: evaluate f at two points $m_1 < m_2$ a third of the way in from each end and discard the third on the side of the smaller value. It works for real-valued unimodal functions with a fixed number of iterations, keeping two thirds of the range each time, so it needs about $\log_{1.5} V \approx 1.7 \log_2 V$ iterations with two evaluations each. On integers the slope search above is simpler and faster.

PITFALL

Both methods need *strict* monotonicity on each side. A flat stretch before the peak breaks them: on $f = [0, 0, 0, 1]$ the first comparison sees $f(1) = f(2)$, decides it is past the peak, and returns index 0, whose value is 0 instead of 1. If a function can have plateaus, binary search cannot tell "flat because at the top" from "flat on the way up", and you need another argument or a full scan.

Summary

- A half-open binary search keeps "everything left of lo fails, everything from hi on passes". The invariant, the shrinking range and the stopping condition $lo = hi$ prove it returns the first passing index in at most $\lfloor \log_2 n \rfloor + 1$ steps.
- Lower bound is the first index with $a[i] \geq x$; upper bound the first with $a[i] > x$. First position, last position and counts all come from these two. `Arrays.binarySearch` returns an arbitrary match and $-(\text{insertion point}) - 1$ for a missing key.
- Pair the branches with the rounding: `hi = mid` with *mid* rounded down, `lo = mid` with *mid* rounded up. Any other pairing can loop for ever.
- Search on the answer in four steps: define ok(x), prove it monotone, choose $lo \leq$ answer and hi with ok(hi) true, write the check.

- "Minimise the maximum" means the smallest x that passes; "maximise the minimum" means the largest. In both, the check is usually a greedy pass whose optimality follows from a "greedy stays ahead" induction.
- Use `long` bounds, `lo + (hi - lo) / 2`, `long` sums inside the check, and `(a + b - 1) / b` or `Math.floorDiv` for ceilings.
- For real answers, run a fixed number of iterations (100 is plenty) instead of testing `hi - lo > eps`.
- The k-th smallest of a huge collection is the smallest x with count(x) $\geq k$, and that x is always an element.
- The cost is $O(C \log V)$ for a check of cost C. Unimodal functions are handled by searching on the slope, provided they have no plateaus.

Key terms

lower bound p. 74
upper bound p. 74
monotone predicate p. 78
unimodal function p. 89
loop invariant p. 74
Binary search on the answer p. 78
ternary search p. 89

Review questions

1. State the invariant of `lowerBound` in Listing 6.1. When x is larger than every element, what are lo and hi after each iteration, and what is returned?
2. Inside the loop of Listing 6.1, why can mid never equal hi? Which line would fail if it could?
3. What does `Arrays.binarySearch` return on [2, 2, 2, 2] for the keys 2, 3 and 1? How do you recover the lower bound from a negative result?
4. In Listing 6.5, the lower bound starts at the heaviest package. Give an input where starting at 0 returns a wrong answer, and explain which assumption of Lemma 6.5 fails.
5. For the eating-speed problem of Example 6.2, why is the largest pile a valid upper bound, and why must the total of hours be a `long`?
6. In Listing 6.6, show what happens with $lo = 6$ and $hi = 7$ on the array of the trace if the midpoint is rounded down.
7. Why does the search of Listing 6.9 always return an entry of the table, even though count(x) can be $\geq k$ for values x that are not entries?
8. A real-valued search over answers near 10^9 uses `while (hi - lo > 1e-9)`. Explain why it may never stop, and give the fix.
9. Give a function on 0..3 that rises and then falls with a plateau, on which Listing 6.10 returns a wrong index.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 3 "Sorting", section 3.3 on binary search, which presents the method as jumps of decreasing length and applies it to finding the smallest solution of a monotone condition. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Binary Search" (cp-algorithms.com/num_methods/binary_search.html): lower bound, search on predicates and the real-valued version with a fixed number of iterations.
- Java 8 API documentation for `java.util.Arrays` (`binarySearch`) and `java.util.Collections` (`binarySearch`), including the insertion-point rule (docs.oracle.com/javase/8/docs/api/).

- Workbook Chapter 6 has traces of Listing 6.1 and Listing 6.5 to complete by hand, and a bug to find in a search on the answer.

Problems

These are the problems for Day 6. Work them in Workbook Chapter 6, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
6.P1	LC 34 · Find First and Last Position of Element in Sorted Array	MEDIUM	CORE
6.P2	LC 875 · Koko Eating Bananas	MEDIUM	CORE
6.P3	LC 1011 · Capacity To Ship Packages Within D Days	MEDIUM	CORE
6.P4	LC 410 · Split Array Largest Sum	HARD	CORE
6.P5	LC 1552 · Magnetic Force Between Two Balls	MEDIUM	BONUS
6.P6	LC 2226 · Maximum Candies Allocated to K Children	MEDIUM	BONUS
6.P7	LC 719 · Find K-th Smallest Pair Distance	HARD	BONUS

CHAPTER 7 · DAY 7 OF 29 · FRIDAY 13 NOVEMBER 2026 · LEARNING DAY

7

Monotonic Stacks

Next or previous greater/smaller element for every index in O(n), and the contribution trick.

Part I · Foundations
Handbook cards · Monotonic stack
Before this chapter · Chapter 1 (ArrayDeque), Chapter 3 (the monotonic deque)
Practice · Workbook Chapter 7 · 7 problems · core ≈ 4.1 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- find the next and previous greater or smaller element of every index in O(n) with a stack, and prove the bound by counting pushes and pops;
- handle circular arrays by scanning the array twice;
- find the largest rectangle in a histogram by treating each bar as the lowest bar of its rectangle;
- compute the sum of the minimums or maximums of all subarrays with the contribution technique, breaking ties so that no subarray is counted twice;
- build the lexicographically smallest result of k deletions with a greedy stack, and justify it with an exchange argument.

A large family of array questions has the same core: for every position, find the nearest position to its right (or left) whose value is larger (or smaller). "How many days until a warmer day?" "How far can this bar extend before a lower bar stops it?" "Where does this element stop being the minimum?" A double loop answers each in $O(n^2)$. A stack that keeps its values in sorted order answers all of them in $O(n)$, and the proof that it does so is one of the cleanest examples of amortised analysis.

The chapter develops the stack once, carefully, and then applies it four ways: to next-greater queries, to the largest rectangle in a histogram, to sums over all subarrays (the contribution technique, which is where most of the subtle bugs live), and to greedy problems that build the smallest possible string. It ends by connecting the stack to the monotonic deque of Chapter 3 and to a tree that the stack builds without trying.

7.1 Next greater element

DEFINITION 7.1 **Next and previous greater or smaller element**

For an array $a[0..n-1]$ and an index i, the **next greater element** of i is the smallest $j > i$ with $a[j] > a[i]$, or "none" (written -1) if no such j exists. The previous greater element is the largest $j < i$ with $a[j] > a[i]$. Next and previous *smaller* elements are defined the same way with $<$. Each has a non-strict version with $\geq$ or $\leq$.

The brute force scans right from each i until it finds a larger value: $O(n^2)$ in the worst case, for example on a decreasing array. The fast method rests on one observation about which indices are still "waiting" for their answer.

Scan from left to right. After reading $a[0..i-1]$, call an index $j < i$ **waiting** if no index between j and i holds a value greater than $a[j]$: its next greater element, if any, is at i or later. Take two waiting indices $j < k$. Since k lies between j and i and j is waiting, $a[k] \le a[j]$. So the values of the waiting indices, read from left to right, never increase. When $a[i]$ arrives, the waiting indices it answers are those with $a[j] < a[i]$, and because the values are sorted, these are exactly the most recent waiting indices. A stack holds the waiting indices with the most recent on top.

DEFINITION 7.2 Monotonic stack

A **monotonic stack** is a stack of array indices whose values are sorted from bottom to top (non-increasing for next-greater queries, non-decreasing for next-smaller queries). Before an index is pushed, every index on top that would break the order is popped, and each pop is the moment that index's answer becomes known.

Listing 7.1 Next strictly greater element of every index (handbook template)

```java
// Index of the next strictly greater element to the right (-1 if none). O(n).
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    int[] st = new int[n];          // indices whose answer is still unknown; values decrease
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (top > 0 && a[st[top - 1]] < a[i]) res[st[--top]] = i;   // a[i] answers them
        st[top++] = i;
    }
    return res;
}
```

The stack is an `int[]` with a `top` counter: `st[top - 1]` is the top, `st[--top]` pops it, `st[top++] = i` pushes. This is faster than `ArrayDeque<Integer>`, which boxes every index, and much faster than `java.util.Stack`, which synchronises every call.

EXAMPLE 7.1 Tracing the stack

For $a = [2, 1, 2, 4, 3]$:

```
i  a[i]  popped (answer = i)  stack after (indices)
0    2   -                    [0]
1    1   -                    [0, 1]
2    2   1                    [0, 2]
3    4   2, 0                 [3]
4    3   -                    [3, 4]
result: [3, 2, 3, -1, -1]
```

At $i = 2$ the value 2 pops index 1 (value 1) but not index 0 (value 2), because the test is strictly greater. At $i = 3$ the 4 answers both remaining indices. Indices 3 and 4 are still waiting at the end, so their answer stays -1.

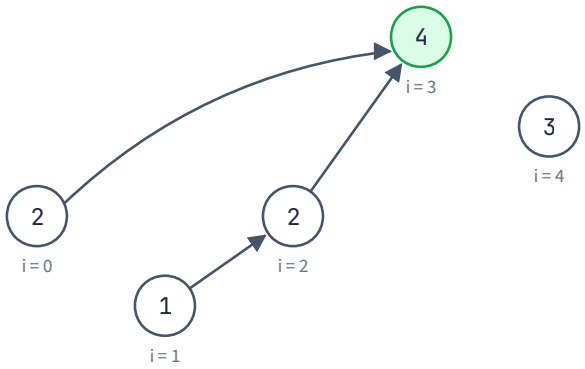


Figure 7.1 Next greater elements of [2, 1, 2, 4, 3]. Each node is an index drawn at the height of its value; an arrow leads to the index's next greater element. Indices 3 and 4 have none.

THEOREM 7.1 Correctness of the next-greater stack

After the iteration for index i of Listing 7.1: (a) the stack holds, in increasing order of index from bottom to top, exactly the waiting indices $j \le i$, those with no value greater than $a[j]$ in positions $j+1..i$; (b) their values are non-increasing from bottom to top; (c) every index not on the stack has `res` *equal to its next greater element. Consequently the method returns the next greater element of every index.*

Proof. By induction on i. Before the loop the stack is empty and nothing has been decided, so (a)–(c) hold. Assume they hold after index $i-1$ and read $a[i]$.

The indices whose next greater element is i are the waiting indices $j < i$ with $a[j] < a[i]$: such a j has no greater value before i, by (a), and $a[i]$ is greater. By (b) these indices form a block at the top of the stack, so the `while` loop pops exactly them and records i for each, which keeps (c). Every index that stays has $a[j] \ge a[i]$, so it is still waiting after i, and pushing i (which is trivially waiting) keeps (a). Since every remaining value is $\ge a[i]$, the new top i keeps the order (b). After the last index, the indices still on the stack have no greater value anywhere to their right, and their `res` is the initial -1, which is correct. ∎

Why the stack costs O(n)

The `while` loop can pop many indices in one iteration. On [10, 9, 8, 7, 6, 5, 4, 3, 2, 100] the final 100 pops nine indices at once. A bound of "n iterations × up to n pops" would suggest $O(n^2)$. The true cost is much smaller, and the argument is worth knowing by name.

DEFINITION 7.3 Amortised analysis

Amortised analysis bounds the *total* cost of a sequence of operations, even when single operations can be expensive. The average cost per operation, the total divided by the number of operations, is the amortised cost.

LEMMA 7.2 Each index is pushed once and popped at most once

Listing 7.1 performs exactly n pushes and at most n pops, so its running time is $O(n)$ and it uses $O(n)$ extra memory.

Proof. Each iteration of the `for` loop pushes one index, so there are n pushes. A pop removes an index that was pushed earlier and is never pushed again, so the number of pops cannot exceed the number of

pushes. Every iteration of the `while` loop performs one pop, so the `while` loop runs at most n times in total over the whole scan. Everything else is O(1) per index. ∎

In the example above, the nine pops at the end are paid for by the nine pushes before them: the scan as a whole does 10 pushes and 9 pops.

The four variants

Changing the comparison and the moment you read the answer gives every nearest-element query. Indices are popped when they can no longer be the answer for anyone; what remains under the new index is its answer on the left, and the index that pops an element is that element's answer on the right.

Table 7.1 One scan from left to right answers each query

Query	Pop while	Answer is recorded	Stack values, bottom to top
next greater (>)	`a[top] < a[i]`	for each popped index: *i*	non-increasing
next smaller (<)	`a[top] > a[i]`	for each popped index: *i*	non-decreasing
previous greater (>)	`a[top] <= a[i]`	for *i*: the top after popping	strictly decreasing
previous smaller (<)	`a[top] >= a[i]`	for *i*: the top after popping	strictly increasing

Swapping `<` and `<=` switches between the strict and non-strict versions: popping on `a[top] <= a[i]` gives "next greater or equal" to the popped indices. Scanning from right to left swaps "next" and "previous".

Listing 7.2 Previous strictly smaller element of every index

```java
// Index of the previous strictly smaller element to the left (-1 if none). O(n).
static int[] prevSmaller(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    int[] st = new int[n];          // values strictly increase from bottom to top
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (top > 0 && a[st[top - 1]] >= a[i]) top--;    // can never answer i or anyone later
        res[i] = top == 0 ? -1 : st[top - 1];
        st[top++] = i;
    }
    return res;
}
```

Why may the popped indices be thrown away? An index j with $a[j] \geq a[i]$ lies left of i, so for any later index k, i is closer to k than j and at least as small. If j would answer k ($a[j] < a[k]$), so does i, and i is nearer. So j can never again be the *previous* smaller element of anyone.

> **EXAM TIP**
>
> One scan can give two answers at once. In Listing 7.2, the index i that pops j is j's next smaller-or-equal element, because j was popped exactly when a value $\leq a[j]$ arrived. Section 7.4 needs precisely this pair: previous strictly smaller on one side, next smaller-or-equal on the other.

Circular arrays

In a circular array the element after $a[n-1]$ is $a[0]$, and LeetCode 503 (Next Greater Element II) asks for the next greater *value* in that circular order. The fix is to scan the array twice: let i run from 0 to $2n-1$ and read $a[i \bmod n]$.

CLAIM 7.3 Two laps are enough

If index j has a next greater element in circular order, it lies within the next $n - 1$ positions after j, all of which are visited after j is pushed during the first lap.

Proof. The circular successors of j are $j + 1, \ldots, j + n - 1$ (taken mod n), which are the scan positions $j + 1..j + n - 1 \le 2n - 2$. Index j is pushed at scan position $j < n$, and the argument of Theorem 7.1 applies to the doubled sequence, so j is popped by the first of those positions with a greater value. ∎

Push only during the first lap ($i < n$); in the second lap the indices only answer waiting ones. For $a = [5, 3, 4, 1, 2]$ the circular next greater values are $[-1, 4, 5, 2, 5]$: the 4 answers the 2 only by wrapping round, and 5, the maximum, has no answer. Turning Listing 7.1 into this version takes three small edits, which are left for the problem itself.

7.2 Worked example: daily temperatures

LeetCode 739 (Daily Temperatures) gives daily temperatures and asks, for each day, how many days you must wait for a warmer one (0 if none). That is the next strictly greater element, reported as a distance: answer[j] = nge(j) − j, or 0. The only change to Listing 7.1 is what is stored when j is popped by i.

EXAMPLE 7.2 Waiting for a warmer day

Take the temperatures [30, 38, 31, 33, 35, 30, 40, 36]. The stack is shown as index:temperature.

```
i  t[i]  popped (wait = i - j)      stack after
0   30   -                         [0:30]
1   38   0 (1)                     [1:38]
2   31   -                         [1:38, 2:31]
3   33   2 (1)                     [1:38, 3:33]
4   35   3 (1)                     [1:38, 4:35]
5   30   -                         [1:38, 4:35, 5:30]
6   40   5 (1), 4 (2), 1 (5)       [6:40]
7   36   -                         [6:40, 7:36]
answer: [1, 5, 1, 1, 2, 1, 0, 0]
```

Day 1 (38°) waits five days, through the smaller peaks at days 3 and 4, until the 40° of day 6. The scan makes 8 pushes and 6 pops. The two days left on the stack at the end never see a warmer day, so their answer stays 0.

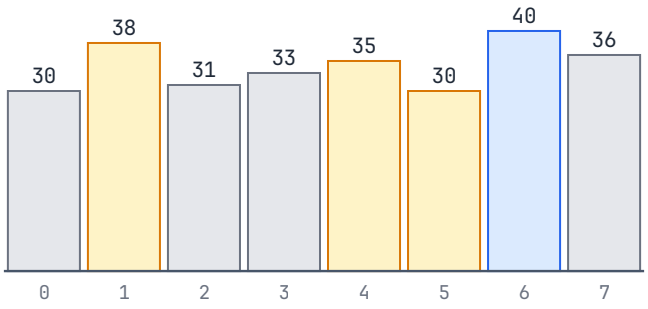


Figure 7.2 Day 6 arrives with 40°. The waiting days 1, 4 and 5 (amber) form a decreasing staircase on the stack; 40 is warmer than all three, so it pops and answers each of them.

Notice what the staircase shape means. Day 2 (31°) and day 3 (33°) left the stack as soon as a warmer day came, and nothing to their right could ever need them again. The stack only remembers days that are warmer than every later day so far; those are the only possible answers for the future.

7.3 Largest rectangle in a histogram

A histogram is a row of bars of width 1 and heights $h[0..n-1]$. LeetCode 84 (Largest Rectangle in Histogram) asks for the largest area of an axis-parallel rectangle that fits inside the bars. A rectangle over the bars $l..r$ can be as tall as the lowest bar in that range, so the task is

$$\max \text{ over } 0 \le l \le r < n \text{ of } (r - l + 1) \cdot \min(h[l..r]).$$

There are $n(n+1)/2$ ranges; carrying a running minimum while extending r gives an $O(n^2)$ brute force. The stack does better by changing the question: instead of asking "how tall can this range be?", ask "if bar i is the lowest bar of the rectangle, how wide can the rectangle be?".

THEOREM 7.4 Each bar as the lowest bar

Let L_i be the index of the previous bar strictly lower than $h[i]$ (-1 if none) and R_i the index of the next bar strictly lower than $h[i]$ (n if none). Then the largest rectangle has area

$$\max \text{ over } i \text{ of } h[i] \cdot (R_i - L_i - 1).$$

Proof. *Every candidate is a real rectangle.* All bars strictly between L_i and R_i are at least $h[i]$ tall, by the definition of L_i and R_i, so a rectangle of height $h[i]$ over the $R_i - L_i - 1$ bars $L_i + 1..R_i - 1$ fits. So the maximum of the candidates is at most the optimum.

The optimum is among the candidates. Take an optimal rectangle over bars $l..r$, with height $H = \min(h[l..r])$ (a lower height would only lose area). Choose i in $l..r$ with $h[i] = H$. No bar in $l..r$ is lower than H, so the previous strictly lower bar is left of l and the next strictly lower bar is right of r: $L_i < l$ and $R_i > r$. Hence the candidate for i has width $R_i - L_i - 1 \ge r - l + 1$ and height H, so its area is at least the optimum. ∎

Both boundary arrays come from Table 7.1, one scan from each side.

Listing 7.3 Largest rectangle in a histogram, two passes

```java
static long largestRectangle(int[] h) {
    int n = h.length;
    int[] left = new int[n], right = new int[n];   // previous / next strictly lower bar
    int[] st = new int[n];
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (top > 0 && h[st[top - 1]] >= h[i]) top--;
        left[i] = top == 0 ? -1 : st[top - 1];
        st[top++] = i;
    }
    top = 0;
    for (int i = n - 1; i >= 0; i--) {
        while (top > 0 && h[st[top - 1]] >= h[i]) top--;
        right[i] = top == 0 ? n : st[top - 1];
        st[top++] = i;
    }
    long best = 0;
    for (int i = 0; i < n; i++) best = Math.max(best, (long) h[i] * (right[i] - left[i] - 1));
    return best;
}
```

EXAMPLE 7.3 Every bar's rectangle

For $h = [3, 1, 4, 5, 4, 2, 3]$:

```
i  h[i]  L(i)  R(i)  width  area
0   3    -1     1     1      3
1   1    -1     7     7      7
2   4     1     5     3     12
3   5     2     4     1      5
4   4     1     5     3     12
5   2     1     7     5     10
6   3     5     7     1      3
largest area: 12
```

Bars 2 and 4 have the same height and give the same rectangle, over bars 2..4. Bar 5 (height 2) reaches from index 2 to 6 for an area of 10, and the lowest bar reaches across everything for 7.

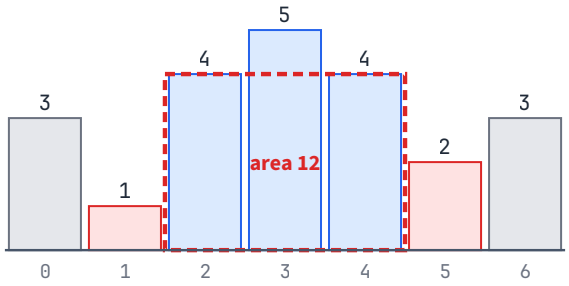


Figure 7.3 The histogram [3, 1, 4, 5, 4, 2, 3]. With bar 2 as the lowest bar, the rectangle extends right until bar 5, which is lower, and left until bar 1: width 3, height 4, area 12.

One pass

The usual exam version computes the areas while popping, in a single scan with a sentinel bar of height 0 after the end that empties the stack.

Listing 7.4 Largest rectangle in one pass: a bar's rectangle is settled when it is popped

```java
static long largestRectangleOnePass(int[] h) {
    int n = h.length;
    int[] st = new int[n + 1];
    int top = 0;
    long best = 0;
    for (int i = 0; i <= n; i++) {
        int cur = i == n ? 0 : h[i];              // height 0 after the end pops everything
        while (top > 0 && h[st[top - 1]] >= cur) {
            int j = st[--top];                    // h[j] is the lowest bar of left+1 .. i-1
            int left = top == 0 ? -1 : st[top - 1];
            best = Math.max(best, (long) h[j] * (i - left - 1));
        }
        st[top++] = i;
    }
    return best;
}
```

When j is popped by i, the bar under it on the stack is the previous strictly lower bar (the stack is strictly increasing), and i is the next bar with height $\le h[j]$. For a bar with an equal neighbour to its right, the width is therefore too short. The trace for the same histogram shows this at $i = 4$, where bar 2 is settled with width 2, while bar 4, of the same height, is settled one step later with the full width 3.

```
i  cur  events                                        stack after
0   3                                                 [0]
1   1   pop 0: h 3, width 1, area 3                   [1]
2   4                                                 [1, 2]
3   5                                                 [1, 2, 3]
4   4   pop 3: h 5, width 1, area 5
        pop 2: h 4, width 2, area 8                   [1, 4]
5   2   pop 4: h 4, width 3, area 12                  [1, 5]
6   3                                                 [1, 5, 6]
7   0   pop 6: area 3; pop 5: area 10; pop 1: area 7
best = 12
```

CLAIM 7.5 Short widths for ties do not matter

Listing 7.4 returns the largest area.

Proof. Every computed area is a real rectangle, since the bars strictly between `left` and i are all $\geq h[j]$. For the optimum over $l..r$ with height H, let i^* be the **rightmost** bar in $l..r$ with height H. Every bar in $i^*+1..r$ is strictly higher than H, so the bar that pops i^* (the next bar with height $\leq H$) is right of r, and the bar below it on the stack (the previous strictly lower bar) is left of l. So the width computed for i^* is at least $r - l + 1$. ∎

JAVA NOTE

Areas can exceed `int`: 10^5 bars of height 10^5 give 10^{10}. Write `(long) h[j] * width`, with the cast on the first factor, so the multiplication itself is done in `long`. `(long) (h[j] * width)` casts after the overflow has already happened.

The same idea solves "largest rectangle of 1s in a 0/1 grid": for each row, let $h[c]$ be the number of consecutive 1s ending at that row in column c, and run the histogram algorithm on every row, in O(rows × columns) total.

7.4 The contribution technique

LeetCode 907 (Sum of Subarray Minimums) asks for the sum, over all $n(n+1)/2$ subarrays, of the minimum of the subarray, modulo $10^9 + 7$. The brute force extends each subarray to the right while carrying the minimum, $O(n^2)$. The fast method turns the sum around.

DEFINITION 7.4 Contribution technique

The **contribution technique** computes a sum over many objects (here, subarrays) by asking, for each element, how many objects it is responsible for. If every object has exactly one responsible element and element i is responsible for c_i objects, each worth $a[i]$, the sum is $\Sigma\, a[i] \cdot c_i$.

For subarray minimums, the responsible element of a subarray is its minimum. The difficulty is **ties**: in [2, 1, 3, 1, 2] the subarray [1, 3, 1] has two minimum positions, and it must be counted once. The fix is a rule that picks exactly one of them, here the rightmost.

THEOREM 7.6 Counting the subarrays each element owns

*Let left[i] be the previous index with a value strictly smaller than a[i] (−1 if none), and right[i] the next index with a value smaller than **or equal to** a[i] (n if none). Then a[i] is the rightmost minimum of exactly (i − left[i]) · (right[i] − i) subarrays, every subarray is counted for exactly one i, and*

> *Σ over subarrays of min = Σ over i of a[i] · (i − left[i]) · (right[i] − i).*

Proof. Call position i the owner of the subarray $a[l..r]$ if $a[i]$ is its minimum and no position to the right of i inside the subarray holds the same value. Each subarray has exactly one owner, its rightmost minimum.

Fix i. A subarray $a[l..r]$ with $l \le i \le r$ is owned by i exactly when every value in $a[l..i-1]$ is $\ge a[i]$ and every value in $a[i+1..r]$ is $> a[i]$. The first condition says no index in $l..i-1$ is strictly smaller, that is, $l > \text{left}[i]$; the second says no index in $i+1..r$ is smaller or equal, that is, $r < \text{right}[i]$. So l ranges over $\text{left}[i]+1..i$ (that is $i - \text{left}[i]$ choices) and r independently over $i..\text{right}[i]-1$ ($\text{right}[i] - i$ choices), giving the product. Summing $a[i]$ times this count over all i adds each subarray's minimum exactly once, by its owner. ∎

The handbook template computes both boundary arrays with the comparisons of the theorem: the left scan pops on `>=`, leaving the previous strictly smaller index; the right-to-left scan pops on `>`, leaving the next smaller-or-equal index.

Listing 7.5 Sum of the minimums of all subarrays (handbook template)

```java
// Sum of min(subarray) over all subarrays: a[i] is the minimum of (i - left) * (right - i) of them
static long sumSubarrayMins(int[] a) {
    int n = a.length;
    int[] left = new int[n], right = new int[n];
    ArrayDeque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {            // left[i]: previous index with a smaller value
        while (!st.isEmpty() && a[st.peek()] >= a[i]) st.pop();
        left[i] = st.isEmpty() ? -1 : st.peek();
        st.push(i);
    }
    st.clear();
    for (int i = n - 1; i >= 0; i--) {       // right[i]: next index with a smaller-or-equal value
        while (!st.isEmpty() && a[st.peek()] > a[i]) st.pop();
        right[i] = st.isEmpty() ? n : st.peek();
        st.push(i);
    }
    long total = 0;
    for (int i = 0; i < n; i++) total += (long) a[i] * (i - left[i]) * (right[i] - i);
    return total;                            // LeetCode 907 wants this mod 1e9+7
}
```

EXAMPLE 7.4 **Contributions in [2, 1, 3, 1, 2]**

i	a[i]	left	right	i - left	right - i	count	a[i] * count
0	2	-1	1	1	1	1	2
1	1	-1	3	2	2	4	4
2	3	1	3	1	1	1	3
3	1	-1	5	4	2	8	8
4	2	3	5	1	1	1	2
					total:	15	19

The counts add up to $15 = 5 \cdot 6/2$, the number of subarrays, which is a quick check that no subarray was lost or counted twice. Index 1 owns only four subarrays because right[1] = 3: any subarray that also contains index 3 has its rightmost minimum there. A brute force over all 15 subarrays also gives 19.

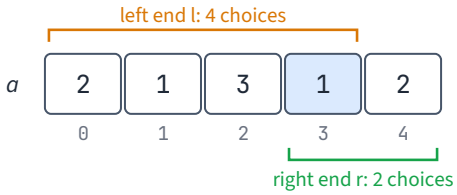


Figure 7.4 The subarrays owned by index 3 of [2, 1, 3, 1, 2]: the left end can be any of indices 0..3 (nothing to the left is strictly smaller) and the right end either 3 or 4 (nothing up to the end is smaller or equal). That is 4 × 2 = 8 subarrays.

Why the tie-breaking matters

Use strict comparisons on both sides, and a subarray with two equal minima is owned by both; use non-strict on both, and it is owned by neither. On [2, 2] the true sum is 2 + 2 + 2 = 6. Strict on both sides gives 8 (the subarray [2, 2] is counted twice); non-strict on both gives 4 (it is not counted at all). The template's rule (strict on the left, non-strict on the right) gives 6. The opposite rule (non-strict on the left, strict on the right) is equally correct; it makes the *leftmost* minimum the owner.

PITFALL

Getting one side wrong passes every test with distinct values and fails as soon as values repeat. Small hand-made examples usually have distinct values, so a wrong tie rule survives them. Always test the contribution formula on an array with ties, such as [2, 2] or [3, 1, 3, 1], against a brute force.

The modulus

LeetCode 907 keeps n and the values at $3 \cdot 10^4$ or less, so the exact total is at most about $4.5 \cdot 10^8$ subarrays $\times 3 \cdot 10^4 \approx 1.4 \cdot 10^{13}$ and fits in `long`; taking the remainder at the end is enough. With $n = 10^5$ and values up to 10^9, one count $(i - \text{left})(\text{right} - i)$ can reach $(n/2)^2 = 2.5 \cdot 10^9$ and the total can approach 10^{19}, beyond `long`. Then reduce every term:

Listing 7.6 The contribution sum modulo 1e9+7, safe for large n and values

```
static final long MOD = 1_000_000_007L;

static long sumOfMinsMod(int[] a, int[] left, int[] right) {    // left, right as in sumSubarrayMins
    long total = 0;
    for (int i = 0; i < a.length; i++) {
        long count = (long) (i - left[i]) * (right[i] - i) % MOD;   // < 2.5e9 before the mod
        total = (total + a[i] % MOD * count) % MOD;                 // both factors < MOD: < 1.1e18
    }
    return total;
}
```

Both factors of the product are below $10^9 + 7$ after the reduction, so the product stays below about $1.0 \cdot 10^{18} < 9.2 \cdot 10^{18}$, the largest `long`.

INFOSYS LENS

The plan's decoding of the official sample paper reports typical limits of $n \le 10^5$ with answers taken mod $10^9 + 7$. Sums over all subarrays or all pairs are the classic reason for that modulus, so reduce each term as above rather than hoping the total fits.

Maxima, ranges and other contributions

For the sum of subarray **maximums**, reverse every comparison: left[i] is the previous strictly *greater* index and right[i] the next greater-or-equal index. The proof is the same with "maximum" in place of "minimum".

The **range** of a subarray is its maximum minus its minimum. Because a sum of differences is the difference of the sums,

Σ over subarrays of (max − min) = Σ over subarrays of max − Σ over subarrays of min,

so the sum of all ranges (LeetCode 2104, Sum of Subarray Ranges, a bonus problem today) is two contribution sums, one for maxima and one for minima. For [2, 1, 3, 1, 2] the maxima sum to 37, the minima to 19, and the ranges to 18.

The counts are useful on their own. The number of subarrays whose minimum is exactly v is the sum of the counts of the indices holding v; the number of subarrays in which $a[i]$ is the minimum is its count. Any question of the form "sum over all subarrays of a function of the minimum" is answered the same way.

7.5 Greedy stacks for the smallest result

The last application looks different but uses the same stack. Delete exactly k characters of a string s so that the remaining string, read in order, is as small as possible in dictionary order.

DEFINITION 7.5 Lexicographic order

For two strings of the same length, x is lexicographically smaller than y if, at the first position where they differ, the character of x is smaller. This is **lexicographic order**, the order of a dictionary. For two digit strings of the same length it is also the numeric order.

All results have length $n - k$, so only the first difference matters. That makes the early positions far more important than the late ones, and suggests deleting a large character as early as possible. The right place to delete turns out to be the first **descent**: the first i with $s[i] > s[i+1]$.

LEMMA 7.7 The best single deletion

Let i be the first index with $s[i] > s[i+1]$, or $i = n - 1$ if s is non-decreasing. Deleting $s[i]$ gives the lexicographically smallest of the n strings obtained by deleting one character.

Proof. Write D_j for s with position j deleted. $D_j[p] = s[p]$ for $p < j$ and $s[p+1]$ for $p \geq j$.

Deleting earlier, $j < i$. For $p < j$ the two strings agree. For $j \leq p < i$, $D_j[p] = s[p+1] \geq s[p] = D_i[p]$, because $s[0..i]$ is non-decreasing. From $p = i$ on they agree again. So $D_i \leq D_j$.

Deleting later, $j > i$. The strings agree before position i, and at position i, $D_i[i] = s[i+1] < s[i] = D_j[i]$. So $D_i < D_j$. (If s is non-decreasing, $i = n - 1$ and only the first case arises.) ∎

One good deletion is not yet k good deletions. The next result says that the greedy choice is never a mistake.

THEOREM 7.8 Greedy deletion is optimal

For $1 \leq k \leq n$, deleting k characters one at a time, each time at the first descent of the current string (or its last character if there is none), gives the lexicographically smallest string obtainable by deleting k characters.

Proof. It is enough to show that some optimal set of k deleted positions contains the position i of Lemma 7.7; then the rest is an optimal deletion of $k-1$ characters from D_i, and induction on k finishes the proof. Take an optimal set S of k positions with $i \notin S$.

Case 1: S contains a position $j < i$. Replace j by i. The two results differ only in the characters kept from positions $j..i$. Let the kept positions in that block be $k_1 < \cdots < k_t = i$ for S; for the new set they are j, k_1, ..., k_{t-1}, each shifted one place to the left in the list. Since $s[j..i]$ is non-decreasing, each character of the new list is $\le$ the character in the same place of the old list. So the new result is no larger.

Case 2: every position of S is after i. Let $j = \min S$ and replace it by i. The results agree before position i. At position i, the old result has $s[i]$ and the new one has $s[i+1]$ (position $i+1$ is kept in the new set, since the only change was to delete i instead of j), and $s[i+1] < s[i]$. So the new result is smaller, contradicting optimality; this case cannot happen.

In both cases there is an optimal set containing i. ∎

Doing k deletions one by one and rescanning from the start each time costs $O(nk)$. A stack removes the rescanning. After deleting position i, the prefix $s[0..i-1]$ is still non-decreasing, so the next descent is at $i-1$ (if $s[i-1]$ is larger than the character that moved next to it) or later, never earlier. Keep the non-decreasing prefix on a stack; when the next character is smaller than the top, the top is the first descent, so pop it and spend one deletion.

Listing 7.7 The smallest string after deleting exactly k characters

```
static String smallestAfterDeleting(String s, int k) {      // 0 <= k <= s.length()
    StringBuilder st = new StringBuilder();                  // the stack: kept characters, non-decreasing
    for (char c : s.toCharArray()) {
        while (k > 0 && st.length() > 0 && st.charAt(st.length() - 1) > c) {
            st.setLength(st.length() - 1);                   // the top is the first descent: delete it
            k--;
        }
        st.append(c);
    }
    st.setLength(st.length() - k);                           // non-decreasing now: delete from the end
    return st.toString();
}
```

Each character is appended once and removed at most once, so the method runs in $O(n)$. When the scan ends with deletions left over, the stack is non-decreasing, its first descent is "none", and Lemma 7.7 says to delete the last character, k times.

EXAMPLE 7.5 **Deleting three letters of "becadfa"**

```
char  popped  stack   k left
b     -       b       3
e     -       be      3
c     e       bc      2
a     c, b    a       0
d     -       ad      0
f     -       adf     0
a     -       adfa    0
result: adfa
```

A brute force over all 35 ways to keep four of the seven letters agrees. Note the last letter: the `a` after `f` would like to pop `f`, but the deletions are used up.

Digits, and letters used once

For digits the same greedy gives the smallest number: equal-length digit strings compare numerically exactly as they compare lexicographically. On “52814” with two deletions, the first descent is the 5 (giving “2814”) and then the 8 (giving “214”). LeetCode 402 (Remove K Digits), a bonus problem today, adds two details that are part of the problem rather than the idea: leading zeros of the result must be removed, and an empty result is written “0”.

LeetCode 316 (Remove Duplicate Letters), the other bonus stack problem, keeps each distinct letter exactly once instead of deleting a fixed number. The stack changes in two ways: a letter already on the stack is skipped, and the top may be popped only if it is larger than the current letter **and** occurs again later in the string, so that it can be taken then. An array of last positions answers “occurs again later” in O(1).

7.6 Relatives: the monotonic deque and the Cartesian tree

The monotonic deque

Chapter 3 found the maximum of every window of length k with a deque of indices whose values decrease from front to back. Its back end behaves exactly like the next-greater stack: a new value pops every smaller value, because those can never again be a window maximum. The only addition is at the front: when the index there falls out of the window, it is removed. Each index still enters once and leaves once, so the cost is O(n) by the argument of Lemma 7.2. A monotonic deque is a monotonic stack that also forgets old indices.

A different structure with a similar name, the *minimum stack*, stores with each pushed value the minimum of the stack so far, so that the current minimum is always on top. Two such stacks make a queue with O(1) amortised minimum. It answers “minimum of everything currently stored”, not “nearest smaller element”.

The Cartesian tree

The stack also builds a tree, almost for free.

DEFINITION 7.6 **Cartesian tree**

The **Cartesian tree** of an array with distinct values is the binary tree whose root is the position of the minimum, whose left subtree is the Cartesian tree of the part to the left of the minimum, and whose right subtree is the Cartesian tree of the part to its right.

Read in order (left subtree, node, right subtree), the tree lists the indices $0..n-1$; and every node’s value is smaller than its children’s. The subtree of node i covers exactly the indices strictly between its previous smaller and next smaller elements, the same range as in Theorem 7.4. So the largest rectangle in a histogram is the maximum over the nodes of value × subtree size.

Listing 7.8 Parents in the Cartesian tree (minimum at the root), built with a stack

```
static int[] cartesianParent(int[] a) {          // distinct values; parent -1 for the root
    int n = a.length;
    int[] par = new int[n];
    Arrays.fill(par, -1);
    int[] st = new int[n];                       // the right spine of the tree built so far
    int top = 0;
    for (int i = 0; i < n; i++) {
        int last = -1;
        while (top > 0 && a[st[top - 1]] > a[i]) last = st[--top];
        if (last != -1) par[last] = i;           // the popped chain hangs below i, on its left
```

```
        if (top > 0) par[i] = st[top - 1];       // i becomes the right child of the new top
        st[top++] = i;
    }
    return par;
}
```

For [5, 2, 6, 1, 4, 3] the parents are [1, 3, 1, −1, 5, 3]: the 1 at index 3 is the root, the 2 at index 1 heads the left part [5, 2, 6], and the 3 at index 5 heads the right part [4, 3].

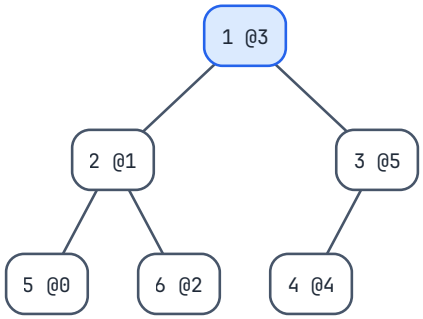


Figure 7.5 The Cartesian tree of [5, 2, 6, 1, 4, 3]. Each node shows its value and index. The root is the minimum; reading the tree from left to right gives back the array.

Two facts make the tree worth knowing later. The minimum of any range $a[l..r]$ is the value at the lowest common ancestor of l and r, which links range-minimum queries to the tree algorithms of Chapter 18. And the stack in Listing 7.8 is the right spine of the tree: each new value climbs the spine while it is smaller, which is the next-smaller scan of Table 7.1 with the pops recorded as edges.

Summary

- A monotonic stack keeps the indices still waiting for an answer, with sorted values. A new value pops every index it answers; those are always on top.
- Each index is pushed once and popped at most once, so a scan costs $O(n)$ in total, even though one step can pop many indices. This is amortised analysis.
- The comparison decides the query: pop on `<` for next greater, on `>` for next smaller; read the popped indices for "next", the top after popping for "previous". `<` versus `<=` decides strict versus non-strict.
- Circular arrays: scan $2n$ positions with `i % n` and push only in the first lap.
- Largest rectangle: every bar is tried as the lowest bar, spreading to its previous and next strictly lower bars; the optimum is among these n candidates.
- Contribution: Σ min over all subarrays $= \Sigma\, a[i] \cdot (i - \text{left}[i]) \cdot (\text{right}[i] - i)$, with strict on one side and non-strict on the other so that each subarray has exactly one owner. Test with ties; use `long` and reduce mod $10^9 + 7$ per term for large inputs.
- Sums of maxima reverse the comparisons; sums of ranges are the sum of maxima minus the sum of minima.
- The smallest string after k deletions deletes at the first descent each time; a stack does all k deletions in $O(n)$, and leftover deletions come off the end.
- Use `int[]` with a top pointer or `ArrayDeque`, never `Stack`.

Key terms

next greater element p. 92

monotonic stack p. 93

Amortised analysis p. 94
contribution technique p. 99
lexicographic order p. 102
Cartesian tree p. 104

Review questions

1. In Listing 7.1, why are the values on the stack non-increasing from bottom to top? Which step of the proof of Theorem 7.1 would fail without this?
2. Give an array of length 6 on which one iteration of the `while` loop in Listing 7.1 pops five indices. How many pops does the whole scan make, and why is the method still O(n)?
3. Which pop condition and which reading of the stack give the previous greater-or-equal element of every index?
4. For a circular array, why does scanning $2n$ positions suffice? Does anything go wrong if indices are also pushed during the second lap?
5. Why is it enough, in the largest-rectangle problem, to consider rectangles whose height equals the height of one of the bars inside them?
6. Compute the sum of subarray minimums of [3, 3, 3] with the template's tie rule, and then with strict comparisons on both sides. Explain the difference.
7. Which comparisons do you use for left and right when summing subarray maximums? Write the sum of subarray ranges in terms of two contribution sums.
8. In Listing 7.7, why are the remaining k deletions taken from the end of the stack after the scan?
9. In the Cartesian tree of an array with distinct values, which indices does the subtree of node i contain? Use the answer to explain the link with Theorem 7.4.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 8 "Amortized analysis", the sections on nearest smaller elements and on the sliding window minimum, which give the same push-and-pop counting argument. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Minimum Stack / Minimum Queue" (cp-algorithms.com/data_structures/stack_queue_modification.html), for the related structure that keeps the minimum of a stack or queue.
- Java 8 API documentation for `java.util.ArrayDeque` (`push`, `pop`, `peek`), docs.oracle.com/javase/8/docs/api/.
- Workbook Chapter 7 has stack traces and a contribution table to complete by hand.

Problems

These are the problems for Day 7. Work them in Workbook Chapter 7, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
7.P1	LC 739 · Daily Temperatures	MEDIUM	CORE
7.P2	LC 503 · Next Greater Element II	MEDIUM	CORE
7.P3	LC 84 · Largest Rectangle in Histogram	HARD	CORE
7.P4	LC 907 · Sum of Subarray Minimums	MEDIUM	CORE
7.P5	LC 402 · Remove K Digits	MEDIUM	BONUS
7.P6	LC 316 · Remove Duplicate Letters	MEDIUM	BONUS

No.	Problem	Level	Set
7.P7	LC 2104 · Sum of Subarray Ranges	MEDIUM	BONUS

CHAPTER 8 · DAY 8 OF 29 · SATURDAY 14 NOVEMBER 2026 · LEARNING DAY

8

Bits, XOR, Tries and Linear Bases

Each bit is independent in a sum of XORs. A trie answers pair questions; a basis answers subset questions.

Part I · Foundations
Handbook cards · Bit tricks and XOR, Binary trie for XOR, XOR basis (maximum subset XOR)
Before this chapter · Chapter 2 (prefix sums and the prefix-count scan)
Practice · Workbook Chapter 8 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- read and write Java's bit operators correctly, including two's complement, the three shifts and the precedence traps;
- use the algebra of XOR to cancel pairs and to count subarrays and triplets through prefix XORs;
- compute sums of XORs by treating every bit separately, and choose a number bit by bit to maximise such a sum under an upper limit;
- build a binary trie and use it for maximum-XOR queries and for counting pairs whose XOR is below a bound, with proofs;
- build a linear XOR basis by elimination, find the maximum XOR of any subset, and say which questions the basis cannot answer.

XOR problems look like puzzles until you see that they are arithmetic in disguise. The exclusive-or of two numbers adds their bits without carries, so every bit position behaves independently of the others, and every number is its own inverse. Those two facts turn "sum of XORs over all pairs" into 31 small counting problems, "subarray with XOR k" into the prefix-count scan of Chapter 2, "pair with the largest XOR" into a walk down a tree of bits, and "subset with the largest XOR" into Gaussian elimination.

The Infosys question bank leans on these ideas: two of its most reported repeat problems are about XOR, and so is the Hard problem of the official sample paper. This chapter builds the toolkit from the bottom. It starts with how Java stores and shifts bits, proves the algebra of XOR, uses bit independence for sums, and then develops two data structures, the binary trie for questions about *pairs* and the linear basis for questions about *subsets*.

8.1 Binary representation and Java's bit operators

A non-negative integer x is written in binary as bits $x_{B-1} \dots x_1 x_0$ with $x = \Sigma\, x_b \cdot 2^b$. Bit b is "set" when $x_b = 1$. Java's `int` has 32 bits and `long` has 64, and both are signed.

DEFINITION 8.1 Two's complement

In **two's complement** with w bits, the bit pattern $b_{w-1} \dots b_0$ stands for $-b_{w-1} \cdot 2^{w-1} + \Sigma_{i<w-1}\, b_i \cdot 2^i$. The top bit is the sign bit, with weight -2^{w-1}.

Three facts follow and all of them matter in practice:

- $-x = \sim x + 1$, where $\sim x$ flips every bit. For example $\sim 12 = -13$, so $-12 = \sim 12 + 1$.

- −1 is all ones: `Integer.toBinaryString(-13)` is `11111111111111111111111111110011`, 32 characters.
- The range is asymmetric: `Integer.MIN_VALUE` = -2^{31} has no positive partner, so `-Integer.MIN_VALUE` and `Math.abs(Integer.MIN_VALUE)` both return −2147483648.

The operators

Table 8.1 Java's bit operators, with a = 12 (binary 1100) and b = 10 (binary 1010)

Expression	Meaning	Value
`a & b`	AND: 1 where both have 1	8 (1000)
`a | b`	OR: 1 where either has 1	14 (1110)
`a ^ b`	XOR: 1 where exactly one has 1	6 (0110)
`~a`	NOT: flip every bit	−13
`a << 2`	shift left, fill with 0 (multiply by 4)	48
`a >> 2`	shift right, copy the sign bit (floor-divide by 4)	3
`-16 >> 2`	arithmetic shift keeps the sign	−4
`-16 >>> 2`	logical shift fills with 0	1073741820

Two shift rules catch people out. First, `>>` copies the sign bit, so it divides negative numbers by powers of two rounding *down*, while `>>>` treats the bits as unsigned. Second, Java uses only the low 5 bits of the shift distance for an `int` (the low 6 for a `long`). So `1 << 31` is −2147483648 (the sign bit), `1 << 32` is 1 (the shift distance 32 becomes 0) and `1 << 33` is 2. To build 2^k for $k \geq 31$, write `1L << k`, which is correct up to $k = 62$ (`1L << 63` is `Long.MIN_VALUE`).

PITFALL

Precedence. Shifts bind more loosely than `+` and `-`, so `1 << k + 1` means `1 << (k + 1)` (8 for $k = 2$, not 5), and `x >> 1 + 1` means `x >> 2`. The bitwise operators `&`, `^`, `|` bind more loosely than `==`, so `a & b == 0` means `a & (b == 0)`; for `int` operands Java rejects this with "bad operand types for binary operator '&'", which at least stops the bug at compile time. Write brackets around every bit expression: `((x >> b) & 1) == 1`.

Library methods and standard tricks

Table 8.2 Useful methods of Integer (the same names exist on Long), for x = 12

Call	Returns	For 12
`Integer.bitCount(x)`	number of set bits	2
`Integer.highestOneBit(x)`	the *value* 2^k of the top set bit (0 for 0)	8
`Integer.lowestOneBit(x)`	the value of the lowest set bit, same as `x & -x`	4
`Integer.numberOfTrailingZeros(x)`	index of the lowest set bit (32 for 0)	2
`Integer.numberOfLeadingZeros(x)`	zeros above the top set bit (32 for 0)	28
`31 - Integer.numberOfLeadingZeros(x)`	index of the top set bit ($x > 0$)	3
`Integer.toBinaryString(x)`	binary digits, no leading zeros	"1100"
`Integer.parseInt(s, 2)`	read a binary string	12 for "1100"

`highestOneBit` returns a power of two, not an index; that confusion is a common bug. Two expressions are worth knowing by heart. `x & -x` keeps only the lowest set bit, because $-x = \sim x + 1$ flips every bit above the lowest 1 and keeps that 1. `x & (x - 1)` clears the lowest set bit, because subtracting 1 turns the lowest 1 into 0 and the zeros below it into ones. Hence $x > 0$ is a power of two exactly when `(x & (x - 1)) == 0`.

Listing 8.1 Bit-level building blocks (handbook template)

```
// Bit b of x:      ((x >> b) & 1) == 1
// Set / clear / flip bit b:  x | (1 << b),  x & ~(1 << b),  x ^ (1 << b)
// Lowest set bit:  x & -x          Number of 1 bits:  Integer.bitCount(x)
// Highest set bit: 31 - Integer.numberOfLeadingZeros(x)   (x > 0)
// Use 1L << b when b can be 31 or more.
static boolean bitIsSet(int x, int b) {
    return ((x >> b) & 1) == 1;
}
```

A set of up to 31 small elements fits in one `int` as a bitmask: bit i says whether element i is in the set. Union is `|`, intersection is `&`, and `for (int m = 0; m < (1 << n); m++)` visits every subset of an n-element set. Chapter 9 uses this for brute force and Chapter 22 for bitmask DP.

8.2 The algebra of XOR

DEFINITION 8.2 **Exclusive or**

The **XOR** of two bits is 1 when exactly one of them is 1. For integers, $a \oplus b$ (Java `a ^ b`) applies this to every bit position separately. Bit by bit, XOR is addition modulo 2: $0 \oplus 0 = 0$, $0 \oplus 1 = 1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

THEOREM 8.1 **Laws of XOR**

For all integers a, b, c: (1) $a \oplus 0 = a$; (2) $a \oplus a = 0$; (3) $a \oplus b = b \oplus a$; (4) $(a \oplus b) \oplus c = a \oplus (b \oplus c)$; (5) $a \oplus b = c$ if and only if $a = b \oplus c$.

Proof. XOR acts on each bit position independently, so it suffices to check the laws for single bits, where $\oplus$ is addition modulo 2. Addition modulo 2 has 0 as identity (1), satisfies $x + x = 2x \equiv 0$ (2), and is commutative (3) and associative (4) because ordinary addition is. For (5), XOR both sides of $a \oplus b = c$ with b: by (4), (2) and (1) the left side becomes a, the right side $c \oplus b = b \oplus c$. The converse is the same step backwards. ∎

Because of (3) and (4), the XOR of a list does not depend on the order or the grouping of its elements, and by (2) equal elements cancel in pairs.

CLAIM 8.2 **Pairs cancel**

The XOR of a list of numbers equals the XOR of the distinct values that occur an odd number of times in it.

Proof. Reorder the list so that equal values are adjacent; by (3) and (4) the result does not change. Each value occurring m times contributes $v \oplus v \oplus \cdots$ (m copies), which is 0 when m is even and v when m is odd, by (2) and (1). ∎

LeetCode 136 (Single Number) is the special case where every value appears twice except one: the XOR of the whole array is the single value, in $O(n)$ time and $O(1)$ memory. For [7, 3, 5, 3, 7] it is 5.

Prefix XOR

DEFINITION 8.3 Prefix XOR

The **prefix XOR** array of $a[0..n-1]$ is $X[0..n]$ with $X[0] = 0$ and $X[i+1] = X[i] \oplus a[i]$, so $X[i]$ is the XOR of the first i elements.

Exactly as for sums in Chapter 2, $\text{xor}(a[l..r]) = X[r+1] \oplus X[l]$: both prefixes contain $a[0..l-1]$, which cancels by (2). The difference from sums is pleasant: the "subtraction" is XOR again, and there is no overflow.

The prefix-count scan of Chapter 2 therefore counts subarrays with XOR equal to k. By law (5), $X[j] \oplus X[i] = k$ exactly when $X[i] = X[j] \oplus k$, so the look-up key is `prefix ^ k`.

Listing 8.2 Subarrays with XOR exactly k (handbook template)

```java
// Number of subarrays with XOR exactly k: prefix XOR + HashMap (same idea as prefix sums)
static long countSubarraysWithXor(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>();
    seen.put(0, 1);
    int prefix = 0;
    long count = 0;
    for (int x : a) {
        prefix ^= x;
        count += seen.getOrDefault(prefix ^ k, 0); // earlier p with prefix ^ p == k
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

Triplets with equal XOR

LeetCode 1442 (Count Triplets That Can Form Two Arrays of Equal XOR) asks for the number of triples $i < j \le k$ with $\text{xor}(a[i..j-1]) = \text{xor}(a[j..k])$. Call the two sides A and B. By law (5) with $c = 0$, $A = B$ exactly when $A \oplus B = 0$, and $A \oplus B$ is the XOR of the whole block $a[i..k]$. In prefix terms:

$$A = B \iff \text{xor}(a[i..k]) = 0 \iff X[k+1] = X[i].$$

The condition does not mention j. So once a block $a[i..k]$ has XOR 0, **every** split point j with $i < j \le k$ works, and there are $k - i$ of them.

THEOREM 8.3 Counting the triplets

Write $p = i$ and $q = k + 1$. The number of triplets is the sum, over all pairs $p < q$ of prefix indices with $X[p] = X[q]$, of $q - p - 1$.

Proof. A triplet (i, j, k) is valid exactly when $X[i] = X[k+1]$ and $i < j \le k$. For a fixed pair $(p, q) = (i, k+1)$ with equal prefixes, the valid j are $i+1, \ldots, k$, which is $k - i = q - p - 1$ values (zero when $q = p + 1$, as it should be, because then $i = k$ leaves no room for j). Different pairs give different triplets, and every triplet arises from its own pair, so the total is the sum. ∎

Summing over all pairs costs $O(n^2)$, already fast enough for the problem's limit of $n \le 300$. The sum can also be collected in one pass: when the scan reaches q, let c be the number of earlier p with $X[p] = X[q]$ and s the sum of those p. Their contribution is $\Sigma\,(q - 1 - p) = c(q-1) - s$, so a map from each prefix value to its (count, index sum) gives $O(n)$.

EXAMPLE 8.1 **Triplets in [6, 2, 4, 1, 7, 6]**

The prefix XORs are $X = [0, 6, 4, 0, 1, 6, 0]$. The pairs with equal values are (0, 3), (0, 6), (3, 6) for the value 0 and (1, 5) for the value 6, contributing 2 + 5 + 2 + 3 = 12. The one-pass version produces the same total:

```
q   X[q]   earlier with same X: count c, index sum s   adds c(q-1) - s   total
0    0                  0, 0                                  0            0
1    6                  0, 0                                  0            0
2    4                  0, 0                                  0            0
3    0                  1, 0                                  2            2
4    1                  0, 0                                  0            2
5    6                  1, 1                                  3            5
6    0                  2, 3                                  7           12
```

A brute force over all triples also counts 12.

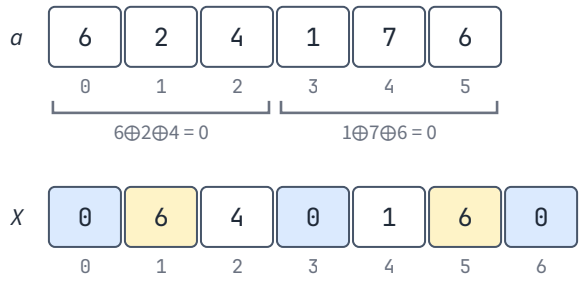


Figure 8.1 Prefix XORs of a = [6, 2, 4, 1, 7, 6]. The three positions where X = 0 (blue) give three blocks with XOR 0: a[0..2], a[3..5] and the whole array. Each block of length m has m − 1 split points.

8.3 Bit independence: sums of XORs and choosing X

A sum of XORs mixes bits only through the final addition. Each bit position can therefore be counted on its own and weighted by its power of two. A second fact about bits makes greedy choices safe.

LEMMA 8.4 **The top bit dominates**

Let x, y be non-negative integers that agree on every bit above position b, with $x_b = 1$ and $y_b = 0$. Then $x > y$, whatever their lower bits are.

Proof. The common high bits contribute equally. Bit b adds 2^b to x and nothing to y. The lower bits add at most $2^0 + \cdots + 2^{b-1} = 2^b - 1$ to y and at least 0 to x. So $x - y \geq 2^b - (2^b - 1) = 1$. ∎

The lemma says that comparing numbers is comparing their bit strings from the top, like words in a dictionary. Every greedy algorithm in the rest of the chapter rests on it: decide the highest bit first, and never give up a higher bit for any combination of lower ones.

The sum of XORs over all pairs

THEOREM 8.5 **Sum of pairwise XORs**

For $a[0..n-1]$ with values below 2^B, let c_b be the number of elements with bit b set. Then

$$\Sigma \text{ over } i < j \text{ of } (a[i] \oplus a[j]) = \Sigma \text{ over } b \text{ of } 2^b \cdot c_b \cdot (n - c_b).$$

Proof. Write each XOR as $\Sigma_b\, 2^b \cdot [\text{bit } b \text{ of } a[i] \neq \text{bit } b \text{ of } a[j]]$, where $[\cdot]$ is 1 if the condition holds and 0 otherwise. Swap the two sums: for a fixed b, the number of pairs whose bits differ is the number of ways to pick one element with the bit and one without, $c_b \cdot (n - c_b)$. ∎

Listing 8.3 Sum of a[i] xor a[j] over all pairs, bit by bit (handbook template)

```
// Sum of (a[i] xor a[j]) over all pairs i < j. Bits are independent, so count per bit.
static long sumPairXor(int[] a) {
    long total = 0;
    for (int b = 0; b < 31; b++) {
        long ones = 0;
        for (int x : a) ones += (x >> b) & 1;
        long pairs = ones * (a.length - ones);     // one number has the bit, the other doesn't
        total += pairs << b;
    }
    return total;
}
```

The cost is $O(31 \cdot n)$ instead of $O(n^2)$. For [5, 9, 12, 7] (binary 0101, 1001, 1100, 0111) the counts of ones are 3, 1, 3, 2 for bits 0 to 3, so the pairs differing at each bit are 3, 3, 3, 4, and the total is $3 \cdot 1 + 3 \cdot 2 + 3 \cdot 4 + 4 \cdot 8 = 53$, the same as adding the six XORs directly. Note `ones` is a `long`, so `ones * (a.length - ones)` cannot overflow even for $n = 10^5$.

Choosing X to maximise a sum of XORs

For a fixed X, the same argument gives $\Sigma_i\, (X \oplus a[i]) = \Sigma_b\, 2^b \cdot g_b(X_b)$, where the gain of bit b is

$g_b(1) = n - c_b$ (bit b of $X = 1$ makes the XOR bit 1 for the elements without the bit), $g_b(0) = c_b$ (bit b of $X = 0$ keeps the bit for the elements that have it).

Without other constraints each bit of X is chosen on its own: set it when $n - c_b > c_b$.

One of the Infosys repeat problems (6 reports in the plan's question bank) adds an upper limit: choose X in $[0, K]$ to maximise $\Sigma\, (X \oplus A[i])$. Now the bits are no longer free, because setting a high bit of X may push it above K. The way out is the structure of the numbers $\le K$.

LEMMA 8.6 **The shape of the numbers below K**

A non-negative integer $X < K$ agrees with K on all bits above some position b, has $X_b = 0$ where $K_b = 1$, and has arbitrary bits below b. Conversely every number of that form is $< K$.

Proof. If $X < K$, let b be the highest position where they differ. Above b they agree, so by Lemma 8.4 the larger number has the 1 at position b; since $X < K$, $K_b = 1$ and $X_b = 0$. The converse is Lemma 8.4 applied to K and X. ∎

So the candidates are $X = K$ itself, plus one family for each set bit b of K: copy K above b, put 0 at b, and below b choose every bit freely by its own gain. Each family's best member is found bit by bit, giving at most 32 candidates and $O(31 \cdot 31)$ work after counting the bits. This is the same "follow the limit, then branch off below it" walk that digit DP formalises in Chapter 22; the full Infosys solution, with its input format and edge cases, is in Chapter 24.

EXAMPLE 8.2 **Choosing X ≤ 6 for A = [1, 4, 6, 3, 2]**

Three bits suffice. With $n = 5$ the counts and gains are:

```
bit b   c_b   gain if X_b = 0 (c_b * 2^b)   gain if X_b = 1 ((n - c_b) * 2^b)   free choice
  2      2              8                                12                          1
  1      3              6                                 4                          0
  0      2              2                                 3                          1
```

The free choice is $X = 101_2 = 5$, which happens to be ≤ 6, but the walk does not know that in advance. $K = 6 = 110_2$ has set bits 2 and 1:

```
candidate                          X (binary)   sum of X xor A[i]
X = K                              110 = 6      12 + 4 + 2 = 18
branch at bit 2: 0, then free      001 = 1       8 + 6 + 3 = 17
branch at bit 1: 1, 0, then free   101 = 5      12 + 6 + 3 = 21
best: X = 5 with 21
```

A brute force over $X = 0..6$ agrees: the maximum is 21 at $X = 5$. With $K = 4$ instead, the branch at bit 2 gives 17 and $X = K = 4$ gives 20, which is then the answer.

Minimising an XOR with a fixed number of bits

LeetCode 2429 (Minimize XOR), a bonus problem today, asks for the X with a given number of set bits that makes $X \oplus$ num1 as small as possible. The key observation is Lemma 8.4 read the other way: a 1 in the XOR at a high position costs more than all lower positions together. So spend the available set bits of X on the highest set bits of num1 first (each one turns a 1 of the XOR into a 0), and if bits remain, place them at the lowest positions where num1 has a 0.

8.4 The binary trie

Per-bit counting handles sums over all pairs. It cannot answer "which partner y in the set gives the largest $x \oplus y$?", because that needs the *same* y for all bits. A tree of bit prefixes can.

DEFINITION 8.4 Binary trie

A **binary trie** over B-bit numbers is a rooted tree in which every node has at most two children, labelled 0 and 1. Each stored number is the root-to-leaf path that reads its bits from bit $B - 1$ down to bit 0. Every node also stores `cnt`, the number of stored numbers whose path passes through it.

A node at depth d stands for one d-bit prefix, and its subtree holds exactly the stored numbers that start with that prefix. Inserting a number creates at most B nodes; numbers that share a prefix share its nodes. Figure 8.2, after the listing, shows a small example.

The handbook template stores the trie in arrays: `child[node][bit]` is the index of a child, with 0 meaning "missing". Node 0 is the root, and the root is never anyone's child, so 0 is free to mean "none".

Listing 8.4 Binary trie for XOR queries over non-negative ints (handbook template)

```
// Binary trie over non-negative ints, highest bit first
static final class XorTrie {
    static final int B = 31;
    final int[][] child;          // child[node][bit], 0 = missing (the root is never a child)
    final int[] cnt;              // how many inserted numbers pass through this node
    int size = 1;

    XorTrie(int maxInserts) {
        child = new int[maxInserts * B + 1][2];
        cnt = new int[child.length];
    }

```

```
    void insert(int x) {
        int node = 0;
        for (int b = B - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (child[node][bit] == 0) child[node][bit] = size++;
            node = child[node][bit];
            cnt[node]++;
        }
    }

    // max of (x ^ y) over inserted y. Trie must not be empty.
    int maxXor(int x) {
        int node = 0, res = 0;
        for (int b = B - 1; b >= 0; b--) {
            int want = ((x >> b) & 1) ^ 1;          // the opposite bit makes this xor bit 1
            if (child[node][want] != 0) {
                res |= 1 << b;
                node = child[node][want];
            } else {
                node = child[node][want ^ 1];
            }
        }
        return res;
    }

    // how many inserted y have (x ^ y) < k
    long countXorLess(int x, int k) {
        long res = 0;
        int node = 0;
        for (int b = B - 1; b >= 0; b--) {
            int xb = (x >> b) & 1, kb = (k >> b) & 1;
            if (kb == 1) {
                if (child[node][xb] != 0) res += cnt[child[node][xb]];   // xor bit 0 < 1: all count
                node = child[node][xb ^ 1];                               // xor bit 1: keep going
            } else {
                node = child[node][xb];                                   // xor bit must be 0
            }
            if (node == 0) break;
        }
        return res;
    }
}
```

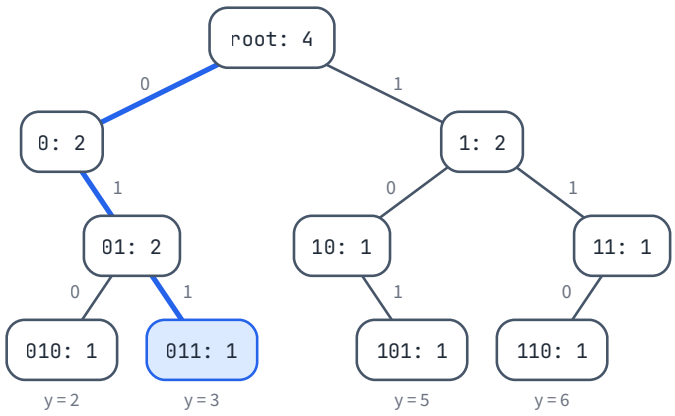


Figure 8.2 A binary trie of the 3-bit numbers 2, 3, 5 and 6. Each node shows the prefix it stands for and, after the colon, its count; edges are labelled with the bit they read. The blue path answers the query “maximum of 4 ⊕ y”: at every level it takes the bit opposite to 4’s bit when that child exists, ending at y = 3.

Maximum XOR with a stored number

THEOREM 8.7 The greedy walk finds the maximum XOR

For a non-empty trie and any x, `maxXor(x)` returns max over stored y of $x \oplus y$.

Proof. Let M be the true maximum. We show by induction on the number of levels processed that, after handling bits $B-1..b$, the bits of `res` above and at b equal those of M, and `node` is the trie node for the prefix of some stored y with $x \oplus y = M$ on those bits. Before any level, both statements are about no bits and `node` is the root. At bit b, the subtree of `node` holds exactly the stored numbers y whose XOR with x matches M on the higher bits. If the child with the opposite bit exists, some such y has XOR bit b equal to 1; by Lemma 8.4 every maximiser must then have bit b equal to 1, so M has it, and the walk sets it and enters that child. If the child does not exist, every y in the subtree has XOR bit b equal to 0, so M has 0 there, and the walk enters the only child. After bit 0, `res` $= M$. ∎

For LeetCode 421 (Maximum XOR of Two Numbers in an Array), insert every number and take the maximum of `maxXor` over all of them; a number paired with itself gives 0, which never beats a real pair unless all numbers are equal. The cost is $O(31 \cdot n)$.

On the trie of Figure 8.2 with $x = 4 = 100_2$: at bit 2 the walk wants 0 and finds it, at bit 1 it wants 1 and finds it, at bit 0 it wants 1 and finds it, reaching $y = 3$ with $4 \oplus 3 = 7$. The other candidates give $4 \oplus 2 = 6$, $4 \oplus 5 = 1$ and $4 \oplus 6 = 2$.

Counting pairs with XOR below a bound

The second query counts stored y with $x \oplus y < k$. By Lemma 8.6, $x \oplus y < k$ exactly when, at the highest bit b where $x \oplus y$ and k differ, k has 1 and $x \oplus y$ has 0. The walk follows the path on which $x \oplus y$ equals k bit by bit, and collects the branches that leave it at a 1-bit of k.

THEOREM 8.8 Counting XOR below k

`countXorLess(x, k)` returns the number of stored y with $x \oplus y < k$.

Proof. For each stored y with $x \oplus y < k$, let b be the highest bit where $x \oplus y$ and k differ; then $k_b = 1$, $y_b = x_b$, and above b, $y_t = x_t \oplus k_t$. So the y counted "at bit b" are exactly those in the subtree reached by following the bits $x_t \oplus k_t$ from the top down to bit $b+1$ and then the bit x_b. The walk does exactly this: on each level it moves to the child $x_t \oplus k_t$ (the code's `xb ^ 1` when $k_t = 1$ and `xb` when $k_t = 0$), and at each level with $k_b = 1$ it first adds the count of the child x_b. Different b give disjoint sets of y, so nothing is counted twice; a y with $x \oplus y = k$ never leaves the path and is never counted, which is right because k is not $< k$. If the path runs into a missing child, no stored y continues it, and the loop stops with all branches already added. ∎

On the trie of Figure 8.2, `countXorLess(4, 3)` runs as follows (with $B = 3$ for the picture):

```
bit  x_b  k_b  action                                          running count
 2    1    0   XOR bit must be 0: go to child 1 {5, 6}              0
 1    0    1   add child 0 {5} (count 1), go to child 1 {6}         1
 0    0    1   add child 0 {6} (count 1), go to child 1 (missing)   2
result 2: the y with 4 xor y < 3 are 5 (4 xor 5 = 1) and 6 (4 xor 6 = 2)
```

Pairs with XOR in a range $[lo, hi]$ are countXorLess($hi+1$) − countXorLess(lo). To count pairs $i < j$ rather than ordered pairs, query each element against the trie *before* inserting it; that is the whole idea of LeetCode 1803 (Count Pairs With XOR in a Range).

Offline queries

LeetCode 1707 (Maximum XOR With an Element From Array), a bonus problem, asks many queries "the maximum of $x \oplus y$ over the array elements $y \le m$". The trie cannot ignore large elements, but the queries do not have to be answered in the given order. The key observation is to sort the queries by m and the array ascending, and before each query insert every element $\le m$ that is not yet in the trie; a query with an empty trie has no answer. Answering queries in a convenient order and putting the answers back in the original positions is called **offline processing**.

Memory

Each insertion creates at most B nodes, so the trie never has more than $n \cdot B + 1$ nodes, which is how the template sizes its arrays. For $n = 2 \cdot 10^5$ and $B = 31$ that is $6.2 \cdot 10^6$ nodes. The layout matters: `new int[nodes][2]` makes one small array object per node, and on a 64-bit JVM each costs about 28 bytes including its reference. Measured on the sandbox JVM, the `child` array took 174 MB and `cnt` another 25 MB. Storing the two children of node v at `next[2 * v]` and `next[2 * v + 1]` of one flat `int[]` needs 8 bytes per node instead, 75 MB for `next` and `cnt` together.

Listing 8.5 The same trie in flat arrays (about 12 bytes per node)

```
static final class FlatXorTrie {
    static final int B = 31;
    final int[] next;              // children of node v: next[2 * v] (bit 0), next[2 * v + 1] (bit 1)
    final int[] cnt;
    int size = 1;

    FlatXorTrie(int maxInserts) {
        int nodes = maxInserts * B + 1;
        next = new int[2 * nodes];
        cnt = new int[nodes];
    }

    void insert(int x) {
        int node = 0;
        for (int b = B - 1; b >= 0; b--) {
            int slot = 2 * node + ((x >> b) & 1);
            if (next[slot] == 0) next[slot] = size++;
            node = next[slot];
            cnt[node]++;
        }
    }

    int maxXor(int x) {            // trie must not be empty
        int node = 0, res = 0;
        for (int b = B - 1; b >= 0; b--) {
            int want = ((x >> b) & 1) ^ 1;
            if (next[2 * node + want] != 0) {
                res |= 1 << b;
                node = next[2 * node + want];
            } else {
                node = next[2 * node + (want ^ 1)];
            }
        }
        return res;
    }
}
```

EXAM TIP

Choose B from the limits. Values below 2^{30} (about $1.07 \cdot 10^9$) need only $B = 30$; values below 2^{20} (about 10^6) need 20, which cuts both time and memory by a third. Non-negative `int`s need 31.

8.5 The XOR basis

The last tool answers questions about **subsets**: "what is the largest XOR of any subset of these numbers?" and "can some subset XOR to v?". There are 2^n subsets, but their XORs have a lot of structure.

Think of a number below 2^B as a vector of B bits. XOR adds two vectors coordinate by coordinate modulo 2. The bits $\{0, 1\}$ with addition and multiplication modulo 2 form the smallest field, written **GF(2)**, and the B-bit numbers form a vector space over it. Multiplying by a scalar is trivial (by 1 keeps a vector, by 0 gives 0), so a "linear combination" of numbers is just the XOR of a subset of them.

DEFINITION 8.5 Span, independence and basis

The **span** of a set S of numbers is the set of XORs of all subsets of S (the empty subset gives 0). S is **independent** if no non-empty subset of it has XOR 0. A **XOR basis** of a span is an independent set whose span is that span.

The handbook template keeps at most one basis number for each highest bit, the arrangement produced by Gaussian elimination.

Listing 8.6 Linear XOR basis (handbook template)

```
// Linear XOR basis: the XOR of any subset can be built from at most 61 basis numbers
static final class XorBasis {
    final long[] b = new long[61];              // b[i] has highest bit i, or is 0

    boolean add(long x) {                       // returns false if x was already reachable
        for (int i = 60; i >= 0; i--) {
            if (((x >> i) & 1) == 0) continue;
            if (b[i] == 0) {
                b[i] = x;
                return true;
            }
            x ^= b[i];
        }
        return false;
    }

    long max() {                                // maximum XOR of any subset
        long r = 0;
        for (int i = 60; i >= 0; i--) r = Math.max(r, r ^ b[i]);
        return r;
    }
}
```

THEOREM 8.9 What the basis maintains

After `add` has been called on $x_1, \ldots, x_m$, in that order, with values below 2^{61}: (1) every non-zero `b[i]` has highest set bit i; (2) the non-zero `b[i]` are independent; (3) their span equals the span of $\{x_1, \ldots, x_m\}$; (4) `add(x)` returns false exactly when x was already in the span.

Proof. Assume (1)–(3) hold before `add(x)`, and let x_0 be the original value. During the loop, x changes only by XOR with basis numbers, so $x \oplus x_0$ always lies in the old span.

When the loop reaches bit i with $x_i = 1$, all higher bits of x are 0: each higher set bit j was either cleared by `x ^= b[j]` (which clears bit j, because $b[j]$ has highest bit j, and changes only lower bits) or would have stopped the loop there. If $b[i] = 0$, the new basis number x has highest bit i, keeping (1). It is not in the old span: by (1), the basis numbers have distinct highest bits, so any non-empty XOR of them has as its highest bit the largest highest bit involved, which is never i. Hence the enlarged set stays independent, giving (2).

The new span is the old span plus x, which is the old span plus x_0 because they differ by an element of the old span, giving (3).

If the loop ends without storing, every set bit met was cleared, so $x = 0$ and $x_0 = x \oplus x_0$ is in the old span: the span does not change and the method returns false. If x_0 was in the old span, then so is the final x, which therefore cannot be stored (shown above), so the method returns false; otherwise it returns true. That is (4). ∎

CLAIM 8.10 Size of the basis and of the span

A basis of numbers below 2^B has at most B elements. If it has r elements, every value in the span is the XOR of exactly one subset of the basis, so the span has exactly 2^r values. Every value in the span is also the XOR of at most r of the original numbers.

Proof. The highest bits are distinct and lie in $0..B-1$, so $r \le B$. If two different subsets of the basis had the same XOR, the XOR of their symmetric difference would be 0, contradicting independence; so the 2^r subsets give 2^r different values. For the last statement, the r original numbers whose `add` returned true have, by (3), the same span as the basis (each successful `add` enlarged the span by exactly its own original number, each failed one by nothing). Any span value is the XOR of a subset of these r numbers. ∎

The maximum subset XOR

THEOREM 8.11 The greedy maximum

`max()` returns the largest XOR of any subset of the inserted numbers.

Proof. By Claim 8.10 each span value is the XOR of a unique subset T of the basis. Bit i of that XOR depends only on whether $b[i] \in T$ and on the basis numbers above i, because the ones below have no bit i. So reading from the top, at each basis position i the choice "include $b[i]$ or not" is exactly the choice of bit i, given the higher choices.

Let M be the largest span value. M has a 1 at every basis position i: otherwise $M \oplus b[i]$ agrees with M above i and has bit i set, so it is larger by Lemma 8.4. The greedy loop processes positions from the top; at a basis position, `r` and `r ^ b[i]` agree above i and differ at bit i, so `Math.max` keeps the one with bit i equal to 1 (and at non-basis positions $b[i] = 0$ changes nothing). So `r` ends with a 1 at every basis position, and since the subset is determined by the bits at the basis positions, `r` $= M$. ∎

The cost is O(61) per insertion and per maximum, or O(B) with a smaller array when the values are small. Membership is the same reduction as `add` without the store.

Listing 8.7 Can some subset XOR to v? Reduce v by the basis

```
static boolean canMake(long[] b, long v) {       // b = the array of an XorBasis
    for (int i = 60; i >= 0; i--)
        if (((v >> i) & 1) == 1 && b[i] != 0) v ^= b[i];
    return v == 0;
}
```

EXAMPLE 8.3 Building a basis for [12, 9, 7, 5]

In 4-bit binary the numbers are 1100, 1001, 0111 and 0101.

```
add(12) 1100:  b[3] empty                              -> b[3] = 1100 (12)
add(9)  1001:  bit 3: xor b[3] -> 0101 (5); b[2] empty -> b[2] = 0101 (5)
add(7)  0111:  bit 2: xor b[2] -> 0010 (2); b[1] empty -> b[1] = 0010 (2)
add(5)  0101:  bit 2: xor b[2] -> 0000                 -> false (5 = 12 xor 9)
max():  r = 0 -> 12 (take b[3]) -> 12 (skip b[2]: 12 xor 5 = 9) -> 14 (take b[1]) -> 14
```

The basis has rank 3, so the subsets of the four numbers produce exactly $2^3 = 8$ distinct XORs: 0, 2, 5, 7, 9, 11, 12, 14. The maximum $14 = 1110_2$ is $9 \oplus 7$. A brute force over all 16 subsets confirms both facts.

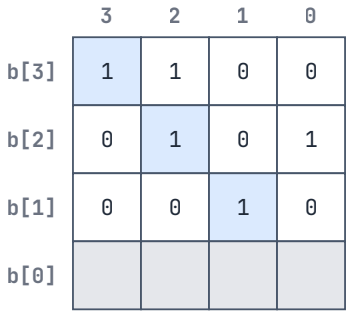


Figure 8.3 The basis after inserting 12, 9, 7 and 5, one row per slot and one column per bit (bit 3 on the left). Each non-empty row starts with its own leading bit (blue); no two rows share a leading position, which is what makes the greedy maximum work.

What the basis cannot answer

The basis forgets *how many* numbers were used and *which positions* they came from. It answers "is v reachable by some subset?" and "what is the largest reachable value?", and with a fully reduced basis also "what is the k-th smallest reachable value?". It cannot answer:

- **Questions with a size limit.** The Infosys repeat "Maximum XOR choosing at most $N/2$ elements" (13 reports in the plan's question bank) has the sample [1, 2] with answer 2: the basis says $1 \oplus 2 = 3$, but that uses two elements and the limit is one. Claim 8.10 shows when the limit is harmless: the maximum is the XOR of at most r original numbers, and $r \le 20$ for values below 2^{20}, so the basis answer is correct whenever $N/2 \ge r$, in particular for $N \ge 40$. For smaller N the problem needs a different method, an XOR knapsack over the 2^{20} possible values, which Chapter 26 develops.
- **Questions about contiguous pieces.** "Maximum XOR of a subarray" is a pair question about prefix XORs ($X[j] \oplus X[i]$), so it belongs to the trie: insert prefixes and query each new one.
- **Exactly k elements**, or any condition on which elements are used. A related counting fact does follow from the rank, though: each reachable value is the XOR of exactly 2^{n-r} of the 2^n subsets, because the $n - r$ numbers that `add` rejected can be included or left out freely, and each choice is completed to the target in exactly one way by the r accepted numbers.

8.6 Choosing the right XOR tool

The four ideas of this chapter cover most XOR questions, and the shape of the question picks the tool.

Table 8.3 Which tool for which XOR question

The question is about	Tool	Cost
one element that does not cancel, parity of counts	XOR of everything (Claim 8.2)	O(n)
subarrays with XOR k, blocks with XOR 0	prefix XOR + hash map (Listing 8.2)	O(n)

The question is about	Tool	Cost
a sum of XORs over all pairs, or $\Sigma\ (X \oplus a[i])$	per-bit counting (Theorem 8.5)	$O(B \cdot n)$
choosing X bit by bit, possibly $\le K$	gains per bit + the walk of Lemma 8.6	$O(B^2)$ after counting
the best partner of x, pairs with XOR $< k$ or in a range	binary trie (Listing 8.4)	$O(B)$ per query
the best subset, reachable values, rank	XOR basis (Listing 8.6)	$O(B)$ per insert
subsets with a size limit	not the basis: XOR knapsack (Chapter 26)	$O(n \cdot 2^B)$

INFOSYS LENS

XOR runs through the plan's Infosys material. Besides the two repeats above (the size-limited subset maximum and choosing $X \le K$), the question bank reports a longest increasing subsequence whose adjacent pairs satisfy $(x \,\&\, y) \cdot 2 < (x \mid y)$, a condition that turns out to depend only on the highest set bits; a reported interview task counts pairs with XOR $< K$ whose sum is prime, where the trie of Section 8.4 replaces the nested loops; and the Hard problem of the official sample paper is a longest non-decreasing subsequence whose XOR is at least M, solved with an extra XOR dimension in a DP. Later chapters come back to them.

Summary

- Java integers are two's complement: $-x = \sim x + 1$, -1 is all ones, and `Integer.MIN_VALUE` has no positive partner. `>>` keeps the sign, `>>>` fills with zeros, and shift distances are taken mod 32 (mod 64 for `long`): use `1L << k` for $k \ge 31$.
- Bracket every bit expression: shifts bind more loosely than `+`, and `&`, `^`, `|` more loosely than `==`.
- XOR is bitwise addition mod 2: it has identity 0, every number is its own inverse, and order does not matter. Equal values cancel in pairs.
- Prefix XORs work like prefix sums: $\text{xor}(l..r) = X[r+1] \oplus X[l]$. A block has XOR 0 exactly when its two end prefixes are equal, which counts the triplets of LeetCode 1442 as $\Sigma\ (q - p - 1)$ over equal pairs.
- In a sum of XORs every bit is independent: count the ones per bit and weight by 2^b. A higher bit outweighs all lower bits together, which justifies every greedy choice from the top.
- The numbers $\le K$ are K itself and, for each 1-bit of K, "copy K above, 0 here, anything below". Walking K's bits this way maximises a per-bit objective under the limit.
- A binary trie answers pair questions in $O(B)$ per query: the maximum XOR by taking the opposite bit when possible, and the count of XOR $< k$ by following k's bits and adding the branches at its 1-bits. Size it as $n \cdot B + 1$ nodes and prefer flat arrays for large n.
- A linear basis keeps at most one number per highest bit. It spans exactly the subset XORs, has at most B elements, gives the maximum subset XOR greedily, and ignores how many elements are used.

Key terms

two's complement p. 108
exclusive or (XOR) p. 110
prefix XOR p. 111
binary trie p. 114
linear basis (XOR basis) p. 118
offline queries p. 117
GF(2) p. 118

Review questions

1. What are `1 << 31`, `1 << 32` and `1L << 32` in Java, and why? Which expression gives 2^k for $31 \le k \le 62$?

2. Explain why `x & -x` isolates the lowest set bit of $x > 0$. What does `x & (x - 1)` do?
3. Prove that the XOR of a list is unchanged when two equal elements are removed from it. Which laws do you use?
4. For $a = [4, 4, 4, 4]$, list the prefix XORs and count the triplets of LeetCode 1442 with the formula of Theorem 8.3.
5. Using Theorem 8.5, why can the sum of XORs over all pairs never exceed $(n^2/4) \cdot (2^B - 1)$?
6. In the "choose $X \le K$" walk, why is it safe to choose the bits below the branching bit independently of each other?
7. In `countXorLess`, why is a number y with $x \oplus y = k$ never counted? How do you count pairs with $lo \le x \oplus y \le hi$?
8. A trie stores 10^5 numbers below 2^{20}. How many nodes can it have at most, if you choose B well?
9. Insert 3, 5, 6 into an XOR basis. What does the basis look like, what is the maximum subset XOR, and how many distinct subset XORs are there?
10. Why can the basis not solve "maximum XOR of at most $N/2$ elements" in general, and when does its answer happen to be correct?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 10 "Bit manipulation", on bit representation, bit operations and representing sets as bits, and Chapter 26 "String algorithms", section 26.2, on the trie structure. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Bit manipulation" (cp-algorithms.com/algebra/bit-manipulation.html), for the standard tricks and their proofs.
- Java 8 API documentation for `java.lang.Integer` and `java.lang.Long` (`bitCount`, `highestOneBit`, `numberOfTrailingZeros`, `toBinaryString`), docs.oracle.com/javase/8/docs/api/.
- Workbook Chapter 8 has trie walks and a basis elimination to trace by hand.

Problems

These are the problems for Day 8. Work them in Workbook Chapter 8, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
8.P1	LC 136 · Single Number	EASY	CORE
8.P2	LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR	MEDIUM	CORE
8.P3	LC 421 · Maximum XOR of Two Numbers in an Array	MEDIUM	CORE
8.P4	LC 1803 · Count Pairs With XOR in a Range	HARD	CORE
8.P5	LC 2429 · Minimize XOR	MEDIUM	BONUS
8.P6	GfG · Maximum Subset XOR	GFG	BONUS
8.P7	LC 1707 · Maximum XOR With an Element From Array	HARD	BONUS

CHAPTER 9 · DAY 9 OF 29 · SUNDAY 15 NOVEMBER 2026 · MOCK DAY

9

Brute Force, Stress Testing and the Mock Exam

3 hours on Phase 1 topics at Q1/Q2/Q3 difficulty, then a full review.

Part I · Foundations
Handbook cards · Brute force first, then stress test
Before this chapter · Chapters 1–8
Practice · Workbook Chapter 9 · 3 problems · core ≈ 4.0 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- write a correct brute force for a problem in 5–10 minutes and say which test sizes it will pass;
- enumerate subarrays, pairs, subsets and permutations, and know the cost and the input size each one fits;
- build a stress tester that compares a fast solution with a brute force on random small inputs, and shrink a failing input to a minimal one;
- turn the limits of a problem into the complexity it needs;
- run a timed mock under exam rules, grade it by the share of tests passed, and classify every error with a rule that prevents it.

Today is the first full mock: three Codeforces problems in three hours, under the rules of the real assessment. The chapter prepares you for it and for what comes after it. It is short on new algorithms and long on method, because the habits it teaches decide how many marks your algorithms actually earn.

The central habit is to write the slow, obviously correct solution first. A brute force earns the marks of the small tests, it tells you whether your fast solution is right, and it often shows you the pattern that leads to the fast idea. Sections Section 9.1 to Section 9.4 develop that habit: what to enumerate, how to compare a fast and a slow program automatically, and how to read the limits so that you know in advance which tests each version will pass. Sections Section 9.5 and Section 9.6 are about the mock itself and the review afterwards. Section 9.7 collects the patterns of Chapters 1–8 into one table for revision, and Section 9.8 gives the key ideas of the three mock problems, to be read only once you have tried them.

9.1 Why brute force first

DEFINITION 9.1 Brute force

A **brute force** solution tries every candidate answer allowed by the statement (every subarray, every subset, every order, every value) and keeps the best or counts the good ones. It is correct by construction, because it checks the definition directly, and slow, because the number of candidates grows quickly with the input size.

On the Infosys paper, according to the plan's summary of the scoring, marks come from the test cases you pass, and partial solutions score. A problem's tests normally range from tiny to the full limits, so a correct

brute force collects every test it can finish in time. The plan's walkthrough of the official sample paper makes the point concrete: for the Hard problem, trying every subsequence works up to $N \approx 20$, and a correct program on those tests scores while a blank one does not. The exam-day rule in the plan says it in one sentence: a correct brute force scores more than a fast wrong solution.

A brute force has two more uses that matter as much as partial marks.

DEFINITION 9.2 Test oracle

A **test oracle** is a program whose output on an input is trusted to be correct. A brute force is the usual oracle in algorithm work: slow, but simple enough to be right.

- **It is an oracle.** When your fast solution passes the samples but fails a hidden test, the brute force is the only tool that can find the input it fails on. Section 9.3 automates the search.
- **It is a thinking tool.** Running a brute force on many small inputs and printing the answers often shows the structure of the problem: an answer that only depends on the maximum, a pattern in powers of two, a greedy order that is always optimal. Several fast ideas in this book were first seen in a table of brute-force outputs.

INFOSYS LENS

The plan's target section quotes one detailed report: a candidate who solved three problems fully and passed 6 of 12 tests on a fourth was shortlisted for the SP L3 interview. Its advice is to plan for Q1 and Q2 fully correct plus real progress on Q3, and the marks reported for the three problems are 20, 30 and 50. Real progress on Q3 usually starts as a brute force.

EXAM TIP

Budget 5–10 minutes for the brute force of every problem, and write it *before* you start on the fast idea. If the fast idea works, the brute force has cost you ten minutes and bought you a test oracle. If it does not, the brute force is already submitted.

9.2 Complete search patterns

Almost every brute force is one of a few enumerations. Knowing their costs lets you say, before writing a line, which limits they will handle.

Table 9.1 The standard enumerations and the sizes they fit (about 10^8 simple steps per second)

Enumerate	Candidates	Typical cost	Fits roughly
all pairs $i < j$	$n(n-1)/2$	$O(n^2)$	$n \le 10^4$
all subarrays, with a running value	$n(n+1)/2$	$O(n^2)$	$n \le 10^4$
all subarrays, recomputed from scratch	$n(n+1)/2$	$O(n^3)$	$n \le 500$
all triples	about $n^3/6$	$O(n^3)$	$n \le 500$
all subsets (bitmask)	2^n	$O(2^n \cdot n)$	$n \le 20$
all orders (permutations)	$n!$	$O(n! \cdot n)$	$n \le 10$
every candidate answer x	V	$O(V \cdot \text{check})$	$V \cdot n \le 10^8$

The numbers in the last column come from counting: $20 \cdot 2^{20} = 20{,}971{,}520$ steps, $10 \cdot 10! = 36{,}288{,}000$ and $500^3 = 1.25 \cdot 10^8$. One more element doubles a subset search and multiplies a permutation search by $n + 1$, which is why the limits are so sharp: $11 \cdot 11!$ is already 439,084,800.

Subsets by bitmask

Number the elements $0..n-1$. Every integer *mask* in $0..2^n - 1$ describes one subset: element i is in it exactly when bit i of *mask* is 1 (Chapter 8).

LEMMA 9.1 Masks and subsets

The map from mask $\in \{0, \ldots, 2^n - 1\}$ to the subset $\{i : \text{bit } i \text{ of mask is } 1\}$ is a bijection onto the subsets of $\{0, \ldots, n-1\}$. So the loop `for (int mask = 0; mask < (1 << n); mask++)` visits every subset exactly once.

Proof. Each subset S is the image of exactly one mask, Σ over $i \in S$ of 2^i, because every integer below 2^n has exactly one binary representation with n digits. ∎

Listing 9.1 Brute force over all subsets: the largest subset sum that does not exceed S

```
static long bestSubsetSum(int[] a, long S) {                // O(2^n * n): n <= 20 or so
    int n = a.length;
    long best = 0;                                          // the empty subset
    for (int mask = 0; mask < (1 << n); mask++) {
        long sum = 0;
        for (int i = 0; i < n; i++) if ((mask & (1 << i)) != 0) sum += a[i];
        if (sum <= S) best = Math.max(best, sum);
    }
    return best;
}
```

`1 << n` is only safe for $n \le 30$, which is far beyond any feasible subset search anyway. The same loop counts subsets with a property, finds the best one, or tests every way to split the elements into two groups (bit i says which group element i joins).

Permutations

Java has no `next_permutation`, so it is worth having one that you trust. It rearranges the array into the next arrangement in lexicographic order (Chapter 7), and reports when there is none.

Listing 9.2 The next permutation in lexicographic order

```
static boolean nextPermutation(int[] a) {          // false (and a sorted again) after the last one
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;        // a[i+1..] is the longest non-increasing suffix
    if (i < 0) {
        reverse(a, 0);
        return false;
    }
    int j = a.length - 1;
    while (a[j] <= a[i]) j--;                      // the rightmost element larger than a[i]
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
    reverse(a, i + 1);                             // the suffix becomes its smallest arrangement
    return true;
}

static void reverse(int[] a, int from) {
    for (int l = from, r = a.length - 1; l < r; l++, r--) {
        int t = a[l];
        a[l] = a[r];
```

```
        a[r] = t;
    }
}
```

LEMMA 9.2 Correctness of the next permutation

If a is not the largest arrangement of its elements, Listing 9.2 turns it into the smallest arrangement that is lexicographically larger than a.

Proof. The suffix $a[i+1..]$ is non-increasing, so it is already the largest arrangement of its elements, and no larger arrangement keeps $a[0..i]$ unchanged. Every larger arrangement that keeps $a[0..i-1]$ must put at position i an element of the suffix that is larger than $a[i]$; the smallest such arrangement puts there the *smallest* element of the suffix larger than $a[i]$, and then the remaining elements in non-decreasing order. Because the suffix is non-increasing, the rightmost element larger than $a[i]$ is the smallest such element (the rightmost of its equal copies). After the swap the suffix is still non-increasing, so reversing it makes it non-decreasing. Any arrangement that changes a position before i to a larger value is larger still, so the result is the next one. ∎

Start from the sorted array and loop `do { ... } while (nextPermutation(a));` to visit every distinct arrangement exactly once, in increasing order. With repeated values this visits fewer than $n!$ arrangements, which is usually what you want. For the array [1, 2, 2] it visits [1, 2, 2], [2, 1, 2] and [2, 2, 1].

Recursion is the other way, and it is easier to adapt when the order is built step by step and some steps are not allowed. The next listing is the brute force for one of the plan's Infosys repeat problems: you start with experience e, monster i can be beaten when $e \ge \text{power}[i]$ and then adds $\text{bonus}[i]$, a lost fight ends the run, and you want the largest number of monsters beaten in some order.

Listing 9.3 Brute force over all orders, by recursion with undo

```
static int bestFights(long e, int[] power, int[] bonus, boolean[] used) {   // O(n! * n): n <= 9 or so
    int best = 0;
    for (int i = 0; i < power.length; i++) {
        if (used[i] || e < power[i]) continue;          // already beaten, or this fight would be lost
        used[i] = true;
        best = Math.max(best, 1 + bestFights(e + bonus[i], power, bonus, used));
        used[i] = false;                                // undo the choice before trying the next one
    }
    return best;
}
```

It never starts a fight it would lose, because a lost fight ends the run and adds nothing. Comparing this with the fast greedy of the repeat card (sort by power, fight while you can) on random small inputs is exactly the stress test of the next section, and the two agree when the bonuses are non-negative.

Trying every answer

The brute force for a "search on the answer" problem (Chapter 6) tries the candidates one by one with the same check: $x = lo$, $lo + 1$, … until $\text{ok}(x)$ is true. It is $O(V \cdot C)$ instead of $O(C \log V)$, but it shares the check with the fast version, so it tests the search and the bounds, not the check. When the check is the doubtful part, the oracle must avoid it: enumerate the schedules, partitions or placements directly.

9.3 Stress testing

DEFINITION 9.3 Stress testing

Stress testing runs a fast solution and a brute force on many random small inputs and stops at the first input where their answers differ. That input is a counterexample: since the brute force is trusted, the fast solution is wrong on it.

A stress tester has four parts: a **generator** of random small inputs, the **brute force**, the **fast solution**, and a **loop** that compares them and prints the first input where they disagree. The handbook template is a complete program with all four. Its example pair is the maximum subarray sum: Kadane's $O(n)$ scan against an $O(n^2)$ loop over all subarrays.

Listing 9.4 Stress tester: fast solve() against brute() on 100,000 tiny random inputs (handbook template)

```java
import java.util.*;

/** Compare a fast solve() with an obviously-correct brute() on thousands of tiny random inputs. */
public class Stress {
    public static void main(String[] args) {
        Random rnd = new Random(1);
        for (int iter = 1; iter <= 100000; iter++) {
            int n = 1 + rnd.nextInt(8);                    // tiny inputs expose bugs fastest
            int[] a = new int[n];
            for (int i = 0; i < n; i++) a[i] = rnd.nextInt(21) - 10;
            long fast = solve(a), slow = brute(a);
            if (fast != slow) {
                System.out.println("Mismatch on iteration " + iter + ": " + Arrays.toString(a));
                System.out.println("fast=" + fast + "  brute=" + slow);
                return;                                     // smallest failing case is now on screen
            }
        }
        System.out.println("All random tests passed");
    }

    // Example: maximum subarray sum. Replace both methods with your problem.
    static long solve(int[] a) {                           // Kadane, O(n)
        long best = Long.MIN_VALUE, cur = 0;
        for (int x : a) { cur = Math.max(x, cur + x); best = Math.max(best, cur); }
        return best;
    }

    static long brute(int[] a) {                           // every subarray, O(n^2)
        long best = Long.MIN_VALUE;
        for (int l = 0; l < a.length; l++) {
            long s = 0;
            for (int r = l; r < a.length; r++) { s += a[r]; best = Math.max(best, s); }
        }
        return best;
    }
}
```

The fixed seed `new Random(1)` makes every run produce the same inputs, so a failure can be reproduced after you change the code. The comparison is with the brute force, never with what you *expect* the answer to be.

EXAMPLE 9.1 A stress test finds an edge-case bug

Change one token of `solve`: initialise `best` to 0 instead of `Long.MIN_VALUE`. The samples of a typical problem still pass, because they contain a positive element. The stress tester prints:

```
Mismatch on iteration 11: [-4]
fast=0  brute=-4
```

An array whose elements are all negative has a negative maximum, but the buggy version never goes below its starting value 0. The input is a single number, so the cause is visible at once.

Shrinking a failing input

Often the first failing input is longer than it needs to be. A smaller one is easier to trace by hand, so it pays to shrink it automatically: try small changes (delete an element, delete two neighbours, move a value towards 0) and keep any change after which the two programs still disagree. Repeat until no change keeps the failure.

Listing 9.5 Shrinking a failing input while fast and slow still disagree

```
static int[] shrink(int[] a, ToLongFunction<int[]> fast, ToLongFunction<int[]> slow) {
    boolean changed = true;
    while (changed) {
        changed = false;
        for (int len = 1; len <= 2 && !changed; len++) {
            for (int i = 0; i + len <= a.length && !changed; i++) {     // try deleting a[i..i+len-1]
                int[] b = new int[a.length - len];
                for (int j = 0, k = 0; j < a.length; j++) if (j < i || j >= i + len) b[k++] = a[j];
                if (b.length > 0 && fast.applyAsLong(b) != slow.applyAsLong(b)) { a = b; changed =
true; }
            }
        }
        for (int i = 0; i < a.length && !changed; i++) {                // try moving a[i] towards 0
            int[] b = a.clone();
            b[i] = a[i] / 2;
            if (b[i] != a[i] && fast.applyAsLong(b) != slow.applyAsLong(b)) { a = b; changed = true; }
        }
    }
    return a;
}
```

Call it from the stress loop as `shrink(a, Stress::solve, Stress::brute)` once a mismatch is found.

LEMMA 9.3 **The shrinker stops, and its result still fails**

Listing 9.5 terminates, returns an input on which `fast` and `slow` disagree, and no single deletion of one or two neighbouring elements and no single halving of one value of the result keeps the disagreement.

Proof. Each accepted change either shortens the array or replaces some value v by $v/2$ (rounded toward zero) with $|v/2| < |v|$, so the pair (length, Σ |values|) decreases in the lexicographic sense and cannot decrease for ever. A change is accepted only if the programs still disagree, so the result disagrees. The loop ends only after a full round in which no change was accepted, which is the last claim. ∎

The result is *locally* minimal, not necessarily the smallest failing input, but in practice it is small enough to trace by hand.

EXAMPLE 9.2 **Shrinking a wrong idea**

The maximum sum of non-adjacent elements (all values ≥ 0) has a tempting wrong solution: take the better of “all even positions” and “all odd positions”. A stress test with values 0..9 fails at once, and the shrinker reduces the input:

```
Mismatch on iteration 1: [8, 7, 3, 4, 4, 4]
fast=15  brute=16
shrunk to [1, 0, 0, 1]   fast=1  brute=2
```

On [1, 0, 0, 1] the right answer takes both ends, positions 0 and 3, which have different parities. That one line shows why the idea is wrong, which the six-element input hid.

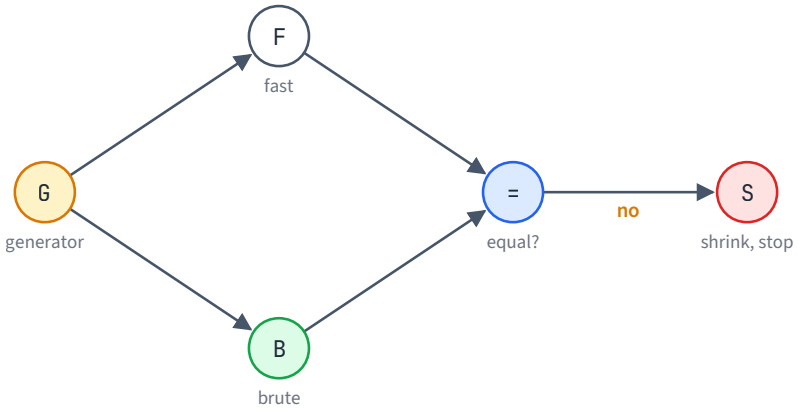


Figure 9.1 The stress-testing loop. The generator makes a tiny input and both programs answer it. If the answers are equal, the loop draws the next input; if not, it stops with a counterexample, which the shrinker then reduces.

Rules that make stress tests work

- **Tiny inputs.** $n \le 8$ and values in $-10..10$ find almost every bug, run the brute force instantly, and give counterexamples you can check on paper.
- **Generate what the statement allows, and nothing else.** If values are positive, generate positive values; if the array is a permutation, generate a permutation. A "failure" on an illegal input wastes time.
- **Add special generators.** Some bugs need a structure that random inputs rarely produce: all values equal, a sorted array, $n = 1$, the largest value. Run each for a few thousand iterations too.
- **Compare outputs that are unique.** If several answers are correct (any optimal subset, any valid order), compare the *value* of the answer, or write a checker that verifies the fast output.

Using it without a judge

The plan's Infosys repeat problems have no online judge, and the plan's advice for them is to run the samples in OneCompiler and then stress-test against a brute force. The template needs no input, so paste it, replace `solve` and `brute` with your two methods and the generator with one that matches the statement, and run it. Java requires a public class to live in a file of the same name, so if the online editor's file is called `Main.java`, rename `Stress` to `Main`.

Reading Codeforces verdicts

Codeforces has no partial marks, but the plan uses its verdict to estimate the share of tests you would pass on a test-case-scored paper. The tests are run in order and judging stops at the first failure, so "Wrong answer on test 7" means tests 1–6 passed, and the samples are normally the first test. "Time limit exceeded on test k" is the most informative verdict for a brute force: the small tests passed and the first large one did not. Two habits make the estimate honest. Record the first failing test number on the score sheet, and after the mock run your stress tester on the problem: a bug that only appears on special inputs (all equal, $n = 1$) would cost a few tests, while one that appears on most random inputs would cost most of them.

9.4 Estimating complexity from the limits

The limits in a statement are a hint about the intended solution. Java performs very roughly 10^8 simple operations per second, so the largest n tells you which complexities fit in a 1–2 second limit.

Table 9.2 From the largest n to a target complexity (the full version is Theory Appendix A)

Largest n	Target complexity	Steps at the limit	Typical technique
≤ 10	$O(n! \cdot n)$	$10 \cdot 10! \approx 3.6 \cdot 10^7$	all orders
≤ 20	$O(2^n \cdot n)$	$20 \cdot 2^{20} \approx 2.1 \cdot 10^7$	all subsets, bitmask DP
≤ 500	$O(n^3)$	$1.25 \cdot 10^8$	triple loops, interval DP
$\le 5{,}000$	$O(n^2)$	$2.5 \cdot 10^7$	all pairs, 2-D DP
$\le 10^5$ to $2 \cdot 10^5$	$O(n \log n)$ or $O(n \sqrt{n})$	$1.7 \cdot 10^6$ to $3.2 \cdot 10^7$	sorting, heaps, binary search, segment trees
$\le 10^6$	$O(n)$ or $O(n \log n)$	10^6 to $2 \cdot 10^7$	prefix sums, two pointers, stacks
$\ge 10^9$ (a value, not a length)	$O(\log n)$ or $O(\sqrt{n})$	30 or $3 \cdot 10^4$	binary search, number theory

Read the table both ways. Before you code, it tells you which idea the setter expects. During the mock, it tells you which tests your brute force will pass: an $O(n^2)$ brute force on a problem with $n \le 2 \cdot 10^5$ passes every test with n up to a few thousand.

PITFALL

Many Codeforces problems have t test cases with a bound on the *sum* of n over all of them (for example $t \le 10^4$ and $\Sigma\, n \le 2 \cdot 10^5$). The work must be proportional to that sum. Resetting a global array of size $2 \cdot 10^5$ with `Arrays.fill` in every test case costs $10^4 \times 2 \cdot 10^5 = 2 \cdot 10^9$ writes, a time-limit failure even though each test case is small. Allocate or clear only the first n entries of each test case.

Memory has a budget too. Count the cells and multiply by their size: an `int[]` of 10^7 entries takes 40 MB, a `long[5000][5000]` takes 200 MB, and the trie layouts of Chapter 8 differ by a factor of two for the same nodes. A `HashMap` costs several times its entries in object overhead (Chapter 2).

9.5 The mock protocol

A mock is only a measurement if it is taken under the same conditions every time. The plan fixes the rules for all five mocks.

CHECKLIST Mock rules

- Timer at 180 minutes, one sitting, phone in another room.
- Minutes 0–15: read every problem. On paper, write the limits, your idea and its complexity for each.
- Solve the problem you are surest of first. That is not always Q1.
- Test on the samples with Codeforces Custom test before you submit.
- By minute 150, every problem has a submission, even if it is only a brute force.

The plan's exam-day schedule divides the 180 minutes the same way, and the mock is the place to practise it.

Table 9.3 The 180 minutes, from the plan's exam-day page

Minutes	Do
0–15	Read every problem with its limits and samples. Write the idea, its complexity and the brute-force fallback for each, and rank them by how sure you are.
15–60	Solve the surest problem fully. Run the samples and two edge cases you write yourself ($n = 1$, all values equal, largest values). Submit.
60–120	The second problem, same routine. Still stuck at minute 100? Submit a brute force and move on.
120–165	The hard problem. No full idea after 15 minutes: write the brute force for small limits, collect those marks, then try to optimise.
165–180	Make sure every problem has a submission that compiles. Do not touch code that already passes.

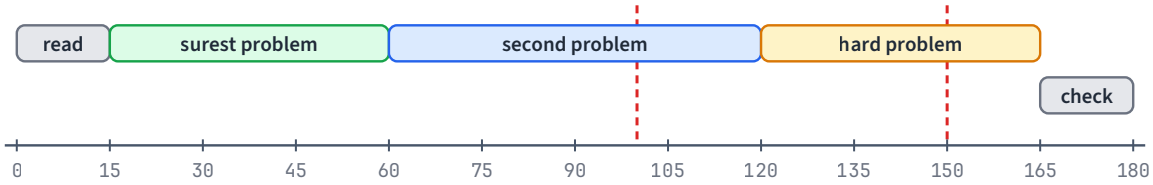


Figure 9.2 The 180-minute plan as a timeline. The dashed lines mark minute 100 (submit a brute force for Q2 if stuck) and minute 150 (every problem has a submission).

Mock 1

Mock 1 uses three Codeforces problems chosen by rating to match the three steps of the paper: CF 1343C (Alternating Subsequence, rated 1200), CF 1475D (Cleaning the Phone, 1800) and CF 1251D (Salary Changing, 1900), with weights 20, 30 and 50. None of them appears in the daily lists. Workbook Chapter 9 has the planning grid, the score sheet and the post-mortem table.

Grading yourself

For each problem, estimate the share of tests your best submission would pass (1 for an accepted solution, an estimate from the verdict otherwise, 0 for nothing). The weighted score is

$$\text{score} = 20 \cdot \text{share}(Q1) + 30 \cdot \text{share}(Q2) + 50 \cdot \text{share}(Q3).$$

For example, Q1 accepted, Q2 failing on about half of its tests and a brute force for Q3 that passes roughly a fifth gives $20 + 15 + 10 = 45$.

> **INFOSYS LENS**
>
> The plan's checkpoint for Mock 1, its own estimate rather than an Infosys figure, is Q1 fully solved and Q2 at least half the tests. If you are below it, the plan says to skip the bonus problems in week 2 and redo the core problems of your weakest day.

After the mock the plan allows about 60 minutes: enter the scores, read the editorial of every unsolved problem, close it and write the solution yourself, log the cause of each failure, and pick the weakest topic to redo two of its core problems the next day.

9.6 The post-mortem and an error taxonomy

A mistake you cannot name is a mistake you will repeat. The plan's error log classifies every failure into one of nine causes, and each entry gets a **rule**, a sentence that would have prevented it. The examples below are small Java fragments that compile and contain the error.

Misread statement. The code solves a different problem from the one asked. The most common forms are 0-based versus 1-based output, "subsequence" read as "subarray", and "at most" read as "exactly".

```
// The statement asks for the 1-based position of the maximum.
int[] a = {4, 9, 2};
int best = 0;
for (int i = 1; i < a.length; i++) if (a[i] > a[best]) best = i;
System.out.println(best);          // prints 1, expected 2: print best + 1
```

Rule: before coding, write the output format of the first sample next to your own trace of it.

Wrong idea. The algorithm is implemented correctly but the idea is false. Greedy choices are the usual suspects: for coins {1, 3, 4} and the amount 6, "always take the largest coin that fits" uses 4 + 1 + 1 (three coins) while 3 + 3 uses two.

```
int[] coins = {4, 3, 1};           // largest first
int amount = 6, used = 0;
for (int c : coins) while (amount >= c) { amount -= c; used++; }
System.out.println(used);          // 3, but the optimum is 2
```

Rule: prove a greedy choice (an exchange argument, Chapter 4) or stress-test it before trusting it.

Overflow / mod. A product or sum exceeds `int` and wraps silently, or a modulus is taken too late.

```
int x = 100000, y = 100000;
long bad = x * y;                  // 1410065408: the int product wrapped before widening
long good = (long) x * y;          // 10000000000
```

Rule: default to `long`; cast the first factor; reduce mod $10^9 + 7$ after every multiplication.

Off-by-one. A loop bound, an inclusive versus half-open range, or a binary search boundary is one step wrong (Chapter 6 lists four classic ones).

```
int[] a = {3, 1, 2};
long sum = 0;
try {
    for (int i = 0; i <= a.length; i++) sum += a[i];      // i = 3 is out of bounds
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("crash: use i < a.length");
}
```

Rule: write every range as half-open $[l, r)$ unless the statement forces otherwise, and test $n = 1$.

TLE / complexity. The program is correct and too slow, either because the idea is too slow for the limits or because of a hidden cost such as per-test resets, `String` concatenation in a loop or `Scanner` on 10^6 numbers.

```
int t = 10000;                     // test cases, with the sum of n bounded by 2e5
int[] cnt = new int[200001];
for (int tc = 0; tc < t; tc++) {
    Arrays.fill(cnt, 0);           // 2e5 writes per test: 2e9 in total
    // ... the work for this test case uses only its own n entries ...
}
```

Rule: before submitting, multiply the cost of one test case by t, and clear only what you used.

Input reading. The program reads the input in the wrong order or format. The plan's walkthrough of the sample paper found one where the written format and the samples disagree, and only the samples' order was right. With `Scanner`, `nextLine()` right after `nextInt()` returns the empty rest of the line.

```
Scanner sc = new Scanner("3\nabc def\n");
int k = sc.nextInt();
String line = sc.nextLine();        // "" : the rest of the first line, not "abc def"
```

Rule: check your reading code against every sample before writing the logic; use FastScanner (Chapter 1).

Edge case. The idea is right for typical inputs and wrong for a special one: $n = 1$, all values equal, all negative, an empty answer, the largest values. Example 9.1 is this kind.

```
int[] a = {-4};
long best = 0, cur = 0;             // should start at Long.MIN_VALUE
for (int v : a) { cur = Math.max(v, cur + v); best = Math.max(best, cur); }
System.out.println(best);           // 0, but the answer is -4
```

Rule: for every problem, test $n = 1$, all equal and the extreme values before submitting.

Stack overflow. Recursion deeper than the call stack allows, typically a DFS on a tree that is a long chain. On the sandbox's JVM with default settings, a simple recursive method overflowed at a depth of about 28,000, and `depthRec` below failed on a chain of 10^5 nodes while `depthLoop` returned 99,999.

Listing 9.6 Recursion versus a loop on a long chain

```
static int depthRec(int[] next, int v) {          // depth of the recursion = length of the chain
    return next[v] == -1 ? 0 : 1 + depthRec(next, next[v]);
}

static int depthLoop(int[] next, int v) {         // the same walk with a loop: no call stack needed
    int d = 0;
    for (; next[v] != -1; v = next[v]) d++;
    return d;
}
```

Rule: if a recursion can be deeper than about 10^4, rewrite it with an explicit stack or a BFS order (Chapter 18), or run it on a thread with a large stack.

Ran out of time. A problem has no submission at the end. This is a process error, not a coding error.

Rule: by minute 150 every problem has a submission, even a brute force; stop debugging Q2 at minute 100.

> **KEY IDEA**
>
> The rule matters more than the label. "Be careful with overflow" is not a rule; "cast the first factor of every product of two ints to long" is. A good rule is specific enough that you could check your code against it in ten seconds.

9.7 A map of Part I

The first eight chapters taught a pattern each, or several. In the mock you will not be told which one applies, so the useful form of that knowledge is a list of **signals**: phrases and limits in a statement that point to a pattern.

Table 9.4 The patterns of Chapters 1–8 and the signals that suggest them

Chapter	Pattern	Signals in the statement	Typical cost
1	Java collections, fast input	"have I seen", "how many of each", "smallest so far", "nearest key below"; more than about 10^5 numbers to read	O(1) or O(log n) per operation
2	prefix sums + hash map	count subarrays with sum, remainder or XOR equal to something; negative values allowed; many range sums	O(n)
2	difference array	many "add v to $a[l..r]$", read once at the end; how many intervals cover each point	O(n + updates)
3	two pointers	pairs or triples with a target sum after sorting; "count pairs with sum ≤ X"	O(n) after sorting
3	sliding window	longest or shortest subarray with a condition that stays true when the window shrinks	O(n)
3	monotonic deque	maximum or minimum of every window of size k; DP over the last k positions	O(n)
4	sorting + greedy on intervals	meetings, trains, bookings; merge, most you can keep, fewest rooms or platforms	O(n log n)
5	greedy with a heap	options unlock over time; "in any order"; "at most k times"	O(n log n)
6	binary search	sorted data; first or last position; "how many ≤ x"	O(log n) per query
6	search on the answer	"minimise the maximum", "maximise the minimum", smallest x such that a check passes	O(n log V)
7	monotonic stack	next or previous greater or smaller; sum over all subarrays of the minimum or maximum; largest rectangle; remove k digits	O(n)
8	bits and XOR	XOR, AND, OR in the statement; sums of XORs over pairs; values below about 2^{20}	O($B \cdot n$)
8	binary trie	maximum XOR of a pair; pairs with XOR below or within a bound	O(B) per query
8	XOR basis	maximum XOR of any subset; which values a subset can reach	O(B) per insert
9	brute force + stress test	every problem first; small limits ($n \le 20$, $n \le 10$); a fast solution that fails hidden tests	as in Table 9.1

EXAM TIP

When no signal fires, go back to the brute force and print its answers for $n = 1, 2, 3, \ldots$ Many statements that look new turn into a known pattern once the small answers are on the page.

9.8 After the mock

PITFALL Read only after the mock

This section gives the key idea of each Mock 1 problem. Reading it before the mock wastes the measurement. Take the mock first, grade it, write your own post-mortem, and only then compare.

Each idea below was checked with my own solution, which passes the problem's samples and agrees with a brute force on 20,000 random small inputs per problem. Write and test your own version before you compare it with an editorial.

CF 1343C (Alternating Subsequence). Cut the sequence into maximal blocks of equal sign. A longest alternating subsequence takes exactly one element from every block, since two elements from one block

cannot be neighbours and skipping a block shortens the result. To maximise the sum, take the largest element of each block. One pass in $O(n)$; the sum needs `long`.

CF 1475D (Cleaning the Phone). Split the applications by their convenience points (1 or 2) and sort each group by memory, largest first; for a fixed number of applications from each group, taking the largest ones is best. For each count j of 2-point applications, the fewest 1-point applications that free enough memory is found from prefix sums, either by binary search (Chapter 6) or with a pointer that only moves left as j grows. The answer is the minimum of $i + 2j$, or -1 if all the memory together is not enough.

CF 1251D (Salary Changing). Search on the answer (Chapter 6): can the median be at least M? Everyone is paid at least their lower bound; at least $(n + 1)/2$ employees must earn $\geq M$; employees whose upper bound is below M cannot help, and among the others the cheapest to raise are those with the largest lower bounds, so take them greedily. The check is monotone because a lower target never costs more, so the largest feasible M is found by binary search with the midpoint rounded up.

Summary

- Marks come from passed tests, so a correct brute force is worth real marks; write it first, in 5–10 minutes, for every problem.
- The standard enumerations and their limits: pairs and subarrays $O(n^2)$ for $n \leq 10^4$, subsets $O(2^n \cdot n)$ for $n \leq 20$, orders $O(n! \cdot n)$ for $n \leq 10$.
- A bitmask from 0 to $2^n - 1$ visits every subset once; `nextPermutation` from the sorted array visits every distinct order once.
- A stress tester compares the fast solution with the brute force on tiny random inputs with a fixed seed and prints the first counterexample; a shrinker reduces it to a locally minimal one.
- The largest n in the limits points to the intended complexity; with multiple test cases, the bound on the sum of n is what the work must respect.
- Take every mock under the same rules: read everything first, solve the surest problem first, a submission for every problem by minute 150.
- Grade by weighted share of tests, and log each failure under one of nine causes with a specific rule that would have prevented it.
- Revise Part I by signals: the phrases and limits that point to each pattern.

Key terms

brute force (complete search) p. 123
test oracle p. 124
Stress testing p. 127

Review questions

1. Why is a correct $O(n^2)$ solution worth writing for a problem with $n \leq 2 \cdot 10^5$? Which tests can it pass?
2. A problem has $n \leq 18$. Which enumeration fits, and roughly how many steps does it take?
3. Why must a stress tester compare with a brute force rather than with answers you work out yourself?
4. In Listing 9.2, why does the method look for the *rightmost* element larger than $a[i]$? What would go wrong with the leftmost one?
5. Why does Listing 9.5 keep only changes after which the two programs still disagree? Could it ever return an input on which they agree?
6. A Codeforces submission gets “Wrong answer on test 3” and another gets “Time limit exceeded on test 12”. Which do you expect to score more on a test-case-scored paper, and why can you not be sure?

7. A problem has $t \le 10^4$ test cases and $\Sigma n \le 2 \cdot 10^5$. Your solution is $O(n \log n)$ per test case but allocates `new int[200001]` in each. Is it fast enough? What about clearing a global array with `Arrays.fill` each time?
8. Classify each bug: (a) `int` sum of 10^5 values up to 10^9; (b) printing indices from 0 when the statement counts from 1; (c) a greedy that sorts by the wrong key; (d) a recursive DFS on a 10^5-node path.
9. Write the weighted score of a mock with Q1 accepted, Q2 passing about 30 % of its tests and nothing for Q3. Is it on track by the Mock 1 checkpoint?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 5 "Complete search" (generating subsets and permutations, backtracking, pruning, meet in the middle) and Chapter 2 "Time complexity" (estimating efficiency from the input size). Free at cses.fi/book.
- Algorithms for Competitive Programming, "Enumerating submasks of a bitmask" (cp-algorithms.com/algebra/all-submasks.html), the next step after enumerating all masks.
- Java 8 API documentation for `java.util.Random` (`nextInt(bound)`, seeded constructors) and `java.util.function.ToLongFunction`, docs.oracle.com/javase/8/docs/api/.
- Workbook Chapter 9 has the mock sheet, the post-mortem table and exercises on brute forces and bug classification.

Problems

These are the problems for Day 9. Work them in Workbook Chapter 9, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
9.P1	CF 1343C · Alternating Subsequence	CF 1200	CORE
9.P2	CF 1475D · Cleaning the Phone	CF 1800	CORE
9.P3	CF 1251D · Salary Changing	CF 1900	CORE

PART II

Number Theory, Dynamic Programming and First Graphs

Days 10–16 · gcd and sieves, five chapters of DP, BFS and topological order

The third problem of an Infosys paper is usually a counting or optimisation question, and its solution is usually number theory, dynamic programming or both. Part II starts with modular arithmetic and factorisation, then develops dynamic programming from first principles: states, transitions, knapsacks, two-string tables, increasing subsequences, intervals and games. Breadth-first search and topological order close the part so that graphs are familiar before the second mock.

10 Number Theory for Programmers

11 Dynamic Programming I: States, Transitions and Knapsacks

12 DP II: two-string DP

13 Dynamic Programming III: Increasing Subsequences and Extra Dimensions

14 Dynamic Programming IV: Divisors, Intervals and Games

15 Graphs I: Breadth-First Search, Depth-First Search and Topological Order

16 Analysing a Problem Before You Code

CHAPTER 10 · DAY 10 OF 29 · MONDAY 16 NOVEMBER 2026 · LEARNING DAY

10

Number Theory for Programmers

gcd, sieves, fast power, inverses, nCr, and the square-free kernel.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · gcd, fast power, inverse, nCr, Sieve, prime factors, square-free kernel
Before this chapter · Chapter 2 (Math.floorMod, counting with maps) and Chapter 8 (bit masks)
Practice · Workbook Chapter 10 · 7 problems · core ≈ 4.1 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- compute gcd and lcm with Euclid's algorithm, prove it correct, and bound its number of steps;
- keep every intermediate value of a calculation modulo 10^9+7 inside a `long`, including subtraction and fast exponentiation in $O(\log e)$;
- compute modular inverses with Fermat's little theorem and binomial coefficients in $O(1)$ after $O(n)$ precomputation, and switch to Pascal's triangle when the modulus is not prime;
- sieve primes and smallest prime factors up to 10^7 and factorise any value in that range in $O(\log x)$;
- turn "the product is a perfect square" into equality of square-free kernels, for values up to 10^9;
- bound gcd scans over subarrays by $O(\log V)$ distinct values and loops over multiples by $O(V \log V)$.

Counting problems rarely ask for the exact count. They ask for it "modulo $10^9 + 7$", because the true number has hundreds of digits. Optimisation problems hide a divisibility condition inside a story: two values "fit together" when their product is a perfect square, a group is "good" when its gcd is exactly k. Each of these needs a small piece of number theory, and each piece comes with a Java trap: a product that overflows before you reduce it, a remainder that comes out negative, an inverse that does not exist because the modulus is not prime.

This chapter collects the number theory a contest programmer needs and proves each piece, because the proofs tell you exactly when a formula may be used. It starts with the greatest common divisor and Euclid's algorithm, moves to arithmetic modulo m, fast exponentiation, inverses and binomial coefficients, then to primes, sieves and factorisation. The last three sections are the ones that appear in disguise in harder problems: the square-free kernel, which decides when a product is a perfect square; the small number of distinct gcds among subarrays; and the harmonic bound that makes loops over multiples fast. Everything here is reused in the counting DPs of Chapters 11 to 14 and in the Infosys problems of Part IV.

10.1 Divisibility and the greatest common divisor

All numbers in this chapter are integers. An integer d **divides** a, written $d \mid a$, when $a = d \cdot k$ for some integer k. Every integer divides 0, and 1 divides everything.

DEFINITION 10.1 Greatest common divisor and least common multiple

For integers a and b, not both zero, the **greatest common divisor** $\gcd(a, b)$ is the largest integer that divides both. By convention $\gcd(0, 0) = 0$. For positive a and b, the least common multiple $\operatorname{lcm}(a, b)$ is the smallest positive integer that both divide.

Two numbers are **coprime** when their gcd is 1. Signs do not matter: $\gcd(-12, 18) = \gcd(12, 18) = 6$.

Euclid's algorithm

Trying every candidate divisor costs $O(\min(a, b))$ operations, which is 10^9 for values near 10^9. Euclid's algorithm replaces the pair (a, b) by a smaller pair with the same gcd, and the pairs shrink very quickly.

LEMMA 10.1 **Euclid's step**

For $b \neq 0$, write $a = qb + r$ with $r = a \bmod b$. Then the common divisors of a and b are exactly the common divisors of b and r. In particular $\gcd(a, b) = \gcd(b, a \bmod b)$.

Proof. If d divides a and b, it divides $a - qb = r$. Conversely, if d divides b and r, it divides $qb + r = a$. The two sets of common divisors are equal, so their largest elements are equal. ∎

The proof uses only $a = qb + r$, which also holds for Java's `%` on negative numbers (`a / b` truncates towards zero and `a % b` takes the sign of `a`). So the lemma holds for every sign, and the template only needs `Math.abs` at the end.

Listing 10.1 Euclid's algorithm and the least common multiple

```
static long gcd(long a, long b) {
    while (b != 0) {
        long t = a % b;
        a = b;
        b = t;
    }
    return Math.abs(a);
}

static long lcm(long a, long b) {
    return a / gcd(a, b) * b;                    // divide first so it doesn't overflow
}
```

THEOREM 10.2 **Correctness of Euclid's algorithm**

For all `long` values a and b, `gcd(a, b)` in Listing 10.1 terminates and returns $\gcd(a, b)$.

Proof. Let g be the gcd of the original arguments. The invariant is "the current pair (a, b) has the same common divisors as the original pair". It holds before the loop, and each iteration replaces (a, b) by $(b, a \bmod b)$, which keeps it by Lemma 10.1. Each iteration makes $|b|$ strictly smaller, because $|a \bmod b| < |b|$, so after finitely many iterations $b = 0$. Every integer divides 0, so the common divisors of $(a, 0)$ are the divisors of a, and the largest of them is $|a|$. By the invariant, $|a| = g$. ∎

EXAMPLE 10.1 **gcd(1071, 462)**

Each row is one iteration of the loop: $q = \lfloor a/b \rfloor$ and $r = a \bmod b$.

```
   a      b     q     r = a mod b
1071    462    2     147          1071 = 2*462 + 147
 462    147    3      21           462 = 3*147 + 21
 147     21    7       0           147 = 7*21
  21      0                        b = 0: the answer is 21
```

Three iterations replace a search over 462 candidates. Check: $1071 = 21 \cdot 51$ and $462 = 21 \cdot 22$, and 51 and 22 are coprime.

How many iterations?

The remainders fall fast, and the reason is a one-line inequality.

LEMMA 10.3 Remainders halve

If $0 < b \le a$, then $a \bmod b < a/2$.

Proof. If $b \le a/2$, then $a \bmod b < b \le a/2$. If $b > a/2$, then $a < 2b$, so $\lfloor a/b \rfloor = 1$ and $a \bmod b = a - b < a - a/2 = a/2$. ∎

Two iterations turn (a, b) into $(a \bmod b, \ldots)$, so the first number at least halves every two iterations. A positive number can halve at most $\log_2 a$ times before it drops below 1, so Euclid's algorithm runs at most about $2 \log_2 a$ iterations: about 60 for values up to 10^9.

The true worst case is better, and it is attained by consecutive Fibonacci numbers $F_1 = F_2 = 1$, $F_{k+1} = F_k + F_{k-1}$. On a Fibonacci pair every quotient is 1, so each step subtracts as little as possible (Figure 10.1).

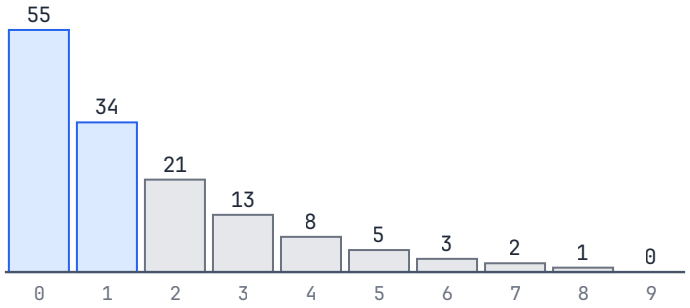


Figure 10.1 The numbers that pass through Euclid's algorithm on $(55, 34) = (F_{10}, F_9)$. Every quotient is 1, so each value is the sum of the next two and the sequence shrinks by only a factor of about 1.6 per step: 8 iterations for a pair below 60.

THEOREM 10.4 Lamé's bound

If Euclid's loop performs $k \ge 1$ iterations on $a > b > 0$, then $b \ge F_{k+1}$ and $a \ge F_{k+2}$. Consequently $k \le 1 + \log_\varphi b \approx 1 + 1.44 \log_2 b$, where $\varphi = (1 + \sqrt{5})/2 \approx 1.618$.

Proof. By induction on k. For $k = 1$: $b \ge 1 = F_2$, and $a > b$ gives $a \ge 2 = F_3$. For $k \ge 2$, the first iteration produces (b, r) with $r = a \bmod b$. The loop continues, so $r > 0$, and $b > r$, so the pair (b, r) satisfies the hypothesis and needs $k - 1 \ge 1$ more iterations. By induction $b \ge F_{k+1}$ and $r \ge F_k$. Because $a > b$, the quotient is at least 1, so $a = qb + r \ge b + r \ge F_{k+1} + F_k = F_{k+2}$.

For the logarithmic form, $F_{k+1} \ge \phi^{k-1}$ for every $k \ge 1$, again by induction: $F_2 = 1 = \phi^0$, $F_3 = 2 \ge \phi$, and using $\phi^2 = \phi + 1$, $F_{k+1} = F_k + F_{k-1} \ge \phi^{k-2} + \phi^{k-3} = \phi^{k-3}(\phi + 1) = \phi^{k-1}$. So $b \ge \phi^{k-1}$, which is $k \le 1 + \log_\phi b$. ∎

In numbers: the worst pair with both values at most 10^9 is $(F_{44}, F_{43}) = (701\,408\,733, 433\,494\,437)$, which takes 42 iterations; over a million random pairs below 10^9 the most any pair needed was 34. For `long` values up to 10^{18} the worst case is 85 iterations. A gcd is therefore an $O(\log \min(a, b))$ operation, and you can call it inside loops of 10^6 iterations without thinking twice.

gcd(0, *x*) = *x*, and the gcd of many numbers

Because every integer divides 0, $\gcd(0, x) = |x|$. This makes 0 the **identity** for gcd, just as 0 is for addition: to fold an array, start from `g = 0` and apply `g = gcd(g, a[i])`. The gcd is also associative and commutative, since $\gcd(a, \gcd(b, c))$ and $\gcd(\gcd(a, b), c)$ are both the largest common divisor of all three numbers,

so you may combine the elements in any order and any grouping. A segment tree of gcds (Chapter 20) uses the same fact: a position that should be ignored can store 0.

lcm without overflow

For positive a and b, $\operatorname{lcm}(a, b) \cdot \gcd(a, b) = a \cdot b$; Section 10.5 proves it from prime factorisations. The tempting formula `a * b / gcd(a, b)` computes the product first, and the product can overflow even when the answer fits.

```
a = 6 000 000 000, b = 4 000 000 000, gcd = 2 000 000 000  (long)
a * b / g   = 2 776 627 963        wrong: a * b = 2.4e19 overflowed
a / g * b   = 12 000 000 000       correct
```

Dividing first keeps every intermediate value at most the final answer. It cannot help when the answer itself is too large: $\operatorname{lcm}(100\,000, 99\,999) = 9\,999\,900\,000$ does not fit in an `int` whichever order you use, and `int` arithmetic silently returns 1 409 965 408. The lcm of many numbers grows quickly, so compute it in `long` and check the bound.

The extended algorithm

Euclid's algorithm can also report *how* the gcd is built from a and b. This is what makes inverses modulo any number possible (Section 10.4).

THEOREM 10.5 Bézout's identity

For integers $a, b \geq 0$, not both zero, there are integers x and y with $ax + by = \gcd(a, b)$. This is **Bézout's identity**.

Listing 10.2 Extended Euclid and an inverse modulo any m

```java
// Returns g = gcd(a, b) for a, b >= 0 and stores x, y with a*x + b*y == g in xy[0], xy[1].
static long extGcd(long a, long b, long[] xy) {
    if (b == 0) {
        xy[0] = 1;
        xy[1] = 0;
        return a;
    }
    long g = extGcd(b, a % b, xy);              // now b*xy[0] + (a % b)*xy[1] == g
    long x = xy[1], y = xy[0] - (a / b) * xy[1];
    xy[0] = x;
    xy[1] = y;
    return g;
}

// Inverse of a modulo any m >= 2, or -1 when gcd(a, m) != 1 (no inverse exists).
static long modInverse(long a, long m) {
    long[] xy = new long[2];
    long g = extGcd(Math.floorMod(a, m), m, xy);
    return g == 1 ? Math.floorMod(xy[0], m) : -1;
}
```

Proof. Listing 10.2 constructs the coefficients by induction on the recursion. When $b = 0$, $a \cdot 1 + 0 \cdot 0 = a = \gcd(a, 0)$. Otherwise the recursive call returns $g = \gcd(b, a \bmod b)$, which equals $\gcd(a, b)$ by Lemma 10.1, together with x', y' such that $bx' + (a \bmod b)y' = g$. Substitute $a \bmod b = a - \lfloor a/b \rfloor b$: $g = bx' + (a - \lfloor a/b \rfloor b)y' = a \cdot y' + b \cdot (x' - \lfloor a/b \rfloor y')$. So $x = y'$ and $y = x' - \lfloor a/b \rfloor y'$, which is exactly what the code stores. ∎

The recursion depth equals the number of Euclid iterations, at most 85 for `long` values, so recursion is safe here. Running it on (240, 46), from the deepest call outwards:

```
call         q    x    y    check a*x + b*y
(2, 0)            1    0    2*1 + 0*0 = 2
(4, 2)       2    0    1    4*0 + 2*1 = 2
(6, 4)       1    1   -1    6*1 + 4*(-1) = 2
(10, 6)      1   -1    2    10*(-1) + 6*2 = 2
(46, 10)     4    2   -9    46*2 + 10*(-9) = 2
(240, 46)    5   -9   47    240*(-9) + 46*47 = 2
```

10.2 Modular arithmetic

DEFINITION 10.2 Congruence and residue

Let $m \geq 1$. Integers a and b are **congruent** modulo m, written $a \equiv b \pmod m$, when m divides $a - b$. Every integer is congruent to exactly one **residue** in $0..m-1$, namely `Math.floorMod(a, m)`.

Working modulo m means replacing each number by its residue. That is only useful if the replacement does not change the answer, and for the three ring operations it does not.

THEOREM 10.6 Congruence respects +, − and ×

If $a \equiv a'$ and $b \equiv b' \pmod m$, then $a + b \equiv a' + b'$, $a - b \equiv a' - b'$ and $ab \equiv a'b' \pmod m$.

Proof. Write $a = a' + sm$ and $b = b' + tm$. Then $(a + b) - (a' + b') = (s + t)m$ and $(a - b) - (a' - b') = (s - t)m$. For the product, $ab - a'b' = a(b - b') + b'(a - a') = atm + b'sm$, also a multiple of m. ∎

By induction over the operations, any expression built from +, − and × gives the same residue whether you reduce once at the end or after every single operation. That is the licence to write `% MOD` after every step and keep the numbers small. Other operations do not have this licence.

Table 10.1 Which operations may be done on residues

Operation	Reduce first?	Example or fix
$a + b$, $a - b$, $a \cdot b$	yes	Theorem 10.6
a / b	no	$6 \equiv 2 \pmod 4$, but $6/2 = 3$ and $2/2 = 1$ differ mod 4; multiply by an inverse instead (Section 10.4)
max, min, comparisons	no	$\max(5, 8) \bmod 7 = 1$, but $\max(5 \bmod 7, 8 \bmod 7) = 5$; compare true values, or exact counts
exponent e in a^e	no	$2^5 \bmod 3 = 2$, but $2^{5 \bmod 3} \bmod 3 = 1$; for prime p reduce e mod $p - 1$ (Section 10.4)
the base a in a^e	yes	it is repeated multiplication

Why $10^9 + 7$

The modulus $10^9 + 7 = 1\,000\,000\,007$ is chosen for three properties. It is **prime**, so every non-zero residue has an inverse (Section 10.4). It is **below 2^{30}**, so the sum of two residues is at most $2\,000\,000\,012$, which still fits in an `int` (maximum $2\,147\,483\,647$). And the product of two residues is at most $(10^9 + 6)^2 = 1\,000\,000\,012\,000\,000\,036 \approx 1.0 \cdot 10^{18}$, which fits in a `long` (maximum about $9.22 \cdot 10^{18}$) with room to spare. A product of *three* residues does not fit, so reduce after every multiplication. The other common prime modulus, 998 244 353, has the same properties.

JAVA NOTE

Two Java traps break modular code silently. First, an `int` times an `int` is computed in `int` before the `%` is applied. With `int x = 1_000_000, y = 1_000_000`, the expression `x * y % MOD` evaluates to −727 379 968, while `(long) x * y % MOD` gives the correct 999 993 007. Declare residues as `long`, or cast one factor. Second, `%` keeps the sign of the left operand. With $a = 5$ and $b = 1\,000\,000\,006$, `(a - b) % MOD` is −1 000 000 001. Use `((a - b) % MOD + MOD) % MOD` or `Math.floorMod(a - b, MOD)`, which both give 6.

The three idioms, as they appear in every counting solution:

```
final long MOD = 1_000_000_007L;
long a = 999_999_999L, b = 123_456_789L;     // residues in 0..MOD-1
long sum  = (a + b) % MOD;
long diff = ((a - b) % MOD + MOD) % MOD;      // never negative
long prod = a * b % MOD;                      // a * b < 1.0e18 fits in a long
```

INFOSYS LENS

The plan's description of the Infosys paper lists typical limits of $n \le 10^5$ "with answers mod 10^9+7", and its exam rules say: default to `long`, and take the modulus after every multiplication. Both habits come from this section.

10.3 Fast exponentiation

Computing $a^e \bmod m$ by multiplying e times costs $O(e)$, hopeless for $e = 10^{15}$. The fix uses the binary representation of e. Write $e = \Sigma\, b_k\, 2^k$ with bits $b_k \in \{0, 1\}$. Then

$a^e = \prod$ over the k with $b_k = 1$ of a^{2^k},

and each power $a^{2^{k+1}} = (a^{2^k})^2$ is one squaring away from the previous one. So you walk through the bits of e from the lowest, squaring a running base at every step and multiplying it into the result when the bit is 1. This is **binary exponentiation**.

Listing 10.3 Binary exponentiation (fast power)

```
1  static final long MOD = 1_000_000_007L;
2
3  static long power(long base, long exp, long m) {  // base^exp mod m in O(log exp)
4      long r = 1 % m;
5      base %= m;
6      if (base < 0) base += m;
7      while (exp > 0) {
8          if ((exp & 1) == 1) r = r * base % m;
9          base = base * base % m;
10         exp >>= 1;
11     }
12     return r;
13 }
```

THEOREM 10.7 **Correctness and cost of fast power**

Let a and $e \ge 0$ be the original arguments. At the start of every iteration, $r \cdot base^{exp} \equiv a^e \pmod m$. Hence `power` *returns $a^e \bmod m$, after $\lfloor \log_2 e \rfloor + 1$ iterations for $e \ge 1$, each with at most two multiplications.*

Proof. Before the loop, $r = 1$ (as a residue), base $\equiv a$ and exp $= e$, so the invariant holds. Suppose it holds and exp > 0. If exp is odd, the iteration sets $r \leftarrow r \cdot \text{base}$, base $\leftarrow \text{base}^2$ and exp $\leftarrow (\text{exp} - 1)/2$, and $r \cdot \text{base} \cdot (\text{base}^2)^{(\text{exp}-1)/2} = r \cdot \text{base}^{\text{exp}}$. If exp is even, r is unchanged, and $r \cdot (\text{base}^2)^{\text{exp}/2} = r \cdot \text{base}^{\text{exp}}$. In both cases `exp >>= 1` computes $\lfloor \text{exp}/2 \rfloor$ for non-negative values, and every product is reduced mod m, which Theorem 10.6 allows. When the loop stops, exp $= 0$ and $r \equiv a^e$. Each iteration removes one bit from exp, so the number of iterations is the bit length of e, $\lfloor \log_2 e \rfloor + 1$. ∎

Three details in the template are deliberate. `r = 1 % m` makes the answer 0 when $m = 1$. `base %= m` followed by the sign fix makes base a residue in $0..m-1$ even when the caller passes a negative or huge base. And because base · base is computed in `long`, the modulus must satisfy $m^2 < 9.22 \cdot 10^{18}$, that is, m below about $3.03 \cdot 10^9$; every modulus you will meet is far below that. For $e = 10^{15}$ the loop runs 50 times.

EXAMPLE 10.2 **$3^{13} \bmod 1000$**

13 is 1101 in binary, so $3^{13} = 3^1 \cdot 3^4 \cdot 3^8$. Each row shows one iteration: the bit read, then r and base after the update.

```
exp  bit   r after   base after (squared)   exp after
 13   1        3          9                    6
  6   0        3         81                    3
  3   1      243        561                    1       (81 * 81 = 6561 -> 561)
  1   1      323        721                    0       (243 * 561 = 136323 -> 323)
```

The answer is 323, and indeed $3^{13} = 1\,594\,323$. Four iterations replace twelve multiplications.

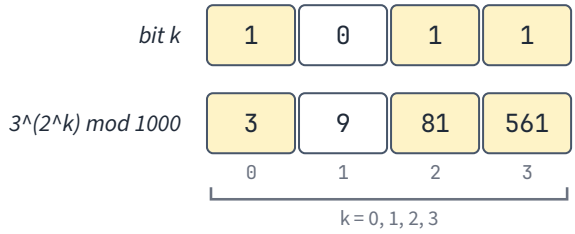


Figure 10.2 Fast power for $3^{13} \bmod 1000$. The bits of $13 = 1101_2$ are read from the lowest (left) to the highest. The second row holds $3^{(2^k)} \bmod 1000$, each the square of the one before. The answer multiplies the powers where the bit is 1: $3 \cdot 81 \cdot 561 \equiv 323$.

EXAMPLE 10.3 **Count Good Numbers (LeetCode 1922)**

A digit string of length n is *good* when every digit at an even index (0, 2, 4, ...) is even and every digit at an odd index is a prime digit (2, 3, 5 or 7). Leading zeros are allowed. Count the good strings of length $n \le 10^{15}$, modulo $10^9 + 7$.

The positions are independent, so by the product rule the count is 5 for each even index times 4 for each odd index. A string of length n has $\lceil n/2 \rceil = (n+1)/2$ even indices and $\lfloor n/2 \rfloor$ odd indices (integer division), so the answer is $5^{\lceil n/2 \rceil} \cdot 4^{\lfloor n/2 \rfloor} \bmod 10^9 + 7$. For $n = 1, 2, 3, 4$ this gives 5, 20, 100, 400, and for $n = 50$ it gives 564 908 303. A brute force that enumerates all 10^n strings agrees for every $n \le 6$.

Listing 10.4 LeetCode 1922: two fast powers

```java
class Solution {
    static final long MOD = 1_000_000_007L;

    public int countGoodNumbers(long n) {
```

```
        long evenPositions = (n + 1) / 2;          // indices 0, 2, 4, ...: digits 0, 2, 4, 6, 8
        long oddPositions = n / 2;                 // indices 1, 3, 5, ...: digits 2, 3, 5, 7
        return (int) (power(5, evenPositions) * power(4, oddPositions) % MOD);
    }

    long power(long base, long exp) {              // Listing 10.3 with m = MOD
        long r = 1;
        base %= MOD;
        while (exp > 0) {
            if ((exp & 1) == 1) r = r * base % MOD;
            base = base * base % MOD;
            exp >>= 1;
        }
        return r;
    }
}
```

The cost is two fast powers of about 50 iterations each, against 10^{15} multiplications for the direct product. Notice that the answer is not $20^{n/2}$ when n is odd: the formula must count the extra even position. Small cases such as $n = 1$ and $n = 3$ catch that slip.

10.4 Inverses and binomial coefficients

Table 10.1 says you cannot divide residues. Yet the most common counting formula, $C(n, r) = n!\,/\,(r!\,(n - r)!)$, divides. The way out is to replace "divide by b" with "multiply by a number that undoes b".

DEFINITION 10.3 Modular inverse

An integer x is a **modular inverse** of a modulo m when $a \cdot x \equiv 1 \pmod{m}$. It is written a^{-1}.

THEOREM 10.8 When an inverse exists

a has an inverse modulo m if and only if $\gcd(a, m) = 1$, and the inverse is unique modulo m.

Proof. If $\gcd(a, m) = 1$, Theorem 10.5 gives x, y with $ax + my = 1$, so $ax \equiv 1 \pmod{m}$. Conversely, if $ax \equiv 1$, then $ax - 1 = mt$ for some t, so $1 = ax - mt$, and any common divisor of a and m divides 1. For uniqueness, if $ax \equiv ax' \equiv 1$, then $x \equiv x(ax') = (xa)x' \equiv x'$. ∎

`modInverse` in Listing 10.2 computes the inverse for any modulus and returns −1 when none exists. Modulo 10 000, the inverse of 3 is 6667 ($3 \cdot 6667 = 20\,001$), while 2 has no inverse at all, since $\gcd(2, 10\,000) = 2$. When the modulus is prime there is a shorter route.

Fermat's little theorem

THEOREM 10.9 Fermat's little theorem

If p is prime and p does not divide a, then $a^{p-1} \equiv 1 \pmod{p}$. Hence a^{p-2} is the inverse of a modulo p.

Proof. Consider the $p - 1$ numbers $a, 2a, 3a, \ldots, (p-1)a$ modulo p. None is 0, because p divides neither a nor any $i < p$, and a prime that divides a product divides one of the factors.[1] No two are equal: if $ia \equiv ja$, then p divides $(i - j)a$, hence $p \mid i - j$, and $|i - j| < p$ forces $i = j$. So the residues of $a, 2a, \ldots, (p-1)a$ are $1, 2, \ldots, p - 1$ in some order. For $p = 7$ and $a = 3$ they are 3, 6, 2, 5, 1, 4. Multiplying all of them gives $a^{p-1} \cdot (p-1)! \equiv (p$

1. This is Euclid's lemma. Proof: if $p \nmid x$, then $\gcd(p, x) = 1$, so $px' + xy' = 1$ by Bézout; multiply by y to get $pyx' + (xy)y' = y$, and p divides both terms on the left when $p \mid xy$.

$-1)!\pmod p$. The number $(p-1)!$ is coprime to p, so it has an inverse; multiplying both sides by it leaves $a^{p-1} \equiv 1$. Finally $a \cdot a^{p-2} = a^{p-1} \equiv 1$. ∎

With Listing 10.3, the inverse of a modulo $10^9 + 7$ is `power(a, MOD - 2, MOD)`: about 30 iterations. For example $3^{-1} \equiv 333\,333\,336$ and $2^{-1} \equiv 500\,000\,004$. The theorem also explains the last row but one of Table 10.1: for prime p and $p \nmid a$, $a^e \equiv a^{e \bmod (p-1)}$, because every full block of $p-1$ factors multiplies to 1.

PITFALL

`a^(m-2)` is an inverse **only when m is prime**. Modulo 10 000, $3^{9998} \bmod 10\,000 = 8889$, and $3 \cdot 8889 \equiv 6667$, not 1. If you need an inverse modulo a composite number, use `modInverse` from Listing 10.2, and expect it not to exist for every a.[2]

Binomial coefficients modulo a prime

THEOREM 10.10 Binomial coefficients from factorials

Let p be a prime and $0 \le r \le n < p$. Then $C(n, r) \equiv n! \cdot (r!)^{-1} \cdot ((n-r)!)^{-1} \pmod p$. Moreover $(i-1)!^{-1} \equiv i \cdot (i!)^{-1}$ for $1 \le i < p$.

Proof. As integers, $r!\,(n-r)! \cdot C(n, r) = n!$. Every factor of $r!$ and $(n-r)!$ is less than p, so by Euclid's lemma p divides neither factorial, and both have inverses by Theorem 10.8. Reduce the equation mod p (Theorem 10.6) and multiply both sides by the two inverses. For the second claim, $(i-1)! \cdot (i \cdot (i!)^{-1}) = i! \cdot (i!)^{-1} \equiv 1$, so $i \cdot (i!)^{-1}$ is the inverse of $(i-1)!$, and inverses are unique. ∎

The second claim is the trick in the template. You compute one inverse, of $n!$, with Fermat, and then walk downwards: invFact[$i-1$] = invFact[i] · i. That replaces n fast powers by n multiplications.

Listing 10.5 Factorials, inverse factorials and nCr modulo a prime

```
static final long MOD = 1_000_000_007L;
static long[] fact, invFact;

static long inverse(long a) {                    // only when MOD is prime (Fermat)
    return power(a, MOD - 2, MOD);
}

static void initFactorials(int n) {
    fact = new long[n + 1];
    invFact = new long[n + 1];
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[n] = inverse(fact[n]);
    for (int i = n; i > 0; i--) invFact[i - 1] = invFact[i] * i % MOD;
}

static long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

static long power(long base, long exp, long m) {  // exactly as in Listing 10.3
    long r = 1 % m;
    base %= m;
    if (base < 0) base += m;
    while (exp > 0) {
        if ((exp & 1) == 1) r = r * base % m;
```

2. The general form is Euler's theorem: $a^{\varphi(m)} \equiv 1 \pmod m$ when $\gcd(a, m) = 1$, where $\varphi(m)$ counts the residues coprime to m. For $m = 10\,000$, $\varphi(m) = 4000$ and $3^{3999} \equiv 6667$ is indeed the inverse of 3.

```
        base = base * base % m;
        exp >>= 1;
    }
    return r;
}
```

Precomputation costs O(n + log p) time and two arrays of n + 1 longs; each `nCr` is then three multiplications. `nCr(10, 3)` returns 120 and `nCr(100000, 50000)` returns 149 033 233. In `nCr`, the product of the first two factors is reduced before the third is multiplied, because three residues multiplied together overflow a `long`.

JAVA NOTE

The backward loop matters for speed. One Fermat inverse is about 30 iterations of two multiplications and two `%` operations. On the machine used to test this book, 10^7 separate inverses took 5.1 seconds, while `initFactorials(10_000_000)`, which computes one inverse and then multiplies, took about 0.2 seconds for both arrays.

When the modulus is not prime: Pascal's triangle

The factorial method needs $n < p$ and a prime p. With a composite modulus it fails completely. Modulo 10 000 = $2^4 \cdot 5^4$, the factorial 20! contains the factors 5, 10, 15, 20 (four 5s) and many 2s, so 20! ≡ 0 (mod 10 000), and so is every larger factorial. Then `fact[30]` is 0, and no inverse can recover $C(30, 15)$ = 155 117 520 ≡ 7520 from it.

Additions never need inverses, and binomial coefficients satisfy an addition rule.

THEOREM 10.11 Pascal's rule

For $1 \le k \le n - 1$, $C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$, with $C(n, 0) = C(n, n) = 1$.

Proof. $C(n, k)$ counts the k-element subsets of {1, ..., n}. Split them by whether they contain n. Those that contain n are a $(k - 1)$-subset of {1, ..., $n - 1$} plus n: $C(n - 1, k - 1)$ of them. Those that do not are k-subsets of {1, ..., $n - 1$}: $C(n - 1, k)$ of them. ∎

Listing 10.6 Binomial coefficients modulo any m

```
// Modulus not prime (Infosys has used 10000)? No inverses: build Pascal's triangle instead.
static long[][] pascal(int n, long m) {
    long[][] c = new long[n + 1][n + 1];
    for (int i = 0; i <= n; i++) {
        c[i][0] = 1 % m;
        for (int j = 1; j <= i; j++) c[i][j] = (c[i - 1][j - 1] + c[i - 1][j]) % m;
    }
    return c;
}
```

When $j = i$, the code reads `c[i - 1][i]`, which was never written and is 0, so the diagonal comes out as 1 without a special case. The table costs O(n^2) time and memory: for n = 1000 that is 10^6 longs, about 8 MB, but for n = 5000 it is 200 MB as `long[][]` (100 MB as `int[][]`), more than a typical limit. If only one row is needed, keep a single array and update it from right to left, the same in-place trick as the 0/1 knapsack of Chapter 11.

Reducing modulo 10 changes the look of the triangle but not the rule: row 5 becomes 1 5 0 0 5 1 and row 6 becomes 1 6 5 0 5 6 1. Over a modulus of 10 000, `pascal(40, 10000)[40][20]` gives 8820, and indeed $C(40, 20)$ = 137 846 528 820.

INFOSYS LENS

One of the most frequently reported Infosys DSE problems (13 independent reports in the plan's question bank) counts arrays and prints the count modulo 10 000. Its solution needs binomial coefficients, and because 10 000 is not prime they must come from Pascal's triangle, not from inverses. Day 24 returns to it; for now, remember to look at the modulus before you reach for Fermat.

10.5 Primes, sieves and factorisation

An integer $p \ge 2$ is **prime** when its only positive divisors are 1 and p; an integer ≥ 2 that is not prime is **composite**. Everything in the rest of the chapter rests on one theorem.

THEOREM 10.12 Fundamental theorem of arithmetic

Every integer $x \ge 2$ can be written as a product of primes $x = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$ with $p_1 < p_2 < \cdots < p_k$ and all $e_i \ge 1$, and this factorisation is unique.

Proof. Existence, by strong induction on x: the smallest divisor $d > 1$ of x is prime (a divisor of d would be a smaller divisor of x), and x/d is 1 or, being smaller than x, a product of primes. Uniqueness: if two factorisations existed, cancel the primes they share; a prime p left on one side divides the product on the other side, so by Euclid's lemma (proved in the footnote to Theorem 10.9) it divides one of those primes, and so equals it, a contradiction. ∎

Write $e_p(x)$ for the exponent of p in x (0 if p does not divide x). Then d divides x exactly when $e_p(d) \le e_p(x)$ for every prime p. It follows that $e_p(\gcd(a,b)) = \min(e_p(a), e_p(b))$ and $e_p(\operatorname{lcm}(a,b)) = \max(e_p(a), e_p(b))$; since min + max = sum, gcd · lcm = $a \cdot b$, the identity used in Section 10.1.

Trial division

LEMMA 10.13 A small factor exists

Every composite n has a prime factor p with $p^2 \le n$.

Proof. Write $n = de$ with $1 < d \le e$. Then $d^2 \le de = n$, and the smallest prime factor of d is at most d. ∎

Listing 10.7 Primality and factorisation by trial division

```java
static boolean isPrime(long n) {                // O(sqrt n)
    if (n < 2) return false;
    for (long d = 2; d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}

// Prime factorisation by trial division: a list of {prime, exponent}, O(sqrt n)
static List<long[]> factorise(long n) {
    List<long[]> f = new ArrayList<>();
    for (long p = 2; p * p <= n; p++) {
        if (n % p != 0) continue;
        int e = 0;
        while (n % p == 0) {
            n /= p;
            e++;
        }
        f.add(new long[]{p, e});
    }
```

```
    if (n > 1) f.add(new long[]{n, 1});          // what is left is a prime
    return f;
}
```

`isPrime` is correct by Lemma 10.13. In `factorise`, the loop condition uses the *current* value of n, which shrinks; the comment on the last line needs an argument.

CLAIM 10.14 The leftover is prime

When the loop of `factorise` ends, n is 1 or a prime, and every p added inside the loop is prime.

Proof. When the loop reaches a value p, every prime smaller than p has been divided out completely, so the current n has no prime factor below p. If $n \% p = 0$ then p is prime, since a composite p has a smaller prime factor that would also divide n. At the end $p^2 > n$. If n were composite, Lemma 10.13 would give a prime factor q with $q^2 \le n < p^2$, so $q < p$, contradicting the first sentence. ∎

Trial division costs $O(\sqrt{n})$: for $n \le 10^{12}$ that is up to 10^6 iterations, fine for one number and far too slow for 10^5 numbers.

INFOSYS LENS

The plan records an interview coding task: count pairs with XOR below K whose sum is prime. The candidate's nested loops called trial division for every pair and timed out. The fix was to sieve once up to $2 \cdot \max(A)$, so that each primality check became one array look-up.

The sieve of Eratosthenes

When you need primality for *every* number up to n, test them all at once. Start with every number unmarked; for each prime p in increasing order, mark its multiples as composite. The **sieve of Eratosthenes** does this with two small optimisations: it skips i when i is already marked, and it starts marking at i^2.

Listing 10.8 The sieve of Eratosthenes

```
// composite[x] == false exactly for the primes 2 <= x <= n (entries 0 and 1 are not used)
static boolean[] sieve(int n) {
    boolean[] composite = new boolean[n + 1];
    for (int i = 2; (long) i * i <= n; i++) {
        if (composite[i]) continue;              // i is composite: its multiples are done
        for (int j = i * i; j <= n; j += i) composite[j] = true;
    }
    return composite;
}
```

THEOREM 10.15 Correctness of the sieve

After `sieve(n)`, for every $2 \le x \le n$, `composite[x]` is true if and only if x is composite.

Proof. Only numbers $j = i \cdot k$ with $k \ge i \ge 2$ are ever marked, and such a j is composite, so primes stay unmarked. Now let x be composite with smallest prime factor p. By Lemma 10.13, $p^2 \le x \le n$, so the outer loop reaches $i = p$. At that moment `composite[p]` is false, because p is prime, so the inner loop runs over $p^2, p^2 + p, p^2 + 2p, \ldots, n$. The number $x = p \cdot (x/p)$ has $x/p \ge p$ (every prime factor of x/p is at least p), so x is one of those values and gets marked. ∎

The proof also explains both optimisations. A composite i is skipped because its multiples are multiples of its smallest prime factor, which was processed earlier. Marking starts at p^2 because a multiple $p \cdot k$ with $k < p$ has a prime factor smaller than p (any prime factor of k) and was marked already.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30

Figure 10.3 The sieve on 1..30, each composite coloured by the prime that marks it first, its smallest prime factor: 2 (blue), 3 (amber), 5 (red). Primes (green) are never marked. Marking by 7 would start at 49, so after 2, 3 and 5 the table is final, as Lemma 10.13 predicts: $\sqrt{30} < 7$.

Why the sieve costs O(n log log n)

For a prime $p \le \sqrt{n}$, the inner loop runs over p^2, $p^2 + p$, ..., at most n/p values. So the total work is at most $n \cdot (1/2 + 1/3 + 1/5 + 1/7 + \cdots)$, the sum running over the primes $p \le \sqrt{n}$.

If the loop ran for every i instead of every prime, the sum would be the harmonic series $1/2 + 1/3 + 1/4 + \cdots$, which grows like $\ln n$ (Theorem 10.23). The primes are much sparser than the integers, and the sum of their reciprocals grows only like $\ln \ln n$: a classical result of Mertens says $\Sigma\, 1/p$ over primes $p \le x$ is $\ln \ln x +$ 0.2615... + (a term that tends to 0). So the sieve costs O(n log log n), and $\ln \ln n$ is below 3 for every n you will ever sieve.

The measured numbers for $n = 10^7$ match the estimate. The sieve as written performs 22 850 051 marking steps; $n \ln \ln n \approx 27\,800\,000$. A version that marks all multiples of every $i \ge 2$, prime or not, would take $\Sigma \lfloor n/i \rfloor = 152\,725\,364$ steps, about 6.7 times more. On the machine used for this book, `sieve(10_000_000)` ran in about 45 ms and found $\pi(10^7) = 664\,579$ primes; it uses one `boolean` per number, about 10 MB.[3]

LeetCode 204 (Count Primes) is this sieve followed by one counting loop. Read its statement carefully: it counts the primes strictly *less than* n, and small n such as 0, 1 and 2 have the answer 0.

The smallest-prime-factor sieve

A boolean sieve answers "is x prime?". A **smallest-prime-factor sieve** stores, for every $x \le n$, the smallest prime dividing it, and that one array factorises every number in the range.

Listing 10.9 Smallest-prime-factor sieve and the number of divisors

```
// spf[x] = smallest prime factor of x, for every x <= n. x is prime when spf[x] == x.
static int[] smallestPrimeFactors(int n) {
    int[] spf = new int[n + 1];
    for (int i = 2; i <= n; i++) {
        if (spf[i] != 0) continue;
        for (int j = i; j <= n; j += i) if (spf[j] == 0) spf[j] = i;
    }
    return spf;
}

// Number of divisors of x from its factorisation: product of (exponent + 1). O(log x).
static int numDivisors(int x, int[] spf) {
    int count = 1;
    while (x > 1) {
        int p = spf[x], e = 0;
        while (x % p == 0) {
```

3. For comparison, $\pi(30) = 10$, $\pi(100) = 25$, $\pi(1000) = 168$ and $\pi(10^6) = 78\,498$.

```
            x /= p;
            e++;
        }
        count *= e + 1;
    }
    return count;
}
```

THEOREM 10.16 Correctness of the SPF sieve and of factorisation

After `smallestPrimeFactors(n)`, `spf[x]` is the smallest prime factor of x for every $2 \le x \le n$. Repeatedly dividing x by `spf[x]` factorises x in at most $\log_2 x$ divisions.

Proof. The outer loop visits $i = 2, 3, \dots$ in increasing order and only runs the inner loop when `spf[i]` is still 0. We show that this happens exactly for prime i. A composite i has a prime factor $q < i$, and the inner loop for q (if it ran) set `spf[i]`. Now take any x with smallest prime factor p. No prime smaller than p divides x, so no inner loop before $i = p$ touches `spf[x]`; when $i = p$ (which is prime, so its loop runs), x is a multiple of p and `spf[x]` is still 0, so it becomes p. Later loops never overwrite a non-zero entry. For factorisation, each division removes one prime factor, at least 2, so after t divisions the remaining value is at most $x/2^t$, and the process ends within $\log_2 x$ steps. ∎

The inner loop runs $\lfloor n/p \rfloor$ times for each prime p, which is again $O(n \log \log n)$ in total: 30 130 317 steps for $n = 10^7$, about 0.2 s in Java. The array holds one `int` per number, 40 MB for $n = 10^7$. Factorising 360 visits

```
360 (spf 2) -> 180 (spf 2) -> 90 (spf 2) -> 45 (spf 3) -> 15 (spf 3) -> 5 (spf 5) -> 1
360 = 2^3 * 3^2 * 5
```

PITFALL

A sieve to 10^7 as `int[]` takes 40 MB, and to 10^8 it would take 400 MB. Check the memory limit before going beyond 10^7, and prefer a `boolean[]` (1 byte per entry) when you only need primality.

Counting and listing divisors

THEOREM 10.17 Number of divisors

If $x = p_1^{e_1} \cdots p_k^{e_k}$, then x has exactly $(e_1 + 1)(e_2 + 1)\cdots(e_k + 1)$ positive divisors.

Proof. By Theorem 10.12, a positive divisor d of x has the form $p_1^{f_1} \cdots p_k^{f_k}$ with $0 \le f_i \le e_i$, and different exponent vectors give different numbers. Each f_i has $e_i + 1$ choices, independently. ∎

So $360 = 2^3 \cdot 3^2 \cdot 5$ has $4 \cdot 3 \cdot 2 = 24$ divisors, which `numDivisors` computes in $O(\log x)$ from the SPF array. Divisor counts stay small, which makes "loop over all divisors" affordable: no number up to 10^5 has more than 128 divisors (83 160 has 128), none up to 10^6 has more than 240 (720 720), and none up to 10^9 has more than 1344 (735 134 400). To list the divisors of a single number without a sieve, pair each $d \le \sqrt{x}$ with x/d:

Listing 10.10 All divisors of one number in O(√x)

```
// All divisors of x in increasing order, O(sqrt x)
static List<Long> divisors(long x) {
    List<Long> small = new ArrayList<>(), big = new ArrayList<>();
    for (long d = 1; d * d <= x; d++) {
        if (x % d != 0) continue;
        small.add(d);
        if (d != x / d) big.add(x / d);
    }
```

```
    Collections.reverse(big);
    small.addAll(big);
    return small;
}
```

The test `d != x / d` stops a perfect square from listing its root twice. For $x = 360$ the method returns the 24 divisors 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, ..., 180, 360.

Table 10.2 Choosing a factorisation tool

Situation	Tool	Cost
one value up to 10^{12}, is it prime / what are its factors?	trial division (Listing 10.7)	$O(\sqrt{x})$
primality of every value up to 10^7	boolean sieve (Listing 10.8)	$O(n \log \log n)$, 1 byte each
factorise many values up to 10^7	SPF sieve (Listing 10.9)	$O(n \log \log n)$ once, then $O(\log x)$ each
kernels of many values up to 10^9	primes up to 1000 (Listing 10.12)	168 divisions per value
all divisors of one value	Listing 10.10	$O(\sqrt{x})$

10.6 Perfect squares and the square-free kernel

A family of problems asks when a product is a perfect square: “count pairs whose product is a square”, “split the array so that no segment contains two numbers whose product is a square”, “choose indices whose pairwise products are squares”. Testing products directly is slow and overflows. Factorisation turns the question into a comparison of two small numbers.

DEFINITION 10.4 Square-free number and square-free kernel

A positive integer is **square-free** when no prime appears in it more than once, that is, when no p^2 divides it. The **square-free kernel** of a positive integer x, written $\ker(x)$, is the product of the primes whose exponent in x is odd.

For example $12 = 2^2 \cdot 3$ has kernel 3, $27 = 3^3$ has kernel 3, $18 = 2 \cdot 3^2$ has kernel 2, and $360 = 2^3 \cdot 3^2 \cdot 5$ has kernel $2 \cdot 5 = 10$. Every x factors as $\ker(x) \cdot s^2$ for some integer s: the even part of each exponent goes into s^2, the odd remainder into the kernel. So the kernel is what is left after dividing out the largest square factor.

LEMMA 10.18 Squares have even exponents

$x \geq 1$ is a perfect square if and only if every exponent $e_p(x)$ is even.

Proof. If $x = s^2$, then $e_p(x) = 2e_p(s)$ for every p. Conversely, if every exponent is even, $s = \prod p^{e_p(x)/2}$ satisfies $s^2 = x$. ∎

THEOREM 10.19 Square products and kernels

For positive integers x and y, $x \cdot y$ is a perfect square if and only if $\ker(x) = \ker(y)$.

Proof. For every prime p, $e_p(xy) = e_p(x) + e_p(y)$, and a sum of two integers is even exactly when they have the same parity. By Lemma 10.18, xy is a square if and only if $e_p(x)$ and $e_p(y)$ have the same parity for every p, that is, if and only if x and y have the same set of primes with odd exponent. The kernel is the product of that set, and by Theorem 10.12 two square-free numbers are equal exactly when their sets of primes are equal. ∎

The theorem has a consequence that is not obvious from the definition: the relation "xy is a perfect square" is transitive. If xy and yz are squares then so is xz, because all three share a kernel. So the numbers split into **groups by kernel**. Two numbers in the same group always have a square product, and two numbers in different groups never do.

PITFALL

The kernel is not the *radical* (the product of *all* distinct prime factors). The radical of 12 is 6, its kernel is 3. With $12 \cdot 3 = 36 = 6^2$ a square and $12 \cdot 6 = 72$ not a square, only the kernel gives the right grouping.

The kernel comes straight out of the SPF factorisation: track the parity of each exponent with `odd ^= 1`.

Listing 10.11 Square-free kernel from the SPF sieve

```java
// Square-free kernel: product of the primes with an odd exponent.
// x * y is a perfect square  <=>  kernel(x) == kernel(y)
static int kernel(int x, int[] spf) {
    int k = 1;
    while (x > 1) {
        int p = spf[x], odd = 0;
        while (x % p == 0) {
            x /= p;
            odd ^= 1;
        }
        if (odd == 1) k *= p;
    }
    return k;
}
```

```
12 = 2^2 * 3    -> kernel 3
27 = 3^3        -> kernel 3     12 * 27 = 324 = 18^2   (same kernel: square)
18 = 2 * 3^2    -> kernel 2     12 * 18 = 216          (different: not a square)
```

Kernels of values up to 10^9

An SPF sieve only reaches about 10^7. For values up to 10^9, you can still compute the kernel without a sieve, because what is left after removing small primes has a very limited shape.

THEOREM 10.20 **The cofactor after removing small primes**

Let $1 \le x \le 10^9$, and let y be what remains of x after dividing out every prime $p \le 1000$ completely. Then y is 1, a prime q, a square q^2 of a prime, or a product $q \cdot r$ of two distinct primes, with $q, r > 1000$.

Proof. Every prime factor of y exceeds 1000, so it is at least 1009, the smallest prime above 1000. If y had three or more prime factors, counted with multiplicity, then $y \ge 1009^3 = 1\,027\,243\,729 > 10^9 \ge x \ge y$, which is impossible. So y has zero, one or two prime factors, which are the four listed shapes. ∎

The kernel contributions of the four shapes are 1, q, 1 and $q \cdot r$. In other words, the cofactor y contributes itself unless it is a perfect square, and a perfect-square test is one integer square root. Listing 10.12 adapts the plan's tested code; it also stops early when p^2 exceeds what is left, in which case the remainder is 1 or a prime by the argument of Claim 10.14.

Listing 10.12 Square-free kernel of $x \le 10^9$ with the 168 primes below 1000

```java
static final int[] SMALL_PRIMES = primesUpTo(1000);   // the 168 primes below 1000

static int[] primesUpTo(int n) {
    List<Integer> ps = new ArrayList<>();
```

```
    for (int i = 2; i <= n; i++) {
        boolean prime = true;
        for (int d = 2; d * d <= i; d++) if (i % d == 0) { prime = false; break; }
        if (prime) ps.add(i);
    }
    int[] r = new int[ps.size()];
    for (int i = 0; i < r.length; i++) r[i] = ps.get(i);
    return r;
}

// Square-free kernel of 1 <= x <= 1e9 without a sieve up to x
static long kernelLarge(long x) {
    long k = 1;
    for (int p : SMALL_PRIMES) {
        if ((long) p * p > x) break;            // then x is 1 or a prime
        int odd = 0;
        while (x % p == 0) {
            x /= p;
            odd ^= 1;
        }
        if (odd == 1) k *= p;
    }
    // x is 1, a prime, or (loop finished) q*q or q*r with primes q, r > 1000
    long s = (long) Math.sqrt((double) x);
    while (s * s > x) s--;
    while ((s + 1) * (s + 1) <= x) s++;
    if (s * s != x) k *= x;                     // a prime or q*r stays; 1 and q*q vanish
    return k;
}
```

```
x = 12 216 972 = 2^2 * 3 * 1009^2       -> kernel 3          (q*q vanishes)
x =  8 176 936 = 2^3 * 1009 * 1013      -> kernel 2 044 234  (2 * q * r)
x = 1 000 000 000 = 2^9 * 5^9           -> kernel 10
x =   999 999 937 (a prime)             -> kernel 999 999 937
```

The two `while` loops around `Math.sqrt` correct any rounding error, so the square-root test is exact. Each call does at most 168 trial divisions plus the loop over exponents, a few hundred operations; 10^5 values cost a few times 10^7 operations. The test harness for this book compared `kernelLarge` with a full trial-division kernel on 20 000 values, including products of primes just above 1000.

INFOSYS LENS

The plan's notes for this day say that the official sample paper's Easy and Medium problems are both number theory at their core. The Easy one concerns products that are perfect squares with values up to 10^9, so Theorem 10.19 and Listing 10.12 are its first step. You will meet the whole paper as Mock 4 on Day 25.

Worked example: grouping indices by kernel

EXAMPLE 10.4 Maximum Element-Sum of a Complete Subset of Indices (LeetCode 2862)

You are given a 1-indexed array *nums* of length $n \le 10^4$. A set of indices is *complete* when the product of every pair of its indices is a perfect square. Return the largest sum of *nums* over a complete set.

By Theorem 10.19, a set of indices is complete exactly when all its indices have the same kernel. So the complete sets are the subsets of the kernel groups, and because every value is positive the best subset of a group is the whole group. The answer is the largest group sum. For *nums* = [8, 7, 3, 5, 7, 2, 4, 9] the kernels of the indices 1..8 are 1, 2, 3, 1, 5, 6, 7, 2, so the group sums are

```
kernel 1: indices {1, 4}  sum 8 + 5 = 13
kernel 2: indices {2, 8}  sum 7 + 9 = 16      <- largest
kernel 3: {3} 3    kernel 5: {5} 7    kernel 6: {6} 2    kernel 7: {7} 4
```

and the answer is 16.

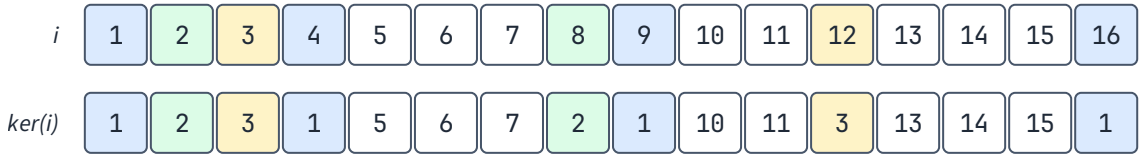


Figure 10.4 Indices 1..16 grouped by square-free kernel. Kernel 1 (blue) holds the squares 1, 4, 9, 16; kernel 2 (green) holds 2 and $8 = 2 \cdot 2^2$; kernel 3 (amber) holds 3 and $12 = 3 \cdot 2^2$. Every other index up to 16 is alone in its group. The product of two indices is a square exactly when they share a colour.

Listing 10.13 LeetCode 2862: sum each kernel group

```java
class Solution {
    public long maximumSum(List<Integer> nums) {
        int n = nums.size();
        int[] spf = new int[n + 1];                    // smallest-prime-factor sieve on 1..n
        for (int i = 2; i <= n; i++) {
            if (spf[i] != 0) continue;
            for (int j = i; j <= n; j += i) if (spf[j] == 0) spf[j] = i;
        }
        long[] groupSum = new long[n + 1];             // indexed by kernel; kernel(i) <= i <= n
        long best = 0;
        for (int i = 1; i <= n; i++) {                 // indices are 1-based in this problem
            int x = i, k = 1;
            while (x > 1) {
                int p = spf[x], odd = 0;
                while (x % p == 0) {
                    x /= p;
                    odd ^= 1;
                }
                if (odd == 1) k *= p;
            }
            groupSum[k] += nums.get(i - 1);
            best = Math.max(best, groupSum[k]);
        }
        return best;
    }
}
```

Because $\ker(i) \le i \le n$, an array of length $n + 1$ replaces a hash map. The sieve costs $O(n \log \log n)$ and each kernel $O(\log n)$, so the whole solution is $O(n \log n)$ with $O(n)$ memory. Group sums can reach $10^4 \cdot 10^9 = 10^{13}$, so they are `long`. The harness checked this listing against an exhaustive search over all 2^n index sets for $n \le 12$ on 3000 random arrays.

The same observation unlocks Codeforces 1497E1 (Square-Free Division (easy version)): a segment contains two numbers with a square product exactly when two of its elements have equal kernels.

10.7 Gcds of subarrays and loops over multiples

Few distinct gcds end at each index

Write $g(l, r)$ for $\gcd(a[l..r])$. An array of length n has $n(n + 1)/2$ subarrays, but their gcds are far from all different.

LEMMA 10.21 The gcd chain

Fix r and let $V = a[r] \geq 1$. As l moves from r down to 0, each value $g(l, r)$ divides the previous one, $g(l + 1, r)$. Consequently the values $g(0, r), \ldots, g(r, r)$ contain at most $\lfloor \log_2 V \rfloor + 1$ distinct numbers.

Proof. $g(l, r) = \gcd(a[l], g(l + 1, r))$, and a gcd divides each of its arguments. So the sequence $g(r, r) = V, g(r - 1, r), \ldots$ is a chain in which each term divides the one before. When a term differs from the one before, it is a proper divisor, hence at most half of it. If the chain takes t distinct values, the smallest is at most $V/2^{t-1}$ and at least 1, so $2^{t-1} \leq V$, that is, $t \leq \log_2 V + 1$. ∎

For values up to 10^9 that is at most 30 distinct gcds per right end, however long the array. The bound is tight: in $[1, 2, 4, \ldots, 2^{29}]$ the subarrays ending at the last index have the 30 gcds $1, 2, 4, \ldots, 2^{29}$.

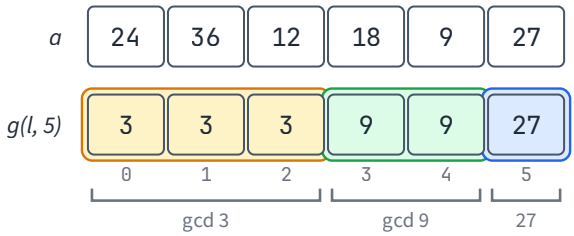


Figure 10.5 The gcds g(l, 5) of the subarrays ending at index 5 of a = [24, 36, 12, 18, 9, 27]. Reading right to left, each value divides the one before, and the six subarrays share only three distinct gcds: 27, 9 and 3.

The lemma supports two techniques.

Early stopping from a fixed start. Fix l and extend r. The running gcd $g(l, r)$ can only move down the divisibility chain. If you need gcd exactly k (LeetCode 2447, Number of Subarrays With GCD Equal to K), note that once k does not divide the running gcd g, it never will again, because every later gcd divides g, and a multiple of k that divides g would make k divide g. So the scan from l can stop at the first r where k fails to divide the gcd. This is the plan's hint for the problem, and with $n \leq 1000$ the $O(n^2)$ pairs, each with one gcd step, fit easily.

Lists of distinct gcds per right end. Keep, for the current right end r, the list of distinct values of $g(l, r)$. Moving to $r + 1$, each value g becomes $\gcd(g, a[r + 1])$, the new single-element subarray adds $a[r + 1]$ itself, and equal values merge. By Lemma 10.21 the list never has more than about 30 entries.

Listing 10.14 Distinct gcds over all subarrays in O(n log V) gcd calls

```
// How many distinct values does gcd(a[l..r]) take over all subarrays? O(n log V) gcd calls.
static int distinctSubarrayGcds(int[] a) {
    Set<Integer> seen = new HashSet<>();
    int[] cur = new int[0];          // distinct gcds of the subarrays ending at the previous index
    for (int x : a) {
        int[] next = new int[cur.length + 1];
        int len = 0;
        next[len++] = x;                              // the subarray that is x alone
        for (int g : cur) {
            int h = gcd(g, x);
            if (h != next[len - 1]) next[len++] = h; // equal values arrive next to each other
        }
        cur = Arrays.copyOf(next, len);
        for (int g : cur) seen.add(g);
    }
    return seen.size();
}

static int gcd(int a, int b) {
```

```
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

CLAIM 10.22 The list is correct

After processing a[r], `cur` holds the distinct values of g(l, r) for 0 ≤ l ≤ r, in the order of decreasing l, which is strictly decreasing order of value.

Proof. By induction on r. The values for right end r are $a[r]$ (for $l = r$) and $\gcd(v, a[r])$ for every value v of $g(l, r-1)$, because $g(l, r) = \gcd(g(l, r-1), a[r])$. The loop produces exactly these, in the order of decreasing l. By Lemma 10.21 this sequence is a divisor chain, so it is non-increasing and equal values are adjacent; comparing each new value with the last one kept therefore removes all repeats. ∎

EXAMPLE 10.5 Distinct gcds of [24, 36, 12, 18, 9, 27, 4]

Each row shows the list after reading $a[i]$. The note "×c" says how many subarrays ending at i have that gcd; Listing 10.14 does not need the counts, but they show where the subarrays went.

```
i  a[i]  list of distinct g(l, i)      new values for the set
0   24   24x1                         24
1   36   36x1  12x1                   36, 12
2   12   12x3                         -
3   18   18x1   6x3                   18, 6
4    9    9x2   3x3                   9, 3
5   27   27x1   9x2   3x3             27
6    4    4x1   1x6                   4, 1
```

The set ends with ten values {1, 3, 4, 6, 9, 12, 18, 24, 27, 36}: the 28 subarrays have only ten different gcds.

The lists hold $O(\log V)$ entries, each costing one gcd of $O(\log V)$ steps, so the scan costs $O(n \log^2 V)$ time and $O(\log V)$ memory besides the set. Keeping a count next to each list entry turns the same scan into a counter of subarrays by gcd.

Loops over multiples and the harmonic bound

A second kind of loop runs, for every d from 1 to V, over the multiples $d, 2d, 3d, \ldots \le V$. It looks like $O(V^2)$, but it is not.

THEOREM 10.23 The harmonic bound

$\sum_{d=1..V} \lfloor V/d \rfloor \le V \cdot H_V \le V(1 + \ln V)$, where $H_V = 1 + 1/2 + \cdots + 1/V$. So a loop over all multiples of every $d \le V$ costs $O(V \log V)$.

Proof. The loop for d visits $\lfloor V/d \rfloor \le V/d$ multiples, so the total is at most $V(1 + 1/2 + \cdots + 1/V)$. For each $d \ge 2$, the function $1/x$ is at least $1/d$ on the interval $[d-1, d]$, so $1/d$ is at most the area under $1/x$ between $d-1$ and d. Adding these for $d = 2..V$ gives $H_V - 1 \le$ (area under $1/x$ from 1 to V) $= \ln V$. ∎

For $V = 10^6$ the sum is 13 970 034, close to $V \ln V \approx 13\,815\,511$. This is the **harmonic bound**, and it is the reason why "for each g, look at the multiples of g" is a standard move in gcd problems.

EXAMPLE 10.6 Pairs with gcd exactly g

Given values $1 \le a[i] \le V$, count for every g the pairs $i < j$ with $\gcd(a[i], a[j]) = g$. Trying all pairs costs $O(n^2)$. Instead, let $m(g)$ be the number of elements divisible by g. A pair has both elements divisible by g exactly when its gcd is a multiple of g, so

$$m(g)(m(g) - 1)/2 = \text{exact}(g) + \text{exact}(2g) + \text{exact}(3g) + \cdots$$

Processing g from V down to 1, every exact(kg) with $k \ge 2$ is already known, so exact(g) is the left side minus them. For $a = [2, 4, 6, 3, 9, 12, 8]$:

```
 g   multiples of g in a   m   C(m,2)   minus exact(2g), exact(3g), ...   exact(g)
12   12                    1     0       0                                0
 6   6, 12                 2     1       0  (exact 12)                    1
 4   4, 12, 8              3     3       0  (exact 8, 12)                 3
 3   6, 3, 9, 12           4     6       1  (exact 6, 9, 12)              5
 2   2, 4, 6, 12, 8        5    10       4  (exact 4, 6, 8, 10, 12)       6
 1   all seven             7    21      15  (6 + 5 + 3 + 1)               6
```

The six coprime pairs are (2, 3), (2, 9), (4, 3), (4, 9), (3, 8) and (9, 8). Rows for $g = 5, 7, 8, 9, 10, 11$ are all 0 and are left out.

Listing 10.15 Pairs by exact gcd with loops over multiples

```
// exact[g] = number of pairs i < j with gcd(a[i], a[j]) == g, for 1 <= a[i] <= V. O(V log V + n).
static long[] pairsWithGcd(int[] a, int V) {
    int[] freq = new int[V + 1];
    for (int x : a) freq[x]++;
    long[] exact = new long[V + 1];
    for (int g = V; g >= 1; g--) {
        long multiples = 0;                          // elements divisible by g
        for (int k = g; k <= V; k += g) multiples += freq[k];
        exact[g] = multiples * (multiples - 1) / 2;  // pairs whose gcd is a multiple of g
        for (int k = 2 * g; k <= V; k += g) exact[g] -= exact[k];
    }
    return exact;
}
```

Correctness is the displayed equation plus induction downwards on g. Both inner loops visit the multiples of g, so the cost is $O(n + V \log V)$ by Theorem 10.23, and the memory is two arrays of size $V + 1$. For $V = 10^6$ that is about $2.8 \cdot 10^7$ simple steps.

LeetCode 1819 (Number of Different Subsequences GCDs) uses the same loop over multiples with one observation: a value g is the gcd of some subsequence exactly when the gcd of all array elements that are multiples of g equals g.

INFOSYS LENS

The plan's notes describe the sample paper's Medium problem as "gcd exactly p". Questions of that shape usually start by keeping only the multiples of p and taking their gcd, which is the reasoning of this section.

10.8 Small primes as bit masks

When every value is small, its set of prime factors is small too, and a set of at most 10 primes fits in the bits of an `int`. The primes up to 30 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29: ten of them. A square-free number up to 30 is the product of a subset of these, so it corresponds to a 10-bit mask (Chapter 8).

Listing 10.16 The prime mask of a small number

```java
static final int[] PRIMES = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};   // all primes <= 30

// Bit i is set <=> PRIMES[i] divides x. Returns -1 if 1 <= x <= 30 is not square-free.
static int primeMask(int x) {
    int mask = 0;
    for (int i = 0; i < PRIMES.length; i++) {
        int p = PRIMES[i];
        if (x % p != 0) continue;
        x /= p;
        if (x % p == 0) return -1;                  // p * p divided the original x
        mask |= 1 << i;
    }
    return mask;
}
```

THEOREM 10.24 **Square-free products as disjoint masks**

A product $x_1 \cdot x_2 \cdots x_k$ of numbers from 1..30 is square-free if and only if every x_i is square-free and their prime masks are pairwise disjoint. Then the mask of the product is the OR of the masks.

Proof. The exponent of p in the product is the sum of its exponents in the factors. The product is square-free when every such sum is at most 1, which happens exactly when no factor contains p^2 and at most one factor contains p. "At most one factor contains p" for every p means that no bit is set in two masks. ∎

For example 6 → 0b11, 5 → 0b100 and 7 → 0b1000 are disjoint, and $6 \cdot 5 \cdot 7 = 210$ is square-free with mask 0b1111. But 6 and 10 (0b101) share bit 0, and $6 \cdot 10 = 60 = 2^2 \cdot 15$ is not. Of the numbers 1..30, 19 are square-free; 4, 8, 9, 12, 16, 18, 20, 24, 25, 27 and 28 get −1.

Disjointness is one AND: `(maskA & maskB) == 0`. This turns "choose numbers whose product is square-free" into "choose masks that do not overlap", which is a knapsack whose capacity is a set of primes instead of a weight. LeetCode 2572 (Count the Number of Square-Free Subsets) counts such choices with a DP over the $2^{10} = 1024$ masks. Chapter 11 develops the knapsack and Chapter 22 the DP over masks; here the point is only that ten primes make the state space $2^{10} = 1024$ sets of used primes, however many numbers the input contains.

Summary

- Euclid's algorithm computes $\gcd(a, b)$ in $O(\log \min(a, b))$ iterations, with consecutive Fibonacci numbers as the worst case; $\gcd(0, x) = x$ makes 0 the starting value for a gcd fold. Compute lcm as `a / gcd(a, b) * b`.
- Extended Euclid finds x, y with $ax + by = \gcd(a, b)$, which gives inverses modulo any m coprime to a.
- Reduce after every +, − and ×, never divide residues, and never compare residues as if they were the true values. With modulus $10^9 + 7$, a product of two residues fits in a `long`; three do not. Fix negative remainders with `+ MOD` or `Math.floorMod`.
- Fast power computes $a^e \bmod m$ in $\lfloor \log_2 e \rfloor + 1$ iterations by squaring the base and reading the bits of e.
- For a prime p, the inverse of a is a^{p-2} (Fermat). One inverse of $n!$ and a backward loop give all inverse factorials, so $C(n, r) \bmod p$ costs $O(1)$ after $O(n)$ work. For a composite modulus such as 10 000, use Pascal's triangle.
- The sieve of Eratosthenes runs in $O(n \log \log n)$ because the reciprocals of the primes sum to about $\ln \ln n$. The SPF sieve factorises any $x \le 10^7$ in $O(\log x)$, and the divisor count is $\prod(e + 1)$.
- xy is a perfect square exactly when $\ker(x) = \ker(y)$, so square-product problems become grouping by kernel. For $x \le 10^9$, divide out the 168 primes below 1000; the cofactor is 1, q, q^2 or $q \cdot r$.

- At most $\lfloor \log_2 V \rfloor + 1$ distinct gcds end at each index, and loops over the multiples of every $d \le V$ cost $O(V \log V)$.

Key terms

greatest common divisor p. 138
Bézout's identity p. 141
congruence p. 142
modular inverse p. 145
square-free kernel p. 152
binary exponentiation p. 143
sieve of Eratosthenes p. 149
smallest-prime-factor sieve p. 150
harmonic bound p. 157

Review questions

1. Why does Euclid's loop always terminate, even for negative arguments? Which pair $a > b$ with both values below 100 needs the most iterations, and how many?
2. Explain why `a * b / gcd(a, b)` can be wrong in `long` arithmetic when `a / gcd(a, b) * b` is right. Can dividing first ever make the result wrong?
3. A solution computes `(prefixA - prefixB) % MOD` and later uses the result as an array index. What can go wrong, and what are two ways to fix it?
4. Why is a^{p-2} the inverse of a modulo a prime p? Give a modulus and a value for which this formula gives a wrong answer, and say what to use instead.
5. In Listing 10.5, why is `invFact` filled from n downwards instead of calling `inverse(fact[i])` for each i? What does the loop rely on?
6. In the sieve of Eratosthenes, why may the inner loop start at i^2 and why may composite i be skipped? What is the cost, and where does the log log come from?
7. Give two numbers with the same radical whose product is not a perfect square, and two numbers with the same kernel. Why does Theorem 10.20 fail if the values may be as large as $2 \cdot 10^9$?
8. For LeetCode 2862, show that the kernel group of k is exactly $\{k \cdot j^2 : j \ge 1,\ k \cdot j^2 \le n\}$ when k is square-free. Use this to give a solution without a sieve, and bound its running time.
9. In Lemma 10.21, why is a new distinct gcd at most half of the previous one? Give an array of length 4 whose subarrays ending at the last index have four distinct gcds.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 21 "Number theory" (primes, factors, modular arithmetic, Euclid's algorithm) and Chapter 22 "Combinatorics" (binomial coefficients). Free at cses.fi/book.
- Algorithms for Competitive Programming: "Sieve of Eratosthenes" (cp-algorithms.com/algebra/sieve-of-eratosthenes.html), which also covers segmented sieving, and "Linear Sieve" (cp-algorithms.com/algebra/prime-sieve-linear.html) for an SPF sieve that touches each number once.
- Algorithms for Competitive Programming: "Binary Exponentiation" (cp-algorithms.com/algebra/binary-exp.html) and "Modular Inverse" (cp-algorithms.com/algebra/module-inverse.html), including a linear-time method for the inverses of $1..n$.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Number-Theoretic Algorithms*, for full proofs of Euclid's algorithm, the extended algorithm and modular exponentiation.

Problems

These are the problems for Day 10. Work them in Workbook Chapter 10, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
10.P1	LC 204 · Count Primes	MEDIUM	CORE
10.P2	LC 1922 · Count Good Numbers	MEDIUM	CORE
10.P3	LC 2447 · Number of Subarrays With GCD Equal to K	MEDIUM	CORE
10.P4	LC 2862 · Maximum Element-Sum of a Complete Subset of Indices	HARD	CORE
10.P5	LC 1819 · Number of Different Subsequences GCDs	HARD	BONUS
10.P6	LC 2572 · Count the Number of Square-Free Subsets	MEDIUM	BONUS
10.P7	CF 1497E1 · Square-Free Division (easy version)	CF 1700	BONUS

CHAPTER 11 · DAY 11 OF 29 · TUESDAY 17 NOVEMBER 2026 · LEARNING DAY

11

Dynamic Programming I: States, Transitions and Knapsacks

Define the state in words before writing any code.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · DP recipe: state, transition, base, answer, Knapsack family
Before this chapter · Chapters 4 and 5 (greedy choices and exchange arguments) and Chapter 10 (arithmetic modulo $10^9 + 7$)
Practice · Workbook Chapter 11 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- recognise overlapping subproblems and optimal substructure, and measure what memoisation saves;
- write a dynamic programme as state, transition, base cases, evaluation order and answer, and prove it correct by induction over the evaluation order;
- solve one-dimensional problems such as House Robber in O(n) time and O(1) extra memory;
- solve the 0/1 and the unbounded knapsack with a one-dimensional table, and prove why the loop direction decides whether an item may be reused;
- count coin combinations and ordered sequences correctly, modulo $10^9 + 7$ when the counts grow;
- reduce partition problems to subset sum, rebuild an optimal choice from the table, and speed up boolean subset sums with 64-bit words.

Greedy algorithms make one choice at a time and never look back. Chapters 4 and 5 showed when that is safe: an exchange argument proves that the greedy choice can always be part of an optimal answer. Many problems have no such choice. To rob the most money from a street of houses without robbing two neighbours, or to pack the most value into a bag of fixed capacity, you must compare whole families of choices, and there are exponentially many of them.

Dynamic programming compares them all without listing them. It names a family of smaller questions, the *states*, answers each one exactly once from the answers to even smaller ones, and stores the results in a table. This chapter develops the method from first principles. It starts with the reason the method works (the same subproblems recur, and optimal answers are built from optimal parts), turns that into a five-line recipe with a proof template, and then applies the recipe to the most frequently tested family in coding assessments: knapsacks, coin change and subset sums. The last section covers what the textbook versions leave out: printing the chosen items, counting modulo a prime, estimating memory, and a bit-level trick that makes boolean subset sums about thirty times faster.

11.1 What dynamic programming is

The Fibonacci numbers are defined by $F(0) = 0$, $F(1) = 1$ and $F(n) = F(n - 1) + F(n - 2)$. Typing the definition as recursion gives a correct program that is astonishingly slow, and counting its calls shows why.

Listing 11.1 Fibonacci three ways, with a call counter

```
static long calls;                                  // incremented on every call

static long fibNaive(int n) {                       // the definition, as recursion
    calls++;
    if (n <= 1) return n;
    return fibNaive(n - 1) + fibNaive(n - 2);
}

static long[] memo;

static long fibMemo(int n) {                        // top-down: remember every answer
    memo = new long[n + 1];
    Arrays.fill(memo, -1);
    return fm(n);
}

static long fm(int n) {
    calls++;
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];              // already solved: no further calls
    return memo[n] = fm(n - 1) + fm(n - 2);
}

static long fibTable(int n) {                       // bottom-up: smallest states first
    long[] f = new long[Math.max(2, n + 1)];
    f[1] = 1;
    for (int i = 2; i <= n; i++) f[i] = f[i - 1] + f[i - 2];
    return f[n];
}
```

THEOREM 11.1 **The cost of naive recursion**

`fibNaive(n)` makes exactly $2F(n+1) - 1$ calls, while `fibMemo(n)` makes $2n - 1$ calls for $n \geq 1$ and `fibTable(n)` performs $n - 1$ additions.

Proof. Let $C(n)$ be the number of calls made by `fibNaive(n)`, including the first. $C(0) = C(1) = 1$, and $2F(1) - 1 = 2F(2) - 1 = 1$. For $n \geq 2$, $C(n) = 1 + C(n-1) + C(n-2) = 1 + (2F(n) - 1) + (2F(n-1) - 1) = 2F(n+1) - 1$ by induction. In `fm`, the body below the memo check runs at most once for each k from 2 to n, because its result is stored, and each run makes two calls; every other call returns at once. So the calls are the first one plus $2(n-1)$, which is $2n - 1$. The table loop runs for $i = 2..n$. ∎

The numbers, from running Listing 11.1 on the test machine:

```
n    F(n)          naive calls      naive time    memo calls   table additions
10   55                    177                          19            9
20   6765               21 891         0.3 ms          39           19
30   832040          2 692 537         5.8 ms          59           29
40   102334155     331 160 281        701 ms           79           39
```

Each step of n multiplies the naive count by about 1.618, the golden ratio of Chapter 10, so at the same rate $n = 50$ would take more than a minute and $n = 90$ several centuries. The memoised and tabulated versions take microseconds for every n up to 92; $F(92) = 7\,540\,113\,804\,746\,346\,429$ is the largest Fibonacci number that fits in a `long`.

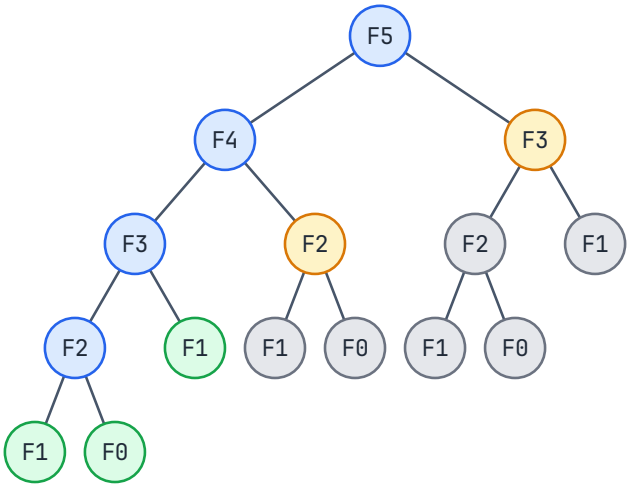


Figure 11.1 The 15 calls of fibNaive(5). The subtree for F(3) appears twice and the one for F(2) three times. With a memo, the calls shaded amber return a stored answer at once and the grey calls are never made: 9 calls remain, as Theorem 11.1 says $(2 \cdot 5 - 1)$.

Two properties made the rescue possible, and every dynamic programme relies on both.

DEFINITION 11.1 Overlapping subproblems, optimal substructure, dynamic programming

A recursive solution has **overlapping subproblems** when the same smaller instance is solved many times, as $F(3)$ is above. An optimisation problem has **optimal substructure** when an optimal solution of an instance is built from optimal solutions of smaller instances. **Dynamic programming** solves each distinct subproblem once, stores its answer, and combines stored answers into the answers of larger subproblems. Storing the answers of a recursive function is called **memoisation**; filling a table in an explicit order is called tabulation.

Without overlap, memoisation saves nothing: merge sort splits into disjoint halves and is a divide-and-conquer algorithm, not a DP. Without optimal substructure, stored answers to smaller problems are useless: the longest *simple* path between two cities is not made of longest simple paths between intermediate cities, and no known polynomial DP solves it.

11.2 The recipe

Every DP in this book is written down in the same five parts before any code is typed.

1. **State**, in words: what one table entry means. "best[i] = the most money from houses $0..i-1$." If you cannot say it in one sentence, the code will not be right.
2. **Transition**: how a state is computed from smaller states. This is the recurrence.
3. **Base cases**: the states that are answered directly, usually the empty prefix or capacity 0.
4. **Evaluation order**: an order in which every state comes after all the states its transition reads.
5. **Answer**: which state, or which combination of states, answers the original question.

The **state** and the **transition** determine the cost: **time = number of states × work per transition**, and **memory = number of states that must be stored at once**. House Robber has $n + 1$ states and two options per state, so $O(n)$; the 0/1 knapsack has $(n + 1)(C + 1)$ states and two options, so $O(nC)$.

It helps to picture the states as the nodes of a directed graph, with an arrow from each state to every state its transition reads. The evaluation order is then a topological order of this graph (Chapter 15), and the

graph must have no cycle. When you find yourself writing a transition that reads a state "at the same size", the state is missing information: add a dimension until every transition moves to strictly smaller states.

A proof template

A DP is correct when two things hold: the recurrence is true, and the program evaluates it in a valid order. The second part has the same proof for every DP.

THEOREM 11.2 Correctness of a tabulated DP

Suppose that (a) every base state is assigned its true value, (b) for every other state s, the transition computes the true value of s from the true values of states that precede s in the evaluation order, and (c) the program visits the states in that order. Then every table entry equals the true value of its state.

Proof. By strong induction along the evaluation order. The first states visited are base states, correct by (a). When the program reaches a non-base state s, every state that the transition reads precedes s, so by (c) it has already been filled in and, by the induction hypothesis, holds its true value; by (b) the value computed for s is then its true value. ∎

All the real work is in (b): proving the recurrence. The method is always the same. Look at the **last decision** of a solution for state s (rob the last house or not; take the last item or not; which coin came last). Then show three things:

- every solution of s falls into one of the cases;
- in each case, removing the last decision leaves a solution of a smaller state, and every solution of that smaller state can be extended by the same decision, so the best value in the case is the best value of the smaller state plus the value of the decision (this is optimal substructure, proved by "cut and paste": if the remaining part were not optimal, replacing it by a better one would improve the whole);
- for counting problems, the cases are disjoint and the correspondence is one-to-one, so counts add.

Top-down or bottom-up?

The same recurrence can be evaluated as a memoised recursion (top-down) or as loops over a table (bottom-up). In Java the choice has practical consequences.

Table 11.1 Top-down and bottom-up evaluation in Java

	Top-down (recursion + memo)	Bottom-up (loops)
evaluation order	found automatically by the recursion	you must choose a valid one
states computed	only those reachable from the answer	all states in the table
call stack	as deep as the longest chain of states	none
memo cost	array: cheap; `HashMap`: boxing and hashing	plain array access
rolling arrays (less memory)	awkward	easy

Two measurements make the table concrete. The top-down House Robber of Listing 11.2 recurses once per house; on the test machine, with Java's default 1 MB thread stack, it threw `StackOverflowError` at depths between about 10^4 and $1.7 \cdot 10^4$ houses (the exact limit depends on how far the JIT compiler had optimised the method), while the loop version handles 10^6 houses without effort. And a top-down 0/1 knapsack with 100 items and capacity 10^4 visited 483 608 states in about 6 ms with a `long[][]` memo, but needed about 100 ms with a `HashMap<Long, Long>` memo, and the bottom-up loop of Listing 11.5 took a few milliseconds.

JAVA NOTE

Write the recurrence top-down first if that is how you think, then convert it to loops when the depth can exceed about 10^4 or the states number in the millions. If the exam clock does not allow a rewrite, run the recursion on a thread with a large stack:

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(null, Main::solve, "solver", 1 << 27);   // 128 MB of stack
        t.start();
        t.join();
    }

    static void solve() {
        // deep recursion is safe here
    }
}
```

On the test machine the same top-down House Robber ran on 10^6 houses inside such a thread.

11.3 One-dimensional DP: House Robber

A street has n houses in a row; house i holds $a[i] \geq 0$. You may rob any set of houses that contains no two neighbours. What is the most you can take? This is LeetCode 198 (House Robber), and it is the cleanest first example of the recipe.

A greedy rule fails at once. “Take the richest house that is still allowed” picks the middle house of [3, 4, 3] and ends with 4, while the two outer houses give 6. The richest house can block two houses that are together worth more.

THEOREM 11.3 The House Robber recurrence

Let best[i] be the most money that can be robbed from houses 0..$i-1$. Then best[0] = 0, best[1] = a[0], and for $i \geq 2$, best[i] = max(best[$i-1$], best[$i-2$] + a[$i-1$]).

Proof. The base cases are the empty street and a single house. For $i \geq 2$, split the allowed sets of houses 0..$i-1$ by the last decision, whether house $i-1$ is robbed. If it is not, the set is an allowed set of houses 0..$i-2$, and every such set is allowed here, so the best in this case is best[$i-1$]. If it is, house $i-2$ is not robbed, so the set is house $i-1$ together with an allowed set of houses 0..$i-3$; conversely, adding house $i-1$ to any allowed set of houses 0..$i-3$ gives an allowed set. The best in this case is therefore $a[i-1]$ + best[$i-2$]. The two cases cover all sets, so best[i] is the larger of the two. ∎

The handbook template writes the recipe as comments and gives both evaluations.

Listing 11.2 House Robber bottom-up and top-down

```
// House Robber. Write these four lines BEFORE any code:
// 1. State:      best[i] = most money from houses 0..i-1
// 2. Transition: best[i] = max(best[i-1], best[i-2] + a[i-1])   (skip house i-1, or rob it)
// 3. Base:       best[0] = 0, best[1] = a[0]
// 4. Answer:     best[n]
static long robBottomUp(int[] a) {
    int n = a.length;
    long[] best = new long[n + 1];
    if (n > 0) best[1] = a[0];
    for (int i = 2; i <= n; i++) best[i] = Math.max(best[i - 1], best[i - 2] + a[i - 1]);
    return best[n];
}
```

```java

// Same recurrence top-down with a memo. Often easier to write first; fine while depth < ~10^4.
static long[] robMemo;

static long robTopDown(int[] a) {
    robMemo = new long[a.length + 1];
    Arrays.fill(robMemo, -1);
    return rob(a, a.length);
}

static long rob(int[] a, int i) {
    if (i == 0) return 0;
    if (i == 1) return a[0];
    if (robMemo[i] != -1) return robMemo[i];
    return robMemo[i] = Math.max(rob(a, i - 1), rob(a, i - 2) + a[i - 1]);
}
```

The evaluation order of the loop is increasing i, and best[i] reads only best[$i-1$] and best[$i-2$], so Theorem 11.2 applies. The cost is O(n) time and O(n) memory.

EXAMPLE 11.1 Robbing [6, 1, 2, 7, 3, 5]

Each entry compares "skip house $i-1$" (best[$i-1$]) with "rob it" (best[$i-2$] + a[$i-1$]).

i	a[i-1]	skip = best[i-1]	rob = best[i-2] + a[i-1]	best[i]
0				0
1	6			6
2	1	6	0 + 1 = 1	6
3	2	6	6 + 2 = 8	8
4	7	8	6 + 7 = 13	13
5	3	13	8 + 3 = 11	13
6	5	13	13 + 5 = 18	18

The answer is 18, from houses 0, 3 and 5 (6 + 7 + 5). Reading the table backwards recovers them: best[6] came from "rob", so house 5 is robbed and we continue from best[4]; best[4] came from "rob", so house 3 is robbed and we continue from best[2]; best[2] = best[1] came from "skip"; best[1] robs house 0.

Rolling variables

best[i] reads only the two entries before it, so the array can shrink to two variables that "roll" forward.

Listing 11.3 House Robber in O(1) extra memory

```java
// best[i] needs only best[i-1] and best[i-2]: keep two variables instead of an array
static long robRolling(int[] a) {
    long prev2 = 0, prev1 = 0;                  // best[i-2] and best[i-1], starting at i = 1
    for (int x : a) {
        long cur = Math.max(prev1, prev2 + x);  // skip this house, or rob it
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Before the first iteration, prev2 = 0 stands for best[−1], which is treated as 0, and prev1 = best[0] = 0; the first iteration therefore computes max(0, 0 + a[0]) = best[1], and from then on the loop repeats the recurrence. On [6, 1, 2, 7, 3, 5] the successive values of `cur` are 6, 6, 8, 13, 13, 18. The price of rolling is that the table is gone, so the robbed houses can no longer be read back.

LeetCode 740 (Delete and Earn) is this recurrence in disguise once the points are added up per value: neighbouring *values* then exclude each other just as neighbouring houses do.

11.4 The 0/1 knapsack

DEFINITION 11.2 0/1 knapsack

Given n items with weights $w[i] \geq 1$ and values $v[i] \geq 0$ and a capacity C, the **0/1 knapsack** problem asks for the largest total value of a set of items whose total weight is at most C. Each item is taken at most once.

Take the items $(w, v) = (1, 1), (3, 4), (4, 5), (5, 7)$ and capacity 7. Greedy by value per unit weight takes (5, 7) first (ratio 1.4), and then only (1, 1) still fits, for a total of 8. The best choice is (3, 4) and (4, 5), worth 9. A heavy item with a good ratio can waste capacity that two lighter items would have filled. So the choice of each item depends on what the other items leave, which is exactly what a table can track.

THEOREM 11.4 The knapsack recurrence

Let best[i][c] be the largest value of a set of items from 0..i − 1 with total weight at most c. Then best[0][c] = 0 for every c, and for i ≥ 1, best[i][c] = best[i − 1][c] if w[i − 1] > c, and otherwise best[i][c] = max(best[i − 1][c], best[i − 1][c − w[i − 1]] + v[i − 1]).

Proof. With no items the only set is empty, worth 0. For $i \geq 1$, split the feasible sets by the last decision: whether item $i - 1$ is in the set. Sets without it are exactly the feasible sets of items $0..i - 2$ with capacity c, best value best$[i - 1][c]$. A set with it has the form $\{i - 1\} \cup T$, where T uses items $0..i - 2$ and has weight at most $c - w[i - 1]$; this is possible only when $w[i - 1] \leq c$. Conversely every such T gives a feasible set. The value is $v[i - 1]$ plus the value of T, which is largest when T is optimal for capacity $c - w[i - 1]$. The two cases cover all feasible sets. ∎

Listing 11.4 The 0/1 knapsack as a two-dimensional table

```
// best[i][c] = most value from items 0..i-1 with total weight <= c
static long[][] knapsackTable(int[] w, int[] v, int cap) {
    int n = w.length;
    long[][] best = new long[n + 1][cap + 1];            // row 0 (no items) is all zeros
    for (int i = 1; i <= n; i++)
        for (int c = 0; c <= cap; c++) {
            best[i][c] = best[i - 1][c];                    // skip item i-1
            if (w[i - 1] <= c)                              // or take it
                best[i][c] = Math.max(best[i][c], best[i - 1][c - w[i - 1]] + v[i - 1]);
        }
    return best;
}
```

Row i reads only row $i - 1$, so filling the rows in increasing order is a valid evaluation order. The table has $(n + 1)(C + 1)$ entries, each computed in O(1): O(nC) time and memory.



Figure 11.2 The table best[i][c] for items (1, 1), (3, 4), (4, 5), (5, 7) and capacity 7. Each entry looks straight up (skip the item) and up-left by the item's weight (take it): best[3][7] = max(best[2][7], best[2][3] + 5) = max(5, 9) = 9. The blue cells are the walk back of Section 11.7, which recovers the chosen items 1 and 2.

One row is enough, if you walk it backwards

Row i depends only on row $i - 1$, so you can keep a single array dp[0..C] and overwrite it in place, item by item. The only danger is that an entry you read might already have been overwritten with the new row's value. The handbook template avoids that by looping the capacity **downwards**.

Listing 11.5 0/1 and unbounded knapsack with one array

```java
// 0/1 knapsack: each item at most once -> loop capacity DOWNWARDS
static long knapsack01(int[] w, int[] v, int cap) {
    long[] dp = new long[cap + 1];                // dp[c] = best value with weight <= c
    for (int i = 0; i < w.length; i++)
        for (int c = cap; c >= w[i]; c--) dp[c] = Math.max(dp[c], dp[c - w[i]] + v[i]);
    return dp[cap];
}

// Unbounded: each item any number of times -> loop capacity UPWARDS
static long knapsackUnbounded(int[] w, int[] v, int cap) {
    long[] dp = new long[cap + 1];
    for (int i = 0; i < w.length; i++)
        for (int c = w[i]; c <= cap; c++) dp[c] = Math.max(dp[c], dp[c - w[i]] + v[i]);
    return dp[cap];
}
```

THEOREM 11.5 **Why the downward loop uses each item once**

In `knapsack01`, just before the pass for item i begins, dp[c] = best[i][c] for every c. During that pass, when the loop reaches capacity c, the entry dp[$c - w[i]$] still holds best[i][$c - w[i]$], so the assignment stores best[i + 1][c]. After the pass, dp[c] = best[i + 1][c] for every c.

Proof. Before the first item, dp is all zeros, which is row 0. Assume dp holds row i when the pass for item i starts. The pass writes only the entries $c = C, C - 1, \ldots, w[i]$, in that order, each once. When it is at c, it has written exactly the entries greater than c. The entry it reads, $c - w[i]$, is smaller than c because $w[i] \geq 1$, so it has not been written in this pass and still holds best[i][$c - w[i]$]; dp[c] itself also still holds best[i][c]. So the assignment computes max(best[i][c], best[i][$c - w[i]$] + $v[i]$), which is best[i + 1][c] by Theorem 11.4. Entries with $c < w[i]$ are not touched, and for them best[i + 1][c] = best[i][c] because the item does not fit. So after the pass, dp holds row i + 1. ∎

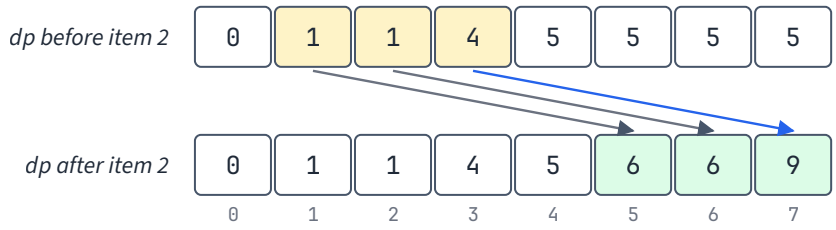


Figure 11.3 The pass for item 2 (w = 4, v = 5) of the example. Each new entry for c ≥ 4 reads the old entry four places to its left. Because c runs downwards, those old entries are still unchanged when they are read: new dp[7] = max(5, old dp[3] + 5) = 9.

Loop upwards and the argument breaks in an instructive way. With one item of weight 2 and value 3 and capacity 4:

```
downwards (0/1):     c=4: dp[2]+3 = 3   c=3: 3   c=2: 3    -> dp[4] = 3  (used once)
upwards (unbounded): c=2: 3   c=3: 3   c=4: dp[2]+3 = 6    -> dp[4] = 6  (used twice)
```

Going up, dp[2] has already been updated to include the item when dp[4] reads it, so dp[4] takes the item a second time. For the 0/1 problem that is a bug; for the unbounded problem of the next section it is exactly what is wanted.

The cost of `knapsack01` is $O(nC)$ time and $O(C)$ memory. This is called **pseudo-polynomial**: it is polynomial in the *value* of C, not in the number of digits needed to write it. With $n = 100$ and $C = 10^5$ it does 10^7 steps; with $C = 10^9$ it is hopeless. When capacities are huge but values are small, swap the roles: let dp[x] be the smallest weight that achieves value exactly x, and answer with the largest x whose weight fits.

INFOSYS LENS

The plan's note for this day says that coin change, subset sum and 0/1 knapsack are among the most-reported Infosys DP problems. One of the reported SP repeats (Day 24) is a knapsack whose "capacity" is an XOR value: there the new entry for x reads the old entry for $x \oplus a$, which may lie on either side of x. Neither loop direction then protects the old values, and the safe method is to write each pass into a fresh array. Theorem 11.5 shows exactly which property, $c - w < c$, the in-place trick needs.

11.5 Unbounded knapsack and coin change

DEFINITION 11.3 Unbounded knapsack

The **unbounded knapsack** problem is the knapsack problem in which every item may be taken any number of times.

THEOREM 11.6 The unbounded recurrence and the upward loop

Let $U_i[c]$ be the largest value of a multiset of items from $0..i-1$ with weight at most c. Then $U_0[c] = 0$ and, for $w[i-1] \le c$, $U_i[c] = \max(U_{i-1}[c], U_i[c - w[i-1]] + v[i-1])$; otherwise $U_i[c] = U_{i-1}[c]$. The upward pass of `knapsackUnbounded` computes exactly this.

Proof. A multiset either contains no copy of item $i-1$, and then it is a multiset of items $0..i-2$ (best $U_{i-1}[c]$), or at least one copy. In the second case remove one copy: what remains is a multiset of items $0..i-1$, still allowed to contain item $i-1$, of weight at most $c - w[i-1]$, and every such multiset plus one copy is feas-

ible. That gives the recurrence, which now reads the *current* row at a smaller capacity. In the upward pass, when the loop reaches c, the entries below c have already been updated in this pass, so dp[$c - w[i-1]$] holds $U_i[c - w[i-1]]$, while dp[c] still holds $U_{i-1}[c]$. By induction on c the pass stores $U_i[c]$. ∎

Fewest coins

Coin change asks: with unlimited coins of the given denominations, what is the fewest number of coins that make an amount exactly (LeetCode 322, Coin Change)? Each coin has "value" 1 and we minimise, and the amount must be hit exactly, so unreachable amounts need a special value.

Listing 11.6 Fewest coins for an amount

```
// Fewest coins to make amount (-1 if impossible)
static int minCoins(int[] coins, int amount) {
    int INF = Integer.MAX_VALUE / 2;
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, INF);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] >= INF ? -1 : dp[amount];
}
```

CLAIM 11.7 **Correctness of minCoins**

After the loop, dp[a] is the fewest coins summing to exactly a, or at least INF when a cannot be made.

Proof. dp[0] = 0 (no coins). For $a \geq 1$, look at any way to make a and pick one of its coins c, say the last one in some order. The other coins make $a - c$, so the way uses at least dp[$a - c$] + 1 coins; conversely, a best way for $a - c$ plus one coin c makes a. So dp[a] = min over coins $c \leq a$ of dp[$a - c$] + 1, and the outer loop over increasing a fills every dp[$a - c$] before dp[a]. Unreachable amounts keep values of at least INF, because INF + 1 is still at least INF, and INF = $2^{30} - 1$ leaves room for the +1 without overflow. ∎

PITFALL

Starting from `Integer.MAX_VALUE` instead of `Integer.MAX_VALUE / 2` makes `dp[a - c] + 1` overflow to a large negative number, which then wins every `Math.min`. The card's trap list puts it first for a reason.

Greedy "largest coin first" works for the coins in your wallet but not in general. With coins {1, 3, 4} and amount 6 it takes 4 + 1 + 1, three coins, while 3 + 3 uses two. The table for amounts 0..6 is [0, 1, 2, 1, 1, 2, 2]. The cost is O(amount × number of coins).

For a minimum, the order of the loops does not matter: a multiset of coins and any rearrangement of it use the same number of coins. For counting, the order is everything.

Combinations or sequences?

How many ways can you make amount 3 from coins 1 and 2? The answer depends on whether 1 + 2 and 2 + 1 are the same way. As **combinations** (multisets) there are 2: {1, 1, 1} and {1, 2}. As **ordered sequences** there are 3: 1 + 1 + 1, 1 + 2 and 2 + 1. LeetCode 518 (Coin Change II) counts combinations. The two counts come from the same update, `ways[a] += ways[a - c]`, with the two loops in different orders.

Listing 11.7 Counting combinations and ordered sequences

```java
// Number of coin combinations (order doesn't matter): coins in the OUTER loop
static long countCombinations(int[] coins, int amount) {
    long[] ways = new long[amount + 1];
    ways[0] = 1;
    for (int c : coins)
        for (int a = c; a <= amount; a++) ways[a] += ways[a - c];
    return ways[amount];
}

// Number of ordered sequences (1+2 and 2+1 differ): amount in the OUTER loop
static long countSequences(int[] coins, int amount) {
    long[] ways = new long[amount + 1];
    ways[0] = 1;
    for (int a = 1; a <= amount; a++)
        for (int c : coins) if (c <= a) ways[a] += ways[a - c];
    return ways[amount];
}
```

THEOREM 11.8 **What each loop order counts**

Let the distinct coin values be c_1, …, c_k in the order of the array. (1) In `countCombinations`*, after the outer loop has processed $c_1..c_j$, ways[a] is the number of multisets of coins from $\{c_1, \ldots, c_j\}$ with sum a. (2) In* `countSequences`*, ways[a] is the number of sequences of coins with sum a.*

Proof. (1) By induction on j. Before any coin, ways = [1, 0, 0, …]: only the empty multiset, with sum 0. For the pass with coin c_j, split the multisets with sum a into those that contain no c_j, counted by the old ways[a] (induction hypothesis), and those that contain at least one. Removing one copy of c_j from a multiset of the second kind gives a multiset from $\{c_1, \ldots, c_j\}$ with sum $a - c_j$, and adding a copy reverses it, so the second kind has as many members as there are multisets from $\{c_1..c_j\}$ with sum $a - c_j$. The inner loop runs upwards, so when it reaches a, ways[$a - c_j$] already holds that new count (induction on a within the pass), and ways[a] still holds the old one. The update adds them. The two kinds are disjoint, so the sum is the total.

(2) Split the sequences with sum $a \geq 1$ by their last coin c. Removing it gives a sequence with sum $a - c$, one-to-one, so the count is the sum over $c \leq a$ of the count for $a - c$. The outer loop over increasing a computes every smaller amount first. ∎

A counting argument shows how far apart the two numbers can be. A multiset that uses coin values with multiplicities m_1, …, m_k can be written as $(m_1 + \cdots + m_k)! / (m_1! \cdots m_k!)$ different sequences. For coins {1, 2} and amount 3, the multiset {1, 1, 1} gives 3!/3! = 1 sequence and {1, 2} gives 2!/(1! 1!) = 2, so 2 combinations become 3 sequences. For coins {1, 2, 5} and amount 5 there are 4 combinations ({5}, {2, 2, 1}, {2, 1, 1, 1}, {1, 1, 1, 1, 1}) but 9 sequences.

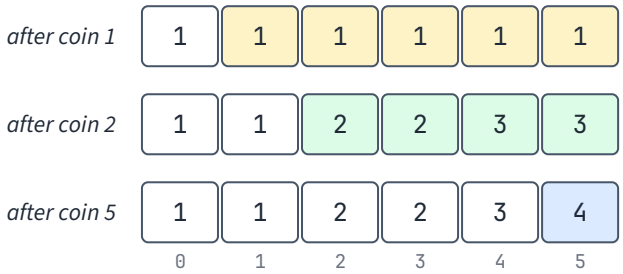


Figure 11.4 countCombinations for coins {1, 2, 5} and amounts 0..5. Each row is the array after one pass of the outer loop. The coloured cells changed in that pass; for example in the pass for coin 2, ways[4] = 1 + ways[2] = 1 + 2 = 3 counts {1,1,1,1}, {1,1,2} and {2,2}.

PITFALL

Swapping the two loops of `countCombinations` does not crash and does not look wrong; it silently counts sequences. For coins {1, 2, 5} and amount 5 it returns 9 instead of 4. Before you submit a counting DP, compute one tiny case by hand, as above.

11.6 Subset sum and partitions

DEFINITION 11.4 Subset sum

The **subset sum** problem asks, for integers $a[0..n-1] \geq 0$ and a target T, whether some subset of the elements has sum exactly T. The counting version asks how many subsets do.

Subset sum is the 0/1 knapsack with the value of each item equal to its weight, asked as a yes/no question. The table becomes boolean, `max` becomes OR, and the loop still runs downwards so that each element is used once.

Listing 11.8 Subset sum, deciding and counting

```java
// Can some subset sum to target?
static boolean subsetSum(int[] a, int target) {
    boolean[] can = new boolean[target + 1];
    can[0] = true;
    for (int x : a)
        for (int s = target; s >= x; s--) if (can[s - x]) can[s] = true;
    return can[target];
}

// How many subsets sum to exactly target? Counted modulo 1e9+7; zeros are allowed.
static long countSubsets(int[] a, int target) {
    final long MOD = 1_000_000_007L;
    long[] ways = new long[target + 1];
    ways[0] = 1;                                  // the empty subset
    for (int x : a)
        for (int s = target; s >= x; s--) ways[s] = (ways[s] + ways[s - x]) % MOD;
    return ways[target];
}
```

Correctness is Theorem 11.5 with OR in place of max: after the pass for element x, can[s] is true exactly when some subset of the elements so far sums to s, because such a subset either avoids x (old can[s]) or is x plus a subset of the earlier elements summing to $s - x$ (old can[$s - x$]). The counting version adds the two disjoint cases instead. An element equal to 0 is handled correctly: its pass runs down to $s = 0$ and doubles every count, once for each choice of taking the zero or not. Both cost $O(nT)$ time and $O(T)$ memory.

EXAMPLE 11.2 Reachable sums of {3, 5, 6, 2}

The sums for which `can` is true after each element:

```
after 3:  0 3
after 5:  0 3 5 8
after 6:  0 3 5 6 8 9 11 14
after 2:  0 2 3 5 6 7 8 9 10 11 13 14 16
```

Each line is the previous line together with the previous line shifted by the new element. Of the sums 0..16, all are reachable except 1, 4, 12 and 15. The total is 16, and 8 is reachable (3 + 5, or 6 + 2), so this array can be split into two halves of equal sum.

That last remark is LeetCode 416 (Partition Equal Subset Sum). A split into two parts with equal sums exists exactly when the total S is even and some subset sums to $S/2$, because the complement of a subset with sum $S/2$ also has sum $S/2$. So the problem is one call of `subsetSum` after a parity check.

Signs: the algebra behind Target Sum

LeetCode 494 (Target Sum) puts a + or a − in front of every element and counts the sign patterns whose total is a given *target*. Let P be the sum of the elements with +, and N the sum of those with −. Then

$P - N$ = target and $P + N = S$ (the total), so $P = (S + \text{target}) / 2$.

A sign pattern is the same thing as the subset of elements that get +, so the count of patterns equals the number of subsets with sum $(S + \text{target})/2$. That number must be an integer between 0 and S: if S + target is odd, or |target| > S, the answer is 0. For example, with elements [2, 1, 3, 1] and target 1, $S = 7$ and $P = 4$. The subsets with sum 4 are {1, 3}, {3, 1′} and {2, 1, 1′}, and they match the three patterns −2 + 1 + 3 − 1, −2 − 1 + 3 + 1 and +2 + 1 − 3 + 1. Zeros need no special care: `countSubsets` already doubles the count for each zero, matching +0 and −0.

Two groups as equal as possible: Last Stone Weight II

In LeetCode 1049 (Last Stone Weight II) you repeatedly smash two stones $x \le y$; both vanish if they are equal, otherwise one stone of weight $y - x$ remains. The question is the smallest possible weight of the last stone (0 if none is left). The plan's hint turns it into a partition problem, and the reason is worth a proof.

THEOREM 11.9 Smashing stones is splitting them into two groups

The smallest possible final weight equals the smallest value of $|S_A - S_B|$ over all ways to split the stones into two groups A and B with sums S_A and S_B.

Proof. *Every final weight is such a difference.* Each stone ever present is a signed sum of original stones: an original stone is +itself, and smashing $x \le y$ produces $y - x$, a signed sum whose terms keep their signs from y and flip them from x. So the last stone, or 0, equals $|\Sigma \pm w_i|$ for some signs, which is $|S_A - S_B|$ for the split "plus signs against minus signs".

The smallest difference is achievable. Take a split (A, B) with the smallest difference $d = S_A - S_B \ge 0$. While both groups are non-empty, smash any stone a of A against any stone b of B, and put the remainder, if any, into the group of the heavier stone. If $a \ge b$, A loses b in total and B loses b; if $a < b$, both lose a. Either way $S_A - S_B$ stays d. Every current stone is a signed sum of original stones. Give each original the group of the current stone that contains it if its sign there is +, and the other group if it is −. Smashing never changes this assignment, so it always reproduces the starting split (A, B), whose difference is d. Each smash removes at least one stone, so eventually one group is empty. Then the other group holds stones of total $d \ge 0$, so it is A (or both are empty and $d = 0$). If A held two or more stones, with some stone x where $0 < x < d$, moving x to B would give a split of the originals with difference $|d - 2x| < d$, contradicting minimality. So A holds at most one stone, of weight d, and smashing within A is never needed. ∎

So the answer is $S - 2s$, where s is the largest subset sum not exceeding $S/2$: one boolean subset-sum table. For stones [4, 9, 2, 6], $S = 21$ and the reachable sums up to 10 are 0, 2, 4, 6, 8, 9 and 10, so the answer is 21 − 20 = 1. The proof's procedure, started from the split A = {9, 2} and B = {4, 6}, really does produce it:

```
smash 9 (A) with 6 (B): A gets 3;  A = {2, 3} sum 5,  B = {4} sum 4,  difference 1
smash 3 (A) with 4 (B): B gets 1;  A = {2} sum 2,     B = {1} sum 1,  difference 1
smash 2 (A) with 1 (B): A gets 1;  A = {1},           B = {}          -> last stone 1
```

The harness for this chapter compared $S - 2s$ with an exhaustive search over all smashing orders on 3000 random sets of up to six stones.

11.7 Solutions, counts, memory and bit-parallel subset sums

Rebuilding the chosen items

The value best[n][C] says how good the best choice is, not what it is. The two-dimensional table of Listing 11.4 keeps enough information to recover it, by asking at each row which case of Theorem 11.4 produced the entry.

Listing 11.9 Recovering an optimal set of items from the table

```
// Which items does an optimal 0/1 knapsack take? Walk back through the table of Listing 11.4.
static List<Integer> chosenItems(long[][] best, int[] w, int cap) {
    List<Integer> items = new ArrayList<>();
    int c = cap;
    for (int i = w.length; i >= 1; i--) {
        if (best[i][c] != best[i - 1][c]) {       // the value changed, so item i-1 was taken
            items.add(i - 1);
            c -= w[i - 1];
        }
    }
    Collections.reverse(items);
    return items;
}
```

CLAIM 11.10 **The walk back returns an optimal set**

At the start of each iteration, the items already collected have total weight $C - c$ and total value best[n][C] − best[i][c]. At the end the collected items form an optimal set.

Proof. Initially $c = C$ and $i = n$, so both claims read 0. If best[i][c] = best[$i - 1$][c], skipping item $i - 1$ is optimal for state (i, c), and moving to $(i - 1, c)$ keeps the invariant. Otherwise the entry was produced by the "take" case, so best[i][c] = best[$i - 1$][$c - w[i - 1]$] + $v[i - 1]$ and $w[i - 1] \le c$; collecting the item and moving to $(i - 1, c - w[i - 1])$ again keeps the invariant. At $i = 0$, best[0][c] = 0, so the collected value is best[n][C] and the weight is at most C. ∎

On the example of Figure 11.2 the walk visits (4, 7) → (3, 7) → (2, 3) → (1, 0) → (0, 0) and returns items 1 and 2, of weights 3 and 4 and values 4 and 5. The one-dimensional table cannot do this, because it keeps only the last row. If memory is tight, store a boolean `took[i][c]` (one bit of decision per state) next to the 1-D array.

Counting modulo $10^9 + 7$

Counts in DP grow exponentially. The number of ordered sums of 1s and 2s equal to a is the Fibonacci number $F(a + 1)$, so `countSequences` with coins {1, 2} overflows `long` from amount 92 on. The number of subsets of 100 ones with sum 50 is $C(100, 50) \approx 1.0 \cdot 10^{29}$; `countSubsets` returns it modulo $10^9 + 7$ as 538 992 043, which matches the exact value reduced. Following Chapter 10, reduce after every addition: the sum of two residues is below $2.1 \cdot 10^9$, so `ways[s] + ways[s - x]` cannot overflow a `long` before the `%`. Problems that print counts without a modulus promise that the answer fits; the promise covers only the final value, and intermediate entries may be larger, so keep them in `long`.

Estimating memory

Multiply the number of entries by the size of one entry: 8 bytes for `long`, 4 for `int`, 1 for `boolean`. A `long[10_000_001]` takes 80 MB, an `int[]` of the same length 40 MB and a `boolean[]` 10 MB. A two-dimensional table `long[101][100001]` takes 80.8 MB, which is why the 1-D knapsack matters: the same problem fits in 0.8 MB. Before you allocate, compare with the memory limit (often 256 MB) and remember that Java needs some of it for itself.

Bit-parallel subset sum

A boolean array wastes seven bits of every byte, and the inner loop of `subsetSum` handles one sum at a time. Store the reachable sums as the bits of a `long[]` instead. Then "the previous set together with the previous set shifted by x" from Example 11.2 becomes a shift and an OR on whole 64-bit words, and one machine instruction processes 64 sums.

Listing 11.10 Subset sums with 64 sums per machine word

```
// Bit s of the result is 1 <=> some subset of a sums to s (0 <= s <= maxSum). O(n * maxSum / 64).
static long[] subsetSumsBits(int[] a, int maxSum) {
    int words = maxSum / 64 + 1;
    long[] reach = new long[words];
    reach[0] = 1L;                                          // sum 0: the empty subset
    for (int x : a) {
        int ws = x >>> 6, bs = x & 63;                      // shift by x = 64 * ws + bs bits
        for (int i = words - 1; i >= ws; i--) {             // high to low: reads old words only
            long moved = reach[i - ws] << bs;
            if (bs != 0 && i - ws - 1 >= 0) moved |= reach[i - ws - 1] >>> (64 - bs);
            reach[i] |= moved;
        }
    }
    int extra = (maxSum & 63) + 1;                          // forget sums above maxSum
    if (extra < 64) reach[words - 1] &= (1L << extra) - 1;
    return reach;
}

static boolean canReach(long[] reach, int s) {
    return (reach[s >>> 6] >>> (s & 63) & 1L) == 1L;
}
```

Shifting the whole bit string left by $x = 64 \cdot \text{ws} + \text{bs}$ moves bit s to bit $s + x$. Word i of the shifted string is made of the low part of word $i - \text{ws}$ shifted left by bs and the high part of word $i - \text{ws} - 1$ shifted right by $64 - \text{bs}$. The loop over i runs from the highest word down for the same reason as Theorem 11.5: it reads only words with index $\le i$, and none of those has been updated in this pass yet. Bits shifted beyond the last word are lost, which is harmless because they stand for sums above maxSum, and the final mask clears the few unused bits of the last word. The `>>>` operator matters: `>>` would copy the sign bit into the vacated positions.

On the test machine, with 200 random elements up to $2 \cdot 10^4$ and a target near 10^6, the boolean loop did $2 \cdot 10^8$ steps in about 230 ms, while Listing 11.10 took about 7 ms. That factor of about 30 can turn a time-out into a pass.

JAVA NOTE

`java.util.BitSet` has `or`, `and` and `get(from, to)`, but no left-shift method, so it cannot do this update directly. A short alternative is `java.math.BigInteger`, whose `shiftLeft` and `or` work on arbitrary bit strings; it allocates a new number on every step, so it was about 3 to 10 times slower than Listing 11.10 in the same test, but still far faster than the boolean loop.

```java
int[] a = {3, 5, 6, 2};
BigInteger reach = BigInteger.ONE;                    // bit 0: the empty subset
for (int x : a) reach = reach.or(reach.shiftLeft(x));
boolean canMake8 = reach.testBit(8);                  // true: 3 + 5 or 6 + 2
```

Summary

- Dynamic programming applies when a recursion solves the same subproblems repeatedly (overlapping subproblems) and optimal answers are assembled from optimal parts (optimal substructure). Memoised Fibonacci makes $2n - 1$ calls instead of $2F(n + 1) - 1$.
- Write five lines before coding: state in words, transition, base cases, evaluation order, answer. Time is states × transitions; memory is the states kept at once.
- Correctness = a proved recurrence (split by the last decision; cases must cover everything, and for counts must be disjoint) + an evaluation order in which every state follows the states it reads.
- In Java, prefer loops when the recursion depth can exceed about 10^4 and arrays over `HashMap` memos; a 128 MB stack thread is the emergency fallback.
- House Robber: best[i] = max(best[$i - 1$], best[$i - 2$] + a[$i - 1$]); two rolling variables suffice.
- 0/1 knapsack: one array with the capacity loop running downwards, so every read sees the previous item's row. Upwards gives the unbounded knapsack. Cost O(nC), pseudo-polynomial.
- Coin change: the minimum ignores loop order; counting does not. Coins outside count combinations, amounts outside count ordered sequences.
- Partition, Target Sum and Last Stone Weight II are subset sum in disguise: total/2, (S + target)/2, and the largest reachable sum not above $S/2$.
- Keep the 2-D table (or a bit per decision) to rebuild a solution, reduce counts modulo $10^9 + 7$ after every addition, estimate memory before allocating, and use 64-bit words for boolean subset sums.

Key terms

overlapping subproblems p. 164
optimal substructure p. 164
dynamic programming p. 164
memoisation p. 164
0/1 knapsack p. 168
unbounded knapsack p. 170
subset sum p. 173
state (DP) p. 164
transition (DP) p. 164

Review questions

1. Why does memoisation turn Fibonacci from exponential into linear time, but give no speed-up to merge sort?
2. State the five parts of the recipe for the 0/1 knapsack table best[i][c]. How many states are there, and how many transitions per state?
3. In the proof of Theorem 11.2, where exactly is the evaluation order used? Give an evaluation order for best[i][c] other than "row by row".
4. What goes wrong if the 0/1 knapsack loop runs the capacity upwards? Give the smallest input on which the answer changes.
5. Why can `minCoins` loop over amounts on the outside while `countCombinations` must loop over coins on the outside?
6. How many ordered sequences of coins from {1, 2, 5} sum to 5, and how many combinations? Explain the difference with the multinomial counting argument.

7. In Target Sum, why must (S + target) be even, and why is the answer 0 when |target| > S? How are elements equal to 0 handled?
8. In Listing 11.10, why does the loop over words run from high to low? What would go wrong with `>>` instead of `>>>`?
9. Estimate the memory of a `long[n + 1][cap + 1]` knapsack table with $n = 200$ and cap = 10^5. Would the 1-D version fit in 1 MB?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 "Dynamic programming": coin problems (including counting solutions), the knapsack problem and the general method. Free at cses.fi/book.
- Algorithms for Competitive Programming: "Introduction to Dynamic Programming" (cp-algorithms.com/dynamic_programming/intro-to-dp.html) and "Knapsack Problem" (cp-algorithms.com/dynamic_programming/knapsack.html), which also covers bounded knapsacks with multiple copies of each item.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Dynamic Programming*, for the elements of DP (optimal substructure and overlapping subproblems) and the cut-and-paste argument in full.

Problems

These are the problems for Day 11. Work them in Workbook Chapter 11, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
11.P1	LC 198 · House Robber	MEDIUM	CORE
11.P2	LC 322 · Coin Change	MEDIUM	CORE
11.P3	LC 518 · Coin Change II	MEDIUM	CORE
11.P4	LC 416 · Partition Equal Subset Sum	MEDIUM	CORE
11.P5	LC 494 · Target Sum	MEDIUM	BONUS
11.P6	LC 1049 · Last Stone Weight II	MEDIUM	BONUS
11.P7	LC 740 · Delete and Earn	MEDIUM	BONUS

CHAPTER 13 · DAY 13 OF 29 · THURSDAY 19 NOVEMBER 2026 · LEARNING DAY

13

Dynamic Programming III: Increasing Subsequences and Extra Dimensions

LIS in O(n log n), DP over values with a Fenwick tree, and an extra state for a side condition.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · Longest increasing subsequence
Before this chapter · Chapter 11 (the DP recipe), Chapter 6 (lower and upper bound)
Practice · Workbook Chapter 13 · 6 problems · core ≈ 4.0 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- find the longest increasing subsequence by an $O(n^2)$ DP and prove the recurrence;
- run patience sorting in O(n log n), state and prove the invariant of the tails array, and switch between strict and non-decreasing order with lower and upper bound;
- recover one optimal subsequence with parent pointers and report the best length ending at every position;
- combine increasing runs from both sides to solve mountain problems, including the condition that both sides have length at least 2;
- replace the binary search with a Fenwick tree of maxima over compressed values, and prove that it computes the same DP;
- handle two-key orders and side conditions by sorting cleverly or by adding a small state dimension.

A *subsequence* keeps some elements of an array in their original order and throws the rest away. Asking for the longest subsequence that rises at every step is one of the oldest problems in dynamic programming, and it keeps coming back in disguise: nesting boxes, scheduling jobs that must be ordered in two ways at once, deleting the fewest elements to make an array sorted or shaped like a mountain. Its Infosys form is the Hard problem of the official sample paper, which asks for a longest non-decreasing subsequence with a condition on the XOR of its elements.

This chapter develops the problem in layers. A quadratic DP comes first, because it is the version you can always derive under pressure. Patience sorting then brings the cost down to $O(n \log n)$ with a short loop whose correctness rests on one invariant. Two extensions follow: recovering the subsequence itself, and gluing an increasing run to a decreasing one. The second half changes the point of view from *positions* to *values*, which turns the DP into prefix-maximum queries on a Fenwick tree. That view handles weights, counts, two keys and extra conditions, and it is the form the sample paper needs.

13.1 Subsequences and the quadratic DP

DEFINITION 13.1 Subsequence and longest increasing subsequence

A **subsequence** of $a[0..n-1]$ is a sequence $a[i_1], a[i_2], \ldots, a[i_k]$ with $i_1 < i_2 < \cdots < i_k$. It is **strictly increasing** if $a[i_1] < a[i_2] < \cdots < a[i_k]$ and **non-decreasing** if each "<" is "≤". The **longest increasing subsequence** (LIS) is a strictly increasing subsequence of maximum length k.

A subsequence need not be contiguous; that is the difference from a *subarray*. For $a = [10, 9, 2, 5, 3, 7, 101, 18]$ the subsequence 2, 3, 7, 18 (indices 2, 4, 5, 7) is strictly increasing and has length 4, and no increasing subsequence is longer.

Every subset of the n indices gives one subsequence, so there are 2^n of them. Trying them all costs $O(2^n \cdot n)$: fine for $n \le 20$, which is exactly the partial-marks fallback for the sample Hard problem, and hopeless for $n = 30$, where $2^{30} \approx 1.07 \cdot 10^9$.

The state "ends at i"

Following the recipe of Chapter 11, first choose a state. The natural guess, "the LIS of the prefix $a[0..i]$", does not work on its own: to decide whether $a[i+1]$ can extend a subsequence you must know that subsequence's *last value*, and the prefix answer forgets it. Fixing where the subsequence ends fixes its last value.

DEFINITION 13.2 The ending-at state

For $0 \le i < n$, let $f[i]$ be the length of the longest strictly increasing subsequence whose last element is $a[i]$.

THEOREM 13.1 The quadratic recurrence

$f[i] = 1 + \max\{ f[j] : j < i \text{ and } a[j] < a[i] \}$, where the maximum of an empty set is 0. The LIS length is max over i of $f[i]$.

Proof. Take a longest increasing subsequence ending at $a[i]$. If it has length 1, then $f[i] = 1$, and the right-hand side is at least 1. Otherwise let $a[j]$ be its second-to-last element. Then $j < i$, $a[j] < a[i]$, and the subsequence without its last element is an increasing subsequence ending at $a[j]$, so its length $f[i] - 1$ is at most $f[j]$. Hence $f[i] \le 1 + \max\{\ldots\}$. Conversely, for any $j < i$ with $a[j] < a[i]$, appending $a[i]$ to a longest increasing subsequence ending at $a[j]$ gives an increasing subsequence of length $f[j] + 1$ ending at $a[i]$, so $f[i] \ge 1 + \max\{\ldots\}$. Every increasing subsequence ends somewhere, so the overall maximum is the maximum of the $f[i]$. ∎

Listing 13.1 Longest increasing subsequence by $O(n^2)$ dynamic programming

```
// f[i] = length of the longest strictly increasing subsequence that ends at index i
static int lisQuadratic(int[] a) {
    int n = a.length, best = 0;
    int[] f = new int[n];
    for (int i = 0; i < n; i++) {
        f[i] = 1;                                   // a[i] on its own
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) f[i] = Math.max(f[i], f[j] + 1);
        best = Math.max(best, f[i]);
    }
    return best;
}
```

EXAMPLE 13.1 The quadratic DP on the card's array

For $a = [10, 9, 2, 5, 3, 7, 101, 18]$ the inner loop finds these earlier, smaller elements:

```
i   a[i]   earlier j with a[j] < a[i]     f[i]
0    10    -                              1
1     9    -                              1
2     2    -                              1
3     5    2                              2     (2, 5)
4     3    2                              2     (2, 3)
5     7    2, 3, 4                        3     (2, 5, 7) or (2, 3, 7)
6   101    0, 1, 2, 3, 4, 5               4
7    18    0, 1, 2, 3, 4, 5               4
```

The answer is $\max f = 4$. Two different subsequences reach $f[5] = 3$; the DP keeps only the length, which is all the recurrence needs.

The double loop makes $n(n-1)/2$ comparisons: about $1.25 \cdot 10^7$ for $n = 5000$, which runs in well under a second, and $2 \cdot 10^{10}$ for $n = 2 \cdot 10^5$, which does not. Memory is one `int` per element. Keep this version in mind for two reasons. It is the brute force you test the fast versions against, and it adapts to almost any change in the condition "$a[j] < a[i]$" (divisibility in Chapter 14, two keys in Section 13.6) without new theory.

13.2 Patience sorting in O(n log n)

The quadratic DP looks at every earlier element. Most of those looks are wasted: among all increasing subsequences of a given length, only the one with the **smallest last value** matters for the future, because any element that extends some subsequence of that length also extends the one with the smallest last value.

DEFINITION 13.3 The tails array

While scanning a from left to right, the **tails array** holds, for each length $L = 1, 2, \ldots, len$, the smallest possible last value of a strictly increasing subsequence of length L among the elements read so far: `tails[L - 1]`. The number *len* is how many lengths are possible so far.

The method is called **patience sorting** after a card game. Deal the elements one at a time as cards; put each card on the leftmost pile whose top card is greater than or equal to it, or start a new pile on the right. The top cards of the piles, read left to right, are exactly the tails array, and the number of piles is the LIS length.

pile 1	pile 2	pile 3	pile 4
10	5	7	101
9	3		18
2			

Figure 13.1 Patience sorting of a = [10, 9, 2, 5, 3, 7, 101, 18]. Cards arrive top to bottom in each column; each goes on the leftmost pile whose current top is ≥ it. The last card of each pile (green) forms tails = [2, 3, 7, 18], and four piles mean an LIS of length 4.

In code, "the leftmost pile whose top is ≥ x" is a lower-bound binary search on `tails`, which works because `tails` is sorted (Lemma 13.2). The template below is the handbook's, unchanged.

Listing 13.2 Patience sorting: the length of the LIS in O(n log n)

```java
// Length of the longest strictly increasing subsequence, O(n log n).
// tails[len-1] = smallest possible last value of an increasing subsequence of length len.
static int lisStrict(int[] a) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int lo = 0, hi = len;
        while (lo < hi) {                       // first tail >= x  (use > x for non-decreasing)
            int mid = (lo + hi) >>> 1;
            if (tails[mid] >= x) hi = mid;
            else lo = mid + 1;
        }
        tails[lo] = x;
        if (lo == len) len++;
    }
    return len;
}
```

```
x      position   tails after
10        0       [10]
9         0       [9]
2         0       [2]
5         1       [2, 5]
3         1       [2, 3]
7         2       [2, 3, 7]
101       3       [2, 3, 7, 101]
18        3       [2, 3, 7, 18]      length 4
```

Why it works

Write $B(L)$ for the smallest last value of a strictly increasing subsequence of length L among the elements read so far (undefined if there is none).

LEMMA 13.2 **The tails are strictly increasing**

Whenever $B(L+1)$ is defined, $B(L) < B(L+1)$.

Proof. Take an increasing subsequence of length $L+1$ whose last value is $B(L+1)$. Removing its last element leaves an increasing subsequence of length L whose last value is its second-to-last element, which is smaller than $B(L+1)$. So $B(L)$ is at most that value, hence smaller than $B(L+1)$. ∎

THEOREM 13.3 **The tails invariant**

After Listing 13.2 has read a[0..i], `len` is the LIS length of a[0..i], and `tails[k]` $= B(k+1)$ for every $0 \le k <$ `len`.

Proof. By induction on i. Before the first element, `len` $= 0$ and there is nothing to check. Assume the statement holds after $a[0..i-1]$ and let $x = a[i]$. The binary search returns p, the smallest index with `tails[p]` $\ge x$, or $p =$ `len` if there is none. By Lemma 13.2, `tails[k]` $< x$ exactly for $k < p$.

The only new subsequences are those that end with x. One of length L exists if $L = 1$, or if some increasing subsequence of length $L-1$ among the earlier elements ends with a value below x, that is, $B(L-1) < x$, that is, $L - 2 < p$. So the lengths that can end with x are exactly $1, 2, \ldots, p+1$. For each such L the new $B(L)$ is the smaller of the old one and x; longer lengths are unchanged.

For $L \le p$ the old $B(L) =$ `tails[L - 1]` is already below x, so nothing changes. For $L = p+1$: if $p <$ `len`, the old value `tails[p]` $\ge x$, so the new best is x; if $p =$ `len`, length $p+1$ is new and its best is x. In both cases

the loop writes `tails[p] = x` and increases `len` exactly when $p =$ `len`, so the invariant holds after $a[i]$. Since `len` counts the possible lengths, it is the LIS length. ∎

Each of the n elements costs one binary search over at most n entries, so the time is $O(n \log n)$ and the memory $O(n)$. For $n = 2 \cdot 10^5$ that is about $3.5 \cdot 10^6$ comparisons.

> **PITFALL**
>
> `tails` is **not** a subsequence of a; only its length means something. For $a = [3, 4, 1]$ the loop ends with `tails` = [1, 4], but 1 comes after 4 in a. If the problem wants the subsequence itself, use the parent pointers of Section 13.3.

Strict or non-decreasing

A non-decreasing subsequence of length $L - 1$ can be extended by x when its last value is $\le x$. Repeating the proof with "≤" in place of "<" shows that the right position is the first tail **strictly greater** than x, the upper bound of Chapter 6, and that `tails` is then non-decreasing rather than strictly increasing. In code, one character changes:

```
int[] tails = {1, 2, 2};
int len = 3, x = 2, lo = 0, hi = len;
while (lo < hi) {
    int mid = (lo + hi) >>> 1;
    if (tails[mid] > x) hi = mid;     // upper bound: first tail > x (non-decreasing LIS)
    else lo = mid + 1;
}
// lo == 3: x = 2 extends the subsequence 1, 2, 2 to length 4
```

For $a = [2, 2, 2]$ the strict version returns 1 and the non-decreasing one returns 3. When a statement says "increasing", check the samples for a repeated value before choosing.

> **JAVA NOTE**
>
> `Arrays.binarySearch` is tempting here but unsafe. On the strict `tails`, which has no duplicates, it returns either the index of x (replace it with itself, harmless) or $-$(insertion point) $- 1$, from which you can recover the lower bound. On a non-decreasing `tails` with repeated values it may return *any* matching index, not the last one, so the upper bound is wrong. Write the five-line loop yourself; it is the same loop as Chapter 6.

13.3 Recovering the subsequence and the length at each position

Many problems want more than one number: the subsequence itself, or the best length ending at every index. Both come from remembering where each element was placed.

Parent pointers

When $a[i]$ lands at position p of `tails`, the element currently at position $p - 1$ is the end of a length-p subsequence with a smaller last value, so $a[i]$ can follow it. Record that element's *index* as the **parent pointer** of i. To do so, store beside each tail the index it came from.

Listing 13.3 Recovering one longest increasing subsequence with parent pointers

```
1 // Returns one longest strictly increasing subsequence of a (its values, in order).
2 static int[] lisSequence(int[] a) {
3     int n = a.length, len = 0;
4     int[] tails = new int[n];      // tails[k]   = smallest last value of a length-(k+1) subsequence
```

```
    int[] tailIdx = new int[n];    // tailIdx[k] = the index in a where that value sits
    int[] parent = new int[n];     // parent[i]  = index before i in the subsequence that ends at i
    for (int i = 0; i < n; i++) {
        int lo = 0, hi = len;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (tails[mid] >= a[i]) hi = mid;
            else lo = mid + 1;
        }
        tails[lo] = a[i];
        tailIdx[lo] = i;
        parent[i] = lo > 0 ? tailIdx[lo - 1] : -1;
        if (lo == len) len++;
    }
    int[] seq = new int[len];
    int i = len > 0 ? tailIdx[len - 1] : -1;
    for (int k = len - 1; k >= 0; k--) {       // walk the parent chain backwards
        seq[k] = a[i];
        i = parent[i];
    }
    return seq;
}
```

CLAIM 13.4 The parent chain is an increasing subsequence

If a[i] was placed at position p, then following parent pointers from i visits p + 1 indices in decreasing order whose values are strictly decreasing. In particular the chain from `tailIdx[len - 1]` is a longest increasing subsequence, read backwards.

Proof. By strong induction on i. If $p = 0$ the parent is -1 and the chain is just i. Otherwise the parent is $j =$ `tailIdx[p - 1]` at the moment $a[i]$ is placed. That entry was written earlier, so $j < i$; its value is `tails[p - 1]`, which is smaller than $a[i]$ because the binary search stopped at p; and $a[j]$ was itself placed at position $p - 1$. By the induction hypothesis the chain from j has p indices with decreasing values, all $\le j$, so prepending i gives $p + 1$ of them. Later overwrites of `tailIdx` do not change `parent[i]`, which was fixed when i was placed. ∎

On the running example the parents are $2 \to 4 \to 5 \to 7$ (values 2, 3, 7, 18): element 18 was placed at position 3 when `tailIdx[2]` = 5, element 7 at position 2 when `tailIdx[1]` = 4, and so on. The method returns [2, 3, 7, 18] in $O(n \log n)$ time and three extra arrays.

The best length ending at every index

The proof of Theorem 13.3 shows more than the final length. When $a[i]$ is placed at position p, the lengths that can end with $a[i]$ are exactly $1, \dots, p + 1$. So $p + 1$ is the longest increasing subsequence ending at index i, which is $f[i]$ of Theorem 13.1, obtained in $O(\log n)$ instead of $O(n)$.

Listing 13.4 Length of the longest increasing subsequence ending at every position

```
// end[i] = length of the longest increasing subsequence that ends exactly at a[i].
// strict = true: first tail >= x (lower bound); strict = false: first tail > x (upper bound).
static int[] lengthEndingAt(int[] a, boolean strict) {
    int n = a.length, len = 0;
    int[] tails = new int[n], end = new int[n];
    for (int i = 0; i < n; i++) {
        int x = a[i], lo = 0, hi = len;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (strict ? tails[mid] >= x : tails[mid] > x) hi = mid;
            else lo = mid + 1;
        }
        tails[lo] = x;
        if (lo == len) len++;
```

```
        end[i] = lo + 1;              // x extends the best subsequence of length lo
    }
    return end;
}
```

EXAMPLE 13.2 Obstacle courses: the non-decreasing length at every position

LeetCode 1964 (Find the Longest Valid Obstacle Course at Each Position) gives obstacle heights and asks, for every i, the longest non-decreasing subsequence that ends with obstacle i itself. That is Listing 13.4 with `strict = false`. On heights [2, 5, 3, 3, 1, 4, 3]:

```
x   upper bound p   answer p + 1   tails after
2         0              1         [2]
5         1              2         [2, 5]
3         1              2         [2, 3]          5 replaced: 3 is a cheaper end for length 2
3         2              3         [2, 3, 3]       equal heights may follow each other
1         0              1         [1, 3, 3]
4         3              4         [1, 3, 3, 4]
3         3              4         [1, 3, 3, 3]    2, 3, 3, 3
```

The output is [1, 2, 2, 3, 1, 4, 4]. With `strict = true` the same heights give [1, 2, 2, 2, 1, 3, 2]: the second 3 can no longer follow the first. With n up to 10^5 in that problem, the quadratic DP would make about $5 \cdot 10^9$ comparisons; this loop makes about $1.7 \cdot 10^6$.

13.4 Peaks: combining increasing runs from both sides

Some problems want a subsequence that rises and then falls. The tool is to compute the best rising part ending at each index from the left, the best falling part starting at each index from the right, and join them at the peak.

DEFINITION 13.4 Mountain

A sequence $b[0..m-1]$ is a **mountain** if $m \geq 3$ and there is a peak index t with $0 < t < m-1$ such that $b[0] < b[1] < \cdots < b[t]$ and $b[t] > b[t+1] > \cdots > b[m-1]$.

The peak may not be the first or the last element. That one condition is where most wrong answers come from.

For each index i let up[i] be the longest strictly increasing subsequence ending at i (Listing 13.4 on a), and down[i] the longest strictly decreasing subsequence starting at i. A decreasing subsequence of a that starts at i is, read backwards, an increasing subsequence of the reversed array that ends at position $n-1-i$. So down comes from Listing 13.4 on the reversed array, read back in reverse.

THEOREM 13.5 Longest mountain subsequence

The longest mountain subsequence of a has length $\max\{\, up[i] + down[i] - 1 : up[i] \geq 2 \text{ and } down[i] \geq 2 \,\}$. If no index satisfies both conditions, a has no mountain subsequence.

Proof. Take any mountain subsequence and let its peak be $a[i]$. Its rising part is an increasing subsequence ending at i of some length $u \geq 2$ (the peak is not first), so $u \leq$ up[i] and up[i] ≥ 2. Its falling part is a decreasing subsequence starting at i of length $d \geq 2$ (the peak is not last), so $d \leq$ down[i] and down[i] ≥ 2. The two parts share only the peak, so the length is $u + d - 1 \leq$ up[i] + down[i] $- 1$. Conversely, for an index i with up[i] ≥ 2 and down[i] ≥ 2, join a longest increasing subsequence ending at i (it uses indices $\leq i$) to a longest decreasing one starting at i (indices $\geq i$). They overlap only in i, the result rises strictly to $a[i]$ and falls

strictly after it, and each part has at least two elements, so the peak is neither first nor last. Its length is up[i] + down[i] − 1. ∎

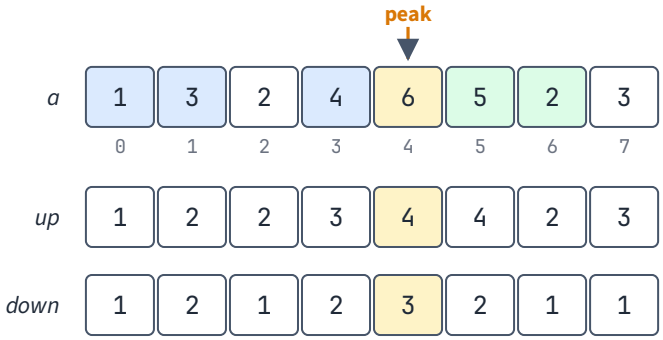


Figure 13.2 up and down for a = [1, 3, 2, 4, 6, 5, 2, 3]. At the peak a[4] = 6, up = 4 and down = 3 give a mountain of length 4 + 3 − 1 = 6, for example 1, 3, 4, 6, 5, 2 (shaded). Removing the other two elements leaves a mountain.

LeetCode 1671 (Minimum Number of Removals to Make Mountain Array) asks for the fewest deletions, which is n minus the longest mountain subsequence. Both arrays cost O(n log n) with Listing 13.4, or O(n^2) with Listing 13.1, which also passes that problem's limits. The join is one more loop.

PITFALL

Dropping the conditions up[i] ≥ 2 and down[i] ≥ 2 lets a purely decreasing or purely increasing run count as a mountain. For a = [6, 4, 3, 6, 2, 1, 2], index 0 has up = 1 and down = 5 (the run 6, 4, 3, 2, 1), and the unchecked formula reports 2 removals. The true answer is 3: the best real mountain has length 4, such as 4, 6, 2, 1 with its peak at index 3. On 2,000 random arrays of length 3 to 10 with values 1..6, the unchecked formula was wrong on 248.

13.5 DP over values with a Fenwick tree of maxima

Patience sorting is fast but rigid. It cannot attach a weight to each element, count subsequences, or carry an extra condition. For those, go back to the quadratic recurrence and look at it from the side of the **values**.

Changing the point of view

Read the elements left to right and keep, for every value v, the number

> $g[v]$ = the longest increasing subsequence found so far whose last element has value v (0 if none).

When $x = a[i]$ arrives, the recurrence of Theorem 13.1 asks for the maximum of $f[j]$ over earlier j with $a[j] < x$. That is the maximum of $g[v]$ over all values $v < x$: a **prefix maximum** over the value axis. Then $g[x]$ is raised to the new length.

THEOREM 13.6 DP over values equals the quadratic DP

Process a left to right. For $x = a[i]$ set $cur = 1 + \max\{ g[v] : v < x \}$ and then $g[x] = \max(g[x], cur)$. Then $cur = f[i]$ at every step, and the answer is the largest cur.

Proof. We show by induction on i that just before $a[i]$ is processed, $g[v] = \max\{ f[j] : j < i, a[j] = v \}$ for every v (0 if the set is empty). It holds at $i = 0$ with every $g[v] = 0$. If it holds before $a[i]$, then max over $v < x$ of $g[v]$ =

$\max\{ f[j] : j < i, a[j] < x \}$, so cur = $f[i]$ by Theorem 13.1. Raising $g[x]$ to $\max(g[x], f[i])$ and leaving every other $g[v]$ alone makes the statement true before $a[i+1]$. ∎

A prefix maximum over all values would cost O(V) per element. A Fenwick tree answers it in O(log V), on two conditions: the values must be the integers 1..V, and each $g[v]$ may only go **up**.

Coordinate compression

Values up to 10^9, or negative, cannot index an array. Only their order matters for "$v < x$", so replace each value by its rank among the distinct values. This is **coordinate compression**.

Listing 13.5 Coordinate compression: values to ranks 1..V

```
// rank[i] = 1 + (number of distinct values smaller than a[i]); equal values get equal ranks.
static int[] compress(int[] a) {
    int[] d = a.clone();
    Arrays.sort(d);
    int m = 0;                                      // d[0..m-1] = distinct values, ascending
    for (int i = 0; i < d.length; i++)
        if (i == 0 || d[i] != d[i - 1]) d[m++] = d[i];
    int[] rank = new int[a.length];
    for (int i = 0; i < a.length; i++) rank[i] = Arrays.binarySearch(d, 0, m, a[i]) + 1;
    return rank;
}
```

Sorting costs O(n log n) and each look-up O(log n). The distinct values have no duplicates, so `Arrays.binarySearch` is safe here. Equal values get equal ranks, so "strictly smaller value" and "strictly smaller rank" mean the same thing.

The Fenwick tree of maxima

A **Fenwick tree of maxima** stores an array `fw[1..V]` in which node j holds the maximum of g over the values j − lowbit(j) + 1, ..., j, where lowbit(j) = `j & -j` is the value of the lowest set bit of j. Chapter 20 develops Fenwick trees for sums in detail; here you need only the two loops and the reason they are correct.

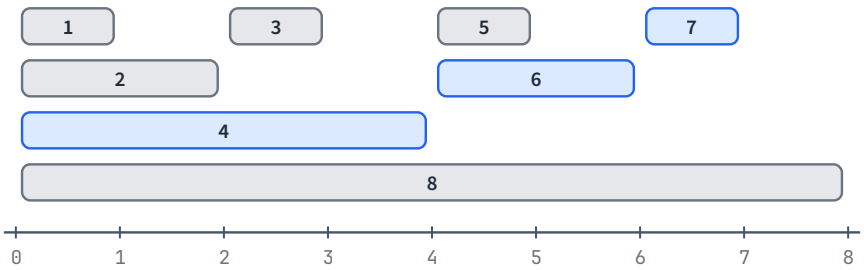


Figure 13.3 The ranges of a Fenwick tree with V = 8. Node j covers the values from j − lowbit(j) + 1 to j, drawn as the bar from j − lowbit(j) to j. A prefix query for values 1..7 reads nodes 7, 6 and 4 (blue), whose ranges are disjoint and cover 1..7 exactly.

LEMMA 13.7 What the two loops visit

(a) The query loop `for (; i > 0; i -= i & -i)` visits nodes whose ranges are disjoint and together cover exactly the values 1..i. (b) The update loop `for (; i <= V; i += i & -i)`, started at v, visits exactly the nodes whose range contains v.

Proof. (a) From node j the loop moves to j − lowbit(j), and the range of j is (j − lowbit(j), j]. So consecutive ranges sit end to end, moving down from i and stopping at 0. They are disjoint and cover 1..i.

(b) First, every visited node contains v. Node v does. If node j contains v and $j' = j + \text{lowbit}(j)$, then $\text{lowbit}(j') \geq 2 \cdot \text{lowbit}(j)$ (adding the lowest bit carries into a higher bit), so $j' - \text{lowbit}(j') \leq j - \text{lowbit}(j) < v \leq j < j'$, and node j' contains v too. Second, no node containing v is skipped. Suppose node j contains v but is not visited. The visited nodes increase from $v \leq j$, so some visited c has $c < j < c + \text{lowbit}(c)$. Write $j = c + t$ with $0 < t < \text{lowbit}(c)$. The low bits of c below $\text{lowbit}(c)$ are zero, so $\text{lowbit}(j) = \text{lowbit}(t) \leq t$, and $j - \text{lowbit}(j) \geq c \geq v$. Then the range of j starts above v, a contradiction. ∎

By (b), raising $g[v]$ touches every node whose range contains v, so each node always holds the maximum of g over its range, provided values only ever go up: a maximum cannot "forget" an old value that has been lowered. By (a), a query combines whole ranges that cover $1..i$. So the query returns $\max\{ g[1..i] \}$. In the example of Figure 13.3, a query for 1..7 reads nodes 7, 6, 4, and raising value 5 writes nodes 5, 6, 8.

Listing 13.6 DP over values with a Fenwick tree of maxima

```java
// Same answer as dp over values: dp(x) = 1 + max dp over values < x.
// A Fenwick tree of maximums gives that max in O(log V). Values must be 1..V (compress first).
// Extra conditions (e.g. "XOR >= M") become an extra dimension: one Fenwick per state.
static int lisWithFenwick(int[] a, int V) {
    int[] fw = new int[V + 1];
    int best = 0;
    for (int x : a) {
        int cur = 1 + prefixMax(fw, x - 1);    // x - 1 for strict, x for non-decreasing
        raise(fw, x, cur);
        best = Math.max(best, cur);
    }
    return best;
}

static int prefixMax(int[] fw, int i) {
    int r = 0;
    for (; i > 0; i -= i & -i) r = Math.max(r, fw[i]);
    return r;
}

static void raise(int[] fw, int i, int v) {    // values only ever go up
    for (; i < fw.length; i += i & -i) fw[i] = Math.max(fw[i], v);
}
```

EXAMPLE 13.3 **The Fenwick DP step by step**

Take $a = [40, 10, 30, 20, 30, 50]$. Compression gives ranks [4, 1, 3, 2, 3, 5] with $V = 5$.

```
rank   prefixMax(rank-1)   cur    fw[1..5] after raise
 4            0             1     [0, 0, 0, 1, 0]
 1            0             1     [1, 1, 0, 1, 0]
 3            1             2     [1, 1, 2, 2, 0]
 2            1             2     [1, 2, 2, 2, 0]
 3            2             3     [1, 2, 3, 3, 0]     10, 20, 30
 5            3             4     [1, 2, 3, 3, 4]     10, 20, 30, 50
```

The answer is 4. Notice the second 30 (rank 3): its query covers ranks 1..2 only, so it cannot extend the first 30. For a non-decreasing subsequence the query would include rank 3 itself, which is the `x` instead of `x - 1` in the comment.

Each element costs one query and one update of $O(\log V)$ steps, so the time is $O(n \log n)$ including compression and the memory $O(V)$. This matches patience sorting asymptotically; the gain is flexibility.

What the value view buys

Weights. If each element has a weight and you want the increasing subsequence of largest total weight, store sums instead of lengths: cur = w + max{ best sum ending in a smaller value }. Patience sorting has no equivalent, because "smallest last value for each length" says nothing about sums.

Listing 13.7 Maximum-sum increasing subsequence: the Fenwick tree stores sums

```
// Largest total of a strictly increasing subsequence. rank[i] in 1..V (from compress), w = a.
static long maxSumIncreasing(int[] a, int[] rank, int V) {
    long[] fw = new long[V + 1];
    long best = Long.MIN_VALUE;
    for (int i = 0; i < a.length; i++) {
        long before = 0;                          // best sum ending in a smaller value (0 = none)
        for (int j = rank[i] - 1; j > 0; j -= j & -j) before = Math.max(before, fw[j]);
        long cur = a[i] + before;
        for (int j = rank[i]; j <= V; j += j & -j) fw[j] = Math.max(fw[j], cur);
        best = Math.max(best, cur);
    }
    return best;
}
```

On [3, −1, 4, 1, 5, −9, 2, 6] it returns 18, from 3 + 4 + 5 + 6. Taking `before` as at least 0 means "start a new subsequence here", which is better than extending a negative total.

Counting. To count the longest increasing subsequences, store a pair (length, count) for each value and combine two pairs by keeping the longer length, adding the counts when the lengths are equal. The combination is associative, so the same Fenwick loops work with pairs. On [2, 1, 4, 3, 5] the pairs ending at each index are (1, 1), (1, 1), (2, 2), (2, 2), (3, 4): four subsequences of length 3, namely 2, 4, 5; 2, 3, 5; 1, 4, 5 and 1, 3, 5. The same pairs in the quadratic DP give an $O(n^2)$ count, which is the idea behind the bonus problem LeetCode 673.

Windows of values. If the previous value must lie in a window, such as $x - k \le v < x$, the query is no longer a prefix and a Fenwick tree of maxima cannot answer it. A segment tree can; that is LeetCode 2407 in Chapter 20.

13.6 Two keys: sort by one, LIS on the other

An envelope (w, h) fits inside another (w', h') when $w < w'$ **and** $h < h'$. The longest chain of nested envelopes is an increasing subsequence in two keys at once. The trick is to fix the order of one key by sorting, so that only the other key needs a LIS.

Sort by width ascending. Any nested chain then appears in sorted order, so its heights form an increasing subsequence of the height sequence. The danger is in the other direction: an increasing run of heights may pick two envelopes with the **same** width, which do not nest.

EXAMPLE 13.4 Why ties need the second key descending

Take the envelopes (3, 3) and (3, 4). Neither fits in the other: the widths are equal. Sorted by (width ascending, height ascending), the heights read 3, 4 and the LIS is 2, a wrong answer. Sorted by (width ascending, height **descending**), the heights read 4, 3 and the LIS is 1, which is right. On 2,000 random sets of up to seven envelopes with sides 1..4, the ascending tie order gave a wrong answer 607 times. The descending order matched a brute force on all of 6,000 random sets of up to eight envelopes.

THEOREM 13.8 Sort, then one LIS

Sort the pairs by w ascending and, for equal w, by h descending. Then the longest chain with both coordinates strictly increasing has the same length as the longest strictly increasing subsequence of the sorted heights.

Proof. A chain with strictly increasing w and h has distinct widths, so in the sorted order its members appear by increasing width, and their heights form a strictly increasing subsequence. Conversely, take a strictly increasing subsequence of the sorted heights. Two members with equal width would appear with non-increasing heights, because ties are sorted by height descending, so they cannot both be in it. Hence all its widths are distinct, and since the order is by width, they increase strictly. Its heights increase strictly by choice, so it is a chain of the same length. ∎

In Java the comparator is the only new line:

```
int[][] env = {{5, 4}, {6, 4}, {6, 7}, {2, 3}};
Arrays.sort(env, (p, q) -> p[0] != q[0] ? Integer.compare(p[0], q[0])    // width ascending
                                        : Integer.compare(q[1], p[1]));  // height descending on ties
```

Then run Listing 13.2 on the heights. The whole method costs $O(n \log n)$. This is the key observation of the bonus problem LeetCode 354 (Russian Doll Envelopes).

If the order were non-strict in both keys ($w \le w'$ and $h \le h'$), the reasoning flips: ties may now chain, so sort ties by h ascending and take the **non-decreasing** LIS of the heights. Always decide the tie order by asking "may two items with equal first key both be in the answer?"

13.7 An extra state for a side condition

So far the state was "where the subsequence ends". Many conditions on a subsequence depend on a little more than the last value, and the general recipe is to add exactly that information to the state.

DEFINITION 13.5 Extra state dimension

An **extra state dimension** is a small piece of information s, taking few possible values, that the transition needs and that the basic state does not record. The DP becomes $g[\text{last}][s]$ or $f[i][s]$, and its cost is multiplied by the number of values of s.

The last letter as the state

When the condition involves only the last element and the values come from a tiny alphabet, the last value *is* the whole state, and you do not need indices at all. Consider subsequences of a digit string in which neighbouring digits differ by exactly 1. Keep best[d], the longest such subsequence read so far that ends with the digit d. A new digit d can follow a subsequence ending in $d-1$ or $d+1$.

Listing 13.8 An extra state for the last digit

```
// Longest subsequence of a digit string in which neighbouring digits differ by exactly 1.
// best[d] = longest such subsequence seen so far that ends with the digit d.
static int longestStepSubsequence(String s) {
    int[] best = new int[10];
    int ans = 0;
    for (int i = 0; i < s.length(); i++) {
        int d = s.charAt(i) - '0';
        int cur = 1;                                  // the digit on its own
        if (d > 0) cur = Math.max(cur, best[d - 1] + 1);
        if (d < 9) cur = Math.max(cur, best[d + 1] + 1);
        best[d] = Math.max(best[d], cur);
```

```
        ans = Math.max(ans, best[d]);
    }
    return ans;
}
```

On `21235422` the values of best[1..5] after each digit are:

```
i  digit  cur   best[1] best[2] best[3] best[4] best[5]
0    2     1       0       1       0       0       0
1    1     2       2       1       0       0       0
2    2     3       2       3       0       0       0
3    3     4       2       3       4       0       0
4    5     1       2       3       4       0       1
5    4     5       2       3       4       5       1      2, 1, 2, 3, 4
6    2     5       2       5       4       5       1      2, 1, 2, 3, 2
7    2     5       2       5       4       5       1
```

The answer is 5. The proof is Theorem 13.6 again with the digit as the value and "differs by exactly 1" as the allowed predecessor set. The cost is O(n) with ten states. The bonus problem LeetCode 2370 has the same shape over 26 letters with a different rule for which letters may be neighbours; its hint is "26 states".

A condition on the whole subsequence

Now suppose the condition is on **all** chosen elements, not just neighbours: the longest strictly increasing subsequence whose sum is divisible by K. The last value alone no longer decides whether a subsequence can be completed, but the last value **plus the sum modulo K** does, because adding x changes the remainder from r to $(r + x) \bmod K$ whatever the earlier elements were.

Listing 13.9 An extra dimension: increasing subsequences whose sum is divisible by K

```
// Longest strictly increasing subsequence whose sum is divisible by K (0 if there is none).
// best[i][r] = longest one that ends at index i with sum % K == r (0 = no such subsequence).
static int lisSumDivisible(int[] a, int K) {
    int n = a.length, ans = 0;
    int[][] best = new int[n][K];
    for (int i = 0; i < n; i++) {
        int v = Math.floorMod(a[i], K);
        best[i][v] = 1;                                  // a[i] on its own
        for (int j = 0; j < i; j++) {
            if (a[j] >= a[i]) continue;
            for (int r = 0; r < K; r++) {
                if (best[j][r] == 0) continue;
                int nr = (r + v) % K;
                best[i][nr] = Math.max(best[i][nr], best[j][r] + 1);
            }
        }
        ans = Math.max(ans, best[i][0]);
    }
    return ans;
}
```

EXAMPLE 13.5 **A remainder as the extra state**

For $a = [1, 6, 7, 4, 8, 3, 5]$ and $K = 3$ the rows best[i][0..2] are:

```
i  a[i]  a[i] mod 3   best[i][0]  best[i][1]  best[i][2]
0    1       1           0           1           0
1    6       0           1           2           0
2    7       1           0           2           3
3    4       1           0           1           2
```

4	8	2	3	4	2
5	3	0	1	2	0
6	5	2	3	3	2

The answer is 3, for example 1, 6, 8 (sum 15) or 1, 3, 5 (sum 9). The plain LIS is 4 (1, 6, 7, 8), but its sum 22 leaves remainder 1, which is the value best[4][1] = 4 in the table. The extra column is what tells the two apart.

The time is $O(n^2 K)$ and the memory $O(nK)$. To reach $O(n K \log n)$, switch to the value view with one Fenwick tree of maxima **per remainder**: tree r answers "the longest subsequence with remainder r ending in a value below x".

Listing 13.10 The same DP with one Fenwick tree of maxima per remainder

```java
// O(n * K * log n). fw[r] is a Fenwick tree over value ranks: at rank t it holds the longest
// subsequence seen so far with sum % K == r whose last value has rank t.
static int lisSumDivisibleFast(int[] a, int K) {
    int n = a.length, V = 0;
    int[] d = a.clone();
    Arrays.sort(d);
    for (int i = 0; i < n; i++) if (i == 0 || d[i] != d[i - 1]) d[V++] = d[i];
    int[][] fw = new int[K][V + 1];
    int[] cand = new int[K];
    for (int x : a) {
        int t = Arrays.binarySearch(d, 0, V, x) + 1, v = Math.floorMod(x, K);
        for (int r = 0; r < K; r++) {                    // 1. read every state ...
            int before = query(fw[r], t - 1);            //    ends in a smaller value, remainder r
            cand[(r + v) % K] = before > 0 ? before + 1 : 0;
        }
        cand[v] = Math.max(cand[v], 1);                  //    x on its own
        for (int r = 0; r < K; r++)                      // 2. ... then write them all
            if (cand[r] > 0) update(fw[r], t, cand[r]);
    }
    return query(fw[0], V);
}

static int query(int[] f, int i) {
    int r = 0;
    for (; i > 0; i -= i & -i) r = Math.max(r, f[i]);
    return r;
}

static void update(int[] f, int i, int v) {
    for (; i < f.length; i += i & -i) f[i] = Math.max(f[i], v);
}
```

Both listings matched a brute force over all subsequences on 6,000 random arrays of length up to 12 (with negative values and K from 1 to 5), and each other on 300 arrays of length up to 2,000.

PITFALL

Read every state for the new element before writing any of them. In the strict version above the reads stop at rank $t - 1$ and the writes start at rank t, so mixing them would happen to be safe. In a **non-decreasing** version the reads include rank t, and a write made for one remainder could be read back for another remainder in the same step, using x twice. The `cand` buffer prevents that at no extra cost.

Where this goes: the sample paper

INFOSYS LENS

The Hard problem of Infosys's official sample paper is "longest non-decreasing subsequence whose XOR is at least M", with $N \le 1000$ and values at most N. The plan's notes list it among the most reported repeats (13 reports, DSE and SP, 2022–2026). Its structure is the one in this section: XOR replaces the sum modulo K as the extra state, and because every value is below 1024, any XOR of values also fits in 10 bits, so there are 1024 states. Chapter 25 solves it in full after you have attempted it as Mock 4. Until then, notice that a brute force over all subsequences works up to $N \approx 20$ and earns the small tests.

Another reported repeat, a longest increasing subsequence in which every pair of neighbours satisfies a bitwise condition (4 reports), also falls to a small state over the last element; Chapter 24 works through it. When you meet a new subsequence condition, ask the same two questions each time. What must I know about the subsequence so far to decide whether x may be appended? How many different values can that information take? If the answer to the second question is small, it becomes a dimension.

EXAM TIP

Under exam pressure, write the quadratic version with the extra dimension first, test it on the samples, and submit it. It is correct and usually earns the medium-sized tests. Then speed it up with a Fenwick tree per state, and compare the two on random inputs before you submit the fast one.

Summary

- A subsequence keeps order but not contiguity. The state "ends at index i" records the last value, which is what an extension needs: $f[i] = 1 + \max f[j]$ over earlier, smaller $a[j]$, in $O(n^2)$.
- Patience sorting keeps `tails[k]`, the smallest last value of an increasing subsequence of length $k+1$. `tails` stays sorted, so each element costs one binary search, and the number of tails is the LIS length: $O(n \log n)$.
- Lower bound (first tail $\ge x$) gives the strict LIS; upper bound (first tail $> x$) gives the non-decreasing one. `tails` itself is not a subsequence.
- The position p where $a[i]$ lands gives the best length ending at i, $p+1$. Storing the index of each tail and a parent pointer per element recovers an optimal subsequence.
- Mountains join up[i] from the left and down[i] from the right: length up + down − 1, valid only when both are at least 2.
- Viewing the DP over values turns the inner loop into a prefix maximum, answered by a Fenwick tree of maxima after coordinate compression. This handles weights and counts, which patience sorting cannot.
- For two keys, sort by the first and run a LIS on the second; for strict order in both, break ties on the second key in descending order.
- A side condition with few possible values becomes an extra state dimension, with one Fenwick tree per state; read all states before writing any. The sample paper's Hard problem is this pattern with 1024 XOR states.

Key terms

subsequence p. 180

longest increasing subsequence p. 180

tails array p. 181
mountain array p. 185
extra state dimension p. 190
patience sorting p. 181
parent pointer p. 183
coordinate compression p. 187
Fenwick tree p. 187

Review questions

1. Why is "the length of the LIS of $a[0..i]$" not a usable DP state on its own? What does "ends at i" add?
2. Run the strict and the non-decreasing versions of patience sorting on [3, 1, 2, 2, 1, 3]. Give `tails` after each element and both final lengths.
3. Prove, or refute with an example, the statement "after the loop, `tails` is an increasing subsequence of a".
4. How would you compute the longest strictly *decreasing* subsequence with Listing 13.2 without changing the listing? Apply it to [5, 2, 8, 6, 3, 6, 9, 7].
5. In Theorem 13.5, which part of the proof uses up$[i] \geq 2$? Give an array with a mountain subsequence on which the formula without the two conditions gives the wrong number of removals.
6. Why must the values in a Fenwick tree of maxima only ever increase? What goes wrong if an update lowers a value?
7. For $V = 8$, which nodes does a prefix query for 1..7 read, and which nodes does raising value 5 write? Check both against Lemma 13.7.
8. Envelopes nest when $w \leq w'$ and $h \leq h'$ (equal sides allowed). How should ties be sorted, and which LIS should you run on the heights? Justify the tie order with a two-envelope example.
9. A problem asks for the longest increasing subsequence whose sum is even. How many values does the extra state take, and what is the cost of the quadratic and the Fenwick versions for $n = 10^5$?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 "Dynamic programming", the section on the longest increasing subsequence (the $O(n^2)$ version), and Chapter 9 "Range queries" for the binary indexed tree. Free at cses.fi/book.
- Algorithms for Competitive Programming, "Longest increasing subsequence" (cp-algorithms.com/sequences/longest_increasing_subsequence.html): the $O(n \log n)$ method, restoring the subsequence, and the version with a data structure over values.
- Algorithms for Competitive Programming, "Fenwick Tree" (cp-algorithms.com/data_structures/fenwick.html), including the prefix-maximum variant used in Section 13.5.
- Workbook Chapter 13 has a tails trace, a Fenwick trace and a mountain table to complete by hand before the problems.

Problems

These are the problems for Day 13. Work them in Workbook Chapter 13, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
13.P1	LC 300 · Longest Increasing Subsequence	MEDIUM	CORE
13.P2	LC 1964 · Find the Longest Valid Obstacle Course at Each Position	HARD	CORE
13.P3	LC 1671 · Minimum Number of Removals to Make Mountain Array	HARD	CORE

No.	Problem	Level	Set
13.P4	LC 2370 · Longest Ideal Subsequence	MEDIUM	BONUS
13.P5	LC 354 · Russian Doll Envelopes	HARD	BONUS
13.P6	LC 673 · Number of Longest Increasing Subsequence	MEDIUM	BONUS

CHAPTER 14 · DAY 14 OF 29 · FRIDAY 20 NOVEMBER 2026 · LEARNING DAY

14

Dynamic Programming IV: Divisors, Intervals and Games

Loops over multiples, dp[l][r] by length, and "my score minus yours" for two-player games.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · Interval DP and game DP
Before this chapter · Chapter 13 (LIS-style DP), Chapter 10 (nCr and Pascal's triangle), Chapter 12 (dp[l][r] on strings)
Practice · Workbook Chapter 14 · 7 problems · core ≈ 4.1 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- turn "every pair divides" into a chain condition and solve it with an LIS-style DP, including reconstruction;
- loop over multiples instead of divisors, and prove the $N(\lfloor \log N \rfloor + 1)$ bound on the total work;
- count divisor arrays by combining divisor chains with stars and bars;
- set up an interval DP, justify its evaluation order, and prove a split recurrence by looking at the first or the last operation;
- solve two-player games with the "my score minus yours" state, and prove when it agrees with "each player maximises their own score";
- choose between memoisation and tables, and estimate time and memory for $n \le 500$ and $n \le 5000$.

This chapter collects three DP families that look different but share a habit of thought: find the right **order** in which subproblems become independent. For divisibility, the order is numerical: sort, and a set in which every pair divides becomes a chain that can be built left to right. For intervals, the order is by length: the answer for a segment depends only on shorter segments inside it. For games, the order is again by length, because every move removes an element and leaves a shorter game for the other player.

The divisor part prepares one of the most frequently reported Infosys problems, counting arrays in which each element divides the next. The interval and game parts give two templates from the handbook that cover a whole page of LeetCode Hards: cutting sticks, bursting balloons, merging piles and taking coins from either end of a row.

14.1 Chains under divisibility

DEFINITION 14.1 **Divisible subset and divisor chain**

A set of positive integers is a **divisible subset** if for every two of its members x, y, either x divides y or y divides x (written $x \mid y$). A **divisor chain** is a sequence $x_1 < x_2 < \cdots < x_k$ with $x_i \mid x_{i+1}$ for every i.

The definition of a divisible subset talks about **all pairs**, which suggests checking $O(k^2)$ conditions. The first step is to see that consecutive pairs are enough once the set is sorted.

LEMMA 14.1 Pairwise divisibility is a chain condition

A set of distinct positive integers is a divisible subset if and only if, listed in increasing order, it is a divisor chain.

Proof. Suppose the sorted list $x_1 < \cdots < x_k$ is a divisor chain, and take $i < j$. Then $x_i \mid x_{i+1}$, $x_{i+1} \mid x_{i+2}$, ..., $x_{j-1} \mid x_j$. Divisibility is transitive (if $x \mid y$ and $y \mid z$, then $y = px$ and $z = qy = pqx$), so $x_i \mid x_j$ by induction on $j - i$. Conversely, in a divisible subset take consecutive members $x_i < x_{i+1}$. One divides the other, and a positive integer cannot be divisible by a larger one, so $x_i \mid x_{i+1}$. ∎

So the largest divisible subset is the longest divisor chain among the sorted values. That is the LIS of Chapter 13 with the condition "$a[j] < a[i]$" replaced by "$a[j] \mid a[i]$". Sorting matters: divisibility implies ≤ for positive integers, so a chain can only be read in increasing order. Without the sort, the DP on [4, 2, 8] links 4 → 8 and 2 → 8 but never 2 → 4, and reports 2 instead of 3.

THEOREM 14.2 The divisibility DP

Sort the values as $a[0] < \cdots < a[n-1]$ and let $f[i]$ be the size of the largest divisible subset whose largest member is $a[i]$. Then $f[i] = 1 + \max\{ f[j] : j < i, a[j] \mid a[i] \}$ (the maximum of an empty set is 0).

Proof. By Lemma 14.1 a divisible subset with largest member $a[i]$ is a divisor chain ending at $a[i]$. If it has more than one element, its second-largest member $a[j]$ has $j < i$ and $a[j] \mid a[i]$, and removing $a[i]$ leaves a chain ending at $a[j]$, so its size is at most $f[j] + 1$. Conversely, if $a[j] \mid a[i]$, every member of a chain ending at $a[j]$ divides $a[j]$ and hence $a[i]$, so appending $a[i]$ gives a chain of size $f[j] + 1$. This is the same exchange as in Chapter 13, with divisibility in place of "smaller". ∎

Listing 14.1 Largest divisible subset: sort, LIS-style DP, parent pointers

```java
// Largest subset of distinct positive values in which every pair divides one way or the other.
static List<Integer> largestDivisibleSubset(int[] nums) {
    int[] a = nums.clone();
    Arrays.sort(a);                                  // a chain must be read in increasing order
    int n = a.length, bestEnd = 0;
    int[] f = new int[n], parent = new int[n];       // f[i] = longest chain ending with a[i]
    for (int i = 0; i < n; i++) {
        f[i] = 1;
        parent[i] = -1;
        for (int j = 0; j < i; j++)
            if (a[i] % a[j] == 0 && f[j] + 1 > f[i]) {
                f[i] = f[j] + 1;
                parent[i] = j;
            }
        if (f[i] > f[bestEnd]) bestEnd = i;
    }
    LinkedList<Integer> out = new LinkedList<>();
    for (int i = (n == 0 ? -1 : bestEnd); i >= 0; i = parent[i]) out.addFirst(a[i]);
    return out;
}
```

EXAMPLE 14.1 A divisible subset step by step

For the values [4, 8, 10, 240, 3, 6, 12], sorting gives [3, 4, 6, 8, 10, 12, 240].

```
a[i]   earlier divisors     f[i]   parent
  3    -                     1     -
  4    -                     1     -
  6    3                     2     3
  8    4                     2     4
```

```
 10    -                    1     -
 12    3, 4, 6              3     6
240    3, 4, 6, 8, 10, 12   4     12
```

Walking back from 240 gives 240 → 12 → 6 → 3, so the method returns [3, 6, 12, 240]. The set {4, 8, 240} is also divisible but smaller. Note that 12 has three earlier divisors; only the one with the largest f matters.

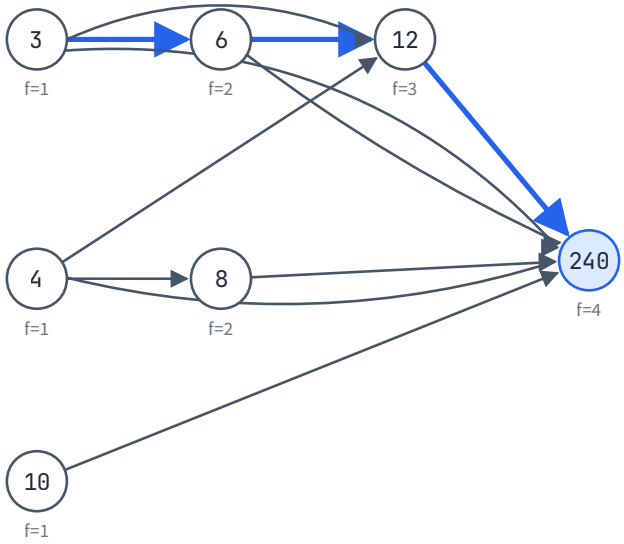


Figure 14.1 The divisibility relation among the sorted values, drawn left to right by the value of f. An arrow x → y means x | y, so a divisible subset is a path. The longest path (blue) is 3 → 6 → 12 → 240.

The double loop makes $n(n-1)/2$ divisibility tests, about $5 \cdot 10^5$ for $n = 1000$, which is the limit in LeetCode 368 (Largest Divisible Subset). Memory is two `int` arrays.

14.2 Looping over multiples

When the values are not a short list but **every** integer from 1 to N, the pairwise loop is too slow: it would test about $N^2/2$ pairs. A better order is to let each value *push* its information to its multiples.

There are two ways to visit all pairs (d, m) with $d \mid m$ and $m \le N$. **Pull**: for each m, find its divisors, by trial division up to $\sqrt{m}$, costing about $N\sqrt{N}$ in total. **Push**: for each d, walk $m = d, 2d, 3d, \ldots$, which visits exactly the pairs and nothing else:

```
int N = 20;
int[] hits = new int[N + 1];
for (int v = 1; v <= N; v++)
    for (int m = 2 * v; m <= N; m += v)      // the proper multiples of v
        hits[m]++;                            // hits[m] = number of proper divisors of m
```

The inner loop runs $\lfloor N/v \rfloor - 1$ times for each v, and the total is governed by the harmonic series.

THEOREM 14.3 The harmonic bound

For every $N \ge 1$, $\lfloor N/1 \rfloor + \lfloor N/2 \rfloor + \cdots + \lfloor N/N \rfloor \le N(\lfloor \log N \rfloor + 1)$.

Proof. Each term $\lfloor N/v \rfloor$ is at most N/v, so the sum is at most $N \cdot H$, where $H = 1 + 1/2 + \cdots + 1/N$. Split the terms of H into blocks of the form $1/2^j + 1/(2^j + 1) + \cdots + 1/(2^{j+1} - 1)$ for $j = 0, 1, 2, \ldots$. Block j has 2^j terms, each

at most $1/2^j$, so it sums to at most 1. The terms up to $1/N$ fill at most $\lfloor \log N \rfloor + 1$ blocks (the last one possibly incomplete). So $H \le \lfloor \log N \rfloor + 1$. ∎

The true constant is smaller: H is close to $\ln N + 0.58$, and $\ln N \approx 0.69 \log N$. The table shows the exact sums.

Table 14.1 Total work of the loop over multiples, computed exactly

N	$\sum \lfloor N/v \rfloor$	$N \ln N$	bound $N(\lfloor \log N \rfloor + 1)$
10^3	7,069	6,908	10,000
10^4	93,668	92,103	140,000
10^5	1,166,750	1,151,293	1,700,000
10^6	13,970,034	13,815,511	20,000,000

So a loop over multiples up to 10^6 costs about $1.4 \cdot 10^7$ steps, a fraction of a second. This is the same loop as the sieve of Eratosthenes in Chapter 10, where the inner loop runs only for primes and the total drops to about $N \ln \ln N$.

Listing 14.2 Looping over multiples: divisor counts and longest divisor chains

```
// divs[v]  = number of divisors of v, for every v in 1..N.
// chain[v] = most distinct values in a chain d1 | d2 | ... | v of numbers in 1..N.
// The inner loop runs N/v times, about N ln N steps in total.
static int[][] divisorsAndChains(int N) {
    int[] divs = new int[N + 1], chain = new int[N + 1];
    for (int v = 1; v <= N; v++) {
        chain[v] = Math.max(chain[v], 1);            // every divisor of v was pushed already
        for (int m = v; m <= N; m += v) {
            divs[m]++;
            if (m > v) chain[m] = Math.max(chain[m], chain[v] + 1);
        }
    }
    return new int[][]{divs, chain};
}
```

The push order is correct because every proper divisor of v is smaller than v, so by the time the outer loop reaches v, all of them have already pushed into `chain[v]`. This is the general rule for DP over divisibility: process values in increasing order, exactly as you process a DAG in topological order (Chapter 15).

For $v = 1, \ldots, 12$ the listing gives divs = [1, 2, 2, 3, 2, 4, 2, 4, 3, 4, 2, 6] and chain = [1, 2, 2, 3, 2, 3, 2, 4, 3, 3, 2, 4]. Up to 10^5, the most divisors is 128 (at 83,160), and the longest chain has 17 values, ending at $65{,}536 = 2^{16}$.

CLAIM 14.4 **How long a divisor chain can be**

The longest divisor chain of distinct values ending at v has $\Omega(v) + 1$ elements, where $\Omega(v)$ is the number of prime factors of v counted with multiplicity. Consequently a chain of distinct values in $1..N$ has at most $\lfloor \log N \rfloor + 1$ elements.

Proof. In a chain $x_1 \mid x_2 \mid \cdots$ with $x_i < x_{i+1}$, each quotient x_{i+1}/x_i is an integer at least 2, so it contributes at least one prime factor: Ω grows by at least 1 at each step, and $\Omega(x_1) \ge 0$. A chain ending at v therefore has at most $\Omega(v) + 1$ elements. Multiplying in the prime factors of v one at a time, starting from 1, gives a chain of exactly that length. Each step at least doubles the value, so a chain of k distinct values in $1..N$ has $x_k \ge 2^{k-1}$, which forces $k \le \lfloor \log N \rfloor + 1$. ∎

The last fact is small but decisive in the next section: however long an array is, it can contain at most 17 *distinct* values if each divides the next and all are at most 10^5.

14.3 Counting divisor arrays

Now count, instead of optimise. How many arrays $b[0..K-1]$ with values in $1..N$ satisfy $b[i] \mid b[i+1]$ for every i? Equal neighbours are allowed, since every number divides itself.

> **INFOSYS LENS**
>
> This is the repeat "Count arrays where each element divides the next", reported 13 times between 2021 and 2025 for DSE, with the answer printed modulo 10000. Its samples are $(N = 2, K = 1) \to 2$, $(2, 2) \to 3$ and $(3, 2) \to 5$. The same problem modulo $10^9 + 7$ with large K is the bonus problem LeetCode 2338. Chapter 24 returns to the Infosys version.

The direct DP

Let $W(k, v)$ be the number of valid arrays of length k that end with v. Then $W(1, v) = 1$, and an array of length k ending in v extends to length $k+1$ by appending any multiple m of v, including v itself. That is the push loop over multiples, once per length, so the cost is $O(K \cdot N \log N)$. For $N = 6$ the rows are $W(2, 1..6) = 1, 2, 2, 3, 2, 4$ and $W(3, 1..6) = 1, 3, 3, 6, 3, 9$, so there are 25 arrays of length 3. This is the method for small K.

Distinct values and stars and bars

When K is large (say 10^4) and N is 10^4 as well, $O(K \cdot N \log N)$ is about 10^9 and too slow. The way out is to separate **which** values appear from **how often** each one repeats.

Squash every run of equal neighbours in a valid array into one value. What remains is a strictly increasing divisor chain of s distinct values, and by Claim 14.4, $s \le \lfloor \log N \rfloor + 1$, a tiny number. Conversely, an array is determined by its chain and the lengths of the s runs, which are positive and sum to K.

> **THEOREM 14.5 Stars and bars**
>
> *The number of ways to write K as an ordered sum of s positive integers is $C(K-1, s-1)$. Hence the number of valid arrays is*
>
> *Σ over $s \ge 1$ of (number of divisor chains with s distinct values in $1..N$) $\times$ $C(K-1, s-1)$.*

Proof. Write K stars in a row. There are $K-1$ gaps between neighbouring stars. Choosing $s-1$ of the gaps and putting a bar in each cuts the row into s non-empty groups, read left to right, and every ordered sum of s positive parts arises from exactly one such choice. So there are $C(K-1, s-1)$ of them. For the arrays: the map "array $\mapsto$ (its chain of distinct values, its run lengths)" is a bijection between valid arrays and pairs (chain with s values, ordered sum of K into s positive parts), because the chain and the run lengths rebuild the array and every such pair gives a valid array. Summing over s gives the formula. This is an instance of **stars and bars**. ∎

The chain counts come from the same push loop with one more index. Let $\text{cnt}(s, v)$ be the number of strictly increasing divisor chains with s values ending at v. Then $\text{cnt}(1, v) = 1$, and $\text{cnt}(s+1, m)$ is the sum of $\text{cnt}(s, v)$ over the proper divisors v of m.

EXAMPLE 14.2 Counting arrays for N = 6, K = 3

The chain counts for $v = 1..6$ are:

```
s    cnt(s, 1..6)        chains with s values   C(K-1, s-1)   contribution
1    1  1  1  1  1  1            6                C(2,0) = 1        6
2    0  1  1  2  1  3            8                C(2,1) = 2       16
3    0  0  0  1  0  2            3                C(2,2) = 1        3
                                                          total  25
```

For instance cnt(2, 6) = 3 counts the chains (1, 6), (2, 6) and (3, 6), and cnt(3, 6) = 2 counts (1, 2, 6) and (1, 3, 6). The chain (1, 2, 6) gives C(2, 2) = 1 array, [1, 2, 6], while (2, 6) gives two, [2, 2, 6] and [2, 6, 6]. The total, 25, matches the direct DP above and a brute force over all $6^3 = 216$ arrays. The same formula gives 2, 3 and 5 on the Infosys samples, and 39 for $N = 6$, $K = 4$.

The chain counts cost $O(S \cdot N \log N)$ with $S = \lfloor \log N \rfloor + 1$, independent of K. The binomials need care with the modulus. $10^9 + 7$ is prime, so $C(n, r)$ can use factorials and modular inverses (Chapter 10). $10000 = 2^4 \cdot 5^4$ is **not** prime, so inverses of many numbers do not exist; build $C(n, r)$ for $r < S$ with Pascal's rule $C(n, r) = C(n-1, r-1) + C(n-1, r)$, which only adds.

PITFALL

Do not reduce the problem to "count chains" and forget the repeats: the samples (2, 2) → 3 already include the arrays [1, 1] and [2, 2]. And do not count arrays of length K with distinct values only; that is the special case $s = K$.

14.4 Interval DP

DEFINITION 14.2 Interval DP

An **interval DP** has one state per contiguous segment, dp[l][r] for $a[l..r]$, whose value is computed from the values of **shorter segments inside it**. The answer is usually dp[0][$n-1$].

You met this shape in Chapter 12: the longest palindromic subsequence of $s[l..r]$ depends on $s[l+1..r-1]$, $s[l+1..r]$ and $s[l..r-1]$. The new ingredient here is the **split point**: many interval problems are answered by trying every k between l and r and combining dp[l][k] with dp[$k+1$][r].

Evaluation order

LEMMA 14.6 Why "by increasing length" works

If every dp[l][r] reads only entries dp[l'][r'] with $l \le l' \le r' \le r$ and $(l', r') \ne (l, r)$, then filling the table in order of increasing length $r - l + 1$ reads only entries that are already filled. So does the order "l from n − 1 down to 0, and for each l, r from l up to n − 1".

Proof. A proper sub-segment of $[l, r]$ is strictly shorter, so it belongs to an earlier length and has been filled. For the second order, a read entry has either $l' > l$, so it belongs to an earlier pass of the outer loop, or $l' = l$ and $r' < r$, so it was filled earlier in the current pass. ∎

The natural-looking order "l from 0 up, r from l up" breaks the rule: dp[0][r] needs dp[1][r], which belongs to a later pass of the outer loop and is still 0. The program compiles, runs and returns a wrong number. This is the handbook's trap for this card.

The template: the cheapest way to multiply matrices

Multiplying a $p \times q$ matrix by a $q \times s$ matrix costs $p \cdot q \cdot s$ scalar multiplications. A product of n matrices can be bracketed in many ways, all giving the same result at very different costs. If matrix i is $d[i] \times d[i+1]$, what is the cheapest bracketing?

THEOREM 14.7 The last multiplication

Let dp[l][r] be the cheapest cost of multiplying matrices l..r. Then dp[l][l] = 0 and, for l < r, dp[l][r] = min over l ≤ k < r of (dp[l][k] + dp[k + 1][r] + d[l] · d[k + 1] · d[r + 1]).

Proof. In any bracketing, some multiplication is performed **last**. It multiplies the product of matrices $l..k$ (a $d[l] \times d[k+1]$ matrix) by the product of $k+1..r$ (a $d[k+1] \times d[r+1]$ matrix) for some k, and costs $d[l] \cdot d[k+1] \cdot d[r+1]$. Before it, the two halves were computed independently, each by some bracketing of its own, costing at least dp[l][k] and dp[$k+1$][r]. So every bracketing costs at least the right-hand side for its k. Conversely, for each k, combining optimal bracketings of the halves achieves exactly that sum. The minimum over k is therefore the optimum. ∎

Listing 14.3 Interval DP: the cheapest order of matrix multiplications

```
// dp[l][r] = best answer for the segment l..r, filled by increasing length.
// Matrix chain: matrix i is d[i] x d[i+1]. Cheapest order of multiplications.
static long matrixChain(int[] d) {
    int n = d.length - 1;
    long[][] dp = new long[n][n];                       // dp[l][l] = 0
    for (int len = 2; len <= n; len++)
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            dp[l][r] = Long.MAX_VALUE;
            for (int k = l; k < r; k++)                 // last multiplication splits at k
                dp[l][r] = Math.min(dp[l][r],
                        dp[l][k] + dp[k + 1][r] + (long) d[l] * d[k + 1] * d[r + 1]);
        }
    return dp[0][n - 1];
}
```

	r=0	r=1	r=2	r=3
l=0	0	120	264	1080
l=1		0	360	1320
l=2			0	720
l=3				0

Figure 14.2 The table dp[l][r] for d = [4, 10, 3, 12, 20], four matrices. It is filled one diagonal at a time: length 1 (grey, all 0), length 2 (blue), length 3 (green), length 4 (amber). The answer 1080 splits at k = 1: (M0 · M1) · (M2 · M3).

Take the corner entry of Figure 14.2. The three candidates are $k=0$: $0 + 1320 + 4 \cdot 10 \cdot 20 = 2120$; $k=1$: $120 + 720 + 4 \cdot 3 \cdot 20 = 1080$; $k=2$: $264 + 0 + 4 \cdot 12 \cdot 20 = 1224$. The minimum is 1080.

The number of (l, r, k) triples is Σ over lengths of $(n - \text{len} + 1)(\text{len} - 1) = (n + 1)n(n - 1)/6$. For $n = 500$ that is 20,833,250, comfortably fast; for $n = 5000$ it is about $2.1 \cdot 10^{10}$, which is not. Memory is n^2 entries, half of them unused ($l > r$).

14.5 Cutting a stick, and the last operation

LeetCode 1547 (Minimum Cost to Cut a Stick) is the core interval problem of the day. A stick of length L must be cut at given positions; cutting a piece costs the piece's current length; the cuts may be made in any order. Find the minimum total cost.

The order matters. For $L = 10$ and cuts at 2, 4, 7: cutting in the order 2, 4, 7 costs $10 + 8 + 6 = 24$; the order 7, 4, 2 costs $10 + 7 + 4 = 21$; the order 4, 2, 7 costs $10 + 4 + 6 = 20$, which is the minimum.

Modelling

The positions along the stick can be up to 10^6, but there are at most 100 cuts, so the state must be over the **cuts**, not over lengths. Add the two ends as extra points and sort: $p[0] = 0 < p[1] < \cdots < p[m] < p[m + 1] = L$, where m is the number of cuts. Every piece that ever exists during the process runs from one point to another, $p[i]$ to $p[j]$, with all the cuts strictly between them still to be made.

THEOREM 14.8 The stick recurrence

Let dp[i][j] be the minimum cost of making all cuts strictly between p[i] and p[j] on the piece [p[i], p[j]]. Then dp[i][i + 1] = 0, and for j ≥ i + 2, dp[i][j] = (p[j] − p[i]) + min over i < k < j of (dp[i][k] + dp[k][j]). The answer is dp[0][m + 1].

Proof. If $j = i + 1$ there is nothing to cut. Otherwise some cut $p[k]$ with $i < k < j$ is made **first** on this piece, and it costs the full length $p[j] - p[i]$, whichever k it is. After it, the two pieces $[p[i], p[k]]$ and $[p[k], p[j]]$ never interact again: every later cut lies in exactly one of them and its cost depends only on that piece. So the remaining cost is at least dp[i][k] + dp[k][j], and an optimal schedule for each piece achieves it. Minimising over the first cut k gives the recurrence. ∎

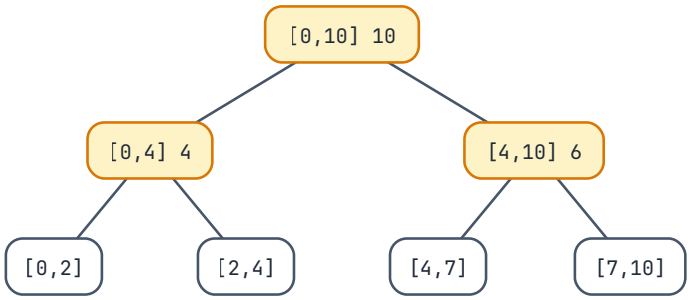


Figure 14.3 The optimal schedule for L = 10 with cuts at 2, 4, 7, drawn as a tree of pieces. Each piece that is cut costs its length (shown after the range); final pieces cost nothing. Total 10 + 4 + 6 = 20.

EXAMPLE 14.3 The stick table

With points $p = [0, 2, 4, 7, 10]$, the table dp[i][j] for $j \geq i + 2$ is filled by increasing $j - i$:

```
piece        points     candidates (dp[i][k] + dp[k][j])            length   dp
[0, 4]       i=0, j=2   k=1: 0 + 0                                     4      4
[2, 7]       i=1, j=3   k=2: 0 + 0                                     5      5
[4, 10]      i=2, j=4   k=3: 0 + 0                                     6      6
```

```
[0, 7]      i=0, j=3   k=1: 0 + 5,   k=2: 4 + 0                      7     11
[2, 10]     i=1, j=4   k=2: 0 + 6,   k=3: 5 + 0                      8     13
[0, 10]     i=0, j=4   k=1: 0 + 13,  k=2: 4 + 6,  k=3: 11 + 0       10     20
```

The best first cut on the whole stick is $k = 2$, the point 4, matching the order 4, 2, 7 found above.

The table has $(m + 2)^2$ entries and the triple loop about $(m + 2)^3/6$ steps: for $m = 100$ cuts, about $1.8 \cdot 10^5$. This is the matrix-chain template with two changes: the cost of a step is the piece's length, and the split point k is shared by both halves. Writing it out is your core exercise for the day.

The last operation instead of the first

In the stick, the **first** cut separates the problem into independent halves. Sometimes the first operation does the opposite. In the bonus problem LeetCode 312 (Burst Balloons), bursting balloon k earns the product of its value and its current neighbours' values, and the burst makes its two neighbours adjacent. Choosing the first balloon to burst therefore changes the neighbours inside both halves, which are no longer independent.

The fix is to choose the balloon burst **last** inside an open interval (l, r), where l and r are balloons that are still standing (pad the row with a 1 at each end). While the balloons strictly between l and k are being burst, k is still there, so the left part's neighbours are fixed as l and k; the right part likewise sees k and r. When k finally goes, its neighbours are l and r. So

$\mathrm{dp}[l][r] = \max$ over $l < k < r$ of ($\mathrm{dp}[l][k] + \mathrm{dp}[k][r] + v[l] \cdot v[k] \cdot v[r]$).

EXAMPLE 14.4 The last balloon

For values [3, 1, 5], padded to $v = [1, 3, 1, 5, 1]$, the three choices for the last balloon on the whole row are:

```
last balloon   left part dp[0][k]   right part dp[k][4]   last burst 1*v[k]*1   total
     3                 0                   30                   3               33
     1                 3                    5                   1                9
     5                30                    0                   5               35
```

The maximum is 35: burst 1 first ($3 \cdot 1 \cdot 5 = 15$), then 3 ($1 \cdot 3 \cdot 5 = 15$), then 5 ($1 \cdot 5 \cdot 1 = 5$). A brute force over all 3! orders agrees.

KEY IDEA

For a split recurrence, ask which single operation, **first** or **last**, leaves the rest of the work in independent pieces whose costs do not depend on each other. Cutting a stick splits on the first cut; bursting balloons and merging piles split on the last operation. The bonus problem LeetCode 664 (Strange Printer) is a third variant, where two prints of the same character can merge.

14.6 Two-player games

In a game DP, two players alternate moves on a shared position, both see everything, and both play perfectly. The DP must describe what perfect play means.

DEFINITION 14.3 Positions, winning and losing

A **position** is everything that determines the rest of the game, including whose turn it is. In a game where the player who cannot move loses, a position is **winning** if the player to move can force a win, and **losing** otherwise.

THEOREM 14.9 Winning means "some move to a losing position"

In a finite game without draws where the player who cannot move loses, a position is losing if and only if every move from it leads to a winning position; in particular a position with no moves is losing.

Proof. By induction on the longest remaining game from the position. If some move leads to a losing position, the player to move makes it and the opponent, now to move in a losing position, cannot force a win; so the position is winning. If every move leads to a winning position, whatever the player does, the opponent can force a win; so the position is losing. A position with no moves falls in the second case vacuously. ∎

Listing 14.4 Winning and losing positions of a take-away game

```
// A pile of s stones; a move removes 1, 3 or 4 stones; a player who cannot move loses.
// win[s] = true if the player to move from s stones wins with perfect play.
static boolean[] winningPositions(int maxS) {
    int[] moves = {1, 3, 4};
    boolean[] win = new boolean[maxS + 1];              // win[0] = false: no move available
    for (int s = 1; s <= maxS; s++)
        for (int m : moves)
            if (m <= s && !win[s - m]) {                // a move to a losing position
                win[s] = true;
                break;
            }
    return win;
}
```

For this game the losing positions up to 14 are 0, 2, 7, 9 and 14: the pattern repeats with period 7 (losing exactly when s mod 7 is 0 or 2), a regularity you can find by printing the table and then prove by induction. Games on arbitrary graphs, and sums of games with the Sprague–Grundy theorem, extend this idea; see the further reading.

Games with scores: minimax

Many problems give points instead of a win or a loss. Two players alternately take a number from the left or the right end of a row $a[0..n-1]$; each adds what they take to their score. This is the setting of LeetCode 486 (Predict the Winner).

The **minimax** principle says that the player to move picks the move whose resulting position is best **for them**, assuming the opponent then does the same. Writing that with two scores is awkward. One number suffices.

DEFINITION 14.4 Score difference

For a segment $a[l..r]$ with some player to move, let $D(l, r)$ be the largest value of (the mover's total from now on) − (the opponent's total from now on) that the mover can guarantee.

THEOREM 14.10 The difference recurrence

*$D(l, l) = a[l]$, and for $l < r$, $D(l, r) = \max(\, a[l] - D(l+1, r),\ a[r] - D(l, r-1)\,)$. Moreover, if each player tries to maximise **their own** total, the first player's total is $(S + D(0, n-1))/2$, where S is the sum of all elements.*

Proof. With one element the mover takes it, so the difference is $a[l]$. Otherwise, suppose the mover takes $a[l]$. The rest of the game is the segment $a[l+1..r]$ with the **opponent** to move, who by induction can guarantee a difference of $D(l+1, r)$ in their own favour, and cannot be held below it. From the mover's side the final difference is $a[l]$ minus that. Taking $a[r]$ is symmetric. The mover chooses the larger.

For the second part: on any segment the two players' totals from now on add up to the segment's sum, which is fixed. If own + other = T, then own − other = 2 · own − T, so maximising one's own total and maximising the difference are the same goal, and the optimal moves coincide. Then the first player's total x satisfies $x + y = S$ and $x - y = D(0, n-1)$, so $x = (S + D)/2$. ∎

So the first player wins or ties exactly when $D(0, n-1) \geq 0$, which is what LeetCode 486 asks (a tie counts as a win for the first player there). Each entry reads two shorter segments, so the table has $n(n+1)/2$ useful entries and the time is $O(n^2)$.

Listing 14.5 Game DP: the first player's best score difference

```java
// Two players take from either end; both play perfectly. dp = my score minus the opponent's.
static long bestScoreDifference(int[] a) {
    int n = a.length;
    long[][] dp = new long[n][n];
    for (int i = 0; i < n; i++) dp[i][i] = a[i];
    for (int len = 2; len <= n; len++)
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            dp[l][r] = Math.max(a[l] - dp[l + 1][r], a[r] - dp[l][r - 1]);
        }
    return dp[0][n - 1];                              // first player wins if >= 0 (LC 486)
}
```

EXAMPLE 14.5 **Why greedy loses**

Take $a = [3, 9, 1, 2]$. If both players greedily take the larger end, the first takes 3, the second takes 9, then 2 and 1 follow, and the first player ends with 5 against 10. The table D tells a different story:

	r=0	r=1	r=2	r=3
l=0	3	6	-5	7
l=1		9	8	8
l=2			1	1
l=3				2

Figure 14.4 The table D(l, r) for a = [3, 9, 1, 2], filled by diagonals. The corner, 7, is the first player's guaranteed margin.

At the corner, taking 3 gives $3 - D(1, 3) = 3 - 8 = -5$, while taking 2 gives $2 - D(0, 2) = 2 - (-5) = 7$. So the first player takes the **smaller** end, 2. The second player then faces [3, 9, 1], where either choice leaves 9 exposed: by $D(0, 2) = -5$ the mover there loses by 5. With perfect play the totals are 11 and 4 (sum 15, difference 7, and indeed $(15 + 7)/2 = 11$).

There is a pretty fact behind the review questions: when n is even, the first player never loses this game. Colour the positions alternately; the first player can always take all even-indexed elements or all odd-indexed elements, whichever sum is larger, because each of their moves exposes only elements of the other colour. On 5,000 random even-length rows the DP never returned a negative value.

When the score of a move depends on what is left

In LeetCode 1690 (Stone Game VII) the player who removes the leftmost or the rightmost stone scores the **sum of the stones that remain**, and the statement says outright that the first player wants to maximise the difference and the second wants to minimise it. The difference recurrence is then the definition of optimal play, not a theorem about it, and only the score of a move changes: removing $a[l]$ from $a[l..r]$ scores the sum of $a[l+1..r]$, which a prefix-sum array gives in O(1) (Chapter 2). A segment with one stone left scores nothing for either player. On the row [2, 7, 1, 4], the first player's best margin works out to 11: removing the 2 scores 7 + 1 + 4 = 12, after which the opponent's best margin on [7, 1, 4] is 1.

> **PITFALL**
>
> Do not mix the two viewpoints. Either store the difference for "the player to move" and subtract the child's value, as above, or store two separate scores and swap them at each level. Storing "player one's score" and adding the child's value treats both players as allies, and gives the answer to a different, cooperative game.

14.7 Choosing the order and the limits

Memoisation or a table

An interval DP can be written top-down: a recursive method `solve(l, r)` that checks a memo table, computes the value from the recurrence by recursive calls, and stores it. Memoisation needs no thought about order, because recursion computes each sub-segment when it is first needed, and it skips states that are never reached. Its costs are a method call per state and a recursion depth equal to the longest chain of nested calls, here at most n (each call shrinks the segment). How deep Java can go depends on the stack size and on the frame size of the method. On the machine used for this book, a recursive method with three `int` parameters overflowed the default stack after about 7,400 nested calls when interpreted and about 11,400 once compiled by the JIT. A depth of 5000 stays below both figures, but not by a wide margin; beyond about 10^4, use a table, or run the recursion on a thread with a large stack (the template is in the plan's Java exam kit).

A bottom-up table avoids both costs and is easy to get right once you apply Lemma 14.6. Prefer it in Java when the whole table is needed anyway, as in every problem of this chapter.

What the limits allow

Table 14.2 Interval and game DPs under typical limits

Shape	Work	$n \le 500$	$n \le 5000$
split DP, $O(n^3)$	$(n+1)n(n-1)/6$ triples	$2.1 \cdot 10^7$: fine	$2.1 \cdot 10^{10}$: too slow
game or end-removal DP, $O(n^2)$	$n(n+1)/2$ states	$1.3 \cdot 10^5$: trivial	$1.25 \cdot 10^7$: fine
`long[n][n]` table	$8n^2$ bytes	2 MB	200 MB: too much for most limits
`int[n][n]` table	$4n^2$ bytes	1 MB	100 MB: risky
rolling `long[n]` (below)	$8n$ bytes	4 KB	40 KB

For $n \le 5000$ a split DP needs a better idea (Knuth's optimisation or a different model) and is outside this plan. An end-removal game, though, only ever reads the previous length, so one array of the current length's values is enough.

Listing 14.6 The same game DP in O(n) memory

```
// d[l] holds dp[l][l + len - 1] for the current length only.
static long bestScoreDifferenceRolling(int[] a) {
    int n = a.length;
    long[] d = new long[n];
    for (int i = 0; i < n; i++) d[i] = a[i];              // length 1
    for (int len = 2; len <= n; len++)
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
            // before this line: d[l] = dp[l][r - 1] (old length), d[l + 1] = dp[l + 1][r] (old length)
            d[l] = Math.max(a[l] - d[l + 1], a[r] - d[l]);
        }
    return n == 0 ? 0 : d[0];
}
```

The comment is the proof. In the pass for length len, `d[l]` still holds the value for $[l, r-1]$ from the previous pass, and `d[l + 1]` holds $[l+1, r]$ because the pass moves left to right and has not reached $l+1$ yet. Overwriting `d[l]` loses nothing that a later step of this pass needs. The rolling version matched Listing 14.5 on random rows of up to 400 elements, and both matched an exhaustive minimax on 4,000 small rows.

EXAM TIP

Before writing an interval DP, write three things on paper: the meaning of dp[l][r] in one sentence, the base case (length 1 or 2), and which single operation, first or last, splits the segment. If you cannot say what is independent after that operation, the recurrence is probably wrong.

Summary

- Sorted, a set in which every pair divides is a divisor chain, so the largest divisible subset is an LIS with "divides" in place of "smaller": $O(n^2)$ with parent pointers.
- To visit all divisor pairs up to N, push from each value to its multiples. The total is $\Sigma \lfloor N/v \rfloor \le N(\lfloor \log N \rfloor + 1)$, about $1.4 \cdot 10^7$ for $N = 10^6$.
- A chain of distinct values in $1..N$ has at most $\lfloor \log N \rfloor + 1$ elements. Arrays in which each element divides the next are chains with repeats: Σ chains$(s) \cdot C(K-1, s-1)$ by stars and bars. With a non-prime modulus, build binomials with Pascal's rule.
- Interval DP stores dp[l][r] and must be filled by increasing length (or l descending, r ascending); "l ascending, r ascending" reads unfilled entries.
- Split recurrences come from the first or the last operation, whichever leaves independent pieces: the first cut for a stick, the last multiplication for matrices, the last balloon for Burst Balloons.
- In a game, a position is winning exactly when some move leads to a losing position. With scores, store the mover's best "my total minus yours": $D(l, r) = \max(a[l] - D(l+1, r), a[r] - D(l, r-1))$. When the total is fixed, this is the same as each player maximising their own score.
- $O(n^3)$ fits $n \le 500$; for $n \le 5000$ only $O(n^2)$ time fits, and a `long[n][n]` table does not fit in memory, so roll the array by length.

Key terms

divisible subset p. 196
divisor chain p. 196
stars and bars p. 200
interval DP p. 201
winning position p. 205
minimax p. 205

Review questions

1. Why must the values be sorted before the divisibility DP? Run the unsorted DP on [4, 2, 8] and compare with the true answer.
2. Compute $\Sigma \lfloor 12/v \rfloor$ for $v = 1..12$ and compare it with the bound of Theorem 14.3.
3. Using the chain counts of Example 14.2, how many arrays of length 4 with values in 1..6 have each element dividing the next?
4. In the matrix-chain template, the loops are changed to "for l from 0 up, for r from $l + 1$ up". Which entry is read before it is written when computing dp[0][2]?
5. Why is the cost of the first cut on a piece the same whichever cut is chosen, and why does that make the stick recurrence possible? Why is the DP over cut indices and not over positions 0..L?
6. Explain in two sentences why "choose the first balloon to burst" does not split Burst Balloons into independent subproblems.
7. Prove that in the take-from-either-end game with an even number of elements, the first player can always score at least half of the total.
8. For the take-away game with moves {1, 3, 4}, list the losing positions up to 14. Prove that $s = 7$ is losing directly from Theorem 14.9.
9. A game DP has $n = 5000$. How much memory does `long[n][n]` need, and how much does Listing 14.6 need? Why can the rolling version not be used for the matrix-chain DP?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 "Dynamic programming", and Chapter 25 "Game theory" for winning and losing positions, the game of nim and the Sprague–Grundy theorem. Free at cses.fi/book.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Dynamic Programming*, whose matrix-chain multiplication section proves the split recurrence in the same way as Theorem 14.7.
- Algorithms for Competitive Programming, "Number of divisors / sum of divisors" (cp-algorithms.com/algebra/divisors.html) and "Games on arbitrary graphs" (cp-algorithms.com/game_theory/games_on_graphs.html).
- Workbook Chapter 14 has a game table, a stick table and a chain-counting table to fill by hand.

Problems

These are the problems for Day 14. Work them in Workbook Chapter 14, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
14.P1	LC 368 · Largest Divisible Subset	MEDIUM	CORE
14.P2	LC 486 · Predict the Winner	MEDIUM	CORE
14.P3	LC 1547 · Minimum Cost to Cut a Stick	HARD	CORE
14.P4	LC 1690 · Stone Game VII	MEDIUM	CORE
14.P5	LC 2338 · Count the Number of Ideal Arrays	HARD	BONUS
14.P6	LC 312 · Burst Balloons	HARD	BONUS
14.P7	LC 664 · Strange Printer	HARD	BONUS

CHAPTER 15 · DAY 15 OF 29 · SATURDAY 21 NOVEMBER 2026 · LEARNING DAY

15

Graphs I: Breadth-First Search, Depth-First Search and Topological Order

Shortest paths by edges, grids, multi-source BFS, Kahn's algorithm and DP on a DAG.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · BFS: fewest edges, grids, many sources, DFS without recursion, components, Topological order (Kahn) and DAG DP, 0-1 BFS
Before this chapter · Chapter 1 (ArrayList, ArrayDeque), Chapter 11 (DP states and order)
Practice · Workbook Chapter 15 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- describe a graph with the standard vocabulary and store it as adjacency lists or as head/next/to arrays, knowing what each costs in memory;
- run breadth-first search from one or many sources, on adjacency lists and on grids, and prove that it finds fewest-edge distances;
- find connected components with an explicit stack, and decide whether a graph is bipartite using the odd-cycle theorem;
- produce a topological order with Kahn's algorithm, detect cycles with it, and run a DP over a DAG in that order;
- use 0-1 BFS when edge costs are 0 or 1, and prove that its deque stays sorted;
- bound every traversal by O(V + E), and avoid the classic bug of marking vertices when they are popped.

A graph is the most general way to say "these things are connected". Cities and roads, cells of a grid and the moves between them, courses and their prerequisites, people and friendships: each is a set of objects with pairwise links, and many questions about them are the same few questions in disguise. How few steps from here to there? Which objects are connected at all? In what order can tasks be done if some must precede others?

This chapter answers those questions with three traversals. Breadth-first search explores in rings of increasing distance and so finds shortest paths when every step costs the same. Depth-first search, written with an explicit stack so that Java's call stack never overflows, finds connected components and two-colourings. Kahn's algorithm peels a directed graph from its sources and either produces an order in which every edge points forward or proves that a cycle exists. A small change to breadth-first search, a deque instead of a queue, then handles edges that cost 0 or 1. Every algorithm here runs in time proportional to the size of the graph, and the last section explains why, and what single line of code destroys that bound.

15.1 Graphs and how to store them

DEFINITION 15.1 Graph

A **graph** $G = (V, E)$ is a set V of **vertices** (also called nodes) and a set E of **edges**, each joining two vertices. In an **undirected** graph an edge $\{u, v\}$ can be used both ways; in a **directed** graph an edge (u, v) goes from u to v only. We write $n = |V|$ and $m = |E|$, and usually number the vertices $0..n-1$.

Some words recur constantly. Two vertices joined by an edge are **neighbours**. The **degree** $\deg(v)$ of a vertex in an undirected graph is its number of incident edges; in a directed graph, the **indegree** counts edges coming in and the **outdegree** edges going out. A **path** is a sequence of vertices $v_0, v_1, \ldots, v_k$ in which consecutive vertices are joined by an edge; its **length** is the number of edges, k. A **cycle** is a path of length at least 1 that returns to its start without repeating an edge (in undirected graphs, of length at least 3, unless there are self-loops or parallel edges).

DEFINITION 15.2 Connected component

In an undirected graph, u and v are **connected** if some path joins them. Being connected is an equivalence relation, and its classes are the **connected components**: maximal sets of vertices in which every two are connected.

One counting fact underlies every running time in this chapter.

LEMMA 15.1 Handshake lemma

In an undirected graph, the degrees add up to twice the number of edges: $\sum \deg(v) = 2m$. In a directed graph, the indegrees add up to m, and so do the outdegrees.

Proof. Count pairs (vertex, incident edge). Each undirected edge $\{u, v\}$ contributes exactly two such pairs, one at u and one at v, so the number of pairs is $2m$. Counting the same pairs vertex by vertex gives $\sum \deg(v)$. A directed edge (u, v) contributes one to the outdegree of u and one to the indegree of v. ∎

Adjacency lists

An adjacency matrix, an $n \times n$ table of booleans, is simple but needs n^2 entries: 10^{10} for $n = 10^5$, far beyond any memory limit. Contest graphs are stored as **adjacency lists**: for each vertex, the list of its neighbours. The handbook's builder takes an edge list, the form in which LeetCode and most exam statements give graphs.

Listing 15.1 Adjacency lists from an edge list

```java
@SuppressWarnings("unchecked")
static List<Integer>[] buildGraph(int n, int[][] edges, boolean directed) {
    List<Integer>[] g = new List[n];
    for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
    for (int[] e : edges) {
        g[e[0]].add(e[1]);
        if (!directed) g[e[1]].add(e[0]);
    }
    return g;
}
```

JAVA NOTE

Java does not allow `new List<Integer>[n]`, because arrays of generic types cannot be created. `new List[n]` creates an array of raw lists, and assigning it to `List<Integer>[]` gives an "unchecked" warning, which `@SuppressWarnings("unchecked")` silences. The code is safe as long as you only put `Integer`s in the lists.

Each list entry is a reference to an `Integer` object. Java caches the objects for −128..127, but larger vertex numbers each get their own 16-byte object, plus a 4-byte slot in the list's array and the spare capacity that `ArrayList` keeps for growth. Measured on the machine used for this book (OpenJDK 25, $n = 10^5$ vertices and $m = 10^6$ directed edges), the lists took about 26 MB, roughly **26 bytes per edge**. That is fine for 10^6 edges and uncomfortable for 10^7.

Three int arrays

When memory or speed matters, store the edges in plain `int` arrays. Edge number e goes to `to[e]`; the edges leaving u form a linked list that starts at `head[u]` and continues through `nxt[e]` until −1. This layout, sometimes called a *forward star*, is what the sample-paper walkthroughs use for trees.

Listing 15.2 Adjacency in three int arrays, and BFS over them

```
// Edge e goes to to[e]; the edges leaving u form a linked list head[u], nxt[head[u]], ..., -1.
static int[] head, nxt, to;
static int edgeCount;

static void initGraph(int n, int maxEdges) {
    head = new int[n];
    Arrays.fill(head, -1);
    nxt = new int[maxEdges];
    to = new int[maxEdges];
    edgeCount = 0;
}

static void addEdge(int u, int v) {            // directed u -> v; add both ways if undirected
    to[edgeCount] = v;
    nxt[edgeCount] = head[u];                  // the new edge goes in front of u's list
    head[u] = edgeCount++;
}

static int[] bfsArrays(int n, int src) {       // the same BFS as Listing 15.3
    int[] dist = new int[n], queue = new int[n];
    Arrays.fill(dist, -1);
    int qh = 0, qt = 0;
    dist[src] = 0;
    queue[qt++] = src;
    while (qh < qt) {
        int u = queue[qh++];
        for (int e = head[u]; e != -1; e = nxt[e]) {
            int v = to[e];
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            queue[qt++] = v;
        }
    }
    return dist;
}
```

The same graph took about 8.7 MB this way: 8 bytes per edge for `nxt` and `to`, and 4 bytes per vertex for `head`. Remember that an undirected edge is stored twice, so `maxEdges` must be $2m$. The neighbours come out in reverse order of insertion, which never matters for the algorithms in this chapter.

Grids are graphs

A grid of R rows and C columns is a graph whose vertices are the cells and whose edges join cells that share a side. Nothing needs to be stored: the neighbours of (r, c) are found with two **direction arrays**.

```
int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};      // down, up, right, left
int rows = 3, cols = 4, r = 0, c = 2;
for (int d = 0; d < 4; d++) {
    int nr = r + DR[d], nc = c + DC[d];
    if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;   // bounds first
    int id = nr * cols + nc;                        // a single int id when you need one
}
```

For eight directions (diagonals too), use the eight pairs (dr, dc) with $dr, dc \in \{-1, 0, 1\}$, not both zero. Check the bounds before you read the cell; reading first throws `ArrayIndexOutOfBoundsException` at the border.

15.2 Breadth-first search

DEFINITION 15.3 Distance

In a graph where every edge has length 1, the **distance** $\delta(s, v)$ from s to v is the fewest edges on a path from s to v, or ∞ if there is no path.

Breadth-first search (BFS) visits the vertices in order of distance from the source: first s, then all its neighbours, then everything two steps away, and so on. A first-in first-out queue produces exactly this order. When a vertex is discovered for the first time, it gets a distance one more than the vertex that discovered it and joins the back of the queue.

Listing 15.3 Breadth-first search: fewest edges from one source

```
// Fewest edges from src to every node (-1 = unreachable)
static int[] bfs(List<Integer>[] g, int src) {
    int[] dist = new int[g.length];
    Arrays.fill(dist, -1);
    ArrayDeque<Integer> q = new ArrayDeque<>();
    dist[src] = 0;
    q.add(src);
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : g[u]) {
            if (dist[v] != -1) continue;       // mark when you PUSH, not when you pop
            dist[v] = dist[u] + 1;
            q.add(v);
        }
    }
    return dist;
}
```

The `dist` array plays two roles: it stores the answer, and −1 means "not yet discovered", so no separate `visited` array is needed.

EXAMPLE 15.1 BFS layer by layer

Take the undirected edges 0–1, 0–2, 1–3, 2–3, 3–4 and start at 0.

```
pop  dist[u]  newly discovered   queue after   dist[0..4]
 0      0     1, 2              [1, 2]        [0, 1, 1, -1, -1]
 1      1     3                 [2, 3]        [0, 1, 1, 2, -1]
```

```
2     1     -   (3 is known)    [3]          [0, 1, 1, 2, -1]
3     2     4                   [4]          [0, 1, 1, 2, 3]
4     3     -                   []           [0, 1, 1, 2, 3]
```

When 2 is popped, its neighbour 3 already has a distance, set when 1 was popped, so it is not queued again.

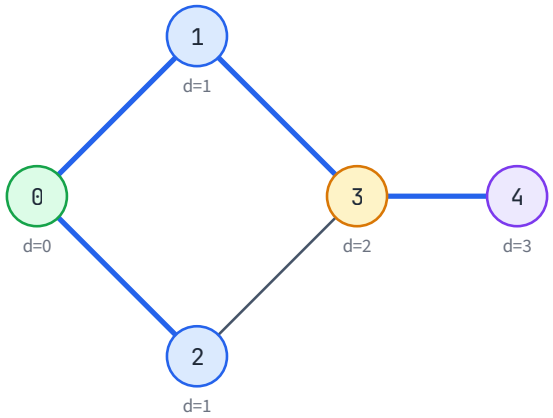


Figure 15.1 The BFS layers of Example 15.1. Vertices at the same distance from 0 share a colour; the blue edges are the ones along which each vertex was first discovered, forming a shortest-path tree.

Why BFS finds shortest paths

LEMMA 15.2 The queue holds at most two consecutive distances

At every moment, if the queue holds $v_1, \dots, v_k$ from front to back, then $dist[v_1] \le dist[v_2] \le \cdots \le dist[v_k] \le dist[v_1] + 1$. Consequently vertices leave the queue in non-decreasing order of dist.

Proof. Initially the queue holds only s. Suppose the statement holds and v_1 is popped with $\text{dist}[v_1] = d$. Every vertex then pushed gets $d + 1$, which is at least every value in the queue (all are d or $d + 1$) and at most (new front) + 1, because the new front has value at least d. So the sequence stays sorted with a spread of at most 1. Since the queue is always sorted and pops come from the front, the popped values never decrease. ∎

THEOREM 15.3 BFS computes distances

After Listing 15.3 finishes, $dist[v] = \delta(s, v)$ for every v reachable from s, and $dist[v] = -1$ for every other vertex.

Proof. First, $\text{dist}[v] \ge \delta(s, v)$ whenever $\text{dist}[v]$ is set. It holds for s; when v gets $\text{dist}[u] + 1$ through the edge u–v, a path to u with $\text{dist}[u]$ edges plus that edge is a path to v with $\text{dist}[u] + 1$ edges. The same argument shows that only reachable vertices are ever discovered.

Now suppose some reachable v ends with a wrong value, and choose one with the smallest $\delta(s, v) = k$. Then $k \ge 1$, since $\text{dist}[s] = 0$ is right. Let u be the vertex before v on a shortest path, so $\delta(s, u) = k - 1$ and, by the choice of v, $\text{dist}[u] = k - 1$. Every discovered vertex is pushed and later popped, so u is popped at some point. At that moment either v is undiscovered, and it receives $\text{dist}[u] + 1 = k$; or v was discovered earlier by some w popped no later than u, and by Lemma 15.2 $\text{dist}[w] \le \text{dist}[u]$, so $\text{dist}[v] = \text{dist}[w] + 1 \le k$. Combined with $\text{dist}[v] \ge k$, either way $\text{dist}[v] = k$, a contradiction. ∎

Each vertex is pushed at most once, because it is marked when pushed. Each adjacency list is scanned once, when its vertex is popped. By Lemma 15.1 the total scanning is $2m$ steps (undirected) or m (directed), so BFS runs in $O(n + m)$ time and $O(n)$ extra memory.

Many sources at once

Sometimes the question is the distance to the **nearest** of several sources: the nearest exit, the nearest hospital, the first minute at which a spreading process reaches each cell.

THEOREM 15.4 Multi-source BFS

If all sources are pushed with distance 0 before the loop starts, the loop computes, for every vertex, the distance to the nearest source.

Proof. Add an imaginary vertex s^* joined to every source. A BFS from s^* would pop s^* first and push all sources with distance 1; from then on it is the same loop, with every distance one larger. By Theorem 15.3 it computes $\delta(s^*, v) = 1 + \min$ over sources t of $\delta(t, v)$, because every path from s^* enters the original graph through some source. Subtracting the 1 gives the claim. ∎

On a grid the template becomes the following, again verbatim from the handbook.

Listing 15.4 Grid BFS from several sources at once

```java
static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};

// Grid BFS from several sources at once ('#' = wall): steps to the nearest source, -1 if none
static int[][] gridBfs(char[][] grid, int[][] sources) {
    int rows = grid.length, cols = grid[0].length;
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, -1);
    ArrayDeque<int[]> q = new ArrayDeque<>();
    for (int[] s : sources) {
        dist[s[0]][s[1]] = 0;
        q.add(s);
    }
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        for (int d = 0; d < 4; d++) {
            int r = cur[0] + DR[d], c = cur[1] + DC[d];
            if (r < 0 || r >= rows || c < 0 || c >= cols) continue;
            if (grid[r][c] == '#' || dist[r][c] != -1) continue;
            dist[r][c] = dist[cur[0]][cur[1]] + 1;
            q.add(new int[]{r, c});
        }
    }
    return dist;
}
```

0	1	2	■	2	1
1	■	3	■	1	0
2	■	4	3	2	1
3	4	5	■	■	2

Figure 15.2 Multi-source BFS on a 4 × 6 grid with two sources (green, distance 0) and walls (dark). Each number is the distance to the nearer source. The amber cell (row 0, column 2) is 2 steps from the left source and 6 from the right one; one BFS finds the smaller value for every cell at once.

EXAMPLE 15.2 A spreading process

In LeetCode 994 (Rotting Oranges), every minute each rotten orange (2) rots its fresh neighbours (1); empty cells (0) block the spread. The time until the last fresh orange rots is the largest multi-source distance over the fresh cells, with all rotten oranges as sources and empty cells as walls. On the grid below, the distances (−1 for empty or never reached) are:

```
grid            distance from the nearest rotten orange
2 1 1 0 1        0   1   2  -1   2
1 0 1 1 1        1  -1   3   2   1
0 1 1 0 2       -1   5   4  -1   0
1 1 0 0 1        7   6  -1  -1   1
```

The answer is 7 minutes, reached by the fresh orange at row 3, column 0. Two edge cases complete the model: if some fresh orange is never reached the answer is −1, and if there are no fresh oranges at all it is 0. Turning this into code is one of the day's core problems.

Eight directions, and counting cells

LeetCode 1091 (Shortest Path in Binary Matrix) changes two details. Moves go to any of the **eight** neighbouring cells, and the length of a path is the number of **cells** on it, not edges, so the start cell has distance 1. If the start or the end cell is blocked, the answer is −1. On the 4 × 4 grid

```
0 0 1 0
1 0 1 0
1 1 0 1
0 0 0 0
```

no four-direction path joins the corners, but with diagonals the path (0, 0) → (1, 1) → (2, 2) → (3, 3) has 4 cells. BFS still applies, because every move still costs the same; only the direction list changes.

15.3 Depth-first search without recursion

Depth-first search (DFS) follows one path as far as it can before backing up. Written recursively it is three lines long: mark u, then for each unmarked neighbour, recurse. In Java that recursion is a trap. Each call uses a stack frame, and the default thread stack holds only some thousands to tens of thousands of frames. A path graph with 10^5 vertices, which is a perfectly legal tree, crashes a recursive DFS with `StackOverflowError`.

INFOSYS LENS

The official sample paper gives trees as parent arrays with up to 10^5 nodes, and its Easy problem needs a traversal of the whole tree. The plan's analysis of the paper names the trap directly: a recursive DFS on a chain overflows the stack in Java. Every traversal in this chapter is therefore written with an explicit `ArrayDeque`, whose size is limited only by the heap.

For connected components the visiting order is irrelevant; what matters is that every vertex is visited exactly once. Replacing the BFS queue with a stack gives the handbook's component labelling.

Listing 15.5 Connected components with an explicit stack

```
1 // Iterative DFS (no stack overflow): comp[v] = component number. Returns how many components.
2 static int components(List<Integer>[] g, int[] comp) {
3     Arrays.fill(comp, -1);
```

```
    int count = 0;
    ArrayDeque<Integer> stack = new ArrayDeque<>();
    for (int s = 0; s < g.length; s++) {
        if (comp[s] != -1) continue;
        comp[s] = count;
        stack.push(s);
        while (!stack.isEmpty()) {
            int u = stack.pop();
            for (int v : g[u]) {
                if (comp[v] != -1) continue;
                comp[v] = count;
                stack.push(v);
            }
        }
        count++;
    }
    return count;
}
```

THEOREM 15.5 The labelling is the component structure

After Listing 15.5, comp[u] = comp[v] if and only if u and v are connected, and the return value is the number of components.

Proof. Consider the round started at s. Every vertex labelled in it was pushed from a labelled neighbour, so by induction it is connected to s. Conversely, let v be connected to s and suppose v is not labelled in this round. On a path from s to v, take the first vertex w that is not labelled in this round, and its predecessor x on the path, which is. Being labelled, x was pushed and therefore popped before the round ended, and at that pop its neighbour w was examined: either w was unlabelled and got labelled now, or it already had a label. That label is from this round, because a vertex labelled in an earlier round would have dragged its whole component, including s, into that round. Either way w is labelled in this round, a contradiction. So each round labels exactly the component of its start vertex, and a new round starts once per component. ∎

The cost is $O(n + m)$ for the same reason as BFS. Note that this stack-based DFS does not visit vertices in the same order as the recursive one, and does not know when a vertex is "finished". For components and colourings that does not matter. For topological order, use Kahn's algorithm of Section 15.4 rather than finishing times.

EXAMPLE 15.3 Islands as components

LeetCode 200 (Number of Islands) asks for the number of connected groups of land cells ('1') in a grid, where cells connect through shared sides. It is Listing 15.5 with the grid's implicit edges in place of `g[u]`: start a new round at every land cell that has no label yet. On the grid

```
1 1 0 0 0
1 1 0 1 1
0 0 1 0 0
1 0 0 1 1
```

there are 5 islands. The single '1' at row 2, column 2 touches the block above-left of it only diagonally, so it is an island of its own.

Two colours: bipartite graphs

DEFINITION 15.4 Bipartite graph

An undirected graph is **bipartite** if its vertices can be coloured with two colours so that every edge joins vertices of different colours.

Bipartite graphs model "two kinds of things, links only across": jobs and workers, two teams that must be separated. Whether a graph is bipartite has a clean characterisation.

LEMMA 15.6 Odd closed walks contain odd cycles

If a graph has a closed walk (a path that may repeat vertices, ending where it starts) of odd length, it has a cycle of odd length.

Proof. By strong induction on the length L of the walk. If no vertex repeats except the start and end, the walk is itself a cycle of odd length. Otherwise some vertex x occurs twice at different positions inside it. Cutting the walk at those two occurrences splits it into two closed walks whose lengths add up to L. Since L is odd, one of them has odd length, and it is shorter than L, so by induction it contains an odd cycle. ∎

THEOREM 15.7 The odd-cycle theorem

An undirected graph is bipartite if and only if it has no cycle of odd length.

Proof. If the graph is bipartite, colours alternate along every cycle, so a cycle returns to its starting colour only after an even number of steps. Conversely, suppose there is no odd cycle, and treat each component separately. Pick a root r, run BFS, and colour each vertex by the parity of its distance from r. If some edge u–v joined two vertices of the same parity, then a shortest path from r to u, the edge, and a shortest path from v back to r would form a closed walk of length $\delta(r, u) + \delta(r, v) + 1$, which is odd. By Lemma 15.6 the graph would contain an odd cycle, a contradiction. So the parity colouring is a valid two-colouring. ∎

The proof is also the algorithm: BFS (or the stack-based DFS) from every uncoloured vertex, give each newly discovered vertex the opposite colour of the vertex that found it, and report "not bipartite" as soon as an edge joins two vertices of the same colour. On the 4-cycle 0–1–2–3–0 the colours alternate 0, 1, 0, 1 and every edge is fine. On the 5-cycle 0–1–2–3–4–0, BFS from 0 gives distances [0, 1, 2, 2, 1], so vertices 2 and 3 both get colour 0 and the edge 2–3 is a conflict, as the theorem predicts for an odd cycle. This is the whole idea of the bonus problem LeetCode 785 (Is Graph Bipartite?); remember that its graph may be disconnected.

15.4 Directed acyclic graphs and topological order

DEFINITION 15.5 DAG and topological order

A **directed acyclic graph** (DAG) is a directed graph with no directed cycle. A **topological order** of a directed graph is an ordering of all its vertices in which every edge (u, v) has u before v.

If a graph has a cycle, no topological order exists: the vertices of the cycle would each have to come before the next one, all the way round. The converse is the content of Kahn's algorithm, and it rests on one small fact.

LEMMA 15.8 Every DAG has a source

Every non-empty DAG has a vertex with indegree 0.

Proof. Start anywhere and repeatedly step backwards along an incoming edge. If every vertex had an incoming edge, this walk could continue for ever; with only n vertices, some vertex would be visited twice, and the part of the walk between the two visits, read forwards, would be a directed cycle. ∎

Kahn's algorithm removes a source, which may create new sources, and repeats. In code, an indegree counter replaces the actual removal: removing u decrements the counter of each of its out-neighbours.

Listing 15.6 Kahn's algorithm: a topological order, or null on a cycle

```java
// Kahn's algorithm on a directed graph. Returns a topological order, or null if there is a cycle.
static int[] topoOrder(List<Integer>[] g) {
    int n = g.length;
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++) for (int v : g[u]) indeg[v]++;
    int[] order = new int[n];
    int head = 0, tail = 0;
    for (int u = 0; u < n; u++) if (indeg[u] == 0) order[tail++] = u;
    while (head < tail) {
        int u = order[head++];
        for (int v : g[u]) if (--indeg[v] == 0) order[tail++] = v;
    }
    return tail == n ? order : null;          // some node never reached indegree 0 -> cycle
}
```

The `order` array doubles as the queue: positions `head..tail - 1` are waiting, positions before `head` are done.

THEOREM 15.9 Correctness of Kahn's algorithm

Listing 15.6 outputs all n vertices if and only if the graph is acyclic, and when it does, the output is a topological order.

Proof. At any moment, indeg[v] equals the number of edges into v from vertices not yet taken out of the queue, because each such removal decrements it once per edge (duplicate edges included). A vertex enters the queue exactly when that number drops to 0.

The output is ordered. For an edge (u, v), v is appended only after its counter reaches 0, which requires u to have been taken out already, and u was appended before it was taken out. So u precedes v.

Acyclic ⇒ everything is output. Suppose the loop ends with some vertices never appended, and let W be that set. The subgraph on W is a DAG, so by Lemma 15.8 some $w \in W$ has no incoming edge from W. All its in-neighbours are outside W, hence were appended and processed, so its counter reached 0 and it was appended: a contradiction.

Cycle ⇒ not everything is output. On a directed cycle, the first vertex of the cycle to be appended would need its predecessor on the cycle to have been processed before it, which is impossible. ∎

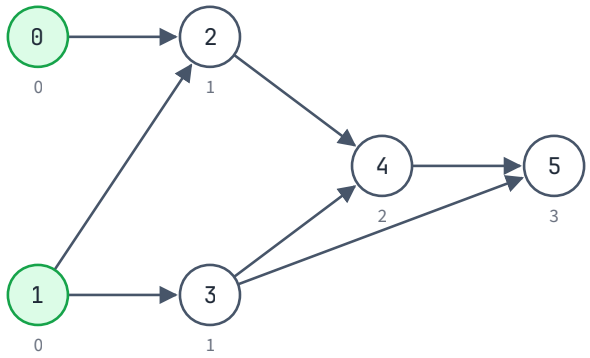


Figure 15.3 A DAG with six vertices. Kahn's algorithm starts with the sources 0 and 1 (green) and outputs 0, 1, 2, 3, 4, 5. The note under each vertex is the longest path, in edges, that ends there.

EXAMPLE 15.4 Kahn's algorithm step by step

For the edges 0→2, 1→2, 1→3, 2→4, 3→4, 4→5, 3→5 of Figure 15.3 the initial indegrees are [0, 0, 2, 1, 2, 2], so the queue starts as [0, 1].

```
take   counters decremented (new value)   queue after   indeg after
 0     2 (1)                              [1]           [0, 0, 1, 1, 2, 2]
 1     2 (0), 3 (0)                       [2, 3]        [0, 0, 0, 0, 2, 2]
 2     4 (1)                              [3]           [0, 0, 0, 0, 1, 2]
 3     4 (0), 5 (1)                       [4]           [0, 0, 0, 0, 0, 1]
 4     5 (0)                              [5]           [0, 0, 0, 0, 0, 0]
 5     -                                  []
```

All six vertices are output, in the order 0, 1, 2, 3, 4, 5. Add the edge 5 → 1 and the graph has the cycle 1 → 3 → 5 → 1: only vertex 0 is ever output, the counters of 1..5 never reach 0, and the method returns `null`.

LeetCode 207 (Course Schedule) is this algorithm with one trap in the input: a pair [*a*, *b*] means "to take *a* you must first take *b*", so the edge goes **from *b* to *a***. The schedule is possible exactly when the order is not `null`. Drawing the edge the wrong way reverses every edge, which preserves cycles, so the yes/no answer happens to survive; any question about the order itself would not.

DP over a DAG

A topological order is exactly the evaluation order a DP needs: when a vertex is processed, every vertex with an edge into it has already been processed.

THEOREM 15.10 DP in topological order

Let $F(v)$ be defined from the values $F(u)$ of the in-neighbours u of v (and a base value for sources). Processing the vertices in topological order and, for each u, pushing $F(u)$ along every edge (u, v) computes every $F(v)$ correctly.

Proof. By induction along the order. When u is processed, all its in-neighbours come earlier, so all their pushes into u have already happened and $F(u)$ is final. Its own pushes then use the final value. ∎

Listing 15.7 DP on a DAG in topological order

```java
// DP on a DAG in topological order: longest path (in edges) ending at each node
static int[] longestPathEndingAt(List<Integer>[] g, int[] order) {
    int[] best = new int[g.length];
    for (int u : order) for (int v : g[u]) best[v] = Math.max(best[v], best[u] + 1);
    return best;
}

// Number of paths from src to every node, modulo 1e9+7, in the same order
static long[] countPaths(List<Integer>[] g, int[] order, int src) {
    final long MOD = 1_000_000_007L;
    long[] ways = new long[g.length];
    ways[src] = 1;
    for (int u : order) for (int v : g[u]) ways[v] = (ways[v] + ways[u]) % MOD;
    return ways;
}
```

On Figure 15.3 the longest paths ending at each vertex are [0, 0, 1, 1, 2, 3], and the numbers of paths starting at vertex 1 are [0, 1, 1, 1, 2, 3]: vertex 5 is reached by 1→3→5, 1→3→4→5 and 1→2→4→5. In a general graph, longest paths are a hard problem; in a DAG they cost O(n + m), because the order removes every cycle from consideration.

The same scheme handles weights on vertices. If each vertex is a task with a duration and an edge means "must finish before", the earliest finishing time of a task is its own duration plus the largest finishing time among its prerequisites. With durations [3, 2, 4, 1, 2, 5] on Figure 15.3 the finishing times are [3, 2, 7, 3, 9, 14], so everything is done at time 14. That one sentence is the key observation of the bonus problem LeetCode 2050 (Parallel Courses III).

15.5 0-1 BFS

Plain BFS assumes every edge has the same cost. If some moves are free and others cost 1, it gives wrong answers. In the directed graph with edges 0→1 (cost 1), 0→2 (0), 2→1 (0), 1→3 (1), 2→4 (1) and 3→4 (0), plain BFS reports [0, 1, 1, 2, 2] (it counts edges), while the cheapest costs are [0, 0, 0, 1, 1].

Dijkstra's algorithm (Chapter 17) handles any non-negative costs with a priority queue in O((n + m) log n). When the costs are only 0 and 1 there is a cheaper way: a deque in which a vertex reached by a free edge goes to the **front** and one reached by a paid edge goes to the **back**. This is **0-1 BFS**.

Listing 15.8 0-1 BFS with a deque

```java
// Weights are only 0 or 1: a deque replaces the heap. Weight 0 -> front, weight 1 -> back.
static int[] zeroOneBfs(List<int[]>[] g, int src) {
    int[] d = new int[g.length];
    Arrays.fill(d, Integer.MAX_VALUE);
    d[src] = 0;
    ArrayDeque<Integer> dq = new ArrayDeque<>();
    dq.add(src);
    while (!dq.isEmpty()) {
        int u = dq.pollFirst();
        for (int[] e : g[u]) {
            int v = e[0], nd = d[u] + e[1];
            if (nd >= d[v]) continue;
            d[v] = nd;
            if (e[1] == 0) dq.addFirst(v);
            else dq.addLast(v);
        }
    }
    return d;
}
```

Here `g[u]` holds pairs {*v*, cost}. Unlike plain BFS, a vertex can be improved after it was pushed, so the test is "is the new distance smaller?", made every time, and a vertex can sit in the deque twice.

EXAMPLE 15.5 The deque at work

On the graph above, starting at 0 (each entry is a vertex; its current *d* is in the last column):

```
pop   d[u]   relaxations                  deque after    d[0..4]
 0     0     1 back (1), 2 front (0)      [2, 1]         [0, 1, 0, -, -]
 2     0     1 front (0), 4 back (1)      [1, 1, 4]      [0, 0, 0, -, 1]
 1     0     3 back (1)                   [1, 4, 3]      [0, 0, 0, 1, 1]
 1     0     -  (an old entry: nothing improves)   [4, 3]
 4     1     -                            [3]
 3     1     4: 1 + 0 is not < 1          []             [0, 0, 0, 1, 1]
```

Vertex 1 was pushed at the back with cost 1, then improved to 0 through the free edge from 2 and pushed again at the front. The second entry for 1 is popped later and changes nothing.

THEOREM 15.11 Correctness and cost of 0-1 BFS

Give each deque entry the key d[v] that v had when the entry was pushed. Then (a) the keys are non-decreasing from front to back and differ by at most 1; (b) when Listing 15.8 ends, d[v] is the cheapest cost from the source; (c) each vertex is pushed at most twice, so the running time is O(n + m).

Proof. (a) An entry whose key is larger than the current $d[u]$ is outdated: u was improved later and the newer entry, with a smaller key, was popped first. When u was popped with its newer key, all its edges were relaxed with that value, and d values only go down, so popping the outdated entry relaxes nothing. Now suppose (a) holds and the front entry, with key $K = d[u]$, is popped. All keys in the deque lie in $[K, K + 1]$. A free edge pushes key K at the front, which is ≤ every key; a paid edge pushes $K + 1$ at the back, which is ≥ every key and at most front + 1. So (a) still holds.

(b) By (a), entries leave in non-decreasing key order, so the deque acts as a priority queue. Suppose some vertex u is popped with key $K = d[u]$ while the true cost $\delta(u)$ is smaller. On a cheapest path from the source to u, let x be the last vertex that has already been popped with $d[x] = \delta(x)$ (the source qualifies), and y the next vertex on the path. When x was popped, the edge to y was relaxed, so $d[y] \le \delta(x) + \text{cost}(x, y) = \delta(y) \le \delta(u) < K$, the middle step because costs are non-negative. The entry pushed for y with that key has not been popped, otherwise y would qualify instead of x. So the deque holds a key below K while K is at the front, contradicting (a). Hence every vertex's first real pop has $d = \delta$, and later relaxations cannot lower it further.

(c) When v first gets a finite d, it is K or $K + 1$, where K is the key being popped. Later keys are at least K by (a), so $d[v]$ can later drop at most once, from $K + 1$ to K. Each drop causes one push, so v is pushed at most twice and its edge list is scanned at most twice: $O(n + m)$ in total. ∎

On 4,000 random graphs with costs 0 and 1 the listing matched a Bellman–Ford brute force, and no vertex was ever pushed more than twice.

On a grid

Grids are where 0-1 BFS appears most. Suppose each cell has a colour, stepping onto a cell of the same colour is free, and stepping onto a different colour costs 1. The fewest colour changes from the top-left to the bottom-right cell is a 0-1 shortest path.

Listing 15.9 0-1 BFS on a grid: fewest colour changes

```java
// Walk from the top-left to the bottom-right cell. Stepping onto a cell with the same letter
// costs 0; stepping onto a different letter costs 1. Cell (r, c) has the id r * C + c.
static int fewestColourChanges(char[][] g) {
    int R = g.length, C = g[0].length;
    int[] dist = new int[R * C];
    Arrays.fill(dist, Integer.MAX_VALUE);
    int[] dr = {1, -1, 0, 0}, dc = {0, 0, 1, -1};
    ArrayDeque<Integer> dq = new ArrayDeque<>();
    dist[0] = 0;
    dq.add(0);
    while (!dq.isEmpty()) {
        int id = dq.pollFirst(), r = id / C, c = id % C;
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            int w = g[nr][nc] == g[r][c] ? 0 : 1, nid = nr * C + nc;
            if (dist[id] + w >= dist[nid]) continue;
            dist[nid] = dist[id] + w;
            if (w == 0) dq.addFirst(nid);
            else dq.addLast(nid);
        }
    }
    return dist[R * C - 1];
}
```

a 0	a 0	b 1	a 2	b 3
a 0	a 0	a 0	b 1	a 2
b 1	a 0	a 0	b 1	b 1
a 2	b 1	b 1	b 1	a 2

Figure 15.4 Fewest colour changes on a 4 × 5 grid of letters a (blue) and b (amber). Each cell shows its cost from the top-left corner. The bottom-right cell costs 2, although walking right along the top row and then down the last column would change colour 6 times.

The bonus problem LeetCode 1368 has the same structure with a different rule for which moves are free: following the arrow printed in a cell costs 0, and moving in any other direction costs 1 (the price of changing the arrow). Once you see the two costs, it is this listing with a different `w`.

EXAM TIP

Use 0-1 BFS when every cost is 0 or 1, or more generally 0 or one fixed value c (divide by c). Use Dijkstra for any other non-negative costs, and plain BFS when all costs are equal. If an edge can have a negative cost, none of the three works.

15.6 Cost, and a checklist for traversals

THEOREM 15.12 **Linear time**

BFS, the stack-based component search, the bipartite check and Kahn's algorithm each run in $O(n + m)$ time on adjacency lists; 0-1 BFS too, by Theorem 15.11. On an $R \times C$ grid with four directions this is $O(RC)$.

Proof. In each algorithm a vertex enters the queue or stack at most once, because it is marked (given a distance, a label, a colour, or a zero counter) at the moment it is pushed, and marked vertices are never pushed again. Each adjacency list is scanned once, when its vertex is removed. So the work is $O(n)$ for pushes and pops plus the total length of the adjacency lists, which is $2m$ or m by Lemma 15.1. A grid has $n = RC$ vertices and at most $4RC$ directed adjacencies. ∎

The proof leans on one line of every template: **mark a vertex when you push it, not when you pop it**. Here is the version that marks on pop.

```
// WRONG: marks a node when it is popped, and overwrites dist on every push
static int[] bfsMarkOnPop(List<Integer>[] g, int src) {
    int[] dist = new int[g.length];
    boolean[] done = new boolean[g.length];
    ArrayDeque<Integer> q = new ArrayDeque<>();
    q.add(src);
    while (!q.isEmpty()) {
        int u = q.poll();
        done[u] = true;                          // marked only now
        for (int v : g[u]) {
            if (done[v]) continue;
            dist[v] = dist[u] + 1;               // may overwrite a shorter distance
            q.add(v);                            // v may already be in the queue
        }
    }
    return dist;
}
```

It is wrong twice over. **Wrong answers:** on the triangle 0–1, 0–2, 1–2 it returns [0, 1, 2], because when 1 is popped, 2 is still waiting in the queue and not yet marked, so its distance 1 is overwritten with 2. On 2,000 random small graphs it gave a wrong distance on 663. **Exploding time:** a vertex can be pushed once by every neighbour that is popped before it, and each copy, when popped, pushes its own neighbours again. On an open 10 × 10 grid this version made 184,755 queue insertions instead of 100, and on a 20 × 20 grid it ran out of memory. A "lazy" repair that skips a popped vertex already marked (`if (done[u]) continue;` right after `poll`) is correct, but still makes up to one insertion per edge: 181 on the 10 × 10 grid. Marking on push keeps it to one per vertex.

CHECKLIST Before you submit a traversal

- Every vertex is marked when it is **pushed**, and marked vertices are never pushed again (except in 0-1 BFS, where the test is "strictly better").
- Grid moves check the bounds **before** reading the cell; walls and already-visited cells are skipped in the same `if`.
- Unreachable vertices keep a sentinel (−1 or `Integer.MAX_VALUE`) and the answer handles it (LeetCode 994 wants −1 if a fresh orange is never reached).
- Edge directions match the statement: "[a, b] means b before a" gives the edge $b \to a$.
- The graph may be disconnected: loop over every vertex as a possible start (components, bipartite check).
- Distances counted in **cells** start at 1, distances in **edges** at 0.
- Vertices numbered 1..n in the input: either allocate $n + 1$ or subtract 1 when reading.
- No recursion deeper than a few thousand calls in Java; use an explicit stack or queue.

Summary

- A graph is vertices and edges, directed or undirected. Store it as `List<Integer>[]` (about 26 bytes per edge) or as head/next/to `int` arrays (8 bytes per edge). Grids are implicit graphs with direction arrays.
- The handshake lemma, $\Sigma \deg = 2m$, is why every traversal costs $O(n + m)$.
- BFS pops vertices in non-decreasing distance and finds the fewest-edge distance to every vertex. Pushing several sources with distance 0 gives the distance to the nearest source.
- DFS with an explicit `ArrayDeque` labels connected components without the risk of `StackOverflowError`.
- A graph is bipartite exactly when it has no odd cycle; colouring by BFS parity either succeeds or finds an edge between equal colours.
- Kahn's algorithm outputs every vertex exactly when the graph is acyclic, and the output is a topological order. DP over a DAG in that order computes longest paths, path counts and finishing times in $O(n + m)$.
- 0-1 BFS puts free moves at the front of a deque and paid moves at the back; the deque stays sorted, each vertex is pushed at most twice, and the cost is $O(n + m)$.
- Mark when you push, not when you pop: marking on pop gives wrong distances and can blow the queue up exponentially.

Key terms

graph p. 211
connected component p. 211
bipartite graph p. 218
directed acyclic graph p. 218
topological order p. 218
adjacency list p. 211
Breadth-first search p. 213
Depth-first search p. 216
0-1 BFS p. 221

Review questions

1. A simple undirected graph has 7 vertices with degrees 3, 3, 2, 2, 2, 1 and 1. How many edges does it have? Could a graph have degrees 3, 3, 3, 2, 2?
2. Estimate the memory of adjacency lists for $n = 10^5$ and $m = 2 \cdot 10^6$ undirected edges, both as `List<Integer>[]` and as head/next/to arrays.
3. In the proof of Theorem 15.3, where is Lemma 15.2 used? What would go wrong if the queue were replaced by a stack?
4. Explain why multi-source BFS is not the same as running BFS from each source and taking the minimum, in cost though not in result. What is the cost of each for k sources?
5. Why is a graph containing a self-loop never bipartite? How does the BFS colouring detect it?
6. Kahn's algorithm is run on a graph with a cycle and returns `null`. Which vertices did it output before stopping? Characterise them.
7. In LeetCode 207, what happens to the answer if every edge is drawn in the wrong direction? Would the same be true for a problem that asks for the order of courses?
8. Why can't plain BFS be used when edge costs are 0 or 1? Give a three-vertex example where it fails.
9. Show that in 0-1 BFS a vertex can be pushed twice, with a concrete graph, and explain why it is never pushed three times.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 11 "Basics of graphs" (terminology and representations), Chapter 12 "Graph traversal" (DFS, BFS, connectivity, bipartiteness) and Chapter 16 "Directed graphs" (topological sorting and dynamic programming on DAGs). Free at cses.fi/book.
- Algorithms for Competitive Programming, "Breadth First Search" (cp-algorithms.com/graph/breadth-first-search.html), "Topological Sorting" (cp-algorithms.com/graph/topological-sort.html) and "0-1 BFS" (cp-algorithms.com/graph/01_bfs.html).
- Algorithms for Competitive Programming, "Bipartite Graph Check" (cp-algorithms.com/graph/bipartite-check.html).
- Workbook Chapter 15 has BFS layers, a Kahn trace and a 0-1 BFS deque to complete by hand.

Problems

These are the problems for Day 15. Work them in Workbook Chapter 15, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
15.P1	LC 200 · Number of Islands	MEDIUM	CORE
15.P2	LC 994 · Rotting Oranges	MEDIUM	CORE
15.P3	LC 207 · Course Schedule	MEDIUM	CORE
15.P4	LC 1091 · Shortest Path in Binary Matrix	MEDIUM	CORE
15.P5	LC 785 · Is Graph Bipartite?	MEDIUM	BONUS
15.P6	LC 1368 · Minimum Cost to Make at Least One Valid Path in a Grid	HARD	BONUS
15.P7	LC 2050 · Parallel Courses III	HARD	BONUS

CHAPTER 16 · DAY 16 OF 29 · SUNDAY 22 NOVEMBER 2026 · MOCK DAY

16

Analysing a Problem Before You Code

3 hours with number theory and DP added, then a full review.

Part II · Number Theory, Dynamic Programming and First Graphs
Handbook cards · —
Before this chapter · Chapters 1–15; Chapter 9 (Mock 1, brute force and stress testing)
Practice · Workbook Chapter 16 · 3 problems · core ≈ 4.0 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- strip a story-style statement down to its objects, its question type and its limits, and name a candidate technique;
- bound the total work of loops whose inner part varies, using the aggregate method, and prove that a doubling array appends in O(1) amortised time;
- estimate the time and memory of a Java solution from the limits, including the hidden costs of boxing, hash maps and recursion;
- choose `int` or `long` from the bounds and check modular arithmetic against exact arithmetic;
- map the signals of a statement to the techniques of Parts I and II, and design edge-case tests before submitting;
- run Mock 2 under exam rules and review it against the SP checkpoint.

Today is the second full mock: three Codeforces problems in three hours, with number theory and dynamic programming now part of what you know. The chapter is about the ten or fifteen minutes before you type anything. Strong contestants do not solve problems faster because they type faster; they spend those minutes deciding *what* the problem is, *how big* it is, and *what could go wrong*, so that the code they write is the right code the first time.

The sections follow that order. First, turning a story into a mathematical model. Then the tools for judging whether an idea is fast enough: amortised analysis for loops whose cost varies, and concrete budgets for Java's operations and memory. Then the two checks that catch most wrong answers after the idea is right: types and modular arithmetic, and hand-made edge cases. A map from signals to techniques collects Parts I and II in one table. The chapter ends with the mock protocol and, behind a warning box, the key ideas of the three mock problems for after you have tried them.

16.1 From story to model

Infosys statements are long stories, and so are many Codeforces ones. The story is there to be removed.

DEFINITION 16.1 Model of a problem

The **model** of a problem is the statement with the story removed. It has three parts: the **objects** (an array, a string, a grid, a graph or tree, a set of intervals, numbers), the **question type** (count, optimise, decide or construct), and the **limits** (the largest sizes and values).

INFOSYS LENS

The plan's analysis of the official sample paper describes the style to expect: long story-style statements, input with one value per line, often a function such as `get_ans(...)` to complete, typical limits of $n \le 10^5$, and answers modulo $10^9 + 7$. Its Lesson 2 is about modelling too: in the sample Medium problem the written input format and the samples disagree about where p comes, and only the order used by the samples gives the expected outputs. Check your reading of the input against the samples before writing any logic.

The objects

Translate every noun of the story into one of a handful of structures. "Tables in a row", "days", "houses along a street": an array or a string. "Cities and roads", "servers and cables", "cells you can walk between": a graph. "Employees and managers", "a parent array": a tree. "Meetings", "trains with arrival and departure times": intervals. A good test is to redraw the sample input as that structure on paper; if the sample answer is obvious on the drawing, the model is right.

The question type

Each question type has a short list of usual techniques.

Table 16.1 Question types and where they usually lead

Type	Typical wording	Usual techniques
count	"how many ways", "number of pairs", "modulo $10^9 + 7$"	prefix counts with a map, DP, combinatorics
optimise	"minimum cost", "maximum length", "fewest moves"	greedy with an exchange proof, DP, BFS, binary search on the answer
decide	"is it possible", "YES or NO"	usually an optimisation in disguise: compute the best and compare
construct	"print any arrangement", "output the order"	a greedy rule with a proof, or DP with reconstruction

The limits

The largest n nearly decides the complexity before you have an idea. The handbook's table is worth knowing by heart; it assumes roughly 10^8 to $3 \cdot 10^8$ simple operations per second in Java.

Table 16.2 From the largest n to the complexity that fits (handbook)

Largest n	Complexity that fits	Usual technique
≤ 10–12	$O(n! \cdot n)$	permutations, backtracking
≤ 20–22	$O(2^n \cdot n)$	bitmask DP, subset enumeration
≤ 500	$O(n^3)$	interval DP, Floyd–Warshall
≤ 5 000	$O(n^2)$	two-dimensional DP, all pairs
≤ $2 \cdot 10^5$	$O(n \log n)$	sorting, heaps, segment trees, binary search
≤ 10^6–10^7	$O(n)$	prefix sums, two pointers, sieve
≤ 10^{12}–10^{18}	$O(\sqrt{n})$ or $O(\log n)$	divisors up to $\sqrt{n}$, binary search, fast power

Values matter as well as sizes. Values up to 10^6 can index an array (counting, sieves, DP over values); values up to 10^9 cannot, and need sorting, compression or a hash map. A sum of 10^5 values of size 10^9 is 10^{14}, which needs `long`.

EXAMPLE 16.1 Three stories, three models

Story A. "A gardener has a row of $n \le 10^5$ trees with heights up to 10^9. He may cut down trees, and wants the remaining heights, read left to right, never to decrease. How few trees must he cut?" Objects: an array. Question: optimise (fewest removals). The kept trees form a non-decreasing subsequence, so the answer is n minus the longest non-decreasing subsequence. Limits: $n = 10^5$ rules out $O(n^2)$ and allows patience sorting with upper bound (Chapter 13).

Story B. "$n \le 10^5$ servers are linked by $m \le 2 \cdot 10^5$ cables. Some servers are infected; each second, every infected server infects the servers linked to it. After how many seconds is every server infected, or will some never be?" Objects: a graph. Question: optimise (a time). The infection time of a server is its distance to the nearest initially infected server, so the answer is the largest multi-source BFS distance, or -1 if some server is unreachable (Chapter 15). Limits: $O(n + m)$ is about $3 \cdot 10^5$ steps.

Story C. "Count the strings of length $n \le 10^5$ over the letters a, b, c with no two equal neighbours, modulo $10^9 + 7$." Objects: strings, but the question only depends on the previous letter. Question: count. A DP over (position, last letter) with 3 states, or the closed form $3 \cdot 2^{n-1}$ for $n \ge 1$, which fast power computes in $O(\log n)$ (Chapter 10).

16.2 Amortised analysis

Many loops do not cost the same at every step. A sliding window's inner `while` sometimes moves the left pointer many times and sometimes not at all; an `ArrayList.add` is usually instant but occasionally copies the whole array. Bounding each step by its worst case gives a uselessly large bound. Amortised analysis bounds the **total** instead.

DEFINITION 16.2 Amortised cost

A sequence of k operations has **amortised cost** c per operation if the total cost of any k consecutive operations from the start is at most $c \cdot k$. Individual operations may cost much more than c.

The aggregate method

The simplest technique is to count the total work directly, usually by charging it to something that can only happen a limited number of times. This is the **aggregate method**.

THEOREM 16.1 Two pointers move O(n) times

If two indices $l \le r$ both start at 0, only ever increase, and never exceed n, then all the loop iterations that move one of them number at most $2n$ in total, however those moves are distributed among the steps of an outer loop.

Proof. Every iteration that moves a pointer increases $l + r$ by at least 1. The sum starts at 0 and never exceeds $2n$, so there are at most $2n$ such iterations. ∎

That is why the sliding window of Chapter 3 is $O(n)$ even though its inner `while` can run n times in one step. The same argument applies to a monotonic stack: charge each operation to the index it pushes or pops.

Listing 16.1 Aggregate counting: a monotonic stack does at most 2n stack operations

```
// next[i] = index of the first element to the right of i that is larger than a[i], or -1.
// ops counts pushes and pops; each index is pushed once and popped at most once.
static long ops;

static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] next = new int[n];
    Arrays.fill(next, -1);
    ArrayDeque<Integer> stack = new ArrayDeque<>();    // indices whose answer is still unknown
    ops = 0;
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && a[stack.peek()] < a[i]) {
            next[stack.pop()] = i;                     // a[i] is the first larger value
            ops++;
        }
        stack.push(i);
        ops++;
    }
    return next;
}
```

Each index is pushed exactly once, and popped at most once because a popped index never returns. So `ops` $\le 2n$. Measured on 10^6 elements: a random array gave 1,999,980 operations, a strictly increasing one 1,999,999 (every index pushed and then popped by its neighbour), and a strictly decreasing one 1,000,000 (nothing is ever popped). A single step of the `for` loop can pop thousands of indices; the total cannot exceed $2n$.

Growing an array by doubling

`ArrayList` stores its elements in an ordinary array. When the array is full, `add` allocates a bigger one and copies every element across. With doubling, the code looks like this.

Listing 16.2 A growable int array that doubles its capacity

```
// What ArrayList does, with doubling instead of its factor 1.5. Counts the copying work.
static final class IntList {
    int[] data = new int[1];
    int size = 0;
    long copies = 0;                                   // elements copied by all resizes so far

    void add(int x) {
        if (size == data.length) {
            data = Arrays.copyOf(data, 2 * data.length);  // every element is copied once
            copies += size;
        }
        data[size++] = x;
    }

    int get(int i) { return data[i]; }
}
```

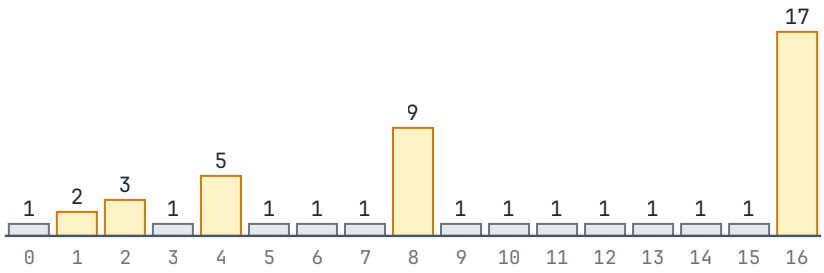


Figure 16.1 The cost of each of the first 17 appends to IntList, counting one unit per element written. Most appends cost 1; the appends that trigger a resize cost 1 plus the number of elements copied (2, 3, 5, 9, 17). The tall bars get rarer exactly as fast as they get taller.

THEOREM 16.2 Doubling gives O(1) amortised appends

Starting from capacity 1, n appends to Listing 16.2 copy fewer than 2n elements in total, so the total work is less than 3n element writes.

Proof. Resizes happen when the size reaches 1, 2, 4, ..., 2^k, where 2^k is the largest power of two below n. They copy $1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1$ elements. Since $2^k < n$, this is less than $2n$. Adding the n ordinary writes gives less than $3n$. ∎

For $n = 10^6$ the count is 1,048,575 copies, just above n. Java's `ArrayList` grows by a factor 1.5 instead of 2 (the new capacity is the old one plus half of it, starting from 10). The same geometric-series argument bounds the copies by about $3n$; simulating its rule for 10^6 appends gives 30 resizes and 2,430,972 copies. A smaller factor wastes less memory and copies more often. If you know the final size, `new ArrayList<>(n)` avoids every resize.

KEY IDEA

"O(1) amortised" is a statement about the total. It is safe inside a loop that runs n times, but it does not promise that every single call is fast. A Java `HashMap` that doubles its table behaves the same way.

16.3 Operation and memory budgets in Java

A complexity class tells you how the cost grows; the limits need an actual number. The planning figure from the handbook is 10^8 to $3 \cdot 10^8$ simple operations per second in Java. Which operations count as "simple" matters as much as how many there are.

Table 16.3 Measured costs on the machine used for this book (one run each; approximate)

Work	Time
10^9 iterations of a simple arithmetic loop	0.40 s
10^8 multiply-then-modulo steps (`x * 31 % MOD`)	0.43 s
10^8 sequential reads of an `int[]`	0.06 s
10^8 random reads of a 64 MB `int[]`	0.80 s
10^6 `HashMap.merge` calls with random `Integer` keys	0.23 s
10^6 increments of an `int[]` counter with the same keys	0.02 s
sorting 10^6 `int`s with `Arrays.sort(int[])`	0.13 s

Work	Time
sorting 10^6 `Integer`s with `Arrays.sort(Integer[])`	0.71 s

The machine was an Intel Xeon at 2.5 GHz running OpenJDK 25; a judge may be two or three times slower, and the just-in-time compiler makes short benchmarks noisy. Treat the table as orders of magnitude. Three lessons survive any machine. Random memory access is more than ten times slower than sequential access. Hash maps cost about ten times more than arrays indexed by the key. **Boxing**, storing an `int` inside an `Integer` object, costs several times more than the plain `int`, in sorting and everywhere else.

Memory

Table 16.4 Approximate memory per item, measured on a 64-bit JVM with compressed references

Structure, 10^6 items	Bytes per item
`int[]`	4
`long[]`	8
`Integer[]` (filled with distinct objects)	about 20: a 16-byte object plus a 4-byte reference
`ArrayList<Integer>`	about 23
`HashSet<Long>`	about 69
`HashMap<Integer, Integer>`	about 77
`int[n][2]` (10^6 rows of two)	about 29 per row
`int[2][n]` (two rows of 10^6)	about 8 per pair

The last two lines are the same data. Every Java array is an object with a header of about 16 bytes, so a million tiny rows cost more than three times as much as two long rows. When you store pairs, prefer two parallel arrays or `int[2][n]` to `int[n][2]`.

To budget memory, multiply cells by bytes before you allocate. A `long[5000][5000]` DP table is $2 \cdot 10^8$ bytes, about 200 MB, too much for a typical 256 MB judge limit once the JVM's own memory is added; `long[2 · 10^5][2]` as two arrays is 3.2 MB.

The call stack

Recursion depth is the third budget. On this machine the default stack held about 24,700 nested calls of a method with one `int` parameter and about 11,400 of a method with three, but only about 9,800 and 7,400 when the same methods ran interpreted, before the JIT compiled them. A tree with 10^5 nodes in a chain needs 10^5 nested calls. Plan for iterative traversals (Chapter 15) whenever depth can exceed a few thousand.

EXAMPLE 16.2 Budgeting before coding

A statement has $n \le 2 \cdot 10^5$ and values up to 10^9, and your idea sorts the values and then does a binary search for each one. Sorting is about $n \log n \approx 3.5 \cdot 10^6$ comparisons and the searches the same again: well under a second, even at the slow end of the budget. Values up to 10^9 do not fit an array index, but sorting does not need one. If instead your idea is "for every pair", that is $(2 \cdot 10^5)^2 = 4 \cdot 10^{10}$ steps, more than a hundred seconds: write it only as the brute force for the stress test. If the idea needs a `HashMap<Integer, Integer>` with one entry per element, the map costs about $2 \cdot 10^5 \times 77$ bytes ≈ 15 MB, which is fine.

16.4 Choosing types and checking modular arithmetic

Integer overflow in Java is silent: `int` and `long` arithmetic wraps around without an exception. A correct idea with a wrong type fails on the large tests only, which is the worst way to fail, because the samples pass.

Table 16.5 Bounds to remember

Quantity	Bound	Type
`Integer.MAX_VALUE`	2,147,483,647 ≈ $2.1 \cdot 10^9$	
`Long.MAX_VALUE`	9,223,372,036,854,775,807 ≈ $9.2 \cdot 10^{18}$	
a sum of 10^5 values up to 10^9	10^{14}	`long`
a product of two values up to 10^9	10^{18}	`long`, just
a product of two residues below $10^9 + 7$	about $1.0 \cdot 10^{18}$	`long`; reduce before the next multiplication
a product of three values up to 10^6	10^{18}	`long`, just

PITFALL

`(long) (a * b)` multiplies in `int` first and converts the already wrong result. For $a = b = 100000$ it gives 1,410,065,408; `(long) a * b` converts first and gives 10,000,000,000. The cast must touch an operand, not the result.

Modular arithmetic, checked

When the answer is "modulo m", reduce after **every** addition and multiplication, so every intermediate value stays below m and every product of two of them below m^2. Subtraction needs care because Java's `%` keeps the sign of the left operand: `-7 % 3` is −1, while `Math.floorMod(-7, 3)` is 2. Division is multiplication by an inverse, which exists for every non-zero value only when m is prime (Chapter 10); for a composite modulus like 10000, avoid division altogether (Chapter 14 builds binomials with Pascal's rule for that reason).

Two cheap checks catch most mistakes while you test.

Listing 16.3 Let Java report overflow while you test

```
// Sum of a[i] * b[i]. The Exact methods throw ArithmeticException instead of wrapping around.
static long dotChecked(int[] a, int[] b) {
    long s = 0;
    for (int i = 0; i < a.length; i++)
        s = Math.addExact(s, Math.multiplyExact((long) a[i], (long) b[i]));
    return s;
}
```

`Math.addExact` and `Math.multiplyExact` behave like `+` and `*` but throw `ArithmeticException` on overflow. Two pairs of $10^9 \cdot 10^9$ give $2 \cdot 10^{18}$, which fits; ten pairs of $2 \cdot 10^9 \cdot 2 \cdot 10^9$ throw "long overflow". Run your solution with them on the largest values once, then switch back if speed matters.

For modular formulas, compare with exact arithmetic on small inputs. `BigInteger` never overflows.

Listing 16.4 Checking modular arithmetic against exact arithmetic

```
// The product of all a[i] modulo m, computed two ways. On small random inputs, both must agree.
static long productMod(long[] a, long m) {
    long p = 1 % m;
    for (long x : a) p = p * Math.floorMod(x, m) % m;     // floorMod: x may be negative
```

```
    return p;
}

static boolean agreesWithBigInteger(long[] a, long m) {
    BigInteger exact = BigInteger.ONE;
    for (long x : a) exact = exact.multiply(BigInteger.valueOf(x));
    return exact.mod(BigInteger.valueOf(m)).longValue() == productMod(a, m);
}
```

On 5,000 random arrays with values between -10^9 and 10^9, the two agreed every time. The same check with `x % m` in place of `Math.floorMod(x, m)` disagreed on 2,532 of 5,000 arrays, every one of them containing a negative value. `BigInteger.mod` always returns a non-negative result, which is what the problem means by "modulo".

16.5 Which pattern?

After the model and the limits, the third question is "which technique does this look like?". The signals below come from the handbook's map, restricted to what Parts I and II have taught, with the chapter where each technique lives.

Table 16.6 Signals and techniques, Chapters 1–15

If the problem says…	Try first	Chapter
count subarrays whose sum, remainder or XOR equals something	prefix sums + `HashMap` of earlier prefixes	2
many "add *v* to *l*..*r*" updates, read once at the end	difference array	2
longest or shortest contiguous part where a condition holds	sliding window	3
maximum or minimum of every window of size *k*	monotonic deque	3
pairs or triples with a target sum	two pointers after sorting	3
meetings, trains, intervals, "in any order"	sort and sweep, or a heap	4, 5
repeatedly take the best available option	greedy with a heap	5
minimise the maximum, maximise the minimum	binary search on the answer	6
next greater or smaller, sums over all subarrays of a minimum	monotonic stack	7
XOR of pairs, "XOR less than *K*"	binary trie, per-bit counting	8
maximum XOR of a subset	XOR basis	8
no idea yet, or you need partial marks	brute force first, then stress test	9
"modulo $10^9 + 7$", count ways, nCr	modular arithmetic and DP	10
product is a perfect square, many factorisations	sieve and square-free kernel	10
items under a capacity, reach a sum, coin change	knapsack	11
two strings: align, transform, common subsequence	two-string DP	12
increasing subsequence, with or without an extra condition	LIS; Fenwick tree of maxima plus an extra dimension	13
every pair divides, multiples, chains of divisors	sort plus LIS-style DP; loops over multiples	14
a segment's answer comes from splitting it, $n \le 500$	interval DP	14
two players take from the ends, both play perfectly	game DP on "my score minus yours"	14
fewest moves, all moves cost the same, grids	BFS (0-1 BFS for costs 0 and 1)	15

If the problem says…	Try first	Chapter
prerequisites, order of tasks, cycle in a directed graph	topological order	15
connected groups, islands, “can I reach”	components with an explicit stack	15

EXAM TIP

When two rows match, let the limits decide. “Increasing subsequence” with $n \le 5000$ can be the quadratic DP; with $n \le 2 \cdot 10^5$ it must be patience sorting or a Fenwick tree. When no row matches, write the brute force (Chapter 9), print its answers for small inputs, and look for a pattern.

16.6 Designing your own tests

The samples in a statement are chosen to explain the problem, not to break your code. Before submitting, run **edge cases**: small inputs that you designed to break it.

CHECKLIST Edge cases to try on every problem

- **The smallest input**: $n = 1$, an empty string, a graph with one vertex. Loops that assume two elements, and answers initialised to the wrong value, fail here.
- **All values equal.** Separates strict from non-strict comparisons (strict LIS is 1, non-decreasing LIS is n) and finds `<` written for `<=`.
- **Sorted and reverse-sorted.** Extreme cases for greedy choices, for monotonic stacks (nothing is popped, or everything is), and for subsequences.
- **Maximum values** at the maximum size. Finds overflow and time limits at once.
- **Negative values and zeros**, if allowed. Find `%` on negatives, sums that start at 0 instead of the first element, and sliding windows that assume positive values.
- **Duplicates.** Break `Arrays.binarySearch` on repeated keys, `TreeSet` (which drops duplicates) and “first occurrence” logic.
- **Disconnected or unreachable** parts, for graph problems: is the answer −1, 0 or something else?

A small generator keeps these cases at hand.

Listing 16.5 Edge-case inputs to run before you submit

```
// A few small arrays that break wrong solutions surprisingly often. maxV = the largest allowed value.
static List<int[]> edgeCases(int n, int maxV) {
    List<int[]> out = new ArrayList<>();
    out.add(new int[]{maxV});                                  // n = 1
    int[] equal = new int[n], up = new int[n], down = new int[n], big = new int[n], signs = new int[n];
    for (int i = 0; i < n; i++) {
        equal[i] = 7;                                          // all values equal
        up[i] = i + 1;                                         // sorted, strictly increasing
        down[i] = n - i;                                       // reverse-sorted
        big[i] = maxV;                                         // every value at the limit: overflow
        signs[i] = i % 2 == 0 ? -maxV : maxV;                  // negatives, alternating signs
    }
    out.add(equal);
    out.add(up);
    out.add(down);
    out.add(big);
    out.add(signs);
    return out;
}
```

For $n = 5$ and $maxV = 10^9$ it produces $[10^9]$, $[7, 7, 7, 7, 7]$, $[1, 2, 3, 4, 5]$, $[5, 4, 3, 2, 1]$, five copies of 10^9, and $[-10^9, 10^9, -10^9, 10^9, -10^9]$. Run your solution on each, and work out the expected answer by hand or with the brute force. Then run the random stress test of Chapter 9 for the cases you did not think of. Edge cases and random tests complement each other: random inputs rarely contain $n = 1$ or all-equal arrays, and hand-made cases rarely contain the strange combination that breaks a greedy rule.

16.7 Mock 2 and its review

The mock follows the plan's rules, the same as Mock 1.

CHECKLIST Mock rules

- Timer at 180 minutes, phone in another room, one sitting.
- Minutes 0–15: read every problem. On paper, write the limits, your idea and its complexity for each. The workbook's planning grid has a row per problem.
- Solve the problem you are surest of first, which is not always Q1.
- Test on the samples with Codeforces *Custom test* before you submit.
- By minute 150, every problem has a submission, even if it is only a brute force.

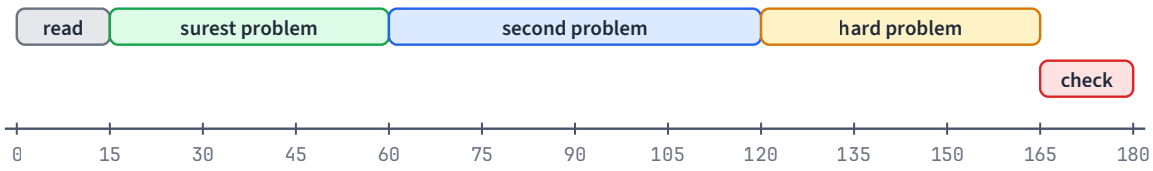


Figure 16.2 The 180 minutes of the exam plan, which the mock rehearses: read and plan, the surest problem, the second problem, the hard problem (brute force first if no idea comes in 15 minutes), and a final check that every submission compiles.

Codeforces gives no partial marks, but its verdict names the first failing test ("Wrong answer on test 7"). Use it to estimate the share of tests you would pass on a judge that does give partial marks, and enter that percentage on the score sheet. The weighted score uses the weights 20, 30 and 50: full marks on Q1 and Q2 and a fifth of Q3 give 20 + 30 + 10 = 60.

The checkpoint

INFOSYS LENS

The plan's checkpoint for Mock 2, on track for the SP role (the plan's own estimate, not an Infosys figure): **Q1 and Q2 fully solved, and a weighted score of at least 50 out of 100**. If you are below it, skip next week's bonus problems, redo the core problems of your weakest day, and spend the review block on the DP problems of Days 11–14.

The review, with a DP focus

The post-mortem takes about an hour, and its order matters.

1. **Score first.** Enter each problem's estimated share of tests and the weighted score.
2. **For each unsolved problem**, read the editorial, close it, and write the solution yourself. Reading is not solving; the solution you can retype from memory is the only one that will be there in December.

3. **Log the cause** of every failure in the error log: misread, wrong idea, overflow, off-by-one, time limit, input, edge case, stack overflow, or ran out of time. The workbook's post-mortem table has a column for it.
4. **For DP failures, find the broken step** of the recipe of Chapter 11: was the state wrong (it forgot something the transition needed, as in Chapter 13's "ends at i"), the transition incomplete, the base case wrong, the evaluation order wrong (Chapter 14's "by increasing length"), or the answer read from the wrong cell?
5. **Pick the weakest topic** and schedule two of its core problems for tomorrow.

EXAM TIP

A mistake that appears twice in the error log deserves a rule written in the margin of your template. "Cast before multiplying" or "lower bound for strict, upper bound for non-decreasing" is cheaper to remember than to rediscover under exam pressure.

16.8 After the mock

PITFALL Read only after the mock

The key idea of each Mock 2 problem, for your review. Each was checked by a solution of my own against a brute force on 5,000 random small inputs.

Codeforces 1367C (Social Distance). Scan the tables from left to right and occupy a free table whenever no occupied table, original or newly placed, lies within distance k on either side. Precompute the position of the next original 1 to the right so that each check is O(1). Taking the leftmost possible table is safe by an exchange argument: any optimal placement can move its first new table left to the greedy one without creating a conflict. The whole scan is O(n) per test.

Codeforces 1373D (Maximum Sum on Even Positions). Reversing a subarray of odd length keeps every element on a position of the same parity, so it never helps. Reversing an even-length subarray swaps the elements of each adjacent pair inside it, so the gain is a sum of pair differences: a[$2j + 1$] − a[$2j$] for pairs that start at an even index, or a[$2j - 1$] − a[$2j$] for pairs that start at an odd one. The best gain is the maximum subarray sum (Kadane, allowing the empty choice) of either difference sequence. Add it to the original sum of even positions, and use `long`.

Codeforces 1475G (Strange Beauty). The kept elements must form a divisibility chain once sorted (Chapter 14), with equal values allowed. Count each value, and let dp[v] be the largest kept set whose maximum is v: dp[v] = cnt[v] + max dp[d] over the proper divisors d of v. Push from each v to its multiples in increasing order, which costs about $2.5 \cdot 10^6$ steps for values up to $2 \cdot 10^5$. The answer is n minus the largest dp value.

Summary

- Before coding, write the model: objects, question type (count, optimise, decide, construct) and limits. Redraw the sample as the model to check it.
- The largest n nearly fixes the complexity; values up to 10^6 can be array indices, values up to 10^9 cannot.
- The aggregate method bounds a loop's total work by charging it to events that happen a limited number of times: pointer moves, pushes and pops. Doubling an array makes appends O(1) amortised, with fewer than $2n$ copies for n appends.

- Budget with 10^8 to $3 \cdot 10^8$ simple operations per second, but remember that hash maps, boxing and random memory access each cost an order of magnitude more than plain arrays.
- Memory is cells times bytes plus object overhead: about 77 bytes per `HashMap<Integer, Integer>` entry and about 16 bytes for every array header. The default stack holds only thousands of frames.
- Use `long` for sums and products, cast before multiplying, reduce after every modular operation, and verify with `Math.multiplyExact` and `BigInteger` on small cases.
- Test $n = 1$, all-equal, sorted, reverse-sorted, maximum values, negatives and duplicates before submitting, then stress test against a brute force.
- Mock 2's checkpoint is Q1 and Q2 fully solved with a weighted score of at least 50.

Key terms

problem model p. 227
amortised analysis p. 229
aggregate method p. 229
Boxing p. 232
Integer overflow p. 233
edge case p. 235

Review questions

1. Model this story: "A library has $n \le 2 \cdot 10^5$ books on a shelf, each with a thickness. A reader wants the longest run of adjacent books whose total thickness is at most W." Give the objects, the question type, the technique and the complexity.
2. A loop has an outer `for` over $i = 0..n-1$ and an inner `while` that increments j, which is never reset. Why is the total work $O(n)$ and not $O(n^2)$?
3. In Listing 16.2, how many elements are copied in total for $n = 17$ appends? Check your answer against the proof of Theorem 16.2.
4. Why would growing an array by a constant amount (say 1000 elements each time) not give $O(1)$ amortised appends? Estimate the copies for $n = 10^6$.
5. A solution stores 10^6 pairs as `int[1000000][2]`. About how much memory does it use, and how would you cut it to a quarter?
6. Which of these overflow, with a, b of type `int` up to 10^9: `long s = a + b;`, `long p = (long) a * b;`, `long q = a * b % 1_000_000_007;`?
7. Why does `BigInteger.mod` make a good reference for a modular computation, while `%` on `long` values does not?
8. Name the edge case from Section 16.6 most likely to break each of: a strict LIS written with upper bound; a sum stored in `int`; a sliding window that assumes positive values.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 2 "Time complexity" (estimating efficiency from the input size) and Chapter 8 "Amortized analysis" (two pointers and nearest smaller elements). Free at cses.fi/book.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Amortized Analysis*, which treats the aggregate, accounting and potential methods and dynamic tables in full.
- Java 8 API documentation, `java.lang.Math` (`addExact`, `multiplyExact`, `floorMod`) and `java.math.BigInteger` (docs.oracle.com/javase/8/docs/api/).
- Workbook Chapter 16 has the mock score sheet, the post-mortem table and exercises on modelling, budgets and edge cases.

Problems

These are the problems for Day 16. Work them in Workbook Chapter 16, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
16.P1	CF 1367C · Social Distance	CF 1300	CORE
16.P2	CF 1373D · Maximum Sum on Even Positions	CF 1600	CORE
16.P3	CF 1475G · Strange Beauty	CF 1900	CORE

PART III

Graphs, Trees and Range Queries

Days 17–23 · Dijkstra, union-find, trees, Fenwick and segment trees, bitmask and digit DP

The official sample paper's hardest material lives here: rooted trees given as parent arrays, answers needed for every subtree, and arrays that change between queries. Part III covers weighted shortest paths, union-find, tree recursion without the call stack, lowest common ancestors, tree DP and re-rooting, small-to-large merging, Fenwick trees, segment trees with and without lazy propagation, and the exponential techniques (backtracking, bitmask DP, digit DP) that earn partial marks when n is small.

17 Graphs II: Shortest Paths, Union-Find and Spanning Trees

18 Trees I: Recursion, Parent Arrays and Ancestors

19 Trees II: Tree DP, Rerooting and Small-to-Large

20 Fenwick trees and segment trees

21 Lazy segment trees and the K-parts DP

22 Backtracking, bitmask DP, digit DP

23 Mock 3 (4 problems) + review

CHAPTER 17 · DAY 17 OF 29 · MONDAY 23 NOVEMBER 2026 · LEARNING DAY

17

Graphs II: Shortest Paths, Union-Find and Spanning Trees

Weighted shortest paths, expanded state graphs, union-find and Kruskal.

Part III · Graphs, Trees and Range Queries
Handbook cards · Dijkstra, Union-find (DSU) and Kruskal
Before this chapter · Chapter 15 (BFS, adjacency lists, 0-1 BFS) and Chapter 5 (heaps)
Practice · Workbook Chapter 17 · 7 problems · core ≈ 3.8 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- prove that Dijkstra's algorithm settles vertices in order of distance, and implement it with a `PriorityQueue` and lazy deletion in $O((V + E) \log V)$;
- change the cost of a path (largest step, number of shortest paths) and check whether the proof of Dijkstra's algorithm still applies;
- turn extra conditions (stops used, a coupon, obstacles removed) into a state-expanded graph, and choose between Dijkstra, Bellman–Ford rounds and 0-1 BFS;
- implement union-find with union by size and path halving, and explain why each operation costs almost $O(1)$;
- prove the cut property and use it to justify Kruskal's and Prim's algorithms, choosing Prim's $O(V^2)$ version for dense graphs;
- answer threshold questions offline by switching vertices on in sorted order and merging them with union-find.

Chapter 15 found shortest paths by breadth-first search, which counts edges. Most real questions put a number on each edge instead: the minutes a signal needs to cross a cable, the price of a flight, the height difference between two cells. Counting edges then gives the wrong answer, because a route with more edges can be cheaper. This chapter develops the algorithms that handle weights.

The chapter has two halves that meet at the end. The first half is about **shortest paths**: Dijkstra's algorithm, a proof of why its greedy choice is safe, the variants that change what "cost" means, and graphs whose vertices carry extra state. The second half is about **connecting things cheaply**: the union-find structure, which tracks groups that merge, and the minimum spanning tree, which connects every vertex at the least total cost. The last section uses union-find to answer questions of the form "using only values up to T" by processing everything in sorted order.

Throughout, a graph has V vertices numbered $0..V-1$ and E edges, and an edge (u, v) has a weight $w(u, v)$. Directed edges go from u to v; undirected edges are stored as two directed ones.

17.1 Weighted shortest paths and Dijkstra's algorithm

DEFINITION 17.1 Path weight and distance

The weight of a path is the sum of the weights of its edges; the empty path from s to itself has weight 0. The **distance** $\mathrm{dist}(s, v)$ is the smallest weight of any path from s to v, or ∞ if v cannot be reached. A path that achieves it is a *shortest path*.

A shortest path with non-negative weights can always be taken to be simple (no repeated vertex): removing a cycle from a path never increases its weight. So there are finitely many candidates, but far too many to enumerate. All the algorithms in this chapter work instead with a tentative distance $d[v]$ for every vertex and improve it step by step.

DEFINITION 17.2 Relaxation

To **relax** the edge (u, v) means: if $d[u] + w(u, v) < d[v]$, set $d[v] = d[u] + w(u, v)$. Initially $d[s] = 0$ and $d[v] = \infty$ for every other vertex.

Relaxation is safe in one direction: it never makes $d[v]$ smaller than the truth.

LEMMA 17.1 Tentative distances are upper bounds

With non-negative weights, at every moment and for every vertex v, either $d[v] = \infty$ or $d[v]$ is the weight of some walk from s to v. In particular $d[v] \geq \mathrm{dist}(s, v)$.

Proof. At the start only $d[s] = 0$ is finite, and it is the weight of the empty walk. A relaxation sets $d[v]$ to $d[u] + w(u, v)$, where $d[u]$ is the weight of a walk to u; appending the edge (u, v) gives a walk to v of exactly that weight. Finally, a walk that repeats a vertex contains a cycle, and cutting the cycle out leaves a walk between the same ends that is no heavier, because the cycle's weight is ≥ 0. Repeating this gives a simple path no heavier than the walk, so every walk weighs at least $\mathrm{dist}(s, v)$. ∎

The question is which edges to relax, and in what order, so that each tentative distance becomes exact quickly. Breadth-first search answers it for unit weights: process vertices in order of distance. Dijkstra's algorithm does the same for arbitrary non-negative weights, using a priority queue to find the closest unfinished vertex.

The algorithm

Keep a min-heap of pairs (d, u). Repeatedly take the pair with the smallest d. If d still equals the current $d[u]$, vertex u is now **settled**: its distance is final, and you relax all of its outgoing edges, pushing a new pair for every vertex whose tentative distance drops. If d is larger than $d[u]$, the pair is out of date and you throw it away.

The second rule exists because Java's `PriorityQueue` cannot lower the key of an element that is already inside it. (`remove(Object)` exists but scans the whole heap, which costs O(size).) Instead of updating the old pair (9, 5) when $d[5]$ falls to 7, the algorithm pushes a new pair (7, 5) and leaves the old one in the heap. The old pair is now a **stale entry**. It is recognised when it surfaces, because $9 \neq d[5]$, and skipped. This technique is called **lazy deletion**.

Listing 17.1 Weighted adjacency lists

```java
@SuppressWarnings("unchecked")
static List<int[]>[] weightedGraph(int n, int[][] edges, boolean directed) {   // edges[i] = {u, v, w}
    List<int[]>[] g = new List[n];
    for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
    for (int[] e : edges) {
        g[e[0]].add(new int[]{e[1], e[2]});              // g[u] holds {to, weight}
        if (!directed) g[e[1]].add(new int[]{e[0], e[2]});
    }
    return g;
}
```

Listing 17.2 Dijkstra's algorithm with lazy deletion (handbook template)

```java
// Non-negative weights. g[u] holds {to, weight}. Returns Long.MAX_VALUE for unreachable nodes.
static long[] dijkstra(List<int[]>[] g, int src) {
    long[] d = new long[g.length];
    Arrays.fill(d, Long.MAX_VALUE);
    d[src] = 0;
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));
    pq.add(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[1];
        if (cur[0] != d[u]) continue;           // stale entry: u was already finalised
        for (int[] e : g[u]) {
            long nd = d[u] + e[1];
            if (nd < d[e[0]]) {
                d[e[0]] = nd;
                pq.add(new long[]{nd, e[0]});
            }
        }
    }
    return d;
}
```

JAVA NOTE

Three details in Listing 17.2 matter in Java. Distances are `long`: a path of 10^5 edges of weight 10^9 weighs 10^{14}. The comparator is `Long.compare(x[0], y[0])`, never `(int) (x[0] - y[0])`, which overflows and silently breaks the heap order. And `Long.MAX_VALUE` is safe as "infinity" because the code only ever adds to `d[u]` after `u` has been taken from the heap, which means `d[u]` is finite.

Why the greedy choice is safe

Dijkstra's algorithm never reconsiders a settled vertex. That is only correct if the distance of a vertex is already exact at the moment it is settled. The proof below is written so that it uses just two facts about path weights, marked (P1) and (P2). Section 17.2 reuses it for other costs.

- (P1) **Extending a path never makes it cheaper**: the weight of a path followed by one more edge is $\geq$ the weight of the path. For sums this is exactly $w \geq 0$.
- (P2) **Extension respects order**: if $x \leq y$, then $x + w \leq y + w$.

THEOREM 17.2 **Correctness of Dijkstra's algorithm**

With non-negative weights, whenever Listing 17.2 settles a vertex u (takes a pair (d[u], u) that is not stale), d[u] = dist(s, u), and d[u] never changes afterwards. When the loop ends, every vertex reachable from s has been settled, so d holds all distances.

Proof. Suppose the first claim fails, and let u be the **first** vertex settled with $d[u] > \text{dist}(s, u)$. Let S be the set of vertices settled before u; by the choice of u, every $x \in S$ has $d[x] = \text{dist}(s, x)$. The source is in S: it is settled first, with $d[s] = 0 = \text{dist}(s, s)$. Take a shortest path P from s to u, let y be the first vertex of P outside S, and let x be the vertex just before y on P, so $x \in S$.

When x was settled, the edge (x, y) was relaxed, so afterwards $d[y] \le \text{dist}(s, x) + w(x, y)$. The part of P from s to x weighs at least $\text{dist}(s, x)$, so by (P2) the part of P from s to y weighs at least $\text{dist}(s, x) + w(x, y) \ge d[y]$. By (P1), the rest of P only adds weight, so $d[y] \le \text{weight}(P) = \text{dist}(s, u) < d[u]$. In particular $y \ne u$.

Every time $d[y]$ receives a value, a pair with that value is pushed. The pair carrying the current value of $d[y]$ cannot have been taken out yet, because taking it out would have settled y, and $y \notin S$. So at the moment u is settled, the heap contains a pair with key $d[y] < d[u]$. A min-heap returns its smallest key, so it could not have returned $(d[u], u)$. This contradiction proves the first claim.

After u is settled, relaxations can only lower $d[u]$, but by Lemma 17.1 it cannot go below $\text{dist}(s, u) = d[u]$. So it never changes. Finally, if some reachable vertex were never settled, apply the same argument to a path from s to it, with S the set of all settled vertices: the first unsettled vertex y on that path received a pair when its predecessor was settled, and that pair would have been taken out and settled y before the heap emptied. ∎

Cost

Each vertex is settled at most once, and settling it scans its outgoing edges once, so the algorithm performs at most E relaxations in total. A pair is pushed only after a successful relaxation, so the heap receives at most $E + 1$ pairs, and at most $E + 1$ pairs are taken out (most of them may be stale). Each heap operation costs $O(\log(E + 1))$. A simple graph has $E < V^2$, so $\log E < 2 \log V$, and the total is

$$O((V + E) \log V) \text{ time and } O(V + E) \text{ memory.}$$

The V term pays for initialising the arrays. For $V = E = 2 \cdot 10^5$ this is a few million heap operations, comfortably under a second in Java.

EXAMPLE 17.1 Network Delay Time by hand

LeetCode 743 (Network Delay Time) sends a signal from node k along directed cables with travel times and asks when the last node receives it, or -1 if some node never does. The answer is the largest distance from k. Take five nodes, $k = 1$, and the cables of Figure 17.1. The table follows Listing 17.2; "heap" lists the pairs (d, v) still waiting, smallest first.

```
step  pair taken  action  relaxations that succeed    d[1..5]           heap afterwards
 1    (0,1)       settle  d[2]: inf->4, d[3]: inf->1  [0, 4, 1, inf, inf] (1,3) (4,2)
 2    (1,3)       settle  d[2]: 4->3,   d[4]: inf->6  [0, 3, 1, 6, inf]   (3,2) (4,2) (6,4)
 3    (3,2)       settle  d[4]: 6->4,   d[5]: inf->9  [0, 3, 1, 4, 9]     (4,2) (4,4) (6,4) (9,5)
 4    (4,2)       stale   -                           unchanged           (4,4) (6,4) (9,5)
 5    (4,4)       settle  d[5]: 9->7                  [0, 3, 1, 4, 7]     (6,4) (7,5) (9,5)
 6    (6,4)       stale   -                           unchanged           (7,5) (9,5)
 7    (7,5)       settle  (no outgoing cables)        [0, 3, 1, 4, 7]     (9,5)
 8    (9,5)       stale   -                           unchanged           empty
```

Three of the eight pairs are stale. Each was pushed when a vertex was first reached by a longer route, and became useless when a shorter route was found. The distances are [0, 3, 1, 4, 7], so the signal reaches every node and the answer is 7.

To turn Listing 17.2 into a LeetCode 743 submission, build the graph from `times` (subtract 1 from node numbers, or allocate $n + 1$ lists and leave list 0 empty), run it from k, and return the maximum distance, or -1 if any distance is still `Long.MAX_VALUE`.

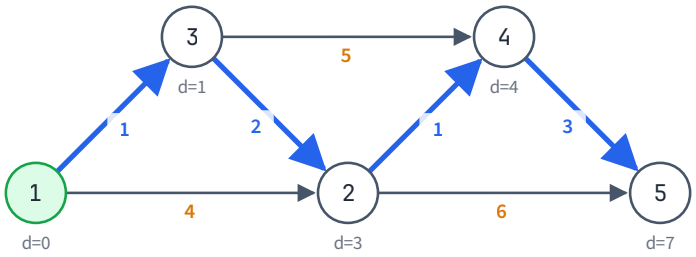


Figure 17.1 The cables of Example 17.1 with travel times. Notes give the final distances from node 1; thick edges form the shortest-path tree (each vertex's last relaxation).

PITFALL

Two shortcuts break the algorithm. Marking a vertex as visited when it is **pushed**, as BFS does, freezes the first tentative distance: in Example 17.1, node 2 would keep the value 4. And forgetting the stale check does not give wrong answers, but every stale pair rescans its vertex's edges, which can multiply the running time by the vertex degree.

Why negative weights break it

The proof used (P1): extending a path never makes it cheaper. A negative edge breaks exactly that step, because a path through an unsettled vertex y can end up cheaper than $d[y]$.

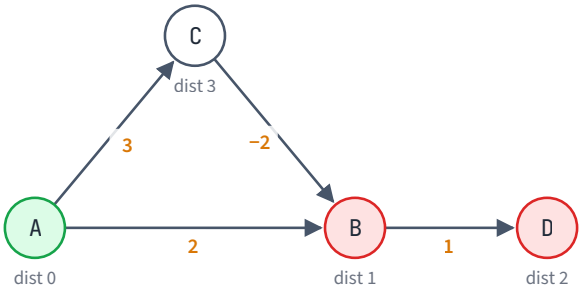


Figure 17.2 A negative edge. True distances from A are in the notes. A version of Dijkstra that settles each vertex once settles B at 2 before it sees the cheaper route A → C → B of weight 1, and then reports d[D] = 3.

In Figure 17.2 a textbook version that settles every vertex exactly once returns $d = [0, 2, 3, 3]$ for A, B, C, D, while the true distances are [0, 1, 3, 2]. The lazy template in Listing 17.2 happens to return the right answer here: when C lowers d[B] to 1, it pushes (1, B), and B is settled a second time. But Theorem 17.2 no longer holds. A vertex can be settled again and again (on graphs built for the purpose, exponentially often), and if the graph has a cycle of negative total weight the loop never ends. For negative weights use the Bellman–Ford algorithm of Section 17.3, which costs $O(VE)$.

17.2 Changing the cost of a path

The proof of Theorem 17.2 never used the fact that path weights are *sums*. It used only (P1) and (P2). Any cost that is built edge by edge and satisfies both can be minimised by the same loop, with `d[u] + e[1]` replaced by the new way of extending a cost.

Minimax paths

In LeetCode 1631 (Path With Minimum Effort) you walk through a grid of heights, and the effort of a route is the **largest** absolute height difference between consecutive cells. You want the route of least effort. This is a **minimax path**: minimise, over all paths, the maximum edge weight on the path.

THEOREM 17.3 Dijkstra for minimax paths

If all weights are ≥ 0 and the cost of a path is its largest edge weight (0 for the empty path), the loop of Listing 17.2 with `d[u] + e[1]` replaced by `Math.max(d[u], e[1])` computes the minimax cost from s to every vertex.

Proof. Extending a cost c by an edge of weight w gives $\max(c, w)$. (P1) holds because $\max(c, w) \geq c$. (P2) holds because if $x \leq y$ then $\max(x, w) \leq \max(y, w)$. Lemma 17.1 also carries over: every finite $d[v]$ is the cost of a walk, and cutting a cycle out of a walk removes edges, which cannot raise the maximum. The proof of Theorem 17.2 then goes through word for word. ∎

Listing 17.3 Minimax paths: Dijkstra with max in place of +

```java
// The cost of a path is its LARGEST edge weight (weights >= 0). Same loop as Dijkstra.
static long[] minimaxPaths(List<int[]>[] g, int src) {
    long[] d = new long[g.length];
    Arrays.fill(d, Long.MAX_VALUE);
    d[src] = 0;                                      // the empty path has no edges
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));
    pq.add(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[1];
        if (cur[0] != d[u]) continue;
        for (int[] e : g[u]) {
            long nd = Math.max(d[u], e[1]);          // the only changed line
            if (nd < d[e[0]]) {
                d[e[0]] = nd;
                pq.add(new long[]{nd, e[0]});
            }
        }
    }
    return d;
}
```

Notice what the proof did *not* need: that every part of an optimal path is itself optimal. For sums this is true, but for maxima it is false. In Figure 17.3 the best route has effort 2; its first step already has difference 2, so every cell on it has minimax cost 2 from the corner, and many other routes to those middle cells are just as good. The proof only needs (P1) and (P2), which is why it survives.

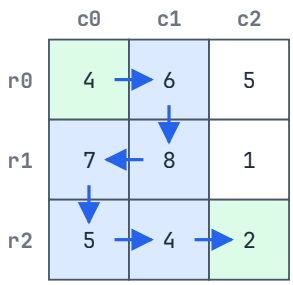


Figure 17.3 Heights in a 3 × 3 grid. The least-effort route from the top-left to the bottom-right corner (shaded) winds through 7 cells, and its largest step is 2. The route with the smallest total climb, 4 → 7 → 5 → 4 → 2, has total 8 but a step of 3.

For a grid, the vertices are the cells and each cell has up to four neighbours, so you do not need adjacency lists at all: give cell (r, c) the id $r \cdot C + c$ and compute the neighbours and step costs inside the loop. LeetCode 1631 is left for you to write in that form.

> **NOTE**
>
> Two other methods solve minimax problems, and both are worth knowing. **Binary search on the answer** (Chapter 6): "is there a route whose every step is $\le T$?" is a plain BFS that ignores larger steps, and the answer is monotone in T, so about 20 BFS runs find the least T for heights up to 10^6. **Kruskal's order** (Section 17.5): add the steps in increasing order of cost with union-find, and stop as soon as the two corners are connected; the cost of the last step added is the answer. Theorem 17.11 proves that this works.

Counting shortest paths

LeetCode 1976 (Number of Ways to Arrive at Destination) asks how many different shortest paths lead from vertex 0 to vertex $n - 1$, modulo $10^9 + 7$. Counting paths directly is hopeless; a grid-like graph can have exponentially many. The count can instead be carried along with the distance.

Call an edge (u, v) **tight** if $\text{dist}(s, u) + w(u, v) = \text{dist}(s, v)$. A path from s is a shortest path exactly when every edge on it is tight, and the last edge of a shortest path to v is a tight edge (u, v) preceded by a shortest path to u. So, writing $\text{ways}(v)$ for the number of shortest paths to v,

$$\text{ways}(s) = 1, \text{ and } \text{ways}(v) = \Sigma\, \text{ways}(u) \text{ over all tight edges } (u, v).$$

With **positive** weights every tight edge goes from a smaller distance to a strictly larger one, so the tight edges form a directed acyclic graph, and Dijkstra settles its vertices in increasing distance, which is a topological order of that graph. The sum can therefore be accumulated during Dijkstra: this is the DAG counting of Chapter 15, with Dijkstra supplying the order.

Listing 17.4 Counting shortest paths modulo a prime (positive weights)

```
static long[] countShortestPaths(List<int[]>[] g, int src, long mod) {
    int n = g.length;
    long[] d = new long[n], ways = new long[n];
    Arrays.fill(d, Long.MAX_VALUE);
    d[src] = 0;
    ways[src] = 1;
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));
    pq.add(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[1];
        if (cur[0] != d[u]) continue;               // u is settled, so ways[u] is final
        for (int[] e : g[u]) {
            int v = e[0];
            long nd = d[u] + e[1];
            if (nd < d[v]) {                        // strictly shorter: the old ways are useless
                d[v] = nd;
                ways[v] = ways[u];
                pq.add(new long[]{nd, v});
            } else if (nd == d[v]) {                // a tie: add the shortest paths through u
                ways[v] = (ways[v] + ways[u]) % mod;
            }
        }
    }
    return ways;
}
```

THEOREM 17.4 Correctness of the counting rule

If every weight is positive, then when Listing 17.4 settles a vertex u, ways[u] equals the number of shortest paths from s to u (modulo mod).

Proof. By induction in the order of settling. The source is settled first with ways[s] = 1, the empty path. Now consider a vertex $v \neq s$ being settled. Every tight edge (u, v) has $\text{dist}(s, u) = \text{dist}(s, v) - w(u, v) < \text{dist}(s, v)$, so by Theorem 17.2 u was settled earlier, and by the induction hypothesis ways[u] was already correct when u relaxed (u, v). At that moment $nd = \text{dist}(s, v)$.

Follow ways[v] over time. Before the first tight edge into v is relaxed, $d[v] > \text{dist}(s, v)$, and any value stored in ways[v] belongs to a longer route. The first tight relaxation finds $nd < d[v]$, so it **overwrites** ways[v] with ways[u]; every later tight relaxation finds $nd = d[v]$ and **adds** ways[u]. Non-tight edges relaxed after that find $nd > d[v]$ and change nothing. So when v is settled, ways[v] is the sum of ways[u] over all tight edges into v, which is the number of shortest paths to v by the recurrence above. ∎

EXAMPLE 17.2 Counting with ties

Take the undirected graph with edges 0–1 (1), 0–2 (1), 1–3 (1), 2–3 (1), 3–4 (2), 1–4 (3), 2–4 (5) and $s = 0$. Each row is one settled vertex; a dash in d means ∞.

```
settle  d[u]  ways[u]  what happens to the neighbours                d          ways
  0      0      1      1: set d=1, ways=1;  2: set d=1, ways=1       [0,1,1,-,-] [1,1,1,0,0]
  1      1      1      3: set d=2, ways=1;  4: set d=4, ways=1       [0,1,1,2,4] [1,1,1,1,1]
  2      1      1      3: tie, ways=1+1=2;  4: 1+5=6 > 4, nothing    [0,1,1,2,4] [1,1,1,2,1]
  3      2      2      4: tie, ways=1+2=3                            [0,1,1,2,4] [1,1,1,2,3]
  4      4      3      (no improvement)                              [0,1,1,2,4] [1,1,1,2,3]
```

Vertex 4 has three shortest paths of length 4: 0–1–4, 0–1–3–4 and 0–2–3–4. The row for vertex 1 shows why the tie rule must wait for settling: at that moment ways[4] = 1 counts only the route through 1–4, and the two routes through vertex 3 arrive later.

PITFALL

The counting rule needs **positive** weights. With a zero-weight edge, two vertices at the same distance can be connected by a tight edge, and the heap may settle the later one first. For the edges $s \to a$ (1), $s \to b$ (1), $a \to b$ (0), $b \to t$ (1) there are two shortest paths to t, but if the input lists $s \to b$ before $s \to a$, Listing 17.4 settles b before a and returns 1. LeetCode 1976 guarantees positive travel times. Its distances need `long`: a shortest route can pass up to 199 roads, each taking up to 10^9 minutes.

INFOSYS LENS

In the official Infosys sample paper, typical limits are $n \le 10^5$ with answers modulo $10^9 + 7$. The exam rules in the plan say: default to `long`, and take the modulus after every operation that can grow a count. In Listing 17.4 the only such operation is the addition in the tie branch.

17.3 State-expanded graphs

Many problems add a condition to the path: at most K stops, one discount coupon, a limited number of walls you may break. The vertex alone no longer tells you everything that matters about a partial route. Two routes that reach city v, one having used the coupon and one not, have different futures, so they must not be compared as if they were the same.

DEFINITION 17.3 State-expanded graph

A **state-expanded graph** has one vertex for every pair (v, x), where v is a vertex of the original graph and x is the extra information a route carries (stops used, coupon used or not, walls broken). There is an edge $(u, x) \to (v, y)$ when the original edge (u, v) can be taken by a route in state x and leaves it in state y.

If the extra information has X possible values, the new graph has $V \cdot X$ vertices and at most $E \cdot X$ edges per kind of move, and any shortest-path algorithm runs on it unchanged. The work is in choosing the state so that it captures *everything* the future depends on, and nothing more.

One coupon

Suppose you may halve the price of one flight on your route (rounding down). The state is "coupon unused" (0) or "coupon used" (1). Every flight $u \to v$ of price w gives three edges: $(u, 0) \to (v, 0)$ and $(u, 1) \to (v, 1)$ at price w, and $(u, 0) \to (v, 1)$ at price $\lfloor w/2 \rfloor$, the moment the coupon is spent. There is no edge from layer 1 back to layer 0.

Listing 17.5 A state-expanded graph: one edge at half price

```java
// Vertex v "coupon unused" keeps the id v; v "coupon used" gets the id v + n.
// Run Dijkstra on the result from s; the answer for t is min(d[t], d[t + n]).
@SuppressWarnings("unchecked")
static List<int[]>[] couponGraph(int n, int[][] edges) {    // directed edges {u, v, w}
    List<int[]>[] h = new List[2 * n];
    for (int i = 0; i < 2 * n; i++) h[i] = new ArrayList<>();
    for (int[] e : edges) {
        int u = e[0], v = e[1], w = e[2];
        h[u].add(new int[]{v, w});                  // unused -> unused: pay in full
        h[u + n].add(new int[]{v + n, w});          // used -> used: pay in full
        h[u].add(new int[]{v + n, w / 2});          // unused -> used: spend the coupon on this edge
    }
    return h;
}
```

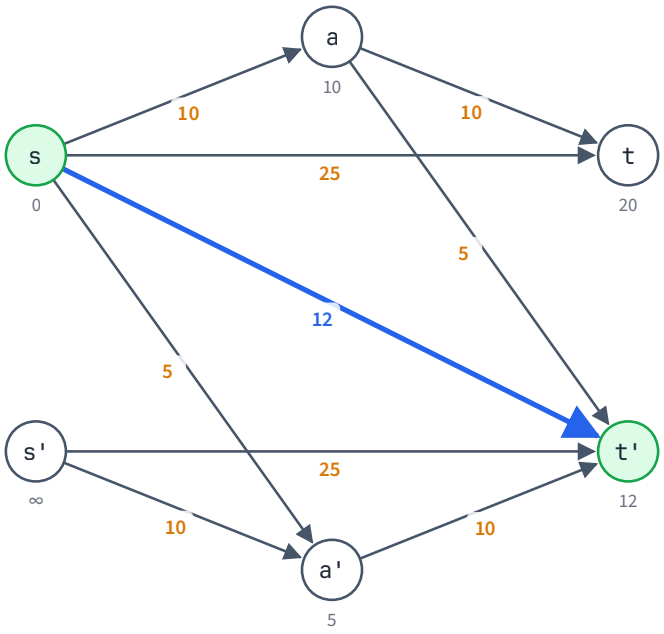


Figure 17.4 The coupon graph for flights s → a (10), a → t (10) and s → t (25). The top row is "coupon unused", the bottom row "coupon used"; the three edges from the top row down to the bottom row spend the coupon. Notes are Dijkstra's distances from s. Without the coupon the best price is 20; with it, the direct flight at 12 wins.

The coupon graph has $2V$ vertices and $3E$ edges, so Dijkstra on it costs $O((V + E) \log V)$, a constant factor more than without the coupon. The same construction handles "c coupons" with $c + 1$ layers.

At most K stops

LeetCode 787 (Cheapest Flights Within K Stops) limits the number of intermediate cities: at most K stops, so at most $K + 1$ flights. Plain Dijkstra on the cities is wrong here, because the cheapest way to reach a city may use too many flights, and Dijkstra then discards a dearer route that would have stayed within the limit. In Figure 17.5 the cheapest route from 0 to 3 is 0 → 1 → 2 → 3 with price 3, but it uses three flights. With $K = 1$ the answer is 6, and Dijkstra on cities never keeps that route, because it reaches 2 more cheaply through 1.

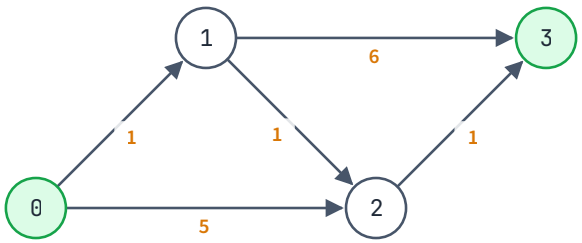


Figure 17.5 Flights with prices. The cheapest route 0 → 1 → 2 → 3 costs 3 but uses three flights; with at most two flights (one stop) the best is 0 → 2 → 3 at 6.

There are two correct ways to respect the limit. The first is a state-expanded graph whose state is the number of flights used: vertices (v, j) for $0 \le j \le K + 1$, edges $(u, j) \to (v, j + 1)$, and the answer is the smallest $d[(t, j)]$ over all j. The second needs no new graph at all. It computes, round by round, the cheapest price using at most i flights.

THEOREM 17.5 Bellman–Ford rounds

Let $b_i[v]$ be the least weight of a walk from s to v with at most i edges (∞ if there is none). Then $b_0[s] = 0$, $b_0[v] = \infty$ for $v \neq s$, and for $i \geq 1$

$b_i[v] = \min(\, b_{i-1}[v],$ *min over edges* (u, v) *of* $b_{i-1}[u] + w(u, v)\,)$.

Proof. A walk with at most i edges either has at most $i - 1$ edges, and is counted by $b_{i-1}[v]$, or has exactly i edges; then it is a walk with $i - 1$ edges to some u followed by the edge (u, v), and the cheapest such choice for a fixed last edge costs $b_{i-1}[u] + w(u, v)$. Conversely, each term on the right is the weight of a walk with at most i edges. So both sides are the same minimum. ∎

The recurrence reads **only** row $i - 1$. Computing row i in a copy of the array and relaxing every edge from the old values gives exactly b_i, at a cost of $O(E)$ per round. For Figure 17.5, relaxing the edges in the order 0→1, 1→2, 0→2, 2→3, 1→3:

```
round i   b_i[0..3]            meaning
   0      [0, inf, inf, inf]   no flights yet
   1      [0, 1, 5, inf]       one flight
   2      [0, 1, 2, 6]         two flights: K = 1 gives 6
   3      [0, 1, 2, 3]         three flights: K = 2 gives 3
```

If instead you update the array **in place**, a value improved earlier in the same round is used again later in that round. With the edge order above, the in-place round 1 already produces [0, 1, 2, 3]: the route 0 → 1 → 2 → 3 slips through in a single "round". That is harmless for ordinary shortest paths but wrong for a limit on the number of edges, which is why the copy matters.

NOTE

Without a limit, $V - 1$ rounds always suffice when there is no negative cycle, because a shortest path is simple and has at most $V - 1$ edges. This is the **Bellman–Ford algorithm**: $O(VE)$ time, correct with negative edges, and a V-th round that still improves something proves that a negative cycle is reachable. It is the right tool when Dijkstra's (P1) fails.

Free and paid moves: 0-1 BFS

When every edge costs 0 or 1, the heap can be replaced by a deque: Chapter 15's **0-1 BFS** pushes a vertex to the front after a 0-edge and to the back after a 1-edge. It is Dijkstra in disguise. Label each deque entry with the distance it was pushed with. Front to back, the labels never decrease and the last is at most the first plus 1: removing the front (label x) keeps this true, a 0-edge pushes label x at the front, and a 1-edge pushes $x + 1$ at the back, which is at most one more than any label left. So the deque always hands out a smallest label, just like the heap, and the proof of Theorem 17.2 applies. Each push follows a successful relaxation, so the cost is $O(V + E)$ with no logarithm.

LeetCode 2290 (Minimum Obstacle Removal to Reach Corner) is this situation: stepping into an empty cell costs 0, stepping into an obstacle costs 1 (you remove it), and the distance to the far corner is the number of obstacles removed. The state is still just the cell; the weights carry the information.

Table 17.1 Which shortest-path algorithm?

Weights	Extra condition	Algorithm	Time
all equal	none	BFS (Chapter 15)	$O(V + E)$
0 or 1	none	0-1 BFS	$O(V + E)$

Weights	Extra condition	Algorithm	Time
≥ 0	none, or a cost satisfying (P1) and (P2)	Dijkstra	$O((V+E)\log V)$
≥ 0	a small state (coupon, keys)	Dijkstra on $(v,$ state$)$	$O(X(V+E)\log(XV))$
any	at most K edges	K Bellman–Ford rounds with a copy	$O(KE)$
any, possibly negative	none	Bellman–Ford	$O(VE)$

17.4 Union-find

The second half of the chapter needs a different kind of structure. Imagine the edges of a graph arriving one at a time, and after each one you are asked "are a and b connected now?" or "how many components are there?". Running a BFS after every edge costs $O(V+E)$ per question. A structure that only ever *merges* groups can do much better.

DEFINITION 17.4 Disjoint-set union

A **union-find** structure maintains a partition of the elements $0..n-1$ into disjoint groups, starting with every element alone. It supports two operations: find(x) returns a representative of x's group, the same element for every member, and union(a, b) merges the groups of a and b.

Each group is stored as a rooted tree in an array `parent`: the root is the representative and has `parent[r] == r`, and every other element points to some element of its group nearer the root. find(x) follows `parent` pointers up to the root. union(a, b) finds both roots and, if they differ, makes one root the child of the other. Two elements are in the same group exactly when their finds return the same root.

How quickly find runs depends on the height of the trees, and two rules keep them low.

Union by size. Store at each root the number of elements in its tree, and always hang the root of the *smaller* tree under the root of the larger one.

Path halving. During find, make every second node on the path point to its grandparent: `parent[x] = parent[parent[x]]`, then step to that grandparent. A node's parent is only ever replaced by one of its own ancestors, so the groups do not change, but the path to the root roughly halves every time it is walked. (Full *path compression*, which points every node on the path straight at the root, needs a second pass or recursion; halving gets the same bound in one loop.)

Listing 17.6 Union-find with union by size and path halving, and Kruskal's algorithm (handbook template)

```
static final class DSU {
    final int[] parent, size;

    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];          // path halving: keeps trees flat
            x = parent[x];
        }
        return x;
    }
```

```
20
21      boolean union(int a, int b) {                   // false if already connected
22          a = find(a);
23          b = find(b);
24          if (a == b) return false;
25          if (size[a] < size[b]) {
26              int t = a;
27              a = b;
28              b = t;
29          }
30          parent[b] = a;                              // attach the smaller tree
31          size[a] += size[b];
32          return true;
33      }
34  }
35
36  // Kruskal: minimum spanning tree weight, -1 if the graph is not connected. edges = {u, v, w}
37  static long mst(int n, int[][] edges) {
38      Arrays.sort(edges, (x, y) -> Integer.compare(x[2], y[2]));
39      DSU dsu = new DSU(n);
40      long total = 0;
41      int used = 0;
42      for (int[] e : edges) {
43          if (dsu.union(e[0], e[1])) {
44              total += e[2];
45              used++;
46          }
47      }
48      return used == n - 1 ? total : -1;
49  }
```

The function `mst` at the bottom is Kruskal's algorithm, the subject of Section 17.5; it is printed here because the handbook keeps the two together. Note that `size[x]` is meaningful only when x is a root.

EXAMPLE 17.3 Union-find arrays by hand

Start with eight singletons and apply eight unions. On equal sizes the listing keeps a (the first argument's root) as the new root.

```
operation    result  parent                    size (only roots matter)
start                [0, 1, 2, 3, 4, 5, 6, 7]  [1, 1, 1, 1, 1, 1, 1, 1]
union(0,1)   true    [0, 0, 2, 3, 4, 5, 6, 7]  [2, 1, 1, 1, 1, 1, 1, 1]
union(2,3)   true    [0, 0, 2, 2, 4, 5, 6, 7]  [2, 1, 2, 1, 1, 1, 1, 1]
union(1,3)   true    [0, 0, 0, 2, 4, 5, 6, 7]  [4, 1, 2, 1, 1, 1, 1, 1]
union(4,5)   true    [0, 0, 0, 2, 4, 4, 6, 7]  [4, 1, 2, 1, 2, 1, 1, 1]
union(6,7)   true    [0, 0, 0, 2, 4, 4, 6, 6]  [4, 1, 2, 1, 2, 1, 2, 1]
union(5,7)   true    [0, 0, 0, 2, 4, 4, 4, 6]  [4, 1, 2, 1, 4, 1, 2, 1]
union(0,3)   false   [0, 0, 0, 0, 4, 4, 4, 6]  [4, 1, 2, 1, 4, 1, 2, 1]
union(3,7)   true    [0, 0, 0, 0, 0, 4, 4, 4]  [8, 1, 2, 1, 4, 1, 2, 1]
```

The rejected union(0, 3) still changed the array: find(3) walked 3 → 2 → 0 and halved the path, so `parent[3]` became 0. In the last union, find(7) moved 7 from 6 up to 4, and then the root 4 (size 4) was hung under the root 0 (size 4). Figure 17.6 shows the forest before and after the last two operations.

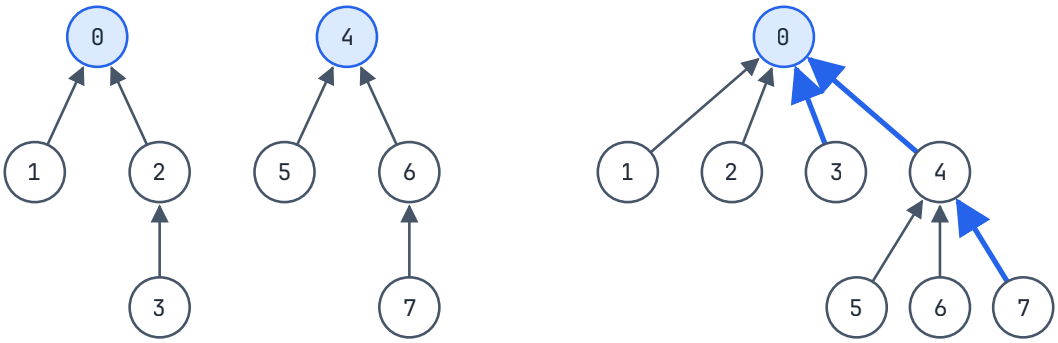


Figure 17.6 The union-find forest of Example 17.3. Left: after union(5,7). Right: after union(0,3) and union(3,7); path halving has lifted 3 and 7, and root 4 now hangs under root 0. Arrows point from a node to its parent.

Why the trees stay flat

LEMMA 17.6 Union by size gives logarithmic height

With union by size, with or without path halving, every element is at depth at most $\log_2 n$ in its tree. Hence find costs $O(\log n)$.

Proof. Path halving only replaces a parent by an ancestor, which never increases any depth. So depths grow only through union. The depth of an element x grows (by exactly 1) only when the root of x's tree is hung under another root. Union by size does that only when x's tree is no larger than the other tree, so the merged tree containing x is at least twice as large as x's old tree. Starting from size 1, a size that at least doubles at every step exceeds n after more than $\log_2 n$ steps, and no tree has more than n elements. So the depth of x is at most $\log_2 n$. ∎

With path halving on top, the bound becomes far better on average. The following result is the reason union-find is described as "almost O(1)".

THEOREM 17.7 Tarjan's bound (stated without proof)

With union by size (or by rank) and path compression or path halving, any sequence of m operations on n elements takes $O(m \cdot \alpha(n))$ time, where α is the inverse of the Ackermann function. α grows so slowly that $\alpha(n) \le 4$ for every n that could ever be stored in a computer.

The proof is long and not needed in practice. What you should remember is the consequence: for 10^5 or 10^6 operations, union-find behaves like an array access with a small constant, and the bound holds for every input, not just on average.

Two direct applications

Counting components. Start with n groups. Each union that returns `true` merges two groups into one, so after processing all edges the number of components is n minus the number of successful unions. This is the same answer as Chapter 15's BFS, but it works when edges arrive over time and you need the count after each one.

Detecting a cycle in an undirected graph. Process the edges one by one.

CLAIM 17.8 Union-find detects the first cycle

When the edges of an undirected graph are added one by one, union(u, v) returns `false` *exactly when the edge (u, v) closes a cycle with edges added before it.*

Proof. The groups are always the connected components of the edges added so far, since every edge merges the components of its two ends. union(u, v) returns `false` exactly when u and v are already in one component, that is, when the earlier edges already contain a path from u to v. That path together with the new edge is a cycle. Conversely, if the new edge lies on a cycle of the graph with it added, the rest of that cycle is a path from u to v made of earlier edges, so u and v were already connected. ∎

PITFALL

Two elements are connected when `find(a) == find(b)`, not when `parent[a] == parent[b]`: the parents are just some nodes on the way to the root. Also, keep `find` iterative as in Listing 17.6. A recursive find is fine with union by size (depth at most 16 for 10^5 elements, by Lemma 17.6) but overflows the stack if you ever drop the size rule and a chain of 10^5 forms.

EXAM TIP

Information about a whole group belongs at its root: its size, its sum, its minimum, the number of special vertices it contains. In `union`, after `parent[b] = a`, combine `b`'s fields into `a`'s, exactly as `size[a] += size[b]` does.

17.5 Minimum spanning trees

"Connect all the cities with roads of least total length" asks for a set of edges that links every vertex, at minimum total weight. The cheapest such set never contains a cycle (dropping any edge of a cycle keeps everything connected and does not increase the weight when weights are non-negative), so it is a tree.

DEFINITION 17.5 Spanning tree and cuts

A *spanning tree* of a connected undirected graph is a subset of its edges that forms a tree containing every vertex; it has $V - 1$ edges. A **minimum spanning tree** (MST) is a spanning tree of least total weight. A **cut** is a split of the vertices into a non-empty set S and its non-empty complement $V \setminus S$. An edge *crosses* the cut if it has one end in each part. A cut *respects* a set of edges A if no edge of A crosses it. A *light* edge of a cut is a crossing edge of least weight.

Both classical MST algorithms are greedy, and both are justified by one exchange argument.

THEOREM 17.9 The cut property

Let A be a set of edges contained in some MST, let $(S, V \setminus S)$ be a cut that respects A, and let e be a light edge of that cut. Then $A \cup \{e\}$ is also contained in some MST.

Proof. Let T be an MST that contains A. If $e \in T$ there is nothing to prove. Otherwise write $e = (u, v)$ with $u \in S$ and $v \notin S$. The tree T contains a unique path from u to v; it starts in S and ends outside S, so some edge f on it crosses the cut. Because the cut respects A, $f \notin A$. Removing f from T splits it into two trees, one containing u and the other v (they were joined only through the path, which used f). Adding e joins them

again, so $T' = T - f + e$ is a spanning tree. Since e is light and f crosses the cut, $w(e) \le w(f)$, so $w(T') \le w(T)$. Hence T' is also an MST, and it contains $A \cup \{e\}$. ∎

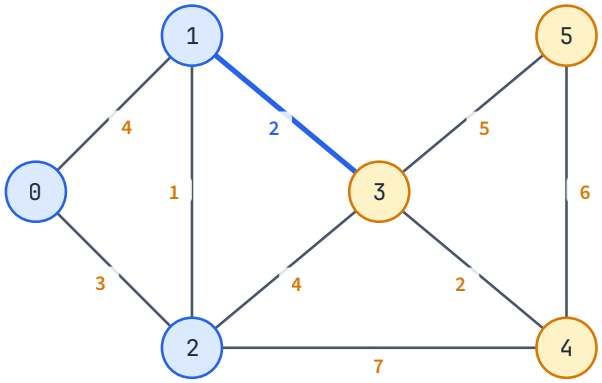


Figure 17.7 A cut with S = {0, 1, 2} (blue) and the other side {3, 4, 5} (amber). Three edges cross it, with weights 2, 4 and 7. The light edge 1–3 (thick) belongs to a minimum spanning tree.

Kruskal's algorithm

Kruskal's algorithm sorts the edges by weight and scans them once, keeping an edge whenever its ends are in different components of the edges kept so far. Union-find answers that question and performs the merge in one call: `mst` in Listing 17.6 keeps exactly the edges for which `union` returns `true`.

THEOREM 17.10 Correctness of Kruskal's algorithm

For a connected undirected graph, the edges kept by Kruskal's algorithm form a minimum spanning tree. If the graph is not connected, fewer than $V - 1$ edges are kept, and `mst` *returns* -1.

Proof. We show that the set A of kept edges is always contained in some MST. This holds at the start, when A is empty. Suppose Kruskal keeps $e = (u, v)$. At that moment u and v are in different groups; let S be the group of u. The groups are the components of A, so no edge of A crosses the cut $(S, V \setminus S)$. Now take any edge f that crosses this cut. If f came before e in the sorted order, it was examined earlier; groups only ever merge, so its ends, which are in different groups now, were in different groups then, and f would have been kept and its ends merged. That contradicts f crossing the cut. So every crossing edge comes after e in sorted order, $w(e) \le w(f)$, and e is light. By the cut property, $A \cup \{e\}$ is contained in some MST.

The kept edges never form a cycle (Claim 17.8). If the graph is connected and two groups remained at the end, some edge would join them; it was examined, its ends were in different groups then as well, and it would have been kept. So at the end A is a spanning tree contained in an MST, hence an MST. If the graph is disconnected, no forest can join its components, fewer than $V - 1$ edges are kept, and `used == n - 1` fails. ∎

Sorting costs $O(E \log E)$ and the E union calls cost $O(E\,\alpha(V))$, so Kruskal runs in $O(E \log E)$ time and $O(V + E)$ memory.

EXAMPLE 17.4 Kruskal by hand

Run `mst` on the graph of Figure 17.7. Java's object sort is stable, so equal weights keep their input order (0–1, 0–2, 1–2, 1–3, 2–3, 3–4, 4–5, 3–5, 2–4).

```
edge   weight  union  kept?  total
1-2      1     true   keep     1
1-3      2     true   keep     3
3-4      2     true   keep     5
0-2      3     true   keep     8
0-1      4     false  skip     8     (0 and 1 joined through 2)
2-3      4     false  skip     8
3-5      5     true   keep    13     (5 edges = V - 1: a spanning tree)
4-5      6     false  skip    13
2-4      7     false  skip    13
```

The minimum spanning tree has weight 13. Its edges are drawn thick in Figure 17.8. The edge 1–3 kept in the second row is the light edge of the cut in Figure 17.7, as the cut property predicts.

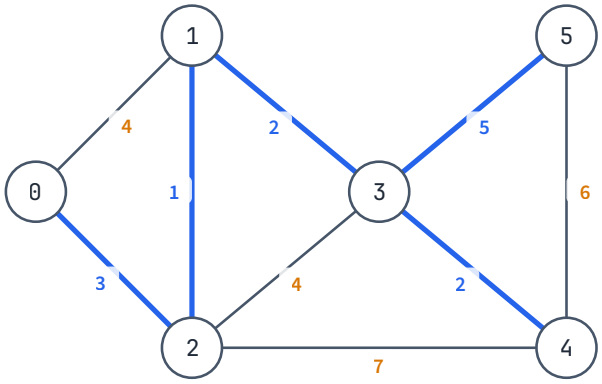


Figure 17.8 The minimum spanning tree found in Example 17.4 (thick edges, total weight 13).

A useful by-product connects back to minimax paths.

THEOREM 17.11 Kruskal's tree contains minimax paths

In a connected undirected graph, for any two vertices a and b, the path from a to b in the tree built by Kruskal's algorithm is a minimax path between them.

Proof. Let M be the minimax cost between a and b, achieved by a path P whose edges all weigh $\le M$. Look at the moment when Kruskal has examined every edge of weight $\le M$. Each edge of P has been examined, and after an edge is examined its two ends are in the same group (it was either kept, or skipped because they already were). So all vertices of P, in particular a and b, are in one group, joined by kept edges of weight $\le M$. That group is a subtree of the final tree, and a tree has only one path between two vertices, so the tree path from a to b uses only edges of weight $\le M$. No path can do better than M, so this one is minimax. ∎

Prim's algorithm and dense graphs

Prim's algorithm grows a single tree. Start with $S = \{0\}$. Repeatedly add the cheapest edge with exactly one end in S, and put its other end into S. Each chosen edge is a light edge of the cut $(S, V \setminus S)$, which respects the tree edges chosen so far, so by the cut property the edges always stay inside some MST, and after $V - 1$ steps they form one.

With a heap, Prim's algorithm looks like Dijkstra and costs $O((V + E) \log V)$. The version worth memorising is the one that uses no heap at all. Keep `best[v]`, the cheapest edge from v to the current tree. Each of the

V steps scans all vertices to find the smallest `best` outside the tree, adds it, and scans its edges to lower the other `best` values.

Listing 17.7 Prim's algorithm for a dense graph in $O(V^2)$

```
// Complete graph given by a weight function; here a matrix w[u][v] = w[v][u].
// best[v] = the cheapest edge from v to the tree built so far.
static long primDense(int[][] w) {
    int n = w.length;
    long[] best = new long[n];
    boolean[] inTree = new boolean[n];
    Arrays.fill(best, Long.MAX_VALUE);
    best[0] = 0;
    long total = 0;
    for (int step = 0; step < n; step++) {
        int u = -1;
        for (int v = 0; v < n; v++)                  // the cheapest vertex outside the tree
            if (!inTree[v] && (u == -1 || best[v] < best[u])) u = v;
        inTree[u] = true;
        total += best[u];
        for (int v = 0; v < n; v++)                  // u's edges may connect others more cheaply
            if (!inTree[v] && w[u][v] < best[v]) best[v] = w[u][v];
    }
    return total;
}
```

Both inner loops run V times per step, so the time is $O(V^2)$ whatever the number of edges, and the extra memory is $O(V)$. This is optimal for a **complete** graph, where $E = V(V-1)/2$ and merely reading the edges costs $\Theta(V^2)$.

LeetCode 1584 (Min Cost to Connect All Points) is that case: up to 1000 points, and every pair is an edge whose weight is the Manhattan distance. Kruskal must first build all 499,500 edges as `int[3]` objects (roughly 18 MB) and sort them, about 10^7 comparisons. Listing 17.7 needs no edge list: replace `w[u][v]` by the distance between points u and v, computed when needed, and the memory is three arrays of length 1000. On the machine used to test this book, Kruskal took about 200–480 ms on 1000 random points and Prim with a precomputed matrix 5–30 ms, with identical answers.

EXAMPLE 17.5 **Prim on four points**

Take the points (0, 0), (1, 3), (4, 1), (5, 5) with Manhattan distances. The distance matrix rows are [0, 4, 5, 10], [4, 0, 5, 6], [5, 5, 0, 5] and [10, 6, 5, 0].

```
step  add  edge cost  total   best[0..3] after updating   in tree
 0     0       0        0     [0, 4, 5, 10]               {0}
 1     1       4        4     [0, 4, 5, 6]                {0, 1}
 2     2       5        9     [0, 4, 5, 5]                {0, 1, 2}
 3     3       5       14     [0, 4, 5, 5]                {0, 1, 2, 3}
```

The tree uses the edges 0–1 (4), 0–2 (5) and 2–3 (5), total 14. At step 2 vertex 2 is equally close to 0 and to 1; either edge gives the same total.

Table 17.2 Kruskal or Prim?

Graph	Choose	Time	Memory
sparse, edges given as a list ($E \approx V$)	Kruskal	$O(E \log E)$	$O(V + E)$
complete or nearly complete, weights computed from coordinates	Prim, array version	$O(V^2)$	$O(V)$
adjacency lists, E between the two	either; Prim with a heap is $O(E \log V)$		

17.6 Offline processing in sorted order

Union-find cannot split a group once it is formed. That sounds like a limitation, but it suggests a strategy: if the questions only ever need groups to grow, arrange to ask them in an order in which the groups do only grow.

A typical question is "can a reach b using only vertices whose value is at most T?". For a single T this is one BFS on the allowed vertices. For 10^5 queries with different thresholds, one BFS each is too slow. But the allowed vertex set for T contains the allowed set for any smaller T. So sort the queries by T, sort the vertices by value, and sweep: before answering a query with threshold T, **activate** every vertex of value $\le T$ that is not yet active, and union each newly active vertex with its already active neighbours. A method that reads all the queries first and answers them in an order of its own choosing is called **offline**.

LEMMA 17.12 Activation keeps the components exact

After every vertex of value $\le T$ has been activated (and no other vertex), the union-find groups of the active vertices are exactly the connected components of the graph formed by the vertices of value $\le T$ and the edges between them.

Proof. Every union joins two active vertices that share an edge, so each group lies inside one component. Conversely, take any edge whose two ends both have value $\le T$. When the later of its two ends was activated, the other end was already active and is a neighbour, so the edge was unioned then. All edges of the component have therefore been unioned, and the whole component is one group. ∎

EXAMPLE 17.6 Switching vertices on by value

The tree in Figure 17.9 has values 4, 2, 3, 1, 2, 1, 3 on vertices 0..6. Activating in increasing value (ties in index order) gives:

```
value  activate  unions with active neighbours  groups of active vertices
  1        3      -                             {3}
  1        5      -                             {3} {5}
  2        1      1-3                           {1,3} {5}
  2        4      4-1                           {1,3,4} {5}
  3        2      2-5                           {1,3,4} {2,5}
  3        6      6-2                           {1,3,4} {2,5,6}
  4        0      0-1, 0-2                      {0,1,2,3,4,5,6}
```

So "can 3 reach 4 with every value ≤ 2?" is answered after row 4: yes, both are in {1, 3, 4}. "Can 3 reach 5 with every value ≤ 3?" is answered after row 6: no, they are in different groups, because the only route passes through vertex 0, whose value is 4.

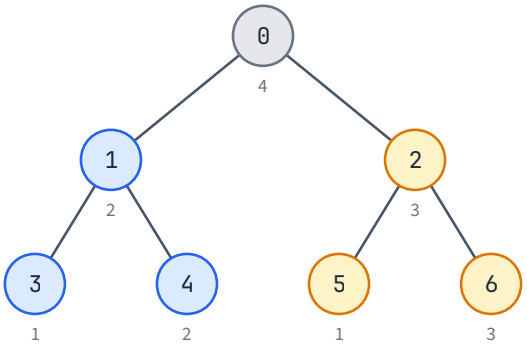


Figure 17.9 The tree of Example 17.6; the notes give each vertex's value. With threshold 3 the active vertices form two groups, {1, 3, 4} (blue) and {2, 5, 6} (amber); vertex 0 (value 4) is still off.

Sorting the vertices and the Q queries costs $O(V \log V + Q \log Q)$, and the sweep performs $O(V + E + Q)$ union-find operations, so the whole method is nearly linear after sorting. The same sweep works with edges instead of vertices ("using only edges of weight $\le T$"): sort the edges and union each one when the threshold passes its weight. That is Kruskal's loop with queries interleaved, and Theorem 17.11 is its special case for a single pair.

LeetCode 2421 (Number of Good Paths) uses this activation order. A good path's ends have the same value x and no vertex on it is larger, so its key observation is: two vertices of value x are the ends of a good path exactly when they lie in the same group right after every vertex of value $\le x$ has been activated. What to count at that moment is the part left for you.

Summary

- Dijkstra's algorithm settles vertices in increasing distance using a min-heap. With non-negative weights a settled distance is final; the proof needs only that extending a path never makes it cheaper (P1) and that extension respects order (P2).
- Java's `PriorityQueue` has no decrease-key, so push a new pair on every improvement and skip pairs whose distance differs from `d[u]`. The heap holds at most $E + 1$ pairs; the cost is $O((V + E) \log V)$. Use `long` distances and `Long.compare`.
- Negative edges violate (P1). Use Bellman–Ford, $O(VE)$, or K rounds of it with a copied array when the number of edges is limited.
- Replacing + by max gives minimax paths with the same loop. Counting shortest paths adds `ways[u]` on ties and resets it on strict improvements; it needs positive weights.
- Extra conditions become extra state: vertices (v, x) in a state-expanded graph. 0/1 weights need only a deque (0-1 BFS, $O(V + E)$).
- Union-find with union by size has depth $\le \log_2 n$; with path halving as well, operations cost $O(\alpha(n))$, effectively constant. A `false` from `union` means the edge closes a cycle.
- The cut property (a light crossing edge belongs to some MST) justifies both Kruskal (sort + union-find, $O(E \log E)$) and Prim (grow one tree). For complete graphs, Prim's array version runs in $O(V^2)$ with $O(V)$ memory.
- Questions with a threshold can be answered offline: sort them, activate vertices or edges in increasing order, and let union-find grow the groups.

Key terms

shortest-path distance p. 242

relaxation p. 242

state-expanded graph p. 249
disjoint-set union (union-find) p. 252
minimum spanning tree p. 255
cut p. 255
stale entry p. 242
minimax path p. 246
0-1 BFS p. 251
Kruskal's algorithm p. 256
Prim's algorithm p. 257
offline algorithm p. 259

Review questions

1. The heap in Listing 17.2 can hold up to $E + 1$ pairs. Why is each heap operation still $O(\log V)$ on a simple graph?
2. What happens to correctness, and to the running time, if the line `if (cur[0] != d[u]) continue;` is deleted from Listing 17.2?
3. Give a directed graph with three vertices on which the route with the fewest edges from vertex 0 to vertex 1 is not the shortest one, and give both answers.
4. Why does Listing 17.3 start with `d[src] = 0`? Give an example with a negative weight where this start value produces a wrong answer, and say what start value would fix it.
5. In Example 17.2, ways[4] equals 1 when vertex 1 is settled but 3 at the end. Explain why the final value can only be trusted once vertex 4 itself is settled.
6. For the flights of Figure 17.5 with $K = 1$, what does a Bellman–Ford loop that updates the array in place return after $K + 1 = 2$ rounds (edges relaxed in the order 0→1, 1→2, 0→2, 2→3, 1→3), and why is it wrong?
7. Give a sequence of unions on 8 elements that makes some element reach depth 3 under union by size. Why can no sequence reach depth 4?
8. Prove that if all edge weights are different, the minimum spanning tree is unique.
9. For LeetCode 1584 with 1000 points, compare the work and memory of Kruskal's algorithm and Listing 17.7.
10. Why must the offline method of Section 17.6 answer the queries in increasing order of T? What could go wrong if it answered them in input order?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 13 "Shortest paths" (Bellman–Ford and Dijkstra) and Chapter 15 "Spanning trees" (Kruskal, the union-find structure and Prim). Free at cses.fi/book.
- Algorithms for Competitive Programming: "Dijkstra - finding shortest paths from given vertex" (cp-algorithms.com/graph/dijkstra.html), "Disjoint Set Union" (cp-algorithms.com/data_structures/disjoint_set_union.html) and "Minimum Spanning Tree - Kruskal with Disjoint Set Union" (cp-algorithms.com/graph/mst_kruskal_with_dsu.html).
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapters *Single-Source Shortest Paths*, *Minimum Spanning Trees* and *Data Structures for Disjoint Sets*; the last contains the proof of the inverse-Ackermann bound.
- Workbook Chapter 17 has a Dijkstra table, union-find arrays and a Kruskal run to complete by hand before you start the problems.

Problems

These are the problems for Day 17. Work them in Workbook Chapter 17, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
17.P1	LC 743 · Network Delay Time	MEDIUM	CORE
17.P2	LC 1631 · Path With Minimum Effort	MEDIUM	CORE
17.P3	LC 1584 · Min Cost to Connect All Points	MEDIUM	CORE
17.P4	LC 1976 · Number of Ways to Arrive at Destination	MEDIUM	CORE
17.P5	LC 787 · Cheapest Flights Within K Stops	MEDIUM	BONUS
17.P6	LC 2290 · Minimum Obstacle Removal to Reach Corner	HARD	BONUS
17.P7	LC 2421 · Number of Good Paths	HARD	BONUS

CHAPTER 18 · DAY 18 OF 29 · TUESDAY 24 NOVEMBER 2026 · LEARNING DAY

18

Trees I: Recursion, Parent Arrays and Ancestors

LeetCode TreeNode recursion, general trees without recursion, and binary lifting.

Part III · Graphs, Trees and Range Queries
Handbook cards · Binary tree recursion (LeetCode TreeNode), General trees from a parent array (no recursion), LCA with binary lifting
Before this chapter · Chapter 15 (BFS, adjacency lists) and Chapter 17 (union-find, for comparison)
Practice · Workbook Chapter 18 · 8 problems · core ≈ 4.1 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- prove the basic facts about trees ($n - 1$ edges, unique paths) and use them to check an input;
- design a recursive function on LeetCode's `TreeNode` by choosing what each node returns, and keep paths that bend in a separate "best" holder;
- process a tree given as a parent array with 10^5 nodes without recursion, by walking its BFS order backwards or forwards;
- compute subtree sizes, depths, entry and exit times and longest paths iteratively, and prove the order of processing correct;
- build a binary-lifting table and answer k-th ancestor, LCA and distance queries in $O(\log n)$;
- turn path questions into root-path prefix values combined at the LCA, and subtree sizes into per-edge answers.

A tree is the simplest connected graph: no cycles, one route between any two vertices. That simplicity is what makes tree problems feel easy on paper and fail in Java. The recursive function you would write for a tree is short and correct, but on a tree that is a long chain of 10^5 vertices it calls itself 10^5 times deep, and the Java call stack runs out.

This chapter has three parts. Sections Section 18.1 and Section 18.2 cover the definitions and the recursion you use on LeetCode's binary trees, where it is safe. Section 18.3 shows how to do the same work on a general tree given as a parent array, which is how the Infosys sample paper delivers its trees, without any recursion at all. Sections Section 18.4 to Section 18.6 then answer questions about ancestors and paths: the lowest common ancestor by binary lifting, path counts from root prefixes, and answers built from subtree sizes. Chapter 19 builds dynamic programming on top of all of this.

18.1 Trees and how they are stored

DEFINITION 18.1 **Tree**

A **tree** is a connected undirected graph with no cycle. A *leaf* is a vertex of degree 1 (in a tree with at least two vertices).

Three facts about trees are used constantly: a tree with n vertices has $n - 1$ edges, any two vertices are joined by exactly one path, and either property, together with connectivity, identifies a tree. All three follow from one small lemma.

LEMMA 18.1 Two leaves

Every tree with $n \geq 2$ vertices has at least two leaves.

Proof. Take a longest simple path $v_0, v_1, \ldots, v_k$ in the tree; $k \geq 1$ because the tree is connected and has an edge. Suppose v_0 had a neighbour x other than v_1. If x is not on the path, then $x, v_0, \ldots, v_k$ is a longer simple path. If $x = v_i$ with $i \geq 2$, then $v_0, v_1, \ldots, v_i, v_0$ is a cycle. Both are impossible, so v_0 has degree 1. The same argument applies to v_k, and $v_0 \neq v_k$. ∎

THEOREM 18.2 Trees have n − 1 edges

A tree with n vertices has exactly $n - 1$ edges.

Proof. By induction on n. A tree with one vertex has no edges. For $n \geq 2$, remove a leaf ℓ and its only edge. The rest has no cycle, and it is still connected: a simple path between two other vertices never passes through ℓ, because passing through a vertex uses two of its neighbours and ℓ has only one. So the rest is a tree with $n - 1$ vertices and, by induction, $n - 2$ edges. Adding ℓ back gives $n - 1$. ∎

THEOREM 18.3 Unique paths

In a tree there is exactly one simple path between any two vertices a and b. Conversely, a connected graph with n vertices and $n - 1$ edges is a tree.

Proof. A path exists because the tree is connected. Suppose P and Q are different simple paths from a to b. Follow both from a; let x be the last vertex before they first differ. From x, follow P until it first reaches a vertex y that lies on Q after x; such a y exists because both paths end at b. The piece of P from x to y meets Q only at x and y, and it is not the same as Q's piece from x to y, because the two paths leave x along different edges. Together the two pieces form a cycle, which a tree does not have.

For the converse, suppose a connected graph has n vertices, $n - 1$ edges and a cycle. Deleting one edge of the cycle keeps the graph connected (the rest of the cycle replaces it). Repeat until no cycle is left. The result is a tree with n vertices and fewer than $n - 1$ edges, contradicting Theorem 18.2. ∎

These facts are practical. An edge list with $n - 1$ edges that connects all n vertices is guaranteed to be a tree, so "the path from u to v" is well defined and every algorithm in this chapter applies.

Rooted trees

Most tree algorithms first pick a vertex as the **root**. Then every other vertex v has a **parent**: the next vertex on the unique path from v to the root.

DEFINITION 18.2 Rooted-tree vocabulary

In a tree rooted at r: the **ancestors** of v are the vertices on the path from v to r, including v itself; v is a *descendant* of each of them. The *children* of v are the vertices whose parent is v. The **depth** of v is the number of edges from v to the root. The **subtree** of v consists of v and all its descendants. The *height* of v is the number of edges on the longest downward path from v to a leaf of its subtree.

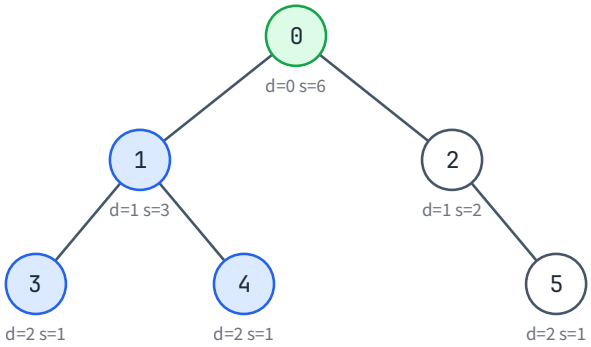


Figure 18.1 The tree of the handbook card, rooted at 0, as the parent array par = [−1, 0, 0, 1, 1, 2]. Notes give depth d and subtree size s. The subtree of 1 (blue) is {1, 3, 4}.

Four ways a tree arrives

Table 18.1 Tree representations and where you meet them

Representation	What you are given	Typical source	Children of v
`TreeNode` objects	a root with `left` and `right` pointers	LeetCode binary-tree problems	`node.left`, `node.right`
parent array	`par[v]` for every v, −1 (or 0 when 1-indexed) at the root	Infosys sample paper, LeetCode 2246	build children lists first
children lists	`List<Integer>[] kids` or `head` / `next` arrays	built by you from a parent array	iterate the list
edge list	$n-1$ pairs $\{u, v\}$ with no direction	LeetCode 834, 1519, 2477	root it by BFS first

A parent array is compact (one `int` per vertex) but answers only "who is above me?". Almost every algorithm needs to go down as well, so the first step is to build children lists. `List<Integer>[]` is easy to write; two `int` arrays, `head[u]` (first child of u) and `next[c]` (next sibling of c), do the same job without creating 10^5 list objects and boxed integers. An edge list does not say which end is the parent, so you choose a root and run a BFS, recording where each vertex was reached from. Section 18.3 gives the code for all of these.

18.2 Recursion on binary trees

On LeetCode a binary tree arrives as `TreeNode` objects, and recursion is the natural tool. A recursive function on a tree is correct by **structural induction**: if it returns the right value for the empty tree (`null`), and it returns the right value for a node whenever the recursive calls returned the right values for its two subtrees, then it is right for every tree. So designing the function comes down to one question.

> **KEY IDEA**
>
> What single fact does a node's parent need to know about the node's subtree? That fact is the return value. Everything else the problem asks for goes into a separate holder that every call can update.

Height and diameter

For the height, the answer to the question is the height itself: a node's height is one more than the larger of its children's heights. The handbook template counts **nodes** on the longest downward path, so that `null` returns 0 and a leaf returns 1; in edges this is one more than the height of Definition 18.2.

The **diameter** of a tree, asked for in LeetCode 543 (Diameter of Binary Tree), is the number of edges on its longest path. A parent cannot compute the diameter of its subtree from the children's diameters alone, because the longest path may pass through the parent. The fix is to notice where a path can bend.

THEOREM 18.4 Every path bends at one node

Let $h(x)$ be the number of nodes on the longest downward path from x, with $h(\texttt{null}) = 0$. Every simple path in a rooted tree has a unique highest node v, and the path consists of a downward path from v into the left subtree and one into the right subtree, either possibly empty. Consequently the diameter is the maximum over all nodes v of $h(v.\text{left}) + h(v.\text{right})$.

Proof. Along a path, consecutive vertices are parent and child, so the depth changes by exactly 1 at each step. A simple path cannot go down from x to a child c and then up again, because the only vertex above c is x. So the path first goes up and then only goes down, and the turning point is its unique highest node v. From v it can enter each child's subtree at most once, since entering a subtree twice would pass through that child twice. A binary node has two children, so the path is a downward chain into the left subtree joined at v to a downward chain into the right subtree. The left chain has at most $h(v.\text{left})$ edges (one edge from v to each of its nodes below v), and similarly on the right, with equality for the longest chains. So the longest path with highest node v has $h(v.\text{left}) + h(v.\text{right})$ edges, and the diameter is the largest of these over all v. ∎

The function therefore returns the height, which the parent needs, and records $l + r$ at every node in a holder `best`, which the parent does not need. A one-element array is the shortest way to give every recursive call write access to the same variable.

Listing 18.1 LeetCode's TreeNode; diameter and maximum path sum (handbook template)

```
static final class TreeNode {                       // LeetCode's binary tree node
    int val;
    TreeNode left, right;

    TreeNode(int val) { this.val = val; }
}

// Pattern: a function returns one fact about a subtree; a holder array collects the answer.
// Diameter (LeetCode 543): longest path in edges between any two nodes.
static int diameter(TreeNode root) {
    int[] best = {0};
    height(root, best);
    return best[0];
}

static int height(TreeNode node, int[] best) {     // nodes on the longest downward path
    if (node == null) return 0;
    int l = height(node.left, best), r = height(node.right, best);
    best[0] = Math.max(best[0], l + r);              // the path that bends at this node
    return 1 + Math.max(l, r);
}

// Maximum path sum (LeetCode 124): same shape, negative branches are dropped
static int maxPathSum(TreeNode root) {
    int[] best = {Integer.MIN_VALUE};
    gain(root, best);
    return best[0];
}

static int gain(TreeNode node, int[] best) {
    if (node == null) return 0;
    int l = Math.max(0, gain(node.left, best)), r = Math.max(0, gain(node.right, best));
```

```
    best[0] = Math.max(best[0], node.val + l + r);
    return node.val + Math.max(l, r);                // a parent can use only one branch
}
```

EXAMPLE 18.1 A diameter that avoids the root

In Figure 18.2 the calls finish in post-order: children before their parent. Each row shows the two children's heights l and r, the bend $l + r$ recorded in `best`, and the returned height.

node	l	r	bend l+r	returns 1+max(l,r)	best
8	0	0	0	1	0
6	1	0	1	2	1
4	2	0	2	3	2
9	0	0	0	1	2
7	0	1	1	2	2
5	0	2	2	3	2
2	3	3	6	4	6
3	0	0	0	1	6
1	4	1	5	5	6

The diameter is 6, the path 8–6–4–2–5–7–9, which bends at node 2. The root only offers 4 + 1 = 5. A solution that returns "the diameter through the root" would answer 5.

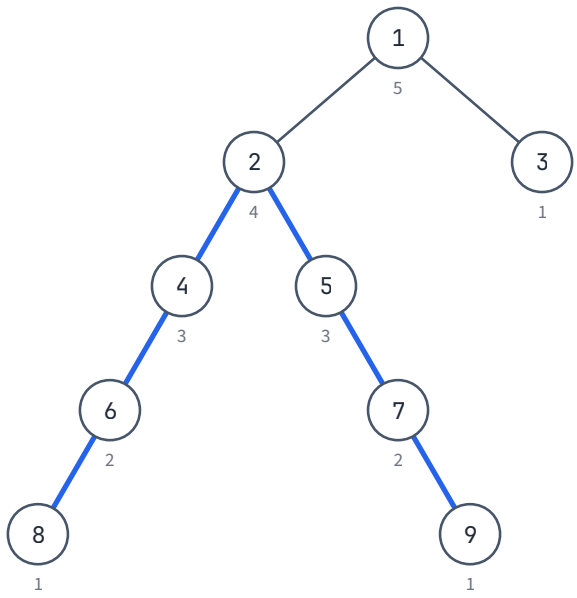


Figure 18.2 A binary tree whose longest path (thick, 6 edges) bends at node 2, not at the root. Notes give h, the number of nodes on the longest downward path from each node.

Maximum path sum with negative values

LeetCode 124 (Binary Tree Maximum Path Sum) asks for the largest sum of node values on any non-empty path, and values may be negative. Theorem 18.4 still says where a path bends, but now a chain into a child's subtree is optional: taking it is worth it only if its best sum is positive.

Let $g(x)$, the **gain** of x, be the largest sum of a downward path that starts at x and contains at least x.

THEOREM 18.5 Gains and the best path

g(x) = x.val + max(0, g(x.left), g(x.right)), where a missing child contributes 0. The best path whose highest node is v has sum v.val + max(0, g(v.left)) + max(0, g(v.right)), and `maxPathSum` *returns the largest of these over all v.*

Proof. A downward path from x is x alone, or x followed by a downward path from one child. The best continuation through a child c is worth $g(c)$, and it should be taken only if it adds something, which gives the formula for $g(x)$ (by induction on the height of x, the recursive calls return the correct gains). By Theorem 18.4, a path with highest node v is v plus an optional downward chain starting at each child, and the two choices are independent, so each child's chain is added exactly when its gain is positive. Every nonempty path has a highest node, so the maximum over v is the answer. ∎

The listing computes `l` and `r` as `Math.max(0, gain(...))`, updates `best` with the bend `node.val + l + r`, and returns `node.val + Math.max(l, r)`: the parent may continue into only one of the two branches.

EXAMPLE 18.2 Dropping negative branches

For the tree of Figure 18.3 the calls return:

```
node   gain(left)  gain(right)  l  r  bend val+l+r  returns val+max(l,r)  best
  5        0           0        0  0       5              5               5
 -2        0           0        0  0      -2             -2               5
  6        5          -2        5  0      11             11              11
 -1        0           0        0  0      -1             -1              11
  3        0           0        0  0       3              3              11
  7       -1           3        0  3      10             10              11
-20       11          10       11 10       1             -9              11
```

The answer is 11, the path 5–6. Node 6 drops its −2 child; node 7 drops its −1 child; and the bend at the root, −20 + 11 + 10 = 1, loses to the bend at 6.

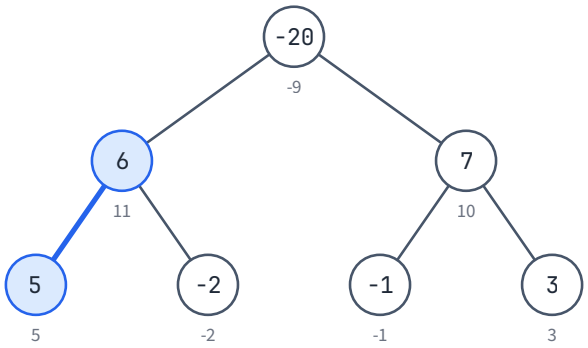


Figure 18.3 Values on a binary tree, with each node's gain g in the notes. The best path (thick) is 5–6 with sum 11; passing through the root would cost −20.

PITFALL

`best` must start at `Integer.MIN_VALUE`, not 0. If every value is negative, the answer is the largest single value (a path must contain at least one node). For the tree with root −3 and one child −1, `maxPathSum` returns −1; starting `best` at 0 would return 0, a path that does not exist.

Lowest common ancestor

DEFINITION 18.3 Lowest common ancestor

The **lowest common ancestor** (LCA) of p and q in a rooted tree is the deepest vertex that is an ancestor of both. Because a vertex is its own ancestor, if q lies in the subtree of p, the LCA is p.

LeetCode 236 (Lowest Common Ancestor of a Binary Tree) gives two nodes of a binary tree and asks for their LCA. The recursive rule is three lines, and its correctness rests on describing precisely what a call returns for *every* subtree, not only for the whole tree.

Listing 18.2 Lowest common ancestor and level order on TreeNode (handbook template)

```
static final class TreeNode { int val; TreeNode left, right; TreeNode(int val) { this.val = val; } }

// Lowest common ancestor (LeetCode 236)
static TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode l = lca(root.left, p, q), r = lca(root.right, p, q);
    if (l != null && r != null) return root;          // p and q are on different sides
    return l != null ? l : r;
}

// Level-order traversal (LeetCode 102): BFS, one level per outer loop
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> levels = new ArrayList<>();
    if (root == null) return levels;
    ArrayDeque<TreeNode> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int k = q.size(); k > 0; k--) {          // exactly the nodes of this level
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.add(node.left);
            if (node.right != null) q.add(node.right);
        }
        levels.add(level);
    }
    return levels;
}
```

THEOREM 18.6 What the recursive LCA returns

For every node x (or `null`), `lca(x, p, q)` returns the LCA of p and q if both lie in the subtree of x; returns whichever of p, q lies in the subtree of x if exactly one does; and returns `null` if neither does. In particular, called on the root of a tree containing both, it returns their LCA.

Proof. By induction on the height of x. For `null` the subtree is empty and the call returns `null`. If x is p (the case q is symmetric), the call returns p. That is correct in both possible cases: if q is in the subtree of p, the LCA is p; if not, exactly one of them, p, is in the subtree.

Otherwise x is neither, and by induction the calls on the children return values described by the theorem. If p and q are in different children's subtrees, each side returns the one it contains, both are non-null, and x is returned; x is a common ancestor, and no child of x is, so x is the LCA. If both are in the same child's subtree, that side returns their LCA and the other side returns `null`, so the LCA is passed up. If exactly one is in the subtree of x, its side returns it and the other side `null`, so it is passed up. If neither is, both sides return `null`, and so does x. ∎

On the tree of Figure 18.2 with $p = 8$ and $q = 7$, the call at 8 returns 8, which 6 and 4 pass up unchanged; on the other side 7 returns itself and 5 passes it up; node 2 receives 8 from the left and 7 from the right and returns itself; node 3 returns `null`; and the root passes 2 up. The LCA is 2.

Level order

The `levelOrder` function in Listing 18.2 is BFS from the root with one extra idea: the size of the queue, read *before* the inner loop starts, is the number of nodes in the current level.

CLAIM 18.7 One level per outer iteration

At the start of the d-th iteration of the outer loop (counting from 0), the queue holds exactly the nodes of depth d, from left to right.

Proof. For $d = 0$ the queue holds only the root. If it holds the depth-d nodes from left to right, the inner loop removes exactly `q.size()` of them, that is, all of them and nothing else, and appends each one's left child and then right child. The nodes of depth $d + 1$ are exactly the children of the depth-d nodes, and listing the children of a left-to-right row in order lists them from left to right. ∎

For the tree of Figure 18.2 (values equal to the labels) the result is [[1], [2, 3], [4, 5], [6, 7], [8, 9]].

JAVA NOTE

Recursion depth is the height of the tree. LeetCode's tests for these problems pass with the recursive templates. A general tree from a parent array is different: it can be a chain. On the JDK used to test this book, the recursive `height` above, run on a chain built from `TreeNode`s with default JVM settings, worked at depth $2 \cdot 10^4$ and threw `StackOverflowError` at depth $5 \cdot 10^4$. The exact limit depends on the JVM and the frame size, so never rely on it for 10^5 nodes.

18.3 General trees without recursion

INFOSYS LENS

In the official sample paper the trees arrive as parent arrays with up to 10^5 nodes, and the plan's Lesson 4 is blunt: a recursive DFS on a chain of that length crashes with `StackOverflowError`. The test data only has to contain one long chain for a recursive solution to score nothing on that file. Everything in this section runs in loops over arrays, so the depth of the tree does not matter.

The replacement for recursion is an **order** in which to visit the vertices. Recursion does two kinds of work: on the way down it passes information from a parent to its children (depths, prefix sums), and on the way back up it combines the children's results into the parent's (sizes, heights, DP values). The breadth-first order supports both.

DEFINITION 18.4 BFS order

The **BFS order** of a rooted tree lists the root, then all vertices of depth 1, then all of depth 2, and so on, as produced by a breadth-first search from the root that appends each vertex's children when the vertex is removed from the queue.

THEOREM 18.8 Parents first, children first backwards

*In the BFS order, every vertex other than the root appears after its parent. Consequently, a loop that walks the order **backwards** reaches every vertex after all of its descendants, and a loop that walks it **forwards** reaches every vertex after all of its ancestors.*

Proof. A vertex c is appended to the order only while its parent u is being processed, and u is processed only after it was itself appended, so u occupies an earlier position. Every vertex is appended: by induction on depth, a vertex of depth $d + 1$ is appended when its parent, of depth d, is processed. For the consequences, apply the first statement repeatedly along the path from a descendant up to the vertex: positions strictly decrease going up, so every ancestor of v is before v and every descendant after it. ∎

The handbook template builds children lists from the parent array, produces the BFS order with an array used as a queue, and computes subtree sizes with one backward loop.

Listing 18.3 Children lists, BFS order and subtree sizes from a parent array (handbook template)

```java
// General tree: nodes 0..n-1, par[root] = -1 (LeetCode style).
// Infosys input is often 1-indexed with par[root] = 0: subtract 1 from every value first.
@SuppressWarnings("unchecked")
static List<Integer>[] childrenOf(int[] par) {
    List<Integer>[] kids = new List[par.length];
    for (int v = 0; v < par.length; v++) kids[v] = new ArrayList<>();
    for (int v = 0; v < par.length; v++) if (par[v] != -1) kids[par[v]].add(v);
    return kids;
}

// BFS order: every parent comes before its children. No recursion, so no stack overflow.
static int[] bfsOrder(int[] par) {
    List<Integer>[] kids = childrenOf(par);
    int n = par.length;
    int[] order = new int[n];
    int head = 0, tail = 0;
    for (int v = 0; v < n; v++) if (par[v] == -1) order[tail++] = v;   // the root
    while (head < tail) {
        int u = order[head++];
        for (int c : kids[u]) order[tail++] = c;
    }
    return order;
}

// "Post-order" without recursion: walk the BFS order BACKWARDS (children before parents)
static int[] subtreeSizes(int[] par) {
    int[] order = bfsOrder(par), size = new int[par.length];
    Arrays.fill(size, 1);
    for (int i = order.length - 1; i > 0; i--) {        // order[0] is the root
        int v = order[i];
        size[par[v]] += size[v];
    }
    return size;
}
```

The size loop is correct because of Theorem 18.8: when vertex v is reached and adds `size[v]` to its parent, all of v's children were reached earlier and have already added their sizes to `size[v]`, so `size[v]` $= 1 + \Sigma$ size[c] is final. The order array doubles as the BFS queue: `head` is the front and `tail` the back, and since each vertex is appended exactly once, an array of length n is enough. Everything costs $O(n)$ time and memory.

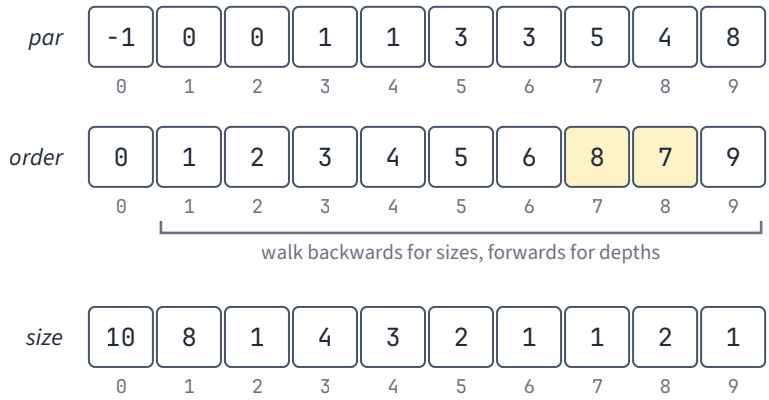


Figure 18.4 A parent array whose BFS order is not the index order (vertex 8 comes before 7), with the subtree sizes computed by the backward loop.

PITFALL

Walking the vertex numbers $n-1, \ldots, 1, 0$ instead of the BFS order is correct only if every parent has a smaller number than its children. Some inputs happen to have that property, but nothing in a parent array guarantees it. If vertex 2 is the parent of vertex 1, the index loop reaches 2 first and adds `size[2]` to its parent before vertex 1 has added its own size to `size[2]`. Use the BFS order every time and the question never arises.

Arrays instead of lists, and pushing values down

For 10^5 vertices, `List<Integer>[]` creates 10^5 list objects and up to 10^5 boxed integers. Two `int` arrays describe the same children lists: `head[u]` is the first child of u (-1 if none) and `next[c]` is the next sibling of c. Inserting the children in decreasing order of number makes each list come out in increasing order, so the BFS order below is identical to the template's.

Listing 18.4 BFS order with head/next arrays, and depths in one forward pass

```
// Children without List objects: head[u] = first child of u, next[c] = the next sibling of c.
static int[] bfsOrderArrays(int[] par) {
    int n = par.length, root = -1;
    int[] head = new int[n], next = new int[n], order = new int[n];
    Arrays.fill(head, -1);
    for (int v = n - 1; v >= 0; v--) {                  // backwards: siblings in increasing order
        if (par[v] == -1) root = v;
        else {
            next[v] = head[par[v]];
            head[par[v]] = v;
        }
    }
    int qh = 0, qt = 0;
    order[qt++] = root;
    while (qh < qt) {
        int u = order[qh++];
        for (int c = head[u]; c != -1; c = next[c]) order[qt++] = c;
    }
    return order;
}

// Walking the order FORWARDS pushes information from parents to children.
static int[] depths(int[] par, int[] order) {
    int[] depth = new int[par.length];
    for (int v : order) depth[v] = par[v] == -1 ? 0 : depth[par[v]] + 1;
    return depth;
}
```

The `depths` loop is the forward half of Theorem 18.8: when v is reached, its parent's depth is already final. Any quantity defined from the parent's value (depth, the sum of values on the root path, the binary-lifting rows of Section 18.4) is computed the same way.

Rooting an edge list

When the tree arrives as an undirected edge list, choose a root and let a BFS discover the parents. The value −2 marks "not reached yet", so the root's −1 is never mistaken for a missing parent.

Listing 18.5 Rooting an edge list: a parent array by BFS

```java
// Edge list (LeetCode style) -> parent array rooted at `root`; par[root] = -1.
@SuppressWarnings("unchecked")
static int[] parentsFromEdges(int n, int[][] edges, int root) {
    List<Integer>[] g = new List[n];
    for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
    for (int[] e : edges) {
        g[e[0]].add(e[1]);
        g[e[1]].add(e[0]);
    }
    int[] par = new int[n];
    Arrays.fill(par, -2);                               // -2 = not visited yet
    par[root] = -1;
    ArrayDeque<Integer> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : g[u]) {
            if (par[v] != -2) continue;
            par[v] = u;
            q.add(v);
        }
    }
    return par;
}
```

Each vertex receives as parent the neighbour from which BFS first reached it, which is the neighbour one step closer to the root, exactly the parent of Definition 18.2.

Entry and exit times

Some questions need a depth-first order instead: "is u an ancestor of v?" for many pairs, or, in Chapter 20, "sum over the subtree of v" while values change. An explicit stack replaces the call stack. Each stack entry is a vertex, and `cur[v]` remembers which child of v to visit next, which is exactly the information a recursive call keeps in its local variables.

DEFINITION 18.5 **Entry and exit times**

In a depth-first traversal from the root, the **entry time** tin[v] is the number of vertices visited before v, and tout[v] is the largest entry time of any vertex in the subtree of v.

Listing 18.6 Iterative DFS with entry and exit times

```java
// tin[v] = time v is first visited; tout[v] = the largest tin inside v's subtree.
// u is an ancestor of v (or v itself) exactly when tin[u] <= tin[v] <= tout[u].
static int[][] entryExitTimes(int[] par) {
    int n = par.length, root = -1, timer = 0, top = 0;
    int[] head = new int[n], next = new int[n];
    Arrays.fill(head, -1);
    for (int v = n - 1; v >= 0; v--) {
        if (par[v] == -1) root = v;
        else {
```

```
            next[v] = head[par[v]];
            head[par[v]] = v;
        }
    }
    int[] tin = new int[n], tout = new int[n], stack = new int[n];
    int[] cur = head.clone();                          // cur[v] = the next child of v to visit
    stack[top++] = root;
    tin[root] = timer++;
    while (top > 0) {
        int v = stack[top - 1];
        if (cur[v] != -1) {                            // go down to the next child
            int c = cur[v];
            cur[v] = next[c];
            tin[c] = timer++;
            stack[top++] = c;
        } else {                                       // every child is finished
            tout[v] = timer - 1;
            top--;
        }
    }
    return new int[][]{tin, tout};
}
```

THEOREM 18.9 A subtree is an interval of entry times

For every vertex v, the vertices of the subtree of v receive exactly the entry times tin[v], tin[v] + 1, ..., tout[v]. Hence u is an ancestor of w exactly when tin[u] ≤ tin[w] ≤ tout[u].

Proof. The stack always holds the path from the root to its top vertex, because every push puts a child of the current top. Consider the time between the push of v and its pop. Every vertex pushed in that time is pushed while v is still on the stack, so it is a child of a vertex on the stack at or above v: a descendant of v. Conversely, v is popped only when `cur[v]` $= -1$, that is, after all its children have been pushed, and each child c is popped (above v) only after all of c's children were pushed; by induction on height, every descendant of v is pushed before v is popped. Each vertex is pushed once, and the timer advances by one per push. So the subtree of v receives consecutive entry times starting at tin[v], and the last one is `timer - 1` at the moment v is popped, which the code stores in tout[v]. The ancestor test follows: w is in the subtree of u exactly when its entry time lies in u's interval. ∎

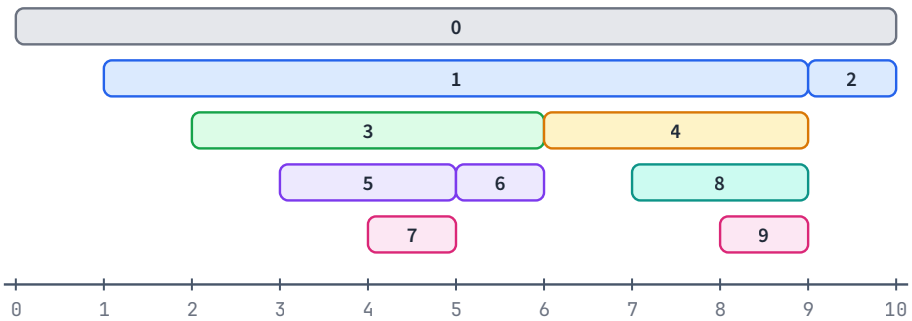


Figure 18.5 Entry and exit times for the tree of Figure 18.4. The bar labelled v covers the unit cells tin[v] to tout[v], the entry times of v's subtree; a bar lies inside another exactly when its vertex is a descendant.

Worked example: the longest path in a general tree

Theorem 18.4 carries over to vertices with any number of children: a path still has a unique highest vertex v, and from v it can enter at most two children's subtrees. So the longest path with highest vertex v uses the **two longest chains** that start at different children. Writing down[v] for the number of edges on the longest

downward path from v, a child c offers a chain of length down[c] + 1 measured from v, and the best bend at v is the sum of the two largest offers (0 for each missing one).

Listing 18.7 Longest path in a general tree: the two longest child chains

```
// down[v] = edges on the longest downward path from v. The longest path bends at its
// highest node v and uses v's two longest child chains. Children are finished first.
static int longestPath(int[] par, int[] order) {
    int n = par.length, best = 0;
    int[] down = new int[n], top1 = new int[n], top2 = new int[n];   // two best (down[c] + 1)
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i];
        down[v] = top1[v];
        best = Math.max(best, top1[v] + top2[v]);         // the path that bends at v
        int p = par[v];
        if (p == -1) continue;
        int chain = down[v] + 1;                          // v's chain extended to p
        if (chain > top1[p]) {
            top2[p] = top1[p];
            top1[p] = chain;
        } else if (chain > top2[p]) {
            top2[p] = chain;
        }
    }
    return best;
}
```

Keeping only the two largest offers instead of sorting all of them makes each vertex O(1) work, so the whole loop is O(n), and it never recurses.

EXAMPLE 18.3 **Two chains on the tree of Figure 18.4**

Take par = [−1, 0, 0, 1, 1, 3, 3, 5, 4, 8] with BFS order [0, 1, 2, 3, 4, 5, 6, 8, 7, 9]. The backward loop gives:

v	top1	top2	down[v]	best so far
9	0	0	0	0
7	0	0	0	0
8	1	0	1	1
6	0	0	0	1
5	1	0	1	1
4	2	0	2	2
3	2	1	2	3
2	0	0	0	3
1	3	3	3	6
0	4	1	4	6

The longest path has 6 edges: 7–5–3–1–4–8–9, bending at vertex 1, where the chains through 3 and through 4 both have length 3. At the root the two best offers are 4 (through 1) and 1 (through 2), only 5.

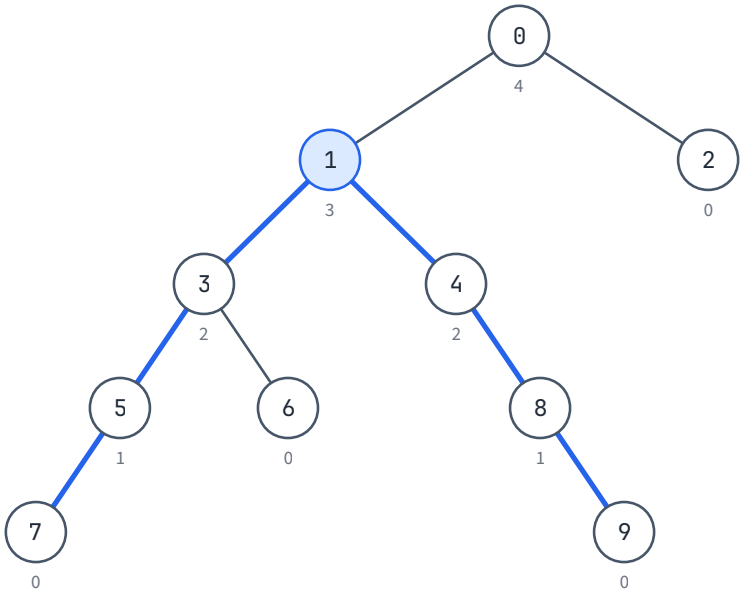


Figure 18.6 The tree of Example 18.3 with down[v] in the notes. The longest path (thick) bends at vertex 1.

LeetCode 2246 (Longest Path With Different Adjacent Characters) is this computation with one extra rule: a child's chain may be offered to its parent only if the two vertices carry different letters, and the answer counts vertices rather than edges. Its input is a parent array, so the backward BFS loop is the natural shape for it; writing it is left to you.

JAVA NOTE

If you have a correct recursive solution and no time to rewrite it, the plan's Java kit gives a fallback: run the work on a new `Thread` created with a large stack size (for example `new Thread(null, task, "solver", 1 << 27)` for 128 MB), which was tested there with recursion depth 10^6. Treat it as an emergency measure; the loops above are shorter than the thread boilerplate once you know them.

18.4 Lowest common ancestors by binary lifting

The recursive LCA of Listing 18.2 answers one query in O(n). With a parent array and depths there is an equally simple method: lift the deeper vertex until both have the same depth, then lift both one step at a time until they meet. That costs O(depth) per query, which on a chain of 10^5 vertices with 10^5 queries is 10^{10} steps. Binary lifting makes each step a jump of a power of two.

DEFINITION 18.6 Binary-lifting table

For a rooted tree, **binary lifting** stores up[k][v], the 2^k-th ancestor of v, for $0 \le k <$ LOG, where LOG is chosen so that $2^{\text{LOG}} \ge n$. Jumps that would pass the root stop at the root: the root's parent is taken to be the root itself.

Write anc(v, j) for the vertex j steps above v, stopping at the root if v has fewer than j ancestors above it. Going up a steps and then b more is the same as going up $a + b$ steps (stopping at the root in both cases), so anc(anc(v, a), b) = anc(v, $a + b$).

LEMMA 18.10 The doubling rule

up[0][v] is the parent of v (the root for the root), and for k ≥ 1, up[k][v] = up[k − 1][up[k − 1][v]].

Proof. up[k][v] = anc(v, 2^k) = anc(anc(v, 2^{k-1}), 2^{k-1}) by the composition rule with $a = b = 2^{k-1}$, and the right-hand side is up[$k-1$] applied twice. ∎

The rule reads row $k-1$ at the vertex x = up[$k-1$][v], an ancestor of v. If the vertices are processed in BFS order, every ancestor of v comes earlier (Theorem 18.8), so all rows of x are already filled when v needs them. That is why the constructor below loops over `order`, filling all LOG rows of each vertex at once.

Listing 18.8 Binary lifting: k-th ancestor, LCA and distance (handbook template)

```
// Binary lifting: up[k][v] = the 2^k-th ancestor of v. O(n log n) build, O(log n) per query.
static final class LCA {
    final int LOG;
    final int[][] up;
    final int[] depth;

    LCA(int[] par, int[] order) {                       // par[root] = -1, order = BFS order
        int n = par.length, lg = 1;
        while ((1 << lg) < n) lg++;
        LOG = lg;
        up = new int[LOG][n];
        depth = new int[n];
        for (int v : order) {                           // parents are filled in first
            up[0][v] = par[v] == -1 ? v : par[v];       // the root points to itself
            depth[v] = par[v] == -1 ? 0 : depth[par[v]] + 1;
            for (int k = 1; k < LOG; k++) up[k][v] = up[k - 1][up[k - 1][v]];
        }
    }

    int kthAncestor(int v, int k) {                     // -1 if v has fewer than k ancestors
        if (k > depth[v]) return -1;
        for (int b = 0; b < LOG; b++) if (((k >> b) & 1) == 1) v = up[b][v];
        return v;
    }

    int lca(int a, int b) {
        if (depth[a] < depth[b]) {
            int t = a;
            a = b;
            b = t;
        }
        a = kthAncestor(a, depth[a] - depth[b]);          // same depth first
        if (a == b) return a;
        for (int k = LOG - 1; k >= 0; k--) {
            if (up[k][a] != up[k][b]) {                 // jump while still below the LCA
                a = up[k][a];
                b = up[k][b];
            }
        }
        return up[0][a];
    }

    int dist(int a, int b) {
        return depth[a] + depth[b] - 2 * depth[lca(a, b)];
    }
}
```

The only change from the handbook card is that the constructor receives the BFS order as a parameter (from Listing 18.3 or Listing 18.4) instead of computing it.

THEOREM 18.11 **The k-th ancestor from binary digits**

If $k \le$ depth[v], `kthAncestor(v, k)` returns the vertex exactly k steps above v.

Proof. Write k as a sum of distinct powers of two, one for each set bit b. The loop jumps up 2^b steps for each set bit, using up[b]. The jumps together cover $k \le$ depth[v] steps, so no jump reaches past the root and none of them is cut short. By the composition rule, the consecutive jumps add up to a single climb of k steps. ∎

THEOREM 18.12 Correctness of the LCA query

`lca(a, b)` *returns the lowest common ancestor of* a *and* b.

Proof. Let a be the deeper vertex. The LCA L of a and b has depth at most depth[b], so it is an ancestor of the vertex depth[a] − depth[b] steps above a; replacing a by that vertex does not change the LCA. Now a and b have the same depth D. If they are equal, that vertex is the LCA.

Otherwise, at any moment of the loop, let $t \ge 1$ be the current distance from a (and from b, which has the same depth) up to L. For $j < t$, the vertices j steps above a and above b are different (otherwise they would be a common ancestor deeper than L); for $j \ge t$ they are equal, since both are $j - t$ steps above L (or both the root). So the test `up[k][a] != up[k][b]` holds exactly when $2^k < t$, that is, $2^k \le t - 1$, and a jump keeps $a \ne b$ and lowers $t - 1$ by 2^k. The loop tries $k = \text{LOG} - 1$ down to 0 and jumps whenever 2^k fits into the current $t - 1$, so it subtracts the binary digits of the initial $t - 1$ from largest to smallest (that value is below $n \le 2^{\text{LOG}}$, so every digit is available). After the loop $t - 1 = 0$: a and b are distinct children of L, and `up[0][a]` is L. ∎

THEOREM 18.13 Distance through the LCA

For vertices a, b *with lowest common ancestor* L, *the path between them has* depth[a] + depth[b] − 2 · depth[L] *edges.*

Proof. Going up from a to L takes depth[a] − depth[L] edges, and going down from L to b takes depth[b] − depth[L]. The two pieces share only L: below L, a and b lie in different children's subtrees (or one of the pieces is empty). So their union is a simple path, which by Theorem 18.3 is *the* path from a to b. ∎

EXAMPLE 18.4 A lifting table by hand

Take par = [−1, 0, 0, 1, 1, 3, 3, 5, 2, 7] (Figure 18.7). Here $n = 10$, so LOG = 4, the BFS order is [0, 1, 2, 3, 4, 8, 5, 6, 7, 9] and the depths are [0, 1, 1, 2, 2, 3, 3, 4, 2, 5].

```
v       0  1  2  3  4  5  6  7  8  9
up[0]   0  0  0  1  1  3  3  5  2  7
up[1]   0  0  0  0  0  1  1  3  0  5
up[2]   0  0  0  0  0  0  0  0  0  1
up[3]   0  0  0  0  0  0  0  0  0  0
```

Each row is the previous row applied twice: up[1][9] = up[0][up[0][9]] = up[0][7] = 5, and up[2][9] = up[1][up[1][9]] = up[1][5] = 1.

```
lca(9, 6): depth 5 vs 3, lift 9 by 2 = binary 10: up[1][9] = 5.   a = 5, b = 6
           k = 3, 2, 1: up[k][5] = up[k][6] (0, 0, 1): no jump
           k = 0: up[0][5] = 3 = up[0][6]: no jump        -> LCA = up[0][5] = 3
           dist = 5 + 3 - 2*2 = 4   (9-7-5-3-6)
lca(9, 8): depth 5 vs 2, lift 9 by 3 = binary 11: up[0][9] = 7, up[1][7] = 3.  a = 3, b = 8
           k = 3, 2, 1: both sides 0: no jump
           k = 0: up[0][3] = 1, up[0][8] = 2 differ: a = 1, b = 2  -> LCA = up[0][1] = 0
           dist = 5 + 2 - 0 = 7
```

In the second query $t = 2$, so $t - 1 = 1$ and the loop makes exactly one jump, of 2^0, as the proof predicts. kthAncestor(9, 3) = 3, and kthAncestor(9, 6) = −1 because vertex 9 has only five ancestors above it.

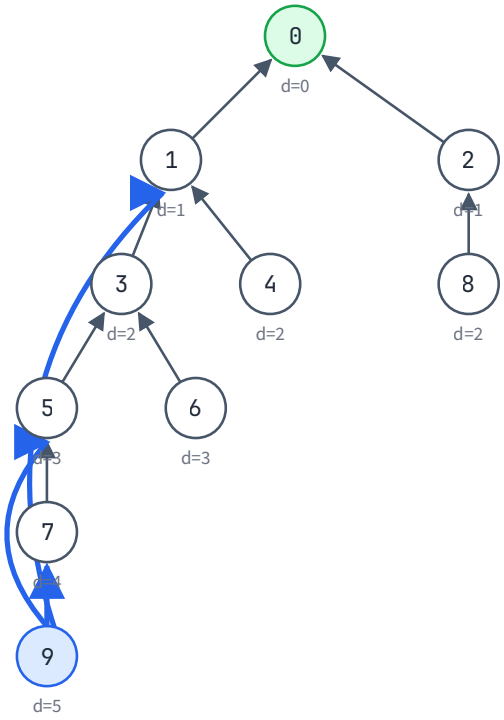


Figure 18.7 The tree of Example 18.4 with depths; arrows point from a vertex to its parent. The thick arrows from 9 are its binary-lifting jumps: 2^0 to 7, 2^1 to 5 and 2^2 to 1.

Cost and memory

Building fills LOG entries for each of n vertices: $O(n \log n)$ time. A k-th ancestor query makes at most LOG jumps, and an LCA query at most $2 \cdot \text{LOG}$, so each is $O(\log n)$. For $n = 10^5$ the loop `while ((1 << lg) < n)` gives LOG = 17, and the table holds $17 \cdot 10^5 = 1.7 \cdot 10^6$ `int`s, about 6.8 MB.

JAVA NOTE

The table is `int[LOG][n]`, not `int[n][LOG]`. Java stores a 2-D array as an array of row objects, each with its own header of about 16 bytes. Seventeen rows of length 10^5 cost almost nothing extra; 10^5 rows of length 17 add about 1.6 MB of headers, 10^5 allocations and an extra pointer step on every access. Put the short dimension first.

LeetCode 1483 (Kth Ancestor of a Tree Node) asks exactly for `kthAncestor` on a fixed tree with many queries, which is what the table is built for.

18.5 Path queries from root prefixes

Binary lifting finds the LCA; combined with values accumulated along root paths, it answers questions about whole paths. The idea is the prefix sum of Chapter 2, laid along every root-to-vertex path at once.

Let every edge $(v, \text{par}[v])$ have a weight $w[v]$, and define the **root prefix** $F[v]$ as the sum of the weights on the path from the root to v ($F[\text{root}] = 0$). It is computed in one forward pass over the BFS order: $F[v] = F[\text{par}[v]] + w[v]$.

THEOREM 18.14 Path values from root prefixes

For vertices u, v with lowest common ancestor L, the sum of the edge weights on the path between u and v is $F[u] + F[v] - 2 \cdot F[L]$. If the values sit on vertices instead, with $S[v]$ the sum of the values from the root to v inclusive, the path sum is $S[u] + S[v] - 2 \cdot S[L] + val[L]$.

Proof. L is an ancestor of u, so the root-to-u path is the root-to-L path followed by the L-to-u path, and $F[u] = F[L]$ + (weights from L down to u). Likewise for v. By the proof of Theorem 18.13 the path between u and v is the union of the L-to-u and L-to-v pieces, so its weight is $(F[u] - F[L]) + (F[v] - F[L])$. With vertex values, $S[u] - S[L]$ and $S[v] - S[L]$ both exclude L's own value, which belongs to the path once, so it is added back. ∎

The same works for **counts**. If each edge has one of a few colours or weights $1..W$, keep one prefix per colour, $C[v][c]$ = the number of edges of colour c on the root path; applying the theorem to the indicator "this edge has colour c" gives the count of colour c on any path as $C[u][c] + C[v][c] - 2 \cdot C[L][c]$. The memory is $W \cdot n$ integers, which for $W = 26$ and $n = 10^4$ is $2.6 \cdot 10^5$.

EXAMPLE 18.5 Counting edge weights on a path

Give the edges of the tree of Figure 18.7 the weights in Figure 18.8, each in 1..3. The root-prefix counts (number of weight-1, weight-2, weight-3 edges from the root) include

```
C[3] = [2, 0, 0]   C[6] = [3, 0, 0]   C[9] = [3, 2, 0]
C[0] = [0, 0, 0]   C[8] = [0, 1, 1]
```

For the path 9–6, $L = 3$: weight 1 appears $3 + 3 - 2 \cdot 2 = 2$ times, weight 2 appears $2 + 0 - 0 = 2$ times and weight 3 never. Directly: the path 9–7–5–3–6 has edges of weights 2, 1, 2, 1. For the path 8–6, $L = 0$ and the counts are simply $C[8] + C[6] = [3, 1, 1]$, the five edges 8–2–0–1–3–6.

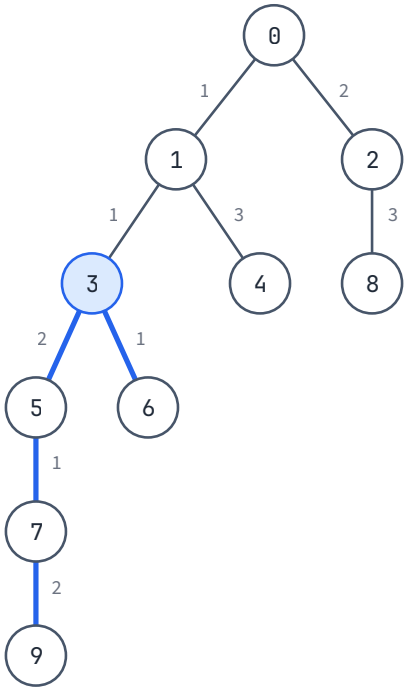


Figure 18.8 The tree of Figure 18.7 with edge weights in 1..3. The path 9–6 (thick) meets at its LCA, vertex 3.

LeetCode 2846 (Minimum Edge Weight Equilibrium Queries in a Tree) needs exactly these counts for the 26 possible weights on each query path; its key observation is that you keep every edge of the most frequent weight on the path and change all the others.

18.6 Subtree sizes as answers

Many tree questions that look global are sums over edges, and the contribution of an edge depends only on how many vertices lie on each side of it. Removing the edge $(v, \text{par}[v])$ splits the tree into the subtree of v, with $\text{size}[v]$ vertices, and the rest, with $n - \text{size}[v]$.

THEOREM 18.15 The sum of all distances

The sum of dist(a, b) over all unordered pairs {a, b} of vertices equals $\Sigma\ \text{size}[v] \cdot (n - \text{size}[v])$ over all non-root vertices v.

Proof. $\text{dist}(a, b)$ is the number of edges on the path between them, so the total counts pairs (pair, edge) with the edge on the pair's path. The edge $(v, \text{par}[v])$ lies on the path between a and b exactly when one of them is in the subtree of v and the other is not: if both are inside, their path stays inside the subtree (it goes through their LCA, which is in the subtree), and if both are outside it never enters the subtree. There are $\text{size}[v] \cdot (n - \text{size}[v])$ such pairs. Summing over edges counts every (pair, edge) once. ∎

Listing 18.9 Sum of all pairwise distances from subtree sizes

```java
// Edge (v, par[v]) lies on the path between a and b exactly when one of them is inside
// v's subtree, so it is used by size[v] * (n - size[v]) pairs.
static long sumOfAllDistances(int[] par, int[] order) {
    int n = par.length;
    int[] size = new int[n];
    Arrays.fill(size, 1);
    long total = 0;
    for (int i = n - 1; i > 0; i--) {                   // children before parents
        int v = order[i];
        total += (long) size[v] * (n - size[v]);
        size[par[v]] += size[v];
    }
    return total;
}
```

For the tree of Figure 18.1 the five edges contribute $3 \cdot 3 = 9$ (edge 1–0), $2 \cdot 4 = 8$ (edge 2–0) and $1 \cdot 5 = 5$ for each of the three leaf edges, 32 in total. The cast to `long` matters: for $n = 10^5$ a single product can reach $2.5 \cdot 10^9$, and the total reaches about $1.7 \cdot 10^{14}$ on a chain.

The same edge-by-edge view solves LeetCode 2477 (Minimum Fuel Cost to Report to the Capital). Every representative in the subtree of v must cross the edge from v to its parent, and a car carries at most *seats* people, so at least $\lceil \text{size}[v] / \textit{seats} \rceil$ cars cross that edge. That is the key observation; showing that the bound can be met at every edge simultaneously, and summing it, is the rest of the problem.

Summary

- A tree with n vertices has $n - 1$ edges and exactly one path between any two vertices; a connected graph with $n - 1$ edges is a tree.
- For recursion on `TreeNode`, decide what one node returns to its parent (height, gain, "p or q is below me") and record paths that bend at the node in a separate holder. Every path has a unique highest node.

- In the maximum path sum, a branch is used only if its gain is positive, and `best` starts at `Integer.MIN_VALUE`.
- The recursive LCA is correct because of a precise statement of what it returns for every subtree, including the case where one node is an ancestor of the other.
- On parent arrays with up to 10^5 vertices, never recurse. Build the BFS order: walking it backwards finishes children before parents (sizes, heights, DP), forwards finishes parents first (depths, prefixes, lifting rows).
- An explicit-stack DFS gives entry and exit times; a subtree is the interval tin[v]..tout[v].
- Binary lifting stores up[k][v] = up[$k-1$][up[$k-1$][v]], built in BFS order: O($n \log n$) build, O($\log n$) for k-th ancestor, LCA and distance = depth[a] + depth[b] − 2 · depth[LCA].
- Path sums and counts are root prefixes combined at the LCA; many global sums are per-edge contributions that depend only on size[v] and n − size[v].

Key terms

tree p. 263
ancestor p. 264
depth p. 264
subtree p. 264
lowest common ancestor p. 269
BFS order p. 270
entry time p. 273
binary lifting p. 276
diameter p. 266
root prefix p. 279

Review questions

1. A parent array of length n has exactly one entry −1, and every other entry is a valid vertex number. Must it describe a tree? Use Theorem 18.3 to say exactly what must be checked, and give a counterexample with $n = 3$.
2. In Listing 18.1, why does `gain` return `node.val + Math.max(l, r)` while it records `node.val + l + r` in `best`?
3. In the recursive LCA, suppose q lies in the subtree of p. What does the call at p return, and why is it correct that the subtree of p is never searched?
4. Would walking the entry-time order of Listing 18.6 backwards also compute subtree sizes correctly? Why?
5. For $n = 10^5$, what value of LOG does the `LCA` constructor choose, and how much memory does the table use? Why is LOG = 17 enough for every query?
6. Why must the rows of `up` be filled in BFS order? Give a parent array for which filling them in index order 0, 1, ..., $n-1$ gives a wrong entry.
7. In the second query of Example 18.4, which jumps does the final loop of `lca` make, and how does that match the proof of Theorem 18.12?
8. For a chain of n vertices (par[v] = $v-1$), give a formula for the sum of all pairwise distances using Theorem 18.15, and evaluate it for $n = 4$.
9. In Theorem 18.14, why is val[L] added back for vertex values but nothing is added back for edge weights?

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 14 "Tree algorithms" (tree traversal, the diameter and its two algorithms) and Chapter 18 "Tree queries" (finding ancestors, subtrees and paths, lowest common ancestor). Free at cses.fi/book.

- Algorithms for Competitive Programming: “Lowest Common Ancestor - Binary Lifting” (cp-algorithms.com/graph/lca_binary_lifting.html) and “Depth First Search” (cp-algorithms.com/graph/depth-first-search.html), which proves the entry/exit-time properties used in Theorem 18.9.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Elementary Graph Algorithms*, for breadth-first and depth-first search and the parenthesis structure of discovery and finishing times.
- Workbook Chapter 18 has a BFS-order table, a binary-lifting table and tree recursions to fill in by hand before you start the problems.

Problems

These are the problems for Day 18. Work them in Workbook Chapter 18, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
18.P1	LC 543 · Diameter of Binary Tree	EASY	CORE
18.P2	LC 236 · Lowest Common Ancestor of a Binary Tree	MEDIUM	CORE
18.P3	LC 124 · Binary Tree Maximum Path Sum	HARD	CORE
18.P4	LC 2246 · Longest Path With Different Adjacent Characters	HARD	CORE
18.P5	LC 102 · Binary Tree Level Order Traversal	MEDIUM	BONUS
18.P6	LC 1483 · Kth Ancestor of a Tree Node	HARD	BONUS
18.P7	LC 2477 · Minimum Fuel Cost to Report to the Capital	MEDIUM	BONUS
18.P8	LC 2846 · Minimum Edge Weight Equilibrium Queries in a Tree	HARD	BONUS

CHAPTER 19 · DAY 19 OF 29 · WEDNESDAY 25 NOVEMBER 2026 · LEARNING DAY

19

Trees II: Tree DP, Rerooting and Small-to-Large

dp[v][state] from the children, answers for every root in O(n), and merging the smaller set into the larger.

Part III · Graphs, Trees and Range Queries
Handbook cards · Tree DP (dp[v][state]), Rerooting (an answer for every root), Small-to-large merging
Before this chapter · Chapter 18 (BFS order, parent arrays, subtree sizes) and Chapter 11 (DP states and transitions)
Practice · Workbook Chapter 19 · 7 problems · core ≈ 4.4 h

LEARNING OBJECTIVES

After this chapter you should be able to:

- define a tree DP with a few states per vertex, prove its transitions by induction on subtrees, and run it in reverse BFS order;
- compute per-subtree aggregates (letter counts, distinct values) and estimate their time and memory;
- compute an answer for every possible root in O(n) by rerooting, both when contributions can be subtracted and when they cannot;
- merge per-subtree sets and maps small-to-large, prove the O(n log n) bound, and count equal-key pairs inside every subtree;
- characterise the vertex sets reachable by operations on both ends of an edge, and solve the resulting parity problem;
- choose between tree DP, rerooting and small-to-large from the wording of a problem.

Chapter 18 showed how to visit a tree without recursion: walk the BFS order backwards to finish children before parents, forwards to finish parents before children. This chapter puts real computations into those two loops. All three of its main techniques answer a question **for every vertex at once**, which is the shape of most hard tree problems.

Tree DP (Section 19.1) gives each vertex a small table of values, one per *state* (chosen or not, matched or not), and computes it from the children's tables. **Rerooting** (Section 19.3) answers "what if the tree were rooted here?" for all n roots with one extra pass instead of n traversals. **Small-to-large merging** (Section 19.4) keeps a whole set or map per subtree, which sounds like O(n^2) memory and time, and shows that pouring the smaller collection into the larger makes it O(n log n). In between, Section 19.2 handles the cheap case of small alphabets, and Section 19.5 treats operations that change both ends of an edge, a pattern behind the sample paper's hardest problem.

Throughout, a tree is given by `par` with `par[root] = -1` and its BFS order `order` from Chapter 18 (`bfsOrder` or `bfsOrderArrays`; `parentsFromEdges` for an edge list). Every listing takes `order` as a parameter so that it never recurses.

19.1 Tree DP with states

DEFINITION 19.1 Tree DP

A **tree DP** assigns to every vertex v and every *state* s from a small set a value dp[v][s]: the best (or the number of) solutions restricted to the subtree of v, among those in which v is in state s. The values of v are computed from the values of its children, so the vertices are processed children first.

The method rests on one structural fact. The subtrees of the children of v share no vertex and no edge; the only edges that connect them to the rest of the tree are the edges from each child to v. So once the state of v is fixed, the choice made inside one child's subtree cannot affect what is allowed inside another's. Each child can be optimised separately, subject only to the rule on the single edge between it and v.

Maximum-weight independent set

DEFINITION 19.2 Independent set

A set of vertices is **independent** if no edge has both ends in it. Given a weight $w[v] \geq 0$ for every vertex, the task is to find an independent set of maximum total weight.

On a general graph this problem is hard (no polynomial algorithm is known). On a tree two states per vertex solve it: take[v] is the best weight in the subtree of v with v in the set, and skip[v] is the best with v not in the set.

THEOREM 19.1 Transitions for the independent set

For every vertex v with children $c_1, \ldots, c_m$,

$$take[v] = w[v] + \Sigma\, skip[c_i] \text{ and } skip[v] = \Sigma\, \max(take[c_i], skip[c_i]),$$

and the answer is $\max(take[root], skip[root])$.

Proof. By induction on the height of v. For a leaf, take = $w[v]$ and skip = 0, which the formulas give with empty sums. Now let v have children, and assume the values of the children are correct. An independent set I of the subtree of v consists of v or not, together with one independent set I_i inside each child's subtree; the only extra condition is on the edges (v, c_i). If $v \in I$, no child may be in I, so each I_i must avoid c_i, and the best total is $w[v]$ plus the best of each, Σ skip[c_i]. If $v \notin I$, the edges (v, c_i) impose nothing, so each I_i is the best set of its subtree, with or without c_i: Σ max(take[c_i], skip[c_i]). Because the subtrees share no edge, any combination of valid I_i is valid, so these sums are achievable as well as upper bounds. ∎

The handbook template computes both sums without children lists: when the backward loop reaches v, both of its values are complete (all children came later in BFS order and were reached earlier), so v adds its contribution to its parent at once.

Listing 19.1 Maximum-weight independent set: dp[v][taken] (handbook template)

```
// dp[v][state] built from the children. Example: maximum-weight set of nodes, no two adjacent.
// take[v] = best in v's subtree with v chosen, skip[v] = best with v not chosen.
static long maxIndependentSet(int[] par, int[] order, int[] w) {   // order = BFS order of the tree
    int n = par.length;
    long[] take = new long[n], skip = new long[n];
    for (int i = n - 1; i >= 0; i--) {                  // children are finished first
        int v = order[i];
        take[v] += w[v];
        int p = par[v];
```

```
        if (p == -1) continue;
        take[p] += skip[v];                          // parent chosen -> child must be skipped
        skip[p] += Math.max(take[v], skip[v]);       // parent skipped -> child is free
    }
    int root = order[0];
    return Math.max(take[root], skip[root]);
}
```

EXAMPLE 19.1 Take or skip, by hand

The tree has par = [−1, 0, 0, 1, 1, 2, 2] and weights w = [5, 3, 4, 2, 6, 1, 3] (Figure 19.1). The BFS order is 0..6, so the loop runs v = 6, 5, ..., 0.

```
v   w[v]  take[v] = w + sum skip(children)   skip[v] = sum max(children)
6    3        3                                 0
5    1        1                                 0
4    6        6                                 0
3    2        2                                 0
2    4        4 + 0 + 0 = 4                     max(1,0) + max(3,0) = 4
1    3        3 + 0 + 0 = 3                     max(2,0) + max(6,0) = 8
0    5        5 + 8 + 4 = 17                    max(3,8) + max(4,4) = 12
```

The answer is max(17, 12) = 17, the set {0, 3, 4, 5, 6}. Taking the root forces both children out, but the four leaves are then all free.

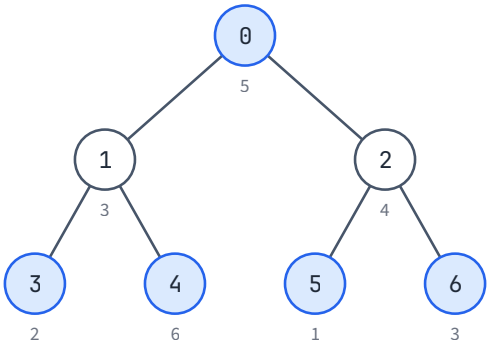


Figure 19.1 The tree of Example 19.1 with weights in the notes. The best independent set (blue) weighs 5 + 2 + 6 + 1 + 3 = 17.

LeetCode 337 (House Robber III) is this DP on a `TreeNode` tree: a house is a vertex, and two directly linked houses cannot both be robbed. There, recursion is fine, and the natural return value is the pair {take, skip} as a `long[2]` (or `int[2]`). Writing it is left to you; the transitions are exactly Theorem 19.1.

The cost of a tree DP

Each edge (v, c) is used once, when c sends its values to v. If there are S states and every state of v looks at every state of c, the work per edge is $O(S^2)$, so the whole DP costs $O(n \cdot S^2)$ time and $O(n \cdot S)$ memory. For the independent set $S = 2$. The point of choosing states carefully is to keep S tiny.

Choosing one child: maximum matching

Some constraints involve *one* child instead of all of them. A **matching** is a set of edges no two of which share a vertex; with edge weights, find a matching of maximum total weight. Now a vertex v is either

matched to one of its children, or not matched to any child (in which case it may still be matched to its parent). Two values per vertex describe this:

- free[v]: the best matching in the subtree of v in which v is **not** matched to a child;
- any[v]: the best matching in the subtree of v, with no restriction.

THEOREM 19.2 Transitions for the matching

With $w(c)$ the weight of the edge (c, v) to the parent,

free[v] = Σ any[c] over the children c, and any[v] = free[v] + max(0, $\max_k$ ($w(c)$ + free[c] − any[c])).

Proof. If v is matched to no child, the children's subtrees are independent and unrestricted, giving Σ any[c]. If v is matched to the child c, then c is used by that edge and may not be matched to one of its own children, so c's subtree contributes free[c] instead of any[c], plus the edge weight $w(c)$, while every other child still contributes any. That total is free[v] − any[c] + free[c] + $w(c)$. any[v] is the best of "no child" and all the single-child choices, which is the formula. Correctness of the children's values follows by induction on height as in Theorem 19.1. ∎

Listing 19.2 Maximum-weight matching in a tree: choose one child

```
// Choose edges, no two sharing a vertex, to maximise the total weight. w[v] = weight of
// the edge (v, par[v]). free[v] = best in v's subtree when v is NOT matched to a child
// (v may still be matched to its parent); any[v] = best with no restriction on v;
// gain[v] = the best extra from matching v to one of its children (0 if none helps).
static long maxMatching(int[] par, int[] order, long[] w) {
    int n = par.length;
    long[] free = new long[n], any = new long[n], gain = new long[n];
    for (int i = n - 1; i >= 0; i--) {                  // children are finished first
        int v = order[i];
        any[v] = free[v] + gain[v];
        int p = par[v];
        if (p == -1) continue;
        free[p] += any[v];                              // p unmatched: each child is unrestricted
        gain[p] = Math.max(gain[p], w[v] + free[v] - any[v]);   // or match p with v
    }
    return any[order[0]];
}
```

Keeping only the best gain over the children turns "try every child" into O(1) work per edge, so the DP is O(n). On the tree of Figure 19.1 with edge weights 4, 2, 3, 5, 6, 1 for the edges from vertices 1..6 to their parents, the loop gives any[2] = 6 (edge 2–5), any[1] = 5 (edge 1–4), free[0] = 11 and gain[0] = 0, so the maximum matching weighs 11: matching the root to either child would give up more than it gains.

INFOSYS LENS

The official sample paper's fourth problem (the "Complex" one, rated 2400+ in the plan) chooses a matching of tree edges and changes both ends of every chosen edge, then asks about patterns on root-to-leaf paths. The plan's advice is to aim for partial marks there. Two ideas from this chapter are the building blocks: the "choose one child" matching DP above, and the parity view of edge operations in Section 19.5.

19.2 Per-subtree aggregates

The simplest "answer for every vertex" is an aggregate over the subtree: its size, its sum, its maximum, or a count for each letter of a small alphabet. Each is computed by the backward loop, adding a child's aggregate into its parent's.

LeetCode 1519 (Number of Nodes in the Sub-Tree With the Same Label) asks, for each vertex v, how many vertices in its subtree carry the same letter as v. With 26 letters, keep a count array per vertex. The listing below solves a neighbouring question, the number of **distinct** letters in each subtree; LeetCode 1519 needs a different read-out of the same `cnt` arrays and an edge list rooted first with `parentsFromEdges` from Chapter 18.

Listing 19.3 Count arrays merged upwards: distinct letters in every subtree

```
// cnt[v][c] = number of vertices in v's subtree whose label is letter c.
static int[] distinctLetters(int[] par, int[] order, String labels) {
    int n = par.length;
    int[][] cnt = new int[n][26];
    int[] distinct = new int[n];
    for (int i = n - 1; i >= 0; i--) {                  // children are merged in before v is used
        int v = order[i];
        cnt[v][labels.charAt(v) - 'a']++;
        for (int c = 0; c < 26; c++) if (cnt[v][c] > 0) distinct[v]++;
        int p = par[v];
        if (p == -1) continue;
        for (int c = 0; c < 26; c++) cnt[p][c] += cnt[v][c];
    }
    return distinct;
}
```

Cost. Each vertex adds its 26 counters into its parent once: $26(n-1)$ additions, plus $26n$ for the read-out, so $O(26\,n)$ time. Memory is the real cost: `new int[n][26]` for $n = 10^5$ is $2.6 \cdot 10^6$ `int`s (10.4 MB) plus 10^5 row objects. That is acceptable, but it grows with the alphabet: for values up to 10^9 instead of letters, a count array per vertex is impossible, and Section 19.4 takes over.

When only the count of **one** key per vertex is needed, as in LeetCode 1519, one global array of 26 counters is enough, thanks to Theorem 18.9.

THEOREM 19.3 Counting with one global array

Run the iterative DFS of Chapter 18 with a global array cnt[26]. When v is entered, record before[v] = cnt[label[v]] and then increment cnt[label[v]]. When v is left, the number of vertices in the subtree of v with the same label as v is cnt[label[v]] − before[v].

Proof. Between entering and leaving v, the DFS enters exactly the vertices of the subtree of v, including v itself, each once (Theorem 18.9). Each entry increments the counter of its own label, so the counter of label[v] grows during that time by the number of subtree vertices with that label. ∎

This uses $O(n)$ memory in total, independent of the alphabet size, but it needs a depth-first order rather than the BFS order.

19.3 Rerooting

DEFINITION 19.3 Rerooting

Rerooting computes, for every vertex r, a quantity defined as if the tree were rooted at r. It roots the tree once, computes answers *inside* subtrees in a backward pass, and then moves the root from each parent to each child in a forward pass, correcting the answer with a formula.

Running a separate traversal from every vertex costs $O(n^2)$, which for $n = 3 \cdot 10^4$ is already $9 \cdot 10^8$ steps and for 10^5 is out of the question. Rerooting does it in $O(n)$.

Sum of distances: the standard example

LeetCode 834 (Sum of Distances in Tree) asks, for every vertex r, for the sum of the distances from r to all other vertices. Root the tree at vertex 0 and define

- size[v]: the number of vertices in the subtree of v;
- down[v]: the sum of the distances from v to the vertices of its own subtree.

LEMMA 19.4 Pass 1: sums inside subtrees

size[v] = 1 + Σ size[c] and down[v] = Σ (down[c] + size[c]) over the children c of v.

Proof. The first formula is the size loop of Listing 18.3. For the second, every vertex x in the subtree of a child c is one edge further from v than from c, since its path to v is its path to c followed by the edge (c, v). Summing over the size[c] vertices of that subtree gives down[c] + size[c]; the vertex v itself contributes distance 0. ∎

At the root, down[0] is the full answer: ans[0] = down[0]. The second pass moves the root.

THEOREM 19.5 Pass 2: moving the root across an edge

If c is a child of p, then ans[c] = ans[p] − size[c] + (n − size[c]).

Proof. Compare each vertex's distance to p and to c. The size[c] vertices in the subtree of c reach p through c, so each is exactly one step closer to c than to p. The other n − size[c] vertices reach c through p, so each is exactly one step further from c. Adding up the changes gives the formula. Here size[c] is the subtree size in the tree rooted at 0, which is correct because the edge (p, c) splits the tree into the same two parts whichever vertex is the root. ∎

The formula needs ans[p] before ans[c], so the second pass walks the BFS order **forwards**.

Listing 19.4 Rerooting: sum of distances from every vertex (handbook template)

```
// Answer for EVERY root in O(n). Sum of distances from each node to all others (LeetCode 834).
static long[] sumOfDistances(int[] par, int[] order) {   // par from parentsFromEdges, BFS order
    int n = par.length;
    int[] size = new int[n];
    long[] down = new long[n];                           // down[v] = sum of distances inside v's
subtree
    Arrays.fill(size, 1);
    for (int i = n - 1; i > 0; i--) {                    // pass 1: children before parents
        int v = order[i], p = par[v];
        size[p] += size[v];
        down[p] += down[v] + size[v];                    // every node below v is 1 step further from p
    }
    long[] ans = new long[n];
```

```
    ans[order[0]] = down[order[0]];
    for (int i = 1; i < n; i++) {                       // pass 2: parents before children
        int v = order[i];
        ans[v] = ans[par[v]] - size[v] + (n - size[v]);  // size[v] nodes get closer, the rest further
    }
    return ans;
}
```

Both passes are single loops over the order, so the time is O(n) and the memory O(n). For LeetCode 834, call `parentsFromEdges(n, edges, 0)` and the BFS order from Chapter 18 first, then copy the `long` answers into the `int[]` the problem returns (its limits keep the sums within `int`, but compute in `long`).

EXAMPLE 19.2 Rerooting the handbook tree

Use the tree of the handbook card, par = [−1, 0, 0, 1, 1, 2] (Figure 19.2).

```
pass 1 (backwards):  size = [6, 3, 2, 1, 1, 1]
                     down = [8, 2, 1, 0, 0, 0]       down[1] = (0+1)+(0+1), down[0] = (2+3)+(1+2)
pass 2 (forwards):   ans[0] = down[0]           = 8
                     ans[1] = 8  - 3 + (6 - 3)   = 8
                     ans[2] = 8  - 2 + (6 - 2)   = 10
                     ans[3] = 8  - 1 + (6 - 1)   = 12      (parent 1)
                     ans[4] = 8  - 1 + (6 - 1)   = 12      (parent 1)
                     ans[5] = 10 - 1 + (6 - 1)   = 14      (parent 2)
```

Check one value directly: from vertex 5 the distances to 0, 1, 2, 3, 4 are 2, 3, 1, 4, 4, which sum to 14.

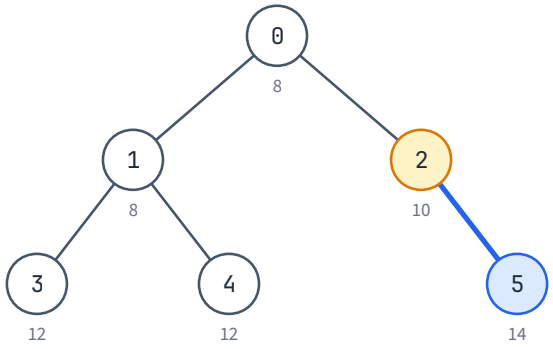


Figure 19.2 Moving the root from 2 to its child 5: the one vertex of 5's subtree comes one step closer, the other five vertices move one step away, so ans[5] = 10 − 1 + 5 = 14. Notes give ans.

A general recipe

The same two passes work for many other quantities. What changes is how the root's answer is split into parts and how a part is moved across an edge.

KEY IDEA

Rerooting recipe. (1) Root at any vertex and compute down[v], the answer for the subtree of v alone, in the backward pass. (2) In the forward pass compute up[v], the contribution of everything **outside** the subtree of v, as seen from v; it is built from up[par[v]] and from the down values of v's siblings. (3) Combine: ans[v] = combine(down[v], up[v]). If contributions are sums, "the siblings of v" is the parent's total minus v's own share, as in Theorem 19.5. If they are maxima, which cannot be subtracted, keep the two best child values at every vertex.

The second case appears when the question is about the **farthest** vertex rather than the sum. The eccentricity of v is the largest distance from v to any vertex, the height of the tree if it were rooted at v. Pass 1 stores the two longest downward chains at each vertex (as in Listing 18.7) and remembers which child gives the longest. Pass 2 computes up[v], the longest path that starts at v and leaves through its parent p: one edge to p, then either further up (up[p]) or down into a sibling of v. The best sibling chain is down1[p], unless that chain goes through v itself, in which case it is down2[p].

Listing 19.5 Rerooting with the two best chains: the farthest distance from every vertex

```
// ecc[v] = the largest distance from v to any vertex = the height of the tree rooted at v.
static int[] eccentricities(int[] par, int[] order) {
    int n = par.length;
    int[] down1 = new int[n], down2 = new int[n], via = new int[n], up = new int[n], ecc = new int[n];
    Arrays.fill(via, -1);                                  // via[p] = the child giving down1[p]
    for (int i = n - 1; i > 0; i--) {                      // pass 1: two longest downward chains
        int v = order[i], p = par[v], chain = down1[v] + 1;
        if (chain > down1[p]) {
            down2[p] = down1[p];
            down1[p] = chain;
            via[p] = v;
        } else if (chain > down2[p]) {
            down2[p] = chain;
        }
    }
    for (int i = 1; i < n; i++) {                          // pass 2: longest path leaving v upwards
        int v = order[i], p = par[v];
        int sideways = via[p] == v ? down2[p] : down1[p];    // best chain from p that avoids v
        up[v] = 1 + Math.max(up[p], sideways);
    }
    for (int v = 0; v < n; v++) ecc[v] = Math.max(down1[v], up[v]);
    return ecc;
}
```

CLAIM 19.6 **Correctness of the eccentricity passes**

For every vertex v, `ecc[v]` is the largest distance from v to any vertex.

Proof. A farthest vertex from v lies either in the subtree of v, at distance down1[v] at most and exactly for the deepest one, or outside it. A path from v to a vertex outside its subtree starts with the edge to $p = \text{par}[v]$ and then either continues up from p (best length up[p], computed earlier because p precedes v in BFS order) or goes down from p into the subtree of a sibling of v. The longest chain from p into a child other than v is down1[p] if the longest chain does not use v, and down2[p] (the best chain through any other child, or 0) if it does. For the root, up = 0 because nothing lies outside its subtree. ∎

For the handbook tree the passes give down1 = [2, 1, 1, 0, 0, 0], up = [0, 3, 3, 4, 4, 4] and ecc = [2, 3, 3, 4, 4, 4]. The vertices of smallest eccentricity are the best roots for a tree of minimum height; LeetCode 310 (Minimum Height Trees), listed on the rerooting card as extra practice, asks for exactly those.

LeetCode 2858 (Minimum Edge Reversals So Every Node Is Reachable) is rerooting over edge directions. Root at 0 and count the edges that point towards the root: those must be reversed for 0 to reach everything. Its key observation is that moving the root from p to a child c changes the count by exactly one, up or down depending only on the direction of the edge between them.

19.4 Small-to-large merging

Count arrays work when keys come from a small alphabet. When values are up to 10^9, each subtree needs a real `HashSet` or `HashMap`. The obvious method, building each vertex's collection by copying all its chil-

dren's collections into a new one, copies every element once for each of its ancestors, which on a path of 10^5 vertices is about $5 \cdot 10^9$ copies.

Two changes fix this. First, a vertex does not build a new collection: it **reuses** one of its children's collections, adding the others into it. Second, it always reuses the **largest** one, so that only the smaller collections are copied.

DEFINITION 19.4 Small-to-large merging

Small-to-large merging combines two collections by inserting every entry of the one with fewer entries into the one with more, and keeping the larger object as the result. Swapping the two references, not their contents, makes "keep the larger one" free.

THEOREM 19.7 Small-to-large costs O(n log n)

Suppose every vertex starts with one entry and collections are combined only by small-to-large merging. Then the total number of entries moved over all merges is $O(n \log n)$.

Proof. Look at one merge, in which a collection with s entries is poured into one with $t \ge s$ entries. Of the s moved entries, say a have keys that are already in the larger collection, so they disappear into existing entries, and $b = s - a$ are new keys.

Suppose first that keys never repeat ($a = 0$, as for sets of distinct colours). Then the merged collection has $t + s \ge 2s$ entries, so every entry that moves lands in a collection at least twice as large as the one it left. A collection never has more than n entries, so an entry moves at most $\log_2 n$ times, and the n entries move $O(n \log n)$ times in total.

With repeated keys, split the merges into two kinds. If $a \ge b$, the merge costs $s \le 2a$; but an entry can disappear only once, and only n entries are ever created, so all merges of this kind together cost at most $2n$. If $a < b$, then $b > s/2$ and the merged collection has $t + b > 1.5 \cdot s$ entries; charge the cost $s < 2b$ to the b surviving entries, two units each. Each of them is now in a collection more than 1.5 times larger than before, so it can be charged at most $\log_{1.5} n$ times. Together the cost is $O(n \log n)$. ∎

With `HashMap` or `HashSet`, each move is O(1) expected, so the merging takes $O(n \log n)$ expected time. The handbook template computes the number of distinct values in every subtree.

Listing 19.6 Small-to-large: distinct values in every subtree (handbook template)

```java
// Distinct colours in every subtree. Merge the SMALLER set into the LARGER one:
// each value moves O(log n) times, so the total is O(n log n).
static int[] distinctInSubtree(int[] par, int[] order, int[] color) {   // order = BFS order
    int n = par.length;
    int[] ans = new int[n];
    @SuppressWarnings("unchecked")
    HashSet<Integer>[] set = new HashSet[n];
    for (int i = n - 1; i >= 0; i--) {                  // children are merged in before v is used
        int v = order[i];
        if (set[v] == null) set[v] = new HashSet<>();
        set[v].add(color[v]);
        ans[v] = set[v].size();
        int p = par[v];
        if (p == -1) continue;
        if (set[p] == null) {
            set[p] = set[v];                            // first child: just hand the set over
        } else {
            HashSet<Integer> big = set[p], small = set[v];
            if (big.size() < small.size()) {
                HashSet<Integer> t = big;
                big = small;
```

```
                small = t;
            }
            big.addAll(small);
            set[p] = big;
        }
        set[v] = null;
    }
    return ans;
}
```

Three details make the listing work. When v is reached by the backward loop, all its children have already poured their sets into `set[v]` (or handed theirs over), so `ans[v]` is read at the right moment, before v's own set leaves. The first child to finish simply hands its set to the parent, so a chain never copies anything. And `set[v] = null` drops the reference to a set that now belongs to the parent; reading it afterwards would be a bug.

PITFALL

When the child's set is the larger one, swap the **references** (`big` and `small`) and store `big` in `set[p]`. Copying the large set into the parent's small one gives the right answer but destroys the O(n log n) bound: on a caterpillar (a long path with a leaf hanging from every vertex), the leaf can finish first and grab its parent's slot, and then every step copies the whole path below.

Counting equal-key pairs in every subtree

A common variant asks, for every vertex v, how many unordered pairs $\{x, y\}$ of vertices in the subtree of v have equal keys. Use a map key → count per subtree instead of a set. The new pairs appear exactly at the moments when two groups of vertices first meet.

THEOREM 19.8 Pairs counted at merges

Merge the children's maps into the parent small-to-large, and when an entry (k, c) of the smaller map meets a count b of the same key in the larger map, add b · c to the parent's pair count. When v adds its own key k to its map, add the count of k already in the map. Then pairs[v] counts every pair of the subtree of v exactly once.

Proof. Take two vertices x, y with equal keys in the subtree of v, and let u be their LCA. If u is x (or y), the pair is counted when u adds its own key, because the other vertex is already in the map collected from u's children, and nowhere else. Otherwise x and y are in the subtrees of different children of u. Before the merges at u they are in different maps; the pair is counted at the merge where the map containing x meets the map containing y (each merge at u combines one child's map with the union of the children before it), and never again, because afterwards they are in one map. Pairs inside one child's subtree were counted below and are passed up with `pairs[p] += pairs[v]`. ∎

Listing 19.7 Small-to-large maps: pairs of equal keys inside every subtree

```
// pairs[v] = number of unordered pairs {x, y} in v's subtree with key[x] == key[y].
static long[] equalKeyPairs(int[] par, int[] order, long[] key) {
    int n = par.length;
    long[] pairs = new long[n];
    @SuppressWarnings("unchecked")
    HashMap<Long, Integer>[] mp = new HashMap[n];        // key -> how many times in the subtree
    for (int i = n - 1; i >= 0; i--) {                   // children are merged in before v is used
        int v = order[i];
        if (mp[v] == null) mp[v] = new HashMap<>();
        int same = mp[v].getOrDefault(key[v], 0);
        pairs[v] += same;                                // v pairs with each equal key below it
```

```
        mp[v].put(key[v], same + 1);
        int p = par[v];
        if (p == -1) continue;
        if (mp[p] == null) {                          // first child: hand the map over
            mp[p] = mp[v];
            pairs[p] += pairs[v];
        } else {
            HashMap<Long, Integer> big = mp[p], small = mp[v];
            if (big.size() < small.size()) {
                HashMap<Long, Integer> t = big;
                big = small;
                small = t;
            }
            long cross = 0;                           // pairs with one end on each side
            for (Map.Entry<Long, Integer> e : small.entrySet()) {
                int have = big.getOrDefault(e.getKey(), 0);
                cross += (long) have * e.getValue();
                big.put(e.getKey(), have + e.getValue());
            }
            mp[p] = big;
            pairs[p] += pairs[v] + cross;
        }
        mp[v] = null;
    }
    return pairs;
}
```

EXAMPLE 19.3 Merging maps with a swap

Take par = [−1, 0, 0, 1, 1, 3, 4, 2] with keys [1, 2, 1, 3, 1, 2, 4, 1] (Figure 19.3). The BFS order is [0, 1, 2, 3, 4, 7, 5, 6], so the loop runs 6, 5, 7, 4, 3, 2, 1, 0.

```
v  key  own pairs  map of v after adding its key    then
6   4      0       {4:1}                            handed to 4
5   2      0       {2:1}                            handed to 3
7   1      0       {1:1}                            handed to 2
4   1      0       {1:1, 4:1}                       handed to 1
3   3      0       {2:1, 3:1}                       merged into 1 (2 entries move, cross 0)
2   1      1       {1:2}                            handed to 0, pairs[0] = 1
1   2      1       {1:1, 2:2, 3:1, 4:1}             merged into 0: 0's map {1:2} is smaller,
                                                    SWAP; 1 entry moves, cross = 1*2 = 2
0   1      3       {1:4, 2:2, 3:1, 4:1}             pairs[0] = 1 + 1 + 2 + 3 = 7
```

The final counts are pairs = [7, 1, 1, 0, 0, 0, 0, 0]. Check the root directly: key 1 occurs at vertices 0, 2, 4 and 7, giving 6 pairs, and key 2 at vertices 1 and 5, giving 1 more. At the last merge the child's map had four entries and the parent's only one, so after the swap a single entry moved instead of four.

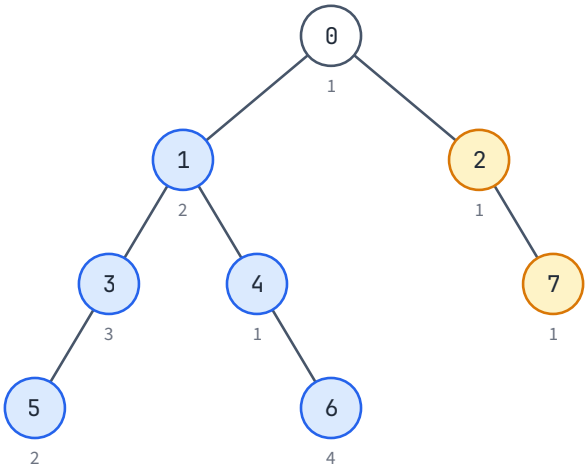


Figure 19.3 The tree of Example 19.3, with keys in the notes. Vertex 1's subtree (blue) has the larger map, so at the merge into 0 it is 0's map that moves.

On the machine used to test this book, both small-to-large listings, together with the rerooting listings, processed trees of 10^5 vertices shaped as a path, a star and a caterpillar in 38–154 ms per tree.

> **INFOSYS LENS**
>
> The official sample paper's Easy problem has exactly this structure: for every node, count the pairs inside its subtree that satisfy a condition, then sum over all nodes, with n up to 10^5 given as a parent array. Once each value is replaced by a key such that "the pair satisfies the condition" becomes "the keys are equal", it is Listing 19.7 plus a final sum. Chapter 25 shows which key to use and the full solution; solve it under exam conditions there first.

Other questions keep more information next to each map. In Codeforces 600E (Lomsat gelral) every vertex needs the sum of the colours that occur most often in its subtree. Counts in the larger map only ever increase during a merge, so the map can carry two extra numbers, the highest count so far and the sum of the colours that reach it, updated whenever a count changes; that is its key observation. Such fields must travel with the map when references are swapped, so it helps to wrap the map and its fields in one small class. LeetCode 2003 (Smallest Missing Genetic Value in Each Subtree) can also be solved with small-to-large sets, although its hint points to a faster idea that uses only the path from the vertex with value 1 to the root.

19.5 Parity and edge operations

LeetCode 3068 (Find the Maximum Sum of Node Values) gives a tree, a value nums[v] on every vertex and a number k. One operation picks an edge (u, v) and replaces both nums[u] and nums[v] by their XOR with k. Operations may be repeated any number of times; maximise the sum of the values. The tree looks essential, but it is not.

Applying the XOR with k twice restores a value, so only the **parity** of the number of operations touching each vertex matters. Let S be the set of vertices touched an odd number of times. The final value of v is nums[v] $\oplus\, k$ if $v \in S$ and nums[v] otherwise.

LEMMA 19.9 Reachable sets are the even-sized sets

In a connected graph, a set S of vertices can be the set of vertices touched an odd number of times exactly when $|S|$ is even.

Proof. Each operation changes the parity of exactly two vertices, so the size of S changes by -2, 0 or $+2$, and it starts at 0. So $|S|$ is always even. Conversely, let $|S|$ be even and pair its vertices up arbitrarily. For a pair (a, b), apply the operation once to every edge of the path from a to b. Each inner vertex of the path lies on two of those edges and is touched twice; a and b lie on one edge each and are touched once. So this flips exactly a and b. Doing this for every pair flips exactly the vertices of S, because parities add up independently. ∎

So the problem is: choose an even-sized set S of vertices to maximise $\Sigma\ \text{nums}[v] + \Sigma_{v \in S}\ g(v)$, where $g(v) = (\text{nums}[v] \oplus k) - \text{nums}[v]$ is the **gain** of flipping v. The edges no longer matter.

THEOREM 19.10 The parity greedy

Let P be the set of vertices with positive gain and m = min over all v of $|g(v)|$. If $|P|$ is even, the answer is Σ nums + $\Sigma_{P}\ g$. If $|P|$ is odd, it is Σ nums + $\Sigma_{P}\ g - m$.

Proof. Without the parity condition, P is the best set, since it contains exactly the vertices that add something. If $|P|$ is even it is allowed. If $|P|$ is odd, every even set S differs from P in at least one vertex. Removing a vertex of P loses its gain $g(v) > 0 = |g(v)|$, and adding a vertex outside P adds $g(v) \le 0$, that is, loses $|g(v)|$. So the value of S is at most the value of P minus $\Sigma\ |g(v)|$ over the vertices where S and P differ, which is at most the value of P minus m. Flipping the membership of a single vertex u with $|g(u)| = m$ gives an even set with exactly that value. ∎

The same result comes out of a two-state DP over the vertices in any order, with best[0] and best[1] the best sum so far using an even or odd number of flips: for each value x, best'[p] = max(best[p] + x, best[$1 - p$] + ($x \oplus k$)), starting from best = [0, $-\infty$]. The answer is best[0]. The DP generalises better: if flipping a vertex had a cost, or the operations were restricted, the greedy would break but the DP would only need new transitions.

EXAMPLE 19.4 Gains and parity

Take nums = [2, 3, 5, 1] and $k = 6$, on any tree with four vertices.

```
x   x XOR 6   gain     DP after x:  even  odd
2      4       +2                    2    4
3      5       +2                    9    7
5      3       -2                   14   12
1      7       +6                   19   21
```

The base sum is 11 and the positive gains are 2, 2 and 6: three of them, an odd number. The smallest $|\text{gain}|$ is 2, so the answer is $11 + 10 - 2 = 19$, the same as the DP's final even value. One optimal choice is to flip the first and the last vertex, giving $4 + 3 + 5 + 7 = 19$; by Lemma 19.9 it is reachable on every tree with four vertices.

Writing LeetCode 3068 from here is left to you; remember that the sum can exceed `int` and must be a `long`.

NOTE

Parity states also appear in tree DPs where the tree *does* matter: if each edge can be used at most once, with a cost, or only edges of a matching may be chosen (as in the sample paper's Complex problem), then dp[v][parity of operations on the edge to the parent] carries the information a child must report to its parent, and the transitions combine children's parities one at a time.

19.6 Choosing a technique

The three techniques answer different shapes of question. The wording of a problem usually tells you which one it needs.

Table 19.1 Signals in the statement and the technique they point to

The statement says	Technique	Typical cost
choose vertices or edges with a rule between neighbours (no two adjacent, a matching, cover every edge)	tree DP with states per vertex	$O(n \cdot S^2)$
an operation changes both ends of an edge	parity of each vertex (Section 19.5), or a DP with a parity state	$O(n)$
for each vertex, something about its subtree with a small alphabet (26 letters, values ≤ 100)	count arrays merged upwards	$O(n \cdot \text{alphabet})$
for each vertex, something about its subtree with large values (distinct values, pairs, most frequent)	small-to-large maps or sets	$O(n \log n)$
"for every vertex as the root", "sum over all other vertices", "the best root"	rerooting	$O(n)$
a sum over all pairs or all paths	per-edge contributions from size[v] (Chapter 18)	$O(n)$
many queries about paths between two given vertices	LCA and root prefixes (Chapter 18)	$O(\log n)$ per query

When two rows seem to apply, try the cheaper one on paper first. Many "for every subtree" questions with a small alphabet do not need small-to-large, and many "for every root" questions turn out to be a sum over edges of size[v] · (n − size[v]), with no second pass at all.

Summary

- A tree DP gives each vertex a few states; because children's subtrees share nothing, each child is optimised separately subject to the rule on its edge. Run it in reverse BFS order; cost $O(n \cdot S^2)$.
- Independent set: take[v] = w[v] + Σ skip[c], skip[v] = Σ max(take[c], skip[c]). Matching: free[v] = Σ any[c], any[v] = free[v] + max(0, best single-child gain).
- Per-subtree counts over a small alphabet merge upwards in O(alphabet · n); for one key per vertex, a global counter with entry and exit times needs only $O(n)$ memory.
- Rerooting computes answers inside subtrees in a backward pass and moves the root in a forward pass. For the sum of distances, ans[c] = ans[p] − size[c] + (n − size[c]). When contributions are maxima, keep the two best children.
- Small-to-large: reuse the largest child's collection and pour the others into it, swapping references. Every entry moves $O(\log n)$ times, so the total is $O(n \log n)$.
- Equal-key pairs are counted when two maps meet: an entry (k, c) meeting count b adds $b \cdot c$.

- Operations that XOR both ends of an edge can flip exactly the even-sized vertex sets of a connected tree, so the tree drops out and a parity greedy or a two-state DP remains.

Key terms

tree DP p. 285
independent set p. 285
Rerooting p. 289
Small-to-large merging p. 292
matching p. 286

Review questions

1. In Theorem 19.1, why is skip[v] built from max(take[c], skip[c]) while take[v] uses only skip[c]? Compute take and skip for the path 0–1–2–3 rooted at 0 with weights [1, 3, 3, 1].
2. Why does the matching DP need the state "not matched to a child" (free) rather than simply "matched" and "unmatched"?
3. Estimate the memory of `int[][] cnt = new int[n][26]` for $n = 10^5$, and explain how Theorem 19.3 avoids it.
4. In Listing 19.4, why must the second pass walk the BFS order forwards? What does the formula give for a leaf c whose parent is p?
5. In Listing 19.5, give a tree on which using down1[p] instead of the "sideways" value gives a wrong eccentricity.
6. In a star with centre 0 and $n - 1$ leaves, how many entries does Listing 19.6 move in total, and why is that consistent with Theorem 19.7?
7. A student's version of Listing 19.7 writes `pairs[p] += cross;` but forgets `pairs[v]`. Which pairs are lost?
8. On the path 0–1–2–3, which operations of LeetCode 3068 flip exactly the vertices 0 and 3? Why can no sequence of operations flip only vertex 0?
9. For each statement, name the technique: (a) for every vertex, the number of distinct values (up to 10^9) in its subtree; (b) for every vertex, the sum of the depths of all vertices if the tree were rooted there; (c) the largest total weight of vertices with no two adjacent.

Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 14 "Tree algorithms" (the section "All longest paths" is rerooting for eccentricities) and Chapter 18 "Tree queries" (the section on offline algorithms, including merging data structures small-to-large). Free at cses.fi/book.
- Algorithms for Competitive Programming, "Introduction to Dynamic Programming" (cp-algorithms.com/dynamic_programming/intro-to-dp.html), for the general discipline of states and transitions that tree DP specialises.
- Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, the chapter *Dynamic Programming*, for optimal substructure, the property Theorem 19.1 proves for trees.
- Workbook Chapter 19 has take/skip tables, a rerooting trace and a small-to-large merge to complete by hand before you start the problems.

Problems

These are the problems for Day 19. Work them in Workbook Chapter 19, which has a worksheet for each one and the hints in its Appendix A. Core problems come first; the bonus problems are the Specialist Programmer level.

No.	Problem	Level	Set
19.P1	LC 337 · House Robber III	MEDIUM	CORE
19.P2	LC 1519 · Number of Nodes in the Sub-Tree With the Same Label	MEDIUM	CORE
19.P3	LC 834 · Sum of Distances in Tree	HARD	CORE
19.P4	LC 3068 · Find the Maximum Sum of Node Values	HARD	CORE
19.P5	LC 2858 · Minimum Edge Reversals So Every Node Is Reachable	HARD	BONUS
19.P6	LC 2003 · Smallest Missing Genetic Value in Each Subtree	HARD	BONUS
19.P7	CF 600E · Lomsat gelral	CF 2300	BONUS

APPENDIX C

Answers to review questions

Answers to the review questions at the end of each chapter.

Chapter 1 · Java for Problem Solving: Collections, Input and Output

1. Take $c = 2$ and $n_0 = 100$: for $n \geq 100$, $100n \leq n^2$, so $n^2 + 100n \leq 2n^2$. It is not O(n): if $n^2 + 100n \leq c \cdot n$ for all large n, then $n + 100 \leq c$ for all large n, which fails for $n > c$.
2. Sorting costs about $n \log_2 n \approx 2 \cdot 10^5 \times 18 = 3.6 \cdot 10^6$ comparisons, and $2 \cdot 10^5$ binary searches of about 18 steps each cost another $3.6 \cdot 10^6$. About $7 \cdot 10^6$ steps in total, far below 10^8: it fits easily.
3. The true product $2.5 \cdot 10^9$ exceeds $2^{31} - 1 \approx 2.147 \cdot 10^9$, so the result wraps: it stores $2{,}500{,}000{,}000 - 2^{32} = -1{,}794{,}967{,}296$. Write `long x = 50_000L * 50_000;` (or `(long) a * b` for variables), so the multiplication happens in `long`.
4. `a == b` is `true` because 100 lies in the cached range −128..127, so both names refer to the same `Integer` object. `c == d` is `false` because 1000 is outside the cache and boxing creates two objects. `c.equals(d)` is `true` because it compares values.
5. An element can then pair with itself. For *nums* = [3, 5] and *target* = 6, index 0 stores 3 and immediately finds 6 − 3 = 3 at index 0, returning [0, 0], which uses one element twice.
6. The proof's first case uses disjointness to conclude that a booking starting before the floor ends before the floor starts. Without it, take bookings [0, 10), [2, 3) and [7, 8) and the request [5, 6). The floor of 5 is the booking starting at 2, which ends at 3 ≤ 5; the ceiling of 5 is the booking starting at 7, which starts at 7 ≥ 6. Neither overlaps, yet [0, 10) does. (My Calendar never stores overlapping bookings, so the case cannot arise there.)
7. Sorting needs all 10^6 numbers in memory and about $10^6 \times 20 = 2 \cdot 10^7$ comparisons. The size-10 heap keeps 11 numbers at most and costs about $10^6 \times \log_2 11 \approx 3.5 \cdot 10^6$ comparisons, and it can process the numbers as they arrive.
8. `FastScanner.next()` keeps reading lines until the current tokenizer has a word, so line breaks are just separators like spaces, and blank lines are skipped. The `readLine` fragment assumes each line holds exactly one value: two values on one line make `parseInt` fail, and a blank line makes it parse an empty string.
9. For μ = 7, λ = 3 the meeting step is the smallest multiple of 3 that is ≥ 7, which is 9. For μ = 0, λ = 4 it is the smallest multiple of 4 that is ≥ 1, which is 4.
10. The failing trees are long chains (or nearly so), where the recursion depth approaches 10^5 and exceeds the thread stack. Fix it by replacing the recursion with an explicit `ArrayDeque` stack (or a parent-array order without recursion, Chapter 18), or by running the solution on a thread with a large stack as in Listing 1.11.

Chapter 2 · Prefix Sums, Difference Arrays and Hashing

1. The leading zero makes $P[i]$ the sum of the half-open range $[0, i)$, so every range is $P[r + 1] - P[l]$, including ranges that start at 0. Without it, with $Q[i] = a[0] + \cdots + a[i]$, the sum is $Q[r] - Q[l - 1]$ and the case $l = 0$ needs a separate branch.
2. The current prefix would be counted as its own earlier prefix. When $k = 0$, every index then contributes one extra "subarray" of length zero. For a = [1, 2] and $k = 0$ the correct answer is 0, but the swapped loop returns 2.
3. The prefix-and-map scan (O(n) memory) and the sliding window of Chapter 3, which uses O(1) extra memory because, with positive values, the window sum moves monotonically with each pointer.
4. Java's `%` returns a negative remainder for a negative left operand, so congruent prefixes could get different keys and array indices could be negative. For `prefix` = −3 and $k = 5$, `-3 % 5` is −3 while `Math.floorMod(-3, 5)` is 2.
5. The prefixes are 0, 1, 2, 3 with parities even, odd, even, odd: two even and two odd, so $2 \times 2 = 4$ odd subarrays. Listing: [1] three times and [1, 1, 1] once, which is 4.

6. For a fixed right end j, the longest subarray uses the smallest left index i with $P[i] = P[j] - k$. Overwriting with later indices would only ever give shorter subarrays.
7. Each update adds +1 at its left end and −1 just after its right end: [0..2] touches $d[0]$ and $d[3]$; [1..3] touches $d[1]$ and $d[4]$; [2..4] touches $d[2]$ and $d[5]$; [0..4] touches $d[0]$ and $d[5]$; [3..3] touches $d[3]$ and $d[4]$. So $d = [2, 1, 1, 0, -2, -2]$, and the running sums give $a = [2, 3, 4, 4, 2]$. Check one entry by hand: index 3 is covered by [1..3], [2..4], [0..4] and [3..3], four ranges.
8. With $r1 = r2 = i$ and $c1 = c2 = j$: $S[i + 1][j + 1] - S[i][j + 1] - S[i + 1][j] + S[i][j]$. By the recurrence in Theorem 2.4 this equals $g[i][j]$.

Chapter 3 · Two Pointers and Sliding Windows

1. By the invariant every valid pair lies inside $[l, r]$, so the only possible partners of l are indices $j \le r$. Since the array is sorted, $a[j] \le a[r]$ for all of them, so $a[l] + a[j] \le a[l] + a[r] < t$. Trying r, the largest partner, already tried them all.
2. Sortedness: for every j in $(l, r]$, $a[j] \le a[r]$, so $a[l] + a[j] \le a[l] + a[r] \le X$. All $r - l$ pairs (l, j) are good.
3. A triple can then be reported once from each copy of its smallest value. For [−1, −1, 0, 1] the method reports (−1, 0, 1) twice: from the first −1 it finds the pair (0, 1) in [−1, 0, 1], and from the second −1 it finds (0, 1) again in [0, 1].
4. (a) Monotone. (b) Not monotone: a valid window can have invalid sub-windows; extending keeps it valid, so it suits the shortest-window loop instead. (c) Monotone, because all values are positive. (d) Not monotone: removing −5 from a window can raise its sum above 10. (e) Not monotone: “aa” is valid but its sub-window “a” is not.
5. Every iteration of the inner loop increases l by one, l never decreases, and l never exceeds n. So across the whole run the inner loop body executes at most n times in total. Adding the n iterations of the outer loop gives $O(n)$ steps.
6. The length only increases at a step where $(\text{length} - M) \le k$, and M is the count of one letter inside a window that really occurred, of length at most the current length (Lemma 3.7 (c)). Any window of the current length that contains that earlier window is valid, so every length the algorithm reports is achievable (first half of Theorem 3.8).
7. atMost(1): the windows with at most one odd value are the three single elements, so 3. atMost(0) = 0, since every element is odd. Exactly one odd value: 3 − 0 = 3.
8. Equal values then stay in the deque, so its values are non-increasing instead of strictly decreasing. The front is still a maximum, so the output stays correct, and each index is still added and removed at most once, so the time stays $O(n)$. The deque may hold more elements.
9. [5, −10, 5] with $X = 0$: the answer is 3 (sum 0), but the template drops the first 5 immediately and returns 2. Use prefix sums: the subarray $a[l..r]$ qualifies when $P[l] \ge P[r + 1] - X$, so for each right end you need the smallest l with that property, which prefix maxima of P and a binary search (Chapter 6) find in $O(\log n)$.

Chapter 4 · Sorting and Greedy Choice: Intervals

1. Two equal `int` values are indistinguishable, so no output can reveal whether they were swapped. Rows of an `int[][]` with the same first column can differ in other columns, so their relative order is visible; Java’s object sort (TimSort) keeps it.
2. The subtraction `x - y` can overflow `long` when the values have opposite signs and large magnitudes, giving the wrong sign. And the cast to `int` keeps only the low 32 bits, so a difference such as 2^{32} becomes 0 (a false tie) and 2^{31} becomes negative. Use `Long.compare(x, y)`.
3. The rule is wrong, but its counterexamples are larger than the ones in Section 4.2. Take the four disjoint intervals [0, 3], [4, 7], [8, 11], [12, 15], the bridge [6, 9], three copies of [2, 5] and three copies of [10, 13] (11 intervals, touching ends allowed). The bridge overlaps only two others, fewer than any other interval, so the rule takes it first; that removes [4, 7] and [8, 11], and at most one interval from each side can still be added, for 3 in total. The optimum is 4 (the top row). A random comparison with the brute force also found a counterexample with 9

intervals, but none with fewer than 9 within 300,000 tries. The lesson: a rule that survives tiny tests can still be wrong, so prove it with an exchange argument or stress-test it on inputs of size 8–10 as well.

4. g_1 has the smallest end of all intervals, so end(g_1) ≤ end(o_1). Every later interval of O starts at or after end(o_1), hence at or after end(g_1), so none of them overlaps g_1. The other intervals of O were already pairwise disjoint.
5. Change `cur[0] >= end` to `cur[0] > end`. Then [1, 2] and [2, 3] conflict, and so do [2, 3] and [3, 4]. The greedy keeps [1, 2] and [3, 4], so the fewest removals is 3 − 2 = 1.
6. Intervals containing a common point pairwise overlap, so each needs its own room: rooms ≥ depth. When the greedy opens room g for interval [s, e], each of the other g − 1 rooms has a last interval that started no later than s and still overlaps [s, e], so it contains s; with the new interval, g intervals contain s, so g ≤ depth.
7. At time 700 the departure of B = [610, 700] would be processed before the arrival of D = [700, 730], so the count would go 2 → 1 → 2 and never reach 3. The method would answer 2, which is wrong for closed intervals: at minute 700 trains B, C and D are all present.
8. $v(7) = 7/9 \approx 0.778$, $v(45) = 45/99 \approx 0.455$, $v(4) = 4/9 \approx 0.444$, $v(40) = 40/99 \approx 0.404$. Decreasing order: 7, 45, 4, 40, giving 745440.
9. Start with $x = 0$. Coin 1 ≤ 1: $x = 1$. Coin 1 ≤ 2: $x = 2$. Coin 3 ≤ 3: $x = 5$. Coin 9 > 6: stop. So every total 0..5 can be paid and 6 is the first that cannot.

Chapter 6 · Binary Search and Searching on the Answer

1. Every index below lo has $a[i] < x$, and every index from hi to n − 1 has $a[i] \geq x$. If x exceeds every element, every test fails, so each iteration sets $lo = mid + 1$ while hi stays n; the loop ends with $lo = hi = n$, and n is returned, meaning "insert at the end". On a = [1, 3, 3, 3, 5, 8, 8, 9] with $x = 10$ the states are (0, 8), (5, 8), (7, 8), (8, 8).
2. $mid = \lfloor (lo + hi)/2 \rfloor$ and $lo < hi$ give $mid \leq (lo + hi - 1)/2 < hi$. If mid could equal hi, then `a[mid]` could read `a[n]`, out of bounds, and `hi = mid` would make no progress.
3. It returns 1 for the key 2 (an arbitrary match, not the first), −5 for 3 (insertion point 4) and −1 for 1 (insertion point 0). For a negative result r, the lower bound is $-r - 1$.
4. Weights [1, 2] with 2 days. The correct answer is 2. With $lo = 0$ the search checks capacity 1: `daysNeeded` closes the first day before the 2, puts the 2 on day 2 although it exceeds the capacity, reports 2 days, and the search returns 1. The lemma assumes cap ≥ every weight, so that each greedy day is a valid load.
5. At a speed equal to the largest pile, every pile takes exactly one hour, so the total is the number of piles, which the problem guarantees is at most h. The total at low speeds can be $10^4 \times 10^9 = 10^{13}$, beyond `int`.
6. With $lo = 6$ and $hi = 7$, a rounded-down midpoint is 6. `piecesAtLeast(a, 6)` = 3 ≥ k, so `lo = mid` sets lo to 6 again and the loop never ends. The rounded-up midpoint is 7, which gives 2 pieces, so hi becomes 6 and the loop stops with 6.
7. By Theorem 6.10: the k-th smallest entry s_k has count ≥ k, and every $x < s_k$ has count ≤ k − 1. A non-entry x with count(x) ≥ k is larger than s_k, so it is never the smallest such value. In the 3 × 4 table, count(5) = 7 but the search returns 4.
8. Near 10^9, neighbouring doubles are `Math.ulp(1e9)` ≈ $1.19 \cdot 10^{-7}$ apart. Once lo and hi are neighbours, their midpoint rounds to one of them, the gap stays about $1.19 \cdot 10^{-7} > 10^{-9}$, and the loop runs for ever. Use a fixed number of iterations, for example 100.
9. f = [0, 0, 0, 1] (peak at index 3). The first midpoint is 1 and $f(1) = f(2)$, so the method decides it is at or past the peak, sets $hi = 1$, and finally returns 0, where $f = 0$. Strictly increasing values before the peak are required.

Chapter 7 · Monotonic Stacks

1. Just before i is pushed, the `while` loop has popped every index with a value smaller than $a[i]$, so the new top below i holds a value ≥ $a[i]$, and the order is kept. The proof uses the order to say that the indices answered by $a[i]$ (those with smaller values) form one block at the top. Without it, a smaller value could sit under a larger one, the `while` loop would stop at the larger one, and the smaller value would miss its answer.
2. [6, 5, 4, 3, 2, 100]: the 100 pops the five indices 4, 3, 2, 1, 0 in one iteration. The whole scan makes 6 pushes and 5 pops. Each index is pushed once and popped at most once, so the `while` loop runs at most n times in total, and the method is O(n).

3. Scan left to right, pop while `a[top] < a[i]`, and after popping read the top (if any) as the answer for i: everything strictly smaller has been removed, so the top is the nearest earlier index with a value ≥ $a[i]$.
4. The circular next greater element of j is among the $n - 1$ positions after it, and the doubled scan visits all of them after j was pushed in the first lap. Pushing during the second lap as well does not change any answer: a second-lap copy of j can only be popped by a value that appears after it, and the first greater value after it is the same one that answered (or will answer) the first-lap copy. It only costs extra pushes and pops, so the first-lap rule is a small saving, not a correctness fix.
5. An optimal rectangle can always be raised until it touches the top of its lowest bar, because the height is limited only by the lowest bar in its range; raising it increases the area. So its height is the height of that bar.
6. With the template's rule, index 0 owns $1 \cdot 1$ subarray, index 1 owns $2 \cdot 1$, index 2 owns $3 \cdot 1$, six in total, and the sum is $3 \cdot 6 = 18$, which is correct (six subarrays, each with minimum 3). With strict comparisons on both sides, every index extends over the whole array: counts $1 \cdot 3$, $2 \cdot 2$, $3 \cdot 1 = 3 + 4 + 3 = 10$, and the sum is 30. Subarrays containing several 3s are counted once for each 3 they contain.
7. left[i] = previous index with a strictly greater value, right[i] = next index with a greater-or-equal value (or the mirror rule). Σ ranges = Σ over i of $a[i] \cdot \text{count_max}(i)$ − Σ over i of $a[i] \cdot \text{count_min}(i)$.
8. After the scan the stack is non-decreasing, so the current string has no descent. By Lemma 7.7 the best single deletion is then the last character, and the same holds after each such deletion, so the remaining k deletions remove the last k characters.
9. The indices strictly between the previous smaller and the next smaller element of i, that is, left[i] + 1..right[i] − 1. In the histogram, that range is exactly the widest rectangle in which bar i is the lowest, so the largest rectangle is the maximum of $a[i]$ × (size of the subtree of i).

Chapter 8 · Bits, XOR, Tries and Linear Bases

1. `1 << 31` is −2147483648, because bit 31 of an `int` is the sign bit. `1 << 32` is 1, because Java uses only the low 5 bits of the shift distance, so 32 acts as 0. `1L << 32` is 4294967296, because the left operand is a `long` and the distance is taken mod 64. Use `1L << k`.
2. $-x = \sim x + 1$. In $\sim x$ the lowest 1 of x becomes 0 and the zeros below it become ones; adding 1 carries through those ones and turns that position back into 1, leaving every higher bit flipped. ANDing with x keeps only that position. `x & (x - 1)` clears the lowest set bit, because $x - 1$ turns the lowest 1 into 0 and the zeros below it into 1s, while the higher bits stay.
3. By commutativity and associativity the two equal elements can be moved next to each other and grouped; their XOR is 0 by $a \oplus a = 0$, and XOR with 0 changes nothing.
4. X = [0, 4, 0, 4, 0]. Equal pairs: (0, 2), (0, 4), (2, 4) for 0 and (1, 3) for 4, contributing 1 + 3 + 1 + 1 = 6. A brute force also gives 6.
5. For each bit, $c_b(n - c_b) \le n^2/4$, because a product of two non-negative numbers with sum n is largest when both are $n/2$. Summing 2^b times this over all bits gives at most $(n^2/4) \cdot \Sigma\, 2^b = (n^2/4)(2^B - 1)$.
6. Below the branching bit X is already smaller than K (by Lemma 8.4), so any lower bits keep $X < K$; there is no constraint left, and the objective is a sum of independent per-bit gains.
7. Such a y follows the path of k's bits all the way down; the walk only adds subtrees that branch off at a 1-bit of k with XOR bit 0, and y never branches off. That is correct because k is not less than k. For a range, count countXorLess(hi + 1) − countXorLess(lo), summed over the elements, querying each before inserting it.
8. With $B = 20$ bits, at most $10^5 \cdot 20 + 1 = 2{,}000{,}001$ nodes; in fact also at most $2^{21} - 1 = 2{,}097{,}151$ because there are only that many prefixes of length ≤ 20, and the first bound is smaller.
9. 3 = 011 goes to b[1]; 5 = 101 goes to b[2]; 6 = 110 reduces by b[2] to 011 and by b[1] to 000, so it is dependent ($6 = 3 \oplus 5$). The rank is 2, so there are $2^2 = 4$ distinct subset XORs (0, 3, 5, 6), and the maximum is 6: `max()` takes b[2] = 5, then $5 \oplus 3 = 6 > 5$.
10. The basis treats every subset as allowed, so its maximum may need more than $N/2$ elements, as in the sample [1, 2] where it gives 3 but only one element may be chosen. Its maximum is always the XOR of at most r original numbers (rank $r \le$ number of bits), so whenever $N/2 \ge r$ the limit does not bind and the basis answer is correct; for values below 2^{20} that holds for all $N \ge 40$.

Chapter 9 · Brute Force, Stress Testing and the Mock Exam

1. Tests are scored one by one, and most problems include small tests. An $O(n^2)$ solution finishes any test with n up to a few thousand ($n = 5{,}000$ is $2.5 \cdot 10^7$ steps) within the time limit, so it earns those marks, and it is the oracle for the fast solution.
2. All subsets by bitmask: $2^{18} \cdot 18 \approx 4.7 \cdot 10^6$ steps, well inside the limit.
3. Your hand-worked answers come from the same understanding that produced the bug, so they share its mistakes, and there are too few of them. The brute force checks the definition directly and can answer thousands of random inputs.
4. The suffix $a[i + 1..]$ is non-increasing, so the rightmost element larger than $a[i]$ is the smallest such element. Swapping with a larger one (for example the leftmost) would give a valid but not the *next* permutation, and the suffix after the swap might not be non-increasing, so reversing it would not give its smallest arrangement.
5. The aim is a smaller input that still shows the bug; a change that makes the programs agree has removed the bug from the input. No: it starts from a disagreeing input and accepts only disagreeing changes (Lemma 9.3).
6. Probably the TLE on test 12: eleven tests passed, and a time-limit failure usually means the remaining tests are the large ones. You cannot be sure because tests are not equal in number or difficulty across problems, and a wrong answer on test 3 could be a small bug that fails only a few later tests too; the stress tester tells you which.
7. Allocating $2 \cdot 10^5$ ints 10^4 times is $2 \cdot 10^9$ cells of zeroing plus garbage, far too slow; so is `Arrays.fill` of the whole array each time ($2 \cdot 10^9$ writes). Allocate or clear only $n + 1$ entries per test case, which totals $O(\Sigma\, n)$.
8. (a) overflow / mod; (b) misread statement (output format); (c) wrong idea; (d) stack overflow.
9. $20 \cdot 1 + 30 \cdot 0.3 + 50 \cdot 0 = 29$. Below the checkpoint, which asks for Q2 at least half the tests; the plan's response is to skip week 2's bonus problems and redo the core problems of your weakest day.

Chapter 10 · Number Theory for Programmers

1. Each iteration replaces b by a mod b, whose absolute value is strictly smaller than $|b|$ (Java's `%` keeps the sign of a but $|a \,\%\, b| < |b|$ still holds), and a non-negative integer cannot decrease forever. Among pairs $a > b$ below 100, $(89, 55) = (F_{11}, F_{10})$ is the only worst case: 9 iterations, as Theorem 10.4 predicts. (With the arguments the other way round, (55, 89) takes one extra iteration, because the first one only swaps them.)
2. The product $a \cdot b$ is formed before the division and can exceed $9.22 \cdot 10^{18}$ even when the lcm fits, as for $6 \cdot 10^9$ and $4 \cdot 10^9$ (the product is $2.4 \cdot 10^{19}$; the lcm is $1.2 \cdot 10^{10}$). Dividing first is exact, because $\gcd(a, b)$ divides a, and every intermediate value is at most the answer. It is wrong only when the lcm itself overflows, and then no order helps.
3. If `prefixA < prefixB` the remainder is negative, and a negative index throws `ArrayIndexOutOfBoundsException` (or a negative key in a map breaks the matching of equal residues). Fix it with `((prefixA - prefixB) % MOD + MOD) % MOD` or with `Math.floorMod(prefixA - prefixB, MOD)`.
4. By Fermat, $a^{p-1} \equiv 1$ when $p \nmid a$, so $a \cdot a^{p-2} \equiv 1$. Modulo 10 000, $3^{9998} \equiv 8889$ and $3 \cdot 8889 \equiv 6667 \not\equiv 1$. Use the extended Euclidean algorithm (`modInverse`), which gives 6667, and expect no inverse when $\gcd(a, m) > 1$, as for $a = 2$.
5. One Fermat inverse costs about 30 squarings, so n of them cost $O(n \log p)$, about 5 seconds for 10^7 values; the backward loop costs one multiplication per entry. It relies on $(i - 1)!^{-1} = i \cdot (i!)^{-1}$ (Theorem 10.10), which needs $n < p$ so that $n!$ is not divisible by p.
6. A multiple $i \cdot k$ with $k < i$ has a prime factor of k, which is smaller than i and has already marked it. A composite i has a smaller prime factor whose loop already marked all multiples of i. The prime p costs at most n/p steps, and the sum of $1/p$ over primes up to n grows like ln ln n (Mertens), so the total is $O(n \log \log n)$.
7. 12 and 18 both have radical 6, but $12 \cdot 18 = 216$ is not a square (their kernels are 3 and 2). The numbers 3 and 12 share kernel 3, and $3 \cdot 12 = 36 = 6^2$. With values up to $2 \cdot 10^9$, the cofactor could have three prime factors above 1000, for example $1009 \cdot 1013 \cdot 1019 = 1\,041\,537\,223 \le 2 \cdot 10^9$, so the "1, q, q^2, $q \cdot r$" case analysis is incomplete.
8. If $\ker(i) = k$, then $i = k \cdot s^2$ by the factorisation $i = \ker(i) \cdot s^2$; conversely $\ker(k \cdot j^2) = k$ when k is square-free, because the extra exponents are even. Loop over every k from 1 to n and add $nums[k \cdot j^2]$ for $j = 1, 2, \ldots$ while $k \cdot j^2 \le n$. For a k that is not square-free the set is a subset of the group of $\ker(k)$, so its sum is never larger than that

group's and taking the maximum over all k is still correct. The work is $\Sigma \sqrt{(n/k)} \le 2n$: for $n = 10^4$ the inner loop runs 16 307 times.

9. A new distinct value is a divisor of the previous one and differs from it, so it is a proper divisor: at most half. In [1, 2, 4, 8] the subarrays ending at index 3 have gcds $g(3, 3) = 8$, $g(2, 3) = 4$, $g(1, 3) = 2$ and $g(0, 3) = 1$, four distinct values, matching $\lfloor \log_2 8 \rfloor + 1 = 4$.

Chapter 11 · Dynamic Programming I: States, Transitions and Knapsacks

1. Fibonacci's recursion tree contains the same arguments many times ($F(3)$ twice in the tree for $F(5)$, and far more often for larger n), so storing each answer removes all but one evaluation per distinct n: $n + 1$ distinct states, $2n - 1$ calls. Merge sort's subproblems are disjoint halves; no subarray is sorted twice, so a memo would never be hit.
2. State: best[i][c] = most value from items $0..i - 1$ with weight at most c. Transition: max(best[$i - 1$][c], best[$i - 1$][$c - w[i - 1]$] + $v[i - 1]$) when the item fits, else best[$i - 1$][c]. Base: best[0][c] = 0. Order: increasing i. Answer: best[n][C]. There are $(n + 1)(C + 1)$ states with at most 2 options each.
3. In the step "every state the transition reads has already been filled in": the order guarantees that the program has visited those states before s. Any order in which row $i - 1$ entries come before row i entries works, for example column by column (increasing c on the outside, increasing i inside), because best[i][c] reads only entries of row $i - 1$ at columns $\le c$.
4. The read of dp[$c - w$] may see a value already updated for the current item, so the item can be taken more than once and the result is the unbounded optimum. The smallest example: one item with $w = 1$, $v = 1$ and capacity 2 gives 2 instead of 1. (The card's example, $w = 2$, $v = 3$, capacity 4, gives 6 instead of 3.)
5. A minimum over all sequences equals the minimum over all multisets, because rearranging coins does not change how many there are; so counting each multiset several times does no harm. A count must count each multiset exactly once, which Theorem 11.8 shows happens only with coins on the outside.
6. 9 sequences and 4 combinations. The multisets are {5} (1 sequence), {2, 2, 1} (3!/(2! 1!) = 3), {2, 1, 1, 1} (4!/(1! 3!) = 4) and {1, 1, 1, 1, 1} (1), and 1 + 3 + 4 + 1 = 9.
7. $P = (S + \text{target})/2$ must be the sum of a set of integers, so it must be an integer; and $0 \le P \le S$ is needed for any subset to have that sum, which fails when $|\text{target}| > S$. Zeros are counted correctly by `countSubsets`, whose pass for $x = 0$ doubles every entry, matching the two signs of a zero.
8. Word i is computed from words $i - \text{ws}$ and $i - \text{ws} - 1$, which have index $\le i$; running downwards means those words have not yet been modified in this pass, so they still hold the old set (the same argument as Theorem 11.5). `>>` is an arithmetic shift: when the top bit of a word is 1, it fills the vacated high bits with 1s and marks sums that are not reachable.
9. $201 \times 100\,001 \times 8$ bytes ≈ 160.8 MB, too much for many limits. The 1-D array has 100 001 longs, about 0.8 MB, so yes.

Chapter 13 · Dynamic Programming III: Increasing Subsequences and Extra Dimensions

1. To decide whether $a[i + 1]$ extends a subsequence you must compare it with that subsequence's last value. The prefix answer is a length only; it forgets which value the best subsequence ends with, and different optimal subsequences may end with different values. Fixing the end index fixes the last value, $a[i]$, so the extension test is a single comparison.
2. Strict (lower bound): [3], [1], [1, 2], [1, 2], [1, 2], [1, 2, 3]; length 3. Non-decreasing (upper bound): [3], [1], [1, 2], [1, 2, 2], [1, 1, 2], [1, 1, 2, 3]; length 4, for example 1, 2, 2, 3. A brute force over all subsequences gives the same two lengths.
3. False. For [3, 4, 1] the loop ends with `tails` = [1, 4], but 1 comes after 4 in the array. `tails[k]` is the best last value for length $k + 1$, and different entries may come from different subsequences.
4. Negate every value: a strictly decreasing subsequence of a is a strictly increasing subsequence of $-a$. For [5, 2, 8, 6, 3, 6, 9, 7] the longest decreasing subsequence has length 3 (for example 8, 6, 3), and the LIS has length 4 (for example 2, 3, 6, 9 or 2, 3, 6, 7).

5. The bounds up[i] ≥ 2 and down[i] ≥ 2 are where the proof uses "the peak is neither first nor last". For [6, 4, 3, 6, 2, 1, 2], index 0 has up = 1, down = 5, and the unchecked formula reports 7 − 5 = 2 removals. The best real mountain has length 4 (for example 4, 6, 2, 1, peak at index 3), so the answer is 3.
6. Each node stores the maximum over its range, and a maximum cannot be "undone": after lowering $g[v]$, a node that covers v does not know the second-largest value in its range, so it would keep reporting the old, larger value. In the LIS DP each $g[v]$ only grows, so this never arises.
7. Query 1..7 reads nodes 7, 6, 4, with ranges [7, 7], [5, 6] and [1, 4], which are disjoint and cover 1..7 (part (a)). Raising value 5 writes nodes 5, 6, 8, with ranges [5, 5], [5, 6], [1, 8]; these are exactly the nodes whose ranges contain 5 (part (b)): node 7 covers only 7, and nodes 1–4 end below 5.
8. Two envelopes with equal width may now both be in a chain, so sort ties by height **ascending** and take the longest **non-decreasing** subsequence of the heights. With (3, 3) and (3, 4), the first fits inside the second under ≤, so the answer is 2. Ascending ties give the heights 3, 4 and a non-decreasing LIS of 2, which is right; descending ties give 4, 3 and the wrong answer 1.
9. Parity has two values, so the state is (end, parity). The quadratic version costs about $2 \cdot n^2/2 = 10^{10}$ steps for $n = 10^5$, too slow; with two Fenwick trees of maxima each element makes two queries and two updates of about 17 steps, roughly $7 \cdot 10^6$ steps in total, plus the $O(n \log n)$ compression.

Chapter 14 · Dynamic Programming IV: Divisors, Intervals and Games

1. A chain can only be read in increasing order, because a positive integer is never divisible by a larger one. Without sorting, the DP only links $j < i$ in the given order. On [4, 2, 8] it links 4 → 8 and 2 → 8 but never 2 → 4, and returns 2; the true answer is 3, the set {2, 4, 8}.
2. 12 + 6 + 4 + 3 + 2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 = 35. The bound is $12 \cdot (\lfloor \log 12 \rfloor + 1) = 12 \cdot 4 = 48$.
3. Chains: 6 with one value, 8 with two, 3 with three. For $K = 4$: $6 \cdot C(3, 0) + 8 \cdot C(3, 1) + 3 \cdot C(3, 2) = 6 + 24 + 9 = 39$, which matches a brute force.
4. dp[0][2] tries $k = 0$, whose candidate dp[0][0] + dp[1][2] + $d[0] \cdot d[1] \cdot d[3]$ needs dp[1][2]. Row 1 is filled in the next pass of the outer loop, so dp[1][2] is still 0 and that candidate, and possibly the minimum, comes out too small.
5. The first cut on a piece always cuts the whole piece, so it costs the piece's length $p[j] - p[i]$ regardless of k. That fixed cost can be taken outside the minimum, and after it the two sub-pieces are independent, which is exactly what a split recurrence needs. Positions go up to 10^6 while there are at most 100 cuts; every piece that can occur runs between two cut points (or ends), so indices into the sorted points describe all states with only $(m + 2)^2$ of them.
6. After bursting balloon k first, its two neighbours become adjacent, so the coins earned later on the left side depend on balloons from the right side and vice versa. Choosing the last balloon instead keeps it standing as a fixed wall between the two sides until both are finished.
7. Colour the positions 0, 1, ..., $n - 1$ alternately even and odd. With n even, the two ends have different colours. If the first player always takes the end of the colour they chose, the two remaining ends after their move are both of the other colour, so the opponent must take that colour, and the first player's colour is again exposed at one end. So the first player can take every element of either colour. Choosing the colour with the larger sum gives at least half the total, so $D(0, n - 1) \geq 0$.
8. The losing positions up to 14 are 0, 2, 7, 9 and 14. From 7 the moves lead to 6, 4 and 3. Each of those is winning: 6 → 2 (remove 4), 4 → 0 (remove 4), 3 → 2 (remove 1), each reaching a losing position. So every move from 7 leads to a winning position, and 7 is losing.
9. `long[5000][5000]` needs $8 \cdot 25 \cdot 10^6 = 2 \cdot 10^8$ bytes, about 200 MB, which exceeds a typical 256 MB limit once the JVM's own memory is counted. The rolling array needs $8 \cdot 5000 = 40{,}000$ bytes. The matrix-chain DP reads dp[l][k] and dp[$k + 1$][r] for **all** shorter lengths, not only the previous one, so the whole table must be kept.

Chapter 15 · Graphs I: Breadth-First Search, Depth-First Search and Topological Order

1. By the handshake lemma the degrees add to $2m$: 3 + 3 + 2 + 2 + 2 + 1 + 1 = 14, so $m = 7$. Degrees 3, 3, 3, 2, 2 add to 13, which is odd, so no graph has them.

2. An undirected edge is stored twice, so there are $4 \cdot 10^6$ list entries. At about 26 bytes per entry (the measurement in Section 15.1) that is roughly 100 MB as `List<Integer>[]`. As arrays: `nxt` and `to` with $4 \cdot 10^6$ entries each is 32 MB, plus 0.4 MB for `head`. These are estimates; the exact figure depends on the JVM.
3. It is used for "w was popped no later than u, so dist[w] ≤ dist[u]": the queue pops in non-decreasing distance. With a stack, vertices are popped in a different order, a vertex can be discovered first along a long path, and its distance is then too large and never corrected (marking on push prevents a second discovery).
4. Both give the same distances. Multi-source BFS is one traversal, $O(n + m)$ in total. Running BFS from each of k sources costs $O(k(n + m))$, which for $k = n$ (every cell a source) is quadratic.
5. A self-loop joins a vertex to itself, so its two endpoints always have the same colour; it is a cycle of length 1, which is odd. In the BFS colouring, when u is popped its neighbour u already has a colour, equal to its own, and the conflict is reported.
6. It outputs exactly the vertices that cannot be reached from any vertex on a cycle. A vertex on a cycle never reaches indegree 0, and by induction along a path neither does any vertex reachable from it; every other vertex has only such "clean" vertices behind it and is output. With the edge 5 → 1 added to Figure 15.3, every vertex except 0 is reachable from the cycle 1 → 3 → 5 → 1, and only 0 is output.
7. Reversing every edge maps each cycle to a cycle and each acyclic graph to an acyclic graph, so the yes/no answer is unchanged. A topological order of the reversed graph is the reverse of an order of the original, so an answer that prints the order would be wrong.
8. BFS counts edges, not costs. With edges 0→1 of cost 1, 0→2 of cost 0 and 2→1 of cost 0, BFS gives vertex 1 distance 1 (one edge), but the path 0→2→1 costs 0.
9. The graph of Example 15.5 does it: vertex 1 is pushed at the back with $d = 1$ when 0 is popped, then improved to 0 through 2 and pushed at the front. By Theorem 15.11, when a vertex first gets a value it is K or $K + 1$ for the key K being popped, and no later key is below K, so it can improve at most once.

Chapter 16 · Analysing a Problem Before You Code

1. Objects: an array of thicknesses. Question: optimise (the longest run). Thicknesses are positive, so the window sum grows when the right end moves and shrinks when the left end moves: a sliding window (Chapter 3) in $O(n)$, with the sum in `long` if thicknesses can be large.
2. Each `while` iteration increments j, which starts at 0, never decreases and is bounded by n. So the inner loop runs at most n times **in total** over all values of i, and the work is $O(n)$ for the inner loop plus $O(n)$ for the outer one (Theorem 16.1).
3. Resizes happen at sizes 1, 2, 4, 8 and 16, copying 1 + 2 + 4 + 8 + 16 = 31 elements, which matches the program's count. The proof gives $2^{k+1} - 1$ with $2^k = 16 < 17$, that is $31 < 2 \cdot 17 = 34$.
4. Resizes then happen every 1000 appends, and the i-th resize copies about $1000i$ elements. For $n = 10^6$ there are 1000 resizes copying about $1000 \cdot (1 + 2 + \cdots + 1000) \approx 5 \cdot 10^8$ elements in total, which is quadratic in n (about $n^2/2000$), not linear.
5. About 29 bytes per row, so roughly 29 MB, because each of the 10^6 rows is a separate array object with its own header. Two parallel `int[10⁶]` arrays, or `int[2][10⁶]`, use about 8 MB.
6. `a + b` is computed in `int`, but with both values at most 10^9 the sum is at most $2 \cdot 10^9$, just below `Integer.MAX_VALUE`, so it does not overflow; three such values would. `(long) a * b` is safe: at most 10^{18}. `a * b % 1_000_000_007` overflows, because the product is computed in `int` before the remainder is taken.
7. `BigInteger` never overflows, so the exact value is right whatever the inputs, and its `mod` always returns a value in [0, m), the mathematical remainder. With `%` on `long`, intermediate products can wrap around and negative operands give negative remainders, so a wrong result can look plausible.
8. Strict LIS with upper bound: all values equal (it returns n instead of 1). A sum in `int`: maximum values at maximum size. A positive-only sliding window: negative values and zeros.

Chapter 17 · Graphs II: Shortest Paths, Union-Find and Spanning Trees

1. Every pair pushed follows a successful relaxation, so there are at most $E + 1$ pairs and each operation costs $O(\log(E + 1))$. A simple graph has $E < V^2$, so $\log(E + 1) \le 2 \log V = O(\log V)$.

2. The answers stay correct. A stale pair is processed with the current `d[u]` (the code uses `d[u]`, not `cur[0]`), which is the value already used when u was settled, so every relaxation from it fails. But each stale pair now rescans all of u's edges, so the running time can grow by a factor of the largest degree.
3. Edges 0→1 (10), 0→2 (1), 2→1 (1). The fewest-edges route is 0→1, one edge of weight 10. Dijkstra finds dist(0, 1) = 2 through vertex 2.
4. The empty path from the source has no edges, and with non-negative weights its cost 0 is the smallest possible, so max(0, w) = w for the first edge. With an edge s→v of weight −3, the true minimax cost of v is −3 but the code computes max(0, −3) = 0. Starting from `Long.MIN_VALUE` (the maximum of an empty set) fixes it: (P1) and (P2) hold for max with any weights.
5. A tight edge into 4 can be relaxed by any vertex settled before 4. When vertex 1 is settled, vertex 3 (distance 2) is not, and the two shortest paths through the edge 3–4 have not been added yet. By Theorem 17.4, ways[4] is final when 4 is settled, because by then every tight predecessor (distance < 4) has been settled.
6. The in-place round 1 already gives [0, 1, 2, 3], and round 2 changes nothing, so it returns 3. That price uses three flights (0→1→2→3), more than the two allowed; the correct answer is 6 (0→2→3). Values improved earlier in a round are reused later in the same round, so a round can add more than one edge.
7. union(0,1), union(2,3), union(4,5), union(6,7), union(0,2), union(4,6), union(0,4) gives `parent` = [0, 0, 0, 2, 0, 4, 4, 6], and element 7 has the path 7 → 6 → 4 → 0, depth 3. By Lemma 17.6 the depth is at most $\log_2 8 = 3$.
8. Suppose $T_1 \neq T_2$ are both minimum spanning trees, and let e be the lightest edge that is in exactly one of them, say $e \in T_1$. Adding e to T_2 creates a cycle. Not all of that cycle's edges are in T_1 (a tree has no cycle), so some edge $f \neq e$ on it is in T_2 but not in T_1. Then f is also in exactly one tree, so $w(f) > w(e)$, as the weights are distinct and e is the lightest such edge. Now $T_2 + e - f$ is a spanning tree lighter than T_2, a contradiction.
9. Kruskal builds 499,500 edges (about 18 MB as `int[3]` objects) and sorts them, about 10^7 comparisons, then runs 499,500 unions. Prim performs 1000 steps, each with two scans of 1000 vertices, about $2 \cdot 10^6$ simple operations, and needs three arrays of length 1000 if distances are computed on demand. In the book's test run Prim was about 10–40 times faster.
10. Union-find can only merge. The groups for a threshold T must contain no vertex of value > T. If a query with a large T were answered first, its activated vertices could join a and b through a vertex that a later query with a small T must not use, and that union cannot be undone.

Chapter 18 · Trees I: Recursion, Parent Arrays and Ancestors

1. Not necessarily. Each non-root vertex contributes one edge to its parent, so there are $n - 1$ edges, and by Theorem 18.3 the graph is a tree exactly when it is connected, which you check by a BFS from the root that must reach all n vertices. Counterexample: par = [−1, 2, 1]. Vertices 1 and 2 point at each other, forming a cycle, and vertex 0 is cut off; the BFS from 0 reaches only one vertex.
2. A path can bend only at its highest node. The value returned to the parent describes a path that continues upwards through the parent, so it may use at most one of the node's two branches. The value recorded in `best` describes a path whose highest node is this node, which may use both.
3. It returns p immediately, without looking below it. By Definition 18.3 the LCA of p and a descendant q is p itself, so nothing below p can change the answer; Theorem 18.6 covers this case.
4. Yes. In a depth-first order every vertex also comes after its parent (a child is pushed while its parent is on the stack, after the parent's entry), so the backward walk again reaches children before parents. The proof of the size loop needs only that property.
5. LOG = 17, because $2^{16} = 65{,}536 < 10^5 \le 131{,}072 = 2^{17}$. The table has $17 \cdot 10^5$ `int`s, about 6.8 MB. A jump distance is at most the depth, which is at most $n - 1 < 2^{17}$, so its binary digits fit in bits 0..16.
6. up[k][v] reads row $k - 1$ at an ancestor of v, which must already be filled. With par = [1, −1, 1] the root is 1. In index order, vertex 0 is processed first, and up[1][0] = up[0][up[0][0]] = up[0][1] reads the root's row before it has been filled, so it reads 0 instead of 1. (`int` arrays start at 0.) The BFS order [1, 0, 2] fills vertex 1 first.
7. After lifting, $a = 3$ and $b = 8$ at depth 2, and the LCA 0 has depth 0, so $t = 2$ and $t - 1 = 1 = 2^0$. The loop finds equal entries for $k = 3, 2, 1$ and different ones for $k = 0$, so it jumps once, to $a = 1$ and $b = 2$, and returns up[0][1] = 0. The jumps are the binary digits of $t - 1$, as in the proof.

8. Rooted at vertex 0, the subtree of vertex v is $\{v, \ldots, n-1\}$, of size $n - v$, so the sum is $\Sigma\,(n-v)\cdot v$ over $v = 1..n-1 = \Sigma\,k(n-k)$ for $k = 1..n-1$. For $n = 4$: $1 \cdot 3 + 2 \cdot 2 + 3 \cdot 1 = 10$. Directly, the six pairs have distances 1, 2, 3, 1, 2, 1, which also sum to 10.
9. With edge weights, $F[u] - F[L]$ and $F[v] - F[L]$ are exactly the edges of the two pieces, and the pieces share no edge. With vertex values, both differences exclude the value of L, yet L is a vertex of the path, so its value must be counted once.

Chapter 19 · Trees II: Tree DP, Rerooting and Small-to-Large

1. If v is taken, its children cannot be, so each child must be in its skip state. If v is skipped, the edge to a child imposes nothing, so each child takes its better state. For the path with weights [1, 3, 3, 1]: vertex 3 gives (take, skip) = (1, 0); vertex 2 gives (3 + 0, max(1, 0)) = (3, 1); vertex 1 gives (3 + 1, max(3, 1)) = (4, 3); vertex 0 gives (1 + 3, max(4, 3)) = (4, 4). The answer is 4, for example {1, 3} or {0, 2}.
2. Whether v may be matched to its parent depends only on whether it is already matched to a child. “Matched” would mix two different situations from the parent’s point of view: matched to a child (the edge to the parent is forbidden) and, after the parent’s decision, matched to the parent. Recording “not matched to a child” (free) and “anything” (any) gives the parent exactly what it needs.
3. $26 \cdot 10^5 = 2.6 \cdot 10^6$ `int`s, about 10.4 MB, plus 10^5 row objects with headers of about 16 bytes each (1.6 MB). Theorem 19.3 keeps one array of 26 counters and two numbers per vertex, because a subtree’s vertices are entered consecutively in a depth-first traversal.
4. ans[c] is computed from ans[p], so the parent must be finished first, and in BFS order every parent precedes its children. For a leaf, size[c] = 1, so ans[c] = ans[p] − 1 + (n − 1) = ans[p] + n − 2.
5. The path 0–1–2 rooted at 0. For $v = 1$, the longest chain from 0 is down1[0] = 2, and it goes through 1 itself. Using it would give up[1] = 1 + 2 = 3, but the farthest vertex from 1 through its parent is 0, at distance 1. The sideways value is down2[0] = 0, giving up[1] = 1 and ecc[1] = max(1, 1) = 1, which is correct.
6. The first leaf to finish hands its set to the centre. Each of the other $n - 2$ leaves has a set of size 1 and the centre’s set is at least as large, so one entry moves per leaf: $n - 2$ moves in total. The bound $O(n \log n)$ allows more; it is an upper bound, and here every moved entry lands in a set at least twice its old size, as in the proof.
7. The pairs whose two vertices both lie inside the child’s subtree v (but not involving p). They were counted in pairs[v] and must be passed up; `cross` contains only pairs with one end on each side of the merge.
8. Apply the operation to the edges 0–1, 1–2 and 2–3 once each: vertices 1 and 2 are touched twice, 0 and 3 once. Flipping only vertex 0 would make $|S| = 1$, which is odd, and Lemma 19.9 shows that $|S|$ is always even.
9. (a) Small-to-large sets (Listing 19.6), because the values are too large for count arrays. (b) Rerooting: it is the sum of distances from each vertex, Listing 19.4. (c) A tree DP with two states, Listing 19.1.

Index

0–9

0-1 BFS, 221, 251
0/1 knapsack, 168

A

adjacency list, 211
aggregate method, 229
amortised analysis, 48, 94, 229
ancestor, 264
autoboxing, 13

B

Bézout's identity, 141
BFS order, 270
big-O notation, 10
binary exponentiation, 143
binary lifting, 276
Binary search on the answer, 78
binary trie, 114
bipartite graph, 218
Boxing, 232
Breadth-first search, 213
brute force (complete search), 123

C

call stack, 28
Cartesian tree, 104
congruence, 142
connected component, 211
contribution technique, 99
coordinate compression, 187
cut, 255

D

depth, 264
depth of intervals, 65
Depth-first search, 216
diameter, 266
difference array, 39
directed acyclic graph, 218
disjoint-set union (union-find), 252
divisible subset, 196
divisor chain, 196
dynamic programming, 164

E

edge case, 235
entry time, 273
Exchange argument, 62
exclusive or (XOR), 110
extra state dimension, 190

F

Fenwick tree, 187

G

GF(2), 118
graph, 211
greatest common divisor, 138
greedy algorithm, 61
Greedy-choice property, 61

H

harmonic bound, 157
hash table, 38

I

immutable, 20
inclusion–exclusion principle, 40
independent set, 285
Integer overflow, 233
interval DP, 201

K

Kruskal's algorithm, 256

L

lexicographic order, 102
linear basis (XOR basis), 118
linked list, 25
load factor, 38
longest increasing subsequence, 180
loop invariant, 74
lower bound, 74
lowest common ancestor, 269

M

matching, 286
memoisation, 164
minimax, 205
minimax path, 246
minimum spanning tree, 255
modular inverse, 145
monotone predicate, 78
monotone window condition, 47
monotonic deque, 54
monotonic stack, 93
mountain array, 185

N

need/have counters, 53
next greater element, 92

O

offline algorithm, 259
offline queries, 117
optimal substructure, 164
ordered map, 15
overlapping subproblems, 164

P

parent pointer, 183
patience sorting, 181
prefix XOR, 111
prefix-sum array, 34
Prim's algorithm, 257
priority queue, 15
problem model, 227

Q

queue, 13

R

relaxation, 242
Rerooting, 289
root prefix, 279

S

shortest-path distance, 242
sieve of Eratosthenes, 149
sliding window, 47
Small-to-large merging, 292
smallest-prime-factor sieve, 150
square-free kernel, 152
stable sort, 60
stack, 13
stale entry, 242
stars and bars, 200
state (DP), 164
state-expanded graph, 249
Stress testing, 127
subsequence, 180
subset sum, 173
subtree, 264
sweep line, 66

T

tails array, 181
ternary search, 89
test oracle, 124
topological order, 218
transition (DP), 164
tree, 263
tree DP, 285
two's complement, 12, 108
two-pointer technique, 43

U

unbounded knapsack, 170
unimodal function, 89
upper bound, 74

W

winning position, 205